Sorting Algorithms — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force walkthroughs, and "why we optimize" reasoning for every problem.

1. Selection Sort

Understanding the Problem NEW

You're given a list of numbers in random order, like `[64, 25, 12, 22, 11]`, and you need to rearrange them so they go from smallest to largest. That's it — no tricks, no hidden constraints. The word "sort" just means: put these numbers in increasing order.

Think of it like this: imagine you have 5 playing cards face-up on a table and you want to line them up from lowest to highest, but the rule is you can only work with a physical deck — you look at all the cards, find the smallest one, and place it at the very left. Then you look at the *remaining* cards, find the next smallest, and place it right after. Selection Sort is exactly this process, done by a computer.

ORIGINAL PROBLEM STATEMENT

Sort an array by repeatedly selecting the minimum element and placing it at the beginning.

Brute Force Approach NEW

What would a beginner try first? The most "obvious" way to sort without knowing any algorithm is: repeatedly scan the entire array, and every time you find two adjacent (or any two) elements that are out of order, swap them. Keep doing random passes until nothing is out of order anymore. This is unstructured and inefficient because you have no system — you might swap the same pair back and forth or scan the array many more times than necessary.

A slightly more structured "brute" idea (closer to what Selection Sort formalizes) is: for every position `i` in the array, compare it against *every other* position to check if a smaller element exists anywhere after it, and if so, swap. This still costs roughly `O(n²)` comparisons but without any guarantee of minimal swaps or a clean invariant.

Why this is "brute force": it doesn't use any clever observation about the structure of the problem — it just brute-scans everything repeatedly until the array happens to be sorted.

Why We Optimize NEW

The unstructured brute-force idea has two problems: (1) you don't know *when* to stop, and (2) you may do far more comparisons and swaps than needed because you're not tracking what's already sorted.

The key insight that leads to Selection Sort: **once you know the smallest element in the whole array, you know for certain it belongs at index 0 — permanently.** That single insight lets you split the array into a "sorted zone" (left) and an "unsorted zone" (right), and shrink the unsorted zone by one every round. This turns a vague repeat-until-sorted process into a precise, guaranteed-to-terminate procedure with exactly `n-1` swaps — far fewer writes than naive repeated swapping, which matters when writes (not just comparisons) are expensive, e.g. writing to flash memory.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [64, 25, 12, 22, 11]

Step 1: Find minimum (11) and swap with first
[11, 25, 12, 22, 64]

Step 2: Find minimum from index 1 (12) and swap
[11, 12, 25, 22, 64]

Step 3: Find minimum from index 2 (22) and swap
[11, 12, 22, 25, 64]

Step 4: Find minimum from index 3 (25) and swap
[11, 12, 22, 25, 64]

Result: [11, 12, 22, 25, 64]
```

The Logic Behind It

Concept: Repeatedly find the smallest element from unsorted part and put it at the beginning.

Step-by-Step Thinking:

1. Find minimum element in unsorted array
2. Swap it with first element of unsorted part
3. Move boundary between sorted and unsorted
4. Repeat until array is sorted

Why This Works:

- Each iteration places one element in correct position
- After i iterations, first i elements are sorted
- Selection sort makes exactly n-1 swaps

Brute Force vs Optimal (original summary)

Brute Force (Naive):

- Compare every element with every other
- $O(n^2)$ time

Optimal (Selection Sort):

- Still $O(n^2)$ but fewer swaps
- $\Theta(n^2)$ comparisons

Algorithm in Theory

```
Algorithm SELECTION_SORT(array):
    For i = 0 to n-1:
        minIndex = i
        For j = i+1 to n-1:
            If array[j] < array[minIndex]:
                 minIndex = j
        Swap array[i] and array[minIndex]
    Return array
```

Pseudocode

```
function selectionSort(arr):
    n = length(arr)
    for i in range(n-1):
        minIdx = i
        for j in range(i+1, n):
            if arr[j] < arr[minIdx]:
                minIdx = j
        swap(arr[i], arr[minIdx])
```

Java Implementation

```
class SelectionSort {
    public void selectionSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n-1; i++) {
            int minIdx = i;
            for (int j = i+1; j < n; j++) {
                if (arr[j] < arr[minIdx]) {
```

```
                    minIdx = j;
                }
            }
            swap(arr, i, minIdx);
        }
    }

    private void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

Time Complexity

- **Best Case:** $O(n^2)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Space Complexity

- O(1) - In-place sorting

When to Use Selection Sort

Use When:

- Small arrays
- Memory is limited
- Minimizing writes is important

Avoid When:

- Large arrays
- Performance is critical
- Data is nearly sorted

2. Bubble Sort

Understanding the Problem NEW

Same goal as before — sort a list of numbers — but this time using a different everyday intuition: look at two numbers standing *next to each other*. If the left one is bigger than the right one, they're "out of order," so swap them. Now move one step to the right and repeat, all the way to the end of the array. That's one full "pass."

Why "bubble"? Picture the largest value as a bubble in water — with every pass, it keeps getting compared with its neighbour and, if bigger, keeps moving one step to the right, until it "floats" all the way to the end. After enough passes, every value has bubbled to its correct spot.

ORIGINAL PROBLEM STATEMENT

Sort an array by repeatedly swapping adjacent elements if they are in wrong order.

Brute Force Approach NEW

The most naive version of this idea: keep making full left-to-right passes over the array, swapping any adjacent out-of-order pair you find, and just *always* do exactly `n-1` passes regardless of whether the array became sorted earlier. This works, and is in fact very close to what Bubble Sort already is — but the "naive" version has no early-exit check, so even if the array becomes sorted after pass 2 out of 5, it keeps blindly comparing pairs in passes 3, 4 and 5 that will never need swapping.

Why We Optimize NEW

The wasted work in the naive version comes from not noticing when the job is already done. The optimization is simple but powerful: **track whether any swap happened during a pass.** If an entire pass completes with zero swaps, that means every adjacent pair is already in the right order — the array is fully sorted, and you can stop immediately instead of running the remaining passes.

This single flag (`swapped`) is what turns Bubble Sort's worst case (fully reversed array, always $O(n^2)$) into a best case of just `O(n)` when the input is already sorted or nearly sorted — because you'll do one pass, find zero swaps, and exit.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [5, 1, 4, 2, 8]

Pass 1:
[5, 1, 4, 2, 8] -> [1, 5, 4, 2, 8] (swap 5,1)
[1, 5, 4, 2, 8] -> [1, 4, 5, 2, 8] (swap 5,4)
[1, 4, 5, 2, 8] -> [1, 4, 2, 5, 8] (swap 5,2)
[1, 4, 2, 5, 8] -> [1, 4, 2, 5, 8] (no swap)
After Pass 1: [1, 4, 2, 5, 8]

Pass 2:
[1, 4, 2, 5, 8] -> [1, 2, 4, 5, 8] (swap 4,2)
After Pass 2: [1, 2, 4, 5, 8]

Result: [1, 2, 4, 5, 8]
```

The Logic Behind It

Concept: Repeatedly step through list, compare adjacent elements and swap if they're in wrong order.

Step-by-Step Thinking:

1. Compare adjacent elements
2. Swap if first is greater than second
3. Largest element "bubbles" to end
4. Reduce range and repeat

Why This Works:

- After each pass, largest element in unsorted part reaches end
- With each pass, one element settles in correct position
- If no swaps in a pass, array is sorted

Brute Force vs Optimal (original summary)

Brute Force:

- Compare all pairs
- $O(n^2)$ always

Optimal (Bubble Sort):

- Can stop early if array becomes sorted
- O(n) best case for already sorted array

Algorithm in Theory

```
Algorithm BUBBLE_SORT(array):
    n = length(array)
    For i = 0 to n-2:
        swapped = false
        For j = 0 to n-i-2:
            If array[j] > array[j+1]:
                 Swap array[j] and array[j+1]
                 swapped = true
        If not swapped:
            Break
    Return array
```

Pseudocode

```
function bubbleSort(arr):
    n = length(arr)
    for i in range(n-1):
        swapped = false
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                swap(arr[j], arr[j+1])
                swapped = true
        if not swapped:
            break
```

Java Implementation

```
class BubbleSort {
    public void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n-1; i++) {
            boolean swapped = false;
            for (int j = 0; j < n-i-1; j++) {
                if (arr[j] > arr[j+1]) {
                    swap(arr, j, j+1);
                    swapped = true;
                }
            }
            if (!swapped) break;
        }
    }

    private void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
```

```
    }
}
```

Time Complexity

- **Best Case:** O(n) - Already sorted
- **Average Case:** O(n²)
- **Worst Case:** O(n²)

Space Complexity

- O(1) - In-place sorting

When to Use Bubble Sort

Use When:

- Small arrays
- Simple implementation needed
- Data is nearly sorted

Avoid When:

- Large arrays
- Performance is important

3. Insertion Sort

Understanding the Problem NEW

This time, imagine sorting playing cards in your hand the way most people actually do it: you hold cards one at a time, and each new card you pick up, you slide it into the correct spot among the cards you're already holding (which are sorted). By the time you've picked up every card, your hand is fully sorted. Insertion Sort is literally that process applied to an array.

ORIGINAL PROBLEM STATEMENT

Sort an array by building the final sorted array one element at a time.

Brute Force Approach NEW

A naive way to "insert" a new element into an already-sorted part of the array is: scan the sorted part from the *beginning*, find the exact index where the new element belongs, then shift everything after that index one place to the right, and drop the element in. This works, but scanning from the beginning every time means you often look at far more elements than necessary, especially if the correct spot is near the end of the sorted part.

Why We Optimize NEW

The optimization here is to scan from the **right end** of the sorted part (the newest neighbour) instead of the left end, and shift elements one-by-one only *while* they're bigger than the element you're inserting. The moment you hit an element that's smaller or equal, you stop — you've found the spot. This means:

- If the array is already nearly sorted, each new element usually needs to move only 0 or 1 positions — giving the excellent `O(n)` best case.
- You never do unnecessary comparisons past the point where the correct position is found.

This is why Insertion Sort is the go-to choice for small or nearly-sorted arrays, and is actually used internally by many real-world sorting libraries (like Java's `Arrays.sort` for small sub-arrays) as a finishing step after another algorithm has done the heavy lifting.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [5, 2, 4, 6, 1, 3]

Step 1: [5] is sorted
[5, 2, 4, 6, 1, 3]

Step 2: Insert 2 into [5]
[2, 5, 4, 6, 1, 3]

Step 3: Insert 4 into [2, 5]
[2, 4, 5, 6, 1, 3]

Step 4: Insert 6 into [2, 4, 5]
[2, 4, 5, 6, 1, 3]

Step 5: Insert 1 into [2, 4, 5, 6]
[1, 2, 4, 5, 6, 3]

Step 6: Insert 3 into [1, 2, 4, 5, 6]
[1, 2, 3, 4, 5, 6]

Result: [1, 2, 3, 4, 5, 6]
```

The Logic Behind It

Concept: Build sorted array by taking one unsorted element and inserting it at correct position.

Step-by-Step Thinking:

1. First element considered sorted
2. Take next element
3. Compare with sorted part from right to left
4. Shift larger elements right
5. Insert element at correct position

Why This Works:

- Maintains invariant: left part is always sorted
- Each element is placed in correct position relative to sorted part
- Shifts are minimal when array is nearly sorted

Brute Force vs Optimal (original summary)

Brute Force:

- Check all positions for each element

- $O(n^2)$

Optimal (Insertion Sort):

- Early exit when position found
- O(n) best case for nearly sorted

Algorithm in Theory

```
Algorithm INSERTION_SORT(array):
    n = length(array)
    For i = 1 to n-1:
        key = array[i]
        j = i - 1
        While j >= 0 and array[j] > key:
            array[j+1] = array[j]
            j = j - 1
        array[j+1] = key
    Return array
```

Pseudocode

```
function insertionSort(arr):
    n = length(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
```

Java Implementation

```
class InsertionSort {
    public void insertionSort(int[] arr) {
        int n = arr.length;
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j+1] = arr[j];
                j--;
            }
            arr[j+1] = key;
        }
    }
}
```

Time Complexity

- **Best Case:** O(n) - Already sorted
- **Average Case:** O(n^2)
- **Worst Case:** O(n^2)

Space Complexity

- O(1) - In-place sorting

When to Use Insertion Sort

Use When:

- Small arrays
- Nearly sorted data
- Online algorithm needed
- Stable sort required

Avoid When:

- Large arrays
- Reverse sorted data

4. Merge Sort

Understanding the Problem NEW

All three algorithms above share a weakness: in the worst case they take O(n²) time, which becomes painfully slow for large arrays (imagine sorting a million numbers — a million squared is a trillion operations). Merge Sort asks a completely different question: **what if, instead of comparing elements one by one across the whole array, we split the problem into smaller and smaller pieces until each piece is trivially "sorted" (a single element), and then carefully glue the pieces back together in order?**

This "split it up, solve small pieces, combine" strategy is called **divide and conquer**, and it's one of the most important ideas in all of computer science — it shows up again and again in later chapters.

ORIGINAL PROBLEM STATEMENT

Sort array using divide and conquer with guaranteed O(n log n) performance.

Brute Force Approach NEW

The "brute force" comparison point here isn't a naive version of merge sort itself — it's the algorithms we already know (Selection/Bubble/Insertion Sort), which are the natural first attempt anyone makes. They're simple to write, but their O(n²) worst case doesn't scale: doubling the input size roughly quadruples the work.

Why We Optimize NEW

The breakthrough insight: **merging two already-sorted lists into one sorted list is very cheap** — you just walk both lists with two pointers, always taking the smaller of the two current front elements, which costs only O(n) total comparisons for n elements combined. So if we can get our hands on sorted halves cheaply, combining them is fast.

And how do we get sorted halves? By recursively applying the exact same idea to each half, all the way down until a "half" is just one element (which is trivially sorted on its own). Splitting a size-n array in half, over and over, takes only `log n` levels of splitting, and at every level we do O(n) total merging work — giving the famous `O(n log n)` total time, which is *guaranteed* no matter how the input

is arranged (unlike Quick Sort below). The trade-off is that merging needs a temporary array, so Merge Sort costs O(n) extra space — it isn't in-place.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [38, 27, 43, 3, 9, 82, 10]

Divide Phase:
[38, 27, 43, 3, 9, 82, 10]
          v
[38, 27, 43] [3, 9, 82, 10]
     v              v
[38] [27, 43]   [3,9] [82,10]
  v    v          v      v
[38] [27] [43] [3] [9] [82] [10]

Merge Phase:
[38] [27,43] [3,9] [10,82]
     v               v
[27,38,43]     [3,9,10,82]
        v
[3,9,10,27,38,43,82]

Result: [3, 9, 10, 27, 38, 43, 82]
```

The Logic Behind It

Concept: Divide array into halves, sort each half, then merge sorted halves.

Step-by-Step Thinking:

1. If array has 1 element, it's sorted
2. Split array into two halves
3. Recursively sort both halves
4. Merge sorted halves back together

Why This Works:

- Divides problem into smaller subproblems
- Merging two sorted arrays is efficient
- Guarantees O(n log n) regardless of input

Brute Force vs Optimal (original summary)

Brute Force (Insertion Sort):

- $O(n^2)$ worst case

- In-place

Optimal (Merge Sort):

- O(n log n) guaranteed
- Uses extra space

Algorithm in Theory

```
Algorithm MERGE_SORT(array, left, right):
    If left < right:
        mid = (left + right) / 2
        MERGE_SORT(array, left, mid)
        MERGE_SORT(array, mid+1, right)
        MERGE(array, left, mid, right)
    Return array

Algorithm MERGE(array, left, mid, right):
    Create temp arrays leftHalf and rightHalf
    Copy array[left..mid] to leftHalf
    Copy array[mid+1..right] to rightHalf
    i = 0, j = 0, k = left
    While i < len(leftHalf) and j < len(rightHalf):
        If leftHalf[i] <= rightHalf[j]:
            array[k] = leftHalf[i]
            i++
        Else:
            array[k] = rightHalf[j]
            j++
        k++
    Copy remaining elements from leftHalf or rightHalf
```

Pseudocode

```
function mergeSort(arr, left, right):
    if left < right:
        mid = (left + right) / 2
        mergeSort(arr, left, mid)
        mergeSort(arr, mid+1, right)
        merge(arr, left, mid, right)

function merge(arr, left, mid, right):
    leftArr = arr[left..mid]
    rightArr = arr[mid+1..right]
    i = 0, j = 0, k = left

    while i < len(leftArr) and j < len(rightArr):
        if leftArr[i] <= rightArr[j]:
            arr[k] = leftArr[i]
            i++
        else:
```

```
            arr[k] = rightArr[j]
            j++
        k++

    while i < len(leftArr):
        arr[k] = leftArr[i]
        i++; k++

    while j < len(rightArr):
        arr[k] = rightArr[j]
        j++; k++
```

Java Implementation

```
class MergeSort {
    public void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);
            merge(arr, left, mid, right);
        }
    }

    private void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] leftArr = new int[n1];
        int[] rightArr = new int[n2];

        System.arraycopy(arr, left, leftArr, 0, n1);
        System.arraycopy(arr, mid + 1, rightArr, 0, n2);

        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
            if (leftArr[i] <= rightArr[j]) {
                 arr[k] = leftArr[i];
                 i++;
            } else {
                 arr[k] = rightArr[j];
                 j++;
            }
            k++;
        }

        while (i < n1) { arr[k] = leftArr[i]; i++; k++; }
        while (j < n2) { arr[k] = rightArr[j]; j++; k++; }
    }
}
```

Time Complexity

- **Best Case:** O(n log n)
- **Average Case:** O(n log n)
- **Worst Case:** O(n log n)

Space Complexity

- O(n) - Needs extra arrays for merging

When to Use Merge Sort

Use When:

- Guaranteed O(n log n) needed
- Stable sort required
- Linked list sorting
- External sorting

Avoid When:

- Memory is limited
- In-place sorting needed
- Small arrays

5. Recursive Bubble Sort

Understanding the Problem NEW

This is not a new sorting idea — it's the exact same Bubble Sort from problem 2, just written using recursion (a function calling itself) instead of a `for` loop. It's included here mainly as recursion practice: the goal is to express "do one pass, then repeat on a smaller array" as a function calling itself with a shrinking size, instead of a loop counting down.

ORIGINAL PROBLEM STATEMENT

Implement bubble sort using recursion.

Brute Force Approach NEW

There isn't really a separate "brute force vs optimized" split for this one — the underlying algorithm *is* Bubble Sort. The only real choice is implementation style: iterative (loop-based, as in problem 2) vs recursive (function calls itself). The "brute" way a beginner might first write this recursively is to forget the base case or recompute the array length inside every call — leading to either infinite recursion or wasted work.

Why We Optimize NEW

The important thing to get right here is the **base case** and the **shrinking parameter**: each call does one bubble pass over the first `n` elements (pushing the current largest of those `n` to position `n-1`), then recurses with `n-1`. The recursion naturally stops when `n <= 1`, since a single element is already sorted. Unlike the divide-and-conquer recursion in Merge Sort, this recursion doesn't reduce the problem size *exponentially* (halving) — it only reduces it by 1 each time, so the time complexity stays $O(n^2)$, and you additionally pay $O(n)$ extra space for the recursion call stack that the iterative version doesn't need.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [5, 1, 4, 2, 8]
```

```
Recursive call stack:
bubbleSort([5,1,4,2,8], 5)
  -> bubbleSort([1,4,2,5,8], 4) [after one pass]
    -> bubbleSort([1,2,4,5,8], 3) [after two passes]
      -> bubbleSort([1,2,4,5,8], 2) [after three passes]
        -> bubbleSort([1,2,4,5,8], 1) [base case]

Result: [1, 2, 4, 5, 8]
```

The Logic Behind It

Concept: Same as iterative bubble sort but implemented recursively.

Step-by-Step Thinking:

1. If array size is 1, return
2. Perform one pass of bubble sort
3. Recursively call on reduced array size
4. Base case: array of size 1 is sorted

Why This Works:

- After each pass, largest element bubbles to end
- Recursion reduces problem size by 1
- Same logic as iterative version

Algorithm in Theory

```
Algorithm RECURSIVE_BUBBLE_SORT(array, n):
    If n <= 1:
        Return
    For i = 0 to n-2:
        If array[i] > array[i+1]:
            Swap array[i] and array[i+1]
    RECURSIVE_BUBBLE_SORT(array, n-1)
    Return array
```

Pseudocode

```
function recursiveBubbleSort(arr, n):
    if n <= 1:
        return

    for i in range(n-1):
        if arr[i] > arr[i+1]:
            swap(arr[i], arr[i+1])

    recursiveBubbleSort(arr, n-1)
```

Java Implementation

```
class RecursiveBubbleSort {
    public void recursiveBubbleSort(int[] arr, int n) {
        if (n <= 1) return;

        for (int i = 0; i < n-1; i++) {
            if (arr[i] > arr[i+1]) {
                swap(arr, i, i+1);
            }
        }
        recursiveBubbleSort(arr, n-1);
    }

    private void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

Time Complexity

- **Best Case:** O(n) - Already sorted
- **Average Case:** O(n^2)
- **Worst Case:** O(n^2)

Space Complexity

- O(n) - Recursion stack space

When to Use Recursive Bubble Sort

Use When:

- Learning recursion
- Simple implementation needed

Avoid When:

- Performance is critical
- Deep recursion concerns

6. Recursive Insertion Sort

Understanding the Problem NEW

Same relationship as the previous problem: this is Insertion Sort (problem 3), rewritten recursively. The clever bit is *which* part gets the recursive call: instead of recursing on "the rest of the array to insert," it recurses on "sort everything except the last element first," and only *after* that recursive call returns does it insert the last element into its correct spot.

ORIGINAL PROBLEM STATEMENT

Implement insertion sort using recursion.

Brute Force Approach NEW

As with recursive bubble sort, there's no separate "brute force" algorithm here — it's about recursion style. A naive attempt might try to recurse "forward" (sort element 0, then recurse on the rest), which actually breaks the natural insertion-sort invariant. The version given here recurses "backward" (assume the first n-1 are already sorted via recursion, then insert the n-th).

Why We Optimize NEW

The reason the recursion is structured as "recurse first, insert after" is that it mirrors the iterative version's invariant exactly: **"the first k elements are always sorted."** By recursing down to `n=1` first (trivially sorted), then inserting element by element as the recursive calls return (unwind), each insertion has a properly sorted left neighbourhood to slot into — exactly like the iterative loop, just expressed with the call stack doing the "looping" instead of a `for` statement. The time complexity remains $O(n^2)$ since insertion itself is unchanged; the only new cost is O(n) recursion stack space.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [5, 2, 4, 6, 1, 3]

Recursive call stack:
```

```
insertionSort([5,2,4,6,1,3], 0) [base case]
  -> insertionSort([5,2,4,6,1,3], 1)
    -> insertionSort([5,2,4,6,1,3], 2)
      -> insertionSort([5,2,4,6,1,3], 3)
        -> insertionSort([5,2,4,6,1,3], 4)
          -> insertionSort([5,2,4,6,1,3], 5) [actual sorting starts here]

Result: [1, 2, 3, 4, 5, 6]
```

The Logic Behind It

Concept: Same as iterative insertion sort but implemented recursively.

Step-by-Step Thinking:

1. Base case: array of size 1 is sorted
2. Recursively sort first n-1 elements
3. Insert last element in correct position
4. Shifts elements right as needed

Why This Works:

- Recursion handles sorting first n-1 elements
- Insertion step places last element correctly
- Same logic as iterative version

Algorithm in Theory

```
Algorithm RECURSIVE_INSERTION_SORT(array, n):
    If n <= 1:
        Return
    RECURSIVE_INSERTION_SORT(array, n-1)
    key = array[n-1]
    j = n-2
    While j >= 0 and array[j] > key:
        array[j+1] = array[j]
        j = j-1
    array[j+1] = key
    Return array
```

Pseudocode

```
function recursiveInsertionSort(arr, n):
    if n <= 1:
        return

    recursiveInsertionSort(arr, n-1)

    key = arr[n-1]
```

```
    j = n-2
    while j >= 0 and arr[j] > key:
        arr[j+1] = arr[j]
        j--
    arr[j+1] = key
```

Java Implementation

```
class RecursiveInsertionSort {
    public void recursiveInsertionSort(int[] arr, int n) {
        if (n <= 1) return;

        recursiveInsertionSort(arr, n-1);

        int key = arr[n-1];
        int j = n-2;
        while (j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
```

Time Complexity

- **Best Case:** O(n) - Already sorted
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Space Complexity

- O(n) - Recursion stack space

When to Use Recursive Insertion Sort

Use When:

- Learning recursion
- Simple implementation needed
- Small arrays

Avoid When:

- Performance is critical
- Deep recursion concerns

7. Quick Sort

Understanding the Problem NEW

Quick Sort asks: can we get Merge Sort's O(n log n) speed *without* paying for the extra O(n) memory that merging needs? The idea: instead of blindly splitting the array in half and merging afterward, pick one element (called the **pivot**) and rearrange (*partition*) the array so that everything smaller than the pivot ends up on its left, and everything bigger ends up on its right. Once that's done, the pivot is already sitting in its final, correct sorted position — no merging step required. Then just repeat the same trick separately on the left group and the right group.

ORIGINAL PROBLEM STATEMENT

Sort an array efficiently using divide and conquer.

Brute Force Approach NEW

The natural comparison point is Merge Sort: it guarantees O(n log n) but always needs a temporary array to merge into, which costs extra memory and extra copying. A "brute" alternative to picking a good pivot is to always pick, say, the first or last element as pivot without any thought. This works correctly, but if the input happens to already be sorted (or reverse-sorted), always picking an end element as pivot is the *worst possible* choice — it causes maximally unbalanced partitions.

Why We Optimize NEW

The whole performance story of Quick Sort comes down to **how balanced the partition is**. If the pivot happens to land near the middle of the sorted order every time, the array roughly halves at each level (just like Merge Sort), giving `O(n log n)` — but this time *in-place*, with no extra array, because partitioning rearranges elements within the same array.

But if you always pick a bad pivot (e.g. the smallest or largest element, which happens naturally on already-sorted input if you always choose the last element), each partition only peels off one element instead of splitting in half — degrading to `O(n²)`, the same as the simple sorts we started with. This is exactly why real implementations use a **random pivot** or **median-of-three** selection (shown at the end of this problem): it makes the worst case astronomically unlikely in

> practice, so Quick Sort behaves like O(n log n) almost all the time while staying in-place — which is why it tends to outperform Merge Sort in practice despite having a worse theoretical worst case.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [50, 30, 80, 20, 70, 10, 60, 40]

Step 1: Choose pivot (last element: 40)
[50, 30, 80, 20, 70, 10, 60, 40]
              Pivot=40

Partition: Arrange elements around pivot
Elements < 40: [30, 20, 10]
Elements > 40: [50, 80, 70, 60]
After partition: [30, 20, 10, 40, 50, 80, 70, 60]

Step 2: Recursively sort left and right
Left: [30, 20, 10] Right: [50, 80, 70, 60]

Step 3: Sort left part
[30, 20, 10] -> pivot=10
Partition: [10, 20, 30]
Sort sub-parts: [10], [20], [30] -> [10, 20, 30]

Step 4: Sort right part
[50, 80, 70, 60] -> pivot=60
Partition: [50, 60, 70, 80]
Sort sub-parts: [50], [70, 80] -> [70, 80]
Final: [50, 60, 70, 80]

Result: [10, 20, 30, 40, 50, 60, 70, 80]
```

The Logic Behind It

Concept: Choose a pivot, place it in its correct position, and recursively sort the elements on both sides.

Step-by-Step Thinking:

1. Choose Pivot: Select an element as pivot
2. Partition: Rearrange array so all elements < pivot are on left, all elements > pivot are on right, and pivot is in its correct position
3. Recursively Sort: Apply same process to left and right subarrays

Why This Works:

- After partitioning, pivot is in correct position
- Left subarray contains smaller elements
- Right subarray contains larger elements
- Each step reduces problem size

Visual Animation

```
Step 1: Choose pivot
[50, 30, 80, 20, 70, 10, 60, 40]
                              ^
                            pivot

Step 2: Partition
i = -1
[50, 30, 80, 20, 70, 10, 60, 40]
  ^
  i

Compare 50 > 40 -> No swap
[50, 30, 80, 20, 70, 10, 60, 40]
      ^
      i

Compare 30 < 40 -> Swap with i+1
[30, 50, 80, 20, 70, 10, 60, 40]
  ^
  i

Compare 80 > 40 -> No swap
Compare 20 < 40 -> Swap with i+1
[30, 20, 80, 50, 70, 10, 60, 40]
      ^
      i

Compare 70 > 40 -> No swap
Compare 10 < 40 -> Swap with i+1
[30, 20, 10, 50, 70, 80, 60, 40]
         ^
         i

Compare 60 > 40 -> No swap
Place pivot in position i+1
[30, 20, 10, 40, 70, 80, 60, 50]
             ^
           pivot at position 3
```

Brute Force vs Optimal (original summary)

Brute Force (Merge Sort):

- O(n log n) but uses extra space
- Always divides exactly in half

Optimal (Quick Sort):

- O(n log n) average
- In-place sort
- Faster in practice due to better cache performance

Algorithm in Theory

```
Algorithm QUICK_SORT(array, low, high):
    If low < high:
        pivotIndex = PARTITION(array, low, high)
        QUICK_SORT(array, low, pivotIndex-1)
        QUICK_SORT(array, pivotIndex+1, high)
    Return array

Algorithm PARTITION(array, low, high):
    pivot = array[high]
    i = low - 1
    For j = low to high-1:
        If array[j] <= pivot:
            i = i + 1
            Swap array[i] and array[j]
    Swap array[i+1] and array[high]
    Return i+1
```

Pseudocode

```
function quickSort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quickSort(arr, low, pi-1)
        quickSort(arr, pi+1, high)

function partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j from low to high-1:
        if arr[j] <= pivot:
            i += 1
            swap(arr[i], arr[j])
    swap(arr[i+1], arr[high])
    return i+1
```

Java Implementation

```
class QuickSort {
    public void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }

    private int partition(int[] arr, int low, int high) {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                swap(arr, i, j);
            }
        }
        swap(arr, i + 1, high);
        return i + 1;
    }

    private void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

Time Complexity Explanation

Why O(n log n) Average?

Best Case (Pivot always splits array in half):

```
T(n) = 2T(n/2) + O(n)      -> T(n) = O(n log n)
```

Worst Case (Pivot always min or max):

```
T(n) = T(n-1) + O(n)       -> T(n) = O(n²)
```

Average Case:

- Assuming random pivot, splits are balanced on average
- T(n) = O(n log n)

Time Complexity:

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: O(n²)

Why Worst Case O(n²)?

- If pivot is always smallest/largest
- Each partition only removes one element
- Causes O(n²) comparisons

Space Complexity Explanation

Why O(log n) Average?

Quick sort:

- In-place sort (no extra array)
- Recursion stack space

Recursion Stack:

- Best/Average: O(log n) - Balanced partitions
- Worst Case: O(n) - Unbalanced partitions

Memory Usage:

```
Input array: n elements
Recursion stack: O(log n) average
Total: O(n) + O(log n)
```

When to Use Quick Sort

Use When:

- Average case performance is good
- In-place sort is needed
- Cache performance matters
- Pivot selection is random

Avoid When:

- Worst-case performance is critical
- Array is already sorted (can use randomization)
- Stable sort is required

Pivot Selection Strategies

```
// 1. First element pivot
int pivot = arr[low];

// 2. Last element pivot (used above)
int pivot = arr[high];

// 3. Middle element pivot
int mid = (low + high) / 2;
int pivot = arr[mid];

// 4. Random pivot
int randomIndex = low + random.nextInt(high - low + 1);
int pivot = arr[randomIndex];

// 5. Median of three
int mid = (low + high) / 2;
int pivot = median(arr[low], arr[mid], arr[high]);
```

Comparison Summary

Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Recursive Bubble	O(n)	O(n²)	O(n²)	O(n)	Yes
Recursive Insertion	O(n)	O(n²)	O(n²)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No

— End of enhanced Chapter 1: Sorting Algorithms — remaining chapters continue in the full edition —

Arrays Problems — Complete Solutions

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

8. Largest Element in an Array

Understanding the Problem NEW

You're handed a list of numbers and asked: which one is biggest? There's no trick here — you just need to look at every number once and remember the biggest you've seen so far. Think of it like scanning a row of people's heights and keeping a mental note of the tallest person you've spotted as you walk down the line.

Brute Force Approach NEW

A beginner's first instinct is often to compare every element with every other element (nested loops) to confirm which one is bigger than all the rest — that's `O(n²)` work for something that doesn't need it.

Why We Optimize NEW

The insight: you don't need to compare every pair. You only need **one running variable** — the biggest value seen so far — and update it as you go. One pass, one comparison per element, done. That takes brute force's O(n²) down to O(n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the maximum element in an array.

Visual Diagram

```
Array: [10, 25, 5, 30, 15]

          Step 1: max = 10
          [10, 25, 5, 30, 15]
           ↑
          max=10

          Step 2: Compare 25 > 10 → max = 25
          [10, 25, 5, 30, 15]
               ↑
             max=25

          Step 3: Compare 5 > 25 → No
          [10, 25, 5, 30, 15]
```

```
                  ↑
                max=25

          Step 4: Compare 30 > 25 → max = 30
          [10, 25, 5, 30, 15]
                       ↑
                    max=30

          Step 5: Compare 15 > 30 → No
          [10, 25, 5, 30, 15]
                           ↑

                        max=30

       Result: 30
```

Solution Approach

Concept: Initialize max with first element and traverse the array.

Algorithm

```
Algorithm FIND_MAX(array):
           max = array[0]
           For i = 1 to n-1:
               If array[i] > max:
                   max = array[i]
           Return max
```

Java Implementation

```
class Solution {
           public int findLargest(int[] arr) {
               int max = arr[0];
               for (int i = 1; i < arr.length; i++) {
                   if (arr[i] > max) {
                        max = arr[i];
                   }
               }
               return max;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

9. Second Largest Element in an Array

Understanding the Problem NEW

Now you want not just the biggest number, but the one right behind it — the second highest. If the array were `[10, 25, 5, 30, 15]`, the largest is 30 and the second largest is 25 (not 15, and not another copy of 30 if it repeated).

Brute Force Approach NEW

The obvious first approach: sort the whole array, then read off the second-to-last element. Sorting costs `O(n log n)` — way more work than this simple question deserves.

Why We Optimize NEW

Just like finding the max needed only one running variable, finding the top-2 needs only **two** running variables — `first` and `second` — updated together in a single pass. Whenever a new number beats `first`, the old `first` slides down into `second` before being replaced. This avoids sorting entirely and gets you to O(n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the second largest element in an array.

Visual Diagram

```
Array: [10, 25, 5, 30, 15]

       Step 1: first = 10, second = -inf
       [10, 25, 5, 30, 15]
        ↑
       first=10, second=-∞

       Step 2: 25 > 10 → second=10, first=25
       [10, 25, 5, 30, 15]
            ↑
          first=25, second=10

       Step 3: 5 between second and first → No
```

```
[10, 25, 5, 30, 15]
         ↑

Step 4: 30 > 25 → second=25, first=30
[10, 25, 5, 30, 15]
             ↑
           first=30, second=25

Step 5: 15 between second and first → No
[10, 25, 5, 30, 15]
                 ↑

Result: 25
```

Solution Approach

Concept: Track both largest and second largest in one pass.

Algorithm

```
Algorithm FIND_SECOND_MAX(array):
           first = -∞, second = -∞
           For each element x in array:
               If x > first:
                   second = first
                   first = x
               Else If x > second and x != first:
                   second = x
           Return second
```

Java Implementation

```
class Solution {
           public int findSecondLargest(int[] arr) {
               int first = Integer.MIN_VALUE;
               int second = Integer.MIN_VALUE;

                 for (int num : arr) {
                     if (num > first) {
                         second = first;
                         first = num;
                     } else if (num > second && num != first) {

                          second = num;
                    }
              }
              return second;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

10. Check if the Array is Sorted

Understanding the Problem NEW

You're checking a simple yes/no question: as you read the array left to right, does every number stay the same or get bigger than the one before it? If even once a number is smaller than its predecessor, the array isn't sorted.

Brute Force Approach NEW

A naive approach some beginners try is comparing every pair of elements (i, j) to check i comes before j only if arr[i] <= arr[j] for all such pairs — unnecessary $O(n^2)$ work.

Why We Optimize NEW

Sortedness is really a statement about **neighbours**, not about every possible pair. If every adjacent pair is in order, the whole array must be in order (this is easy to prove — if a ≤ b and b ≤ c, then a ≤ c automatically). So checking only n-1 adjacent pairs is both correct and sufficient, giving O(n) instead of $O(n^2)$.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if an array is sorted in non-decreasing order.

Visual Diagram

```
Array: [1, 2, 3, 4, 5]

       Step 1: 1 <= 2 → Continue
       [1, 2, 3, 4, 5]
        ↑ ↑

       Step 2: 2 <= 3 → Continue
       [1, 2, 3, 4, 5]
           ↑ ↑

       Step 3: 3 <= 4 → Continue
       [1, 2, 3, 4, 5]
              ↑ ↑
```

```
Step 4: 4 <= 5 → Continue
[1, 2, 3, 4, 5]
          ↑ ↑

Result: Sorted

Array: [1, 3, 2, 4, 5]
Step: 3 > 2 → Not Sorted
[1, 3, 2, 4, 5]
    ↑ ↑
```

Solution Approach

Concept: Check if each element is less than or equal to next element.

Algorithm

```
Algorithm IS_SORTED(array):
          For i = 0 to n-2:
              If array[i] > array[i+1]:
                  Return false
          Return true
```

Java Implementation

```
class Solution {
          public boolean isSorted(int[] arr) {
              for (int i = 0; i < arr.length - 1; i++) {
                  if (arr[i] > arr[i+1]) {
                       return false;
                  }
              }
              return true;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

11. Remove Duplicates from Sorted Array

Understanding the Problem NEW

Given a sorted array with repeated values like `[1,1,2,2,3,4,4,5]`, you need to squeeze out the duplicates and shift the remaining unique values to the front, in-place (without allocating a new array), then report how many unique values remain.

Brute Force Approach NEW

A beginner might use a separate hash set or list to collect unique values, then copy them back into the original array. This works but uses O(n) extra space, when the sorted structure of the input makes that unnecessary.

Why We Optimize NEW

Because the array is **already sorted**, duplicates are always sitting right next to each other. That means you can use two pointers: one (`i`) marking the last confirmed-unique position, and one (`j`) scanning ahead. Whenever `arr[j]` differs from `arr[i]`, you've found a new unique value — advance `i` and write it there. This gets the job done in-place in O(n) time and O(1) extra space, exploiting the sorted order instead of ignoring it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Remove duplicates from sorted array in-place.

Visual Diagram

```
Array: [1, 1, 2, 2, 3, 4, 4, 5]

       Step 1: i=0, j=1
       [1, 1, 2, 2, 3, 4, 4, 5]
        ↑ ↑
       i j

       Step 2: arr[0] == arr[1] → j++
       [1, 1, 2, 2, 3, 4, 4, 5]
         ↑     ↑
       i     j
```

```
Step 3: arr[1] != arr[2] → arr[++i] = arr[j]
[1, 2, 2, 2, 3, 4, 4, 5]
    ↑     ↑
    i     j

Step 4: Continue...
[1, 2, 3, 4, 5, 4, 4, 5]
          ↑       ↑
          i       j

Result: [1, 2, 3, 4, 5]
```

Solution Approach

Concept: Use two-pointer technique to overwrite duplicates.

Algorithm

```
Algorithm REMOVE_DUPLICATES(array):
          If array empty: Return 0
          i = 0
          For j = 1 to n-1:
              If array[j] != array[i]:
                  i++
                  array[i] = array[j]
          Return i+1
```

Java Implementation

```
class Solution {
          public int removeDuplicates(int[] arr) {
              if (arr.length == 0) return 0;

                int i = 0;
                for (int j = 1; j < arr.length; j++) {
                    if (arr[j] != arr[i]) {
                        i++;
                        arr[i] = arr[j];
                    }
                }
                return i + 1;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

12. Left Rotate an Array by One Place

Understanding the Problem NEW

You need to shift every element one position to the left, and whatever falls off the front wraps around to become the new last element. `[1,2,3,4,5]` becomes `[2,3,4,5,1]`.

Brute Force Approach NEW

Beginners sometimes build a brand-new array, copy elements 1..n-1 into positions 0..n-2, then place the old first element at the end — correct, but it uses O(n) extra space unnecessarily.

Why We Optimize NEW

You can do this in-place: save the first element in a temporary variable (so it isn't lost), shift everything else left by writing over it, then drop the saved value into the last slot. Same O(n) time, but O(1) space instead of O(n) — no separate array needed.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Rotate array elements left by one position.

Visual Diagram

```
Array: [1, 2, 3, 4, 5]

       Step 1: Store first element
       temp = 1
       [1, 2, 3, 4, 5]

       Step 2: Shift left
       [2, 3, 4, 5, 5]

       Step 3: Place temp at end
       [2, 3, 4, 5, 1]

       Result: [2, 3, 4, 5, 1]
```

Solution Approach

Concept: Store first element, shift all elements left, place stored element at end.

Algorithm

```
Algorithm LEFT_ROTATE_BY_ONE(array):
           temp = array[0]
           For i = 0 to n-2:
               array[i] = array[i+1]
           array[n-1] = temp
           Return array
```

Java Implementation

```
class Solution {
           public void leftRotateByOne(int[] arr) {
               int temp = arr[0];
               for (int i = 0; i < arr.length - 1; i++) {
                   arr[i] = arr[i+1];
               }
               arr[arr.length - 1] = temp;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

13. Left Rotate an Array by D Places

Understanding the Problem NEW

Same idea as before, but now you rotate left by `D` places instead of just one — e.g. rotating `[1,2,3,4,5,6,7]` left by 3 gives `[4,5,6,7,1,2,3]`.

Brute Force Approach NEW

The straightforward brute idea: rotate by one place, D separate times, in a loop. Each single rotation costs O(n), so doing it D times costs O(n·D) — slow when D is large.

Why We Optimize NEW

The elegant trick is the **reversal algorithm**: reverse the first D elements, reverse the remaining n-D elements, then reverse the whole array. Each reversal is O(n), and you only do 3 of them regardless of D, giving a flat O(n) total instead of O(n·D). This is a good example of how a clever transformation (reversing pieces) can replace repeated brute work.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Rotate array elements left by D positions.

Visual Diagram

```
Array: [1, 2, 3, 4, 5, 6, 7], D = 3

       Step 1: Reverse first D elements
       [3, 2, 1, 4, 5, 6, 7]

       Step 2: Reverse remaining elements
       [3, 2, 1, 7, 6, 5, 4]

       Step 3: Reverse whole array
       [4, 5, 6, 7, 1, 2, 3]

       Result: [4, 5, 6, 7, 1, 2, 3]
```

Solution Approach

Concept: Use reversal algorithm (three reverses).

Algorithm

```
Algorithm LEFT_ROTATE_D(array, d):
           d = d % n
           REVERSE(array, 0, d-1)
           REVERSE(array, d, n-1)
           REVERSE(array, 0, n-1)
           Return array
```

Java Implementation

```
class Solution {
           public void leftRotate(int[] arr, int d) {
               int n = arr.length;
               d = d % n;
               reverse(arr, 0, d-1);
               reverse(arr, d, n-1);
               reverse(arr, 0, n-1);
           }

           private void reverse(int[] arr, int start, int end) {
               while (start < end) {
                   int temp = arr[start];
                   arr[start] = arr[end];
                   arr[end] = temp;
                   start++;
                   end--;
               }
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

14. Move Zeros to End

Understanding the Problem NEW

Given an array with some zeros scattered in it, like `[0,1,0,3,12]`, push all the zeros to the end while keeping the relative order of the non-zero numbers the same, producing `[1,3,12,0,0]`.

Brute Force Approach NEW

A beginner approach: create a new array, copy all non-zero elements into it first, then fill the rest with zeros, and copy back. This works but needs O(n) extra space.

Why We Optimize NEW

You can do it in-place with two pointers: one pointer marks where the next non-zero element should go; you scan through the array, and every time you see a non-zero value you swap it into that position and advance the marker. By the end, all non-zero values have been shifted left in order, and the zeros naturally end up at the back — all in O(n) time and O(1) space.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Move all zeros to the end while maintaining relative order of non-zero elements.

Visual Diagram

```
Array: [0, 1, 0, 3, 12]

       Step 1: i=0, j=0
       [0, 1, 0, 3, 12]
        ↑
       i,j

       Step 2: arr[j]==0 → j++
       [0, 1, 0, 3, 12]
          ↑  ↑
       i    j

       Step 3: arr[j]!=0 → swap arr[i], arr[j]
```

```
       [1, 0, 0, 3, 12]
           ↑ ↑
           i j

       Step 4: i++, j++
       [1, 0, 0, 3, 12]
              ↑ ↑
              i j

       Step 5: arr[j]!=0 → swap arr[i], arr[j]
       [1, 3, 0, 0, 12]
                 ↑ ↑
                 i j

       Step 6: i++, j++
       [1, 3, 0, 0, 12]
                    ↑ ↑
                    i j

       Step 7: arr[j]!=0 → swap arr[i], arr[j]
       [1, 3, 12, 0, 0]
                     ↑ ↑
                     i j

       Result: [1, 3, 12, 0, 0]
```

Solution Approach

Concept: Two-pointer technique - one pointer for non-zero elements.

Algorithm

```
Algorithm MOVE_ZEROS(array):
          i = 0
          For j = 0 to n-1:
              If array[j] != 0:
                  Swap array[i] and array[j]
                  i++
          Return array
```

Java Implementation

```
class Solution {
         public void moveZeroes(int[] arr) {
             int i = 0;
             for (int j = 0; j < arr.length; j++) {
                 if (arr[j] != 0) {
                      int temp = arr[i];
                      arr[i] = arr[j];
                      arr[j] = temp;
```

```
                    i++;
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

15. Linear Search

Understanding the Problem NEW

You're given an array and a target value, and you need to find the index of that value in the array (or report that it isn't there). There's no sorted order assumed here.

Brute Force Approach NEW

This *is* the brute-force-friendly problem: without any extra structure (sorting, hashing), the only correct way to guarantee finding a value in an unsorted array is to check every element one at a time.

Why We Optimize NEW

Linear Search is a case where the 'brute force' approach is actually already optimal — there's no way to search an arbitrary unsorted array faster than O(n) in the worst case, because the target could be anywhere, including the very last position. The lesson here is important: **optimization is only possible when there's exploitable structure** (sorted data → binary search, repeated lookups → hashing). Plain unsorted, one-shot search has no shortcut.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the index of a target element in an array.

Visual Diagram

```
Array: [10, 25, 5, 30, 15], Target: 30

       Step 1: Compare arr[0]=10 == 30? No
       [10, 25, 5, 30, 15]
        ↑

       Step 2: Compare arr[1]=25 == 30? No
       [10, 25, 5, 30, 15]
            ↑

       Step 3: Compare arr[2]=5 == 30? No
       [10, 25, 5, 30, 15]
```

```
                ↑

       Step 4: Compare arr[3]=30 == 30? Yes!
       [10, 25, 5, 30, 15]
                    ↑

       Result: Index = 3
```

Solution Approach

Concept: Traverse array and compare each element with target.

Algorithm

```
Algorithm LINEAR_SEARCH(array, target):
           For i = 0 to n-1:
               If array[i] == target:
                   Return i
           Return -1
```

Java Implementation

```
class Solution {
           public int linearSearch(int[] arr, int target) {
               for (int i = 0; i < arr.length; i++) {
                   if (arr[i] == target) {
                        return i;
                   }
               }
               return -1;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

16. Find the Union

Understanding the Problem NEW

You're given two *sorted* arrays and need to produce a single sorted array containing every value that appears in either one, with no duplicates — like combining two sorted guest lists into one, without repeating a name.

Brute Force Approach NEW

A naive approach: dump both arrays into a hash set to remove duplicates, then sort the result. That costs O((n+m) log(n+m)) for the final sort, ignoring the fact both inputs were already sorted.

Why We Optimize NEW

Because both arrays are sorted, you can walk them simultaneously with **two pointers**, always advancing whichever pointer has the smaller current value (and skipping over duplicates as you go). This produces the merged, duplicate-free result in a single O(n+m) pass with no extra sorting needed — reusing the sorted order you were already given instead of throwing it away.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find union of two sorted arrays.

Visual Diagram

```
Array1: [1, 2, 3, 4, 5]
       Array2: [2, 3, 5, 6, 7]

       Step-by-step:
       i=0, j=0: 1<2 → add 1, i++
       i=1, j=0: 2==2 → add 2, i++, j++
       i=2, j=1: 3==3 → add 3, i++, j++
       i=3, j=2: 4<5 → add 4, i++
       i=4, j=2: 5==5 → add 5, i++, j++
       i=5, j=3: j<len → add 6, j++
       i=5, j=4: j<len → add 7, j++
```

```
       Result: [1, 2, 3, 4, 5, 6, 7]
```

Solution Approach

Concept: Use two-pointer technique for sorted arrays.

Algorithm

```
Algorithm UNION(arr1, arr2):
           i=0, j=0
           result = []
           While i < n and j < m:
               If arr1[i] < arr2[j]:
                    Add arr1[i] to result if not duplicate
                    i++
               Else If arr1[i] > arr2[j]:
                    Add arr2[j] to result if not duplicate
                    j++
               Else:
                    Add arr1[i] to result
                    i++; j++
           Add remaining elements
           Return result
```

Java Implementation

```
class Solution {
           public List<Integer> union(int[] arr1, int[] arr2) {
               List<Integer> result = new ArrayList<>();
               int i = 0, j = 0;

                 while (i < arr1.length && j < arr2.length) {
                     if (arr1[i] < arr2[j]) {
                         if (result.isEmpty() || result.get(result.size()-1) !=
arr1[i]) {
                              result.add(arr1[i]);
                         }
                         i++;
                     } else if (arr1[i] > arr2[j]) {
                         if (result.isEmpty() || result.get(result.size()-1) !=
arr2[j]) {
                              result.add(arr2[j]);
                         }
                         j++;
                     } else {

                      if (result.isEmpty() || result.get(result.size()-1) !=
arr1[i]) {
                          result.add(arr1[i]);
```

```
                    }
                    i++; j++;
                }
            }

            while (i < arr1.length) {
                if (result.isEmpty() || result.get(result.size()-1) != arr1[i]) {
                     result.add(arr1[i]);
                }
                i++;
            }

            while (j < arr2.length) {
                if (result.isEmpty() || result.get(result.size()-1) != arr2[j]) {
                     result.add(arr2[j]);
                }
                j++;
            }

            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n+m) | Space Complexity: O(n+m)

17. Find Missing Number in an Array

Understanding the Problem NEW

You're given an array that's supposed to contain every number from 0 to n exactly once, but one number is missing. Find which one. E.g. `[3,0,1]` is missing 2.

Brute Force Approach NEW

The obvious brute approach: for every number from 0 to n, scan the whole array checking if it's present. That's O(n) numbers × O(n) scan each = O(n²) total.

Why We Optimize NEW

There's a much cleverer shortcut: you know exactly what the sum of 0..n *should* be (using the formula n(n+1)/2). Subtract the *actual* sum of the array from that expected sum, and whatever's left over is the missing number. (XOR-ing all numbers 0..n together with all array elements works the same way and avoids overflow concerns.) This turns an O(n²) search into a single O(n) pass with basic arithmetic.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the missing number from 0 to n in an array of size n.

Visual Diagram

```
Array: [3, 0, 1, 4], n=4

       Method 1: Sum
       Expected sum = 0+1+2+3+4 = 10
       Actual sum = 3+0+1+4 = 8
       Missing = 10-8 = 2

       Method 2: XOR
       XOR all numbers: 3^0^1^4 = 6
       XOR all indices: 0^1^2^3^4 = 4
       Missing = 6^4 = 2
```

```
Result: 2
```

Solution Approach

Concept: Use sum formula or XOR.

Algorithm (Sum Method)

Algorithm MISSING_NUMBER(array, n):

expected_sum = n * (n+1) / 2

actual_sum = sum(array)

Return expected_sum - actual_sum

Algorithm (XOR Method)

Algorithm MISSING_NUMBER_XOR(array, n):

xor_all = 0

For i = 0 to n:

xor_all ^= i

For each num in array:

xor_all ^= num

Return xor_all

Java Implementation

```
class Solution {
           // Sum Method
           public int missingNumber(int[] arr) {
               int n = arr.length;
               int expectedSum = n * (n + 1) / 2;
               int actualSum = 0;
               for (int num : arr) {
                   actualSum += num;
               }
               return expectedSum - actualSum;
           }

           // XOR Method (prevents overflow)
           public int missingNumberXOR(int[] arr) {
               int xor = 0;
               for (int i = 0; i <= arr.length; i++) {
```

```
                xor ^= i;
            }

            for (int num : arr) {
                xor ^= num;
            }
            return xor;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

18. Maximum Consecutive Ones

Understanding the Problem NEW

Given a binary array (only 0s and 1s), find the longest unbroken run of consecutive 1s. E.g. in `[1,1,0,1,1,1]` the answer is 3 (the last three 1s).

Brute Force Approach NEW

A brute way: for every starting index, keep extending forward counting 1s until you hit a 0, tracking the best length found. That's checking every possible subarray start, costing $O(n^2)$ in the worst case.

Why We Optimize NEW

You don't need to restart from every index — you can track a **running counter** that increases by 1 for each consecutive 1, and resets to 0 the moment you see a 0, updating a 'best so far' as you go. One single left-to-right pass, O(n) time, no nested loops needed.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the maximum number of consecutive 1s in a binary array.

Visual Diagram

```
Array: [1, 1, 0, 1, 1, 1, 0, 1, 1]

       Step 1: count=1, max=1
       [1, 1, 0, 1, 1, 1, 0, 1, 1]
        ↑

       Step 2: count=2, max=2
       [1, 1, 0, 1, 1, 1, 0, 1, 1]
           ↑

       Step 3: 0 → reset count=0
       [1, 1, 0, 1, 1, 1, 0, 1, 1]
              ↑

       Step 4: count=1
       [1, 1, 0, 1, 1, 1, 0, 1, 1]
```

```
                ↑

       Step 5: count=2
       [1, 1, 0, 1, 1, 1, 0, 1, 1]
                   ↑

       Step 6: count=3, max=3
       [1, 1, 0, 1, 1, 1, 0, 1, 1]
                      ↑

       Step 7: 0 → reset
       Step 8-9: count=2

       Result: 3
```

Solution Approach

Concept: Count consecutive ones and track maximum.

Algorithm

```
Algorithm MAX_CONSECUTIVE_ONES(array):
           max_count = 0, count = 0
           For each element x in array:
               If x == 1:
                   count++
                   max_count = max(max_count, count)
               Else:
                   count = 0
           Return max_count
```

Java Implementation

```
class Solution {
           public int findMaxConsecutiveOnes(int[] arr) {
               int maxCount = 0;
               int count = 0;

                 for (int num : arr) {

                  if (num == 1) {
                      count++;
                      maxCount = Math.max(maxCount, count);
                  } else {
                      count = 0;
                  }
              }
              return maxCount;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

19. Find the Number that Appears Once

Understanding the Problem NEW

Every number in the array appears exactly twice, except one number that appears only once — find that one number. E.g. in `[4,1,2,1,2]` the answer is 4.

Brute Force Approach NEW

A beginner approach: for each element, count how many times it appears elsewhere in the array (nested loop), and return the one whose count is 1. That's $O(n^2)$. A slightly better brute idea uses a hashmap to count frequencies in O(n) time but O(n) extra space.

Why We Optimize NEW

There's a beautifully cheap trick using the **XOR** operation: XOR-ing a number with itself gives 0 (`a ^ a = 0`), and XOR-ing with 0 leaves a number unchanged (`a ^ 0 = a`). If you XOR every element in the array together, every pair cancels itself out to 0, leaving only the lone number standing. This gets you O(n) time *and* O(1) space — better than even the hashmap version — purely by exploiting a mathematical property of XOR.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the element that appears once in an array where every other element appears twice.

Visual Diagram

```
Array: [4, 1, 2, 1, 2]

       Step 1: XOR all elements
       4 ^ 1 = 5
       5 ^ 2 = 7
       7 ^ 1 = 6
       6 ^ 2 = 4

       XOR Properties:
       a ^ a = 0
       a ^ 0 = a
```

```
       4 ^ 1 ^ 2 ^ 1 ^ 2 = 4 ^ (1^1) ^ (2^2) = 4 ^ 0 ^ 0 = 4

       Result: 4
```

Solution Approach

Concept: Use XOR property where a^a=0.

Algorithm

```
Algorithm SINGLE_NUMBER(array):
           result = 0
           For each num in array:
               result ^= num
           Return result
```

Java Implementation

```
class Solution {
           public int singleNumber(int[] arr) {
               int result = 0;
               for (int num : arr) {
                   result ^= num;
               }
               return result;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

20. Longest Subarray with Given Sum K (Positive Numbers)

Understanding the Problem NEW

Given an array of **positive** numbers and a target sum K, find the length of the longest contiguous subarray whose elements add up to exactly K.

Brute Force Approach NEW

The naive approach checks every possible subarray (every pair of start and end indices), summing each one and comparing to K — $O(n^2)$ or, if you recompute the sum from scratch each time, even $O(n^3)$.

Why We Optimize NEW

Because all numbers are **positive**, growing the window (adding an element) only ever increases the sum, and shrinking it only ever decreases the sum — this monotonic behaviour is exactly what a **sliding window** needs. You expand the right edge of a window, and whenever the sum exceeds K you shrink from the left, tracking the best length seen. Each pointer only moves forward, so the whole process is O(n) — the positivity of the numbers is precisely the property that makes this optimization valid (it wouldn't work with negative numbers, which is why problem 21 needs a different technique).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the length of longest subarray with sum K (positive numbers only).

Visual Diagram

```
Array: [1, 2, 3, 1, 1, 1], K=3

       Brute Force:
       Subarray [1,2] sum=3 length=2
       Subarray [3] sum=3 length=1
       Subarray [1,1,1] sum=3 length=3

       Optimal (Two Pointer):
       [1, 2, 3, 1, 1, 1]
```

```
        ↑
       left, right
       sum=1

       [1, 2, 3, 1, 1, 1]
        ↑ ↑
       left right
       sum=3 length=2

       [1, 2, 3, 1, 1, 1]
        ↑     ↑
       left   right
       sum=6 > 3 → left++

       [1, 2, 3, 1, 1, 1]
           ↑ ↑
          left right
       sum=5 > 3 → left++

       [1, 2, 3, 1, 1, 1]
              ↑ ↑
             left right
       sum=3 length=1

       Continue...
       [1, 2, 3, 1, 1, 1]
                 ↑ ↑
                left right
       sum=2

       [1, 2, 3, 1, 1, 1]
                 ↑    ↑
                left right
       sum=3 length=3

       Result: 3
```

Solution Approach

Concept: Two-pointer sliding window for positive numbers.

Algorithm

```
Algorithm LONGEST_SUBARRAY_SUM_K(arr, K):
           left=0, right=0, sum=0, max_len=0
           While right < n:

              sum += arr[right]
              While sum > K and left <= right:
                  sum -= arr[left]
                  left++
```

```
            If sum == K:
                max_len = max(max_len, right-left+1)
            right++
        Return max_len
```

Java Implementation

```
class Solution {
        public int longestSubarrayWithSumK(int[] arr, int K) {
            int left = 0, right = 0;
            int sum = 0, maxLen = 0;

            while (right < arr.length) {
                sum += arr[right];

                while (sum > K && left <= right) {
                    sum -= arr[left];
                    left++;
                }

                if (sum == K) {
                    maxLen = Math.max(maxLen, right - left + 1);
                }

                right++;
            }
            return maxLen;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

21. Longest Subarray with Sum K (Positive and Negative)

Understanding the Problem NEW

Same question as above, but now the array can contain **negative** numbers too, which breaks the sliding-window trick from problem 20 (a window's sum is no longer guaranteed to grow as it expands).

Brute Force Approach NEW

Checking every subarray directly is still O(n²) and doesn't get any easier just because negatives are allowed.

Why We Optimize NEW

The fix is to track **prefix sums**: the running total from the start of the array up to each index. If two prefix sums differ by exactly K, the subarray between those two points sums to K. Store the *first* index where each prefix sum was seen in a hashmap; then for each new prefix sum `S`, check if `S - K` has been seen before, and if so, compute the subarray length. This turns the problem into one hashmap lookup per element — O(n) time (with O(n) space for the map) — and works correctly even with negative numbers, unlike the sliding-window approach.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the length of longest subarray with sum K (including negative numbers).

Visual Diagram

```
Array: [1, 2, -1, 2, 3], K=4

       Prefix Sum Method:
       Index:   0 1 2 3      4
       Array:   1 2 -1 2     3
       Prefix: 1 3 2 4       7

       For target K=4:
       Check prefix[i] - K exists in map
```

```
       i=3: prefix=4, 4-4=0 exists? No
       But we can check: prefix[3] - prefix[0] = 4-1=3 (not K)
       prefix[3] - prefix[2] = 4-2=2 (not K)
       prefix[3] - prefix[1] = 4-3=1 (not K)

       Actually: subarray [1,2,-1,2] sum=4 length=4
       Found: prefix[3] - prefix[-1] (where we store 0 at index -1)

       Result: 4
```

Solution Approach

Concept: Prefix sum with hashmap.

Algorithm

```
Algorithm LONGEST_SUBARRAY_K(arr, K):
           map = {0: -1}
           sum = 0, max_len = 0
           For i = 0 to n-1:
               sum += arr[i]
               If map contains (sum - K):
                   max_len = max(max_len, i - map[sum-K])
               If map doesn't contain sum:
                   map[sum] = i
           Return max_len
```

Java Implementation

```
class Solution {
           public int longestSubarrayWithSumK(int[] arr, int K) {
               Map<Integer, Integer> map = new HashMap<>();
               map.put(0, -1);

                 int sum = 0, maxLen = 0;

                 for (int i = 0; i < arr.length; i++) {
                     sum += arr[i];

                    if (map.containsKey(sum - K)) {
                        maxLen = Math.max(maxLen, i - map.get(sum - K));
                    }

                    if (!map.containsKey(sum)) {
                        map.put(sum, i);

                 }
             }

             return maxLen;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

22. Two Sum Problem

Understanding the Problem NEW

You're given an array and a target sum, and need to find the indices of the two numbers that add up to exactly that target. E.g. `[2,7,11,15]`, target 9 → indices [0,1] since 2+7=9.

Brute Force Approach NEW

The obvious approach: for every pair of elements, check if they add up to the target — two nested loops, $O(n^2)$ comparisons.

Why We Optimize NEW

For each number `x` you visit, what you actually need to know is: 'have I already seen the number `target - x`?' Instead of searching the whole array again to answer that (which is what the nested-loop approach effectively does), you can store every number you've seen so far in a **hashmap** as you go, so that lookup becomes instant (O(1) average). This drops the total time to a single O(n) pass, at the cost of O(n) extra space for the map.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find two numbers that sum to target.

Visual Diagram

```
Array: [2, 7, 11, 15], Target: 9

       Brute Force:
       2+7=9 ✓ (0,1)
       2+11=13 ✗
       2+15=17 ✗
       7+11=18 ✗
       7+15=22 ✗
       11+15=26 ✗

       HashMap Method:
       Step 1: 2 → need=7 (store 2 at index 0)
       Step 2: 7 → need=2 (found at index 0)
```

```
       Return [0, 1]

       Result: [0, 1]
```

Solution Approach

Concept: Hashmap to store complement.

Algorithm

```
Algorithm TWO_SUM(arr, target):
           map = {}
           For i = 0 to n-1:
               complement = target - arr[i]
               If complement in map:
                    Return [map[complement], i]
               map[arr[i]] = i
           Return []
```

Java Implementation

```
class Solution {
           public int[] twoSum(int[] nums, int target) {
               Map<Integer, Integer> map = new HashMap<>();

                 for (int i = 0; i < nums.length; i++) {
                     int complement = target - nums[i];
                     if (map.containsKey(complement)) {
                         return new int[]{map.get(complement), i};
                     }
                     map.put(nums[i], i);
                 }
                 return new int[0];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

23. Sort an Array of 0's, 1's and 2's

Understanding the Problem NEW

An array contains only the values 0, 1, and 2 in random order, and you need to sort it — but the challenge is to do it without calling a generic sorting algorithm, ideally in a single pass.

Brute Force Approach NEW

A beginner's first idea is to just run any comparison sort (like the ones from Chapter 1) on it — correct, but O(n log n), which ignores the fact there are only 3 distinct values, so much less work should be needed. A slightly better brute idea: count the 0s, 1s, and 2s, then overwrite the array in three passes — this works in O(n) but needs two full passes.

Why We Optimize NEW

The **Dutch National Flag** algorithm gets it done in a *single* pass using three pointers — `low`, `mid`, and `high` — that partition the array into 'confirmed 0s', 'confirmed 1s' (being scanned), and 'confirmed 2s' regions as you go, swapping elements into the right region the moment you see them. No sorting comparisons at all — just constant-time swaps based on which of the 3 fixed values you're looking at, giving O(n) time and O(1) space in one pass.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Sort array containing only 0, 1, 2 without using sorting algorithm.

Visual Diagram

```
Array: [2, 0, 2, 1, 1, 0]

       Dutch National Flag Algorithm:
       low=0, mid=0, high=5

       Step 1: arr[mid]=2 → swap with high
       [0, 0, 2, 1, 1, 2] high=4
            ↑
            mid
```

```
       Step 2: arr[mid]=0 → swap with low
       [0, 0, 2, 1, 1, 2] low=1, mid=1
               ↑
               mid

       Step 3: arr[mid]=0 → swap with low
       [0, 0, 2, 1, 1, 2] low=2, mid=2
                  ↑
                  mid

       Step 4: arr[mid]=2 → swap with high
       [0, 0, 1, 1, 2, 2] high=3
                  ↑
                  mid

       Step 5: arr[mid]=1 → mid++
       [0, 0, 1, 1, 2, 2] mid=4
                     ↑
                     mid

       Step 6: arr[mid]=1 → mid++
       [0, 0, 1, 1, 2, 2] mid=5

       Result: [0, 0, 1, 1, 2, 2]
```

Solution Approach

Concept: Dutch National Flag algorithm (3-pointer).

Algorithm

```
Algorithm SORT_012(arr):
          low=0, mid=0, high=n-1
          While mid <= high:
              If arr[mid] == 0:
                  Swap arr[low] and arr[mid]
                  low++; mid++
              Else If arr[mid] == 1:
                  mid++
              Else: // arr[mid] == 2
                  Swap arr[mid] and arr[high]
                  high--
          Return arr
```

Java Implementation

```
class Solution {
         public void sortColors(int[] nums) {
             int low = 0, mid = 0, high = nums.length - 1;
```

```
            while (mid <= high) {
                if (nums[mid] == 0) {
                    swap(nums, low, mid);
                    low++;
                    mid++;
                } else if (nums[mid] == 1) {
                    mid++;
                } else {
                    swap(nums, mid, high);
                    high--;
                }
            }
        }

        private void swap(int[] arr, int i, int j) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

24. Majority Element (>n/2 times)

Understanding the Problem NEW

Find the element that appears **more than n/2 times** in the array (it's guaranteed such an element exists). E.g. in `[2,2,1,1,1,2,2]`, 2 appears more than half the time.

Brute Force Approach NEW

Beginners often count occurrences of every distinct element (nested loop or hashmap) and check which count exceeds n/2 — this works in O(n) with a hashmap, but uses O(n) extra space.

Why We Optimize NEW

Since the majority element appears more than *half* the array, it's guaranteed to 'outlast' every other element combined if you pair them off and cancel them out. That's the idea behind **Moore's Voting Algorithm**: keep a running candidate and a counter; increment the counter when you see the candidate again, decrement it for anything else, and switch candidates when the counter hits zero. Whatever candidate survives to the end must be the majority element. This achieves O(n) time with just O(1) space — no hashmap needed — because the 'more than half' guarantee is exactly what makes the cancellation trick mathematically safe.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find element that appears more than n/2 times.

Visual Diagram

```
Array: [2, 2, 1, 1, 1, 2, 2]

       Moore's Voting Algorithm:
       Step 1: candidate=2, count=1
       [2, 2, 1, 1, 1, 2, 2]
        ↑

       Step 2: 2==candidate → count=2
```

```
       [2, 2, 1, 1, 1, 2, 2]
           ↑

       Step 3: 1!=candidate → count=1
       [2, 2, 1, 1, 1, 2, 2]
              ↑

       Step 4: 1!=candidate → count=0
       [2, 2, 1, 1, 1, 2, 2]
                 ↑

       Step 5: count=0 → candidate=1, count=1
       [2, 2, 1, 1, 1, 2, 2]
                    ↑

       Step 6: 2!=candidate → count=0
       [2, 2, 1, 1, 1, 2, 2]
                       ↑

       Step 7: count=0 → candidate=2, count=1
       [2, 2, 1, 1, 1, 2, 2]
                          ↑

       Result: candidate=2 (verify: 2 appears 3 times > 3.5)
```

Solution Approach

Concept: Moore's Voting Algorithm.

Algorithm

```
Algorithm MAJORITY_ELEMENT(arr):
           candidate = -1, count = 0
           For each num in arr:
               If count == 0:
                   candidate = num
                   count = 1
               Else If num == candidate:
                   count++
               Else:
                   count--
           Return candidate
```

Java Implementation

```
class Solution {
           public int majorityElement(int[] nums) {

              int candidate = -1;
              int count = 0;
```

```
            for (int num : nums) {
                if (count == 0) {
                    candidate = num;
                    count = 1;
                } else if (num == candidate) {
                    count++;
                } else {
                    count--;
                }
            }

            // Verify majority
            count = 0;
            for (int num : nums) {
                if (num == candidate) count++;
            }
            return count > nums.length / 2 ? candidate : -1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

25. Kadane's Algorithm (Maximum Subarray Sum)

Understanding the Problem NEW

Find the maximum possible sum of any contiguous subarray (a run of consecutive elements) within the array — including the possibility that the best subarray is just a single element, if all numbers are negative.

Brute Force Approach NEW

The naive approach checks every possible subarray's sum (all start/end pairs), which is $O(n^2)$, or $O(n^3)$ if you recompute each subarray's sum from scratch instead of reusing previous work.

Why We Optimize NEW

Kadane's Algorithm asks a smarter local question at each position: **'should I extend the previous subarray by including this element, or is it better to start a fresh subarray here?'** If the running sum so far has gone negative, it can only be dragging future sums down, so it's better to restart from the current element. This is a dynamic-programming insight — the best subarray ending at position i only depends on the best subarray ending at position i-1, not on recomputing everything from scratch — collapsing $O(n^2)$ work into a single O(n) pass.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the maximum sum of a contiguous subarray.

Visual Diagram

```
Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

       Step 1: current=-2, max=-2
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
        ↑

       Step 2: current=max(1, -2+1)=1, max=1
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
            ↑
```

```
       Step 3: current=max(-3, 1-3)=-2, max=1
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                ↑

       Step 4: current=max(4, -2+4)=4, max=4
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                   ↑

       Step 5: current=max(-1, 4-1)=3, max=4
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                        ↑

       Step 6: current=max(2, 3+2)=5, max=5
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                           ↑

       Step 7: current=max(1, 5+1)=6, max=6
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                              ↑

       Step 8: current=max(-5, 6-5)=1, max=6
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                                 ↑

       Step 9: current=max(4, 1+4)=5, max=6
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                                    ↑

       Result: 6 (subarray [4, -1, 2, 1])
```

Solution Approach

Concept: Dynamic programming - either start new subarray or extend existing.

Algorithm

```
Algorithm KADANE(arr):
           current_max = arr[0]
           global_max = arr[0]
           For i = 1 to n-1:
               current_max = max(arr[i], current_max + arr[i])
               global_max = max(global_max, current_max)
           Return global_max
```

Java Implementation

```
class Solution {
          public int maxSubarraySum(int[] nums) {
              int currentMax = nums[0];
              int globalMax = nums[0];
```

```
            for (int i = 1; i < nums.length; i++) {
                currentMax = Math.max(nums[i], currentMax + nums[i]);
                globalMax = Math.max(globalMax, currentMax);
            }
            return globalMax;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

26. Print Subarray with Maximum Subarray Sum

Understanding the Problem NEW

This is the same problem as Kadane's Algorithm above, except now you also need to report *which* subarray (the actual start and end indices) produced that maximum sum, not just the sum itself.

Brute Force Approach NEW

Without tracking indices, you'd need to re-scan the array afterward trying to find a subarray that matches the maximum sum you calculated — extra, redundant work.

Why We Optimize NEW

The fix is simple: while running Kadane's algorithm, whenever you decide to 'restart' the running sum, also update a `start` marker to the current index; and whenever the running sum beats the best-so-far, save the current `start` and current index as the best `(start, end)` pair. No extra pass is needed — you get the actual subarray boundaries for free, in the same single O(n) sweep.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print the subarray that gives maximum sum.

Visual Diagram

```
Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

       Tracking:
       Start: 0, End: 0, Max: -2
       i=1: curr=1, global=1 → start=1, end=1
       i=2: curr=-2 → no change
       i=3: curr=4, global=4 → start=3, end=3
       i=4: curr=3 → no change
       i=5: curr=5, global=5 → start=3, end=5
       i=6: curr=6, global=6 → start=3, end=6
       i=7: curr=1 → no change
       i=8: curr=5 → no change
```

```
Subarray: indices 3 to 6 → [4, -1, 2, 1]
Sum: 6
```

Solution Approach

Concept: Kadane's algorithm with tracking indices.

Algorithm

```
Algorithm MAX_SUBARRAY_PRINT(arr):
           current_sum = arr[0]
           max_sum = arr[0]
           start = 0, end = 0, temp_start = 0

           For i = 1 to n-1:
               If arr[i] > current_sum + arr[i]:
                   current_sum = arr[i]
                   temp_start = i
               Else:
                   current_sum = current_sum + arr[i]

                 If current_sum > max_sum:
                     max_sum = current_sum
                     start = temp_start
                     end = i

           Return subarray from start to end
```

Java Implementation

```
class Solution {
           public List<Integer> maxSubarrayPrint(int[] nums) {
               int currentSum = nums[0];
               int maxSum = nums[0];
               int start = 0, end = 0, tempStart = 0;

                 for (int i = 1; i < nums.length; i++) {
                     if (nums[i] > currentSum + nums[i]) {
                         currentSum = nums[i];
                         tempStart = i;
                     } else {
                         currentSum = currentSum + nums[i];

                  }

                  if (currentSum > maxSum) {
                      maxSum = currentSum;
                      start = tempStart;
                      end = i;
                  }
```

```
            }

            List<Integer> result = new ArrayList<>();
            for (int i = start; i <= end; i++) {
                result.add(nums[i]);
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

27. Stock Buy and Sell

Understanding the Problem NEW

You're given the price of a stock on each day, and you may buy once and sell once (buy before you sell). Find the maximum profit possible. E.g. prices `[7,1,5,3,6,4]` → buy at 1, sell at 6, profit 5.

Brute Force Approach NEW

The obvious approach: for every pair of buy-day and later sell-day, compute the profit and track the best — that's checking all $O(n^2)$ pairs of days.

Why We Optimize NEW

As you scan through the prices left to right, the best profit if you *sold today* only depends on the **lowest price you've seen so far** (since you must buy before you sell). So you just track the running minimum price seen, and at each day compute `currentPrice - minSoFar`, keeping the best result. This reduces the problem to a single O(n) pass with O(1) space, because you never need to look back at all previous days individually — only the cheapest one matters.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find maximum profit from buying and selling stock once.

Visual Diagram

```
Prices: [7, 1, 5, 3, 6, 4]

       Step 1: min_price=7, max_profit=0
       [7, 1, 5, 3, 6, 4]
        ↑
       min=7, profit=0

       Step 2: 1 < 7 → min=1
       [7, 1, 5, 3, 6, 4]
           ↑
         min=1, profit=0

       Step 3: profit = 5-1=4 → max=4
```

```
       [7, 1, 5, 3, 6, 4]
              ↑
           min=1, profit=4

       Step 4: profit = 3-1=2 → max=4
       [7, 1, 5, 3, 6, 4]
                 ↑

       Step 5: profit = 6-1=5 → max=5
       [7, 1, 5, 3, 6, 4]
                    ↑

       Step 6: profit = 4-1=3 → max=5
       [7, 1, 5, 3, 6, 4]
                       ↑

       Result: 5 (Buy at 1, Sell at 6)
```

Solution Approach

Concept: Track minimum price and calculate profit.

Algorithm

```
Algorithm STOCK_PROFIT(prices):
           min_price = INF
           max_profit = 0
           For each price in prices:
               If price < min_price:
                   min_price = price
               max_profit = max(max_profit, price - min_price)
           Return max_profit
```

Java Implementation

```
class Solution {
           public int maxProfit(int[] prices) {
               int minPrice = Integer.MAX_VALUE;
               int maxProfit = 0;

                 for (int price : prices) {
                     if (price < minPrice) {

                      minPrice = price;
                  }
                     maxProfit = Math.max(maxProfit, price - minPrice);
              }
              return maxProfit;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

28. Rearrange Array in Alternating Positive/Negative

Understanding the Problem NEW

Rearrange the array so that positive and negative numbers alternate in position, starting with a positive number, while keeping the relative order of positives among themselves and negatives among themselves.

Brute Force Approach NEW

A common first approach is to sort or repeatedly scan-and-swap to interleave elements in place, which can involve a lot of shifting and easily become $O(n^2)$ if done carelessly.

Why We Optimize NEW

Since you need to *preserve relative order*, the clean approach is to first separate all positives into one list and all negatives into another (a single O(n) pass), then interleave them into the result array two-at-a-time. This costs O(n) time, at the price of O(n) extra space for the two temporary lists — a reasonable trade given that maintaining order in-place without extra space is significantly trickier.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Rearrange array so positive and negative elements alternate.

Visual Diagram

```
Array: [3, 1, -2, -5, 2, -4]

       Step 1: Separate positives and negatives
       Positives: [3, 1, 2]
       Negatives: [-2, -5, -4]

       Step 2: Place alternately
       Index 0: 3 (positive)
       Index 1: -2 (negative)
       Index 2: 1 (positive)
       Index 3: -5 (negative)
       Index 4: 2 (positive)
       Index 5: -4 (negative)
```

```
       Result: [3, -2, 1, -5, 2, -4]
```

Solution Approach

Concept: Separate arrays for positive and negative.

Algorithm

```
Algorithm REARRANGE_ALTERNATE(arr):
           positives = []
           negatives = []
           For each num in arr:
               If num >= 0: positives.add(num)
               Else: negatives.add(num)

           i=0, j=0, k=0
           While i < positives.size() and j < negatives.size():
               arr[k++] = positives[i++]
               arr[k++] = negatives[j++]
           While i < positives.size():
               arr[k++] = positives[i++]
           While j < negatives.size():
               arr[k++] = negatives[j++]
           Return arr
```

Java Implementation

```
class Solution {
           public void rearrange(int[] arr) {
               List<Integer> positives = new ArrayList<>();
               List<Integer> negatives = new ArrayList<>();

                 for (int num : arr) {
                     if (num >= 0) {
                         positives.add(num);
                     } else {
                         negatives.add(num);
                     }
                 }

                 int i = 0, j = 0, k = 0;

              while (i < positives.size() && j < negatives.size()) {
                  arr[k++] = positives.get(i++);
                  arr[k++] = negatives.get(j++);
              }

              while (i < positives.size()) {
                  arr[k++] = positives.get(i++);
```

```
            }

            while (j < negatives.size()) {
                arr[k++] = negatives.get(j++);
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

29. Next Permutation

Understanding the Problem NEW

Given a permutation (arrangement) of numbers, find the *next* lexicographically greater permutation — the very next arrangement you'd reach if you listed all permutations in sorted order. E.g. `[1,2,3]` → `[1,3,2]`.

Brute Force Approach NEW

The naive approach: generate *all* permutations of the array, sort them, and find the one right after the current arrangement. Generating all permutations costs O(n!) — completely impractical for anything but tiny arrays.

Why We Optimize NEW

You don't need every permutation — you only need to make the **smallest possible increase**. The trick: scan from the right to find the first place where the sequence stops increasing (the 'pivot'); swap that pivot with the smallest element to its right that's still bigger than it; then reverse everything after the pivot's position (since that suffix was in decreasing order, reversing it makes it the smallest possible arrangement). This turns an O(n!) brute enumeration into a clever O(n) sequence of scans, swaps, and a reversal.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the next lexicographically greater permutation.

Visual Diagram

```
Array: [1, 2, 3]

       Step 1: Find first decreasing element from right
       [1, 2, 3]
           ↑
         2 < 3

       Step 2: Find element just greater than 2 from right
       [1, 2, 3]
           ↑ ↑
          2 3
```

```
Step 3: Swap
[1, 3, 2]

Step 4: Reverse suffix after position of 2
[1, 3, 2]

Result: [1, 3, 2]

Example 2:
Array: [3, 2, 1] → Next: [1, 2, 3]

Example 3:
Array: [1, 1, 5] → Next: [1, 5, 1]
```

Solution Approach

Concept: Find pivot, swap with smallest larger element, reverse suffix.

Algorithm

```
Algorithm NEXT_PERMUTATION(arr):
           i = n-2
           While i >= 0 and arr[i] >= arr[i+1]:
               i--

           If i >= 0:
               j = n-1
               While arr[j] <= arr[i]:
                   j--
               Swap arr[i] and arr[j]

           Reverse arr[i+1..n-1]
           Return arr
```

Java Implementation

```
class Solution {
           public void nextPermutation(int[] nums) {
               int i = nums.length - 2;

                 while (i >= 0 && nums[i] >= nums[i+1]) {
                     i--;
                 }

              if (i >= 0) {
                  int j = nums.length - 1;
                  while (nums[j] <= nums[i]) {
                      j--;
                  }
```

```
                swap(nums, i, j);
            }

            reverse(nums, i+1, nums.length - 1);
        }

        private void swap(int[] nums, int i, int j) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
        }

        private void reverse(int[] nums, int start, int end) {
            while (start < end) {
                swap(nums, start, end);
                start++;
                end--;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

30. Leaders in an Array

Understanding the Problem NEW

A 'leader' is an element that is greater than every element to its right. Find all the leaders in the array. E.g. in `[16,17,4,3,5,2]`, the leaders are 17, 5, and 2 (each is bigger than everything after it).

Brute Force Approach NEW

The direct approach: for every element, scan all elements to its right and check if it's bigger than all of them — $O(n^2)$ comparisons.

Why We Optimize NEW

Instead of looking forward from each element (which forces rechecking a lot of the same territory), scan **from the right side** and keep a running 'maximum seen so far.' Any element that beats this running maximum is a leader (since everything to its right, by definition, has already been accounted for in that maximum). This collapses the problem into a single right-to-left O(n) pass.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find elements that are greater than all elements to their right.

Visual Diagram

```
Array: [16, 17, 4, 3, 5, 2]

       Traverse from right:
       Step 1: 2 is leader (rightmost)
       [16, 17, 4, 3, 5, 2]
                         ↑
                      leader=2

       Step 2: 5 > 2 → 5 is leader
       [16, 17, 4, 3, 5, 2]
                      ↑
                   leader=5

       Step 3: 3 < 5 → Not leader
```

```
       [16, 17, 4, 3, 5, 2]
                    ↑

       Step 4: 4 < 5 → Not leader
       [16, 17, 4, 3, 5, 2]
                 ↑

       Step 5: 17 > 5 → 17 is leader
       [16, 17, 4, 3, 5, 2]
            ↑
          leader=17

       Step 6: 16 < 17 → Not leader
       [16, 17, 4, 3, 5, 2]
        ↑

       Result: [17, 5, 2]
```

Solution Approach

Concept: Traverse from right, track maximum.

Algorithm

```
Algorithm FIND_LEADERS(arr):
           leaders = []
           max_from_right = -INF

            For i = n-1 down to 0:
                If arr[i] > max_from_right:
                    leaders.add(arr[i])
                    max_from_right = arr[i]

            Reverse leaders
            Return leaders
```

Java Implementation

```
class Solution {
           public List<Integer> findLeaders(int[] arr) {
               List<Integer> leaders = new ArrayList<>();

              int maxFromRight = Integer.MIN_VALUE;

              for (int i = arr.length - 1; i >= 0; i--) {
                  if (arr[i] > maxFromRight) {
                      leaders.add(arr[i]);
                      maxFromRight = arr[i];
                  }
              }
```

```
            Collections.reverse(leaders);
            return leaders;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

31. Longest Consecutive Sequence

Understanding the Problem NEW

Find the length of the longest run of consecutive integers present in the array, regardless of their order or position. E.g. `[100,4,200,1,3,2]` contains the consecutive run 1,2,3,4 — length 4.

Brute Force Approach NEW

A first attempt might sort the array, then scan for the longest consecutive run — that works but costs O(n log n) just for the sort, when the problem can be solved without ordering the whole array.

Why We Optimize NEW

Put every number into a **hash set** for O(1) lookups. Then, for each number, only *start* counting a sequence if that number's predecessor (`num - 1`) is *not* in the set — this guarantees you only ever count each consecutive run starting from its true beginning, instead of re-counting the same run from every one of its members. Each number therefore gets visited a small constant number of times overall, giving true O(n) time — better than sorting, using O(n) extra space for the set.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the length of longest consecutive elements sequence.

Visual Diagram

```
Array: [100, 4, 200, 1, 3, 2]

       HashSet: {100, 4, 200, 1, 3, 2}

       Check each number:
       100: prev=99 not found → sequence starts here
         length=1
         101 not found → length=1

       4: prev=3 found → skip (start of sequence is 1)
```

```
       200: prev=199 not found → length=1

       1: prev=0 not found → sequence starts here
         len=1, check 2→len=2, check 3→len=3, check 4→len=4, check 5 not found
         max=4

       3: prev=2 found → skip

       2: prev=1 found → skip

       Result: 4 (sequence [1, 2, 3, 4])
```

Solution Approach

Concept: Use HashSet and only start sequence from smallest element.

Algorithm

```
Algorithm LONGEST_CONSECUTIVE(arr):
           set = createSet(arr)
           max_len = 0
           For each num in set:
               If set doesn't contain num-1:
                   current_num = num
                   current_len = 1
                   While set contains current_num+1:
                       current_num++
                       current_len++
                   max_len = max(max_len, current_len)
           Return max_len
```

Java Implementation

```
class Solution {
           public int longestConsecutive(int[] nums) {
               Set<Integer> set = new HashSet<>();
               for (int num : nums) {
                   set.add(num);
               }

                 int maxLen = 0;
                 for (int num : set) {
                     if (!set.contains(num - 1)) {
                         int currentNum = num;

                      int currentLen = 1;

                      while (set.contains(currentNum + 1)) {
                          currentNum++;
                          currentLen++;
```

```
                    }

                    maxLen = Math.max(maxLen, currentLen);
                }
            }
            return maxLen;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

32. Set Matrix Zeros

Understanding the Problem NEW

Given a matrix, if any cell contains a 0, then its entire row and entire column must be set to 0. You need to do this for the whole matrix, ideally without needing a full separate copy of it.

Brute Force Approach NEW

A naive approach: scan the whole matrix, and the moment you see a 0, immediately zero out its row and column right away. The catch is this corrupts the matrix for later checks, causing incorrect over-zeroing. A safer brute fix uses an extra full-size boolean matrix to record which rows/columns need zeroing — this works but costs O(m×n) extra space.

Why We Optimize NEW

Instead of a separate marker matrix, you can reuse the matrix's own **first row and first column** as the marker space (with two extra flags for whether row 0 / column 0 themselves need zeroing, handled separately since they double as markers). You first record markers into the first row/column, then use those markers to zero out the rest of the matrix. This achieves the same O(m×n) time using only O(1) extra space, by cleverly repurposing existing memory instead of allocating new memory.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Set entire row and column to zero if any element is zero.

Visual Diagram

```
Matrix:
       [1, 1, 1]   [1, 0, 1]
       [1, 0, 1] → [0, 0, 0]
       [1, 1, 1]   [1, 0, 1]

       Step 1: Find zero at (1,1)
       [1, 1, 1]
       [1, 0, 1]
```

```
       [1, 1, 1]

       Step 2: Mark row 1 and column 1
       [1, 0, 1]
       [0, 0, 0]
       [1, 0, 1]
```

Solution Approach

Concept: Use first row and column as markers.

Algorithm

```
Algorithm SET_MATRIX_ZERO(matrix):
           m = rows, n = cols
           first_row_zero = false, first_col_zero = false

           Check first row for zero
           Check first column for zero

           For i = 1 to m-1:
               For j = 1 to n-1:
                   If matrix[i][j] == 0:
                       matrix[i][0] = 0
                       matrix[0][j] = 0

           For i = 1 to m-1:
               If matrix[i][0] == 0:
                   Set row i to zero

           For j = 1 to n-1:
               If matrix[0][j] == 0:
                   Set column j to zero

           If first_row_zero: Set first row to zero
           If first_col_zero: Set first column to zero
```

Java Implementation

```
class Solution {
           public void setZeroes(int[][] matrix) {
               int m = matrix.length, n = matrix[0].length;
               boolean firstRowZero = false, firstColZero = false;

                 // Check first row
                 for (int j = 0; j < n; j++) {
                     if (matrix[0][j] == 0) firstRowZero = true;

              }
```

```
            // Check first column
            for (int i = 0; i < m; i++) {
                if (matrix[i][0] == 0) firstColZero = true;
            }

            // Mark zeros
            for (int i = 1; i < m; i++) {
                for (int j = 1; j < n; j++) {
                    if (matrix[i][j] == 0) {
                        matrix[i][0] = 0;
                        matrix[0][j] = 0;
                    }
                }
            }

            // Set rows to zero
            for (int i = 1; i < m; i++) {
                if (matrix[i][0] == 0) {
                    for (int j = 1; j < n; j++) {
                        matrix[i][j] = 0;
                    }
                }
            }

            // Set columns to zero
            for (int j = 1; j < n; j++) {
                if (matrix[0][j] == 0) {
                    for (int i = 1; i < m; i++) {
                        matrix[i][j] = 0;
                    }
                }
            }

            if (firstRowZero) {
                for (int j = 0; j < n; j++) matrix[0][j] = 0;
            }
            if (firstColZero) {
                for (int i = 0; i < m; i++) matrix[i][0] = 0;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(1)

33. Rotate Matrix by 90 Degrees

Understanding the Problem NEW

Rotate a square matrix 90 degrees clockwise, in-place, without using a second matrix to build the rotated result.

Brute Force Approach NEW

The straightforward approach: create a brand-new matrix, and copy each element from its old position directly to its rotated position using an index formula. This is easy to get right, but costs $O(n^2)$ extra space for the new matrix.

Why We Optimize NEW

You can achieve the same rotation in-place using two simpler operations you may already know: first **transpose** the matrix (swap rows and columns, i.e. flip across the diagonal), then **reverse each row**. Doing these two well-understood $O(n^2)$ operations back-to-back produces exactly a 90-degree clockwise rotation, but using only O(1) extra space, because both transposing and row-reversal can be done by swapping elements within the same matrix.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Rotate a matrix by 90 degrees clockwise.

Visual Diagram

```
Original:           Transpose:           Rotate:
       [1, 2, 3]           [1, 4, 7]           [7, 4, 1]
       [4, 5, 6]      →     [2, 5, 8]      →    [8, 5, 2]
       [7, 8, 9]           [3, 6, 9]           [9, 6, 3]

       Step 1: Transpose
       [1, 4, 7]
       [2, 5, 8]
       [3, 6, 9]

       Step 2: Reverse each row
       [7, 4, 1]
```

```
       [8, 5, 2]
       [9, 6, 3]
```

Solution Approach

Concept: Transpose matrix, then reverse each row.

Algorithm

```
Algorithm ROTATE_90(matrix):
           n = matrix.length

           // Transpose
           For i = 0 to n-1:
               For j = i+1 to n-1:
                   Swap matrix[i][j] and matrix[j][i]

           // Reverse each row
           For i = 0 to n-1:
               For j = 0 to n/2-1:
                   Swap matrix[i][j] and matrix[i][n-1-j]
```

Java Implementation

```
class Solution {
           public void rotate(int[][] matrix) {
               int n = matrix.length;

                 // Transpose
                 for (int i = 0; i < n; i++) {
                     for (int j = i + 1; j < n; j++) {
                         int temp = matrix[i][j];
                         matrix[i][j] = matrix[j][i];
                         matrix[j][i] = temp;
                     }
                 }

                 // Reverse each row
                 for (int i = 0; i < n; i++) {
                     for (int j = 0; j < n/2; j++) {
                         int temp = matrix[i][j];
                         matrix[i][j] = matrix[i][n-1-j];
                         matrix[i][n-1-j] = temp;

                   }
               }
           }
      }
```

Time & Space Complexity

Time Complexity: $O(n^2)$ | Space Complexity: O(1)

34. Print Matrix in Spiral Manner

Understanding the Problem NEW

Print all elements of a matrix in spiral order — starting from the top-left, going right along the top row, then down the right column, then left along the bottom row, then up the left column, and repeating inward.

Brute Force Approach NEW

Without a clear plan, a beginner might try to track direction changes with ad-hoc conditionals scattered everywhere, which quickly gets tangled and error-prone (easy to double-print or skip a cell) — this isn't really 'brute force vs optimal' so much as 'unstructured vs structured.'

Why We Optimize NEW

The clean approach: maintain four boundaries — `top`, `bottom`, `left`, `right` — representing the current unprinted rectangle. Print along the top row, then shrink `top` down by one; print down the right column, then shrink `right` in by one; and so on for bottom and left, repeating until the boundaries cross. This 'shrinking rectangle' structure guarantees every cell is visited exactly once in the correct order, in a clean O(m×n) pass with O(1) extra space (aside from the output).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print matrix elements in spiral order.

Visual Diagram

```
Matrix:
       [1, 2, 3, 4]
       [5, 6, 7, 8]
       [9, 10, 11, 12]
       [13,14, 15, 16]

       Spiral Order:
       1 → 2 → 3 → 4 ↓
                       8 ↓
                      12 ↓
       16 ← 15 ← 14 ← 13 ↑
```

```
        ↓
        9 → 10 → 11

        Result: [1, 2, 3, 4, 8, 12, 16, 15, 14, 13, 9, 5, 6, 7, 11, 10]
```

Solution Approach

Concept: Traverse in layers (top, right, bottom, left).

Algorithm

```
Algorithm SPIRAL_MATRIX(matrix):
           result = []
           top = 0, bottom = m-1
           left = 0, right = n-1

            While top <= bottom and left <= right:
                // Top row
                For j = left to right:
                    Add matrix[top][j]
                top++

                 // Right column
                 For i = top to bottom:
                     Add matrix[i][right]
                 right--

                 // Bottom row
                 If top <= bottom:
                     For j = right down to left:
                         Add matrix[bottom][j]
                     bottom--

                 // Left column
                 If left <= right:
                     For i = bottom down to top:
                         Add matrix[i][left]
                     left++

             Return result
```

Java Implementation

```
class Solution {

          public List<Integer> spiralOrder(int[][] matrix) {
              List<Integer> result = new ArrayList<>();
              int top = 0, bottom = matrix.length - 1;
              int left = 0, right = matrix[0].length - 1;
```

```
            while (top <= bottom && left <= right) {
                // Top row
                for (int j = left; j <= right; j++) {
                    result.add(matrix[top][j]);
                }
                top++;

                // Right column
                for (int i = top; i <= bottom; i++) {
                    result.add(matrix[i][right]);
                }
                right--;

                // Bottom row
                if (top <= bottom) {
                    for (int j = right; j >= left; j--) {
                        result.add(matrix[bottom][j]);
                    }
                    bottom--;
                }

                // Left column
                if (left <= right) {
                    for (int i = bottom; i >= top; i--) {
                        result.add(matrix[i][left]);
                    }
                    left++;
                }
            }

            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(1)

35. Count Subarrays with Given Sum

Understanding the Problem NEW

Count how many subarrays (contiguous runs) have a sum equal to exactly K. This version assumes the array elements are structured in a way that lets you use prefix sums directly (works for both positive and general integers).

Brute Force Approach NEW

The direct approach checks every possible subarray's sum and counts the matches — O(n²) pairs of start/end indices.

Why We Optimize NEW

As in problem 21, track the running **prefix sum** as you scan left to right, and use a hashmap to remember how many times each prefix sum value has occurred so far. For each new prefix sum `S`, the number of previous positions where the prefix sum was `S - K` tells you exactly how many subarrays ending here sum to K — because the difference between any two prefix sums *is* the sum of the subarray between them. This turns O(n²) pair-checking into a single O(n) pass with a hashmap.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count number of subarrays with sum equal to K.

Visual Diagram

```
Array: [1, 1, 1], K=2

       Brute Force:
       [1] sum=1 ✗
       [1,1] sum=2 ✓ (1)
       [1,1,1] sum=3 ✗
       [1] sum=1 ✗
       [1,1] sum=2 ✓ (2)
       [1] sum=1 ✗

       Total = 2
```

```
HashMap Method:
prefix=0, map={0:1}

i=0: prefix=1, need=-1 (not found), store 1
i=1: prefix=2, need=0 (found: 1), count=1, store 2
i=2: prefix=3, need=1 (found: 1), count=2, store 3

Result: 2
```

Solution Approach

Concept: Prefix sum with hashmap.

Algorithm

```
Algorithm COUNT_SUBARRAYS(arr, K):
           map = {0: 1}
           prefix_sum = 0
           count = 0

           For each num in arr:
               prefix_sum += num
               If map contains (prefix_sum - K):
                   count += map[prefix_sum - K]
               map[prefix_sum] = map.getOrDefault(prefix_sum, 0) + 1

           Return count
```

Java Implementation

```
class Solution {
           public int countSubarraysWithSumK(int[] nums, int K) {
               Map<Integer, Integer> map = new HashMap<>();
               map.put(0, 1);

                 int prefixSum = 0;
                 int count = 0;

                 for (int num : nums) {
                     prefixSum += num;
                     count += map.getOrDefault(prefixSum - K, 0);
                     map.put(prefixSum, map.getOrDefault(prefixSum, 0) + 1);
                 }

              return count;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

36. Pascal's Triangle

Understanding the Problem NEW

Generate the first n rows of Pascal's Triangle, where each number is the sum of the two numbers directly above it (with edges always being 1). Row 3 (0-indexed) is `[1,3,3,1]`.

Brute Force Approach NEW

A beginner might try to compute each entry directly using the binomial coefficient formula `C(row, col) = row! / (col! × (row-col)!)` from scratch for every cell — this involves repeated, redundant factorial computations and can be slow or prone to overflow for larger rows.

Why We Optimize NEW

Because each entry is defined in terms of the row above it, you can build the triangle **incrementally**: start each row with a 1, and compute every other entry as the sum of the two values above it from the previous row (already computed). This reuses previously computed results instead of recalculating factorials from scratch every time — a simple but classic 'build on what you already know' dynamic-programming pattern, costing O(n²) total for the full triangle, which is optimal since there are O(n²) numbers to produce.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Generate Pascal's triangle with n rows.

Visual Diagram

```
n=5

       Row 0:                    1
       Row 1:                 1     1
       Row 2:              1     2     1
       Row 3:           1     3     3     1
       Row 4:        1     4     6     4     1

       Construction:
       Row 0: [1]
```

```
       Row 1: [1, 1]
       Row 2: [1, 1+1=2, 1] = [1, 2, 1]
       Row 3: [1, 1+2=3, 2+1=3, 1] = [1, 3, 3, 1]
       Row 4: [1, 1+3=4, 3+3=6, 3+1=4, 1] = [1, 4, 6, 4, 1]
```

Solution Approach

Concept: Each element is sum of two elements above.

Algorithm

```
Algorithm PASCAL_TRIANGLE(numRows):
           result = []
           For row = 0 to numRows-1:
               current = []
               For col = 0 to row:
                   If col == 0 or col == row:
                       current.add(1)
                   Else:
                       prev = result[row-1]
                       current.add(prev[col-1] + prev[col])
               result.add(current)
           Return result
```

Java Implementation

```
class Solution {
           public List<List<Integer>> generate(int numRows) {
               List<List<Integer>> result = new ArrayList<>();

                 for (int row = 0; row < numRows; row++) {
                     List<Integer> current = new ArrayList<>();

                       for (int col = 0; col <= row; col++) {
                           if (col == 0 || col == row) {
                               current.add(1);
                           } else {
                               List<Integer> prev = result.get(row - 1);
                               current.add(prev.get(col - 1) + prev.get(col));
                           }
                       }
                       result.add(current);

               }

               return result;
           }
       }
```

Time & Space Complexity

Time Complexity: $O(n^2)$ | Space Complexity: $O(n^2)$

37. Majority Element (n/3 times)

Understanding the Problem NEW

Find all elements that appear **more than n/3 times** in the array. Note there can be at most 2 such elements (since three elements each appearing more than n/3 times would need more than n total elements).

Brute Force Approach NEW

As with the n/2 majority problem, a beginner approach counts occurrences of every distinct value with a hashmap and filters by the n/3 threshold — correct, O(n) time, but O(n) space.

Why We Optimize NEW

The same cancellation idea from Moore's Voting Algorithm can be **extended** to track two candidates and two counters instead of one, since at most two elements can qualify. Elements are paired off against combinations of the two current candidates, with counters incremented/decremented similarly. After one pass, do a second verification pass to confirm the surviving candidates truly occur more than n/3 times (since the voting process can produce false positives without this check). This still runs in O(n) time but drops the extra space down to O(1), using the mathematical guarantee about how many candidates can possibly qualify.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find elements appearing more than n/3 times.

Visual Diagram

```
Array: [3, 2, 3, 1, 3, 4, 2]
       n=7, n/3=2.33

       Boyer-Moore Extended:
       Step 1: candidate1=3, count1=1
       Step 2: 2 != 3 → candidate2=2, count2=1
       Step 3: 3 == candidate1 → count1=2
       Step 4: 1 != both → count1=1, count2=1
```

```
Step 5: 3 == candidate1 → count1=2
Step 6: 4 != both → count1=1, count2=1
Step 7: 2 == candidate2 → count2=2

Candidates: 3 and 2
Verify: 3 appears 3 times, 2 appears 2 times
3 > 2.33 ✓, 2 < 2.33 ✗

Result: [3]
```

Solution Approach

Concept: Extended Boyer-Moore voting for n/3.

Algorithm

```
Algorithm MAJORITY_ELEMENT_N3(arr):
           candidate1 = -1, candidate2 = -1
           count1 = 0, count2 = 0

           For each num in arr:
               If num == candidate1: count1++
               Else If num == candidate2: count2++
               Else If count1 == 0: candidate1 = num; count1 = 1
               Else If count2 == 0: candidate2 = num; count2 = 1
               Else: count1--; count2--

           Verify candidates and return
```

Java Implementation

```
class Solution {
           public List<Integer> majorityElement(int[] nums) {
               int candidate1 = -1, candidate2 = -1;
               int count1 = 0, count2 = 0;

                 for (int num : nums) {
                     if (num == candidate1) {
                         count1++;
                     } else if (num == candidate2) {
                         count2++;
                     } else if (count1 == 0) {
                         candidate1 = num;
                         count1 = 1;
                     } else if (count2 == 0) {
                         candidate2 = num;
                         count2 = 1;

                  } else {
                      count1--;
```

```
                    count2--;
                }
            }

            List<Integer> result = new ArrayList<>();
            count1 = 0; count2 = 0;
            for (int num : nums) {
                if (num == candidate1) count1++;
                else if (num == candidate2) count2++;
            }

            if (count1 > nums.length / 3) result.add(candidate1);
            if (count2 > nums.length / 3) result.add(candidate2);

            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

38. 3-Sum Problem

Understanding the Problem NEW

Find all unique triplets of numbers in the array that add up to zero. E.g. in `[-1,0,1,2,-1,-4]`, valid triplets include `[-1,-1,2]` and `[-1,0,1]`.

Brute Force Approach NEW

The most direct approach: three nested loops trying every possible triplet and checking if it sums to zero — O(n³) time, plus extra care needed to avoid counting duplicate triplets.

Why We Optimize NEW

First, **sort** the array (O(n log n)) — this unlocks the two-pointer technique. Then fix one element with a single loop, and for the remaining two, use two pointers moving from both ends of the rest of the array inward: if the sum is too small, move the left pointer right; if too big, move the right pointer left; if it matches, record the triplet and skip over duplicates. This reduces the innermost search from O(n²) (nested loop) to O(n) (two-pointer), giving O(n²) total instead of O(n³) — and sorting also makes duplicate-skipping trivial.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all triplets that sum to zero.

Visual Diagram

```
Array: [-1, 0, 1, 2, -1, -4]
        Sorted: [-4, -1, -1, 0, 1, 2]

        i=0, target=-(-4)=4, left=1, right=5
        -1 + 2 = 1 < 4 → left++
        -1 + 2 = 1 < 4 → left++
        0 + 2 = 2 < 4 → left++
        1 + 2 = 3 < 4 → left++

        i=1, target=-(-1)=1, left=2, right=5
        -1 + 2 = 1 ✓ → [-1, -1, 2]
        -1 + 1 = 0 < 1 → left++ (skip duplicates)
```

```
        i=2, target=-(-1)=1, left=3, right=5
        0 + 2 = 2 > 1 → right--
        0 + 1 = 1 ✓ → [-1, 0, 1]

        Result: [[-1, -1, 2], [-1, 0, 1]]
```

Solution Approach

Concept: Sort array, fix one element, use two-pointer.

Algorithm

```
Algorithm THREE_SUM(arr):
            Sort arr
            result = []
            For i = 0 to n-3:
                If i > 0 and arr[i] == arr[i-1]: Continue
                left = i+1, right = n-1
                While left < right:
                     sum = arr[i] + arr[left] + arr[right]
                     If sum == 0:
                         result.add([arr[i], arr[left], arr[right]])
                         left++; right--
                         Skip duplicates
                     Else If sum < 0:
                         left++
                     Else:
                         right--
            Return result
```

Java Implementation

```
class Solution {
            public List<List<Integer>> threeSum(int[] nums) {
                Arrays.sort(nums);
                List<List<Integer>> result = new ArrayList<>();

                 for (int i = 0; i < nums.length - 2; i++) {
                     if (i > 0 && nums[i] == nums[i-1]) continue;

                     int left = i + 1, right = nums.length - 1;
                     while (left < right) {

                       int sum = nums[i] + nums[left] + nums[right];

                       if (sum == 0) {
                           result.add(Arrays.asList(nums[i], nums[left],
nums[right]));
                           left++;
```

```
                            right--;

                            while (left < right && nums[left] == nums[left-1]) left+
+;
                            while (left < right && nums[right] == nums[right+1])
right--;
                        } else if (sum < 0) {
                            left++;
                        } else {
                            right--;
                        }
                    }
                }

                return result;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(1)

39. 4-Sum Problem

Understanding the Problem NEW

Same idea as 3-Sum, but now find all unique quadruplets (four numbers) that add up to a given target.

Brute Force Approach NEW

The brute approach extends naturally: four nested loops trying every combination — O(n^4) time.

Why We Optimize NEW

Apply the exact same trick as 3-Sum, one level deeper: sort the array, fix the *first two* elements with nested loops, then use the two-pointer technique on the remaining two — turning the innermost O(n^2) search into O(n). This brings total time down from O(n^4) to O(n^3), the same 'fix-and-shrink-with-two-pointers' pattern applied recursively to one more dimension.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all quadruplets that sum to target.

Visual Diagram

```
Array: [1, 0, -1, 0, -2, 2], Target: 0
       Sorted: [-2, -1, 0, 0, 1, 2]

       i=0, j=1: left=2, right=5
       Sum = -2-1+0+2 = -1 < 0 → left++

       Continue searching...
       Found: [-2, -1, 1, 2]
       Found: [-2, 0, 0, 2]
       Found: [-1, 0, 0, 1]

       Result: [[-2, -1, 1, 2], [-2, 0, 0, 2], [-1, 0, 0, 1]]
```

Solution Approach

Concept: Two nested loops + two-pointer technique.

Algorithm

```
Algorithm FOUR_SUM(arr, target):
           Sort arr
           result = []
           For i = 0 to n-4:
               If i > 0 and arr[i] == arr[i-1]: Continue
               For j = i+1 to n-3:
                    If j > i+1 and arr[j] == arr[j-1]: Continue
                    left = j+1, right = n-1
                    While left < right:
                        sum = arr[i] + arr[j] + arr[left] + arr[right]
                        If sum == target:
                            Add quadruplet
                            left++; right--
                            Skip duplicates
                        Else If sum < target:
                            left++
                        Else:
                            right--
           Return result
```

Java Implementation

```
class Solution {
           public List<List<Integer>> fourSum(int[] nums, int target) {
               Arrays.sort(nums);
               List<List<Integer>> result = new ArrayList<>();

                 for (int i = 0; i < nums.length - 3; i++) {
                     if (i > 0 && nums[i] == nums[i-1]) continue;

                    for (int j = i + 1; j < nums.length - 2; j++) {
                        if (j > i + 1 && nums[j] == nums[j-1]) continue;

                         int left = j + 1, right = nums.length - 1;

                         while (left < right) {

                           long sum = (long) nums[i] + nums[j] + nums[left] +
nums[right];

                           if (sum == target) {
                               result.add(Arrays.asList(nums[i], nums[j],
nums[left], nums[right]));
                               left++;
                               right--;
```

```
                                while (left < right && nums[left] == nums[left-1])
left++;
                                while (left < right && nums[right] == nums[right+1])
right--;
                            } else if (sum < target) {
                                left++;
                            } else {
                                right--;
                            }
                        }
                    }
                }

                return result;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n³) | Space Complexity: O(1)

40. Largest Subarray with 0 Sum

Understanding the Problem NEW

Find the length of the longest contiguous subarray whose elements sum to exactly zero (a special case of the 'sum equals K' family of problems, with K=0).

Brute Force Approach NEW

Checking every subarray's sum directly costs $O(n^2)$.

Why We Optimize NEW

Using prefix sums again: if the *same* prefix sum value occurs at two different indices, the subarray between them must sum to zero (since the running total didn't change between those points). Store the **first** occurrence index of each prefix sum in a hashmap; whenever a prefix sum repeats, compute the distance between the current index and that first occurrence to get a candidate length, keeping the maximum. This is a single O(n) pass with a hashmap, reusing exactly the same prefix-sum insight from problems 21, 35, and 40.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the length of largest subarray with sum 0.

Visual Diagram

```
Array: [15, -2, 2, -8, 1, 7, 10, 23]

       Prefix Sum:
       i=0: sum=15, store at index 0
       i=1: sum=13, store at index 1
       i=2: sum=15, found at index 0 → length=2-0=2
       i=3: sum=7, store at index 3
       i=4: sum=8, store at index 4
       i=5: sum=15, found at index 0 → length=5-0=5
       i=6: sum=25, store at index 6
       i=7: sum=48, store at index 7

       Max length = 5 (subarray: [-2, 2, -8, 1, 7])
```

Solution Approach

Concept: Prefix sum with hashmap.

Algorithm

```
Algorithm LARGEST_SUBARRAY_0(arr):
           map = {0: -1}
           sum = 0, max_len = 0
           For i = 0 to n-1:
               sum += arr[i]
               If map contains sum:
                   max_len = max(max_len, i - map[sum])
               Else:
                   map[sum] = i
           Return max_len
```

Java Implementation

```
class Solution {
           public int largestSubarrayWithZeroSum(int[] nums) {
               Map<Integer, Integer> map = new HashMap<>();
               map.put(0, -1);

                 int sum = 0, maxLen = 0;

                 for (int i = 0; i < nums.length; i++) {
                     sum += nums[i];

                     if (map.containsKey(sum)) {
                         maxLen = Math.max(maxLen, i - map.get(sum));
                     } else {
                         map.put(sum, i);
                     }
                 }

                 return maxLen;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

41. Count Number of Subarrays with Given Sum

Understanding the Problem NEW

This is functionally the same question as problem 35 — count subarrays summing to K — just emphasized here for arrays that may include negative numbers, where the earlier sliding-window idea (problem 20) wouldn't be valid.

Brute Force Approach NEW

As before, checking every subarray directly is $O(n^2)$.

Why We Optimize NEW

The same prefix-sum-plus-hashmap technique from problem 35 applies unchanged here, and it works correctly regardless of whether the array has negative numbers — because prefix sums and their differences are well-defined either way, unlike the sliding-window approach which specifically relies on all-positive values to guarantee the window only grows as you add elements.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count subarrays with sum equal to K (including negative numbers).

Visual Diagram

```
Array: [1, -1, 0, 1], K=0

       HashMap:
       prefix=0, map={0:1}

       i=0: sum=1, need=1, map.get(1)=0, count=0, store 1
       i=1: sum=0, need=0, map.get(0)=1, count=1, store 0
       i=2: sum=0, need=0, map.get(0)=2, count=3, store 0
       i=3: sum=1, need=1, map.get(1)=1, count=4, store 1

       Result: 4 subarrays
       Subarrays: [1,-1], [1,-1,0], [0], [0,1,-1,0]
```

Solution Approach

Concept: Prefix sum with hashmap (same as problem 35).

Algorithm

```
Algorithm COUNT_SUBARRAYS_K(arr, K):
           map = {0: 1}
           sum = 0, count = 0
           For each num in arr:
               sum += num
               count += map.getOrDefault(sum - K, 0)
               map.put(sum, map.getOrDefault(sum, 0) + 1)
           Return count
```

Java Implementation

```
class Solution {
           public int countSubarraysWithSumK(int[] nums, int K) {
               Map<Integer, Integer> map = new HashMap<>();
               map.put(0, 1);

                 int sum = 0, count = 0;

                 for (int num : nums) {
                     sum += num;
                     count += map.getOrDefault(sum - K, 0);
                     map.put(sum, map.getOrDefault(sum, 0) + 1);
                 }

                 return count;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

42. Merge Overlapping Subintervals

Understanding the Problem NEW

Given a list of intervals (like meeting times) that may overlap, merge all overlapping ones into a minimal set of non-overlapping intervals. E.g. `[[1,3],[2,6],[8,10]]` → `[[1,6],[8,10]]`.

Brute Force Approach NEW

A naive approach compares every pair of intervals to check for overlap and merges them repeatedly — this can require multiple passes and cost up to $O(n^2)$ or worse, since merging one pair can create new overlaps with other intervals.

Why We Optimize NEW

The trick that unlocks a clean solution: **sort the intervals by start time** first (O(n log n)). Once sorted, you only ever need to compare each interval with the *most recently merged* interval — if it overlaps, extend that merged interval's end; if not, start a new merged interval. This single left-to-right pass after sorting guarantees correctness because overlapping intervals must be adjacent once sorted by start time, avoiding the need to re-compare every pair.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Merge all overlapping intervals.

Visual Diagram

```
Intervals: [[1,3], [2,6], [8,10], [15,18]]

       Step 1: Sort by start time
       [[1,3], [2,6], [8,10], [15,18]]

       Step 2: Merge
       [1,3] → merge with [2,6] → [1,6]
       [1,6] → next [8,10] doesn't overlap → add [1,6]
       [8,10] → next [15,18] doesn't overlap → add [8,10]
       [15,18] → add [15,18]

       Result: [[1,6], [8,10], [15,18]]
```

```
Visual:
[1,3]   [2,6]     [8,10]    [15,18]
[----] [----]     [----]    [----]
[---------]                 [----]
```

Solution Approach

Concept: Sort intervals and merge overlapping ones.

Algorithm

```
Algorithm MERGE_INTERVALS(intervals):
           Sort intervals by start
           result = []
           current = intervals[0]

           For each interval in intervals[1..n-1]:
               If interval.start <= current.end:
                   current.end = max(current.end, interval.end)
               Else:
                   result.add(current)
                   current = interval
           result.add(current)
           Return result
```

Java Implementation

```
class Solution {
           public int[][] merge(int[][] intervals) {
               Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

                 List<int[]> result = new ArrayList<>();
                 int[] current = intervals[0];
                 result.add(current);

                 for (int[] interval : intervals) {
                     if (interval[0] <= current[1]) {
                         current[1] = Math.max(current[1], interval[1]);
                     } else {
                         current = interval;
                         result.add(current);
                     }

               }

               return result.toArray(new int[result.size()][]);
           }
       }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

43. Merge Two Sorted Arrays Without Extra Space

Understanding the Problem NEW

You have two already-sorted arrays and need to merge them into one fully sorted sequence — but with the constraint that you cannot use any extra array; everything must be rearranged within the two given arrays' existing memory.

Brute Force Approach NEW

The typical merge (like in Merge Sort) uses a temporary array to hold the merged result before copying it back — simple and O(n+m) time, but it violates the 'no extra space' constraint here.

Why We Optimize NEW

The **gap method** (inspired by Shell Sort) solves this in-place: start with a gap roughly half the combined length, and compare-and-swap elements that are 'gap' positions apart across both arrays (treating them conceptually as one combined sequence); shrink the gap each round (dividing by 2, rounding up) until the gap reaches 1. This is a clever reuse of an old sorting idea (Shell Sort's gap sequence) to solve a merging problem without extra memory, trading a bit of extra time complexity (O((n+m) log(n+m))) for the O(1) space guarantee.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Merge two sorted arrays without using extra space.

Visual Diagram

```
arr1: [1, 3, 5, 7]
       arr2: [2, 4, 6, 8]

       Step 1: Compare arr1[3]=7 with arr2[0]=2
       7 > 2 → swap → arr1=[1,3,5,2], arr2=[7,4,6,8]

       Step 2: Sort arr2 → arr2=[4,6,7,8]
       arr1=[1,3,5,2] → arr2=[4,6,7,8]

       Step 3: arr1[2]=5 with arr2[0]=4
       5 > 4 → swap → arr1=[1,3,4,2], arr2=[5,6,7,8]
```

```
Step 4: Sort arr2 → arr2=[5,6,7,8]
arr1=[1,3,4,2] → arr2=[5,6,7,8]

Step 5: arr1[1]=3 with arr2[0]=5 → 3 < 5 (continue)

After full merging:
arr1 = [1,2,3,4], arr2 = [5,6,7,8]
```

Solution Approach

Concept: Gap method (Shell sort).

Algorithm

```
Algorithm MERGE_WITHOUT_SPACE(arr1, arr2):
           n = len(arr1), m = len(arr2)
           gap = ceil((n+m)/2)

           While gap > 0:
               i = 0
               While i + gap < n+m:
                   If i < n and i+gap < n:
                       Compare arr1[i] and arr1[i+gap]
                   Else If i < n and i+gap >= n:
                       Compare arr1[i] and arr2[i+gap-n]
                   Else:
                       Compare arr2[i-n] and arr2[i+gap-n]
                   i++
               gap = gap <= 1 ? 0 : (gap+1)/2
```

Java Implementation

```
class Solution {
           public void merge(int[] arr1, int[] arr2) {
               int n = arr1.length, m = arr2.length;
               int gap = (n + m + 1) / 2;

                 while (gap > 0) {
                     int i = 0, j = gap;

                    while (j < n + m) {
                        if (j < n) {
                            if (arr1[i] > arr1[j]) {

                                swap(arr1, i, j);
                            }
                      } else if (i < n) {
                           if (arr1[i] > arr2[j - n]) {
                               int temp = arr1[i];
```

```
                            arr1[i] = arr2[j - n];
                            arr2[j - n] = temp;
                        }
                    } else {
                        if (arr2[i - n] > arr2[j - n]) {
                            swap(arr2, i - n, j - n);
                        }
                    }
                    i++;
                    j++;
                }

                gap = gap <= 1 ? 0 : (gap + 1) / 2;
            }
        }

        private void swap(int[] arr, int i, int j) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
```

Time & Space Complexity

Time Complexity: O((n+m) log(n+m)) | Space Complexity: O(1)

44. Find Repeating and Missing Number

Understanding the Problem NEW

An array of size n should contain every number from 1 to n exactly once, but due to a data error, one number is duplicated (appears twice) and another is missing entirely. Find both numbers.

Brute Force Approach NEW

A beginner approach: use a hashmap or frequency array to count occurrences of every number, then scan through 1..n to find which count is 2 (the repeating number) and which is 0 (the missing one) — correct, O(n) time, but O(n) extra space.

Why We Optimize NEW

You can solve this with pure arithmetic and O(1) space: let `S` and `S2` be the actual sum and sum-of-squares of the array, and `S_expected` / `S2_expected` be what they'd be for a perfect 1..n array. The differences `(S - S_expected)` and `(S2 - S2_expected)` give you two equations in two unknowns (the missing and repeating numbers), which you can solve directly. (An XOR-based method achieves the same result while avoiding overflow issues with large sums of squares.) Either way, you go from O(n) space down to O(1) by using number-theoretic relationships instead of explicit counting.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

In array of size n with numbers 1 to n, find the one repeating and one missing number.

Visual Diagram

```
Array: [3, 1, 2, 5, 3], n=5
       Expected: [1, 2, 3, 4, 5]
       Actual:   [3, 1, 2, 5, 3]

       Method 1: Sum and Square Sum
       Expected Sum = 15, Actual Sum = 14
       Expected Square Sum = 55, Actual Square Sum = 48
```

```
Let x = missing, y = repeating
y - x = 14-15 = -1 ... (1)
y² - x² = 48-55 = -7 ... (2)

From (1): y = x - 1
Substitute in (2): (x-1)² - x² = -7
x² - 2x + 1 - x² = -7
-2x + 1 = -7
-2x = -8
x = 4 (missing)
y = 3 (repeating)

Method 2: XOR
XOR all numbers: 3^1^2^5^3 = 4
XOR 1..5: 1^2^3^4^5 = 1
XOR result = 4^1 = 5

Result: Missing=4, Repeating=3
```

Solution Approach

Concept: Use XOR or sum methods.

Algorithm (XOR Method)

Algorithm REPEATING_MISSING(arr):

xor_all = 0

For i in arr: xor_all ^= i

For i = 1 to n: xor_all ^= i

// xor_all = missing ^ repeating

Find rightmost set bit of xor_all

xor1 = 0, xor2 = 0

For each num in arr:

If num has set bit: xor1 ^= num

Else: xor2 ^= num

For i = 1 to n:

If i has set bit: xor1 ^= i

Else: xor2 ^= i

Check which is missing and which is repeating

Java Implementation

```
class Solution {

          public int[] findErrorNums(int[] nums) {
              int n = nums.length;
              int xor = 0;

              // XOR all numbers in array and 1..n
              for (int i = 0; i < n; i++) {
                  xor ^= nums[i];
                  xor ^= (i + 1);
              }
              // xor = missing ^ repeating

              // Find rightmost set bit
              int setBit = xor & -xor;

              int xor1 = 0, xor2 = 0;

              // Partition numbers
              for (int num : nums) {
                  if ((num & setBit) != 0) {
                      xor1 ^= num;
                  } else {
                      xor2 ^= num;
                  }
              }

              for (int i = 1; i <= n; i++) {
                  if ((i & setBit) != 0) {
                      xor1 ^= i;
                  } else {
                      xor2 ^= i;
                  }
              }

              // Find which is repeating
              for (int num : nums) {
                  if (num == xor1) {
                      return new int[]{xor1, xor2};
                  }
              }
              return new int[]{xor2, xor1};
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

45. Count Inversions

Understanding the Problem NEW

Count the number of 'inversions' in the array — pairs of indices (i, j) where i comes before j but arr[i] > arr[j]. This is essentially a measure of 'how far the array is from being sorted.'

Brute Force Approach NEW

The direct approach checks every pair (i, j) and counts how many are inverted — $O(n^2)$ comparisons, which is fine for small arrays but too slow for large ones.

Why We Optimize NEW

This is a beautiful reuse of Merge Sort's structure: while merging two already-sorted halves, if an element from the right half gets placed *before* an element still remaining in the left half, that single placement actually represents *multiple* inversions at once — specifically, one inversion for every remaining element in the left half (since the whole left half comes before that position and is bigger than the current right-half element). By counting these during the merge step instead of comparing every pair separately, you piggyback inversion-counting onto Merge Sort's existing O(n log n) structure, avoiding the $O(n^2)$ brute pairwise check entirely.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count the number of inversions (i < j and arr[i] > arr[j]).

Visual Diagram

```
Array: [2, 4, 1, 3, 5]

        Brute Force:
        (2,1), (4,1), (4,3) = 3 inversions

        Merge Sort Method:
        [2, 4, 1, 3, 5]
        Divide: [2, 4] [1, 3, 5]
          [2] [4] [1, 3, 5]
            [1, 3] [5]
```

```
              [1] [3]

        Merge and count:
        [2,4] and [1,3,5]:
        2 > 1 → inversion +2 (2,1 and 4,1)
        2 < 3 → continue
        4 > 3 → inversion +1 (4,3)
        Total inversions = 3

        Result: 3
```

Solution Approach

Concept: Modified merge sort.

Algorithm

```
Algorithm COUNT_INVERSIONS(arr):
            Return mergeSort(arr, 0, n-1)

        Algorithm mergeSort(arr, low, high):
            If low >= high: Return 0
            mid = (low+high)/2
            inv = mergeSort(arr, low, mid)
            inv += mergeSort(arr, mid+1, high)
            inv += merge(arr, low, mid, high)
            Return inv

        Algorithm merge(arr, low, mid, high):
            temp = [], inv = 0
            i=low, j=mid+1
            While i <= mid and j <= high:
                If arr[i] <= arr[j]:
                    temp.add(arr[i++])
                Else:
                    temp.add(arr[j++])
                    inv += (mid - i + 1)
            Add remaining elements
            Copy temp back to arr
            Return inv
```

Java Implementation

```
class Solution {
            public int countInversions(int[] arr) {

                return mergeSort(arr, 0, arr.length - 1);
            }

            private int mergeSort(int[] arr, int low, int high) {
```

```
        if (low >= high) return 0;

        int mid = (low + high) / 2;
        int inv = mergeSort(arr, low, mid);
        inv += mergeSort(arr, mid + 1, high);
        inv += merge(arr, low, mid, high);

        return inv;
    }

    private int merge(int[] arr, int low, int mid, int high) {
        int[] temp = new int[high - low + 1];
        int i = low, j = mid + 1, k = 0;
        int inv = 0;

        while (i <= mid && j <= high) {
            if (arr[i] <= arr[j]) {
                temp[k++] = arr[i++];
            } else {
                temp[k++] = arr[j++];
                inv += (mid - i + 1);
            }
        }

        while (i <= mid) temp[k++] = arr[i++];
        while (j <= high) temp[k++] = arr[j++];

        System.arraycopy(temp, 0, arr, low, temp.length);
        return inv;
    }
}
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

46. Reverse Pairs

Understanding the Problem NEW

A variant of counting inversions: count pairs (i, j) with i < j where arr[i] > 2 × arr[j] — a stricter 'more than double' condition instead of simply 'bigger.'

Brute Force Approach NEW

Checking every pair directly costs $O(n^2)$, same as basic inversion counting.

Why We Optimize NEW

The same modified-merge-sort trick from Count Inversions applies here, with one twist: because the condition involves a factor of 2 rather than a simple >, you can't count matches with the same single merge pointer used for the normal merge step — you need a **separate pointer** that specifically walks ahead checking the '> 2×' condition, before doing the standard merge comparison. Counting these matches during the merge (rather than via nested loops) again keeps the total time at O(n log n) instead of $O(n^2)$, reusing the divide-and-conquer structure but adapting the counting condition to the specific inequality being asked about.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count pairs where i < j and arr[i] > 2*arr[j].

Visual Diagram

```
Array: [1, 3, 2, 3, 1]

        Brute Force:
        i=0: 1 > 2*1=2? No
        i=1: 3 > 2*2=4? No, 3 > 2*3=6? No, 3 > 2*1=2? Yes → 1 pair
        i=2: 2 > 2*3=6? No, 2 > 2*1=2? No
        i=3: 3 > 2*1=2? Yes → 1 pair
        Total: 2 pairs

        Modified Merge Sort:
        [1, 3, 2, 3, 1]
        Count pairs during merge: (3,1), (3,1) = 2 pairs
```

```
        Result: 2
```

Solution Approach

Concept: Modified merge sort with two-pointer counting.

Algorithm

```
Algorithm REVERSE_PAIRS(arr):
            Return mergeSort(arr, 0, n-1)

        Algorithm mergeSort(arr, low, high):
            If low >= high: Return 0
            mid = (low+high)/2
            count = mergeSort(arr, low, mid)
            count += mergeSort(arr, mid+1, high)
            count += merge(arr, low, mid, high)
            Return count

        Algorithm merge(arr, low, mid, high):
            count = 0
            j = mid+1

            // Count reverse pairs
            For i = low to mid:
                While j <= high and arr[i] > 2*arr[j]:
                    j++
                count += (j - (mid+1))

            // Normal merge
            Merge two halves
            Return count
```

Java Implementation

```
class Solution {
            public int reversePairs(int[] nums) {
                return mergeSort(nums, 0, nums.length - 1);
            }

            private int mergeSort(int[] nums, int low, int high) {
                if (low >= high) return 0;

               int mid = (low + high) / 2;
               int count = mergeSort(nums, low, mid);
               count += mergeSort(nums, mid + 1, high);
               count += merge(nums, low, mid, high);

               return count;
```

```
        }

        private int merge(int[] nums, int low, int mid, int high) {
            int count = 0;
            int j = mid + 1;

            for (int i = low; i <= mid; i++) {
                while (j <= high && (long) nums[i] > 2L * nums[j]) {
                    j++;
                }
                count += (j - (mid + 1));
            }

            // Normal merge
            int[] temp = new int[high - low + 1];
            int i = low;
            j = mid + 1;
            int k = 0;

            while (i <= mid && j <= high) {
                if (nums[i] <= nums[j]) {
                    temp[k++] = nums[i++];
                } else {
                    temp[k++] = nums[j++];
                }
            }

            while (i <= mid) temp[k++] = nums[i++];
            while (j <= high) temp[k++] = nums[j++];

            System.arraycopy(temp, 0, nums, low, temp.length);
            return count;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

47. Maximum Product Subarray

Understanding the Problem NEW

Find the contiguous subarray whose product (not sum) of elements is the largest. This is trickier than the max-sum version because multiplying by a negative number can flip a very small (very negative) product into the largest one, and multiplying by zero wipes everything out.

Brute Force Approach NEW

The naive approach checks every subarray's product directly — $O(n^2)$, and it's easy to make sign-handling mistakes even then.

Why We Optimize NEW

A single running-maximum (like in Kadane's Algorithm) isn't enough here, because a very negative running product can become the *best* possible product if it gets multiplied by another negative number. The fix: track **both** a running maximum *and* a running minimum product ending at each position. When you hit a negative number, swap max and min (since multiplying flips their roles), then update both using the current element. This dual-tracking adapts Kadane's single-value dynamic-programming idea to handle the sign-flipping behaviour of multiplication, still in a single O(n) pass.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the subarray with maximum product.

Visual Diagram

```
Array: [2, 3, -2, 4]

      All subarrays:
      [2] = 2
      [2,3] = 6
      [2,3,-2] = -12
      [2,3,-2,4] = -48
      [3] = 3
      [3,-2] = -6
      [3,-2,4] = -24
```

```
      [-2] = -2
      [-2,4] = -8
      [4] = 4

      Maximum = 6 (subarray [2,3])

      Array: [-2, 0, -1]
      Maximum = 0

      Array: [-2, 3, -4]
      All subarrays:
      [-2] = -2
      [-2,3] = -6
      [-2,3,-4] = 24
      [3] = 3
      [3,-4] = -12
      [-4] = -4
      Maximum = 24
```

Solution Approach

Concept: Track max and min products.

Algorithm

```
Algorithm MAX_PRODUCT_SUBARRAY(arr):
          max_product = arr[0]
          min_product = arr[0]
          result = arr[0]

          For i = 1 to n-1:
              If arr[i] < 0:
                  Swap max_product and min_product

                 max_product = max(arr[i], max_product * arr[i])
                 min_product = min(arr[i], min_product * arr[i])

                 result = max(result, max_product)

          Return result
```

Java Implementation

```
class Solution {
          public int maxProduct(int[] nums) {

              int maxProduct = nums[0];
              int minProduct = nums[0];
              int result = nums[0];
```

```
            for (int i = 1; i < nums.length; i++) {
                if (nums[i] < 0) {
                    int temp = maxProduct;
                    maxProduct = minProduct;
                    minProduct = temp;
                }

                maxProduct = Math.max(nums[i], maxProduct * nums[i]);
                minProduct = Math.min(nums[i], minProduct * nums[i]);

                result = Math.max(result, maxProduct);
            }

            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

Summary Table

Problem	Time	Space	Key Technique
Largest Element	O(n)	O(1)	Simple iteration
Second Largest	O(n)	O(1)	Two variables
Check Sorted	O(n)	O(1)	Compare adjacent
Remove Duplicates	O(n)	O(1)	Two-pointer
Left Rotate by One	O(n)	O(1)	Store and shift
Left Rotate by D	O(n)	O(1)	Reversal algorithm
Move Zeros	O(n)	O(1)	Two-pointer
Linear Search	O(n)	O(1)	Traversal
Union	O(n+m)	O(n+m)	Two-pointer
Missing Number	O(n)	O(1)	Sum/XOR
Max Consecutive Ones	O(n)	O(1)	Counting
Single Number	O(n)	O(1)	XOR
Longest Subarray K (Pos)	O(n)	O(1)	Sliding window
Longest Subarray K	O(n)	O(n)	Prefix sum
Two Sum	O(n)	O(n)	HashMap
Sort 0,1,2	O(n)	O(1)	DNF Algorithm
Majority Element	O(n)	O(1)	Moore's Voting
Kadane's	O(n)	O(1)	DP
Stock Buy Sell	O(n)	O(1)	Greedy
Rearrange Alt	O(n)	O(n)	Separation
Next Permutation	O(n)	O(1)	Finding pivot
Leaders	O(n)	O(n)	Right traversal
Longest Consecutive	O(n)	O(n)	HashSet
Set Matrix Zero	O(m*n)	O(1)	Marking
Rotate Matrix	$O(n^2)$	O(1)	Transpose + Reverse
Spiral Matrix	O(m*n)	O(1)	Layer traversal
Count Subarrays	O(n)	O(n)	Prefix sum
Pascal's Triangle	$O(n^2)$	$O(n^2)$	DP

Majority n/3	O(n)	O(1)	Extended Voting
3-Sum	$O(n^2)$	O(1)	Two-pointer
4-Sum	$O(n^3)$	O(1)	Two-pointer
Largest 0 Sum	O(n)	O(n)	Prefix sum
Merge Intervals	O(n log n)	O(n)	Sorting
Merge Arrays	O((n+m)log(n+m))	O(1)	Gap method
Repeating Missing	O(n)	O(1)	XOR
Count Inversions	O(n log n)	O(n)	Merge Sort
Reverse Pairs	O(n log n)	O(n)	Merge Sort
Max Product	O(n)	O(1)	DP

— End of enhanced Chapter 2: Arrays Problems — remaining chapters continue in the full edition —

Binary Search — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

48. Binary Search to Find X in Sorted Array

Understanding the Problem NEW

You're given a sorted array and a value X, and you need to check whether X exists in it. Because the array is sorted, you don't have to check every element one by one — think of how you'd look up a word in a paper dictionary: you don't start at page 1, you flip to roughly the middle and decide whether to go left or right from there.

Brute Force Approach NEW

The naive way to search any array — sorted or not — is **Linear Search**: check every element from left to right until you find X or reach the end. That's O(n) time, and critically, it never makes use of the fact the array is sorted.

Why We Optimize NEW

Because the array is sorted, comparing X to the **middle** element instantly tells you which half X could possibly be in — if X is bigger than the middle, it can only be in the right half; if smaller, only the left half. You can then throw away the other half entirely and repeat on the remaining half. Each comparison eliminates half of the remaining search space, so instead of checking n elements one at a time, you check only about $\log_2 n$ of them — turning O(n) into O(log n). This is the single most important idea in this whole chapter, and every later problem is a variation of it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find if element X exists in a sorted array using binary search.

Visual Diagram

```
Array: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91], Target: 23

        Step 1: low=0, high=9, mid=4
        [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
                          ↑
                        mid=16
        16 < 23 → Search right half
```

```
        Step 2: low=5, high=9, mid=7
        [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
                              ↑
                            mid=56
        56 > 23 → Search left half

        Step 3: low=5, high=6, mid=5
        [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
                              ↑
                            mid=23
        23 == 23 → Found at index 5

        Result: Index 5
```

Solution Approach

Concept: Divide and conquer - compare with middle element.

Algorithm

```
Algorithm BINARY_SEARCH(arr, target):
           low = 0, high = n-1
           While low <= high:
               mid = (low + high) / 2
               If arr[mid] == target:
                   Return mid
               Else If arr[mid] < target:
                   low = mid + 1
               Else:
                   high = mid - 1
           Return -1
```

Java Implementation

```
class Solution {
           public int binarySearch(int[] arr, int target) {
               int low = 0, high = arr.length - 1;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                     if (arr[mid] == target) {
                         return mid;
                     } else if (arr[mid] < target) {
                         low = mid + 1;
                     } else {
                         high = mid - 1;
                     }
                 }
```

```
                return -1;
            }
        }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

49. Implement Lower Bound

Understanding the Problem NEW

Find the **first index** where you could insert the target value while keeping the array sorted — equivalently, the first position holding a value that is greater than or equal to the target. E.g. in `[1,2,2,3]` the lower bound of 2 is index 1.

Brute Force Approach NEW

Scan the array left to right and stop at the first element ≥ target — O(n) time, ignoring the sorted structure that's available to exploit.

Why We Optimize NEW

This is standard Binary Search with a small twist: instead of stopping the moment you find a match, you keep searching the **left half** even after finding an element equal to the target, because there might be an earlier occurrence. You track the best (leftmost) qualifying index found so far and keep narrowing. Same O(log n) benefit as basic binary search, just with a modified 'keep looking left' rule.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the first position where target can be inserted to maintain sorted order (first >= target).

Visual Diagram

```
Array: [1, 2, 3, 3, 5, 7, 8], Target: 3

       Step 1: low=0, high=6, mid=3
       [1, 2, 3, 3, 5, 7, 8]
                ↑
              arr[3]=3
       3 >= 3 → ans=3, high=2

       Step 2: low=0, high=2, mid=1
       [1, 2, 3, 3, 5, 7, 8]
             ↑
           arr[1]=2
       2 < 3 → low=2
```

```
Step 3: low=2, high=2, mid=2
[1, 2, 3, 3, 5, 7, 8]
          ↑
        arr[2]=3
3 >= 3 → ans=2, high=1

Result: 2 (first position where 3 can be inserted)

Target: 4 → Lower Bound = 4 (position after 3s)
Target: 0 → Lower Bound = 0
Target: 9 → Lower Bound = 7 (length)
```

Solution Approach

Concept: Find first element >= target.

Algorithm

```
Algorithm LOWER_BOUND(arr, target):
           low = 0, high = n
           ans = n
           While low < high:
               mid = (low + high) / 2
               If arr[mid] >= target:
                   ans = mid
                   high = mid
               Else:
                   low = mid + 1
           Return ans
```

Java Implementation

```
class Solution {
           public int lowerBound(int[] arr, int target) {
               int low = 0, high = arr.length;
               int ans = arr.length;

                 while (low < high) {
                     int mid = low + (high - low) / 2;

                     if (arr[mid] >= target) {

                       ans = mid;
                       high = mid;
                   } else {
                       low = mid + 1;
                   }
               }
               return ans;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

50. Implement Upper Bound

Understanding the Problem NEW

Very similar to lower bound, but now find the first index whose value is **strictly greater than** the target (not equal to it). In `[1,2,2,3]`, the upper bound of 2 is index 3 (where the 3 sits).

Brute Force Approach NEW

Same naive idea as before — scan until you find the first element strictly greater than target, O(n).

Why We Optimize NEW

Again a binary search variant: when the middle element is less than *or equal to* the target, you now search right (since equal values don't qualify for 'strictly greater'); when it's greater, you search left but remember this position as a candidate. The only change from Lower Bound is the comparison operator, but that single tweak changes what boundary you land on — it's worth noticing how small a change in the binary search condition can redirect it to solve a related but distinct problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the first position where target can be inserted (first > target).

Visual Diagram

```
Array: [1, 2, 3, 3, 5, 7, 8], Target: 3

       Step 1: low=0, high=6, mid=3
       [1, 2, 3, 3, 5, 7, 8]
                ↑
              arr[3]=3
       3 > 3? No → low=4

       Step 2: low=4, high=6, mid=5
       [1, 2, 3, 3, 5, 7, 8]
                     ↑
                   arr[5]=7
```

```
        7 > 3 → ans=5, high=4

        Step 3: low=4, high=4, mid=4
        [1, 2, 3, 3, 5, 7, 8]
                    ↑
                  arr[4]=5
        5 > 3 → ans=4, high=3

        Result: 4 (first position > 3)

        Target: 6 → Upper Bound = 5 (position of 7)
        Target: 0 → Upper Bound = 0
        Target: 9 → Upper Bound = 7 (length)
```

Solution Approach

Concept: Find first element > target.

Algorithm

```
Algorithm UPPER_BOUND(arr, target):
            low = 0, high = n
            ans = n
            While low < high:
                mid = (low + high) / 2
                If arr[mid] > target:
                    ans = mid
                    high = mid
                Else:
                    low = mid + 1
            Return ans
```

Java Implementation

```
class Solution {
            public int upperBound(int[] arr, int target) {
                int low = 0, high = arr.length;
                int ans = arr.length;

                  while (low < high) {
                      int mid = low + (high - low) / 2;

                      if (arr[mid] > target) {

                        ans = mid;
                        high = mid;
                    } else {
                        low = mid + 1;
                    }
                }
```

```
                return ans;
            }
        }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

51. Search Insert Position

Understanding the Problem NEW

Given a sorted array and a target, find the index where the target is (if present) or the index where it would be inserted to keep the array sorted (if absent). This is essentially the Lower Bound problem in disguise.

Brute Force Approach NEW

Linear scan comparing each element until you pass the target's rightful position — O(n).

Why We Optimize NEW

This problem *is* Lower Bound: the first index where `arr[i] >= target` is exactly the answer, whether target is present or not. Recognizing that two differently-worded problems are secretly the same problem is a valuable skill — it means you already have an O(log n) solution ready to reuse instead of designing something new.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find position where target should be inserted in sorted array.

Visual Diagram

```
Array: [1, 3, 5, 6], Target: 5

       Search:
       Step 1: low=0, high=3, mid=1
       [1, 3, 5, 6]
           ↑
         arr[1]=3
       3 < 5 → low=2

       Step 2: low=2, high=3, mid=2
       [1, 3, 5, 6]
                ↑
              arr[2]=5
       5 == 5 → Return 2
```

```
Result: 2

Array: [1, 3, 5, 6], Target: 2
Step 1: mid=1, arr[1]=3 > 2 → high=1
Step 2: mid=0, arr[0]=1 < 2 → low=1
Result: 1 (insert between 1 and 3)

Array: [1, 3, 5, 6], Target: 7
Search through all → low=4
Result: 4 (insert at end)

Array: [1, 3, 5, 6], Target: 0
Search → high becomes -1
Result: 0 (insert at beginning)
```

Solution Approach

Concept: Binary search for insertion point.

Algorithm

```
Algorithm SEARCH_INSERT(arr, target):
           low = 0, high = n-1
           While low <= high:
               mid = (low + high) / 2
               If arr[mid] == target:
                   Return mid
               Else If arr[mid] < target:
                   low = mid + 1
               Else:
                   high = mid - 1
           Return low
```

Java Implementation

```
class Solution {
           public int searchInsert(int[] nums, int target) {
               int low = 0, high = nums.length - 1;

                 while (low <= high) {

                   int mid = low + (high - low) / 2;

                   if (nums[mid] == target) {
                       return mid;
                   } else if (nums[mid] < target) {
                       low = mid + 1;
                   } else {
                       high = mid - 1;
```

```
                }
            }
            return low;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

52. Floor/Ceil in Sorted Array

Understanding the Problem NEW

Floor is the largest value in the array that is ≤ target; Ceil is the smallest value that is ≥ target. E.g. in `[1,2,8,10,11]` with target 5: floor is 2, ceil is 8.

Brute Force Approach NEW

Scan the whole array once tracking the closest value below the target and the closest value above it — O(n), correct but ignores sortedness.

Why We Optimize NEW

Floor and Ceil are just **two separate binary searches** with a slightly adjusted condition each time — Ceil is close to the Lower Bound idea (first element ≥ target), and Floor is its mirror image searching from the other direction (last element ≤ target). Once you've internalized the Lower/Upper Bound pattern, Floor/Ceil become almost free — same log n technique, just pointed at slightly different boundaries.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find floor (largest <= target) and ceil (smallest >= target).

Visual Diagram

```
Array: [2, 3, 5, 9, 14, 16, 18], Target: 15

       Floor (largest ≤ 15):
       [2, 3, 5, 9, 14, 16, 18]
                   ↑
              floor = 14

       Ceil (smallest ≥ 15):
       [2, 3, 5, 9, 14, 16, 18]
                      ↑
                 ceil = 16

       Target: 10
       Floor: 9, Ceil: 14
```

```
Target: 1
Floor: -∞, Ceil: 2

Target: 20
Floor: 18, Ceil: ∞
```

Solution Approach

Concept: Binary search for both floor and ceil.

Algorithm

```
Algorithm FLOOR_CEIL(arr, target):
           floor = -∞, ceil = ∞
           low = 0, high = n-1

            While low <= high:
                mid = (low + high) / 2
                If arr[mid] == target:
                    Return {arr[mid], arr[mid]}
                Else If arr[mid] < target:
                    floor = arr[mid]
                    low = mid + 1
                Else:
                    ceil = arr[mid]
                    high = mid - 1

            Return {floor, ceil}
```

Java Implementation

```
class Solution {
           public int[] floorAndCeil(int[] arr, int target) {
               int floor = -1, ceil = -1;
               int low = 0, high = arr.length - 1;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                     if (arr[mid] == target) {

                       return new int[]{arr[mid], arr[mid]};
                   } else if (arr[mid] < target) {
                       floor = arr[mid];
                       low = mid + 1;
                   } else {
                       ceil = arr[mid];
                       high = mid - 1;
                   }
```

```
                }
                return new int[]{floor, ceil};
            }
        }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

53. Find First or Last Occurrence of a Number

Understanding the Problem NEW

Find both the first and the last position at which a target value occurs in a sorted array that may contain duplicates. E.g. in `[5,7,7,8,8,10]`, target 8 occurs at first index 3 and last index 4.

Brute Force Approach NEW

Scan the array once left to right to find the first occurrence, and once right to left (or continue the same scan) to find the last — O(n), fine, but the sorted order makes something faster possible.

Why We Optimize NEW

First occurrence is exactly Lower Bound of the target (the first index ≥ target, verified to actually equal target); last occurrence is Upper Bound of the target, minus one. Reusing the Lower/Upper Bound binary searches gets both answers in O(log n) total, instead of a full O(n) scan.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find first and last occurrence of target in sorted array.

Visual Diagram

```
Array: [2, 4, 4, 4, 6, 8, 10], Target: 4

       First Occurrence:
       Step 1: mid=3, arr[3]=4 → first=3, high=2
       Step 2: mid=1, arr[1]=4 → first=1, high=0
       Step 3: mid=0, arr[0]=2 < 4 → low=1
       Result: first=1

       Last Occurrence:
       Step 1: mid=3, arr[3]=4 → last=3, low=4
       Step 2: mid=5, arr[5]=8 > 4 → high=4
       Step 3: mid=4, arr[4]=6 > 4 → high=3
       Result: last=3
```

```
Range: [1, 3]
```

Solution Approach

Concept: Binary search with modified condition.

Algorithm

```
Algorithm FIRST_OCCURRENCE(arr, target):
          low = 0, high = n-1
          ans = -1
          While low <= high:
              mid = (low + high) / 2
              If arr[mid] == target:
                   ans = mid
                   high = mid - 1
              Else If arr[mid] < target:
                   low = mid + 1
              Else:
                   high = mid - 1
          Return ans

      Algorithm LAST_OCCURRENCE(arr, target):
          low = 0, high = n-1
          ans = -1
          While low <= high:
              mid = (low + high) / 2
              If arr[mid] == target:
                   ans = mid
                   low = mid + 1
              Else If arr[mid] < target:
                   low = mid + 1
              Else:
                   high = mid - 1
          Return ans
```

Java Implementation

```
class Solution {
          public int[] searchRange(int[] nums, int target) {
              int first = findFirst(nums, target);

              int last = findLast(nums, target);
              return new int[]{first, last};
          }

          private int findFirst(int[] nums, int target) {
              int low = 0, high = nums.length - 1;
              int ans = -1;
```

```
            while (low <= high) {
                int mid = low + (high - low) / 2;

                if (nums[mid] == target) {
                    ans = mid;
                    high = mid - 1;
                } else if (nums[mid] < target) {
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
            return ans;
        }

        private int findLast(int[] nums, int target) {
            int low = 0, high = nums.length - 1;
            int ans = -1;

            while (low <= high) {
                int mid = low + (high - low) / 2;

                if (nums[mid] == target) {
                    ans = mid;
                    low = mid + 1;
                } else if (nums[mid] < target) {
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
            return ans;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

54. Count Occurrences of a Number in a Sorted Array

Understanding the Problem NEW

Count how many times a target value appears in a sorted array. E.g. target 8 in `[5,7,7,8,8,8,10]` appears 3 times.

Brute Force Approach NEW

Count occurrences with a full linear scan, incrementing a counter whenever you see the target — O(n).

Why We Optimize NEW

Once you have the first and last occurrence indices (problem 53, both computed in O(log n) via binary search), the count is simply `lastIndex - firstIndex + 1` — pure arithmetic, no additional scanning needed. This is a nice example of how solving one problem (find first/last) can make a seemingly separate problem (counting) almost trivial.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count total occurrences of target in sorted array.

Visual Diagram

```
Array: [2, 4, 4, 4, 6, 8, 10], Target: 4

       First = 1, Last = 3
       Count = Last - First + 1 = 3 - 1 + 1 = 3

       Visual:
       [2, 4, 4, 4, 6, 8, 10]
           ↑ ↑ ↑
           First=1, Last=3
           Count = 3

       Target: 5 → First=-1, Last=-1 → Count=0
```

Solution Approach

Concept: Find first and last occurrence, calculate difference.

Algorithm

```
Algorithm COUNT_OCCURRENCES(arr, target):
           first = FIRST_OCCURRENCE(arr, target)
           If first == -1: Return 0
           last = LAST_OCCURRENCE(arr, target)
           Return last - first + 1
```

Java Implementation

```
class Solution {
           public int countOccurrences(int[] nums, int target) {
               int first = findFirst(nums, target);
               if (first == -1) return 0;
               int last = findLast(nums, target);
               return last - first + 1;
           }

            private int findFirst(int[] nums, int target) {
                int low = 0, high = nums.length - 1;
                int ans = -1;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;
                     if (nums[mid] == target) {
                         ans = mid;
                         high = mid - 1;
                     } else if (nums[mid] < target) {
                         low = mid + 1;
                     } else {
                         high = mid - 1;
                     }
                 }
                 return ans;
            }

            private int findLast(int[] nums, int target) {
                int low = 0, high = nums.length - 1;

               int ans = -1;

               while (low <= high) {
                   int mid = low + (high - low) / 2;
                   if (nums[mid] == target) {
                       ans = mid;
                       low = mid + 1;
                   } else if (nums[mid] < target) {
```

```
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
            return ans;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

55. Search in Rotated Sorted Array I

Understanding the Problem NEW

The array was originally sorted but has been 'rotated' at some pivot point (e.g. `[4,5,6,7,0,1,2]` is `[0,1,2,4,5,6,7]` rotated). Search for a target value in this rotated array, assuming all elements are unique.

Brute Force Approach NEW

Since the array isn't fully sorted anymore, a beginner might fall back to Linear Search across the whole thing — O(n), abandoning binary search entirely because 'it's not sorted.'

Why We Optimize NEW

The key realization: even after rotation, **at least one half** (left of mid, or right of mid) is always still perfectly sorted at every step. So at each step, first figure out which half is sorted by comparing the endpoints to the middle; then check whether the target could possibly lie within that sorted half's value range. If yes, recurse into that half; if no, the target must be in the other (still-rotated) half, so recurse there instead. This preserves the halving-every-step property of binary search, keeping it at O(log n) despite the array not being globally sorted.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Search target in a rotated sorted array (unique elements).

Visual Diagram

```
Array: [4, 5, 6, 7, 0, 1, 2], Target: 0

       Step 1: low=0, high=6, mid=3
       [4, 5, 6, 7, 0, 1, 2]
                 ↑
              arr[3]=7

       Check which half is sorted:
       Left half [4,5,6,7] is sorted
       Target 0 not in left half (0 < 4) → low=4
```

```
Step 2: low=4, high=6, mid=5
[4, 5, 6, 7, 0, 1, 2]
               ↑
             arr[5]=1

Right half [0,1,2] is sorted
Target 0 in right half → high=4

Step 3: low=4, high=4, mid=4
[4, 5, 6, 7, 0, 1, 2]
            ↑
          arr[4]=0
0 == 0 → Found at index 4

Result: 4
```

Solution Approach

Concept: Identify sorted half and search accordingly.

Algorithm

```
Algorithm SEARCH_ROTATED(arr, target):
           low = 0, high = n-1
           While low <= high:
               mid = (low + high) / 2
               If arr[mid] == target:
                   Return mid

               If arr[low] <= arr[mid]: // Left half sorted
                   If arr[low] <= target && target < arr[mid]:
                       high = mid - 1
                   Else:
                       low = mid + 1
               Else: // Right half sorted
                   If arr[mid] < target && target <= arr[high]:
                       low = mid + 1
                   Else:
                       high = mid - 1
           Return -1
```

Java Implementation

```
class Solution {

           public int search(int[] nums, int target) {
               int low = 0, high = nums.length - 1;

               while (low <= high) {
                   int mid = low + (high - low) / 2;
```

```
                if (nums[mid] == target) return mid;

                // Left half is sorted
                if (nums[low] <= nums[mid]) {
                    if (nums[low] <= target && target < nums[mid]) {
                        high = mid - 1;
                    } else {
                        low = mid + 1;
                    }
                }
                // Right half is sorted
                else {
                    if (nums[mid] < target && target <= nums[high]) {
                        low = mid + 1;
                    } else {
                        high = mid - 1;
                    }
                }
            }
            return -1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

56. Search in Rotated Sorted Array II

Understanding the Problem NEW

Same as above, but now the rotated array may contain **duplicate** values, which can make it ambiguous which half is sorted (e.g. `[3,1,2,3,3,3,3]`).

Brute Force Approach NEW

Falling back to full Linear Search, O(n), is the 'safe' brute option once duplicates seem to break the halving trick.

Why We Optimize NEW

The fix is small: when `arr[low] == arr[mid] == arr[high]`, you genuinely can't tell which half is sorted from these three values alone — so simply shrink both boundaries inward by one (`low++, high--`) and try again, without losing correctness. In the rare worst case (all duplicates), this degrades to O(n), but in the general case it stays close to O(log n) — a good example of a small patch that restores an algorithm's applicability at the cost of a rare worst-case penalty.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Search target in rotated sorted array with duplicates.

Visual Diagram

```
Array: [2, 5, 6, 0, 0, 1, 2], Target: 0

       Step 1: low=0, high=6, mid=3
       [2, 5, 6, 0, 0, 1, 2]
                ↑
              arr[3]=0
       Found at index 3

       Array: [1, 0, 1, 1, 1], Target: 0

       Step 1: low=0, high=4, mid=2
       [1, 0, 1, 1, 1]
             ↑
           arr[2]=1
```

```
nums[low] == nums[mid] == nums[high] → Cannot decide
Shrink: low++, high--

Step 2: low=1, high=3, mid=2
[1, 0, 1, 1, 1]
      ↑
    arr[2]=1

arr[low]=0, arr[mid]=1, arr[high]=1
Left half [0] is sorted, target=0 in left
high=1

Step 3: low=1, high=1, mid=1
[1, 0, 1, 1, 1]
    ↑
  arr[1]=0
Found at index 1
```

Solution Approach

Concept: Handle duplicates by shrinking boundaries.

Algorithm

```
Algorithm SEARCH_ROTATED_II(arr, target):
           low = 0, high = n-1
           While low <= high:
               mid = (low + high) / 2
               If arr[mid] == target:
                   Return true

                 // Handle duplicates
                 If arr[low] == arr[mid] && arr[mid] == arr[high]:
                     low++; high--
                     Continue

                 If arr[low] <= arr[mid]: // Left half sorted
                     If arr[low] <= target && target < arr[mid]:
                         high = mid - 1
                     Else:

                         low = mid + 1
               Else: // Right half sorted
                     If arr[mid] < target && target <= arr[high]:
                         low = mid + 1
                     Else:
                         high = mid - 1
           Return false
```

Java Implementation

```
class Solution {
           public boolean search(int[] nums, int target) {
               int low = 0, high = nums.length - 1;

               while (low <= high) {
                   int mid = low + (high - low) / 2;

                   if (nums[mid] == target) return true;

                   // Handle duplicates
                   if (nums[low] == nums[mid] && nums[mid] == nums[high]) {
                       low++;
                       high--;
                       continue;
                   }

                   // Left half is sorted
                   if (nums[low] <= nums[mid]) {
                       if (nums[low] <= target && target < nums[mid]) {
                           high = mid - 1;
                       } else {
                           low = mid + 1;
                       }
                   }
                   // Right half is sorted
                   else {
                       if (nums[mid] < target && target <= nums[high]) {
                           low = mid + 1;
                       } else {
                           high = mid - 1;
                       }
                   }
               }
               return false;
           }
       }
```

Time & Space Complexity

Time Complexity: O(log n) average, O(n) worst | Space Complexity: O(1)

57. Find Minimum in Rotated Sorted Array

Understanding the Problem NEW

Find the minimum element in a rotated sorted array (unique elements) — this minimum is exactly the 'pivot' point where the rotation happened. E.g. in `[4,5,6,7,0,1,2]`, the minimum is 0.

Brute Force Approach NEW

Scan every element and track the smallest seen — O(n), which works but again ignores the structure.

Why We Optimize NEW

Compare the middle element to the **rightmost** element: if `arr[mid] > arr[high]`, the minimum must be somewhere to the right of mid (because the left portion is 'too high' to contain the rotation point); otherwise, the minimum is at or before mid. This single comparison rule lets you discard half the array each step, finding the pivot point in O(log n) instead of scanning everything.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum element in rotated sorted array.

Visual Diagram

```
Array: [4, 5, 6, 7, 0, 1, 2]

       Step 1: low=0, high=6, mid=3
       [4, 5, 6, 7, 0, 1, 2]
                ↑
              arr[3]=7

       arr[mid] > arr[high] → Minimum in right half
       low = mid + 1 = 4

       Step 2: low=4, high=6, mid=5
       [4, 5, 6, 7, 0, 1, 2]
                     ↑
                   arr[5]=1
```

```
       arr[mid] <= arr[high] → Minimum in left half
       high = mid = 5

       Step 3: low=4, high=5, mid=4
       [4, 5, 6, 7, 0, 1, 2]
                   ↑
                 arr[4]=0

       arr[mid] <= arr[high] → high=4

       Step 4: low=4, high=4
       Return arr[low] = 0

       Result: 0
```

Solution Approach

Concept: Compare mid with high to find pivot.

Algorithm

```
Algorithm FIND_MIN_ROTATED(arr):
           low = 0, high = n-1
           While low < high:
               mid = (low + high) / 2
               If arr[mid] > arr[high]:
                   low = mid + 1
               Else:
                   high = mid
           Return arr[low]
```

Java Implementation

```
class Solution {
           public int findMin(int[] nums) {
               int low = 0, high = nums.length - 1;

                 while (low < high) {
                     int mid = low + (high - low) / 2;
                     if (nums[mid] > nums[high]) {

                       low = mid + 1;
                   } else {
                       high = mid;
                   }
               }
               return nums[low];
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

58. Find How Many Times Array is Rotated

Understanding the Problem NEW

Figure out how many positions the original sorted array was rotated by. This turns out to be the same question as finding the *index* of the minimum element.

Brute Force Approach NEW

Scan for the index where a value is smaller than its predecessor — O(n).

Why We Optimize NEW

This directly reuses the Find-Minimum-in-Rotated-Array technique (problem 57): the index of the minimum element *is* the rotation count. No new algorithm needed — just recognize the equivalence and reuse the O(log n) binary search.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the number of rotations in a rotated sorted array.

Visual Diagram

```
Array: [4, 5, 6, 7, 0, 1, 2]

       Find minimum element index:
       Minimum = 0 at index 4
       Number of rotations = index = 4

       Verification:
       Original: [0, 1, 2, 4, 5, 6, 7]
       Rotated 4 times: [4, 5, 6, 7, 0, 1, 2]
       Index of minimum = 4

       Array: [11, 12, 15, 18, 2, 5, 6, 8]
       Minimum = 2 at index 4
       Rotations = 4

       Array: [1, 2, 3, 4, 5]
       Minimum = 1 at index 0
       Rotations = 0
```

Solution Approach

Concept: Find index of minimum element.

Algorithm

```
Algorithm COUNT_ROTATIONS(arr):
           low = 0, high = n-1
           While low < high:
               mid = (low + high) / 2
               If arr[mid] > arr[high]:
                   low = mid + 1
               Else:
                   high = mid
           Return low // Number of rotations
```

Java Implementation

```
class Solution {
           public int countRotations(int[] nums) {
               int low = 0, high = nums.length - 1;

                 while (low < high) {
                     int mid = low + (high - low) / 2;

                    if (nums[mid] > nums[high]) {
                        low = mid + 1;
                    } else {
                        high = mid;
                    }
                 }
                 return low; // index of minimum = number of rotations
           }
       }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

59. Single Element in a Sorted Array

Understanding the Problem NEW

In a sorted array, every element appears exactly twice except one, which appears once — find that lone element. E.g. `[1,1,3,3,4,4,8,8,10,10,11]` → answer `11`. Note this differs from the earlier 'find the single number' array problem because this array is *sorted*, which changes what's possible.

Brute Force Approach NEW

Since finding a 'lone' element among pairs generally uses XOR (as in the unsorted version), a beginner might XOR everything together — O(n), which works but completely ignores the sorted structure that's now available.

Why We Optimize NEW

Because the array is sorted, every proper pair sits at positions (even, odd) *before* the lone element, and (odd, even) *after* it. So you can binary search on this pattern: check whether `arr[mid]` matches its 'expected' pair partner (based on whether `mid` is even or odd); if the pattern holds, the lone element is to the right; if it's broken, the lone element is at or before mid. This exploits the sorted order to get O(log n), better than the O(n) XOR approach that's forced to look at everything.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the single element in a sorted array where every other appears twice.

Visual Diagram

```
Array: [1, 1, 2, 3, 3, 4, 4, 8, 8]

       Observation:
       Index: 0 1 2 3 4 5 6 7 8
              [1, 1, 2, 3, 3, 4, 4, 8, 8]
       Pair: (0,1) (2,3) (4,5) (6,7) 8

       Single element at index 2

       Binary Search:
```

```
       Step 1: low=0, high=8, mid=4
       [1, 1, 2, 3, 3, 4, 4, 8, 8]
                   ↑
                 arr[4]=3
       Check pair: arr[4] == arr[5] (3==3)
       Single element is to the left → high=3

       Step 2: low=0, high=3, mid=1
       [1, 1, 2, 3, 3, 4, 4, 8, 8]
             ↑
           arr[1]=1
       Check pair: arr[0] == arr[1] (1==1)
       Single element is to the right → low=2

       Step 3: low=2, high=3, mid=2
       [1, 1, 2, 3, 3, 4, 4, 8, 8]
                ↑
              arr[2]=2
       Check pair: arr[2] != arr[3] (2!=3)
       Single element found → Return 2
```

Solution Approach

Concept: Binary search using pair pattern.

Algorithm

```
Algorithm SINGLE_ELEMENT(arr):
           low = 0, high = n-1
           While low < high:
               mid = (low + high) / 2
               If mid % 2 == 1:
                   mid-- // Make mid even

               If arr[mid] == arr[mid+1]:
                   low = mid + 2
               Else:
                   high = mid
           Return arr[low]
```

Java Implementation

```
class Solution {
           public int singleNonDuplicate(int[] nums) {
               int low = 0, high = nums.length - 1;

               while (low < high) {
                   int mid = low + (high - low) / 2;

                   // Make mid even to check pairs
```

```
                if (mid % 2 == 1) {
                    mid--;
                }

                // If pair exists, single is on right
                if (nums[mid] == nums[mid + 1]) {
                    low = mid + 2;
                } else {
                    high = mid;
                }
            }
            return nums[low];
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

60. Find Peak Element

Understanding the Problem NEW

A 'peak' element is one that's bigger than both its neighbours (treating out-of-bounds neighbours as negative infinity). Find *any* peak — there might be several, you just need one. E.g. in `[1,2,3,1]`, index 2 (value 3) is a peak.

Brute Force Approach NEW

Scan left to right checking each element against both neighbours — O(n), correct but unnecessary given what binary search can offer here.

Why We Optimize NEW

The trick: if `arr[mid] < arr[mid+1]`, the array is 'still rising' at mid, which guarantees a peak exists somewhere to the right (since values can't rise forever); so you search right. Otherwise, a peak is guaranteed to exist at mid or to its left, so you search left. This guarantee — that a peak must exist in the direction you're 'still rising' toward — is precisely what makes binary search valid here, even though the array as a whole isn't sorted, giving O(log n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find a peak element (greater than its neighbors).

Visual Diagram

```
Array: [1, 2, 3, 1]

       Step 1: low=0, high=3, mid=1
       [1, 2, 3, 1]
           ↑
         arr[1]=2

       arr[1] < arr[2] → Peak on right
       low = mid + 1 = 2

       Step 2: low=2, high=3, mid=2
       [1, 2, 3, 1]
              ↑
```

```
             arr[2]=3

       arr[2] > arr[3] → arr[2] is peak
       Return 2

       Array: [1, 2, 1, 3, 5, 6, 4]
       Peaks: 2 (index 1), 6 (index 5)

       Binary Search:
       Step 1: mid=3, arr[3]=3, arr[4]=5
       3 < 5 → Peak on right

       Step 2: mid=5, arr[5]=6, arr[6]=4
       6 > 4 → Peak at index 5
```

Solution Approach

Concept: Binary search comparing mid with mid+1.

Algorithm

```
Algorithm FIND_PEAK(arr):
           low = 0, high = n-1
           While low < high:
               mid = (low + high) / 2
               If arr[mid] < arr[mid+1]:
                   low = mid + 1
               Else:
                   high = mid
           Return low
```

Java Implementation

```
class Solution {
           public int findPeakElement(int[] nums) {
               int low = 0, high = nums.length - 1;

                 while (low < high) {
                     int mid = low + (high - low) / 2;

                    if (nums[mid] < nums[mid + 1]) {
                        low = mid + 1;

                   } else {
                       high = mid;
                   }
               }
               return low;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

61. Find Square Root of a Number in log n

Understanding the Problem NEW

Find the integer square root of a number N (the largest integer whose square is ≤ N) without using a built-in sqrt function. E.g. sqrt(28) → 5, since $5^2=25 \leq 28$ but $6^2=36>28$.

Brute Force Approach NEW

Try every integer from 1 upward, squaring it and checking if it exceeds N, stopping at the last one that didn't — O(√N) time (or O(N) if you're even less careful).

Why We Optimize NEW

This is **binary search on the answer** rather than binary search on an array — a very important generalization of the technique. The 'answer space' (possible square roots from 1 to N) is monotonic: if some value k satisfies $k^2 \leq N$, every smaller value does too. That monotonic yes/no property is exactly what lets you binary search over the space of possible answers, checking the middle candidate's square against N and discarding half the remaining candidates each time — O(log N) instead of O(√N).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find integer square root of a number.

Visual Diagram

```
Number: 36

       Step 1: low=0, high=36, mid=18
       18² = 324 > 36 → high=17

       Step 2: low=0, high=17, mid=8
       8² = 64 > 36 → high=7

       Step 3: low=0, high=7, mid=3
       3² = 9 < 36 → ans=3, low=4
```

```
Step 4: low=4, high=7, mid=5
5² = 25 < 36 → ans=5, low=6

Step 5: low=6, high=7, mid=6
6² = 36 ≤ 36 → ans=6, low=7

Result: 6

Number: 10
Step 1: mid=5, 25>10 → high=4
Step 2: mid=2, 4<10 → ans=2, low=3
Step 3: mid=3, 9<10 → ans=3, low=4
Result: 3 (floor sqrt)
```

Solution Approach

Concept: Binary search on answer space.

Algorithm

```
Algorithm SQRT(x):
           If x < 2: Return x
           low = 0, high = x
           ans = 0
           While low <= high:
               mid = (low + high) / 2
               If mid * mid <= x:
                   ans = mid
                   low = mid + 1
               Else:
                   high = mid - 1
           Return ans
```

Java Implementation

```
class Solution {
           public int mySqrt(int x) {
               if (x < 2) return x;

                 int low = 0, high = x;
                 int ans = 0;

               while (low <= high) {
                   int mid = low + (high - low) / 2;

                   // Use long to prevent overflow
                   long square = (long) mid * mid;

                   if (square <= x) {
                       ans = mid;
```

```
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
            return ans;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

62. Find the Nth Root of a Number

Understanding the Problem NEW

A generalization of the previous problem: find the integer N-th root of M (the integer x such that x^n equals M, or determine none exists).

Brute Force Approach NEW

Try every integer from 1 upward, raising it to the N-th power and comparing to M — O(M) in the worst case (or O(M^(1/N)) with a smarter loop bound).

Why We Optimize NEW

Exactly the same 'binary search on the answer' idea as square root, generalized: the candidate range is 1 to M, and whether `x^N ≤ M` is monotonic in x, so you can binary search over candidate values of x, checking each candidate's N-th power — giving roughly O(N log M) instead of a linear scan over all possible roots.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find N-th root of M using binary search.

Visual Diagram

```
N=3, M=27 (Cube root)

       Step 1: low=1, high=27, mid=14
       14³ = 2744 > 27 → high=13

       Step 2: low=1, high=13, mid=7
       7³ = 343 > 27 → high=6

       Step 3: low=1, high=6, mid=3
       3³ = 27 ≤ 27 → ans=3, low=4

       Result: 3

       N=2, M=10
       Step 1: mid=5, 25>10 → high=4
       Step 2: mid=2, 4<10 → ans=2, low=3
```

```
       Step 3: mid=3, 9<10 → ans=3, low=4
       Result: 3 (floor root)
```

Solution Approach

Concept: Binary search on answer with power check.

Algorithm

```
Algorithm NTH_ROOT(n, m):
           low = 0, high = m
           ans = 0
           While low <= high:
               mid = (low + high) / 2
               If pow(mid, n) <= m:
                   ans = mid
                   low = mid + 1
               Else:
                   high = mid - 1
           Return ans
```

Java Implementation

```
class Solution {
           public int nthRoot(int n, int m) {
               int low = 0, high = m;
               int ans = 0;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                     if (Math.pow(mid, n) <= m) {
                         ans = mid;
                         low = mid + 1;
                     } else {
                         high = mid - 1;
                     }
                 }
                 return ans;

           }
       }
```

Time & Space Complexity

Time Complexity: O(log m * n) | Space Complexity: O(1)

63. Koko Eating Bananas

Understanding the Problem NEW

Koko has piles of bananas and H hours to eat them all; each hour she picks one pile and eats up to 'speed' bananas from it (if the pile has fewer, she finishes that pile and doesn't eat more that hour). Find the minimum integer eating speed that lets her finish within H hours.

Brute Force Approach NEW

Try every possible speed starting from 1 upward, simulating how many hours it would take at that speed, and stop at the first speed that fits within H hours. This can be O(max(piles) × n) in the worst case — trying many candidate speeds, each requiring a full simulation pass.

Why We Optimize NEW

Here's the pattern-recognition moment for this whole sub-chapter: 'find the minimum X such that condition(X) is achievable' is a classic **binary search on the answer** setup, whenever achievability is monotonic (if speed S works, every faster speed also works). So instead of trying every candidate speed one at a time, binary search the speed range from 1 to max(pile) — for each candidate speed, simulate in O(n) how many hours it takes, and narrow the search based on whether that's within H hours or not. This turns O(max(piles) × n) into O(n × log(max(piles))).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum eating speed to eat all bananas in H hours.

Visual Diagram

```
Piles: [3, 6, 7, 11], H=8

       Speed = 4:
       Hour 1: pile 3 (3/4=1 hour)
       Hour 2-3: pile 6 (6/4=2 hours)
       Hour 4-5: pile 7 (7/4=2 hours)
       Hour 6-8: pile 11 (11/4=3 hours)
```

```
        Total: 1+2+2+3 = 8 hours ✓

        Speed = 3:
        Pile 3: 1 hour
        Pile 6: 2 hours
        Pile 7: 3 hours (ceil 7/3=3)
        Pile 11: 4 hours (ceil 11/3=4)
        Total: 1+2+3+4 = 10 hours ✗

        Binary Search:
        low=1, high=11
        mid=6: total=1+1+2+2=6 ✓ → high=5
        mid=3: total=1+2+3+4=10 ✗ → low=4
        mid=4: total=1+2+2+3=8 ✓ → high=3
        Result: 4
```

Solution Approach

Concept: Binary search on eating speed.

Algorithm

```
Algorithm KOKO(piles, H):
            low = 1, high = max(piles)
            ans = high

            While low <= high:
                mid = (low + high) / 2
                hours = 0
                For each pile in piles:
                    hours += ceil(pile / mid)

                If hours <= H:
                    ans = mid
                    high = mid - 1
                Else:
                    low = mid + 1
            Return ans
```

Java Implementation

```
class Solution {
            public int minEatingSpeed(int[] piles, int h) {
                int low = 1, high = 0;
                for (int pile : piles) high = Math.max(high, pile);

                  int ans = high;
                  while (low <= high) {

                    int mid = low + (high - low) / 2;
```

```
                long hours = 0;

                for (int pile : piles) {
                    hours += (pile + mid - 1) / mid; // ceil division
                }

                if (hours <= h) {
                    ans = mid;
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            }
            return ans;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log max) | Space Complexity: O(1)

64. Minimum Days to Make M Bouquets

Understanding the Problem NEW

Given flowers that bloom on specific days, find the minimum number of days needed to be able to make M bouquets, each requiring K **adjacent** already-bloomed flowers.

Brute Force Approach NEW

Try every possible day count starting from the earliest bloom day, simulating whether M bouquets are achievable by that day — potentially checking every day one at a time, each simulation costing O(n), giving O(n × maxDay) in the worst case.

Why We Optimize NEW

Same binary-search-on-the-answer pattern as Koko Eating Bananas: whether 'M bouquets are possible by day D' is monotonic in D (if it's possible by day D, it's also possible by any later day), so binary search over the range of possible days, using an O(n) feasibility check at each candidate day, instead of testing every day individually — turning a potentially huge linear scan over days into O(n × log(maxDay)).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum days to make M bouquets with K adjacent flowers.

Visual Diagram

```
Bloom: [1, 10, 3, 10, 2], M=3, K=1

       Need: 3 bouquets × 1 flower = 3 flowers
       Adjacent requirement: K=1 (any flower)

       Day 1: [1] → 1 bouquet
       Day 2: [1, 2] → 2 bouquets
       Day 3: [1, 2, 3] → 3 bouquets ✓

       Result: 3
```

```
Example 2: Bloom: [1, 10, 3, 10, 2], M=3, K=2
Need: 3 bouquets × 2 flowers = 6 flowers
But only 5 flowers total → Impossible

Binary Search:
low=1, high=10
mid=5: At day 5, flowers [1,3,2] bloom
Adjacent groups: [1] (1), [3] (1), [2] (1)
Bouquets = 0 ✗ → low=6

mid=8: At day 8, flowers [1,3,2] bloom
Still not enough → low=9

mid=9: Same → low=10

Result: -1 (Impossible)
```

Solution Approach

Concept: Binary search on days.

Algorithm

```
Algorithm MIN_DAYS(bloom, M, K):
           If M*K > n: Return -1
           low = min(bloom), high = max(bloom)
           ans = high

           While low <= high:
               mid = (low + high) / 2
               If possible(mid, bloom, M, K):
                   ans = mid
                   high = mid - 1
               Else:
                   low = mid + 1
           Return ans
```

Java Implementation

```
class Solution {
           public int minDays(int[] bloomDay, int m, int k) {
               int n = bloomDay.length;
               if (m * k > n) return -1;
               int low = Integer.MAX_VALUE, high = 0;

              for (int day : bloomDay) {
                  low = Math.min(low, day);
                  high = Math.max(high, day);
              }
```

```
            int ans = high;

            while (low <= high) {
                int mid = low + (high - low) / 2;

                if (canMakeBouquets(bloomDay, mid, m, k)) {
                    ans = mid;
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            }
            return ans;
        }

        private boolean canMakeBouquets(int[] bloomDay, int day, int m, int k) {
            int bouquets = 0;
            int adjacent = 0;

            for (int bloom : bloomDay) {
                if (bloom <= day) {
                    adjacent++;
                    if (adjacent == k) {
                         bouquets++;
                         adjacent = 0;
                    }
                } else {
                    adjacent = 0;
                }
            }
            return bouquets >= m;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log max) | Space Complexity: O(1)

65. Find the Smallest Divisor

Understanding the Problem NEW

Given an array and a threshold, find the smallest divisor such that if every element is divided by it (rounded up) and summed, the total doesn't exceed the threshold.

Brute Force Approach NEW

Try every possible divisor starting from 1 upward, computing the resulting sum each time, and stop at the first divisor whose sum fits — potentially O(max(nums) × n).

Why We Optimize NEW

Again the same recognizable shape: 'find the smallest divisor such that condition holds' is monotonic (a bigger divisor only ever produces a smaller or equal sum), so binary search over possible divisor values from 1 to max(nums), checking the resulting sum in O(n) at each candidate — O(n × log(max(nums))) instead of a full linear scan over every possible divisor.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find smallest divisor such that sum of ceil(nums[i]/divisor) <= threshold.

Visual Diagram

```
Array: [1, 2, 5, 9], Threshold: 6

       Divisor 1: sum = 1+2+5+9 = 17 > 6 ✗
       Divisor 2: sum = 1+1+3+5 = 10 > 6 ✗
       Divisor 3: sum = 1+1+2+3 = 7 > 6 ✗
       Divisor 4: sum = 1+1+2+3 = 7 > 6 ✗
       Divisor 5: sum = 1+1+1+2 = 5 ≤ 6 ✓

       Binary Search:
       low=1, high=9
       mid=5: sum=1+1+1+2=5 ✓ → ans=5, high=4
       mid=3: sum=1+1+2+3=7 ✗ → low=4
```

```
       mid=4: sum=1+1+2+3=7 ✗ → low=5
       Result: 5
```

Solution Approach

Concept: Binary search on divisor.

Algorithm

```
Algorithm SMALLEST_DIVISOR(nums, threshold):
           low = 1, high = max(nums)
           ans = high

           While low <= high:
               mid = (low + high) / 2
               sum = 0
               For each num in nums:
                   sum += ceil(num / mid)

               If sum <= threshold:
                   ans = mid
                   high = mid - 1
               Else:
                   low = mid + 1
           Return ans
```

Java Implementation

```
class Solution {
           public int smallestDivisor(int[] nums, int threshold) {
               int low = 1, high = 0;
               for (int num : nums) high = Math.max(high, num);

                 int ans = high;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;
                     int sum = 0;

                     for (int num : nums) {
                         sum += (num + mid - 1) / mid; // ceil division
                     }
                     if (sum <= threshold) {

                      ans = mid;
                      high = mid - 1;
                   } else {
                       low = mid + 1;
                   }
              }
```

```
            return ans;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log max) | Space Complexity: O(1)

66. Capacity to Ship Packages Within D Days

Understanding the Problem NEW

Find the minimum ship capacity needed to deliver all packages (in their given order) within D days, where each day's shipment can't exceed the ship's capacity.

Brute Force Approach NEW

Try every possible capacity value starting from the largest single package weight upward, simulating how many days it takes at that capacity — up to O(sum(weights) × n) in the worst case.

Why We Optimize NEW

The same monotonic pattern again: a bigger capacity only ever needs the same or fewer days, so binary search the capacity range between max(weight) and sum(weights), simulating the day-count at each candidate capacity in O(n) — O(n × log(sum(weights))) instead of testing every possible capacity value individually. By this point in the chapter, it's worth noticing that problems 63-66 (and several more ahead) are really the *same* algorithmic template wearing different clothes.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum capacity to ship all packages within D days.

Visual Diagram

```
Weights: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Days: 5

       Capacity = 15:
       Day 1: 1+2+3+4+5=15
       Day 2: 6+7=13
       Day 3: 8=8
       Day 4: 9=9
       Day 5: 10=10
       Total days: 5 ✓

       Capacity = 14:
       Day 1: 1+2+3+4+5=15 > 14 ✗
```

```
Binary Search:
low=10, high=55
mid=32: days=4 ✓ → high=31
mid=21: days=5 ✓ → high=20
mid=15: days=5 ✓ → high=14
mid=13: days=6 ✗ → low=14
mid=14: days=5 ✓ → high=13
Result: 14
```

Solution Approach

Concept: Binary search on capacity.

Algorithm

```
Algorithm SHIP_CAPACITY(weights, days):
           low = max(weights), high = sum(weights)
           ans = high

           While low <= high:
               mid = (low + high) / 2
               if canShip(weights, mid, days):
                   ans = mid
                   high = mid - 1
               Else:
                   low = mid + 1
           Return ans
```

Java Implementation

```
class Solution {
           public int shipWithinDays(int[] weights, int days) {
               int low = 0, high = 0;
               for (int weight : weights) {
                   low = Math.max(low, weight);
                   high += weight;
               }

                 int ans = high;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                  if (canShip(weights, mid, days)) {
                      ans = mid;
                      high = mid - 1;
                  } else {
                      low = mid + 1;
                  }
             }
```

```
            return ans;
        }

        private boolean canShip(int[] weights, int capacity, int days) {
            int requiredDays = 1;
            int currentWeight = 0;

            for (int weight : weights) {
                if (currentWeight + weight > capacity) {
                    requiredDays++;
                    currentWeight = 0;
                }
                currentWeight += weight;
            }
            return requiredDays <= days;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log sum) | Space Complexity: O(1)

67. Kth Missing Positive Number

Understanding the Problem NEW

Given a sorted array of positive integers, find the k-th positive integer that's *missing* from the array. E.g. array `[2,3,4,7,11]`, k=5 → the missing numbers in order are 1,5,6,8,9,10,... so the 5th missing is 9.

Brute Force Approach NEW

Walk through positive integers 1, 2, 3, ... one at a time, checking whether each is present in the array (possibly with a linear scan or pointer), counting misses until you reach the k-th — O(n + answer size), which can be slow if k is large.

Why We Optimize NEW

At any array index i, you can directly **compute** how many numbers are missing up to that point: it's `arr[i] - (i+1)` (the difference between what the value 'should' be if nothing were missing, and its actual value). This missing-count is monotonically increasing as i increases, so you can binary search on the array index for the point where the missing-count first reaches or exceeds k, then do simple arithmetic to pin down the exact missing number — O(log n) instead of counting misses one at a time.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find kth missing positive number in a sorted array.

Visual Diagram

```
Array: [2, 3, 4, 7, 11], K=5

       Missing numbers:
       1 (missing), 5 (missing), 6 (missing), 8 (missing), 9 (missing)
            ↑           ↑          ↑           ↑           ↑
            1st         2nd        3rd         4th         5th

       5th missing = 9

       Binary Search:
       For each index i:
```

```
       missing_count = arr[i] - (i+1)

       Index 0: 2-1=1 missing
       Index 1: 3-2=1 missing
       Index 2: 4-3=1 missing
       Index 3: 7-4=3 missing
       Index 4: 11-5=6 missing

       Find first index where missing_count >= K:
       Index 3: missing=3 < 5 ✗
       Index 4: missing=6 >= 5 ✓

       ans = arr[4] - (missing_count - K) - 1
       ans = 11 - (6-5) - 1 = 9
```

Solution Approach

Concept: Binary search on missing count.

Algorithm

```
Algorithm KTH_MISSING(arr, K):
           low = 0, high = n-1
           ans = n

           While low <= high:
               mid = (low + high) / 2
               missing = arr[mid] - (mid + 1)

                 If missing >= K:
                     ans = mid
                     high = mid - 1
                 Else:
                     low = mid + 1

           Return ans + K
```

Java Implementation

```
class Solution {
           public int findKthPositive(int[] arr, int k) {
               int low = 0, high = arr.length - 1;
               int ans = arr.length;
               while (low <= high) {

                   int mid = low + (high - low) / 2;
                   int missing = arr[mid] - (mid + 1);

                   if (missing >= k) {
                       ans = mid;
```

```
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            }
            return ans + k;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

68. Aggressive Cows

Understanding the Problem NEW

Given stall positions and a number of cows, place all cows in stalls to **maximize the minimum distance** between any two cows (so they're as spread out as possible). A classic 'maximize the minimum' optimization.

Brute Force Approach NEW

Try every possible minimum-distance value starting from 1 upward, and for each, greedily check if that many cows can be placed with at least that spacing — potentially checking every possible distance value one at a time, O(maxDistance × n).

Why We Optimize NEW

'Maximize the minimum distance such that placement is feasible' is monotonic in the opposite direction from the previous problems: a *smaller* distance is always easier to satisfy, a larger one is harder — but the feasibility still flips cleanly at one threshold, which is exactly what binary search needs. Binary search over possible distances, using an O(n) greedy check (can I fit all cows with at least this spacing?) at each candidate — O(n × log(maxDistance)) instead of testing every distance individually. This is the first 'maximize' version of the pattern, as opposed to the 'minimize' versions in problems 63-66.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Place cows in stalls with maximum minimum distance.

Visual Diagram

```
Stalls: [1, 2, 4, 8, 9], Cows: 3

       Distance = 3:
       Place cow 1: stall 1
       Place cow 2: stall 4 (distance = 3)
       Place cow 3: stall 8 (distance = 4)
       Minimum distance = 3 ✓
```

```
Distance = 4:
Cow 1: stall 1
Cow 2: stall 4 (distance=3 < 4) ✗

Binary Search:
low=1, high=8
mid=4: Can place? ✗ → high=3
mid=2: Can place? ✓ → ans=2, low=3
mid=3: Can place? ✓ → ans=3, low=4
Result: 3
```

Solution Approach

Concept: Binary search on answer (distance).

Algorithm

```
Algorithm AGGRESSIVE_COWS(stalls, cows):
           Sort stalls
           low = 1, high = stalls[n-1] - stalls[0]
           ans = 0

           While low <= high:
               mid = (low + high) / 2
               If canPlace(stalls, mid, cows):
                   ans = mid
                   low = mid + 1
               Else:
                   high = mid - 1
           Return ans
```

Java Implementation

```
class Solution {
           public int aggressiveCows(int[] stalls, int cows) {
               Arrays.sort(stalls);

                 int low = 1, high = stalls[stalls.length - 1] - stalls[0];
                 int ans = 0;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                    if (canPlace(stalls, mid, cows)) {
                        ans = mid;
                        low = mid + 1;
                    } else {

                       high = mid - 1;
                   }
```

```
            }
            return ans;
        }

        private boolean canPlace(int[] stalls, int distance, int cows) {
            int placed = 1;
            int lastPlaced = stalls[0];

            for (int i = 1; i < stalls.length; i++) {
                if (stalls[i] - lastPlaced >= distance) {
                    placed++;
                    lastPlaced = stalls[i];

                    if (placed >= cows) return true;
                }
            }
            return false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log range) | Space Complexity: O(1)

69. Book Allocation Problem

Understanding the Problem NEW

Distribute books (given as page counts) among students, giving each student a contiguous set of books, so as to **minimize the maximum** number of pages any single student has to read.

Brute Force Approach NEW

Try every possible way of splitting the books among students (all possible partitions) and track the split that minimizes the worst student's total — combinatorially expensive, potentially exponential if done by brute enumeration.

Why We Optimize NEW

Just like Koko's eating speed, binary search on the answer: guess a maximum-pages-per-student value, then greedily check in O(n) whether that many students are enough to cover all books without exceeding the guessed maximum per student. Feasibility is monotonic (a higher 'maximum allowed' is always at least as easy to satisfy), so you binary search over the range of possible maximums — O(n × log(sum of pages)) instead of exploring every partition.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Allocate books to students with minimum maximum pages.

Visual Diagram

```
Books: [12, 34, 67, 90], Students: 2

      Allocate:
      Student 1: 12+34=46, Student 2: 67+90=157 → Max=157
      Student 1: 12, Student 2: 34+67+90=191 → Max=191
      Student 1: 12+34+67=113, Student 2: 90 → Max=113

      Minimum possible maximum = 113

      Binary Search:
      low=90 (max book), high=203 (sum)
      mid=146: Can allocate? ✓ → high=145
```

```
      mid=113: Can allocate? ✓ → high=112
      mid=101: Can allocate? ✗ → low=102
      mid=113: Can allocate? ✓ → high=112
      Result: 113
```

Solution Approach

Concept: Binary search on maximum pages.

Algorithm

```
Algorithm BOOK_ALLOCATION(books, students):
          If students > n: Return -1
          low = max(books), high = sum(books)
          ans = high

          While low <= high:
              mid = (low + high) / 2
              If canAllocate(books, mid, students):
                  ans = mid
                  high = mid - 1
              Else:
                  low = mid + 1
          Return ans
```

Java Implementation

```
class Solution {
          public int allocateBooks(int[] books, int students) {
              if (students > books.length) return -1;

                 int low = 0, high = 0;
                 for (int book : books) {
                     low = Math.max(low, book);
                     high += book;
                 }

                 int ans = high;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                     if (canAllocate(books, mid, students)) {

                      ans = mid;
                      high = mid - 1;
                  } else {
                      low = mid + 1;
                  }
              }
```

```
            return ans;
        }

        private boolean canAllocate(int[] books, int maxPages, int students) {
            int requiredStudents = 1;
            int currentPages = 0;

            for (int book : books) {
                if (currentPages + book > maxPages) {
                    requiredStudents++;
                    currentPages = 0;
                }
                currentPages += book;
            }
            return requiredStudents <= students;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log sum) | Space Complexity: O(1)

70. Split Array - Largest Sum

Understanding the Problem NEW

Split an array into K contiguous subarrays so as to minimize the largest subarray sum — structurally identical to Book Allocation, just phrased with numbers instead of book pages.

Brute Force Approach NEW

Try every possible way to split the array into K parts and track the best (minimum of the maximums) — combinatorially expensive.

Why We Optimize NEW

This is a straight reuse of the Book Allocation technique: binary search over possible 'maximum subarray sum' values, checking feasibility (can this array be split into ≤K parts each summing to at most this value?) in O(n) at each candidate. Recognizing that a new-looking problem is a renamed version of one you've already solved is one of the most valuable pattern-matching skills in this style of problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Split array into k subarrays minimizing maximum sum.

Visual Diagram

```
Array: [7, 2, 5, 10, 8], K=2

       Possible splits:
       [7] [2,5,10,8] → max=25
       [7,2] [5,10,8] → max=23
       [7,2,5] [10,8] → max=18
       [7,2,5,10] [8] → max=24

       Minimum possible maximum = 18

       Binary Search:
       low=10 (max element), high=32 (sum)
       mid=21: Can split? ✓ → high=20
```

```
       mid=15: Can split? ✗ → low=16
       mid=18: Can split? ✓ → high=17
       mid=17: Can split? ✗ → low=18
       Result: 18
```

Solution Approach

Concept: Same as book allocation.

Algorithm

```
Algorithm SPLIT_ARRAY(nums, k):
           low = max(nums), high = sum(nums)
           ans = high

           While low <= high:
               mid = (low + high) / 2
               If canSplit(nums, mid, k):
                   ans = mid
                   high = mid - 1
               Else:
                   low = mid + 1
           Return ans
```

Java Implementation

```
class Solution {
           public int splitArray(int[] nums, int k) {
               int low = 0, high = 0;
               for (int num : nums) {
                   low = Math.max(low, num);
                   high += num;
               }

                 int ans = high;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                     if (canSplit(nums, mid, k)) {
                         ans = mid;
                         high = mid - 1;

                    } else {
                        low = mid + 1;
                    }
              }
              return ans;
          }
```

```
        private boolean canSplit(int[] nums, int maxSum, int k) {
            int required = 1;
            int currentSum = 0;

            for (int num : nums) {
                if (currentSum + num > maxSum) {
                    required++;
                    currentSum = 0;
                }
                currentSum += num;
            }
            return required <= k;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log sum) | Space Complexity: O(1)

71. Painter's Partition Problem

Understanding the Problem NEW

Given boards of different lengths and a number of painters (each painting a contiguous set of boards, one unit length per unit time), find the minimum time to paint all boards. Yet another disguise of the same underlying problem.

Brute Force Approach NEW

Try every possible allocation of boards to painters and track the best worst-case time — combinatorially expensive.

Why We Optimize NEW

Same binary-search-on-the-answer template once more: binary search over possible 'maximum time' values, and for each candidate, greedily check in O(n) how many painters would be needed to keep every painter's total within that time. This is the third appearance of essentially the same technique (Book Allocation, Split Array, Painter's Partition) — a strong hint that once you recognize the shape ('minimize the maximum, subarray-based, monotonic feasibility'), you can solve a whole family of problems with one template.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum time to paint all boards with k painters.

Visual Diagram

```
Boards: [10, 20, 30, 40], Painters: 2

       Time per unit = 1 unit

       Same as split array:
       Possible allocations:
       [10] [20,30,40] → max=90
       [10,20] [30,40] → max=70
       [10,20,30] [40] → max=60

       Minimum time = 60
```

```
Binary Search:
low=40, high=100
mid=70: Can paint? ✓ → high=69
mid=55: Can paint? ✗ → low=56
mid=60: Can paint? ✓ → high=59
Result: 60
```

Solution Approach

Concept: Same as book allocation.

Algorithm

```
Algorithm PAINTER_PARTITION(boards, painters):
           low = max(boards), high = sum(boards)
           ans = high

           While low <= high:
               mid = (low + high) / 2
               If canPaint(boards, mid, painters):
                   ans = mid
                   high = mid - 1
               Else:
                   low = mid + 1
           Return ans
```

Java Implementation

```
class Solution {
           public int painterPartition(int[] boards, int painters) {
               int low = 0, high = 0;
               for (int board : boards) {
                   low = Math.max(low, board);
                   high += board;
               }

                 int ans = high;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                     if (canPaint(boards, mid, painters)) {
                         ans = mid;

                          high = mid - 1;
                    } else {
                         low = mid + 1;
                    }
              }
              return ans;
```

```
        }

        private boolean canPaint(int[] boards, int maxTime, int painters) {
            int required = 1;
            int currentTime = 0;

            for (int board : boards) {
                if (currentTime + board > maxTime) {
                    required++;
                    currentTime = 0;
                }
                currentTime += board;
            }
            return required <= painters;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log sum) | Space Complexity: O(1)

72. Minimize Max Distance to Gas Station

Understanding the Problem NEW

Given existing gas station positions along a road and a number of new stations to add, place the new stations to **minimize the maximum distance** between any two adjacent stations.

Brute Force Approach NEW

Try adding stations one at a time, always splitting the currently-largest gap — a greedy simulation that can require re-scanning for the largest gap repeatedly, or trying every possible max-distance value with a slow feasibility check, giving a poor overall complexity.

Why We Optimize NEW

Binary search on the answer again, this time over possible real-valued (not just integer) maximum distances: for a candidate maximum distance D, you can compute in O(n) how many new stations would be needed to make every existing gap ≤ D (each gap of length g needs `floor(g/D)` extra stations); check if that's ≤ the available stations. Feasibility is monotonic in D (a larger D always needs fewer or equal new stations), so binary search narrows in on the smallest feasible D efficiently, even over a continuous range of real numbers.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Place k stations to minimize maximum distance between stations.

Visual Diagram

```
Stations: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], K=1

       Distances: 1,1,1,1,1,1,1,1,1
       Max distance = 1

       Place station between 1 and 10 → distances: 4.5,4.5
       Max distance = 4.5

       Binary Search:
       low=0, high=9
```

```
mid=4.5: Can place? ✓ → ans=4.5, high=4.5
mid=2.25: Can place? ✓ → ans=2.25, high=2.25
mid=1.125: Can place? ✓ → ans=1.125
Result: 1.125 (approximately)
```

Solution Approach

Concept: Binary search on distance.

Algorithm

```
Algorithm MIN_MAX_DISTANCE(stations, K):
           low = 0, high = max gap
           precision = 1e-6

           While high - low > precision:
               mid = (low + high) / 2
               If canPlace(stations, mid, K):
                   high = mid
               Else:
                   low = mid
           Return high
```

Java Implementation

```
class Solution {
           public double minmaxGasDist(int[] stations, int k) {
               int n = stations.length;
               double low = 0, high = stations[n-1] - stations[0];

                 while (high - low > 1e-6) {
                     double mid = low + (high - low) / 2;

                     int count = 0;
                     for (int i = 0; i < n - 1; i++) {
                         count += (int) ((stations[i+1] - stations[i]) / mid);
                     }

                     if (count <= k) {
                         high = mid;
                     } else {
                         low = mid;
                     }
                 }
                 return high;

           }
      }
```

Time & Space Complexity

Time Complexity: O(n * log precision) | Space Complexity: O(1)

73. Median of 2 Sorted Arrays

Understanding the Problem NEW

Given two sorted arrays, find the median of all their elements combined, ideally without actually merging them into one big array first.

Brute Force Approach NEW

Merge both arrays into one sorted array (O(n+m)) and then just read off the middle element(s) — correct and simple, but throws away an opportunity for something faster.

Why We Optimize NEW

The key insight: the median splits the combined array into a left half and right half of equal size. You can binary search on **how many elements to take from the smaller array's left side** — for each candidate split, the corresponding split point in the other array is determined by arithmetic (since the halves must add up correctly), and you check whether the boundary elements satisfy the ordering that a valid partition requires. This lets you find the median without merging anything, in O(log(min(n,m))) time — dramatically better than the O(n+m) merge-based approach, purely by binary searching over 'how much to take' rather than over array values directly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find median of two sorted arrays.

Visual Diagram

```
Arr1: [1, 3], Arr2: [2]
       Sorted: [1, 2, 3] → Median = 2

       Arr1: [1, 2], Arr2: [3, 4]
       Sorted: [1, 2, 3, 4] → Median = (2+3)/2 = 2.5

       Binary Search (Partition Method):
       Arr1: [1, 3, 8, 9, 15], Arr2: [7, 11, 18, 19, 21, 25]

       We partition both arrays:
```

```
       Left half size = (5+6)/2 = 5

       Try partition in Arr1:
       part1 = 2, part2 = 3
       [1,3] | [8,9,15]
       [7,11,18] | [19,21,25]
       Left max = max(3,18) = 18
       Right min = min(8,19) = 8
       18 > 8 → need smaller partition in arr1

       part1 = 1, part2 = 4
       [1] | [3,8,9,15]
       [7,11,18,19] | [21,25]
       Left max = max(1,19) = 19
       Right min = min(3,21) = 3
       19 > 3 → need smaller partition

       part1 = 0, part2 = 5
       [] | [1,3,8,9,15]
       [7,11,18,19,21] | [25]
       Left max = max(-∞,21) = 21
       Right min = min(1,25) = 1
       21 > 1 → need smaller

       Not possible? Actually we need to ensure binary search works correctly.
```

Solution Approach

Concept: Partition method with binary search.

Algorithm

```
Algorithm MEDIAN_2_ARRAYS(A, B):
           If n > m: Swap arrays
           low = 0, high = n

           While low <= high:
               partitionA = (low + high) / 2
               partitionB = (n + m + 1) / 2 - partitionA

                 maxLeftA = partitionA == 0 ? -∞ : A[partitionA-1]
                 minRightA = partitionA == n ? ∞ : A[partitionA]
                 maxLeftB = partitionB == 0 ? -∞ : B[partitionB-1]
                 minRightB = partitionB == m ? ∞ : B[partitionB]
                 If maxLeftA <= minRightB and maxLeftB <= minRightA:

                    If (n+m)%2 == 0:
                        Return (max(maxLeftA, maxLeftB) + min(minRightA,
minRightB)) / 2
                    Else:
                        Return max(maxLeftA, maxLeftB)
```

```
                Else If maxLeftA > minRightB:
                     high = partitionA - 1
                Else:
                     low = partitionA + 1
           Return 0
```

Java Implementation

```
class Solution {
           public double findMedianSortedArrays(int[] nums1, int[] nums2) {
               int n = nums1.length, m = nums2.length;

               // Ensure nums1 is smaller
               if (n > m) {
                   return findMedianSortedArrays(nums2, nums1);
               }

               int low = 0, high = n;
               int total = n + m;

               while (low <= high) {
                   int partition1 = low + (high - low) / 2;
                   int partition2 = (total + 1) / 2 - partition1;

                   int maxLeft1 = partition1 == 0 ? Integer.MIN_VALUE :
nums1[partition1 - 1];
                   int minRight1 = partition1 == n ? Integer.MAX_VALUE :
nums1[partition1];
                   int maxLeft2 = partition2 == 0 ? Integer.MIN_VALUE :
nums2[partition2 - 1];
                   int minRight2 = partition2 == m ? Integer.MAX_VALUE :
nums2[partition2];

                   if (maxLeft1 <= minRight2 && maxLeft2 <= minRight1) {
                       if (total % 2 == 0) {
                            return (Math.max(maxLeft1, maxLeft2) +
Math.min(minRight1, minRight2)) / 2.
                       } else {
                            return Math.max(maxLeft1, maxLeft2);
                       }
                   } else if (maxLeft1 > minRight2) {
                       high = partition1 - 1;
                   } else {
                       low = partition1 + 1;
                   }
               }
               return 0;
           }
       }
```

Time & Space Complexity

Time Complexity: O(log min(n,m)) | Space Complexity: O(1)

74. Kth Element of 2 Sorted Arrays

Understanding the Problem NEW

Find the k-th smallest element across two sorted arrays combined, without necessarily merging them.

Brute Force Approach NEW

Merge both arrays (O(n+m)) and read off the k-th element directly — simple and correct, but again ignores a smarter option.

Why We Optimize NEW

Similar spirit to the median problem: you can binary search on how elements are partitioned between the two arrays to directly locate the k-th smallest without a full merge, achieving roughly O(log(min(n,m))) instead of O(n+m). (A simpler, though slightly less optimal, alternative is to repeatedly eliminate k/2 elements from whichever array has the smaller candidate at that position — also avoiding a full merge.) Either way, the lesson is the same as problem 73: full merging is a safe brute-force default, but exploiting the sorted structure of both arrays can beat it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find kth smallest element in two sorted arrays.

Visual Diagram

```
Arr1: [2, 3, 6, 7, 9], Arr2: [1, 4, 8, 10], K=5

       Merged: [1, 2, 3, 4, 6, 7, 8, 9, 10]
       5th element = 6

       Binary Search:
       Take k/2 = 2 elements from each array
       Arr1: 2,3 → compare with Arr2: 1,4
       3 < 4 → remove 2 elements from arr1

       Remaining: Arr1: [6,7,9], Arr2: [1,4,8,10]
       K = 3
```

```
Take k/2 = 1 from each
Arr1[0]=6, Arr2[0]=1
1 < 6 → remove from arr2

Remaining: Arr1: [6,7,9], Arr2: [4,8,10]
K = 2

Take 1 from each
6 > 4 → remove from arr2

Remaining: Arr1: [6,7,9], Arr2: [8,10]
K = 1 → min(6,8) = 6

Result: 6
```

Solution Approach

Concept: Binary search removing elements.

Algorithm

```
Algorithm KTH_ELEMENT(A, B, K):
           If n > m: Swap arrays
           low = max(0, K-m), high = min(K, n)

           While low <= high:
               cut1 = (low + high) / 2
               cut2 = K - cut1

                 left1 = cut1 == 0 ? -∞ : A[cut1-1]
                 right1 = cut1 == n ? ∞ : A[cut1]
                 left2 = cut2 == 0 ? -∞ : B[cut2-1]
                 right2 = cut2 == m ? ∞ : B[cut2]

               If left1 <= right2 and left2 <= right1:
                    Return max(left1, left2)
               Else If left1 > right2:
                    high = cut1 - 1
               Else:
                    low = cut1 + 1
           Return 0
```

Java Implementation

```
class Solution {
           public int kthElement(int[] arr1, int[] arr2, int k) {
               int n = arr1.length, m = arr2.length;

               if (n > m) {
```

```
                return kthElement(arr2, arr1, k);
            }

            int low = Math.max(0, k - m), high = Math.min(k, n);

            while (low <= high) {
                int cut1 = low + (high - low) / 2;
                int cut2 = k - cut1;

                int left1 = cut1 == 0 ? Integer.MIN_VALUE : arr1[cut1 - 1];
                int right1 = cut1 == n ? Integer.MAX_VALUE : arr1[cut1];
                int left2 = cut2 == 0 ? Integer.MIN_VALUE : arr2[cut2 - 1];
                int right2 = cut2 == m ? Integer.MAX_VALUE : arr2[cut2];

                if (left1 <= right2 && left2 <= right1) {
                    return Math.max(left1, left2);
                } else if (left1 > right2) {
                    high = cut1 - 1;
                } else {
                    low = cut1 + 1;
                }
            }
            return 0;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log min(n,m)) | Space Complexity: O(1)

75. Find the Row with Maximum Number of 1's

Understanding the Problem NEW

Given a matrix where each row is sorted (containing 0s then 1s), find the row that contains the most 1s.

Brute Force Approach NEW

Scan every cell in the matrix, counting 1s per row and tracking the max — O(rows × cols).

Why We Optimize NEW

Because each row is sorted, you can find the number of 1s in a row instantly with a binary search per row (finding the first 1, i.e. a Lower-Bound-style search) — O(log cols) per row instead of O(cols). Even better, since rows are 'sorted' relative to each other in a matrix like this, you can start scanning from the top-right corner and only move left or down as needed, visiting each row and column at most once — giving an overall O(rows + cols) traversal that beats even the per-row binary search version. This mirrors the 'staircase search' idea used again in problem 77.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find row with maximum number of 1's in a sorted binary matrix.

Visual Diagram

```
Matrix:
       Row 0: [0, 1, 1, 1]   → 3 ones
       Row 1: [0, 0, 1, 1]   → 2 ones
       Row 2: [0, 0, 0, 1]   → 1 one
       Row 3: [1, 1, 1, 1]   → 4 ones

       Result: Row 3

       Optimized Approach:
       Start from top-right corner
       Row 0: [0,1,1,1] → col=3, row=0
       Move left to first 1 at index 1 → max=3, ansRow=0
```

```
Row 1: [0,0,1,1] → col=3, row=1
At row 1, col=3: 1 → col=2: 1 → col=1: 0 (stop)
max=3 stays

Row 2: [0,0,0,1] → col=1, row=2
col=1: 0 → stop
max=3

Row 3: [1,1,1,1] → col=1, row=3
col=1: 1 → col=0: 1 → col=-1: 0
max=4, ansRow=3

Result: Row 3
```

Solution Approach

Concept: Traverse from top-right.

Algorithm

```
Algorithm MAX_ONES_ROW(matrix):
           rows = m, cols = n
           row = 0, col = n-1
           max_ones = 0, ans_row = 0

           While row < m and col >= 0:
               If matrix[row][col] == 1:
                   max_ones = col + 1
                   ans_row = row
                   col--
               Else:
                   row++
           Return ans_row
```

Java Implementation

```
class Solution {
           public int maxOnesRow(int[][] matrix) {
               int m = matrix.length, n = matrix[0].length;

              int row = 0, col = n - 1;
              int maxOnes = 0, ansRow = 0;

              while (row < m && col >= 0) {
                  if (matrix[row][col] == 1) {
                      maxOnes = col + 1;
                      ansRow = row;
                      col--;
                  } else {
```

```
                    row++;
                }
            }
            return ansRow;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m+n) | Space Complexity: O(1)

76. Search in a 2D Matrix

Understanding the Problem NEW

Search for a target in a matrix where each row is sorted, and the first element of each row is greater than the last element of the previous row — meaning the whole matrix, read row by row, is really just one big sorted sequence in disguise.

Brute Force Approach NEW

Scan every cell individually — O(rows × cols), completely ignoring that the matrix is secretly one sorted array.

Why We Optimize NEW

Because reading the matrix row-by-row produces a fully sorted sequence, you can treat the 2D matrix as a 1D sorted array of length rows×cols using simple index arithmetic (`row = idx / cols` , `col = idx % cols`) and just run ordinary binary search over that virtual array. This collapses a 2D search into the exact same O(log(rows×cols)) binary search from problem 48, just with an extra index-mapping step.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Search target in matrix where each row is sorted and first element of each row > last of previous.

Visual Diagram

```
Matrix:
       [1, 3, 5, 7]
       [10, 11, 16, 20]
       [23, 30, 34, 60]

       Target: 16

       Flattened: [1,3,5,7,10,11,16,20,23,30,34,60]

       Binary Search on flattened:
       low=0, high=11, mid=5 → arr[5]=11 < 16
       low=6, high=11, mid=8 → arr[8]=23 > 16
```

```
        low=6, high=7, mid=6 → arr[6]=16 == 16 ✓

        Map index to matrix:
        index=6 → row=6/4=1, col=6%4=2
        matrix[1][2] = 16 ✓

        Result: True
```

Solution Approach

Concept: Treat as 1D array with mapping.

Algorithm

```
Algorithm SEARCH_2D_MATRIX(matrix, target):
            m = rows, n = cols
            low = 0, high = m*n - 1

            While low <= high:
                mid = (low + high) / 2
                row = mid / n, col = mid % n
                val = matrix[row][col]

                If val == target:
                    Return true
                Else If val < target:
                    low = mid + 1
                Else:
                    high = mid - 1
            Return false
```

Java Implementation

```
class Solution {
            public boolean searchMatrix(int[][] matrix, int target) {
                int m = matrix.length, n = matrix[0].length;
                int low = 0, high = m * n - 1;

                  while (low <= high) {
                      int mid = low + (high - low) / 2;
                      int row = mid / n;

                   int col = mid % n;

                   if (matrix[row][col] == target) {
                       return true;
                   } else if (matrix[row][col] < target) {
                       low = mid + 1;
                   } else {
                       high = mid - 1;
```

```
                }
            }
            return false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log(m*n)) | Space Complexity: O(1)

77. Search in a Row and Column Wise Sorted Matrix

Understanding the Problem NEW

Search for a target in a matrix where each row is sorted left-to-right and each column is sorted top-to-bottom — but unlike problem 76, rows aren't globally ordered relative to each other, so you can't flatten it into one sorted array.

Brute Force Approach NEW

Scan every cell — O(rows × cols), or run a binary search independently on every row — O(rows × log cols), both ignoring the column-sorted structure.

Why We Optimize NEW

Start from the **top-right corner**: if the current value is bigger than the target, the whole column below it must also be too big (column is sorted), so move left; if it's smaller, the whole row to its left is too small, so move down. Each step eliminates either a full row or a full column, so you reach the answer (or run out of matrix) in at most `rows + cols` steps — O(rows + cols), better than the row-by-row binary search approach, by exploiting *both* sorted directions at once instead of just one.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Search in matrix sorted row and column wise.

Visual Diagram

```
Matrix:
       [1, 4, 7, 11, 15]
       [2, 5, 8, 12, 19]
       [3, 6, 9, 16, 22]
       [10, 13, 14, 17, 24]
       [18, 21, 23, 26, 30]

       Target: 14

       Start from top-right: matrix[0][4] = 15
       15 > 14 → move left
       [1,4,7,11,15] → 11 < 14 → move down
```

```
[2,5,8,12,19] → 12 < 14 → move down
[3,6,9,16,22] → 16 > 14 → move left
[3,6,9,16,22] → 9 < 14 → move down
[10,13,14,17,24] → 14 == 14 ✓

Path: (0,4) → (0,3) → (1,3) → (2,3) → (2,2) → (3,2)
Result: True
```

Solution Approach

Concept: Start from top-right and move.

Algorithm

```
Algorithm SEARCH_ROW_COL_SORTED(matrix, target):
           row = 0, col = n-1

           While row < m and col >= 0:
               If matrix[row][col] == target:
                   Return true
               Else If matrix[row][col] > target:
                   col--
               Else:
                   row++
           Return false
```

Java Implementation

```
class Solution {
           public boolean searchMatrix(int[][] matrix, int target) {
               int row = 0, col = matrix[0].length - 1;

                 while (row < matrix.length && col >= 0) {
                     if (matrix[row][col] == target) {
                         return true;
                     } else if (matrix[row][col] > target) {
                         col--;
                     } else {
                         row++;
                     }
                 }
                 return false;
           }

      }
```

Time & Space Complexity

Time Complexity: O(m+n) | Space Complexity: O(1)

78. Find Peak Element in 2D Matrix

Understanding the Problem NEW

Find a 'peak' element in a 2D matrix — one that's greater than or equal to all four of its direct neighbours (up, down, left, right).

Brute Force Approach NEW

Scan every cell and check its four neighbours — O(rows × cols), correct but doesn't exploit any structure.

Why We Optimize NEW

You can binary search on **columns** instead of scanning every cell: pick the middle column, find the maximum element in that column (O(rows)), and compare it to its left and right neighbours. If it's bigger than both, it's a 2D peak — done. If a neighbour to one side is bigger, a peak is guaranteed to exist somewhere in that direction (following the same 'still rising, so a peak must exist ahead' logic as the 1D peak problem), so recurse into that half of the columns. This turns an O(rows×cols) full scan into O(rows × log(cols)) — binary searching over columns while doing a linear scan within each candidate column.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find a peak element in a 2D matrix (greater than neighbors).

Visual Diagram

```
Matrix:
       [1, 2, 3, 4]
       [5, 6, 7, 8]
       [9, 10, 11, 12]
       [13, 14, 15, 16]

       Peaks: 16 at (3,3) is a peak

       Find maximum in middle column:
       mid = 1 → column 1: [2,6,10,14] → max=14 at row 3
       Check neighbors: left=13, right=15
       14 < 15 → peak on right side
```

```
       low = mid + 1 = 2

       mid = 2 → column 2: [3,7,11,15] → max=15 at row 3
       Check neighbors: left=14, right=16
       15 < 16 → peak on right side
       low = 3

       mid = 3 → column 3: [4,8,12,16] → max=16 at row 3
       Check neighbors: left=15, right=-∞
       16 > 15 ✓ → Peak at (3,3)

       Result: (3,3)
```

Solution Approach

Concept: Binary search on columns.

Algorithm

```
Algorithm PEAK_2D(matrix):
           m = rows, n = cols
           low = 0, high = n-1

           While low <= high:
               mid = (low + high) / 2
               row = findMaxInColumn(mid)

                 left = mid == 0 ? -∞ : matrix[row][mid-1]
                 right = mid == n-1 ? -∞ : matrix[row][mid+1]

               If matrix[row][mid] > left and matrix[row][mid] > right:
                   Return (row, mid)
               Else If left > matrix[row][mid]:
                   high = mid - 1
               Else:
                   low = mid + 1
           Return (-1, -1)
```

Java Implementation

```
class Solution {
           public int[] findPeakGrid(int[][] matrix) {
               int m = matrix.length, n = matrix[0].length;

              int low = 0, high = n - 1;

              while (low <= high) {
                  int mid = low + (high - low) / 2;
                  int row = findMaxRow(matrix, mid);
```

```
                int left = mid == 0 ? -1 : matrix[row][mid - 1];
                int right = mid == n - 1 ? -1 : matrix[row][mid + 1];

                if (matrix[row][mid] > left && matrix[row][mid] > right) {
                    return new int[]{row, mid};
                } else if (left > matrix[row][mid]) {
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            }
            return new int[]{-1, -1};
        }

        private int findMaxRow(int[][] matrix, int col) {
            int maxRow = 0;
            for (int i = 1; i < matrix.length; i++) {
                if (matrix[i][col] > matrix[maxRow][col]) {
                    maxRow = i;
                }
            }
            return maxRow;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m log n) | Space Complexity: O(1)

79. Matrix Median

Understanding the Problem NEW

Find the median of all elements in a matrix where each row is individually sorted (rows aren't necessarily related to each other, unlike problem 76).

Brute Force Approach NEW

Flatten every row into one big list, sort the whole thing, and read off the middle — O(rows×cols × log(rows×cols)) for the sort.

Why We Optimize NEW

This reuses the 'binary search on the answer' idea, but over the *range of possible values* in the matrix rather than positions: guess a candidate median value, and use each row's sortedness to count in O(log cols) how many elements in that row are ≤ the candidate (via a binary search per row, so O(rows × log cols) total per candidate). If the total count is less than half the elements, the true median must be higher, otherwise lower — binary search over the value range (not the index range) using this counting check, giving roughly O(rows × log(cols) × log(valueRange)) — much better than fully sorting every element.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find median of a row-wise sorted matrix.

Visual Diagram

```
Matrix:
       [1, 3, 5]
       [2, 6, 9]
       [3, 6, 9]

       Flattened: [1,2,3,3,5,6,6,9,9]
       Median = 5 (5th element)

       Binary Search:
       low=1, high=9
       mid=5: Count elements <= 5
       Row 0: 1,3,5 → 3
```

```
        Row 1: 2 → 1
        Row 2: 3 → 1
        Total: 5 <= floor(9/2)=4? No, 5 > 4
        ans=5, high=4

        mid=3: Count <= 3
        Row 0: 1,3 → 2
        Row 1: 2 → 1
        Row 2: 3 → 1
        Total: 4 <= 4 ✓
        ans=3, low=4

        mid=4: Count <= 4
        Row 0: 1,3 → 2
        Row 1: 2 → 1
        Row 2: 3 → 1
        Total: 4 <= 4 ✓
        ans=4, low=5

        Result: 5
```

Solution Approach

Concept: Binary search on value range.

Algorithm

```
Algorithm MATRIX_MEDIAN(matrix):
           m = rows, n = cols
           low = min(matrix), high = max(matrix)
           required = (m*n) / 2

           While low <= high:
               mid = (low + high) / 2
               count = 0

                 For each row in matrix:
                     count += upper_bound(row, mid)

                 If count <= required:
                     low = mid + 1
                 Else:
                     high = mid - 1

           Return low
```

Java Implementation

```
class Solution {
           public int matrixMedian(int[][] matrix) {
```

```
            int m = matrix.length, n = matrix[0].length;
            int low = Integer.MAX_VALUE, high = Integer.MIN_VALUE;

            for (int i = 0; i < m; i++) {
                low = Math.min(low, matrix[i][0]);
                high = Math.max(high, matrix[i][n-1]);
            }

            int required = (m * n) / 2;

            while (low <= high) {
                int mid = low + (high - low) / 2;
                int count = 0;

                for (int i = 0; i < m; i++) {
                    count += upperBound(matrix[i], mid);
                }

                if (count <= required) {
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
            return low;
        }

        private int upperBound(int[] row, int target) {
            int low = 0, high = row.length;

            while (low < high) {
                int mid = low + (high - low) / 2;
                if (row[mid] <= target) {
                    low = mid + 1;
                } else {
                    high = mid;
                }
            }
            return low;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m log n log range) | Space Complexity: O(1)

Binary Search Summary Table

Problem	Type	Time	Space	Key Technique
Binary Search	1D	O(log n)	O(1)	Simple BS
Lower Bound	1D	O(log n)	O(1)	First >= target
Upper Bound	1D	O(log n)	O(1)	First > target
Search Insert	1D	O(log n)	O(1)	Lower bound
Floor/Ceil	1D	O(log n)	O(1)	Two separate BS
First/Last Occurrence	1D	O(log n)	O(1)	Modified BS
Count Occurrences	1D	O(log n)	O(1)	First + Last
Search Rotated I	1D	O(log n)	O(1)	Identify sorted half
Search Rotated II	1D	O(log n) avg, O(n) worst	O(1)	Handle duplicates
Min in Rotated	1D	O(log n)	O(1)	Compare mid with high
Count Rotations	1D	O(log n)	O(1)	Index of min
Single Element	1D	O(log n)	O(1)	Pair pattern
Peak Element	1D	O(log n)	O(1)	Compare mid with mid+1
Square Root	Answer	O(log n)	O(1)	BS on answer
Nth Root	Answer	O(log n * n)	O(1)	BS on answer
Koko Eating	Answer	O(n log max)	O(1)	BS on speed
Make Bouquets	Answer	O(n log max)	O(1)	BS on days
Smallest Divisor	Answer	O(n log max)	O(1)	BS on divisor
Ship Capacity	Answer	O(n log sum)	O(1)	BS on capacity
Kth Missing	Answer	O(log n)	O(1)	Missing count
Aggressive Cows	Answer	O(n log range)	O(1)	BS on distance
Book Allocation	Answer	O(n log sum)	O(1)	BS on pages
Split Array	Answer	O(n log sum)	O(1)	BS on max sum
Painter's Partition	Answer	O(n log sum)	O(1)	BS on time
Gas Station	Answer	O(n log precision)	O(1)	BS on distance
Median 2 Arrays	2 Arrays	O(log min(n,m))	O(1)	Partition method

Kth 2 Arrays	2 Arrays	O(log min(n,m))	O(1)	Partition method
Max Ones Row	2D	O(m+n)	O(1)	Top-right traversal
Search 2D Matrix	2D	O(log(m*n))	O(1)	Flattened view
Search Row/Col Sorted	2D	O(m+n)	O(1)	Top-right traversal
Peak 2D	2D	O(m log n)	O(1)	Column BS
Matrix Median	2D	O(m log n log range)	O(1)	BS on value

Strings — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

80. Remove Outermost Parentheses

Understanding the Problem NEW

A 'primitive' valid parentheses string is one that doesn't split into smaller valid pieces on its own (like `(()())` or `()`). You're given a string made of several primitive pieces glued together, e.g. `(()())(())`, which splits into two primitive pieces: `(()())` and `(())`. Removing the outermost bracket from each piece leaves `()()` and `()`, so the final answer is `()()()`. The point: every primitive chunk has exactly one outermost opening and closing bracket that should be dropped, keeping everything inside untouched.

Brute Force Approach NEW

A beginner might try to first locate every primitive substring explicitly (by matching each opening bracket with its exact partner using a stack), slice out the substring, strip its first and last characters, and concatenate the results — correct, but more bookkeeping than necessary.

Why We Optimize NEW

You don't need to find and slice substrings at all — you just need to track your current **nesting depth** with a single counter as you scan character by character. Any `(` is kept only if depth is currently > 0 *before* incrementing (i.e. it's not an outermost opening bracket), and any `)` is kept only if depth > 1 *before* decrementing. This lets you build the answer in a single O(n) left-to-right pass with O(1) extra bookkeeping (just an integer counter), instead of explicitly identifying and slicing substrings.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Remove the outermost parentheses from every primitive valid parentheses string.

Visual Diagram

```
Input: "(()())(())"
       Primitive decompositions:
       (()()) → ( ) removed → ()()
       (())   → ( ) removed → ()
       Result: "()()()"
```

```
Step-by-step:
String: ( ( ) ( ) ) ( ( ) )
Index: 0 1 2 3 4 5 6 7 8 9

Initialize count=0, result=""
i=0: '(' → count=1 (skip first)
i=1: '(' → count=2, add '('
i=2: ')' → count=1, add ')'
i=3: '(' → count=2, add '('
i=4: ')' → count=1, add ')'
i=5: ')' → count=0 (skip last)
i=6: '(' → count=1 (skip first)
i=7: '(' → count=2, add '('
i=8: ')' → count=1, add ')'
i=9: ')' → count=0 (skip last)

Result: "()()()"
```

Solution Approach

Concept: Use counter to track depth, skip outermost parentheses.

Algorithm

```
Algorithm REMOVE_OUTER_PARENS(s):
           result = ""
           count = 0

           For each char c in s:
               If c == '(':
                    If count > 0: result += c

                   count++
              Else: // c == ')'
                   count--
                   If count > 0: result += c

          Return result
```

Java Implementation

```
class Solution {
          public String removeOuterParentheses(String s) {
              StringBuilder result = new StringBuilder();
              int count = 0;

              for (char c : s.toCharArray()) {
                  if (c == '(') {
                      if (count > 0) {
```

```
                        result.append(c);
                    }
                    count++;
                } else { // c == ')'
                    count--;
                    if (count > 0) {
                        result.append(c);
                    }
                }
            }
            return result.toString();
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

81. Reverse Words in a Given String

Understanding the Problem NEW

Given a sentence, reverse the order of the words (not the letters within each word) while collapsing any extra spaces. E.g. `"the sky is blue"` → `"blue is sky the"`.

Brute Force Approach NEW

Manually scanning character by character to identify word boundaries and rebuild the reversed order in-place is fiddly and error-prone, especially handling multiple spaces, leading/trailing spaces, etc.

Why We Optimize NEW

The clean approach: split the string into a list of words (which naturally handles multiple spaces if you use a split that collapses whitespace), reverse that list, and join it back together with single spaces. This isn't really a 'brute vs optimal complexity' story so much as 'use the right built-in operations' — splitting and reversing a list of words is O(n) overall and far less error-prone than manual in-place character shuffling.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse the order of words in a string.

Visual Diagram

```
Input: "the sky is blue"
       Output: "blue is sky the"

       Step-by-step:
       1. Split into words: ["the", "sky", "is", "blue"]
       2. Reverse array: ["blue", "is", "sky", "the"]
       3. Join with spaces: "blue is sky the"

       Input: " hello world     "
       Output: "world hello"

       Step-by-step:
```

```
1. Trim spaces and split: ["hello", "world"]
2. Reverse: ["world", "hello"]
3. Join: "world hello"
```

Solution Approach

Concept: Split by spaces, reverse, and join.

Algorithm

```
Algorithm REVERSE_WORDS(s):
           words = split(s.trim())
           reverse(words)
           Return join(words, " ")
```

Java Implementation

```
class Solution {
           public String reverseWords(String s) {
               // Split by spaces and remove empty strings
               String[] words = s.trim().split("\\s+");

                 // Reverse words
                 StringBuilder result = new StringBuilder();
                 for (int i = words.length - 1; i >= 0; i--) {
                     result.append(words[i]);
                     if (i > 0) {
                         result.append(" ");
                     }
                 }
                 return result.toString();
           }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

82. Largest Odd Number in a String

Understanding the Problem NEW

Given a string of digits, find the largest possible number formed by taking some prefix of the string, with the requirement that the resulting number is odd. E.g. `"52"` → the only odd-ending prefix option is none... consider `"35427"` → largest odd prefix is `"35427"` itself since it ends in 7 (odd); but `"4206"` has no odd digit at all so the answer is empty.

Brute Force Approach NEW

A beginner might generate every possible prefix of the string, convert each to a number, check if it's odd, and track the largest — expensive if done via actual numeric conversion (numbers can be huge) and unnecessary work.

Why We Optimize NEW

The 'largest' prefix is always the *longest* valid one (since more digits at the front always makes a bigger number, assuming no leading-zero edge cases to worry about here). So you just need to find the **rightmost digit that is odd**, scanning from the end of the string backward, and take everything up to and including that digit. One O(n) backward scan, no numeric conversion or prefix enumeration needed.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the largest odd number in a string.

Visual Diagram

```
Input: "35427"
       Output: "35427" (all digits form odd number)

       Check from right:
       '7' is odd → entire string "35427" is odd
       Result: "35427"

       Input: "4206"
       Output: "" (no odd number)
```

```
Check from right:
'6' even, '0' even, '2' even, '4' even
No odd found → return ""

Input: "52"
Output: "5"

Check from right:
'2' even
'5' odd → substring "5"
Result: "5"
```

Solution Approach

Concept: Find rightmost odd digit.

Algorithm

```
Algorithm LARGEST_ODD(s):
           For i = s.length-1 down to 0:
               If (s[i] - '0') % 2 == 1:
                   Return s.substring(0, i+1)
           Return ""
```

Java Implementation

```
class Solution {
           public String largestOddNumber(String num) {
               for (int i = num.length() - 1; i >= 0; i--) {
                   if ((num.charAt(i) - '0') % 2 == 1) {
                        return num.substring(0, i + 1);
                   }
               }
               return "";
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

83. Longest Common Prefix

Understanding the Problem NEW

Given an array of strings, find the longest string that is a prefix of *every* string in the array. E.g. `["flower","flow","flight"]` → common prefix is `"fl"`.

Brute Force Approach NEW

Compare every string against every other string letter by letter to find matches — this can be done but is often written inefficiently, e.g. re-comparing pairs of strings repeatedly rather than tracking one shrinking answer.

Why We Optimize NEW

A clean way to think about it: start by assuming the *first* string entirely is the common prefix, then walk through the **rest** of the strings one at a time, shrinking your candidate prefix down every time it doesn't match the current string, until either the candidate becomes empty (no common prefix) or you've checked every string. This visits each character a bounded number of times overall, giving roughly O(sum of string lengths) — you're incrementally narrowing one answer rather than comparing every pair separately.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the longest common prefix among strings.

Visual Diagram

```
Input: ["flower", "flow", "flight"]
       Output: "fl"

       Step 1: Compare "flower" and "flow"
       Common: "flow"
       Step 2: Compare "flow" and "flight"
       Common: "fl"
       Result: "fl"

       Input: ["dog", "racecar", "car"]
       Output: ""
```

```
       Step 1: Compare "dog" and "racecar"
       Common: ""
       Result: ""

       Horizontal Scanning:
       Take first word "flower"
       Compare with "flow": common "flow"
       Compare with "flight": common "fl"
       Return "fl"
```

Solution Approach

Concept: Compare first string with all others.

Algorithm

```
Algorithm LONGEST_COMMON_PREFIX(strs):
           If empty: Return ""
           prefix = strs[0]
           For i = 1 to n-1:
               While strs[i].indexOf(prefix) != 0:
                   prefix = prefix.substring(0, prefix.length-1)
                   If prefix == "": Return ""
           Return prefix
```

Java Implementation

```
class Solution {
           public String longestCommonPrefix(String[] strs) {
               if (strs.length == 0) return "";

                 String prefix = strs[0];

                 for (int i = 1; i < strs.length; i++) {
                     while (strs[i].indexOf(prefix) != 0) {
                         prefix = prefix.substring(0, prefix.length() - 1);
                         if (prefix.isEmpty()) return "";
                     }
                 }
                 return prefix;
            }
       }
```

Time & Space Complexity

Time Complexity: O(n * m) | Space Complexity: O(1)

84. Isomorphic String

Understanding the Problem NEW

Two strings are 'isomorphic' if you can consistently replace every character in one string with a corresponding character in the other, such that the pattern of repetition matches exactly. E.g. `"egg"` and `"add"` are isomorphic (e→a, g→d) but `"foo"` and `"bar"` are not.

Brute Force Approach NEW

A beginner might try building a mapping from the first string's characters to the second's, but forget to check the mapping is **consistent in both directions** — e.g. two different characters in string 1 both mapping to the same character in string 2 would wrongly be accepted without a check.

Why We Optimize NEW

You need **two** hashmaps, not one: one mapping characters from string 1 → string 2, and another mapping string 2 → string 1. At each position, you check that both mappings agree with what's already been recorded (or create a new mapping entry if this character pair hasn't been seen yet). This bidirectional check is what correctly enforces a strict one-to-one correspondence, still in a single O(n) pass with O(1) extra space (since there are a bounded number of possible characters).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if two strings are isomorphic (characters can be mapped one-to-one).

Visual Diagram

```
Input: "egg", "add"
       Output: true

       Mapping: e→a, g→d
       "egg" → "add" ✓

       Input: "foo", "bar"
       Output: false

       Mapping: f→b, o→a, but o→r (conflict)
```

```
        "foo" → "bar" ✗

        Input: "paper", "title"
        Output: true

        Mapping: p→t, a→i, p→t (consistent), e→l, r→e
        "paper" → "title" ✓

        Two maps approach:
        Map1: char in s → char in t
        Map2: char in t → char in s

        For "egg", "add":
        e→a, a→e
        g→d, d→g
        Both maps consistent ✓
```

Solution Approach

Concept: Use two maps for bidirectional mapping.

Algorithm

```
Algorithm IS_ISOMORPHIC(s, t):
            If length different: Return false
            map_s = {}, map_t = {}
            For i = 0 to n-1:
                char_s = s[i], char_t = t[i]
                If map_s contains char_s:
                    If map_s[char_s] != char_t: Return false
                Else:
                    If map_t contains char_t: Return false
                    map_s[char_s] = char_t
                    map_t[char_t] = char_s
            Return true
```

Java Implementation

```
class Solution {
            public boolean isIsomorphic(String s, String t) {
                if (s.length() != t.length()) return false;

                  Map<Character, Character> mapS = new HashMap<>();
                  Map<Character, Character> mapT = new HashMap<>();
                  for (int i = 0; i < s.length(); i++) {

                    char c1 = s.charAt(i);
                    char c2 = t.charAt(i);

                    if (mapS.containsKey(c1)) {
```

```
                    if (mapS.get(c1) != c2) return false;
                } else {
                    if (mapT.containsKey(c2)) return false;
                    mapS.put(c1, c2);
                    mapT.put(c2, c1);
                }
            }
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1) (max 256 characters)

85. Check Whether One String is a Rotation of Another

Understanding the Problem NEW

Check whether string B can be obtained by rotating string A some number of positions. E.g. `"waterbottle"` rotated becomes things like `"erbottlewat"` — is a given target a valid rotation?

Brute Force Approach NEW

A beginner might literally generate every possible rotation of A (shifting one character at a time and rebuilding the string) and compare each one to B — O(n) rotations, each requiring an O(n) string comparison, giving $O(n^2)$ total.

Why We Optimize NEW

Here's a neat trick: if you concatenate A with itself (`A+A`), every possible rotation of A appears somewhere inside that doubled string as a contiguous substring. So the whole problem reduces to a single substring-search: is B a substring of `A+A` ? Using an efficient substring search algorithm, this check can be done in roughly O(n) time instead of generating and comparing every rotation individually — the doubling trick avoids explicit rotation entirely.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if one string is a rotation of another.

Visual Diagram

```
Input: s1 = "abcde", s2 = "cdeab"
       Output: true

       s1 = "abcde"
       Possible rotations:
       "abcde"
       "bcdea"
       "cdeab" ✓
       "deabc"
       "eabcd"
```

```
Approach: Check if s2 is substring of s1+s1
s1 + s1 = "abcdeabcde"
"cdeab" is substring ✓

Input: s1 = "abcde", s2 = "abced"
Output: false

s1 + s1 = "abcdeabcde"
"abced" not substring ✗
```

Solution Approach

Concept: Check if s2 is substring of s1+s1.

Algorithm

```
Algorithm IS_ROTATION(s1, s2):
          If length different: Return false
          Return (s1 + s1).contains(s2)
```

Java Implementation

```
class Solution {
          public boolean isRotation(String s1, String s2) {
              if (s1.length() != s2.length()) return false;
              return (s1 + s1).contains(s2);
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) (contains uses KMP internally) | Space Complexity: O(n)

86. Check if Two Strings are Anagrams

Understanding the Problem NEW

Check whether two strings are anagrams of each other — meaning they contain exactly the same characters with the same frequency, just possibly in a different order. E.g. `"listen"` and `"silent"`.

Brute Force Approach NEW

A beginner approach: sort both strings alphabetically and check if the sorted versions are equal. This works, but sorting costs O(n log n) when something cheaper is available.

Why We Optimize NEW

Since you only care about *frequency*, not order, you can build a **frequency count** (an array of size 26 for lowercase letters, or a hashmap in general) for each string in O(n), then compare the two frequency counts directly in O(26) or O(alphabet size). This avoids sorting entirely, dropping the total from O(n log n) down to O(n) — trading 'sort and compare' for 'count and compare,' which is cheaper because comparing fixed-size frequency tables is much faster than sorting variable-length strings.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if two strings are anagrams of each other.

Visual Diagram

```
Input: s = "anagram", t = "nagaram"
       Output: true

       Frequency count:
       a: 3 (s) → 3 (t)
       n: 1 → 1
       g: 1 → 1
       r: 1 → 1
       m: 1 → 1
       All match ✓
```

```
Input: s = "rat", t = "car"
Output: false

Frequency count:
r: 1 (s) → 0 (t) ✗
a: 1 → 1
t: 1 → 1
r mismatch ✗
```

Solution Approach

Concept: Use character frequency array.

Algorithm

```
Algorithm IS_ANAGRAM(s, t):
           If length different: Return false
           freq = array of size 26

           For each char in s: freq[char-'a']++
           For each char in t: freq[char-'a']--

           For each count in freq:
               If count != 0: Return false
           Return true
```

Java Implementation

```
class Solution {
           public boolean isAnagram(String s, String t) {
               if (s.length() != t.length()) return false;

                 int[] freq = new int[26];

                 for (char c : s.toCharArray()) {
                     freq[c - 'a']++;
                 }

                 for (char c : t.toCharArray()) {
                     freq[c - 'a']--;
                     if (freq[c - 'a'] < 0) return false;
                 }

                 return true;

           }
     }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

87. Sort Characters by Frequency

Understanding the Problem NEW

Given a string, rearrange its characters so they appear in order of decreasing frequency — the most common character first, repeated as many times as it appears, then the next most common, and so on. E.g. `"tree"` → `"eert"` or `"eetr"`.

Brute Force Approach NEW

A beginner might repeatedly scan the whole remaining string to find the currently most frequent character, append it, and remove it — repeating this scan-and-remove process character by character, which is O(n²) since each scan costs O(n) and you do it up to n times.

Why We Optimize NEW

Instead, count all character frequencies once in O(n) using a hashmap, then **sort the distinct characters** by their frequency (descending) — O(k log k) where k is the number of distinct characters (at most 26 for lowercase letters, so effectively constant). Finally, build the output string by repeating each character according to its count. This turns repeated O(n) rescans into a single count pass plus one small sort, giving roughly O(n + k log k) instead of O(n²).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Sort characters by frequency in descending order.

Visual Diagram

```
Input: "tree"
       Output: "eert" or "eetr"

       Frequency:
       'e': 2
       't': 1
       'r': 1
       Sort by frequency: e(2), t(1), r(1)

       Input: "cccaaa"
```

```
Output: "cccaaa" or "aaaccc"

Frequency:
'c': 3
'a': 3
Both frequency 3 → either order

Input: "Aabb"
Output: "bbAa" or "bbaA"

Frequency:
'b': 2
'A': 1
'a': 1
Sort: bb then Aa
```

Solution Approach

Concept: Count frequencies and sort.

Algorithm

```
Algorithm SORT_BY_FREQUENCY(s):
           freq = array of size 256
           For each char in s: freq[char]++

           Sort characters by frequency desc
           Build result by repeating characters
           Return result
```

Java Implementation

```
class Solution {
           public String frequencySort(String s) {
               Map<Character, Integer> freq = new HashMap<>();
               for (char c : s.toCharArray()) {
                   freq.put(c, freq.getOrDefault(c, 0) + 1);
               }

                 // Sort by frequency
                 List<Character> chars = new ArrayList<>(freq.keySet());
                 chars.sort((a, b) -> freq.get(b) - freq.get(a));

               StringBuilder result = new StringBuilder();
               for (char c : chars) {
                   int count = freq.get(c);
                   for (int i = 0; i < count; i++) {
                       result.append(c);
                   }
               }
```

```
            return result.toString();
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

88. Maximum Nesting Depth of Parentheses

Understanding the Problem NEW

Given a string of matched parentheses (possibly with other characters), find the maximum nesting depth — how many levels of parentheses are open at the deepest point. E.g. `"(1+(2*3)+((8)/4))"` has max depth 3.

Brute Force Approach NEW

Without a plan, a beginner might try to explicitly identify matching bracket pairs first (e.g. using a stack to record positions) and then compute nesting levels from those pairs — more machinery than the problem actually needs.

Why We Optimize NEW

You don't need to match pairs at all — just keep a single running counter: increment it on every `(`, decrement on every `)`, and track the maximum value the counter ever reaches. The maximum value the counter hits *is* the maximum nesting depth, computed in a single O(n) left-to-right pass with O(1) space — no stack of positions or explicit pair-matching required, since you only care about the depth number, not which brackets matched which.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the maximum nesting depth of parentheses in a string.

Visual Diagram

```
Input: "(1+(2*3)+((8)/4))+1"
       Output: 3

       Trace:
       String: ( 1 + ( 2 * 3 ) + ( ( 8 ) / 4 ) ) + 1
       Depth: 1 1 2 2 2 2 1 2 3 3 2 2 1 1 1 0

       Maximum depth = 3

       Input: "(1)+((2))+(((3)))"
       Output: 3
```

```
Trace:
(1)    → depth 1
((2)) → depth 2
(((3))) → depth 3
Maximum = 3
```

Solution Approach

Concept: Track current depth with a counter.

Algorithm

```
Algorithm MAX_DEPTH(s):
           max_depth = 0
           current_depth = 0

           For each char in s:
               If char == '(':
                   current_depth++
                   max_depth = max(max_depth, current_depth)
               Else If char == ')':
                   current_depth--

           Return max_depth
```

Java Implementation

```
class Solution {
           public int maxDepth(String s) {
               int maxDepth = 0;
               int currentDepth = 0;

                 for (char c : s.toCharArray()) {
                     if (c == '(') {
                         currentDepth++;
                         maxDepth = Math.max(maxDepth, currentDepth);
                     } else if (c == ')') {
                         currentDepth--;
                     }
                 }
                 return maxDepth;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

89. Roman Number to Integer and Vice Versa

Understanding the Problem NEW

Convert between Roman numerals (like `"MCMXCIV"`) and integers (1994), in both directions.

Brute Force Approach NEW

For integer-to-Roman, a beginner might try to handle every possible digit combination with a giant set of if/else conditions per digit position — workable but repetitive and error-prone. For Roman-to-integer, a common mistake is to just add up each symbol's value without accounting for subtractive cases like `IV` (4, not 6).

Why We Optimize NEW

Both directions have a clean **greedy** solution. For integer→Roman: keep a table of value/symbol pairs sorted from largest to smallest (including subtractive combinations like 'CM'=900, 'IV'=4), and repeatedly subtract the largest value that fits, appending its symbol, until the number reaches 0 — this greedily builds the correct Roman numeral without per-digit special-casing. For Roman→integer: scan left to right, and if a symbol's value is smaller than the value that comes right after it, subtract it instead of adding it (this correctly captures subtractive notation like IV, IX, XL, etc.). Both run in O(1) time in practice (Roman numerals are bounded in length) and avoid a giant hand-written case explosion.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert Roman numerals to integer and vice versa.

Visual Diagram

```
Roman to Integer:
       Input: "MCMXCIV"
       Output: 1994

       M = 1000
       CM = 900 (C=100 before M=1000)
       XC = 90 (X=10 before C=100)
```

```
IV = 4 (I=1 before V=5)
Total = 1000 + 900 + 90 + 4 = 1994

Rules:
I(1), V(5), X(10), L(50), C(100), D(500), M(1000)
Smaller before larger = subtract

Integer to Roman:
Input: 1994
Output: "MCMXCIV"

1994 = 1000 + 900 + 90 + 4
1000 → M
900 → CM
90 → XC
4 → IV
Result: MCMXCIV
```

Solution Approach

Concept: Greedy approach for both conversions.

Algorithm (Roman to Integer)

Algorithm ROMAN_TO_INT(s):

map = {'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000}

result = 0

For i = 0 to n-1:

If i+1 < n and map[s[i]] < map[s[i+1]]:

result -= map[s[i]]

Else:

result += map[s[i]]

Return result

Algorithm (Integer to Roman)

Algorithm INT_TO_ROMAN(num):

values = [1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1]

symbols = ["M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"]

result = ""

For i = 0 to 12:

While num >= values[i]:

result += symbols[i]

num -= values[i]

Return result

Java Implementation

```
class Solution {
            // Roman to Integer
            public int romanToInt(String s) {
                Map<Character, Integer> map = new HashMap<>();
                map.put('I', 1);
                map.put('V', 5);
                map.put('X', 10);
                map.put('L', 50);
                map.put('C', 100);
                map.put('D', 500);
                map.put('M', 1000);

                int result = 0;
                for (int i = 0; i < s.length(); i++) {
                    if (i + 1 < s.length() && map.get(s.charAt(i)) <
map.get(s.charAt(i + 1))) {
                        result -= map.get(s.charAt(i));
                    } else {
                        result += map.get(s.charAt(i));
                    }
                }
                return result;
            }

            // Integer to Roman
            public String intToRoman(int num) {
                int[] values = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4,
1};
                String[] symbols = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X",
"IX", "V", "IV", "

                StringBuilder result = new StringBuilder();
                for (int i = 0; i < values.length; i++) {
                    while (num >= values[i]) {
                        result.append(symbols[i]);
                        num -= values[i];
                    }
                }
                return result.toString();
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) for roman, O(1) for integer | Space Complexity: O(1)

90. Implement Atoi

Understanding the Problem NEW

Implement the classic `atoi` function: convert a string (which may have leading whitespace, an optional +/- sign, then digits, possibly followed by non-digit junk) into an integer, clamping to the valid 32-bit integer range if it overflows.

Brute Force Approach NEW

This isn't really a 'brute force vs optimized' problem — the real challenge is just being thorough about all the edge cases: leading whitespace, sign handling, stopping at the first non-digit character, and overflow/underflow clamping. A beginner's first attempt often forgets one or more of these (especially overflow, or strings that are entirely non-numeric).

Why We Optimize NEW

The reliable approach is to process the string in clearly separated stages, in order: (1) skip leading whitespace, (2) check for an optional sign, (3) consume digits one at a time while building up the number, and (4) at each digit, check *before* multiplying whether you're about to overflow the valid range, clamping and stopping early if so. Handling overflow proactively (checking before it happens) rather than after the fact (letting the number wrap around) is the key correctness detail here — this remains a simple O(n) single pass, but the 'optimization' is really about correctness robustness rather than speed.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert string to integer (atof/atoi implementation).

Visual Diagram

```
Input: "   -42"
       Output: -42

       Steps:
       1. Skip leading spaces: "   -42" → "-42"
       2. Check sign: '-' → negative
       3. Parse digits: "42" → 42
```

```
4. Apply sign: -42
5. Check range: -42 within [-2^31, 2^31-1]

Input: "4193 with words"
Output: 4193

Steps:
1. Skip spaces: "4193 with words"
2. No sign
3. Parse digits: "4193" → 4193
4. Stop at non-digit space
5. Result: 4193

Input: "words and 987"
Output: 0

Steps:
1. Skip spaces: "words and 987"
2. No digits found → return 0

Input: "-91283472332"
Output: -2147483648

Steps:
1. Parse "-91283472332"
2. Less than Integer.MIN_VALUE
3. Return Integer.MIN_VALUE
```

Solution Approach

Concept: Handle whitespace, sign, digits, and overflow.

Algorithm

```
Algorithm MY_ATOI(s):
           i = 0, n = s.length()
           sign = 1, result = 0

            // Skip spaces
            While i < n and s[i] == ' ': i++

            // Handle sign
            If i < n and (s[i] == '+' or s[i] == '-'):
                sign = s[i] == '-' ? -1 : 1
                i++

            // Parse digits
            While i < n and s[i] >= '0' and s[i] <= '9':

              digit = s[i] - '0'
              // Check overflow
```

```
            If result > (Integer.MAX_VALUE - digit) / 10:
                Return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE
            result = result * 10 + digit
            i++

        Return result * sign
```

Java Implementation

```
class Solution {
        public int myAtoi(String s) {
            int i = 0, n = s.length();
            int sign = 1;
            long result = 0;

            // Skip leading whitespace
            while (i < n && s.charAt(i) == ' ') {
                i++;
            }

            // Handle sign
            if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
                sign = s.charAt(i) == '-' ? -1 : 1;
                i++;
            }

            // Parse digits
            while (i < n && s.charAt(i) >= '0' && s.charAt(i) <= '9') {
                int digit = s.charAt(i) - '0';
                result = result * 10 + digit;

                // Check overflow
                if (result * sign > Integer.MAX_VALUE) {
                     return Integer.MAX_VALUE;
                }
                if (result * sign < Integer.MIN_VALUE) {
                     return Integer.MIN_VALUE;
                }
                i++;
            }

            return (int)(result * sign);
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

91. Count Number of Substrings

Understanding the Problem NEW

Count the number of substrings that contain at least K distinct characters. This is a variation of many substring-counting problems, using a sliding window.

Brute Force Approach NEW

Check every possible substring (all start/end pairs), count its distinct characters, and tally the ones meeting the K threshold — $O(n^2)$ or $O(n^3)$ if you recompute distinct-character counts from scratch for every substring.

Why We Optimize NEW

Use a **sliding window with a frequency map**: as you expand the window's right edge, update the count of distinct characters seen; whenever the window has at least K distinct characters, every further expansion to the right still keeps that property, so you can count all valid substrings starting from the current left edge in O(1) rather than re-scanning. By shrinking from the left when needed and reusing the frequency map incrementally (rather than recomputing it per substring), the whole count can be computed in O(n) or O(n × alphabet size), instead of re-examining every substring independently.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count substrings with at least K distinct characters.

Visual Diagram

```
Input: s = "abc", K = 2
       Output: 3

       Substrings with ≥ 2 distinct:
       "ab" (a,b)
       "abc" (a,b,c)
       "bc" (b,c)
       Total: 3

       Input: s = "abac", K = 2
       Output: 7
```

```
All substrings with ≥ 2 distinct:
"ab", "aba", "abac", "ba", "bac", "ac", "abc"
Wait, "abc" not in string... Let's list carefully:

All substrings:
a: 1 distinct ✗
ab: 2 distinct ✓
aba: 2 distinct ✓
abac: 3 distinct ✓
b: 1 ✗
ba: 2 ✓
bac: 3 ✓
a: 1 ✗
ac: 2 ✓
c: 1 ✗
Total: 7
```

Solution Approach

Concept: Sliding window with frequency map.

Algorithm

```
Algorithm COUNT_SUBSTRINGS(s, K):
           count = 0
           For i = 0 to n-1:
               freq = {}
               distinct = 0
               For j = i to n-1:
                   If freq[s[j]] == 0: distinct++
                   freq[s[j]]++
                   If distinct >= K: count++
           Return count
```

Java Implementation

```
class Solution {
           public int countSubstrings(String s, int k) {
               int count = 0;
               int n = s.length();

                 for (int i = 0; i < n; i++) {
                     int[] freq = new int[26];

                   int distinct = 0;

                   for (int j = i; j < n; j++) {
                       int index = s.charAt(j) - 'a';
                       if (freq[index] == 0) distinct++;
```

```
                    freq[index]++;

                    if (distinct >= k) count++;
                }
            }
            return count;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(1)

92. Longest Palindromic Substring

Understanding the Problem NEW

Find the longest substring of the given string that reads the same forwards and backwards (a palindrome). E.g. in `"babad"`, both `"bab"` and `"aba"` are valid answers (length 3).

Brute Force Approach NEW

The most direct approach: check every possible substring (all start/end pairs) to see if it's a palindrome — $O(n^2)$ substrings, and checking each one takes up to $O(n)$ time, giving $O(n^3)$ total.

Why We Optimize NEW

A palindrome is symmetric around its center, so instead of checking substrings from the outside in, you can pick every possible **center** (there are 2n-1 of them, since a center can be a single character for odd-length palindromes, or a gap between two characters for even-length ones) and **expand outward** from each center as long as the characters on both sides keep matching. Each expansion naturally stops as soon as symmetry breaks, and you track the longest one found. This still checks $O(n)$ centers, but each expansion only costs as much as the palindrome actually found (not a fixed $O(n)$ check regardless), bringing the total down to $O(n^2)$ instead of $O(n^3)$ — a smarter way of generating candidates rather than checking every possible substring blindly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the longest palindromic substring in a string.

Visual Diagram

```
Input: "babad"
       Output: "bab" or "aba"

       Step 1: Check all substrings
       "b" → palindrome length 1
       "bab" → palindrome length 3 ✓
       "babad" → not palindrome
```

```
"aba" → palindrome length 3 ✓

Expanding around center:
Center: index 1 ('a')
Left=0 ('b'), Right=2 ('b') → "bab" ✓
Left=-1, Right=3 → stop
Max length = 3

Center: between index 1 and 2
Left=1 ('a'), Right=2 ('b') → no match
Max length = 1

Input: "cbbd"
Output: "bb"

Center between index 1 and 2:
Left=1 ('b'), Right=2 ('b') → "bb" ✓
Left=0 ('c'), Right=3 ('d') → no match
Max length = 2
```

Solution Approach

Concept: Expand around centers (2n-1 centers).

Algorithm

```
Algorithm LONGEST_PALINDROME(s):
           n = length(s)
           start = 0, max_len = 1

            For i = 0 to n-1:
                // Odd length palindrome
                left = i, right = i
                While left >= 0 and right < n and s[left] == s[right]:
                    If right - left + 1 > max_len:
                        start = left
                        max_len = right - left + 1
                    left--; right++

                 // Even length palindrome
                 left = i, right = i+1
                 While left >= 0 and right < n and s[left] == s[right]:
                     If right - left + 1 > max_len:
                         start = left
                         max_len = right - left + 1
                     left--; right++

            Return s.substring(start, start + max_len)
```

Java Implementation

```
class Solution {
           public String longestPalindrome(String s) {
               if (s == null || s.length() < 2) return s;

               int start = 0, maxLen = 1;
               int n = s.length();

               for (int i = 0; i < n; i++) {
                   // Odd length
                   int left = i, right = i;
                   while (left >= 0 && right < n && s.charAt(left) ==
s.charAt(right)) {
                       if (right - left + 1 > maxLen) {
                           start = left;
                           maxLen = right - left + 1;
                       }
                       left--;
                       right++;
                   }

                   // Even length
                   left = i;
                   right = i + 1;
                   while (left >= 0 && right < n && s.charAt(left) ==
s.charAt(right)) {
                       if (right - left + 1 > maxLen) {
                           start = left;
                           maxLen = right - left + 1;
                       }
                       left--;
                       right++;
                   }
               }

               return s.substring(start, start + maxLen);
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(1)

93. Sum of Beauty of All Substrings

Understanding the Problem NEW

For every substring of the given string, define its 'beauty' as (frequency of its most common character) minus (frequency of its least common character); sum this beauty value over every substring in the string.

Brute Force Approach NEW

Directly following the definition: for every substring, build a frequency count of its characters (O(n) per substring in the worst case), then compute max-freq minus min-freq — with O(n²) substrings, each costing up to O(n) to analyze, giving O(n³) total.

Why We Optimize NEW

You can avoid recomputing frequencies from scratch for every substring: fix a starting index, and as you extend the substring one character at a time to the right, **incrementally update** a running frequency count instead of rebuilding it — after each single-character extension, updating the max and min frequency values only needs an O(alphabet size) scan (since there are only up to 26 lowercase letters to check), not a full recount of the substring. This turns the per-substring cost from O(n) down to O(26), reducing the overall complexity from O(n³) to O(n² × 26), i.e. effectively O(n²).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate sum of beauty (max_freq - min_freq) for all substrings.

Visual Diagram

```
Input: "aabcb"
       Output: 5

       All substrings:
       "a": 1-1=0
       "aa": 2-2=0
       "aab": 2-1=1
       "aabc": 2-1=1
```

```
"aabcb": 2-1=1
"a": 1-1=0
"ab": 1-1=0
"abc": 1-1=0
"abcb": 2-1=1
"b": 1-1=0
"bc": 1-1=0
"bcb": 2-1=1
"c": 1-1=0
"cb": 1-1=0
"b": 1-1=0
Total: 1+1+1+1+1+1 = 5
```

Solution Approach

Concept: For each substring, track min and max frequency.

Algorithm

```
Algorithm SUM_OF_BEAUTY(s):
           total = 0
           For i = 0 to n-1:
               freq = array of size 26
               For j = i to n-1:
                   freq[s[j]-'a']++
                   min_freq = INF, max_freq = -INF
                   For each count in freq:
                        If count > 0:
                            min_freq = min(min_freq, count)
                            max_freq = max(max_freq, count)
                   total += max_freq - min_freq
           Return total
```

Java Implementation

```
class Solution {
           public int beautySum(String s) {
               int total = 0;
               int n = s.length();

                 for (int i = 0; i < n; i++) {
                     int[] freq = new int[26];

                    for (int j = i; j < n; j++) {
                        freq[s.charAt(j) - 'a']++;

                         int minFreq = Integer.MAX_VALUE;

                       int maxFreq = Integer.MIN_VALUE;
```

```
                    for (int count : freq) {
                        if (count > 0) {
                            minFreq = Math.min(minFreq, count);
                            maxFreq = Math.max(maxFreq, count);
                        }
                    }

                    total += maxFreq - minFreq;
                }
            }
            return total;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(1)

94. Reverse Every Word in a String

Understanding the Problem NEW

Given a sentence, reverse the letters within each word individually, while keeping the words themselves in their original order (the opposite emphasis from problem 81, which reversed word order but not the letters). E.g. `"Let's take LeetCode contest" → "s'teL ekat edoCteeL tsetnoc"`.

Brute Force Approach NEW

A beginner might manually track index boundaries and reverse characters in-place with careful pointer arithmetic — doable, but easy to get off-by-one errors on word boundaries, especially with multiple spaces.

Why We Optimize NEW

This is cleanly handled by splitting the string into words (by spaces), reversing the *characters within* each word individually (leaving word order untouched), and joining them back together with single spaces. Contrasting this directly with problem 81 (which reverses word order but keeps letters intact) makes it easy to remember the distinction: 'reverse the list of words' vs 'reverse each word in the list' are two different operations on the same underlying split-word representation, and confusing the two is a common source of mistakes.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse each word in a string while preserving word order.

Visual Diagram

```
Input: "Let's take LeetCode contest"
       Output: "s'teL ekat edoCteeL tsetnoc"

       Step 1: Split words
       ["Let's", "take", "LeetCode", "contest"]

       Step 2: Reverse each word
       "Let's" → "s'teL"
       "take" → "ekat"
       "LeetCode" → "edoCteeL"
```

```
       "contest" → "tsetnoc"

       Step 3: Join with spaces
       "s'teL ekat edoCteeL tsetnoc"

       Input: "God Ding"
       Output: "doG gniD"

       "God" → "doG"
       "Ding" → "gniD"
       Result: "doG gniD"
```

Solution Approach

Concept: Split, reverse each, join.

Algorithm

```
Algorithm REVERSE_EACH_WORD(s):
           words = split(s, " ")
           For each word in words:
               Reverse word
           Return join(words, " ")
```

Java Implementation

```
class Solution {
           public String reverseWords(String s) {
               String[] words = s.split(" ");
               StringBuilder result = new StringBuilder();

                 for (int i = 0; i < words.length; i++) {
                     String reversed = new
StringBuilder(words[i]).reverse().toString();
                     result.append(reversed);
                     if (i < words.length - 1) {
                         result.append(" ");
                     }
                 }
                 return result.toString();
             }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

String Algorithms Summary

Problem	Type	Time	Space	Key Technique
Remove Outer Parentheses	Easy	O(n)	O(n)	Counter tracking
Reverse Words	Easy	O(n)	O(n)	Split and reverse
Largest Odd Number	Easy	O(n)	O(1)	Right-to-left scan
Longest Common Prefix	Easy	O(n*m)	O(1)	Compare first string
Isomorphic Strings	Easy	O(n)	O(1)	Two hash maps
Rotation Check	Easy	O(n)	O(n)	Check substring in s+s
Anagram Check	Easy	O(n)	O(1)	Frequency array
Sort by Frequency	Medium	O(n log n)	O(n)	Frequency + sorting
Max Nesting Depth	Medium	O(n)	O(1)	Counter tracking
Roman to/from Integer	Medium	O(n)	O(1)	Greedy mapping
Implement Atoi	Medium	O(n)	O(1)	Handle edge cases
Count Substrings	Medium	$O(n^2)$	O(1)	Sliding window
Longest Palindrome	Medium	$O(n^2)$	O(1)	Center expansion
Sum of Beauty	Medium	$O(n^2)$	O(1)	Frequency tracking
Reverse Each Word	Medium	O(n)	O(n)	Split and reverse

LinkedList — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

95. Introduction to LinkedList

Understanding the Problem NEW

A LinkedList is a completely different way of storing a sequence of values compared to an array. Instead of sitting in one contiguous block of memory (where you can jump straight to index 5 with simple arithmetic), each element ('node') is its own separate little box containing a value *and* a pointer/reference to the next box. To get to the 5th element, you must walk from the first box, through the second, third, and fourth, following the 'next' pointers each time.

Brute Force Approach NEW

There's no brute-vs-optimized story for the concept itself — this section is purely about understanding the building block. But it's worth contrasting directly with arrays: an array lets you jump to any index in O(1) (direct memory address calculation), while a linked list can only be walked one step at a time, O(n) to reach an arbitrary position.

Why We Optimize NEW

So why use a linked list at all if array access is faster? Because linked lists make **inserting and deleting** elements (especially at the front, or in the middle once you already have a reference to the right spot) an O(1) operation — you just rewire a couple of pointers, with no need to shift every subsequent element like an array requires. This is the fundamental trade-off to keep in mind for the rest of this chapter: linked lists trade away fast random access in exchange for fast insertion/deletion.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding LinkedList structure and basic operations.

Visual Diagram

```
LinkedList Node Structure:
          ■■■■■■■■■■■■■■■■■■■■
          ■ data ■ next ■      ■
          ■■■■■■■■■■■■■■■■■■■■
          ■   Node Structure ■
```

```
          ■■■■■■■■■■■■■■■■■■

          Example: [1] → [2] → [3] → null

                ↑        ↑       ↑
              Head     Node2   Node3

       Singly Linked List:
       Head → [10] → [20] → [30] → null

       Traversal:
       Start at Head, follow next pointers until null
```

Solution Approach

Concept: Node contains data and reference to next node.

Java Implementation

```
// Node class
       class Node {
           int data;
           Node next;

           Node(int data) {
               this.data = data;
               this.next = null;
           }
       }

       // LinkedList class
       class LinkedList {
           Node head;

           // Initialize empty list
           LinkedList() {
               this.head = null;
           }
       }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

96. Inserting a Node in LinkedList

Understanding the Problem NEW

Insert a new node into a linked list — at the beginning, at the end, or at a specific position in the middle.

Brute Force Approach NEW

Without linked lists, a beginner used to arrays might imagine you'd need to 'shift everything over' to make room for a new element — that's exactly the O(n) cost arrays pay for insertion.

Why We Optimize NEW

Because a linked list is just a chain of pointers, inserting anywhere only requires updating **at most two pointers**: the new node's 'next' pointer (pointing it at whatever came after the insertion point), and the previous node's 'next' pointer (pointing it at the new node instead). No shifting of existing elements is needed at all. Inserting at the very front or with a direct reference to the insertion point is O(1); inserting at an arbitrary *position* (by index) still costs O(n) just to walk there first, but the insertion itself, once you've arrived, is instant.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Insert node at beginning, end, or specific position.

Visual Diagram

```
Original: [10] → [20] → [30] → null

       Insert at beginning (5):
       Step 1: Create new node [5]
       Step 2: new.next = head
       Step 3: head = new
       Result: [5] → [10] → [20] → [30] → null

       Insert at end (40):
       Step 1: Create new node [40]
       Step 2: Traverse to end
       Step 3: last.next = new
```

```
Result: [10] → [20] → [30] → [40] → null

Insert at position 2 (value 25):
Step 1: Create new node [25]
Step 2: Traverse to position 1
Step 3: new.next = temp.next
Step 4: temp.next = new
Result: [10] → [25] → [20] → [30] → null
```

Solution Approach

Concept: Handle all insertion cases with proper pointer updates.

Algorithm

```
Algorithm INSERT_AT_BEGINNING(head, data):
           new_node = Node(data)
           new_node.next = head
           Return new_node

       Algorithm INSERT_AT_END(head, data):
           new_node = Node(data)
           If head == null: Return new_node
           current = head
           While current.next != null:
               current = current.next
           current.next = new_node
           Return head
```

Java Implementation

```
class LinkedList {
           Node head;

            // Insert at beginning
            public void insertAtBeginning(int data) {
                Node newNode = new Node(data);
                newNode.next = head;
                head = newNode;
            }

            // Insert at end
            public void insertAtEnd(int data) {

               Node newNode = new Node(data);
               if (head == null) {
                   head = newNode;
                   return;
               }
               Node current = head;
```

```
            while (current.next != null) {
                current = current.next;
            }
            current.next = newNode;
        }

        // Insert at specific position (0-based)
        public void insertAtPosition(int data, int position) {
            Node newNode = new Node(data);
            if (position == 0) {
                newNode.next = head;
                head = newNode;
                return;
            }

            Node current = head;
            for (int i = 0; i < position - 1 && current != null; i++) {
                current = current.next;
            }

            if (current == null) return;
            newNode.next = current.next;
            current.next = newNode;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) at beginning, O(n) at end | Space Complexity: O(1)

97. Deleting a Node in LinkedList

Understanding the Problem NEW

Delete a node from a linked list — from the beginning, the end, or a specific position — handling the tricky edge cases (deleting the head, deleting the only node, deleting the tail) correctly.

Brute Force Approach NEW

An array-based mental model would suggest 'shift everything after the deleted position back by one' — O(n) work, again unnecessary for a linked list.

Why We Optimize NEW

Deletion in a linked list just means **re-pointing** the previous node's 'next' to skip over the node being removed (so it points directly to what came after it) — the deleted node simply becomes unreachable and gets cleaned up. This is O(1) once you have a reference to the node just before the one you're deleting; the only O(n) cost is walking to find that position in the first place if you're given an index rather than a direct pointer.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Delete node from beginning, end, or specific position.

Visual Diagram

```
Original: [10] → [20] → [30] → [40] → null

       Delete from beginning:
       Step 1: head = head.next
       Result: [20] → [30] → [40] → null

       Delete from end:
       Step 1: Traverse to second last
       Step 2: secondLast.next = null
       Result: [10] → [20] → [30] → null

       Delete at position 2 (delete 30):
       Step 1: Traverse to position 1 (20)
```

```
       Step 2: temp.next = temp.next.next
       Result: [10] → [20] → [40] → null
```

Solution Approach

Concept: Handle edge cases and update pointers.

Algorithm

```
Algorithm DELETE_FROM_BEGINNING(head):
           If head == null: Return null
           Return head.next

       Algorithm DELETE_FROM_END(head):
           If head == null: Return null
           If head.next == null: Return null
           current = head
           While current.next.next != null:
               current = current.next
           current.next = null
           Return head
```

Java Implementation

```
class LinkedList {
           Node head;

           // Delete from beginning
           public void deleteFromBeginning() {
               if (head == null) return;
               head = head.next;
           }

           // Delete from end
           public void deleteFromEnd() {
               if (head == null) return;
               if (head.next == null) {
                   head = null;
                   return;
               }
               Node current = head;
               while (current.next.next != null) {

                   current = current.next;
               }
               current.next = null;
           }

           // Delete at specific position
           public void deleteAtPosition(int position) {
```

```
            if (head == null) return;
            if (position == 0) {
                head = head.next;
                return;
            }

            Node current = head;
            for (int i = 0; i < position - 1 && current.next != null; i++) {
                current = current.next;
            }

            if (current.next == null) return;
            current.next = current.next.next;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) at beginning, O(n) at end | Space Complexity: O(1)

98. Find the Length of the LinkedList

Understanding the Problem NEW

Count how many nodes are in the linked list.

Brute Force Approach NEW

There isn't a faster alternative here — a linked list has no built-in 'length' field by default (unlike some language's array types), so counting requires visiting every node.

Why We Optimize NEW

Since there's no way to jump ahead or skip nodes, you simply walk from the head to the end, incrementing a counter at each node, until you reach a null 'next' pointer — an unavoidable O(n) traversal. (Some linked list implementations choose to maintain a running length counter that's updated on every insert/delete, turning this into an O(1) lookup at the cost of a little bookkeeping on every mutation — a classic time-vs-maintenance trade-off worth knowing about.)

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count number of nodes in LinkedList.

Visual Diagram

```
List: [10] → [20] → [30] → [40] → null

       Traverse:
       current = head (10)
       count = 1

       current = 20
       count = 2

       current = 30
       count = 3

       current = 40
       count = 4
```

```
current = null
Result: 4

Empty List:
head = null
count = 0
Result: 0
```

Solution Approach

Concept: Traverse and count nodes.

Algorithm

```
Algorithm LENGTH(head):
           count = 0
           current = head
           While current != null:
               count++
               current = current.next
           Return count
```

Java Implementation

```
class LinkedList {
           Node head;

           public int length() {
               int count = 0;
               Node current = head;
               while (current != null) {
                   count++;
                   current = current.next;
               }
               return count;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

99. Search an Element in the LL

Understanding the Problem NEW

Check whether a given value exists anywhere in the linked list.

Brute Force Approach NEW

As with counting, there's no shortcut available purely from the data structure itself — you must inspect nodes one at a time since there's no way to 'jump' to check a specific value without visiting the nodes in between.

Why We Optimize NEW

Walk from the head, comparing each node's value to the target, and stop the moment you find a match (or reach the end without finding one) — O(n) in the worst case, same as Linear Search on an array. The lesson here (echoing problem 15 in the Arrays chapter) is that unsorted, unstructured data — whether stored as an array or a linked list — fundamentally requires an O(n) search unless you build extra structure (like a hashmap) on top of it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find if element exists in LinkedList.

Visual Diagram

```
List: [10] → [20] → [30] → [40] → null

       Search for 30:
       current = 10 → not match
       current = 20 → not match
       current = 30 → match ✓
       Return true

       Search for 25:
       current = 10 → not match
       current = 20 → not match
       current = 30 → not match
       current = 40 → not match
```

```
        current = null → end
        Return false
```

Solution Approach

Concept: Traverse and compare each node's data.

Algorithm

```
Algorithm SEARCH(head, target):
            current = head
            While current != null:
                If current.data == target:
                    Return true
                current = current.next
            Return false
```

Java Implementation

```
class LinkedList {
            Node head;

             public boolean search(int target) {
                 Node current = head;
                 while (current != null) {
                     if (current.data == target) {
                         return true;
                     }
                     current = current.next;
                 }
                 return false;
             }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

100. Introduction to DLL

Understanding the Problem NEW

A Doubly LinkedList (DLL) is like a singly linked list, but each node also keeps a pointer **back** to the previous node, not just forward to the next one. This lets you traverse in both directions.

Brute Force Approach NEW

Nothing to optimize here conceptually — it's a structural extension. Worth noting the trade-off though: a DLL uses extra memory (one more pointer per node) in exchange for new capabilities.

Why We Optimize NEW

The extra 'prev' pointer unlocks operations that are awkward or impossible efficiently in a singly linked list — like deleting a node when you're only given a reference to that node itself (without needing to first walk from the head to find its predecessor), or traversing backward from the tail. This is why several later problems in this chapter (delete, reverse, remove duplicates) become simpler or faster on a DLL than on a singly linked list.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding Doubly LinkedList structure.

Visual Diagram

```
Doubly Node Structure:
       ■■■■■■■■■■■■■■■■■■■■■■■■■■■
       ■ prev ■ data ■ next ■    ■
       ■■■■■■■■■■■■■■■■■■■■■■■■■■■
       ■    DLL Node Structure ■
       ■■■■■■■■■■■■■■■■■■■■■■■■■■■

       Example: null ← [10] ↔ [20] ↔ [30] → null
                       ↑      ↑       ↑
                     Head   Node2  Node3

       Traversal:
```

```
       Forward: Start at Head, follow next
       Backward: Start at Tail, follow prev
```

Solution Approach

Concept: Node has prev and next pointers.

Java Implementation

```
// Node class for DLL
       class DLLNode {
           int data;
           DLLNode prev;
           DLLNode next;

           DLLNode(int data) {
               this.data = data;
               this.prev = null;
               this.next = null;
           }
       }

       // DoublyLinkedList class
       class DoublyLinkedList {
           DLLNode head;
           DLLNode tail;

           DoublyLinkedList() {
               this.head = null;
               this.tail = null;
           }
       }
```

101. Insert a Node in DLL

Understanding the Problem NEW

Insert a node into a Doubly LinkedList, correctly wiring up both the 'next' pointer of the preceding node *and* the 'prev' pointer of the following node.

Brute Force Approach NEW

A common beginner mistake is to update only the 'next' pointers (as you would in a singly linked list) and forget the 'prev' pointers entirely, silently corrupting the backward-traversal capability of the list.

Why We Optimize NEW

Every insertion in a DLL requires updating **up to four** pointer connections instead of two: the new node's next and prev, plus the neighbours' pointers pointing back at the new node. Getting the order of these pointer updates right (so you don't accidentally lose a reference to a node before you've rewired everything) is the key skill here — still O(1) once you're at the right position, just with twice the bookkeeping of a singly linked list.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Insert node in Doubly LinkedList.

Visual Diagram

```
Original: null ← [10] ↔ [20] ↔ [30] → null

       Insert at beginning (5):
       Step 1: new.next = head
       Step 2: head.prev = new
       Step 3: head = new
       Step 4: head.prev = null
       Result: null ← [5] ↔ [10] ↔ [20] ↔ [30] → null

       Insert at end (40):
       Step 1: new.prev = tail
       Step 2: tail.next = new
       Step 3: tail = new
```

```
       Result: null ← [10] ↔ [20] ↔ [30] ↔ [40] → null

       Insert after node with value 20 (25):
       Step 1: Create new node [25]
       Step 2: new.next = current.next
       Step 3: new.prev = current
       Step 4: current.next.prev = new
       Step 5: current.next = new
       Result: null ← [10] ↔ [20] ↔ [25] ↔ [30] → null
```

Solution Approach

Concept: Handle prev and next pointers carefully.

Algorithm

```
Algorithm INSERT_IN_DLL(head, data, position):
           new_node = Node(data)
           If position == 0:
               new_node.next = head
               If head != null: head.prev = new_node
               Return new_node

           current = head
           For i = 0 to position-1:
               current = current.next
               If current == null: Return head

           new_node.prev = current
           new_node.next = current.next
           If current.next != null: current.next.prev = new_node
           current.next = new_node
           Return head
```

Java Implementation

```
class DoublyLinkedList {
           DLLNode head;
           DLLNode tail;

           // Insert at beginning
           public void insertAtBeginning(int data) {

               DLLNode newNode = new DLLNode(data);
               if (head == null) {
                   head = tail = newNode;
                   return;
               }
               newNode.next = head;
               head.prev = newNode;
```

```
            head = newNode;
        }

        // Insert at end
        public void insertAtEnd(int data) {
            DLLNode newNode = new DLLNode(data);
            if (tail == null) {
                head = tail = newNode;
                return;
            }
            newNode.prev = tail;
            tail.next = newNode;
            tail = newNode;
        }

        // Insert after specific node
        public void insertAfter(DLLNode prevNode, int data) {
            if (prevNode == null) return;
            DLLNode newNode = new DLLNode(data);
            newNode.next = prevNode.next;
            newNode.prev = prevNode;
            if (prevNode.next != null) {
                prevNode.next.prev = newNode;
            } else {
                tail = newNode;
            }
            prevNode.next = newNode;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) for position, O(1) at ends | Space Complexity: O(1)

102. Delete a Node in DLL

Understanding the Problem NEW

Delete a node from a Doubly LinkedList, correctly handling the head, the tail, and middle nodes.

Brute Force Approach NEW

As above, forgetting to update the 'prev' pointer of the node *after* the deleted one (not just the 'next' pointer of the node before it) is the classic mistake.

Why We Optimize NEW

Because you have direct access to both neighbours via the deleted node's own prev/next pointers, you can splice it out by connecting those two neighbours directly to each other (in both directions) — O(1), and notably, unlike a singly linked list, you don't need a separate reference to the 'previous' node walked in from the head, since the node being deleted already carries that reference itself. This is the concrete payoff of the extra memory a DLL uses.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Delete node from Doubly LinkedList.

Visual Diagram

```
Original: null ← [10] ↔ [20] ↔ [30] ↔ [40] → null

       Delete first node (10):
       Step 1: head = head.next
       Step 2: head.prev = null
       Result: null ← [20] ↔ [30] ↔ [40] → null

       Delete last node (40):
       Step 1: tail = tail.prev
       Step 2: tail.next = null
       Result: null ← [10] ↔ [20] ↔ [30] → null

       Delete node with value 20:
       Step 1: current.next.prev = current.prev
```

```
       Step 2: current.prev.next = current.next
       Result: null ← [10] ↔ [30] ↔ [40] → null
```

Solution Approach

Concept: Handle all cases including head and tail.

Algorithm

```
Algorithm DELETE_IN_DLL(head, node):
           If head == null or node == null: Return head
           If node == head: head = head.next
           If node.next != null: node.next.prev = node.prev
           If node.prev != null: node.prev.next = node.next
           Return head
```

Java Implementation

```
class DoublyLinkedList {
           DLLNode head;
           DLLNode tail;

           // Delete first node
           public void deleteFirst() {
               if (head == null) return;
               if (head == tail) {
                   head = tail = null;
                   return;
               }
               head = head.next;
               head.prev = null;
           }

           // Delete last node
           public void deleteLast() {
               if (tail == null) return;
               if (head == tail) {
                   head = tail = null;
                   return;
               }
               tail = tail.prev;

               tail.next = null;
           }

           // Delete specific node
           public void deleteNode(DLLNode node) {
               if (node == null) return;
               if (node == head) {
                   deleteFirst();
```

```
                return;
            }
            if (node == tail) {
                deleteLast();
                return;
            }
            node.prev.next = node.next;
            node.next.prev = node.prev;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) if node known, O(n) if search needed | Space Complexity:

103. Reverse a DLL

Understanding the Problem NEW

Reverse a Doubly LinkedList so that traversing it forward now visits nodes in the opposite order.

Brute Force Approach NEW

A beginner might try to build an entirely new reversed list by reading the original front-to-back and inserting new nodes at the front of a fresh list — this works but needlessly allocates a whole new set of nodes when the existing ones can simply be rewired.

Why We Optimize NEW

Since every node already has both 'next' and 'prev' pointers, reversing the whole list is just a matter of **swapping next and prev on every single node** — what used to be 'forward' becomes 'backward' and vice versa — then updating the head/tail references. This is done in-place in a single O(n) pass with no new memory allocation, exploiting the fact a DLL already has all the pointer information needed in both directions.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse the Doubly LinkedList.

Visual Diagram

```
Original: null ← [10] ↔ [20] ↔ [30] → null
       Reverse: null ← [30] ↔ [20] ↔ [10] → null

       Step-by-step:
       Step 1: current = head (10)
       Step 2: Swap prev and next of each node
               [10] prev=null, next=null → prev=20, next=null
       Step 3: Move to next node
               [20] prev=10, next=30 → prev=30, next=10
       Step 4: Continue...
       Result: null ← [30] ↔ [20] ↔ [10] → null
```

Solution Approach

Concept: Swap prev and next for each node.

Algorithm

```
Algorithm REVERSE_DLL(head):
           current = head
           temp = null

           While current != null:
               temp = current.prev
               current.prev = current.next
               current.next = temp
               current = current.prev

           If temp != null:
               head = temp.prev

           Return head
```

Java Implementation

```
class DoublyLinkedList {
           DLLNode head;

           public void reverse() {
               DLLNode current = head;
               DLLNode temp = null;

                 while (current != null) {
                     temp = current.prev;
                     current.prev = current.next;
                     current.next = temp;
                     current = current.prev;
                 }

                 if (temp != null) {
                     head = temp.prev;
                 }
           }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

104. Middle of a LinkedList (Tortoise & Hare)

Understanding the Problem NEW

Find the middle node of a singly linked list in a single pass, without first counting the total length.

Brute Force Approach NEW

The obvious two-pass approach: first traverse the whole list once just to count its length, then traverse again to the length/2 position — correct, but two full passes when one might suffice.

Why We Optimize NEW

The classic **Tortoise and Hare** trick: use two pointers starting at the head, one ('slow') moving one step at a time, the other ('fast') moving two steps at a time. By the time 'fast' reaches the end, 'slow' will be exactly at the middle — because fast covers ground twice as quickly. This finds the middle in a single O(n) pass instead of two, and this same two-speed-pointer idea reappears repeatedly later in this chapter (cycle detection, palindrome checking).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the middle node of a LinkedList.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → null

       Tortoise (slow) moves 1 step, Hare (fast) moves 2 steps:
       Step 1: slow=1, fast=1
       Step 2: slow=2, fast=3
       Step 3: slow=3, fast=5
       Step 4: slow=4, fast=null (stop)
       Result: slow=4 (middle)

       List: [1] → [2] → [3] → [4] → [5] → [6] → null
       Step 1: slow=1, fast=1
       Step 2: slow=2, fast=3
       Step 3: slow=3, fast=5
```

```
       Step 4: slow=4, fast=null (stop)
       Result: slow=4 (second middle for even length)
```

Solution Approach

Concept: Two pointers moving at different speeds.

Algorithm

```
Algorithm MIDDLE(head):
           slow = head
           fast = head

           While fast != null and fast.next != null:
               slow = slow.next
               fast = fast.next.next

           Return slow
```

Java Implementation

```
class Solution {
           public Node middleNode(Node head) {
               Node slow = head;
               Node fast = head;

                 while (fast != null && fast.next != null) {
                     slow = slow.next;
                     fast = fast.next.next;
                 }
                 return slow;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

105. Reverse a LinkedList [Iterative]

Understanding the Problem NEW

Reverse a singly linked list in-place, so the last node becomes the new head and every 'next' pointer points backward relative to the original order.

Brute Force Approach NEW

As with the DLL version, a beginner might build a brand-new list by reading the original and prepending nodes to a fresh list — correct, but allocates n new nodes when the existing ones can just be rewired.

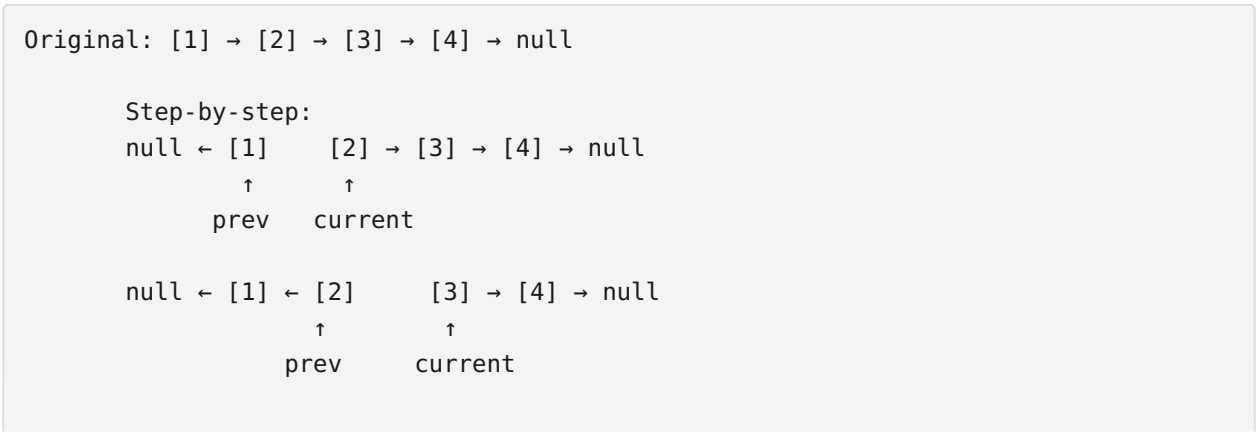
Why We Optimize NEW

Since a singly linked list only has 'next' pointers (no 'prev'), you can't just swap two fields like in the DLL case — instead, you walk through the list with **three pointers**: `prev` (initially null), `current` (starting at head), and a temporary `next` to remember where to go before you overwrite `current`'s pointer. At each step, you redirect `current.next` to point backward at `prev`, then advance all three pointers forward. This reverses the entire list in-place in a single O(n) pass with O(1) extra space — the 'three pointers' pattern here is foundational and reused in problem 122 (reverse in groups).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse LinkedList using iterative approach.

Visual Diagram



```
       null ← [1] ← [2] ← [3]       [4] → null
                          ↑          ↑
                        prev       current

       null ← [1] ← [2] ← [3] ← [4]
                                 ↑
                               head

       Result: [4] → [3] → [2] → [1] → null
```

Solution Approach

Concept: Use three pointers: prev, current, next.

Algorithm

```
Algorithm REVERSE_ITERATIVE(head):
           prev = null
           current = head

           While current != null:
               next = current.next
               current.next = prev
               prev = current
               current = next

           Return prev
```

Java Implementation

```
class Solution {
           public Node reverseList(Node head) {
               Node prev = null;
               Node current = head;

                 while (current != null) {
                     Node next = current.next;
                     current.next = prev;
                     prev = current;
                     current = next;
                 }
                 return prev;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

106. Reverse a LL [Recursive]

Understanding the Problem NEW

Same reversal as above, but implemented recursively instead of with an explicit loop and three pointers.

Brute Force Approach NEW

Nothing 'brute' to contrast against here — it's the same problem as 105, just a different implementation style, useful for practicing recursive thinking on linked structures.

Why We Optimize NEW

The recursive version reverses everything *after* the current node first (via the recursive call), and only then fixes up the current node's own pointer, once the rest of the list is already reversed and its new tail is known. This mirrors the 'recurse first, fix up after' pattern seen in Recursive Insertion Sort back in Chapter 1 — same underlying idea, applied to pointers instead of array values. It still achieves O(n) time, but costs O(n) extra space for the recursion call stack, unlike the iterative version's O(1) space.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse LinkedList using recursion.

Visual Diagram

```
Original: [1] → [2] → [3] → [4] → null

       Recursive calls:
       reverse(1):
         reverse(2):
           reverse(3):
             reverse(4):
               return 4 (base case)

                 head = 4
                 3.next.next = 3 (4.next = 3)
                 3.next = null
```

```
                return 4

            2.next.next = 2 (3.next = 2)
            2.next = null
            return 4

          1.next.next = 1 (2.next = 1)
          1.next = null
          return 4

      Result: [4] → [3] → [2] → [1] → null
```

Solution Approach

Concept: Recursively reverse smaller list.

Algorithm

```
Algorithm REVERSE_RECURSIVE(head):
          If head == null or head.next == null:
              Return head

            newHead = REVERSE_RECURSIVE(head.next)
            head.next.next = head
            head.next = null
            Return newHead
```

Java Implementation

```
class Solution {
          public Node reverseList(Node head) {
              if (head == null || head.next == null) {
                  return head;
              }

                  Node newHead = reverseList(head.next);
                  head.next.next = head;
                  head.next = null;

                  return newHead;
            }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n) recursion stack

107. Detect a Loop in LL

Understanding the Problem NEW

Determine whether a linked list has a cycle (some node's 'next' pointer loops back to an earlier node instead of eventually reaching null).

Brute Force Approach NEW

A beginner might try storing every visited node's reference in a hashset, and check before visiting each node whether it's already been seen — correct, O(n) time, but O(n) extra space to remember every node visited.

Why We Optimize NEW

Floyd's Cycle Detection (the Tortoise and Hare again) solves this with O(1) space: use two pointers moving at different speeds (one step vs two steps) through the list. If there's no cycle, the fast pointer will simply reach the end (null) first. But if there *is* a cycle, the fast pointer will eventually 'lap' the slow pointer and they'll land on the exact same node at the same time — a mathematical guarantee for any cycle length. This gets you the same O(n) time as the hashset approach, but without needing any extra memory.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Detect if LinkedList has a cycle.

Visual Diagram

```
List with cycle: [1] → [2] → [3] → [4] → [5]
                               ↑                ↓
                               [8] ← [7] ← [6]

       Floyd's Cycle Detection:
       Step 1: slow=1, fast=1
       Step 2: slow=2, fast=3
       Step 3: slow=3, fast=5
       Step 4: slow=4, fast=7
       Step 5: slow=5, fast=4 (met at 4)
       → Cycle detected!
```

```
No cycle: [1] → [2] → [3] → [4] → null
slow=1, fast=1
slow=2, fast=3
slow=3, fast=5
fast reaches null → no cycle
```

Solution Approach

Concept: Floyd's cycle detection algorithm.

Algorithm

```
Algorithm HAS_CYCLE(head):
           slow = head
           fast = head

            While fast != null and fast.next != null:
                slow = slow.next
                fast = fast.next.next
                If slow == fast:
                    Return true

            Return false
```

Java Implementation

```
class Solution {
           public boolean hasCycle(Node head) {
               Node slow = head;
               Node fast = head;

                 while (fast != null && fast.next != null) {
                     slow = slow.next;
                     fast = fast.next.next;

                     if (slow == fast) {
                         return true;
                     }
                 }
                 return false;
             }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

108. Find the Starting Point of Loop in LL

Understanding the Problem NEW

Given that a linked list has a cycle (from problem 107), find the exact node where the cycle actually begins — not just detect that one exists.

Brute Force Approach NEW

Once you know a cycle exists, tracking every visited node in a hashset and reporting the first one you see twice would find the start — correct, O(n) time, O(n) space.

Why We Optimize NEW

After the slow/fast pointers meet (as in Floyd's algorithm), there's an elegant mathematical fact: if you now move the **slow pointer back to the head** and advance both the slow and fast pointers *one step at a time* from here, they are guaranteed to meet exactly at the start of the cycle. This relies on the specific distance relationships built up during the initial meeting, and again avoids any extra hashset — still O(n) time, O(1) space, extending Floyd's algorithm rather than falling back to extra memory.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the node where cycle begins.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → [6]
                            ↑               ↓
                            [9] ← [8] ← [7]

       Step 1: Find meeting point (Floyd's)
       slow=1, fast=1
       slow=2, fast=3
       slow=3, fast=5
       slow=4, fast=7
       slow=5, fast=4
       Meeting point = 4
```

```
       Step 2: Find cycle start
       Initialize slow = head (1)
       Move slow and fast 1 step each
       slow=1, fast=4
       slow=2, fast=5
       slow=3, fast=6
       slow=4, fast=7
       Now slow=fast=4 → Cycle starts at 4!

       Result: Cycle starts at node with value 4
```

Solution Approach

Concept: After meeting, move slow to head, both step one by one.

Algorithm

```
Algorithm CYCLE_START(head):
           If no cycle: Return null

           slow = head, fast = head
           While fast != null and fast.next != null:
               slow = slow.next
               fast = fast.next.next
               If slow == fast:
                   slow = head
                   While slow != fast:
                       slow = slow.next
                       fast = fast.next
                   Return slow
           Return null
```

Java Implementation

```
class Solution {
           public Node detectCycle(Node head) {
               Node slow = head;
               Node fast = head;

                 // Find meeting point
                 while (fast != null && fast.next != null) {
                     slow = slow.next;
                     fast = fast.next.next;

                  if (slow == fast) {
                      // Find cycle start
                      slow = head;
                      while (slow != fast) {
                          slow = slow.next;
                          fast = fast.next;
```

```
                    }
                    return slow;
                }
            }
            return null;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

109. Length of Loop in LL

Understanding the Problem NEW

Given a cyclic linked list, find how many nodes are in the cycle itself (not the whole list, just the looping part).

Brute Force Approach NEW

Store visited nodes with a step counter in a hashset; once you revisit a node, the difference between the current step count and that node's stored step count is the cycle length — O(n) time, O(n) space.

Why We Optimize NEW

Once Floyd's algorithm finds any meeting point inside the cycle (from problem 107), you can compute the cycle's length for free without extra memory: just keep one pointer fixed at the meeting point, advance a second pointer one step at a time, and count steps until it comes back around to the same node. This reuses the meeting point you've already found, giving O(cycle length) additional work and O(1) space, instead of tracking every node visited.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the length of the cycle in LinkedList.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → [6]
                            ↑               ↓
                            [9] ← [8] ← [7]

       Step 1: Find meeting point (Floyd's)
       Meeting point = 4

       Step 2: Count length
       temp = meeting.next (5)
       count = 1
       temp=5 → count=1
       temp=6 → count=2
       temp=7 → count=3
```

```
temp=8 → count=4
temp=9 → count=5
temp=4 (meeting) → stop
Loop length = 5

Result: Length = 5
```

Solution Approach

Concept: Find meeting point, then count nodes until back.

Algorithm

```
Algorithm LOOP_LENGTH(head):
           If no cycle: Return 0
           slow = head, fast = head

            While fast != null and fast.next != null:
                slow = slow.next
                fast = fast.next.next
                If slow == fast:
                     count = 1
                     temp = slow.next
                     While temp != slow:
                         count++
                         temp = temp.next
                     Return count
            Return 0
```

Java Implementation

```
class Solution {
           public int loopLength(Node head) {
               Node slow = head;
               Node fast = head;

                 while (fast != null && fast.next != null) {
                     slow = slow.next;
                     fast = fast.next.next;

                     if (slow == fast) {
                         int count = 1;

                      Node temp = slow.next;
                      while (temp != slow) {
                          count++;
                          temp = temp.next;
                      }
                      return count;
                  }
```

```
            }
            return 0;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

110. Check if LL is Palindrome or Not

Understanding the Problem NEW

Check whether a linked list reads the same forwards and backwards (like an array palindrome check), but linked lists can't be indexed backward directly the way arrays can.

Brute Force Approach NEW

A beginner might copy all the values into an array first, then check the array for palindrome-ness using two pointers from both ends — correct, O(n) time, but O(n) extra space for the copy.

Why We Optimize NEW

You can do this with O(1) extra space by combining two tricks you've already learned in this chapter: find the middle of the list (problem 104's Tortoise and Hare), **reverse the second half in-place** (problem 105's technique), and then walk the first half and the reversed second half together, comparing values node by node. If you want to restore the list afterward, you can reverse the second half back once the comparison is done. This avoids the O(n) array copy entirely, reusing two techniques already built earlier in the chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if LinkedList is palindrome.

Visual Diagram

```
List: [1] → [2] → [3] → [2] → [1] → null
       Palindrome ✓

       Step 1: Find middle (3)
       Step 2: Reverse second half [2] → [1] → null becomes [1] → [2] → null
       Step 3: Compare first and second half
       [1] = [1] ✓
       [2] = [2] ✓
       Result: true

       List: [1] → [2] → [3] → [4] → null
```

```
Not palindrome ✗

Step 1: Find middle (3)
Step 2: Reverse second half [4] → null becomes [4] → null
Step 3: Compare
[1] ≠ [4] ✗
Result: false
```

Solution Approach

Concept: Find middle, reverse second half, compare.

Algorithm

```
Algorithm IS_PALINDROME(head):
           If head == null: Return true

           // Find middle
           slow = head, fast = head
           While fast.next != null and fast.next.next != null:
               slow = slow.next
               fast = fast.next.next

           // Reverse second half
           second = reverseList(slow.next)
           slow.next = null

           // Compare
           first = head
           while second != null:
               If first.data != second.data: Return false
               first = first.next
               second = second.next

           Return true
```

Java Implementation

```
class Solution {
           public boolean isPalindrome(Node head) {
               if (head == null) return true;

                 // Find middle
                 Node slow = head;

              Node fast = head;
              while (fast.next != null && fast.next.next != null) {
                  slow = slow.next;
                  fast = fast.next.next;
              }
```

```
            // Reverse second half
            Node second = reverseList(slow.next);
            slow.next = null;

            // Compare
            Node first = head;
            Node temp = second;
            while (temp != null) {
                if (first.data != temp.data) return false;
                first = first.next;
                temp = temp.next;
            }

            return true;
        }

        private Node reverseList(Node head) {
            Node prev = null;
            Node current = head;
            while (current != null) {
                Node next = current.next;
                current.next = prev;
                prev = current;
                current = next;
            }
            return prev;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

111. Segregate Odd and Even Nodes in LL

Understanding the Problem NEW

Rearrange a linked list so all nodes originally at odd positions (1st, 3rd, 5th...) come first, followed by all nodes originally at even positions (2nd, 4th, 6th...), preserving their relative order within each group.

Brute Force Approach NEW

A beginner might copy all node values into two separate arrays (odd-position values and even-position values), then rebuild the linked list from those arrays — correct, O(n) time, but O(n) extra space, and it discards the existing nodes unnecessarily.

Why We Optimize NEW

Since you already have direct pointer access to every node, you can do this **in-place** by maintaining two running pointers — one tracking the current tail of the 'odd chain,' one tracking the current tail of the 'even chain' — and re-wiring 'next' pointers as you walk through the original list once, finally connecting the end of the odd chain to the head of the even chain. This achieves O(n) time with O(1) extra space, by rewiring existing nodes instead of copying values into new structures.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Segregate odd-indexed and even-indexed nodes.

Visual Diagram

```
Original: [1] → [2] → [3] → [4] → [5] → null

       Odd positions: 1, 3, 5
       Even positions: 2, 4

       Result: [1] → [3] → [5] → [2] → [4] → null

       Step-by-step:
       Step 1: odd = head (1), even = head.next (2)
       Step 2: oddHead = 1, evenHead = 2
```

```
Step 3: Link odd nodes
1.next = 3
3.next = 5
5.next = null
Step 4: Link even nodes
2.next = 4
4.next = null
Step 5: Combine
5.next = 2
Result: 1 → 3 → 5 → 2 → 4
```

Solution Approach

Concept: Separate odd and even position nodes.

Algorithm

```
Algorithm SEGREGATE_ODD_EVEN(head):
           If head == null: Return null

           odd = head
           even = head.next
           evenHead = even

           While even != null and even.next != null:
               odd.next = even.next
               odd = odd.next
               even.next = odd.next
               even = even.next

           odd.next = evenHead
           Return head
```

Java Implementation

```
class Solution {
           public Node oddEvenList(Node head) {
               if (head == null) return null;

                 Node odd = head;
                 Node even = head.next;
                 Node evenHead = even;

                 while (even != null && even.next != null) {
                     odd.next = even.next;

                  odd = odd.next;
                  even.next = odd.next;
                  even = even.next;
              }
```

```
            odd.next = evenHead;
            return head;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

112. Remove Nth Node from the Back

Understanding the Problem NEW

Remove the N-th node counting from the *end* of the list (not the beginning), ideally in a single pass.

Brute Force Approach NEW

The natural two-pass approach: first traverse the whole list to find its total length L, then traverse again to the (L-N)-th node from the start to perform the deletion — correct, but two full passes.

Why We Optimize NEW

Use two pointers with a fixed **gap of N** between them: first advance one pointer N steps ahead on its own, then move both pointers forward together, one step at a time. When the front pointer reaches the end, the back pointer will be sitting exactly at the node just before the one to delete. This achieves the deletion in a single O(n) pass instead of two — the fixed gap does the work that counting the total length would otherwise require.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Remove the Nth node from the end of LinkedList.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → null, N=2
       Remove: 2nd from end = 4

       Step 1: fast = head
       Move fast N steps: fast = 3

       Step 2: slow = head
       Move both until fast reaches end
       slow=1, fast=3
       slow=2, fast=4
       slow=3, fast=5
       slow=4, fast=null
```

```
Step 3: slow is at node before target
Remove target: slow.next = slow.next.next
Result: [1] → [2] → [3] → [5] → null

N=1 (remove last):
fast = 1 → move to 5
slow = 1 → move to 4
Remove: 4.next = null
Result: [1] → [2] → [3] → [4] → null
```

Solution Approach

Concept: Two pointers with N gap.

Algorithm

```
Algorithm REMOVE_NTH_FROM_END(head, n):
           dummy = Node(0)
           dummy.next = head
           slow = dummy
           fast = dummy

           For i = 1 to n+1:
               fast = fast.next

           While fast != null:
               slow = slow.next
               fast = fast.next

           slow.next = slow.next.next
           Return dummy.next
```

Java Implementation

```
class Solution {
           public Node removeNthFromEnd(Node head, int n) {
               Node dummy = new Node(0);
               dummy.next = head;
               Node slow = dummy;
               Node fast = dummy;

                 // Move fast n+1 steps ahead

              for (int i = 0; i <= n; i++) {
                  fast = fast.next;
              }

              // Move both until fast reaches end
              while (fast != null) {
                  slow = slow.next;
```

```
                fast = fast.next;
            }

            // Remove nth node
            slow.next = slow.next.next;
            return dummy.next;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

113. Delete the Middle Node of LL

Understanding the Problem NEW

Delete the middle node of a linked list directly, without first computing the list's length.

Brute Force Approach NEW

Just like the earlier length-then-traverse approach, you could count nodes first (one pass), then traverse again to the middle position for deletion (a second pass).

Why We Optimize NEW

Reuse the Tortoise and Hare technique from problem 104 (with a small pointer-start adjustment) to locate the middle in a single pass, while also keeping a reference to the node just *before* the middle so you can immediately unlink it the moment you arrive — combining 'find the middle' and 'delete a node' into one O(n) traversal instead of two separate ones.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Delete the middle node of LinkedList.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → null
       Middle: 3

       Step 1: Find middle
       slow=1, fast=1
       slow=2, fast=3
       slow=3, fast=5
       slow=4, fast=null
       Middle = 3

       Step 2: Delete middle
       Result: [1] → [2] → [4] → [5] → null

       List: [1] → [2] → [3] → [4] → [5] → [6] → null
```

```
Middle: 3 (for even length, first middle)

Step 1: Find middle
slow=1, fast=1
slow=2, fast=3
slow=3, fast=5
slow=4, fast=null
Middle = 3

Result: [1] → [2] → [4] → [5] → [6] → null
```

Solution Approach

Concept: Use two pointers to find middle.

Algorithm

```
Algorithm DELETE_MIDDLE(head):
           If head == null or head.next == null:
               Return null

           slow = head
           fast = head
           prev = null

           While fast != null and fast.next != null:
               prev = slow
               slow = slow.next
               fast = fast.next.next

           prev.next = slow.next
           Return head
```

Java Implementation

```
class Solution {
           public Node deleteMiddle(Node head) {
               if (head == null || head.next == null) {
                   return null;
               }
               Node slow = head;

              Node fast = head;
              Node prev = null;

              while (fast != null && fast.next != null) {
                  prev = slow;
                  slow = slow.next;
                  fast = fast.next.next;
              }
```

```
            prev.next = slow.next;
            return head;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

114. Sort LinkedList

Understanding the Problem NEW

Sort a linked list into ascending order — but unlike an array, you can't randomly access elements by index, which changes which sorting algorithms are practical here.

Brute Force Approach NEW

You could copy all values into an array, sort the array with any standard algorithm (like Chapter 1's Quick Sort), and rebuild the linked list from the sorted array — correct, O(n log n) time, but O(n) extra space for the copy, and it throws away the existing nodes.

Why We Optimize NEW

Merge Sort turns out to be an exceptionally good fit for linked lists specifically, because merging two sorted lists doesn't require random access — you just walk both lists with pointers and relink nodes as you go, and finding the 'middle' to split the list uses the same Tortoise and Hare trick from problem 104. This lets you sort the linked list in-place (rewiring existing nodes) in O(n log n) time with only O(log n) extra space for the recursion, instead of O(n) space for an array copy — Quick Sort, by contrast, doesn't translate nearly as well to linked lists because its partitioning step benefits from random access that a linked list doesn't offer.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Sort a LinkedList using merge sort.

Visual Diagram

```
List: [4] → [2] → [1] → [3] → null

       Step 1: Find middle → [2]
       Step 2: Split into left and right
       Left: [4] → [2] → null
       Right: [1] → [3] → null
```

```
Step 3: Sort each half
Left: [2] → [4] → null
Right: [1] → [3] → null

Step 4: Merge sorted halves
Compare 2 and 1 → 1
Compare 2 and 3 → 2
Compare 4 and 3 → 3
Append 4
Result: [1] → [2] → [3] → [4] → null
```

Solution Approach

Concept: Merge sort on LinkedList.

Algorithm

```
Algorithm SORT_LL(head):
           If head == null or head.next == null:
               Return head

           // Find middle
           slow = head, fast = head
           While fast.next != null and fast.next.next != null:
               slow = slow.next
               fast = fast.next.next

           // Split
           middle = slow.next
           slow.next = null

           // Sort halves
           left = SORT_LL(head)
           right = SORT_LL(middle)

           // Merge
           Return MERGE(left, right)
```

Java Implementation

```
class Solution {
           public Node sortList(Node head) {
               if (head == null || head.next == null) {
                   return head;
               }

                 // Find middle
                 Node slow = head;

               Node fast = head;
```

```
            while (fast.next != null && fast.next.next != null) {
                slow = slow.next;
                fast = fast.next.next;
            }

            // Split
            Node middle = slow.next;
            slow.next = null;

            // Sort halves
            Node left = sortList(head);
            Node right = sortList(middle);

            // Merge
            return merge(left, right);
        }

        private Node merge(Node l1, Node l2) {
            Node dummy = new Node(0);
            Node current = dummy;

            while (l1 != null && l2 != null) {
                if (l1.data < l2.data) {
                    current.next = l1;
                    l1 = l1.next;
                } else {
                    current.next = l2;
                    l2 = l2.next;
                }
                current = current.next;
            }

            current.next = (l1 != null) ? l1 : l2;
            return dummy.next;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(log n)

115. Sort a LL of 0's, 1's and 2's

Understanding the Problem NEW

Sort a linked list that only contains the values 0, 1, and 2 — the linked-list version of the Dutch National Flag problem from the Arrays chapter (problem 23).

Brute Force Approach NEW

You could count the 0s, 1s, and 2s in one pass, then overwrite each node's *value* in a second pass based on the counts — this works (O(n) time) but relies on being able to freely overwrite values, which isn't always the intent when the goal is to rearrange nodes.

Why We Optimize NEW

A cleaner approach true to the spirit of linked lists: create three separate dummy chains (for 0s, 1s, and 2s), walk through the original list once, and append each node to the appropriate chain by rewiring pointers (no value copying needed) — then splice the three chains together in order at the end. This is a direct pointer-based translation of the Dutch National Flag idea, achieving O(n) time and O(1) extra space (beyond the three temporary head pointers) by moving existing nodes around instead of overwriting their contents.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Sort LinkedList containing 0, 1, 2.

Visual Diagram

```
List: [2] → [1] → [0] → [2] → [1] → [0] → null
       Result: [0] → [0] → [1] → [1] → [2] → [2] → null

       Method 1: Count frequencies
       Count 0s: 2, 1s: 2, 2s: 2
       Build list: 0,0,1,1,2,2

       Method 2: Three pointers (same as Dutch Flag)
       zeroHead = null, oneHead = null, twoHead = null
```

```
       Traverse and add to respective lists
       Then combine: zero → one → two
```

Solution Approach

Concept: Use three separate lists for 0, 1, 2.

Algorithm

```
Algorithm SORT_012(head):
           zero = new Node(0), one = new Node(0), two = new Node(0)
           zeroHead = zero, oneHead = one, twoHead = two

            current = head
            While current != null:
                If current.data == 0:
                    zero.next = current
                    zero = zero.next
                Else If current.data == 1:
                    one.next = current
                    one = one.next
                Else:
                    two.next = current
                    two = two.next
                current = current.next

            zero.next = oneHead.next ? oneHead.next : twoHead.next
            one.next = twoHead.next
            two.next = null

            Return zeroHead.next
```

Java Implementation

```
class Solution {
           public Node sort012(Node head) {
               if (head == null) return null;

                 Node zero = new Node(0);
                 Node one = new Node(0);
                 Node two = new Node(0);
                 Node zeroHead = zero, oneHead = one, twoHead = two;

                 Node current = head;
                 while (current != null) {
                     if (current.data == 0) {

                      zero.next = current;
                      zero = zero.next;
                  } else if (current.data == 1) {
```

```
                    one.next = current;
                    one = one.next;
                } else {
                    two.next = current;
                    two = two.next;
                }
                current = current.next;
            }

            zero.next = (oneHead.next != null) ? oneHead.next : twoHead.next;
            one.next = twoHead.next;
            two.next = null;

            return zeroHead.next;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

116. Find the Intersection Point of Y LinkedList

Understanding the Problem NEW

Two singly linked lists eventually merge into the same nodes, forming a Y shape. Find the exact node where they intersect (not just a node with the same *value*, but literally the same node in memory).

Brute Force Approach NEW

A beginner might compare every node in list A against every node in list B, checking for a matching reference — O(n×m) time, or use a hashset of all of list A's node references and check list B's nodes against it — O(n+m) time but O(n) extra space.

Why We Optimize NEW

Because both lists share the exact same tail once they merge, their lengths only differ in the 'unshared' portion before the merge point. If you first find each list's length, then advance the pointer of the *longer* list by the length difference so both pointers now have the same number of remaining nodes to the end, walking both remaining pointers forward together one step at a time will land them on the intersection node simultaneously. This achieves O(n+m) time with O(1) extra space, avoiding the hashset entirely by aligning the two lists' remaining lengths first.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find where two LinkedLists intersect.

Visual Diagram

```
List A: [1] → [2] → [3]
                            ↓
                            [7] → [8] → [9] → null
                            ↑
       List B: [4] → [5] → [6]

       Intersection node: 7
```

```
Method 1: HashSet
Add all nodes of list A to set
Traverse list B and check if node in set

Method 2: Two pointers
Step 1: Find length difference = 3
Step 2: Move longer list (A) by 3
Step 3: Move both until they meet
Result: Intersection at 7
```

Solution Approach

Concept: Align lengths and traverse together.

Algorithm

```
Algorithm INTERSECTION(headA, headB):
           lenA = length(headA), lenB = length(headB)
           diff = abs(lenA - lenB)

           If lenA > lenB:
               Move headA by diff
           Else:
               Move headB by diff

           While headA != null and headB != null:
               If headA == headB:
                   Return headA
               headA = headA.next
               headB = headB.next

           Return null
```

Java Implementation

```
class Solution {
           public Node getIntersectionNode(Node headA, Node headB) {
               int lenA = getLength(headA);
               int lenB = getLength(headB);

                 while (lenA > lenB) {
                     headA = headA.next;
                     lenA--;
                 }
                 while (lenB > lenA) {
                     headB = headB.next;
                     lenB--;

             }
```

```
            while (headA != null && headB != null) {
                if (headA == headB) {
                    return headA;
                }
                headA = headA.next;
                headB = headB.next;
            }
            return null;
        }

        private int getLength(Node head) {
            int len = 0;
            while (head != null) {
                len++;
                head = head.next;
            }
            return len;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n+m) | Space Complexity: O(1)

117. Add 1 to a Number Represented by LL

Understanding the Problem NEW

A number is represented digit-by-digit as a linked list (e.g. `1 -> 2 -> 3` represents 123). Add 1 to this number and return the updated linked list, correctly handling carries (e.g. `1->9->9` becomes `2->0->0`).

Brute Force Approach NEW

Convert the linked list into an actual integer (or string), add 1 using normal arithmetic, then rebuild a new linked list from the result — works for small numbers, but breaks down for very large numbers that don't fit in a standard integer type, which is exactly the scenario this problem is designed to represent.

Why We Optimize NEW

Since a carry can potentially ripple all the way from the last digit back to the first (like `999` becoming `1000`), the natural approach is to **reverse the list, add 1 with carry propagation from the new front (originally the last digit), then reverse back** — or equivalently, use recursion to process from the last node first (unwinding back to the front, similar to Recursive Insertion Sort's structure) so the carry naturally propagates backward through the return values. Either way, this handles arbitrarily large numbers correctly in O(n) time without ever converting to a fixed-size integer type that could overflow.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Add 1 to a number represented by LinkedList.

Visual Diagram

```
List: [1] → [2] → [3] (represents 123)
       Add 1: [1] → [2] → [4] (124)

       List: [9] → [9] → [9] (999)
       Add 1: [1] → [0] → [0] → [0] (1000)

       Method: Reverse and add
       Reverse: [3] → [2] → [1] → null
```

```
Add 1: carry=1
3+1=4, carry=0
2+0=2
1+0=1
Reverse back: [1] → [2] → [4]

Alternative: Recursive
AddOne(head):
  If head == null: Return 1
  carry = AddOne(head.next)
  If carry == 0: Return 0
  head.data += 1
  If head.data == 10:
    head.data = 0
    Return 1
  Return 0
```

Solution Approach

Concept: Use recursion or reverse and add.

Algorithm

```
Algorithm ADD_ONE(head):
           carry = ADD_CARRY(head)
           If carry == 1:
               newNode = Node(1)
               newNode.next = head
               Return newNode
           Return head

       Algorithm ADD_CARRY(node):
           If node == null:
               Return 1
           carry = ADD_CARRY(node.next)
           If carry == 0:
               Return 0
           node.data += 1
           If node.data == 10:
               node.data = 0
               Return 1
           Return 0
```

Java Implementation (Recursive)

class Solution {

public Node addOne(Node head) {

int carry = addCarry(head);

```
if (carry == 1) {
Node newNode = new Node(1);
newNode.next = head;
return newNode;
}
return head;
}
private int addCarry(Node node) {
if (node == null) {
return 1;
}
int carry = addCarry(node.next);
if (carry == 0) {
return 0;
}
node.data += 1;
if (node.data == 10) {
node.data = 0;
return 1;
}
return 0;
}
}
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n) recursion stack

118. Add 2 Numbers in LL

Understanding the Problem NEW

Two numbers are each represented as linked lists of digits (commonly stored in reverse order, so the head is the least significant digit), and you must add them together and return the sum as a new linked list, again handling carries correctly.

Brute Force Approach NEW

Converting both linked lists to actual integers, adding them normally, and converting the result back to a linked list works for small numbers but fails the same way as problem 117 for very large numbers that overflow standard integer types.

Why We Optimize NEW

Walk both lists simultaneously from their heads (least significant digits, if stored in reverse order), adding corresponding digits together along with any carry from the previous position, creating a new result node for each position's digit, and continuing past the end of the shorter list if one list is longer than the other (treating missing digits as 0). Any leftover carry after both lists are exhausted becomes one final extra digit. This is a direct digit-by-digit simulation of elementary-school long addition, run in O(max(n,m)) time — never needing to represent the whole number as a single integer at once, which is exactly why linked-list digit representation is used for arbitrarily large numbers in the first place.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Add two numbers represented by LinkedLists.

Visual Diagram

```
List 1: [2] → [4] → [3] (243)
       List 2: [5] → [6] → [4] (564)
       Result: [8] → [0] → [7] (807)

       Step-by-step:
       2 + 5 = 7 → carry 0
```

```
4 + 6 = 10 → digit 0, carry 1
3 + 4 + 1 = 8 → digit 8, carry 0

List 1: [9] → [9] (99)
List 2: [1] (1)
Result: [1] → [0] → [0] (100)

9 + 1 = 10 → digit 0, carry 1
9 + 0 + 1 = 10 → digit 0, carry 1
carry = 1 → new node [1]
```

Solution Approach

Concept: Traverse and add digit by digit with carry.

Algorithm

```
Algorithm ADD_TWO_NUMBERS(l1, l2):
           dummy = Node(0)
           current = dummy
           carry = 0

           While l1 != null or l2 != null or carry > 0:
               sum = carry
               If l1 != null:
                   sum += l1.data
                   l1 = l1.next
               If l2 != null:
                   sum += l2.data
                   l2 = l2.next

                 carry = sum / 10
                 current.next = Node(sum % 10)
                 current = current.next

           Return dummy.next
```

Java Implementation

```
class Solution {
           public Node addTwoNumbers(Node l1, Node l2) {
               Node dummy = new Node(0);
               Node current = dummy;
               int carry = 0;

                 while (l1 != null || l2 != null || carry > 0) {
                     int sum = carry;

                    if (l1 != null) {
```

```
                    sum += l1.data;
                    l1 = l1.next;
                }
                if (l2 != null) {
                    sum += l2.data;
                    l2 = l2.next;
                }

                carry = sum / 10;
                current.next = new Node(sum % 10);
                current = current.next;
            }

            return dummy.next;
        }
    }
```

Time & Space Complexity

Time Complexity: O(max(n,m)) | Space Complexity: O(max(n,m))

119. Delete All Occurrences of a Key in DLL

Understanding the Problem NEW

Delete every node containing a specific key value from a Doubly LinkedList (there may be multiple occurrences, unlike a simple single-delete).

Brute Force Approach NEW

Convert to an array, filter out matching values, and rebuild the list from scratch — O(n) time but O(n) extra space and discards all existing nodes unnecessarily.

Why We Optimize NEW

Because DLL nodes have direct access to both their previous and next neighbours, you can walk through the list once and, whenever you find a matching node, immediately unlink it in O(1) by connecting its neighbours directly to each other (using the same technique as problem 102) — no need to track a separate 'previous' pointer manually the way you would in a singly linked list, since each node already knows its own predecessor. This deletes all occurrences in a single O(n) pass with O(1) extra space, reusing DLL's key advantage directly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Delete all nodes with a specific key in Doubly LinkedList.

Visual Diagram

```
Original: null ← [1] ↔ [2] ↔ [3] ↔ [2] ↔ [4] → null
       Delete key = 2

       Step 1: Check head
       head=1 → not delete
       Step 2: Check 2 → delete
       1.next = 3, 3.prev = 1
       Step 3: Check 3 → not delete
       Step 4: Check 2 → delete
       3.next = 4, 4.prev = 3
       Step 5: Check 4 → not delete
```

```
Result: null ← [1] ↔ [3] ↔ [4] → null
```

Solution Approach

Concept: Traverse and delete matching nodes.

Algorithm

```
Algorithm DELETE_KEY_DLL(head, key):
           current = head
           While current != null:
               If current.data == key:
                   If current == head:
                       head = head.next
                       If head != null: head.prev = null
                   Else:
                       current.prev.next = current.next
                       If current.next != null:
                           current.next.prev = current.prev
               current = current.next
           Return head
```

Java Implementation

```
class Solution {
           public DLLNode deleteAllOccurrences(DLLNode head, int key) {
               DLLNode current = head;

                 while (current != null) {
                     if (current.data == key) {
                         if (current == head) {
                             head = head.next;
                             if (head != null) {
                                  head.prev = null;
                             }
                         } else {
                             current.prev.next = current.next;
                             if (current.next != null) {
                                  current.next.prev = current.prev;
                             }

                        }
                    }
                    current = current.next;
               }
               return head;
           }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

120. Find Pairs with Given Sum in DLL

Understanding the Problem NEW

Given a **sorted** Doubly LinkedList, find all pairs of nodes whose values sum to a given target — the linked-list version of the Two Sum problem, but with a sorted, bidirectional structure available instead of a hashmap.

Brute Force Approach NEW

Just like Two Sum on an unsorted array, you could use a hashset of values seen so far as you traverse once — O(n) time, O(n) extra space — but this doesn't take advantage of the list being sorted and bidirectional.

Why We Optimize NEW

Because the DLL is sorted *and* supports backward traversal, you can apply the exact **two-pointer** technique used for sorted arrays in problems like 3-Sum: one pointer starts at the head, one at the tail, and you move them toward each other based on whether the current pair's sum is too small or too large. This achieves O(n) time with O(1) extra space (no hashset needed), directly exploiting the DLL's sortedness and backward pointers — something a singly linked list couldn't offer as cleanly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all pairs in sorted DLL that sum to target.

Visual Diagram

```
List: null ← [1] ↔ [2] ↔ [3] ↔ [4] ↔ [5] → null
       Target: 5

       Pairs:
       1 + 4 = 5 ✓
       2 + 3 = 5 ✓

       Step 1: left = head (1), right = tail (5)
       Sum = 6 > 5 → right--
       Sum = 1+4=5 → pair found, left++, right--
       Sum = 2+3=5 → pair found, left++, right--
```

```
       Result: (1,4), (2,3)
```

Solution Approach

Concept: Two pointers from both ends.

Algorithm

```
Algorithm PAIRS_IN_DLL(head, target):
           left = head
           right = head
           While right.next != null:
               right = right.next

            pairs = []
            While left != right and left.prev != right:
                sum = left.data + right.data
                If sum == target:
                    pairs.add([left.data, right.data])
                    left = left.next
                    right = right.prev
                Else If sum < target:
                    left = left.next
                Else:
                    right = right.prev
            Return pairs
```

Java Implementation

```
class Solution {
           public List<int[]> findPairs(DLLNode head, int target) {
               List<int[]> pairs = new ArrayList<>();
               DLLNode left = head;
               DLLNode right = head;

                 while (right.next != null) {
                     right = right.next;
                 }

                 while (left != null && right != null && left != right &&
left.prev != right) {
                     int sum = left.data + right.data;

                     if (sum == target) {

                      pairs.add(new int[]{left.data, right.data});
                      left = left.next;
                      right = right.prev;
                  } else if (sum < target) {
```

```
                    left = left.next;
                } else {
                    right = right.prev;
                }
            }
            return pairs;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

121. Remove Duplicates from Sorted DLL

Understanding the Problem NEW

Given a sorted Doubly LinkedList, remove any duplicate values, keeping only one copy of each distinct value.

Brute Force Approach NEW

Convert to an array, deduplicate (as in the Arrays-chapter Remove Duplicates problem), then rebuild the list — O(n) time but O(n) extra space and needless node reconstruction.

Why We Optimize NEW

Because the list is sorted, duplicates are always adjacent — so you can walk through the list once, and whenever the current node's value equals the next node's value, unlink that next node directly (an O(1) DLL deletion, same as problem 102) instead of advancing past it. This is a direct linked-list translation of the two-pointer 'remove duplicates from sorted array' technique (problem 11), achieving O(n) time and O(1) extra space by rewiring pointers instead of copying to an array.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Remove duplicate nodes from sorted Doubly LinkedList.

Visual Diagram

```
Original: null ← [1] ↔ [2] ↔ [2] ↔ [3] ↔ [3] ↔ [4] → null
       Result: null ← [1] ↔ [2] ↔ [3] ↔ [4] → null

       Step-by-step:
       current = 1
       next = 2 (1 != 2)
       current = 2
       next = 2 (duplicate)
       Skip duplicate: 2.next = 3, 3.prev = 2
       current = 2
       next = 3 (2 != 3)
       current = 3
```

```
        next = 3 (duplicate)
        Skip duplicate: 3.next = 4, 4.prev = 3
        current = 3
        next = 4 (3 != 4)
        current = 4
        next = null (done)
```

Solution Approach

Concept: Traverse and skip duplicates.

Algorithm

```
Algorithm REMOVE_DUPLICATES_DLL(head):
           current = head
           While current != null and current.next != null:
               If current.data == current.next.data:
                   duplicate = current.next
                   current.next = duplicate.next
                   If duplicate.next != null:
                       duplicate.next.prev = current
               Else:
                   current = current.next
           Return head
```

Java Implementation

```
class Solution {
           public DLLNode removeDuplicates(DLLNode head) {
               if (head == null) return null;

                 DLLNode current = head;

                 while (current != null && current.next != null) {
                     if (current.data == current.next.data) {
                         DLLNode duplicate = current.next;
                         current.next = duplicate.next;
                         if (duplicate.next != null) {
                              duplicate.next.prev = current;
                         }
                     } else {
                         current = current.next;
                     }

              }
              return head;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

122. Reverse LL in Groups of Given Size K

Understanding the Problem NEW

Reverse a linked list not all at once, but in chunks of size K — e.g. reversing `[1,2,3,4,5]` in groups of 2 gives `[2,1,4,3,5]` (the last group, if shorter than K, may or may not be reversed depending on the exact problem variant).

Brute Force Approach NEW

A beginner might extract each group of K nodes' values into a temporary array, reverse the array, and write the values back into the same nodes — this works (O(n) time) but relies on overwriting values rather than truly rewiring the list's node structure, which misses the point of practicing pointer manipulation.

Why We Optimize NEW

This combines two ideas you already have: the **three-pointer reversal** technique from problem 105, applied to just the current group of K nodes at a time, and **recursion** (or an iterative loop) to move on to the next group once the current one is reversed, connecting the reversed group's new tail to the result of reversing the next group. This is a genuine pointer-rewiring solution, still O(n) time overall (each node is only touched a constant number of times), reusing the core reversal building block from earlier in the chapter rather than treating each group as a separate array-conversion problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse LinkedList in groups of K nodes.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → [6] → [7] → null, K=3

       Group 1: [1,2,3] → [3,2,1]
       Group 2: [4,5,6] → [6,5,4]
       Group 3: [7] → [7]

       Result: [3] → [2] → [1] → [6] → [5] → [4] → [7] → null
```

```
Step-by-step:
Step 1: Reverse first K nodes
[3] → [2] → [1] → [4] → [5] → [6] → [7] → null

Step 2: Reverse next K nodes
[3] → [2] → [1] → [6] → [5] → [4] → [7] → null

Step 3: Last group (K=1) remains same
[3] → [2] → [1] → [6] → [5] → [4] → [7] → null
```

Solution Approach

Concept: Recursively reverse groups.

Algorithm

```
Algorithm REVERSE_K_GROUPS(head, K):
           current = head
           count = 0

           // Check if K nodes exist
           While current != null and count < K:
               current = current.next
               count++

           If count < K:
               Return head

           // Reverse first K nodes
           prev = null
           current = head
           count = 0
           While current != null and count < K:
               next = current.next
               current.next = prev
               prev = current
               current = next
               count++

           // Recursively reverse rest
           head.next = REVERSE_K_GROUPS(current, K)
           Return prev
```

Java Implementation

```
class Solution {
           public Node reverseKGroup(Node head, int k) {
               Node current = head;
               int count = 0;
```

```
            // Check if enough nodes
            while (current != null && count < k) {
                current = current.next;
                count++;
            }
            if (count < k) {
                return head;
            }

            // Reverse first k nodes
            Node prev = null;
            current = head;
            count = 0;
            while (current != null && count < k) {
                Node next = current.next;
                current.next = prev;
                prev = current;
                current = next;
                count++;
            }

            // Recursively reverse rest
            head.next = reverseKGroup(current, k);
            return prev;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n/k) recursion stack

123. Rotate a LinkedList

Understanding the Problem NEW

Rotate a linked list to the right by K positions — e.g. rotating `[1,2,3,4,5]` right by 2 gives `[4,5,1,2,3]`. This is the linked-list equivalent of the array rotation problems (12-13) from Chapter 2.

Brute Force Approach NEW

A beginner might repeatedly move the last node to the front, one rotation at a time, K times — each single rotation requires walking to the end of the list (O(n)), so doing this K times costs O(n×K) total, mirroring the naive 'rotate by one, repeated' approach from the array version.

Why We Optimize NEW

You can do this in a single pass: first find the list's length and connect the tail back to the head temporarily (forming a circle), then walk forward to the position that will become the **new head** (which is `length - (K % length)` steps from the original head — using modulo in case K is larger than the list's length), and break the circle at exactly that point. This achieves the rotation in O(n) time regardless of how large K is, instead of O(n×K), by directly computing the new head position rather than simulating each individual rotation step.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Rotate LinkedList by K positions to the right.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → null, K=2

       Step 1: Find length = 5
       Step 2: K = 2 % 5 = 2
       Step 3: Find new head position = 5 - 2 = 3
       Step 4: Traverse to node at position 3 (node 3)
       Step 5: Break at position 3, connect tail to head

       Original: [1] → [2] → [3] → [4] → [5] → null
                        ↓
```

```
       Break at 3: [1] → [2] → [3] → null
                                [4] → [5] → null
                        ↓
       Connect: [4] → [5] → [1] → [2] → [3] → null

       Result: [4] → [5] → [1] → [2] → [3] → null
```

Solution Approach

Concept: Find new head and reconnect.

Algorithm

```
Algorithm ROTATE_LL(head, k):
           If head == null: Return null

            // Find length
            tail = head
            len = 1
            While tail.next != null:
                tail = tail.next
                len++

            k = k % len
            If k == 0: Return head

            // Find new head
            steps = len - k
            current = head
            For i = 1 to steps-1:
                current = current.next

            // Rotate
            newHead = current.next
            current.next = null
            tail.next = head

            Return newHead
```

Java Implementation

```
class Solution {
           public Node rotateRight(Node head, int k) {
               if (head == null) return null;
               // Find length and tail

              Node tail = head;
              int len = 1;
              while (tail.next != null) {
                  tail = tail.next;
```

```
                len++;
            }

            k = k % len;
            if (k == 0) return head;

            // Find new head
            int steps = len - k;
            Node current = head;
            for (int i = 1; i < steps; i++) {
                current = current.next;
            }

            // Rotate
            Node newHead = current.next;
            current.next = null;
            tail.next = head;

            return newHead;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

124. Flattening of LinkedList

Understanding the Problem NEW

Each node in this linked list can have not just a 'next' pointer, but also a 'child' pointer to a separate sub-list, which can itself have further nested sub-lists (multiple levels deep, like a tree of linked lists). Flatten this whole structure into one single-level linked list, in the correct visiting order.

Brute Force Approach NEW

A beginner might try to handle this with ad-hoc, deeply nested loops trying to track 'depth' manually — this quickly becomes unmanageable and error-prone as the nesting depth grows unpredictably.

Why We Optimize NEW

This structure is naturally suited to a **depth-first, recursive** approach: whenever you encounter a node with a child pointer, recursively flatten that child sub-list *first*, then splice the fully-flattened child list in between the current node and whatever it originally pointed to next, before continuing the traversal. Recursion elegantly handles the arbitrary, unpredictable nesting depth without you needing to manually track how deep you are — each level 'trusts' the recursive call to fully flatten what's below it, similar in spirit to how divide-and-conquer trusts smaller subproblems to already be solved.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Flatten a multilevel LinkedList into a single level.

Visual Diagram

```
Original:
       1 → 2 → 3 → 4 → 5 → 6 → null
           ↓
           7 → 8 → 9 → null
               ↓
               10 → 11 → null

       Flattened:
```

```
1 → 2 → 7 → 8 → 10 → 11 → 9 → 3 → 4 → 5 → 6 → null

Approach:
1. Start from head (1)
2. Traverse to tail (6)
3. Connect tail to child head (7)
4. Continue until all flattened
```

Solution Approach

Concept: Use DFS to flatten recursively.

Algorithm

```
Algorithm FLATTEN_LL(head):
           current = head

            While current != null:
                If current.child != null:
                    // Find tail of child list
                    tail = current.child
                    While tail.next != null:
                        tail = tail.next

                     // Connect tail to current.next
                     tail.next = current.next
                     if current.next != null:
                         current.next.prev = tail

                     // Connect current to child
                     current.next = current.child
                     current.child.prev = current
                     current.child = null

                 current = current.next

            Return head
```

Java Implementation

```
class Node {
           int data;
           Node next;
           Node prev;
           Node child;

            Node(int data) {

              this.data = data;
         }
```

```
}

class Solution {
    public Node flatten(Node head) {
        Node current = head;

        while (current != null) {
            if (current.child != null) {
                // Find tail of child
                Node tail = current.child;
                while (tail.next != null) {
                    tail = tail.next;
                }

                // Connect tail to current.next
                tail.next = current.next;
                if (current.next != null) {
                    current.next.prev = tail;
                }

                // Connect current to child
                current.next = current.child;
                current.child.prev = current;
                current.child = null;
            }
            current = current.next;
        }
        return head;
    }
}
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

125. Clone a Linked List with Random and Next Pointers

Understanding the Problem NEW

Each node has a normal 'next' pointer, but also a 'random' pointer that can point to *any* other node in the list (or null) — arbitrarily, not necessarily in sequence. Create a complete, independent deep copy of this list, correctly replicating both pointer types.

Brute Force Approach NEW

A beginner's first idea might use a hashmap from original nodes to their newly created clone nodes, built in one pass, then a second pass to wire up both the next and random pointers on the clones by looking up the corresponding clone in the hashmap — this works correctly in O(n) time, but costs O(n) extra space for the hashmap.

Why We Optimize NEW

There's a clever O(1)-extra-space trick: first, interleave a clone of each node directly *after* its original (so the list becomes original→clone→original→clone...); this lets you set each clone's 'random' pointer easily, since `original.random.next` is exactly that random target's clone (because every clone was placed right after its original); finally, in a third pass, carefully unweave the interleaved list back into two separate lists — the original (restored) and the fully-wired clone. This achieves the same O(n) time as the hashmap approach, but O(1) extra space, by using the list's own structure as temporary storage instead of an external map.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Clone LinkedList with next and random pointers.

Visual Diagram



```
        ■■■random■■→    ■■■random■■→   ■■■random■■→       ■■■random■■→

       Approach 1: HashMap
       Step 1: Create mapping old → new
       [1] → [1'], [2] → [2'], [3] → [3'], [4] → [4']

       Step 2: Copy random pointers
       [1'].random = map.get([1].random)

       Approach 2: Interleaving
       Step 1: Insert clone nodes
       [1] → [1'] → [2] → [2'] → [3] → [3'] → [4] → [4']

       Step 2: Set random pointers
       [1'].random = [1].random.next

       Step 3: Extract cloned list
       [1'] → [2'] → [3'] → [4']
```

Solution Approach

Concept: Three-step interleaving approach.

Algorithm

```
Algorithm CLONE_RANDOM_LL(head):
           If head == null: Return null

           // Step 1: Insert clone nodes
           current = head
           While current != null:
               clone = Node(current.data)
               clone.next = current.next
               current.next = clone
               current = clone.next

           // Step 2: Set random pointers
           current = head
           While current != null:
               If current.random != null:
                   current.next.random = current.random.next
               current = current.next.next

           // Step 3: Extract cloned list
           dummy = Node(0)
           clone = dummy
           current = head
           While current != null:
               clone.next = current.next
               clone = clone.next
               current.next = clone.next
```

```
                current = current.next

            Return dummy.next
```

Java Implementation

```
class Node {
            int data;
            Node next;
            Node random;

            Node(int data) {
                this.data = data;
            }
        }

        class Solution {
            public Node copyRandomList(Node head) {
                if (head == null) return null;

                // Step 1: Insert clone nodes
                Node current = head;
                while (current != null) {
                    Node clone = new Node(current.data);
                    clone.next = current.next;
                    current.next = clone;
                    current = clone.next;
                }

                // Step 2: Set random pointers
                current = head;
                while (current != null) {
                    if (current.random != null) {
                        current.next.random = current.random.next;
                    }
                    current = current.next.next;
                }

                // Step 3: Extract cloned list
                Node dummy = new Node(0);
                Node clone = dummy;
                current = head;
                while (current != null) {
                    clone.next = current.next;
                    clone = clone.next;
                    current.next = clone.next;
                    current = current.next;
                }

                return dummy.next;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1) (excluding cloned list)

LinkedList Summary — Complexity Comparison

Problem	Type	Time	Space	Key Technique
Basic Operations	Easy	O(1)-O(n)	O(1)	Pointer manipulation
Middle Node	Medium	O(n)	O(1)	Tortoise-Hare
Reverse (Iterative)	Medium	O(n)	O(1)	Three pointers
Reverse (Recursive)	Medium	O(n)	O(n)	Recursion
Detect Cycle	Medium	O(n)	O(1)	Floyd's Algorithm
Cycle Start	Medium	O(n)	O(1)	Floyd's + reset
Palindrome	Medium	O(n)	O(1)	Reverse half
Odd-Even	Medium	O(n)	O(1)	Two lists
Remove Nth	Medium	O(n)	O(1)	Two pointers
Sort LL	Medium	O(n log n)	O(log n)	Merge sort
Sort 0,1,2	Medium	O(n)	O(1)	Dutch flag
Intersection	Medium	O(n+m)	O(1)	Length difference
Add Two Numbers	Medium	O(n)	O(n)	Digit addition
DLL Operations	Medium	O(n)	O(1)	Prev/Next
Reverse K Groups	Hard	O(n)	O(n/k)	Recursive groups
Rotate	Hard	O(n)	O(1)	Find new head
Flattening	Hard	O(n)	O(1)	DFS insertion
Clone with Random	Hard	O(n)	O(1)	Interleaving

Recursion — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

126. Recursive Implementation of Atoi

Understanding the Problem NEW

This is the same Atoi conversion from the Strings chapter (problem 90), but now implemented recursively instead of with an explicit loop — process one character, then recursively handle 'the rest of the string,' carrying along whatever state (sign, accumulated number so far) is needed between calls.

Brute Force Approach NEW

Nothing new to contrast conceptually here — the actual parsing logic (skip whitespace, handle sign, accumulate digits, watch for overflow) is identical to problem 90. The only difference is implementation style: iterative loop vs recursive calls.

Why We Optimize NEW

Recursion is a genuinely useful practice exercise here because it forces you to think about what **state must be threaded through** each recursive call (the running number built so far, the sign, whether you've started reading digits yet) — since there's no shared loop variable to fall back on, every recursive call must explicitly receive and pass along everything it needs. This is good preparation for later problems in this chapter, where recursive state-passing becomes essential (like backtracking problems further down).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert string to integer using recursion.

Visual Diagram

```
Input: " -42"
       Output: -42

       Recursive steps:
       atoi(" -42", 0)
         → skip spaces: atoi("-42", 1)
           → handle sign: atoi("42", 2, sign=-1)
             → parse digits: atoi("2", 3, result=4)
```

```
            → parse digits: atoi("", 4, result=42)
              → return 42 * sign = -42

    Input: "4193 with words"
    atoi("4193 with words", 0)
      → parse digits: atoi("193 with words", 1, result=4)
        → parse digits: atoi("93 with words", 2, result=41)
          → parse digits: atoi("3 with words", 3, result=419)
            → parse digits: atoi(" with words", 4, result=4193)
              → stop at space, return 4193
```

Solution Approach

Concept: Recursively process characters with state tracking.

Algorithm

```
Algorithm ATOI(str, index, sign, result):
          If index >= n:
              Return result * sign

          If str[index] == ' ':
              Return ATOI(str, index+1, sign, result)
          Else If str[index] == '+' or str[index] == '-':
              If result == 0:
                  sign = str[index] == '-' ? -1 : 1
                  Return ATOI(str, index+1, sign, result)
          Else If str[index] >= '0' and str[index] <= '9':
              digit = str[index] - '0'
              If result > (Integer.MAX_VALUE - digit) / 10:
                  Return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE
              result = result * 10 + digit
              Return ATOI(str, index+1, sign, result)
          Else:
              Return result * sign

          Return result * sign
```

Java Implementation

```
class Solution {
          public int myAtoi(String s) {
              return atoi(s, 0, 1, 0);
          }

          private int atoi(String s, int index, int sign, int result) {
              if (index >= s.length()) {
                  return result * sign;
              }
```

```
                char c = s.charAt(index);

                // Skip spaces
                if (c == ' ') {

                    return atoi(s, index + 1, sign, result);
                }

                // Handle sign
                if ((c == '+' || c == '-') && result == 0) {
                    sign = (c == '-') ? -1 : 1;
                    return atoi(s, index + 1, sign, result);
                }

                // Parse digits
                if (c >= '0' && c <= '9') {
                    int digit = c - '0';
                    // Check overflow
                    if (result > (Integer.MAX_VALUE - digit) / 10) {
                        return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE;
                    }
                    result = result * 10 + digit;
                    return atoi(s, index + 1, sign, result);
                }

                // Any other character stops parsing
                return result * sign;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n) recursion stack

127. Pow(x, n)

Understanding the Problem NEW

Compute x raised to the power n (x^n) — e.g. $2^5 = 512$. The obvious method of multiplying x by itself n times works, but the recursive version here is designed to be dramatically faster for large n.

Brute Force Approach NEW

The direct approach: multiply x by itself n times in a loop (or n recursive calls, one multiplication each) — O(n) multiplications for x^n.

Why We Optimize NEW

The key insight: **$x^n = (x^{n/2})^2$** when n is even, and $x \times x^{n-1}$ when n is odd. This means you can compute x^n by first computing $x^{n/2}$ recursively (a problem *half the size*) and then just squaring that one result — instead of n multiplications, you only need about $\log_2 n$ of them, since each recursive call halves the exponent. This 'fast exponentiation' trick (also called exponentiation by squaring) turns O(n) into O(log n), and the same halving-the-problem idea is exactly what made Merge Sort and Binary Search fast earlier in this book.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate x raised to power n using recursion.

Visual Diagram

```
pow(2, 10):
         → pow(2, 5) * pow(2, 5)
           → pow(2, 2) * 2 * pow(2, 2) * 2
             → pow(2, 1) * pow(2, 1) * 2 * pow(2, 1) * pow(2, 1) * 2
               → 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 = 1024

       Optimized:
       pow(2, 10):
         temp = pow(2, 5) = 32
         return temp * temp = 1024

       pow(2, 5):
```

```
          temp = pow(2, 2) = 4
          return temp * temp * 2 = 32

        pow(2, 2):
          temp = pow(2, 1) = 2
          return temp * temp = 4
```

Solution Approach

Concept: Divide and conquer with fast exponentiation.

Algorithm

```
Algorithm POW(x, n):
           If n == 0: Return 1
           If n < 0:
               x = 1 / x
               n = -n

           temp = POW(x, n/2)
           If n % 2 == 0:
               Return temp * temp
           Else:
               Return temp * temp * x
```

Java Implementation

```
class Solution {
           public double myPow(double x, int n) {
               if (n == 0) return 1.0;
               if (n < 0) {
                   x = 1 / x;
                   // Handle Integer.MIN_VALUE overflow
                   if (n == Integer.MIN_VALUE) {
                        return x * myPow(x, -(n + 1));
                   }
                   n = -n;
               }

                 double temp = myPow(x, n / 2);
                 if (n % 2 == 0) {
                     return temp * temp;
                 } else {

                   return temp * temp * x;
               }
           }
       }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(log n)

128. Count Good Numbers

Understanding the Problem NEW

Count how many n-digit numbers exist where every digit at an even index (0, 2, 4...) is one of the 5 even digits (0,2,4,6,8), and every digit at an odd index is one of the 4 prime digits (2,3,5,7). Since this count can be astronomically large, the answer is typically required modulo 10^9+7.

Brute Force Approach NEW

A beginner might try to actually generate every valid n-digit number and count them — completely impossible for even moderately large n, since the count grows exponentially.

Why We Optimize NEW

You don't need to generate anything — this is purely a counting problem, and the count has a simple closed form: since even positions independently have 5 choices and odd positions independently have 4 choices, the total is `5^(number of even positions) × 4^(number of odd positions)`. Computing these two large powers efficiently reuses the fast exponentiation trick from problem 127 (with modular arithmetic applied at each squaring step to keep numbers manageable), turning what looks like a combinatorial explosion into two O(log n) power computations.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count numbers of length n where even positions have even digits and odd positions have prime digits.

Visual Diagram

```
n = 1 (single digit):
       Positions: 0 (even)
       Even positions: can be 0,2,4,6,8 → 5 options
       Total = 5

       n = 2:
       Position 0 (even): 5 options (0,2,4,6,8)
```

```
Position 1 (odd): 4 options (2,3,5,7)
Total = 5 * 4 = 20

n = 3:
Even positions: 5 options each
Odd positions: 4 options each
Total = 5^(ceil(n/2)) * 4^(floor(n/2))

For n=3:
5^2 * 4^1 = 25 * 4 = 100
```

Solution Approach

Concept: Power calculation with modular arithmetic.

Algorithm

```
Algorithm GOOD_NUMBERS(n):
          MOD = 10^9 + 7
          even_positions = (n + 1) / 2
          odd_positions = n / 2
          Return (pow(5, even_positions) * pow(4, odd_positions)) % MOD
```

Java Implementation

```
class Solution {
          private static final int MOD = 1_000_000_007;

          public int countGoodNumbers(long n) {
              long even = (n + 1) / 2;
              long odd = n / 2;
              return (int)(pow(5, even) * pow(4, odd) % MOD);
          }

          private long pow(long base, long exp) {
              long result = 1;
              while (exp > 0) {
                  if (exp % 2 == 1) {
                      result = (result * base) % MOD;
                  }
                  base = (base * base) % MOD;
                  exp /= 2;
              }
              return result;
          }
      }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

129. Sort a Stack Using Recursion

Understanding the Problem NEW

Sort a stack (a last-in-first-out structure) into order, using *only* recursion — no loops, and no other data structure like an array to dump the values into.

Brute Force Approach NEW

A beginner might reach for an array: pop every element off the stack into an array, sort the array (Chapter 1 style), and push the values back — this works, but it isn't really 'using recursion to sort' in the spirit intended, and it uses O(n) extra space for the array plus the cost of a general sort.

Why We Optimize NEW

The trick is to use the **call stack itself** as your only extra storage: recursively pop the top element off, recursively sort the rest of the (now smaller) stack first, and then insert the popped element back into its correct position within the now-sorted remainder — which itself requires a second small recursive helper that pops elements until it finds the right insertion point, inserts, and then pushes the popped elements back on top. This achieves the sort using only the stack itself and the recursion call stack (no separate array), at the cost of a fairly inefficient $O(n^2)$ time complexity, since each insertion may need to temporarily remove several elements.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Sort a stack using only recursion (no loops).

Visual Diagram

```
Stack: [5, 2, 8, 1, 3] (top)
       Sort: [1, 2, 3, 5, 8]

       Recursive steps:
       sort(stack):
         1. Pop top: 3
         2. Sort remaining: [5,2,8,1] → [1,2,5,8]
         3. Insert 3 in sorted order: [1,2,3,5,8]
```

```
      insert(stack, element):
        1. If stack empty or top <= element:
           Push element, return
        2. Pop top: 8
        3. Insert element in remaining
        4. Push 8 back
```

Solution Approach

Concept: Recursively pop and insert in sorted position.

Algorithm

```
Algorithm SORT_STACK(stack):
          If stack is empty: Return
          top = stack.pop()
          SORT_STACK(stack)
          INSERT_SORTED(stack, top)

      Algorithm INSERT_SORTED(stack, element):
          If stack empty or stack.peek() <= element:
              stack.push(element)
              Return
          top = stack.pop()
          INSERT_SORTED(stack, element)
          stack.push(top)
```

Java Implementation

```
class Solution {
          public void sortStack(Stack<Integer> stack) {
              if (stack.isEmpty()) return;

                int top = stack.pop();
                sortStack(stack);
                insertSorted(stack, top);
          }

          private void insertSorted(Stack<Integer> stack, int element) {
              if (stack.isEmpty() || stack.peek() <= element) {
                  stack.push(element);
                  return;
              }

                int top = stack.pop();
                insertSorted(stack, element);

              stack.push(top);
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

130. Reverse a Stack Using Recursion

Understanding the Problem NEW

Reverse the order of elements in a stack using only recursion, again no explicit loops or auxiliary data structures like arrays.

Brute Force Approach NEW

As above, a beginner might use an auxiliary array or a second stack to reverse order and rebuild the stack — correct but doesn't showcase pure recursive manipulation.

Why We Optimize NEW

Recursively pop all elements off the stack (down to empty, via recursive calls), and then, as the recursion unwinds, insert each popped element at the very **bottom** of the stack instead of the top — which itself needs a small recursive helper (pop everything, recursively place the new element at the bottom once the stack is empty, then push everything back on top in the reverse order they were popped). The stack ends up reversed purely by controlling *where* each element gets reinserted during the unwind phase of recursion, with no extra array needed — just the recursion call stack itself as implicit storage.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse a stack using only recursion.

Visual Diagram

```
Stack: [1, 2, 3, 4, 5] (top)
       Reverse: [5, 4, 3, 2, 1]

       Recursive steps:
       reverse(stack):
         1. Pop top: 1
         2. Reverse remaining: [5,4,3,2]
         3. Insert at bottom: [5,4,3,2,1]

       insertAtBottom(stack, element):
         1. If stack empty: Push element, return
```

```
2. Pop top: 2
3. Insert element at bottom of remaining
4. Push 2 back
```

Solution Approach

Concept: Pop all elements and insert at bottom.

Algorithm

```
Algorithm REVERSE_STACK(stack):
           If stack is empty: Return
           top = stack.pop()
           REVERSE_STACK(stack)
           INSERT_AT_BOTTOM(stack, top)

       Algorithm INSERT_AT_BOTTOM(stack, element):
           If stack is empty:
               stack.push(element)
               Return
           top = stack.pop()
           INSERT_AT_BOTTOM(stack, element)
           stack.push(top)
```

Java Implementation

```
class Solution {
           public void reverseStack(Stack<Integer> stack) {
               if (stack.isEmpty()) return;

                 int top = stack.pop();
                 reverseStack(stack);
                 insertAtBottom(stack, top);
           }

           private void insertAtBottom(Stack<Integer> stack, int element) {
               if (stack.isEmpty()) {
                   stack.push(element);
                   return;
               }

                 int top = stack.pop();
                 insertAtBottom(stack, element);
                 stack.push(top);

           }
       }
```

Time & Space Complexity

Time Complexity: $O(n^2)$ | Space Complexity: $O(n)$

131. Generate All Binary Strings

Understanding the Problem NEW

Generate every possible binary string (made of 0s and 1s) of a given length n. E.g. for n=2: `00, 01, 10, 11`.

Brute Force Approach NEW

You could compute this without recursion by counting from 0 to 2^n-1 in binary and formatting each number as an n-digit binary string — this actually works reasonably well, but doesn't build the recursive 'branching decision' pattern that's foundational for the harder backtracking problems ahead in this chapter.

Why We Optimize NEW

The recursive approach frames this as a series of decisions: at each position, you can either place a '0' *or* a '1', and recursively fill in the remaining positions for each choice — this creates a branching 'decision tree' with exactly 2 choices at each of the n levels, producing all 2^n combinations. This 'try each choice, recurse, then move on' pattern — sometimes just called exhaustive recursive generation — is the direct ancestor of the backtracking technique used extensively later in this chapter (subsequences, combination sum, N-Queens, and more), so it's worth internalizing the pattern here even though a non-recursive shortcut exists for this specific problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Generate all binary strings of length n.

Visual Diagram

```
n = 2
       [0,0]
       [0,1]
       [1,0]
       [1,1]

       Recursive tree:
                ""
```

```
              /    \
            "0"    "1"
           / \     / \
         "00" "01" "10" "11"

       n = 3 (8 strings):
       000, 001, 010, 011, 100, 101, 110, 111
```

Solution Approach

Concept: Recursively build strings.

Algorithm

```
Algorithm BINARY_STRINGS(n):
           result = []
           generate("", n, result)
           Return result

       Algorithm generate(current, n, result):
           If length(current) == n:
               result.add(current)
               Return
           generate(current + "0", n, result)
           generate(current + "1", n, result)
```

Java Implementation

```
class Solution {
           public List<String> generateBinaryStrings(int n) {
               List<String> result = new ArrayList<>();
               generate("", n, result);
               return result;
           }

            private void generate(String current, int n, List<String> result) {
                if (current.length() == n) {
                    result.add(current);
                    return;
                }

                 generate(current + "0", n, result);
                 generate(current + "1", n, result);
            }

       }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(2^n * n)

132. Generate Parentheses

Understanding the Problem NEW

Generate every valid combination of n pairs of well-formed parentheses. E.g. for n=2: `(())`, `()()` — but not `)(()(` or other invalid arrangements.

Brute Force Approach NEW

A beginner might generate *all* possible strings of 2n parenthesis characters (open or close at each position, like problem 131's binary strings) and then filter out the invalid ones afterward by checking each generated string — this works but wastes a lot of effort generating and then discarding strings that could never have been valid in the first place.

Why We Optimize NEW

Instead of generating everything and filtering afterward, you can **prune invalid branches early** during the recursive decision-making itself: at each step, only add an opening bracket if you haven't used all n yet, and only add a closing bracket if doing so wouldn't create more closing brackets than opening ones seen so far. This is the essence of **backtracking** — track just enough state (count of opens used, count of closes used) to reject bad choices immediately, rather than generating a complete (possibly invalid) string and checking it only at the very end. This dramatically reduces wasted work compared to generate-then-filter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Generate all valid combinations of n pairs of parentheses.

Visual Diagram

```
n = 1: ["()"]
       n = 2: ["(())", "()()"]
       n = 3: ["((()))", "(()())", "(())()", "()(())", "()()()"]

       Recursive tree (n=2):
                       ""
                       "("
                    /      \
```

```
                "(("      "()"
              /   \         |
            "((("  "(()"    "()("
              |      |        |
            "((()))" "(()())" "()()"
              |
            "((()))"
```

Solution Approach

Concept: Track open and close counts.

Algorithm

```
Algorithm GENERATE_PARENTHESES(n):
           result = []
           generate("", 0, 0, n, result)
           Return result

       Algorithm generate(current, open, close, n, result):
           If open == n and close == n:
               result.add(current)
               Return
           If open < n:
               generate(current + "(", open+1, close, n, result)
           If close < open:
               generate(current + ")", open, close+1, n, result)
```

Java Implementation

```
class Solution {
           public List<String> generateParenthesis(int n) {
               List<String> result = new ArrayList<>();
               generate("", 0, 0, n, result);
               return result;
           }

           private void generate(String current, int open, int close, int n,
List<String> result) {
               if (open == n && close == n) {
                   result.add(current);
                   return;
               }

                 if (open < n) {
                     generate(current + "(", open + 1, close, n, result);
                 }
                 if (close < open) {

                  generate(current + ")", open, close + 1, n, result);
```

```
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(4^n/√n) | Space Complexity: O(4^n/√n)

133. Print All Subsequences / Power Set

Understanding the Problem NEW

Print every possible subsequence (subset, keeping relative order) of an array — including the empty subsequence and the full array itself. For an array of size n, there are always exactly 2^n subsequences (the 'Power Set').

Brute Force Approach NEW

There isn't really a slower alternative to contrast against — generating all subsequences inherently requires visiting all 2^n possibilities, since that's the actual size of the answer. The main risk for beginners is an inefficient implementation (e.g. rebuilding a new list/array at every recursive call instead of reusing one shared list with add/remove).

Why We Optimize NEW

The clean recursive framing, closely related to the binary-strings idea in problem 131: for each element, make a binary decision — **include it, or exclude it** — and recurse on the rest of the array for both choices. Using a single shared temporary list that you append to before recursing and remove from after returning (rather than copying a new list at every level) keeps the extra work per call to O(1), so the total time is proportional to the total output size, $O(2^n \times n)$ (the n factor from copying each subsequence when you record it) — this 'include or exclude' framing recurs constantly for the rest of this chapter's subset/combination problems.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print all subsequences of an array.

Visual Diagram

```
Array: [3, 1, 2]
       Subsequences:
       []
       [3]
       [1]
       [2]
```

```
       [3,1]
       [3,2]
       [1,2]
       [3,1,2]

       Recursive tree:
                    []
                /       \
              [3]           []
             /    \       /    \
           [3,1] [3]    [1]    []
          / \     / \ / \ / \
       [3,1,2][3,1][3,2][3][1,2][1][2][]
```

Solution Approach

Concept: Include or exclude each element.

Algorithm

```
Algorithm SUBSETS(arr):
           result = []
           generate(0, [], arr, result)
           Return result

       Algorithm generate(index, current, arr, result):
           If index == n:
               result.add(new ArrayList(current))
               Return
           // Exclude
           generate(index+1, current, arr, result)
           // Include
           current.add(arr[index])
           generate(index+1, current, arr, result)
           current.remove(current.size()-1)
```

Java Implementation

```
class Solution {
           public List<List<Integer>> subsets(int[] nums) {
               List<List<Integer>> result = new ArrayList<>();
               generate(0, new ArrayList<>(), nums, result);
               return result;
           }

           private void generate(int index, List<Integer> current, int[] nums,
List<List<Integer>> res
               if (index == nums.length) {
                   result.add(new ArrayList<>(current));
                   return;
```

```
            }

            // Exclude
            generate(index + 1, current, nums, result);

            // Include
            current.add(nums[index]);
            generate(index + 1, current, nums, result);
            current.remove(current.size() - 1);
        }
    }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(2^n * n)

134. All Patterns of Subsequences (Print, Count, Check)

Understanding the Problem NEW

This is really three closely related variations built on the exact same include/exclude recursion from problem 133: (1) **print** every subsequence, (2) **count** how many subsequences satisfy some condition, and (3) **check** whether at least one subsequence satisfies some condition.

Brute Force Approach NEW

Treating these as three separate problems requiring three separate solutions would be wasteful — they share almost identical recursive structure.

Why We Optimize NEW

The valuable lesson here is recognizing that the *same* include/exclude recursion tree can be adapted for different purposes just by changing what happens at the base case and what gets returned: printing accumulates and outputs each valid path, counting adds 1 (or the sum of two recursive counts) instead of printing, and existence-checking can **short-circuit** — returning true immediately the moment any valid path is found, instead of continuing to explore the rest of the tree. That short-circuit for existence checks is a genuine optimization over the print/count versions, since you can stop early instead of exploring all 2^n branches.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Learn patterns: print, count, check existence.

Visual Diagram

```
Array: [1, 2, 1], K=2

       Pattern 1: Print all subsequences with sum K
       [2], [1,1]

       Pattern 2: Count subsequences with sum K
       [2], [1,1] → Count = 2
```

```
Pattern 3: Check if any subsequence with sum K exists
Yes (2 exists)

Recursive patterns:
// Print
if (index == n && sum == K) print

// Count
if (index == n) return sum == K ? 1 : 0
return count(index+1, sum) + count(index+1, sum+arr[index])

// Check
if (index == n) return sum == K
if (check(index+1, sum)) return true
if (check(index+1, sum+arr[index])) return true
return false
```

Solution Approach

Concept: Three variations of subsequence generation.

Java Implementation

```
class Solution {
           // Print all subsequences with sum K
           public void printSubsequences(int[] arr, int K) {
               print(0, 0, new ArrayList<>(), arr, K);
           }

           private void print(int index, int sum, List<Integer> current, int[] arr,
int K) {
               if (index == arr.length) {
                   if (sum == K) {
                       System.out.println(current);
                   }
                   return;
               }

                // Exclude
                print(index + 1, sum, current, arr, K);

                // Include
                current.add(arr[index]);
                print(index + 1, sum + arr[index], current, arr, K);
                current.remove(current.size() - 1);
           }

           // Count subsequences with sum K

          public int countSubsequences(int[] arr, int K) {
              return count(0, 0, arr, K);
```

```
        }

        private int count(int index, int sum, int[] arr, int K) {
            if (index == arr.length) {
                return sum == K ? 1 : 0;
            }
            return count(index + 1, sum, arr, K) + count(index + 1, sum +
arr[index], arr, K);
        }

        // Check if any subsequence with sum K exists
        public boolean existsSubsequence(int[] arr, int K) {
            return exists(0, 0, arr, K);
        }

        private boolean exists(int index, int sum, int[] arr, int K) {
            if (index == arr.length) {
                return sum == K;
            }
            if (exists(index + 1, sum, arr, K)) return true;
            if (exists(index + 1, sum + arr[index], arr, K)) return true;
            return false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(n)

135. Count All Subsequences with Sum K

Understanding the Problem NEW

Count how many subsequences of the array have elements summing to exactly K.

Brute Force Approach NEW

This is direct: use the include/exclude recursion, and at each 'leaf' of the decision tree (once every element has been decided), check if the accumulated sum equals K — this correctly counts all valid subsequences, but explores the full 2^n tree regardless of whether a sum could still possibly reach K.

Why We Optimize NEW

A useful, cheap optimization here is to prune early: if the running sum has already exceeded K partway through building a subsequence (assuming non-negative numbers), there's no point continuing to include more elements in that branch — you can stop exploring it immediately rather than waiting until the base case. This kind of early pruning doesn't change the theoretical worst-case complexity (still $O(2^n)$ in the worst case) but can meaningfully cut down real-world running time by avoiding provably-doomed branches — a preview of the pruning ideas used more aggressively in problems 137-141.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count number of subsequences with sum equal to K.

Visual Diagram

```
Array: [1, 2, 1], K=2
      Subsequences with sum 2:
      [2] (index 1)
      [1,1] (index 0,2)
      Count = 2

      Recursive approach:
      count(0, 0):
        count(1, 0):
          count(2, 0):
```

```
            count(3, 0): sum=0 != 2 → 0
            count(3, 1): sum=1 != 2 → 0
          count(2, 1):
            count(3, 1): sum=1 != 2 → 0
            count(3, 2): sum=2 == 2 → 1
        count(1, 1):
          count(2, 1):
            count(3, 1): sum=1 != 2 → 0
            count(3, 2): sum=2 == 2 → 1
          count(2, 2):
            count(3, 2): sum=2 == 2 → 1
            count(3, 3): sum=3 != 2 → 0
      Total = 2
```

Solution Approach

Concept: Standard subsequence count with sum.

Algorithm

```
Algorithm COUNT_SUM_K(arr, K):
          Return count(0, 0, arr, K)

      Algorithm count(index, sum, arr, K):
          If index == n:
              Return sum == K ? 1 : 0
          // Exclude
          exclude = count(index+1, sum, arr, K)
          // Include
          include = count(index+1, sum+arr[index], arr, K)
          Return exclude + include
```

Java Implementation

```
class Solution {
          public int countSubsequencesWithSumK(int[] arr, int K) {
              return count(0, 0, arr, K);
          }

          private int count(int index, int sum, int[] arr, int K) {
              if (index == arr.length) {
                  return sum == K ? 1 : 0;
              }

                 // Exclude current element

              int exclude = count(index + 1, sum, arr, K);
              // Include current element
              int include = count(index + 1, sum + arr[index], arr, K);
```

```
            return exclude + include;
        }
    }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(n)

136. Check if There Exists a Subsequence with Sum K

Understanding the Problem NEW

Check whether *any* subsequence of the array sums to exactly K (a yes/no answer, not a count).

Brute Force Approach NEW

Using the same include/exclude recursion and checking every leaf for a matching sum works, but explores the entire 2^n tree even after finding an answer, if you don't explicitly stop.

Why We Optimize NEW

Since you only need to know if *any* valid subsequence exists, the recursive calls should **return true immediately** the moment one branch reports success, rather than dutifully continuing to explore every remaining branch. This 'short-circuit on success' idea (mentioned in problem 134) can turn a full $O(2^n)$ exploration into something that finishes much earlier in practice, especially if a valid subsequence appears early in the search order — even though the worst case (no valid subsequence exists at all) still requires exploring everything to be sure.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if any subsequence sums to K.

Visual Diagram

```
Array: [1, 2, 1], K=3
       Subsequence [1,2] sum=3 exists → true

       Early termination optimization:
       exists(0, 0):
         exists(1, 0):
           exists(2, 0):
             exists(3, 0): false
             exists(3, 1): false
```

```
          exists(2, 1):
            exists(3, 1): false
            exists(3, 2): false
        exists(1, 1):
          exists(2, 1):
            exists(3, 1): false
            exists(3, 2): false
          exists(2, 2):
            exists(3, 2): false
            exists(3, 3): true → return true
```

Solution Approach

Concept: Return early when found.

Algorithm

```
Algorithm EXISTS_SUM_K(arr, K):
          Return exists(0, 0, arr, K)

      Algorithm exists(index, sum, arr, K):
          If index == n:
              Return sum == K
          If exists(index+1, sum, arr, K):
              Return true
          If exists(index+1, sum+arr[index], arr, K):
              Return true
          Return false
```

Java Implementation

```
class Solution {
          public boolean existsSubsequenceWithSumK(int[] arr, int K) {
              return exists(0, 0, arr, K);
          }

          private boolean exists(int index, int sum, int[] arr, int K) {
              if (index == arr.length) {
                  return sum == K;
              }

                // Exclude
                if (exists(index + 1, sum, arr, K)) return true;
                // Include
                if (exists(index + 1, sum + arr[index], arr, K)) return true;
                return false;

          }
     }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(n)

137. Combination Sum

Understanding the Problem NEW

Find all unique combinations of numbers from a given array that sum to a target, where the **same number can be reused multiple times**. E.g. array `[2,3,6,7]`, target 7 → `[7]` and `[2,2,3]` are both valid.

Brute Force Approach NEW

Trying to generate every possible combination without any structure risks both missing valid combinations and generating massive numbers of duplicate combinations (like `[2,3,2]` and `[2,2,3]` being counted as different when they're really the same set).

Why We Optimize NEW

The key structural decision that avoids duplicates: at each recursive step, when you choose to include an element again, you allow the *same index* to be revisited (permitting reuse), but when you choose to move to a *different* element, you only ever move forward in the array (never backward) — this ordering rule guarantees every valid combination is generated in exactly one canonical order, never as a rearranged duplicate. Combined with early pruning (stop exploring a branch once the running sum exceeds the target, assuming positive numbers), this keeps the search efficient despite allowing repeated elements.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all combinations that sum to target (unlimited use of elements).

Visual Diagram

```
Array: [2, 3, 5], Target: 8

       Combinations:
       [2,2,2,2] → 8 ✓
       [2,3,3] → 8 ✓
       [3,5] → 8 ✓

       Recursive tree:
```

```
                    []
              /      |      \
            [2]     [3]    [5]
           / \     / \     / \
         [2,2] [2,3] [3,3] [3,5] [5,5]
        / \    / \     / \    / \
      [2,2,2] [2,2,3] [2,3,3] [3,3,3] ...
```

Solution Approach

Concept: Allow repeated elements, maintain index.

Algorithm

```
Algorithm COMBINATION_SUM(candidates, target):
           result = []
           Arrays.sort(candidates)
           findCombinations(0, target, [], result, candidates)
           Return result

       Algorithm findCombinations(index, remaining, current, result, candidates):
           If remaining == 0:
               result.add(new ArrayList(current))
               Return
           For i = index to n-1:
               If candidates[i] > remaining: Break
               current.add(candidates[i])
               findCombinations(i, remaining-candidates[i], current, result,
candidates)
               current.remove(current.size()-1)
```

Java Implementation

```
class Solution {
           public List<List<Integer>> combinationSum(int[] candidates, int target)
{
               List<List<Integer>> result = new ArrayList<>();
               Arrays.sort(candidates);
               findCombinations(0, target, new ArrayList<>(), result, candidates);
               return result;
           }

           private void findCombinations(int index, int remaining, List<Integer>
current,
                                         List<List<Integer>> result, int[]
candidates) {
               if (remaining == 0) {
                   result.add(new ArrayList<>(current));
                   return;
               }
```

```
                for (int i = index; i < candidates.length; i++) {

                    if (candidates[i] > remaining) break;

                    current.add(candidates[i]);
                    findCombinations(i, remaining - candidates[i], current, result,
candidates);
                    current.remove(current.size() - 1);
                }
            }
        }
```

Time & Space Complexity

Time Complexity: O(2^(target/min)) | Space Complexity: O(target/min)

138. Combination Sum-II

Understanding the Problem NEW

Same as Combination Sum, but now each number in the array can be used **at most once**, and the array may contain duplicate values that still shouldn't produce duplicate combinations in the output.

Brute Force Approach NEW

Generating all combinations without any duplicate-avoidance strategy and then filtering out duplicate combinations afterward (e.g. via a hashset of sorted tuples) works, but wastes significant effort generating and then discarding a large number of duplicate branches.

Why We Optimize NEW

The standard fix: **sort the array first**, then during the recursive exploration, whenever you're deciding whether to include the current element at a given recursion depth, skip over it if it's identical to the element you just skipped at the *same* depth (not the same value used earlier at a different depth — that's fine). This 'skip duplicates at the same recursion level, after sorting' pattern is exactly the same duplicate-avoidance trick used for 3-Sum and 4-Sum in the Arrays chapter, just applied to a recursive search instead of a two-pointer sweep — and it prevents duplicate combinations from ever being generated in the first place, rather than filtering them out after the fact.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find unique combinations (no repeated elements, each used once).

Visual Diagram

```
Array: [10,1,2,7,6,1,5], Target: 8
       Unique combinations:
       [1,1,6]
       [1,2,5]
       [1,7]
       [2,6]
```

```
       Recursive approach:
       1. Sort: [1,1,2,5,6,7,10]
       2. Skip duplicates at same level
       3. Each element can be used once

       Combination [1,1,6]:
       Index 0: use 1
       Index 1: use 1 (skip duplicate)
       Index 5: use 6
```

Solution Approach

Concept: Sort and skip duplicates.

Algorithm

```
Algorithm COMBINATION_SUM_II(candidates, target):
           Arrays.sort(candidates)
           result = []
           findCombinations(0, target, [], result, candidates)
           Return result

       Algorithm findCombinations(index, remaining, current, result, candidates):
           If remaining == 0:
               result.add(new ArrayList(current))
               Return
           For i = index to n-1:
               If i > index and candidates[i] == candidates[i-1]:
                   Continue
               If candidates[i] > remaining: Break
               current.add(candidates[i])
               findCombinations(i+1, remaining-candidates[i], current, result,
candidates)
               current.remove(current.size()-1)
```

Java Implementation

```
class Solution {
           public List<List<Integer>> combinationSum2(int[] candidates, int target)
{
               List<List<Integer>> result = new ArrayList<>();
               Arrays.sort(candidates);
               findCombinations(0, target, new ArrayList<>(), result, candidates);
               return result;
           }

           private void findCombinations(int index, int remaining, List<Integer>
current,
                                         List<List<Integer>> result, int[]
candidates) {
```

```
                if (remaining == 0) {
                    result.add(new ArrayList<>(current));

                    return;
                }

                for (int i = index; i < candidates.length; i++) {
                    // Skip duplicates
                    if (i > index && candidates[i] == candidates[i - 1]) continue;
                    if (candidates[i] > remaining) break;

                    current.add(candidates[i]);
                    findCombinations(i + 1, remaining - candidates[i], current,
result, candidates);
                    current.remove(current.size() - 1);
                }
            }
        }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(n)

139. Subset Sum-I

Understanding the Problem NEW

Find all possible sums achievable by any subset of the array (not the subsets themselves, just their sums). This is a direct application of the include/exclude recursion from problem 133, now tracking sums instead of the elements.

Brute Force Approach NEW

There isn't a slower alternative to contrast — this problem inherently requires exploring all 2^n subsets to enumerate every possible sum, since any one of them could produce a sum not achievable another way.

Why We Optimize NEW

Straightforward include/exclude recursion, adding the current element's value to a running sum when included, and recording the final sum at each leaf of the tree — $O(2^n)$ time since there are that many subsets to enumerate. The main practical note: if duplicate sums are undesirable in the output, sorting the resulting list of sums at the end (or using a set) removes them, though the array itself has no ordering requirement here (unlike problem 140).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all possible subset sums.

Visual Diagram

```
Array: [3, 1, 2]
       Subset sums:
       [] → 0
       [3] → 3
       [1] → 1
       [2] → 2
       [3,1] → 4
       [3,2] → 5
       [1,2] → 3
       [3,1,2] → 6

       Sorted sums: [0, 1, 2, 3, 3, 4, 5, 6]
```

```
       Recursive:
       subsetSum(0, 0):
         exclude: subsetSum(1, 0)
           exclude: subsetSum(2, 0)
             exclude: subsetSum(3, 0) → add 0
             include: subsetSum(3, 3) → add 3
           include: subsetSum(2, 1)
             exclude: subsetSum(3, 1) → add 1
             include: subsetSum(3, 4) → add 4
         include: subsetSum(1, 3)
           exclude: subsetSum(2, 3)
             exclude: subsetSum(3, 3) → add 3
             include: subsetSum(3, 6) → add 6
           include: subsetSum(2, 4)
             exclude: subsetSum(3, 4) → add 4
             include: subsetSum(3, 7) → add 7
```

Solution Approach

Concept: Include/exclude elements.

Algorithm

```
Algorithm SUBSET_SUMS(arr):
           result = []
           generate(0, 0, arr, result)
           Collections.sort(result)
           Return result

       Algorithm generate(index, sum, arr, result):
           If index == n:
               result.add(sum)
               Return
           // Exclude
           generate(index+1, sum, arr, result)
           // Include
           generate(index+1, sum+arr[index], arr, result)
```

Java Implementation

```
class Solution {
           public List<Integer> subsetSums(int[] arr) {

              List<Integer> result = new ArrayList<>();
              generate(0, 0, arr, result);
              Collections.sort(result);
              return result;
          }
```

```
          private void generate(int index, int sum, int[] arr, List<Integer>
result) {
              if (index == arr.length) {
                  result.add(sum);
                  return;
              }

              // Exclude
              generate(index + 1, sum, arr, result);
              // Include
              generate(index + 1, sum + arr[index], arr, result);
          }
      }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(2^n)

140. Subset Sum-II

Understanding the Problem NEW

Same idea as Subset Sum-I, but now find the unique *subsets themselves* (not just sums), where the array may contain duplicate values that shouldn't produce duplicate subsets in the output.

Brute Force Approach NEW

Generate every subset via plain include/exclude recursion and then deduplicate the results afterward using a hashset of sorted subsets — correct, but wastes effort generating duplicates before throwing them away.

Why We Optimize NEW

Exactly the same fix as Combination Sum-II (problem 138): sort the array first, then at each recursion level, skip over duplicate values at that *same* level (not across different levels) to avoid ever generating the same subset twice. This is the third appearance of the 'sort, then skip same-level duplicates' pattern in this chapter — worth recognizing as one reusable technique rather than three separate ones.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find unique subsets (Power Set without duplicates).

Visual Diagram

```
Array: [1, 2, 2]
       Unique subsets:
       []
       [1]
       [2]
       [1,2]
       [2,2]
       [1,2,2]

       Duplicate handling:
       At each level, skip duplicate elements
       Index 0: use 1, skip 2 duplicate
```

```
       Index 1: use first 2
       Index 2: use second 2
```

Solution Approach

Concept: Sort and skip duplicates at each level.

Algorithm

```
Algorithm UNIQUE_SUBSETS(arr):
           Arrays.sort(arr)
           result = []
           generate(0, [], arr, result)
           Return result

       Algorithm generate(index, current, arr, result):
           result.add(new ArrayList(current))
           For i = index to n-1:
               If i > index and arr[i] == arr[i-1]:
                   Continue
               current.add(arr[i])
               generate(i+1, current, arr, result)
               current.remove(current.size()-1)
```

Java Implementation

```
class Solution {
           public List<List<Integer>> subsetsWithDup(int[] nums) {
               List<List<Integer>> result = new ArrayList<>();
               Arrays.sort(nums);
               generate(0, new ArrayList<>(), nums, result);
               return result;
           }

           private void generate(int index, List<Integer> current, int[] nums,
List<List<Integer>> res
               result.add(new ArrayList<>(current));

                 for (int i = index; i < nums.length; i++) {
                     if (i > index && nums[i] == nums[i - 1]) continue;

                    current.add(nums[i]);
                    generate(i + 1, current, nums, result);
                    current.remove(current.size() - 1);

               }
           }
       }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(2^n * n)

141. Combination Sum - III

Understanding the Problem NEW

Find all combinations of exactly K numbers (each from 1-9, each used at most once) that sum to a given target n. E.g. k=3, n=7 → `[1,2,4]`.

Brute Force Approach NEW

Since the numbers are fixed to 1-9, you could enumerate every possible k-sized combination from those 9 digits and check which ones sum to n — with only 9 possible digits this is small enough to brute-force reasonably, but it's still cleaner to build the combination incrementally with pruning.

Why We Optimize NEW

This is a more constrained version of Combination Sum (problem 137): because digits can't repeat and only 9 exist, and you need exactly k of them, you can prune very aggressively during the recursive search — stop exploring a branch the moment the running sum exceeds n, the moment you've already picked k numbers (whether or not the sum matches, in which case you check and record or discard), or the moment there aren't enough remaining digits left to reach k total picks. These early exits keep the search fast despite the exhaustive-looking structure, since the small fixed digit range (1-9) means the search tree is naturally shallow and narrow.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all combinations of k numbers that sum to n (1-9 each used once).

Visual Diagram

```
k = 3, n = 7
       Combinations:
       [1,2,4] → 7 ✓

       k = 3, n = 9
       [1,2,6]
       [1,3,5]
       [2,3,4]
```

```
       Recursive:
       combination(1, 9, 3):
         i=1: combination(2, 8, 2)
           i=2: combination(3, 6, 1)
             i=3: combination(4, 3, 0) → not valid
             i=4: combination(5, 2, 0) → not valid
             i=5: combination(6, 1, 0) → not valid
             i=6: combination(7, 0, 0) → [1,2,6] ✓
           i=3: combination(4, 5, 1)
             i=4: combination(5, 1, 0) → not valid
             i=5: combination(6, 0, 0) → [1,3,5] ✓
         i=2: combination(3, 7, 2)
           i=3: combination(4, 4, 1)
             i=4: combination(5, 0, 0) → [2,3,4] ✓
```

Solution Approach

Concept: Generate combinations with fixed length.

Algorithm

```
Algorithm COMBINATION_SUM_3(k, n):
           result = []
           findCombinations(1, k, n, [], result)
           Return result

       Algorithm findCombinations(start, remaining_k, remaining_sum, current,
result):
           If remaining_k == 0 and remaining_sum == 0:
               result.add(new ArrayList(current))
               Return
           If remaining_k == 0 or remaining_sum < 0:
               Return
           For i = start to 9:
               If i > remaining_sum: Break
               current.add(i)
               findCombinations(i+1, remaining_k-1, remaining_sum-i, current,
result)
               current.remove(current.size()-1)
```

Java Implementation

```
class Solution {
           public List<List<Integer>> combinationSum3(int k, int n) {
               List<List<Integer>> result = new ArrayList<>();
               findCombinations(1, k, n, new ArrayList<>(), result);
               return result;
           }
```

```
            private void findCombinations(int start, int remainingK, int
remainingSum,
                                          List<Integer> current,
List<List<Integer>> result) {
                if (remainingK == 0 && remainingSum == 0) {
                    result.add(new ArrayList<>(current));
                    return;
                }
                if (remainingK == 0 || remainingSum < 0) {
                    return;
                }

                for (int i = start; i <= 9; i++) {
                    if (i > remainingSum) break;

                    current.add(i);
                    findCombinations(i + 1, remainingK - 1, remainingSum - i,
current, result);
                    current.remove(current.size() - 1);
                }
            }
        }
```

Time & Space Complexity

Time Complexity: O(9!/(9-k)!) | Space Complexity: O(k)

142. Letter Combinations of a Phone Number

Understanding the Problem NEW

Given a string of digits (like an old phone keypad, e.g. '23'), where each digit maps to a set of letters (2→abc, 3→def), generate every possible letter combination the digit string could represent. E.g. '23' → `ad, ae, af, bd, be, bf, cd, ce, cf`.

Brute Force Approach NEW

You could compute the total number of combinations in advance and try to generate them with nested loops, one loop per digit — this works but becomes unwieldy since the number of digits (and therefore nested loops) isn't fixed in advance.

Why We Optimize NEW

Recursion naturally handles a variable, unknown-in-advance number of 'positions to fill': at each recursive call, you're deciding which letter to place at the *current* digit's position, trying every letter that digit maps to, and recursing to handle the remaining digits — building up one combination per full root-to-leaf path through the recursion tree. This avoids needing to write a different number of nested loops depending on the input string's length, letting recursion adapt naturally to any length input.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Generate all letter combinations from phone digits.

Visual Diagram

```
Digits: "23"
       2 → a,b,c
       3 → d,e,f

       Combinations:
       ad, ae, af, bd, be, bf, cd, ce, cf

       Recursive tree:
```

```
             ""
          /  |   \
         a   b    c
        /|\ /|\ /|\
       ad ae af bd be bf cd ce cf
```

Solution Approach

Concept: Map digits to letters, backtrack.

Algorithm

```
Algorithm LETTER_COMBINATIONS(digits):
           If digits empty: Return []
           map = {"2":"abc", "3":"def", ...}
           result = []
           generate(0, "", digits, map, result)
           Return result

       Algorithm generate(index, current, digits, map, result):
           If index == len(digits):
               result.add(current)
               Return
           For each char in map[digits[index]]:
               generate(index+1, current+char, digits, map, result)
```

Java Implementation

```
class Solution {
           public List<String> letterCombinations(String digits) {
               if (digits.isEmpty()) return new ArrayList<>();

                 String[] mapping = {"", "", "abc", "def", "ghi", "jkl", "mno",
"pqrs", "tuv", "wxyz"};
                 List<String> result = new ArrayList<>();
                 generate(0, "", digits, mapping, result);
                 return result;
            }

            private void generate(int index, String current, String digits,
                                  String[] mapping, List<String> result) {
                if (index == digits.length()) {
                    result.add(current);
                    return;
                }

                 int digit = digits.charAt(index) - '0';
                 for (char c : mapping[digit].toCharArray()) {

                  generate(index + 1, current + c, digits, mapping, result);
```

```
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(4^n) | Space Complexity: O(4^n)

143. Palindrome Partitioning

Understanding the Problem NEW

Partition a string into pieces such that every piece is itself a palindrome, and find every way to do this. E.g. `"aab"` → `["a","a","b"]` and `["aa","b"]` are both valid partitions.

Brute Force Approach NEW

A beginner might try to generate every possible way to split the string into pieces first (all possible partition boundaries) and then filter out the ones where some piece isn't a palindrome — this generates many invalid partitions before rejecting them.

Why We Optimize NEW

Instead, use backtracking with early pruning: at each recursive step, try every possible *prefix* of the remaining string, but only recurse further if that prefix is itself already a valid palindrome (checked immediately, before committing to it) — invalid prefixes are rejected right away rather than being included in a full partition that gets checked only at the very end. This is the same 'prune invalid choices as early as possible' philosophy from Generate Parentheses (problem 132), applied here to partition boundaries instead of bracket counts.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Partition string into palindromic substrings.

Visual Diagram

```
Input: "aab"
       Partitions:
       ["a", "a", "b"]
       ["aa", "b"]

       Recursive tree:
                   "aab"
                 /       \
               "a"       "aa"
              /   \        |
```

```
          "a"    "ab"     "b"
          /        |
        "b"      "b"
```

Solution Approach

Concept: Check palindrome and partition.

Algorithm

```
Algorithm PALINDROME_PARTITION(s):
           result = []
           partition(0, [], s, result)
           Return result

       Algorithm partition(start, current, s, result):
           If start == n:
               result.add(new ArrayList(current))
               Return
           For end = start to n-1:
               If isPalindrome(s, start, end):
                   current.add(s.substring(start, end+1))
                   partition(end+1, current, s, result)
                   current.remove(current.size()-1)

       Algorithm isPalindrome(s, left, right):
           While left < right:
               If s[left] != s[right]: Return false
               left++; right--
           Return true
```

Java Implementation

```
class Solution {
           public List<List<String>> partition(String s) {
               List<List<String>> result = new ArrayList<>();
               partition(0, new ArrayList<>(), s, result);
               return result;
           }

            private void partition(int start, List<String> current, String s,
                                   List<List<String>> result) {

               if (start == s.length()) {
                   result.add(new ArrayList<>(current));
                   return;
               }

               for (int end = start; end < s.length(); end++) {
                   if (isPalindrome(s, start, end)) {
```

```
                    current.add(s.substring(start, end + 1));
                    partition(end + 1, current, s, result);
                    current.remove(current.size() - 1);
                }
            }
        }

        private boolean isPalindrome(String s, int left, int right) {
            while (left < right) {
                if (s.charAt(left) != s.charAt(right)) return false;
                left++;
                right--;
            }
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * 2^n) | Space Complexity: O(n)

144. Word Search

Understanding the Problem NEW

Given a 2D grid of letters and a target word, determine if the word can be traced out by moving between horizontally/vertically adjacent cells, using each cell at most once in a single trace.

Brute Force Approach NEW

Trying every possible starting cell and blindly exploring every direction without remembering which cells are already used in the current path could revisit the same cell twice within one trace, producing incorrect matches.

Why We Optimize NEW

This needs **DFS with backtracking and a 'visited' marker**: from each candidate starting cell matching the word's first letter, recursively try all four directions for the next letter, temporarily marking the current cell as visited (so it can't be reused later in this same path) — and crucially, **un-marking** it again once that recursive branch is done exploring (this is the 'backtrack' step), so the cell remains available for other paths that don't go through it. Forgetting to un-mark cells after backtracking out of a branch is the single most common bug in this kind of grid search — the search must restore the grid's original state before trying the next possibility.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find if word exists in a grid by adjacent cells.

Visual Diagram

```
Grid:
       [['A','B','C','E'],
        ['S','F','C','S'],
        ['A','D','E','E']]

       Word: "ABCCED"
       Path: (0,0)→(0,1)→(0,2)→(1,2)→(2,2)→(2,3)
       A→B→C→C→E→D
```

```
Search:
Start at (0,0) 'A'
  → (0,1) 'B'
    → (0,2) 'C'
      → (1,2) 'C'
        → (2,2) 'E'
          → (2,3) 'D' ✓
```

Solution Approach

Concept: DFS with backtracking and visited marking.

Algorithm

```
Algorithm WORD_SEARCH(board, word):
           For each cell:
               If board[i][j] == word[0] and dfs(i, j, 0):
                   Return true
           Return false

       Algorithm dfs(i, j, index):
           If index == len(word): Return true
           If out of bounds: Return false
           If board[i][j] != word[index]: Return false

            char = board[i][j]
            board[i][j] = '#' // Mark visited
            // Check all 4 directions
            if dfs(i+1,j,index+1) or dfs(i-1,j,index+1) or
               dfs(i,j+1,index+1) or dfs(i,j-1,index+1):
                Return true
            board[i][j] = char // Backtrack
            Return false
```

Java Implementation

```
class Solution {
           public boolean exist(char[][] board, String word) {
               int m = board.length, n = board[0].length;

                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         if (board[i][j] == word.charAt(0) && dfs(i, j, 0, board,
word)) {
                             return true;
                         }
                     }

             }
```

```
                return false;
            }

            private boolean dfs(int i, int j, int index, char[][] board, String word)
{
                if (index == word.length()) return true;
                if (i < 0 || i >= board.length || j < 0 || j >= board[0].length)
return false;
                if (board[i][j] != word.charAt(index)) return false;

                char temp = board[i][j];
                board[i][j] = '#';

                boolean found = dfs(i+1, j, index+1, board, word) ||
                                dfs(i-1, j, index+1, board, word) ||
                                dfs(i, j+1, index+1, board, word) ||
                                dfs(i, j-1, index+1, board, word);

                board[i][j] = temp;
                return found;
            }
        }
```

Time & Space Complexity

Time Complexity: O(m*n*4^L) | Space Complexity: O(L)

145. N Queen

Understanding the Problem NEW

Place N queens on an N×N chessboard such that no two queens attack each other (no shared row, column, or diagonal). Find all such arrangements.

Brute Force Approach NEW

Trying every possible placement of N queens among all N^2 squares (checking all $C(N^2, N)$ combinations for validity) is astronomically expensive even for modest N.

Why We Optimize NEW

The key structural insight: since no two queens can share a row, you only ever need to place **exactly one queen per row** — this alone eliminates a huge number of invalid possibilities by construction, rather than generating them and checking afterward. Then, placing queens row by row, at each row you only try columns that don't conflict with any queen already placed in an earlier row (checking column and both diagonals) — if a row has no valid column, you **backtrack** to the previous row and try a different column there instead. This 'place one at a time, backtrack on failure' approach explores only partially-valid board states, pruning the search space dramatically compared to generating full boards and validating them after the fact.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Place N queens on NxN board so no two attack each other.

Visual Diagram

```
N = 4 Queens:
       Q . . .
       . . Q .
       . . . Q
       . Q . .

       Board:
       0: Q . . .
       1: . . Q .
       2: . . . Q
```

```
3: . Q . .

Backtracking:
Row 0: Place at col 0
Row 1: Try col 0 (invalid), 1 (invalid), 2 (valid)
Row 2: Try col 0 (invalid), 1 (invalid), 2 (invalid), 3 (valid)
Row 3: Try col 0 (invalid), 1 (valid) ✓
```

Solution Approach

Concept: Place queens row by row with conflict checking.

Algorithm

```
Algorithm N_QUEEN(n):
           board = new char[n][n]
           result = []
           solve(0, board, result)
           Return result

       Algorithm solve(row, board, result):
           If row == n:
               result.add(createBoard(board))
               Return
           For col = 0 to n-1:
               If isSafe(row, col, board):
                   board[row][col] = 'Q'
                   solve(row+1, board, result)
                   board[row][col] = '.'

       Algorithm isSafe(row, col, board):
           Check column
           Check upper-left diagonal
           Check upper-right diagonal
```

Java Implementation

```
class Solution {
           public List<List<String>> solveNQueens(int n) {
               char[][] board = new char[n][n];
               for (int i = 0; i < n; i++) {
                   Arrays.fill(board[i], '.');
               }
               List<List<String>> result = new ArrayList<>();
               solve(0, board, result);

               return result;
           }

           private void solve(int row, char[][] board, List<List<String>> result) {
```

```
            if (row == board.length) {
                result.add(construct(board));
                return;
            }

            for (int col = 0; col < board.length; col++) {
                if (isSafe(row, col, board)) {
                    board[row][col] = 'Q';
                    solve(row + 1, board, result);
                    board[row][col] = '.';
                }
            }
        }

        private boolean isSafe(int row, int col, char[][] board) {
            // Check column
            for (int i = 0; i < row; i++) {
                if (board[i][col] == 'Q') return false;
            }

            // Check upper-left diagonal
            for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
                if (board[i][j] == 'Q') return false;
            }

            // Check upper-right diagonal
            for (int i = row - 1, j = col + 1; i >= 0 && j < board.length; i--,
j++) {
                if (board[i][j] == 'Q') return false;
            }

            return true;
        }

        private List<String> construct(char[][] board) {
            List<String> result = new ArrayList<>();
            for (char[] row : board) {
                result.add(new String(row));
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n!) | Space Complexity: O(n²)

146. Rat in a Maze

Understanding the Problem NEW

Find a path for a rat to travel from the top-left corner to the bottom-right corner of a maze (grid), moving only through open cells, and avoiding cells already visited in the current path.

Brute Force Approach NEW

Similar to Word Search, blindly trying all four directions from every cell without tracking visited cells in the current path risks infinite loops (bouncing back and forth between two cells) or invalid paths that revisit the same cell.

Why We Optimize NEW

Same DFS-with-backtracking-and-visited-marking pattern as Word Search (problem 144): try each direction from the current cell, mark the cell as visited before recursing, and un-mark it when backtracking out of a dead end so other paths can still use that cell. This structural template — mark, recurse, unmark — is the standard shape for essentially all grid/maze pathfinding problems, and it's worth explicitly recognizing that problems 144 and 146 are really the same technique applied to two differently-flavoured problems.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find path for rat from (0,0) to (n-1,n-1).

Visual Diagram

```
Maze:
        1 0 0 0
        1 1 0 1
        1 1 0 0
        0 1 1 1

        Path: D D R R D D
        (0,0)→(1,0)→(2,0)→(2,1)→(2,2)→(3,2)→(3,3)

        Backtracking:
        Start: (0,0)
```

```
          → Down: (1,0)
            → Down: (2,0)
              → Right: (2,1)
                → Right: (2,2)
                  → Down: (3,2)
                     → Down: (3,3) ✓
```

Solution Approach

Concept: DFS with movement directions.

Algorithm

```
Algorithm RAT_IN_MAZE(maze):
            n = maze.length
            path = []
            If solve(0, 0, maze, path):
                Return path
            Return []

        Algorithm solve(i, j, maze, path):
            If i == n-1 and j == n-1:
                return true
            If not safe: return false

            // Try all directions: D, R, U, L
            For each direction:
                path.add(dir)
                If solve(new_i, new_j, maze, path):
                    return true
                path.remove(path.size()-1)
            return false
```

Java Implementation

```
class Solution {
            public List<String> findPath(int[][] maze) {
                int n = maze.length;
                List<String> result = new ArrayList<>();
                if (maze[0][0] == 0) return result;

                  boolean[][] visited = new boolean[n][n];
                  solve(0, 0, maze, visited, "", result);
                  return result;

           }

           private void solve(int i, int j, int[][] maze, boolean[][] visited,
                              String path, List<String> result) {
               int n = maze.length;
```

```
            if (i == n - 1 && j == n - 1) {
                result.add(path);
                return;
            }

            visited[i][j] = true;

            // Down
            if (i + 1 < n && !visited[i+1][j] && maze[i+1][j] == 1) {
                solve(i + 1, j, maze, visited, path + "D", result);
            }
            // Right
            if (j + 1 < n && !visited[i][j+1] && maze[i][j+1] == 1) {
                solve(i, j + 1, maze, visited, path + "R", result);
            }
            // Up
            if (i - 1 >= 0 && !visited[i-1][j] && maze[i-1][j] == 1) {
                solve(i - 1, j, maze, visited, path + "U", result);
            }
            // Left
            if (j - 1 >= 0 && !visited[i][j-1] && maze[i][j-1] == 1) {
                solve(i, j - 1, maze, visited, path + "L", result);
            }

            visited[i][j] = false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(4^(n²)) | Space Complexity: O(n²)

147. Word Break

Understanding the Problem NEW

Given a string and a dictionary of words, determine if the string can be segmented into a sequence of one or more dictionary words. E.g. `"leetcode"` with dictionary `["leet","code"]` → yes.

Brute Force Approach NEW

The direct recursive approach: try every possible prefix of the remaining string, and if it's a dictionary word, recursively check if the rest can also be segmented — this is correct, but without any memory of previous results, the same 'remaining substring' can be re-checked repeatedly across different branches of the recursion tree, leading to exponential blowup ($O(2^n)$ in the worst case) from redundant recomputation.

Why We Optimize NEW

The critical optimization: whenever you compute whether a particular *starting index* in the string can be successfully segmented, **cache that result** (in an array or hashmap keyed by index) so that if the same starting index comes up again in a different branch of the recursion, you return the cached answer instantly instead of recomputing it from scratch. This is a form of **memoization** — turning what would otherwise be exponential repeated work into roughly $O(n^2)$ (one computation per starting index, each costing up to $O(n)$ to try every prefix) by remembering answers to subproblems you've already solved, a preview of dynamic programming ideas explored more fully in the DP chapter ahead.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Break string into words from dictionary.

Visual Diagram

```
s = "leetcode", wordDict = ["leet", "code"]
       Result: ["leet code"]

       s = "catsanddog", wordDict = ["cat","cats","and","sand","dog"]
       Results: ["cat sand dog", "cats and dog"]
```

```
      Recursive:
      "leetcode":
        "leet" + "code" → ["leet","code"] ✓

      "catsanddog":
        "cat" + "sanddog"
          "sand" + "dog" → ["cat","sand","dog"] ✓
        "cats" + "anddog"
          "and" + "dog" → ["cats","and","dog"] ✓
```

Solution Approach

Concept: Recursively check prefixes.

Algorithm

```
Algorithm WORD_BREAK(s, wordDict):
           result = []
           breakWord(0, [], s, wordDict, result)
           Return result

      Algorithm breakWord(start, current, s, wordDict, result):
           If start == n:
               result.add(join(current))
               Return
           For end = start+1 to n:
               substr = s.substring(start, end)
               If wordDict contains substr:
                   current.add(substr)
                   breakWord(end, current, s, wordDict, result)
                   current.remove(current.size()-1)
```

Java Implementation

```
class Solution {
           public List<String> wordBreak(String s, List<String> wordDict) {
               Set<String> dict = new HashSet<>(wordDict);
               List<String> result = new ArrayList<>();
               breakWord(0, new ArrayList<>(), s, dict, result);
               return result;
           }

           private void breakWord(int start, List<String> current, String s,
                                  Set<String> dict, List<String> result) {
               if (start == s.length()) {
                   result.add(String.join(" ", current));
                   return;
               }
               for (int end = start + 1; end <= s.length(); end++) {
```

```
                String word = s.substring(start, end);
                if (dict.contains(word)) {
                    current.add(word);
                    breakWord(end, current, s, dict, result);
                    current.remove(current.size() - 1);
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(n)

148. M Coloring Problem

Understanding the Problem NEW

Given a graph and a number of available colors m, determine if you can color every vertex such that no two directly connected vertices share the same color (and if so, find one valid coloring).

Brute Force Approach NEW

Trying every possible assignment of m colors to every vertex (m^V total combinations for V vertices) and checking validity only after a complete assignment is enormously wasteful, since most combinations will violate a constraint almost immediately.

Why We Optimize NEW

Backtracking again: assign a color to one vertex at a time, but only ever choose a color that doesn't conflict with any *already-colored neighbour* — checking this immediately rather than waiting until the whole graph is colored. If a vertex has no valid color available given the current partial coloring, backtrack and try a different color for an earlier vertex. This prunes away entire branches of invalid colorings the moment a conflict appears, rather than fully coloring the graph and discovering the conflict only at the very end.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Color graph with at most m colors (adjacent vertices different).

Visual Diagram

```
Graph: 4 vertices, edges: (0-1), (0-2), (1-2), (1-3), (2-3)

       Can color with 3 colors?
       0: Red
       1: Blue
       2: Green
       3: Red

       Check:
       0-1: Red-Blue ✓
```

```
       0-2: Red-Green ✓
       1-2: Blue-Green ✓
       1-3: Blue-Red ✓
       2-3: Green-Red ✓

       Backtracking:
       Color vertex 0: try color 0
       Color vertex 1: try color 1
       Color vertex 2: try color 2
       Color vertex 3: try color 0 ✓
```

Solution Approach

Concept: Assign colors vertex by vertex.

Algorithm

```
Algorithm M_COLORING(graph, m):
           colors = new int[n]
           If solve(0, graph, colors, m):
               Return colors
           Return []

       Algorithm solve(vertex, graph, colors, m):
           If vertex == n: Return true
           For color = 0 to m-1:
               If isSafe(vertex, color, graph, colors):
                   colors[vertex] = color
                   If solve(vertex+1, graph, colors, m):
                        Return true
                   colors[vertex] = -1
           Return false

       Algorithm isSafe(vertex, color, graph, colors):
           For neighbor in graph[vertex]:
               If colors[neighbor] == color:
                   Return false
           Return true
```

Java Implementation

```
class Solution {
           public int[] graphColoring(int[][] graph, int m) {
               int n = graph.length;
               int[] colors = new int[n];

              Arrays.fill(colors, -1);

              if (solve(0, graph, colors, m)) {
                  return colors;
```

```
            }
            return new int[0];
        }

        private boolean solve(int vertex, int[][] graph, int[] colors, int m) {
            if (vertex == colors.length) return true;

            for (int color = 0; color < m; color++) {
                if (isSafe(vertex, color, graph, colors)) {
                    colors[vertex] = color;
                    if (solve(vertex + 1, graph, colors, m)) {
                        return true;
                    }
                    colors[vertex] = -1;
                }
            }
            return false;
        }

        private boolean isSafe(int vertex, int color, int[][] graph, int[]
colors) {
            for (int neighbor : graph[vertex]) {
                if (colors[neighbor] == color) {
                    return false;
                }
            }
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m^n) | Space Complexity: O(n)

149. Sudoku Solver

Understanding the Problem NEW

Solve a partially-filled Sudoku puzzle by filling in the remaining cells so every row, column, and 3×3 box contains each digit 1-9 exactly once.

Brute Force Approach NEW

Trying every possible digit in every empty cell independently (9^(number of empty cells) total combinations) and validating the entire board only at the end is computationally infeasible for real puzzles.

Why We Optimize NEW

Backtracking with immediate validation: for each empty cell, try digits 1-9 one at a time, but only proceed with a digit if it doesn't *immediately* violate the row/column/box constraints given what's already filled in — if it's valid, recursively try to fill in the rest of the board; if that recursive attempt fails (some later cell has no valid options), backtrack, undo the current cell's digit, and try the next candidate digit instead. This immediate, per-cell validation (rather than filling the whole board blindly and checking once at the end) is what makes backtracking Sudoku solvers tractable even though the raw search space is enormous.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Solve a Sudoku puzzle.

Visual Diagram

```
Input:
       5 3 . | . 7 . | . . .
       6 . . | 1 9 5 | . . .
       . 9 8 | . . . | . 6 .
       ------+-------+------
       8 . . | . 6 . | . . 3
       4 . . | 8 . 3 | . . 1
       7 . . | . 2 . | . . 6
       ------+-------+------
       . 6 . | . . . | 2 8 .
       . . . | 4 1 9 | . . 5
```

```
. . . | . 8 . | . 7 9

Solution after backtracking:
5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
------+-------+------
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
------+-------+------
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9
```

Solution Approach

Concept: Place numbers and validate.

Algorithm

```
Algorithm SUDOKU_SOLVER(board):
           solve(board)
           Return board

       Algorithm solve(board):
           For each empty cell:
               For num = 1 to 9:
                   If isValid(board, row, col, num):
                       board[row][col] = num
                       If solve(board): Return true
                       board[row][col] = 0
               Return false
           Return true

       Algorithm isValid(board, row, col, num):
           Check row, column, and 3x3 box
```

Java Implementation

```
class Solution {
           public void solveSudoku(char[][] board) {
               solve(board);
           }

           private boolean solve(char[][] board) {
               for (int row = 0; row < 9; row++) {
                   for (int col = 0; col < 9; col++) {
                       if (board[row][col] == '.') {
                            for (char num = '1'; num <= '9'; num++) {
```

```
                            if (isValid(board, row, col, num)) {
                                board[row][col] = num;
                                if (solve(board)) return true;
                                board[row][col] = '.';
                            }
                        }
                        return false;
                    }
                }
            }
            return true;
        }

        private boolean isValid(char[][] board, int row, int col, char num) {
            // Check row
            for (int i = 0; i < 9; i++) {
                if (board[row][i] == num) return false;
            }

            // Check column
            for (int i = 0; i < 9; i++) {
                if (board[i][col] == num) return false;
            }

            // Check 3x3 box
            int boxRow = (row / 3) * 3;
            int boxCol = (col / 3) * 3;
            for (int i = boxRow; i < boxRow + 3; i++) {
                for (int j = boxCol; j < boxCol + 3; j++) {
                    if (board[i][j] == num) return false;
                }
            }

            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(9^(n²)) | Space Complexity: O(n²)

150. Expression Add Operators

Understanding the Problem NEW

Given a string of digits and a target value, insert the operators +, -, and × between the digits (without rearranging them) so the resulting expression evaluates to the target. Find every way to do this.

Brute Force Approach NEW

Since there are 3 possible operators to insert at each of the gaps between digits, generating every possible operator arrangement means the search space grows exponentially (3^(number of gaps)) — but there's no way around exploring this space in general, since almost any subset of insertions could be the right one.

Why We Optimize NEW

The main practical technique here (beyond plain exhaustive recursion trying each operator at each gap) is careful handling of **multi-digit numbers** and **multiplication's precedence**: at each recursive step, you must also consider grouping multiple consecutive digits together as one multi-digit number (not just single digits), and when evaluating running totals, you need to track the *previous operand* separately so that a multiplication can correctly 'undo and redo' the last addition/subtraction it affects (since × binds tighter than + or -). Getting this precedence bookkeeping right, layered on top of the exhaustive recursive search over operator choices, is what separates a working solution here from a subtly incorrect one — this is less about complexity optimization and more about correctly modeling arithmetic precedence within a backtracking search.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Add operators to get target value.

Visual Diagram

```
Input: "123", Target: 6
       Expressions:
       1+2+3 = 6 ✓
       1*2*3 = 6 ✓
```

```
Input: "232", Target: 8
2+3*2 = 8 ✓
2*3+2 = 8 ✓

DFS:
"123" at position 0:
  Take "1":
     expr = "1", val = 1, prev = 1
     at pos 1:
       Take "2":
         +: expr="1+2", val=3, prev=2
         -: expr="1-2", val=-1, prev=-2
         *: expr="1*2", val=2, prev=2
```

Solution Approach

Concept: DFS with operator choices.

Algorithm

```
Algorithm ADD_OPERATORS(num, target):
           result = []
           dfs(0, "", 0, 0, num, target, result)
           Return result

       Algorithm dfs(index, expr, val, prev, num, target, result):
           If index == n:
               If val == target: result.add(expr)
               Return
           For i = index to n-1:
               curr_str = num.substring(index, i+1)
               curr = Integer(curr_str)
               If index == 0:
                   dfs(i+1, curr_str, curr, curr, num, target, result)
               Else:
                   dfs(i+1, expr+"+"+curr_str, val+curr, curr, ...)
                   dfs(i+1, expr+"-"+curr_str, val-curr, -curr, ...)
                   dfs(i+1, expr+"*"+curr_str, val-prev+prev*curr, prev*curr, ...)
               If curr == 0: Break
```

Java Implementation

```
class Solution {
           public List<String> addOperators(String num, int target) {
               List<String> result = new ArrayList<>();
               dfs(0, "", 0, 0, num, target, result);
               return result;
           }

           private void dfs(int index, String expr, long val, long prev,
```

```
                        String num, int target, List<String> result) {
            if (index == num.length()) {
                if (val == target) {
                    result.add(expr);
                }
                return;
            }

            for (int i = index; i < num.length(); i++) {
                // Skip numbers with leading zeros
                if (i > index && num.charAt(index) == '0') break;

                String currStr = num.substring(index, i + 1);
                long curr = Long.parseLong(currStr);

                if (index == 0) {
                    dfs(i + 1, currStr, curr, curr, num, target, result);
                } else {
                    dfs(i + 1, expr + "+" + currStr, val + curr, curr, num,
target, result);
                    dfs(i + 1, expr + "-" + currStr, val - curr, -curr, num,
target, result);
                    dfs(i + 1, expr + "*" + currStr, val - prev + prev * curr,
prev * curr, num, ta
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(4^n) | Space Complexity: O(n)

Recursion Summary

Problem	Type	Time	Space	Key Technique
Atoi	Easy	O(n)	O(n)	State tracking
Pow(x,n)	Easy	O(log n)	O(log n)	Fast exponentiation
Good Numbers	Easy	O(log n)	O(1)	Modular arithmetic
Sort Stack	Medium	$O(n^2)$	O(n)	Insert sorted
Reverse Stack	Medium	$O(n^2)$	O(n)	Insert at bottom
Binary Strings	Subsequence	$O(2^n)$	$O(2^n*n)$	Include/Exclude
Generate Parentheses	Subsequence	$O(4^n/\sqrt{n})$	$O(4^n/\sqrt{n})$	Valid combinations
Subsequences	Subsequence	$O(2^n)$	$O(2^n*n)$	Include/Exclude
Subsequence Patterns	Subsequence	$O(2^n)$	O(n)	Print/Count/Check
Count Sum K	Subsequence	$O(2^n)$	O(n)	Counting
Exists Sum K	Subsequence	$O(2^n)$	O(n)	Early termination
Combination Sum	Subsequence	O(2^(target/min))	O(target/min)	Unlimited elements
Combination Sum II	Subsequence	$O(2^n)$	O(n)	Unique, skip duplicates
Subset Sum I	Subsequence	$O(2^n)$	$O(2^n)$	Sum calculation
Subset Sum II	Subsequence	$O(2^n)$	$O(2^n*n)$	Unique subsets
Combination Sum III	Subsequence	O(9!/(9-k)!)	O(k)	Fixed length
Letter Combinations	Subsequence	$O(4^n)$	$O(4^n)$	Mapping
Palindrome Partitioning	Hard	$O(n*2^n)$	O(n)	Check palindrome
Word Search	Hard	$O(m*n*4_3)$	O(L)	DFS with backtracking
N Queen	Hard	O(n!)	$O(n^2)$	Column validation
Rat in Maze	Hard	$O(4^{(n^2)})$	$O(n^2)$	Direction exploration
Word Break	Hard	$O(2^n)$	O(n)	Prefix matching
M Coloring	Hard	$O(m^n)$	O(n)	Vertex coloring
Sudoku Solver	Hard	$O(9^{(n^2)})$	$O(n^2)$	Cell validation

Expression Operators	Hard	$O(4^n)$	O(n)	Operator combinations

— End of enhanced Chapter 6: Recursion — remaining chapters continue in the full edition —

Bit Manipulation — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

151. Introduction to Bit Manipulation

Understanding the Problem NEW

Every number your computer works with is ultimately stored as a sequence of bits (0s and 1s) in binary. Bit manipulation means directly operating on those individual bits — using operators like AND (`&`), OR (`|`), XOR (`^`), NOT (`~`), and shifts (`<<` , `>>`) — instead of working with the number as a whole via normal arithmetic.

Brute Force Approach NEW

There's no 'brute' alternative to introduce here — this section is about learning the vocabulary and behaviour of each bitwise operator, which is foundational for every problem that follows in this chapter.

Why We Optimize NEW

Bitwise operations matter because they let you solve certain problems dramatically faster and with less memory than arithmetic-based approaches — checking or toggling a single bit is O(1), and operations like XOR have almost magical cancellation properties (as you'll see in problems 160 and 163) that let you solve 'find the odd one out' problems without any extra memory at all. This chapter is really about recognizing when a problem is secretly asking a question about individual bits, even if it's not phrased that way on the surface.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding bitwise operators and their applications.

Visual Diagram

```
Bitwise Operators:
       AND (&) : 1 & 1 = 1, else 0
       OR (|)   : 0 | 0 = 0, else 1
       XOR (^) : Same = 0, Different = 1
       NOT (~) : Flips bits
       Left Shift (<<) : x << 1 = x * 2
       Right Shift (>>): x >> 1 = x / 2
```

```
       Example: 5 (0101) & 3 (0011) = 1 (0001)
                5 (0101) | 3 (0011) = 7 (0111)
                5 (0101) ^ 3 (0011) = 6 (0110)
                5 << 1 = 10 (1010)
                5 >> 1 = 2 (0010)

       Binary Representation:
       0 = 0000
       1 = 0001
       2 = 0010
       3 = 0011
       4 = 0100
       5 = 0101
       6 = 0110
       7 = 0111
       8 = 1000
```

Solution Approach

Concept: Understanding binary operations.

Java Implementation

```
class BitManipulation {
           // Bitwise AND
           public int bitwiseAnd(int a, int b) {
               return a & b;
           }

           // Bitwise OR
           public int bitwiseOr(int a, int b) {
               return a | b;
           }

           // Bitwise XOR
           public int bitwiseXor(int a, int b) {
               return a ^ b;
           }

           // Left Shift (multiply by 2)
           public int leftShift(int a, int positions) {

              return a << positions;
          }

          // Right Shift (divide by 2)
          public int rightShift(int a, int positions) {
              return a >> positions;
          }
      }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

152. Check if the i-th Bit is Set or Not

Understanding the Problem NEW

Given a number and a bit position i (counting from 0 at the rightmost bit), determine whether that specific bit is a 1 ('set') or a 0 ('unset'). E.g. in binary `1010`, bit 1 (second from the right) is set.

Brute Force Approach NEW

Without bitwise tools, you might convert the number to a binary string (via repeated division by 2) and check the character at the corresponding position — this works but is far more roundabout than necessary.

Why We Optimize NEW

You can check any bit directly with two operators: shift 1 left by i positions (`1 << i`) to create a 'mask' with only that one bit set, then AND (`&`) it with the number — the result is non-zero exactly when that bit was set. This is an O(1) operation using the hardware's native bit operations, completely avoiding any string conversion or division.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if the i-th bit (0-indexed) is set in a number.

Visual Diagram

```
Number: 13 (1101)
       Check bit positions:
       bit 0: 1 ✓ set
       bit 1: 0 ✗ not set
       bit 2: 1 ✓ set
       bit 3: 1 ✓ set

       Method: n & (1 << i)
       13 & (1 << 0) = 13 & 1 = 1 ✓
       13 & (1 << 1) = 13 & 2 = 0 ✗
       13 & (1 << 2) = 13 & 4 = 4 ✓
       13 & (1 << 3) = 13 & 8 = 8 ✓
```

```
       Check for bit 2:
       1 << 2 = 4 (0100)
       13 (1101) & 4 (0100) = 4 (0100) != 0 → Set
```

Solution Approach

Concept: Use left shift and AND operator.

Algorithm

```
Algorithm IS_BIT_SET(n, i):
           Return (n & (1 << i)) != 0
```

Java Implementation

```
class Solution {
           public boolean isBitSet(int n, int i) {
               return (n & (1 << i)) != 0;
           }
       }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

153. Check if a Number is Odd or Not

Understanding the Problem NEW

Determine if a number is odd or even.

Brute Force Approach NEW

The everyday way: compute `number % 2` and check if it's 1 or 0 — this works and is already quite fast, but a modulo operation is doing more work than strictly necessary.

Why We Optimize NEW

A number is odd exactly when its **very last bit** (the least significant bit) is 1 — this is just what binary place value means (the last bit represents 'is there a 1's place contribution'). So `number & 1` checks that single bit directly, which is typically an even simpler, faster operation for a CPU than a full modulo division — a good first example of how a bitwise trick can replace a basic arithmetic operation with something conceptually (and often practically) cheaper.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Determine if a number is odd using bit manipulation.

Visual Diagram

```
Binary representation:
       Even numbers end with 0:
       2 = 10
       4 = 100
       6 = 110

       Odd numbers end with 1:
       1 = 1
       3 = 11
       5 = 101

       Check: n & 1
```

```
5 (101) & 1 (001) = 1 → Odd ✓
4 (100) & 1 (001) = 0 → Even ✓
```

Solution Approach

Concept: Check the least significant bit.

Algorithm

```
Algorithm IS_ODD(n):
          Return (n & 1) == 1
```

Java Implementation

```
class Solution {
          public boolean isOdd(int n) {
              return (n & 1) == 1;
          }
      }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

154. Check if a Number is Power of 2

Understanding the Problem NEW

Determine whether a number is a power of 2 (like 1, 2, 4, 8, 16...).

Brute Force Approach NEW

You could repeatedly divide the number by 2 in a loop, checking that it divides evenly each time, until you reach 1 (power of 2) or hit an odd remainder before reaching 1 (not a power of 2) — O(log n) time, correct but requiring a loop for something that has a slicker one-line trick.

Why We Optimize NEW

Every power of 2 has *exactly one* bit set in binary (e.g. 8 is `1000`). There's a classic bit trick: `n & (n-1)` always clears the lowest set bit of n. If n is a power of 2 (only one bit set), this operation results in exactly 0. So the one-line check `n > 0 && (n & (n-1)) == 0` answers the question in O(1) — no loop needed at all, compared to the O(log n) repeated-division approach.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if a number is a power of two.

Visual Diagram

```
Powers of 2:
       1 = 1        (1)
       2 = 10       (2)
       4 = 100      (4)
       8 = 1000     (8)
       16 = 10000   (16)

       Property: n & (n-1) == 0 for powers of 2
       8 (1000) & 7 (0111) = 0 ✓
       16 (10000) & 15 (01111) = 0 ✓

       Not powers of 2:
       6 (0110) & 5 (0101) = 4 (0100) ≠ 0 ✗
       10 (1010) & 9 (1001) = 8 (1000) ≠ 0 ✗
```

```
       Edge case: n == 0 → false
```

Solution Approach

Concept: Check if exactly one bit is set.

Algorithm

```
Algorithm IS_POWER_OF_TWO(n):
           If n <= 0: Return false
           Return (n & (n-1)) == 0
```

Java Implementation

```
class Solution {
           public boolean isPowerOfTwo(int n) {
               if (n <= 0) return false;
               return (n & (n - 1)) == 0;
           }
       }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

155. Count the Number of Set Bits

Understanding the Problem NEW

Count how many bits are set to 1 in a number's binary representation (also called its 'popcount' or 'Hamming weight'). E.g. 13 in binary is `1101`, which has 3 set bits.

Brute Force Approach NEW

The straightforward approach: check every bit position one at a time (shift and AND, as in problem 152) across all 32 (or 64) bits of the number, counting how many are set — O(number of bits in the number's type), which for a 32-bit integer is a fixed 32 iterations even if the number itself is small.

Why We Optimize NEW

Brian Kernighan's algorithm is a clever shortcut: repeatedly apply `n = n & (n-1)` (which, as noted in problem 154, clears the lowest set bit each time) and count how many times you can do this before n becomes 0. Instead of always checking a fixed 32 bit positions regardless of the number, this loop runs exactly once per *set* bit — so a number like 8 (only one set bit) finishes in a single iteration instead of 32, making the algorithm's speed proportional to how many 1s are actually present rather than the total bit-width of the number's type.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count the number of 1s in binary representation.

Visual Diagram

```
Number: 13 (1101)
       Set bits: positions 3, 2, 0 → 3 bits

       Method 1: Brian Kernighan's Algorithm
       13 (1101)
       n & (n-1): 13 & 12 = 12 (1100) → count=1
       n & (n-1): 12 & 11 = 8 (1000) → count=2
       n & (n-1): 8 & 7 = 0 (0000) → count=3

       Method 2: Loop through bits
```

```
        13 (1101) & 1 = 1 → count=1, n>>=1
        6 (0110) & 1 = 0 → count=1, n>>=1
        3 (0011) & 1 = 1 → count=2, n>>=1
        1 (0001) & 1 = 1 → count=3, n>>=1
```

Solution Approach

Concept: Brian Kernighan's algorithm or bit loop.

Algorithm

```
Algorithm COUNT_SET_BITS(n):
           count = 0
           While n > 0:
               n = n & (n-1)
               count++
           Return count
```

Java Implementation

```
class Solution {
           // Brian Kernighan's Algorithm
           public int countSetBits(int n) {
               int count = 0;
               while (n > 0) {
                   n = n & (n - 1);
                   count++;
               }
               return count;
           }
       }
```

Time & Space Complexity

Time Complexity: O(number of set bits) | Space Complexity: O(1)

156. Set/Unset the Rightmost Unset Bit

Understanding the Problem NEW

Given a number, find and set (turn to 1) the rightmost bit that is currently 0 (unset). E.g. `1001` → the rightmost unset bit is at position 1, giving `1011`.

Brute Force Approach NEW

You could scan bit positions one at a time from the right (using the check from problem 152), find the first unset one, then set it with a separate OR operation — two passes conceptually (find, then set), or one pass with a conditional break.

Why We Optimize NEW

There's an elegant single-expression trick: `n | (n + 1)`. Adding 1 to n causes a carry to ripple through all the trailing 1-bits (flipping them to 0) until it hits the first 0-bit, which becomes a 1 — and OR-ing the result back with the original n preserves everything that was already set while incorporating that newly-flipped bit. This does in one arithmetic-plus-bitwise expression what would otherwise take an explicit bit-by-bit scan-and-set.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Set the rightmost unset bit to 1.

Visual Diagram

```
Number: 10 (1010)
       Rightmost unset bit: position 1 (0)
       Set: 10 (1010) → 14 (1110)

       Step-by-step:
       n = 10 (1010)
       ~n = 0101 (bitwise complement)
       n+1 = 1011
       ~(n+1) = 0100
       n | (~(n+1)) = 1010 | 0100 = 1110 = 14

       Simpler: n | (n+1)
       10 | 11 = 1010 | 1011 = 1011 = 11 (not correct)
```

```
Wait, correct method:
n | (n+1) works for rightmost unset bit:
10 (1010) | 11 (1011) = 1011 = 11

Examples:
9 (1001) → 1011 = 11
5 (0101) → 0111 = 7
3 (0011) → 0111 = 7 (all bits set)
```

Solution Approach

Concept: Use n | (n+1).

Algorithm

```
Algorithm SET_RIGHTMOST_UNSET(n):
          Return n | (n + 1)
```

Java Implementation

```
class Solution {
          public int setRightmostUnsetBit(int n) {
              return n | (n + 1);
          }
      }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

157. Swap Two Numbers

Understanding the Problem NEW

Swap the values of two variables without using a third temporary variable to hold one of them during the swap.

Brute Force Approach NEW

The everyday way to swap uses a temporary variable: `temp = a; a = b; b = temp;` — perfectly correct and actually the clearest, safest approach in real code; this problem exists purely as a bit-manipulation exercise.

Why We Optimize NEW

Using XOR's self-cancelling property (`x ^ x = 0` and `x ^ 0 = x` , the same property that powers problem 160): `a = a ^ b; b = a ^ b; a = a ^ b;` swaps the two values using no extra variable, because after the first line, `a` holds `original_a ^ original_b` , and XOR-ing that against either original value 'cancels out' that value, leaving the other one. Worth noting as a caveat: this trick has a subtle bug if `a` and `b` are the same memory location (XOR-ing a variable with itself zeroes it out), so in real production code the plain temp-variable swap is usually still preferred for clarity and safety — this problem is really about understanding XOR's properties, not about writing better swap code.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Swap two numbers without using third variable.

Visual Diagram

```
Using XOR:
       a = 5 (0101), b = 3 (0011)

       Step 1: a = a ^ b
       a = 0101 ^ 0011 = 0110 (6)

       Step 2: b = a ^ b
       b = 0110 ^ 0011 = 0101 (5)

       Step 3: a = a ^ b
```

```
        a = 0110 ^ 0101 = 0011 (3)

        Result: a=3, b=5

        Using Addition/Subtraction:
        a = 5, b = 3
        Step 1: a = a + b = 8
        Step 2: b = a - b = 8 - 3 = 5
        Step 3: a = a - b = 8 - 5 = 3
        Result: a=3, b=5
```

Solution Approach

Concept: XOR or addition/subtraction.

Algorithm (XOR)

Algorithm SWAP(a, b):

a = a ^ b

b = a ^ b

a = a ^ b

Return {a, b}

Java Implementation

```
class Solution {
            // Using XOR
            public void swapXOR(int[] arr, int i, int j) {
                arr[i] = arr[i] ^ arr[j];
                arr[j] = arr[i] ^ arr[j];
                arr[i] = arr[i] ^ arr[j];
            }

            // Using addition/subtraction
            public void swapAdd(int[] arr, int i, int j) {
                arr[i] = arr[i] + arr[j];
                arr[j] = arr[i] - arr[j];
                arr[i] = arr[i] - arr[j];
            }
        }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

158. Divide Two Integers Without Using Multiplication

Understanding the Problem NEW

Divide two integers and return the integer quotient, without using multiplication, division, or modulo operators directly.

Brute Force Approach NEW

The most literal way without division: repeatedly subtract the divisor from the dividend, counting how many subtractions you can do before the remainder is smaller than the divisor — correct, but if the dividend is much larger than the divisor, this can take an enormous number of subtractions (e.g. dividing a billion by 1 would take a billion steps).

Why We Optimize NEW

Instead of subtracting the divisor one copy at a time, subtract the **largest possible power-of-2 multiple** of the divisor that still fits, using left-shifts to quickly generate those large multiples (`divisor << k` effectively computes `divisor × 2^k` without using multiplication) — subtracting off the biggest chunk you can at each step, then trying progressively smaller powers of 2. This is conceptually similar to how long division works in decimal, but using powers of 2 instead of powers of 10, and it turns a potentially O(dividend/divisor) subtraction count into O(log(dividend)) — since each step at least halves the remaining problem, much like fast exponentiation in problem 168.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Divide two integers using bit manipulation.

Visual Diagram

```
Divide: 10 / 3 = 3
        Binary division:
        10 = 1010, 3 = 0011

        Method: Repeated subtraction with shift
        quotient = 0, temp = 3
```

```
        10 >= 3 → quotient = 1
        temp <<= 1 → 6
        10 >= 6 → quotient = 2 (1+1)
        temp <<= 1 → 12
        10 < 12 → stop

        Alternative bit manipulation:
        10 / 3:
        10 - 3*2 = 4 → quotient += 2
        4 - 3*1 = 1 → quotient += 1
        Result: 3

        Edge cases:
        Positive divisor, positive dividend
        Negative handling
        Overflow handling
```

Solution Approach

Concept: Bit shifting for division.

Algorithm

```
Algorithm DIVIDE(dividend, divisor):
            If divisor == 0: Return MAX_VALUE
            sign = (dividend < 0) ^ (divisor < 0)
            n = abs(dividend), d = abs(divisor)
            quotient = 0

            While n >= d:
                temp = d, multiple = 1
                While n >= (temp << 1):
                    temp <<= 1
                    multiple <<= 1
                n -= temp
                quotient += multiple

            Return sign ? -quotient : quotient
```

Java Implementation

```
class Solution {
            public int divide(int dividend, int divisor) {
                if (divisor == 0) return Integer.MAX_VALUE;
                if (dividend == Integer.MIN_VALUE && divisor == -1) {
                    return Integer.MAX_VALUE;
                }

                 boolean isNegative = (dividend < 0) ^ (divisor < 0);
```

```
            long n = Math.abs((long) dividend);
            long d = Math.abs((long) divisor);

            long quotient = 0;
            while (n >= d) {
                long temp = d;
                long multiple = 1;
                while (n >= (temp << 1)) {
                    temp <<= 1;
                    multiple <<= 1;
                }
                n -= temp;
                quotient += multiple;
            }

            return (int)(isNegative ? -quotient : quotient);
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

159. Count Number of Bits to be Flipped to Convert A to B

Understanding the Problem NEW

Given two numbers A and B, count how many bit positions need to be flipped to turn A into B.

Brute Force Approach NEW

You could convert both numbers to binary strings and compare them character by character, counting mismatches — correct, but involves an unnecessary string-conversion step.

Why We Optimize NEW

XOR-ing A and B directly produces a number where every bit that *differs* between them is set to 1, and every bit that's the *same* is 0 (this is literally XOR's definition: 'true when inputs differ'). So the answer is simply the **count of set bits** in `A ^ B` — directly reusing the bit-counting technique from problem 155 (Brian Kernighan's algorithm), turning a two-step 'convert to binary strings and diff' process into a one-line XOR followed by a bit count.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count how many bits need to be flipped to convert a to b.

Visual Diagram

```
a = 10 (1010), b = 7 (0111)

       XOR: a ^ b = 1010 ^ 0111 = 1101 (13)
       Count set bits in XOR = 3

       Bits to flip: positions where bits differ
       a: 1 0 1 0
       b: 0 1 1 1
          ^ ^   ^
          3 bits differ
```

```
      Method: countSetBits(a ^ b)
```

Solution Approach

Concept: XOR to find differing bits.

Algorithm

```
Algorithm COUNT_FLIPS(a, b):
          diff = a ^ b
          Return countSetBits(diff)
```

Java Implementation

```
class Solution {
          public int countBitsToFlip(int a, int b) {
              int diff = a ^ b;
              return countSetBits(diff);
          }

           private int countSetBits(int n) {
               int count = 0;
               while (n > 0) {
                   n = n & (n - 1);
                   count++;
               }
               return count;
           }
      }
```

Time & Space Complexity

Time Complexity: O(number of set bits) | Space Complexity: O(1)

160. Find the Number that Appears Odd Number of Times

Understanding the Problem NEW

In an array where every number appears an even number of times except one, which appears an odd number of times, find that one number. (This is the same 'Single Number' problem seen in the Arrays chapter, revisited here explicitly as a bit-manipulation technique.)

Brute Force Approach NEW

A hashmap-based frequency count finds the answer in O(n) time but O(n) extra space, tracking every distinct value's count explicitly.

Why We Optimize NEW

XOR every number in the array together: pairs of equal numbers XOR to 0 and vanish (`x ^ x = 0`), and XOR-ing with 0 leaves any value unchanged (`x ^ 0 = x`), so after XOR-ing everything, only the number that appeared an odd number of times survives. This is exactly the same technique as the Arrays-chapter 'Single Number' problem, but placed here to make explicit *why* it works — it's a direct consequence of XOR's self-cancelling algebra, achieving O(n) time and O(1) space, better than the hashmap's O(n) space.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the number that appears odd number of times.

Visual Diagram

```
Array: [1, 2, 3, 2, 1, 3, 4]
       All appear even times except 4

       XOR property:
       1 ^ 1 = 0
       2 ^ 2 = 0
       3 ^ 3 = 0
       4 ^ 0 = 4
```

```
XOR all numbers:
1 ^ 2 ^ 3 ^ 2 ^ 1 ^ 3 ^ 4 = 4

Result: 4 (appears once → odd times)

Array: [4, 3, 4, 4, 3, 4, 5, 6, 6]
XOR all: 5 (appears once → odd)

Array: [1, 1, 2, 2, 3, 3, 3]
XOR all: 3 (appears three times → odd)
```

Solution Approach

Concept: XOR all numbers, duplicates cancel.

Algorithm

```
Algorithm FIND_ODD_OCCURRENCE(arr):
           result = 0
           For each num in arr:
               result ^= num
           Return result
```

Java Implementation

```
class Solution {
           public int findOddOccurrence(int[] arr) {
               int result = 0;
               for (int num : arr) {
                   result ^= num;
               }
               return result;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

161. Power Set

Understanding the Problem NEW

Generate the power set (every possible subset) of a given array — the same goal as the Recursion chapter's 'Print All Subsequences' problem (133), but this time using bit manipulation instead of recursive include/exclude calls.

Brute Force Approach NEW

The recursive include/exclude approach from problem 133 works fine ($O(2^n)$ time, matching the number of subsets that must exist), but requires building a recursion tree with function call overhead for each branching decision.

Why We Optimize NEW

Since there are exactly 2^n subsets of an n-element array, and each subset can be represented by an n-bit number where **bit i tells you whether element i is included**, you can generate every subset just by counting from 0 to 2^n-1 and checking which bits are set in each count value (using the technique from problem 152) to decide which elements to include. This replaces recursive function calls with a simple loop and bitmask checks — same $O(2^n)$ total time as the recursive version, but often with lower constant-factor overhead since there's no recursive call stack involved, and it directly connects the abstract idea of 'include or exclude each element' to a concrete numeric representation (the bitmask).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Generate all subsets using bit manipulation.

Visual Diagram

```
Array: [1, 2, 3]
       All subsets: 2³ = 8 subsets

       Bit representation:
       0: 000 → []
       1: 001 → [1]
       2: 010 → [2]
```

```
3: 011 → [1,2]
4: 100 → [3]
5: 101 → [1,3]
6: 110 → [2,3]
7: 111 → [1,2,3]

For each i from 0 to (1<<n)-1:
  For each bit j:
    If (i >> j) & 1:
      Add arr[j] to subset

n=3, 1<<3 = 8
i=5 (101):
  j=0: (5>>0)&1=1 → include arr[0]
  j=1: (5>>1)&1=0 → skip arr[1]
  j=2: (5>>2)&1=1 → include arr[2]
  → [1,3]
```

Solution Approach

Concept: Use bitmask to represent subsets.

Algorithm

```
Algorithm POWER_SET(arr):
           n = arr.length
           result = []
           For mask = 0 to (1<<n)-1:
               subset = []
               For i = 0 to n-1:
                   If (mask & (1<<i)) != 0:
                       subset.add(arr[i])
               result.add(subset)
           Return result
```

Java Implementation

```
class Solution {
           public List<List<Integer>> powerSet(int[] arr) {
               int n = arr.length;
               List<List<Integer>> result = new ArrayList<>();

                 for (int mask = 0; mask < (1 << n); mask++) {
                     List<Integer> subset = new ArrayList<>();
                     for (int i = 0; i < n; i++) {
                         if ((mask & (1 << i)) != 0) {
                             subset.add(arr[i]);
                         }

                   }
```

```
                result.add(subset);
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * 2^n) | Space Complexity: O(2^n * n)

162. Find XOR of Numbers from L to R

Understanding the Problem NEW

Compute the XOR of every integer from L to R inclusive (e.g. L=3, R=5 → 3^4^5), without looping through and XOR-ing every single number one at a time.

Brute Force Approach NEW

Loop from L to R, XOR-ing each number into a running result — O(R-L) time, which becomes slow if R-L is very large (e.g. billions of numbers).

Why We Optimize NEW

XOR from 1 to N follows a repeating 4-step pattern based on N modulo 4 (it cycles through N, 1, N+1, 0 depending on N mod 4) — a fact you can derive and then use as an O(1) formula. Using this, `XOR(L to R) = XOR(1 to R) ^ XOR(1 to L-1)` (since XOR-ing the same prefix twice cancels it out, similar in spirit to how prefix sums let you compute range sums instantly). This collapses a potentially billions-long loop into a small constant number of O(1) formula evaluations — the same 'prefix technique' idea from the Arrays chapter's prefix-sum problems, just applied to XOR instead of addition.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate XOR of all numbers from L to R inclusive.

Visual Diagram

```
XOR from 0 to n pattern:
       0: 0
       1: 1
       2: 3
       3: 0
       4: 4
       5: 1
       6: 7
       7: 0
       8: 8

       Pattern:
```

```
n % 4 == 0 → n
n % 4 == 1 → 1
n % 4 == 2 → n+1
n % 4 == 3 → 0

Find XOR(L,R) = XOR(0,R) ^ XOR(0,L-1)

Example: XOR(2,5)
XOR(0,5) = 1 (5%4=1 → 1)
XOR(0,1) = 1 (1%4=1 → 1)
XOR(2,5) = 1 ^ 1 = 0
Verify: 2^3^4^5 = 2^3=1, 1^4=5, 5^5=0 ✓
```

Solution Approach

Concept: Use XOR prefix property.

Algorithm

```
Algorithm XOR_0_TO_N(n):
          If n % 4 == 0: Return n
          If n % 4 == 1: Return 1
          If n % 4 == 2: Return n+1
          Return 0

      Algorithm XOR_L_R(L, R):
          Return XOR_0_TO_N(R) ^ XOR_0_TO_N(L-1)
```

Java Implementation

```
class Solution {
          public int xorFrom0ToN(int n) {
              if (n % 4 == 0) return n;
              if (n % 4 == 1) return 1;
              if (n % 4 == 2) return n + 1;
              return 0;
          }

          public int xorLtoR(int L, int R) {
              return xorFrom0ToN(R) ^ xorFrom0ToN(L - 1);
          }
      }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

163. Find the Two Numbers Appearing Odd Number of Times

Understanding the Problem NEW

Now two numbers each appear an odd number of times (everything else appears an even number of times) — find *both* of them. A direct extension of problem 160's single-odd-number version.

Brute Force Approach NEW

A hashmap frequency count finds both numbers in O(n) time, O(n) space — correct, but this problem exists specifically to explore how far the XOR trick can be pushed with a clever twist.

Why We Optimize NEW

First, XOR everything together — this gives you `x ^ y` (the XOR of the two odd-occurring numbers), since every evenly-occurring number cancels out, leaving the combination of the two numbers you're looking for. Since `x` and `y` are different, `x ^ y` must have *at least one* bit set — pick any such set bit, and use it to **partition** the entire array into two groups: numbers with that bit set, and numbers without. Because x and y differ at that bit, they land in different groups, and every pair of evenly-occurring duplicates lands in the *same* group as each other (since duplicates are bit-identical) — so XOR-ing each group separately isolates x in one group and y in the other. This still achieves O(n) time and O(1) space, extending the simple cancellation trick with one extra partitioning step to separate two 'survivors' instead of just one.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find two numbers that appear odd number of times.

Visual Diagram

```
Array: [1, 2, 3, 2, 1, 4, 3, 5]
       Odd appearing: 4, 5

       Step 1: XOR all numbers
```

```
1^2^3^2^1^4^3^5 = 4^5 = 1 (0001)

Step 2: Find rightmost set bit
1 & -1 = 1

Step 3: Partition numbers by this bit
Numbers with bit 0 set:
1, 3, 1, 3, 5 → XOR = 5
Numbers with bit 1 set:
2, 2, 4 → XOR = 4

Result: [4, 5]

Wait, let's recalc:
4^5 = 1 (0001) → rightmost set bit is position 0
Numbers with bit 0 = 1:
1, 3, 1, 3, 5 → xor = 5
Numbers with bit 0 = 0:
2, 2, 4 → xor = 4
```

Solution Approach

Concept: XOR all numbers, find set bit, partition.

Algorithm

```
Algorithm TWO_ODD_OCCURRING(arr):
           xor_all = 0
           For num in arr: xor_all ^= num

            rightmost_bit = xor_all & -xor_all

            x = 0, y = 0
            For num in arr:
                If (num & rightmost_bit) == 0:
                    x ^= num
                Else:
                    y ^= num

            Return {x, y}
```

Java Implementation

```
class Solution {
           public int[] twoOddOccurring(int[] arr) {
               int xorAll = 0;
               for (int num : arr) {
                   xorAll ^= num;
               }
```

```
                // Rightmost set bit

            int rightmostBit = xorAll & -xorAll;

            int x = 0, y = 0;
            for (int num : arr) {
                if ((num & rightmostBit) == 0) {
                    x ^= num;
                } else {
                    y ^= num;
                }
            }
            return new int[]{x, y};
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

164. Print Prime Factors of a Number

Understanding the Problem NEW

Find all prime factors of a number (the prime numbers that multiply together to produce it). E.g. 60 = 2×2×3×5.

Brute Force Approach NEW

Check every integer from 2 up to the number itself to see if it divides evenly, and if so, whether it's prime — testing primality for every candidate divisor individually is expensive and redundant.

Why We Optimize NEW

You don't need to separately test primality at all: starting from the smallest possible factor (2), repeatedly divide the number by it as many times as it evenly divides (each successful division is automatically a prime factor, since you've already removed all smaller factors first), then move to the next candidate divisor. You only need to check candidate divisors up to **√n**, because if n still has a factor larger than √n remaining after removing all smaller ones, that remaining value itself must be prime. This turns an expensive 'check every number for primality' approach into a direct O(√n) factor-extraction process, with primality of each factor guaranteed for free by the order of extraction rather than checked explicitly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print all prime factors of a number.

Visual Diagram

```
Number: 12
       Factors: 2, 2, 3
       Prime factors: 2, 3

       Number: 36
       36 = 2 × 2 × 3 × 3
       Prime factors: 2, 3
```

```
Algorithm:
n = 12
i = 2: 12%2==0 → print 2, n=6
i = 2: 6%2==0 → print 2, n=3
i = 3: 3%3==0 → print 3, n=1
Stop

Number: 13 (prime)
i = 2: 13%2 !=0
i = 3: 13%3 !=0
i = sqrt(13) ≈ 3.6
After loop, if n > 1: print n
Result: 13
```

Solution Approach

Concept: Divide by smallest prime factors.

Algorithm

```
Algorithm PRINT_PRIME_FACTORS(n):
           While n % 2 == 0:
               print 2
               n /= 2

            For i = 3 to sqrt(n) step 2:
                While n % i == 0:
                    print i
                    n /= i

            If n > 1: print n
```

Java Implementation

```
class Solution {
           public List<Integer> primeFactors(int n) {
               List<Integer> factors = new ArrayList<>();

                 while (n % 2 == 0) {
                     factors.add(2);
                     n /= 2;
                 }

                 for (int i = 3; i * i <= n; i += 2) {

                   while (n % i == 0) {
                       factors.add(i);
                       n /= i;
                   }
               }
```

```
            if (n > 1) factors.add(n);
            return factors;
        }
    }
```

Time & Space Complexity

Time Complexity: O(√n) | Space Complexity: O(number of factors)

165. All Divisors of a Number

Understanding the Problem NEW

Find every divisor of a number (every integer that divides it evenly). E.g. divisors of 12: 1, 2, 3, 4, 6, 12.

Brute Force Approach NEW

Check every integer from 1 to n to see if it divides evenly — O(n) time, checking many candidates that obviously can't be factors (like checking if 7 divides 12 when you already know 12/7 isn't an integer, but doing this check for every number up to n regardless).

Why We Optimize NEW

Divisors come in **pairs**: if `d` divides n, then `n/d` is also a divisor. This means you only need to check candidate divisors up to **√n** — for each one that divides evenly, you get two divisors at once (`d` and `n/d`), except when d equals √n exactly (a perfect square), where you only count it once. This drops the search from O(n) to O(√n), exploiting the pairing structure instead of checking every candidate individually.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all divisors of a number.

Visual Diagram

```
Number: 36
       Divisors: 1, 2, 3, 4, 6, 9, 12, 18, 36

       Method: Check up to sqrt(n)
       i = 1: 36%1==0 → add 1 and 36
       i = 2: 36%2==0 → add 2 and 18
       i = 3: 36%3==0 → add 3 and 12
       i = 4: 36%4==0 → add 4 and 9
       i = 5: 36%5!=0
       i = 6: 36%6==0 → add 6 (36/6=6, same)

       Sorted: 1,2,3,4,6,9,12,18,36
```

```
Number: 28
Divisors: 1,2,4,7,14,28
sqrt(28) ≈ 5.29
i=1: 1,28
i=2: 2,14
i=4: 4,7
i=5: no
```

Solution Approach

Concept: Check up to √n.

Algorithm

```
Algorithm ALL_DIVISORS(n):
           divisors = []
           For i = 1 to sqrt(n):
               If n % i == 0:
                   divisors.add(i)
                   If i != n/i:
                       divisors.add(n/i)
           Sort(divisors)
           Return divisors
```

Java Implementation

```
class Solution {
           public List<Integer> allDivisors(int n) {
               List<Integer> divisors = new ArrayList<>();

                 for (int i = 1; i * i <= n; i++) {
                     if (n % i == 0) {
                         divisors.add(i);
                         if (i != n / i) {
                             divisors.add(n / i);
                         }
                     }
                 }

                 Collections.sort(divisors);
                 return divisors;
            }

       }
```

Time & Space Complexity

Time Complexity: O(√n) | Space Complexity: O(number of divisors)

166. Sieve of Eratosthenes

Understanding the Problem NEW

Find all prime numbers up to some limit N — e.g. all primes up to 30.

Brute Force Approach NEW

Check every number from 2 to N individually for primality (e.g. by testing divisibility up to its square root, as in a per-number primality test) — O(N√N) total, since you repeat a full primality check for every single number.

Why We Optimize NEW

The **Sieve of Eratosthenes** flips the approach: instead of asking 'is this number prime?' for each number individually, start with every number marked as 'potentially prime,' and for each prime you find (starting from 2), **cross off all of its multiples** as definitely-not-prime, since any multiple of a prime can't itself be prime. Whatever survives uncrossed by the end must be prime. This achieves roughly O(N log log N) — dramatically better than checking every number separately — because you're marking off composite numbers in bulk based on their prime factors, rather than individually verifying every candidate from scratch.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all prime numbers up to n.

Visual Diagram

```
n = 20

       Initialize: [2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20]

       Step 1: 2 is prime → mark multiples of 2
       [2,3,5,7,9,11,13,15,17,19]

       Step 2: 3 is prime → mark multiples of 3
       [2,3,5,7,11,13,17,19]

       Step 3: 5 is prime → mark multiples of 5
```

```
       [2,3,5,7,11,13,17,19]

       Continue up to sqrt(20) ≈ 4.47

       Primes: 2,3,5,7,11,13,17,19

       Array representation:
       Index: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
       Value: F F T T F T F T F F F T F T F F F T F T F
```

Solution Approach

Concept: Mark multiples of each prime.

Algorithm

```
Algorithm SIEVE(n):
           isPrime = boolean[n+1]
           Arrays.fill(isPrime, true)
           isPrime[0] = isPrime[1] = false

           For i = 2 to sqrt(n):
               If isPrime[i]:
                   For j = i*i to n step i:
                       isPrime[j] = false

           primes = []
           For i = 2 to n:
               If isPrime[i]:
                   primes.add(i)
           Return primes
```

Java Implementation

```
class Solution {
           public List<Integer> sieveOfEratosthenes(int n) {
               boolean[] isPrime = new boolean[n + 1];
               Arrays.fill(isPrime, true);
               isPrime[0] = isPrime[1] = false;

                 for (int i = 2; i * i <= n; i++) {
                     if (isPrime[i]) {
                         for (int j = i * i; j <= n; j += i) {
                             isPrime[j] = false;

                         }
                    }
               }

               List<Integer> primes = new ArrayList<>();
```

```
            for (int i = 2; i <= n; i++) {
                if (isPrime[i]) {
                    primes.add(i);
                }
            }
            return primes;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log log n) | Space Complexity: O(n)

167. Find Prime Factorisation of a Number

Understanding the Problem NEW

Find the prime factorization of a number, but this time set up to answer this efficiently for *many* different query numbers, not just once.

Brute Force Approach NEW

Running the per-number O(√n) factorization from problem 164 separately for every single query number works fine for one query, but if you have many queries, you're redoing similar work (checking small primes repeatedly) over and over.

Why We Optimize NEW

Precompute a **Smallest Prime Factor (SPF)** table for every number up to some maximum, using a sieve-like process similar to problem 166 (for each prime found, mark it as the smallest prime factor for all of its yet-unmarked multiples). Once this table is built once (in O(N log log N)), factorizing *any* number up to N becomes fast: repeatedly divide by its SPF and look up the SPF of the result, peeling off prime factors in O(log n) per query instead of O(√n) — a valuable trade-off when you expect to answer many factorization queries, paying a one-time upfront cost to make every individual query much cheaper.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find prime factorization of a number using sieve.

Visual Diagram

```
Number: 36
       Using sieve, find smallest prime factor (SPF)

       SPF array for n=36:
       2:2, 3:3, 4:2, 5:5, 6:2, 8:2, 9:3, 10:2, 12:2, 14:2, 15:3, 16:2, 18:2, 20:2,
21:3, 22:2, 24:2,

       Factorization:
       n = 36
       spf[36] = 2 → divide → n=18
```

```
        spf[18] = 2 → divide → n=9
        spf[9] = 3 → divide → n=3
        spf[3] = 3 → divide → n=1
        Result: 2,2,3,3

        Number: 84
        84 → 2 → 42
        42 → 2 → 21
        21 → 3 → 7
        7 → 7 → 1
        Result: 2,2,3,7
```

Solution Approach

Concept: Precompute smallest prime factor.

Algorithm

```
Algorithm SPF_SIEVE(n):
            spf = new int[n+1]
            For i = 0 to n: spf[i] = i
            spf[0] = spf[1] = 0

            For i = 2 to sqrt(n):
                If spf[i] == i:
                    For j = i*i to n step i:
                        If spf[j] == j:
                            spf[j] = i

            Return spf

        Algorithm FACTORIZE(n, spf):
            factors = []
            While n > 1:
                factors.add(spf[n])
                n /= spf[n]
            Return factors
```

Java Implementation

```
class Solution {
            public int[] smallestPrimeFactor(int n) {
                int[] spf = new int[n + 1];
                for (int i = 0; i <= n; i++) {
                    spf[i] = i;
                }

                 spf[0] = spf[1] = 0;

                 for (int i = 2; i * i <= n; i++) {
```

```
                if (spf[i] == i) {
                    for (int j = i * i; j <= n; j += i) {
                        if (spf[j] == j) {
                            spf[j] = i;
                        }
                    }
                }
            }
            return spf;
        }

        public List<Integer> primeFactorization(int n) {
            int[] spf = smallestPrimeFactor(n);
            List<Integer> factors = new ArrayList<>();

            while (n > 1) {
                factors.add(spf[n]);
                n /= spf[n];
            }
            return factors;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log log n) for sieve, O(log n) for factorization | Space

168. Power(n, x) - Fast Exponentiation

Understanding the Problem NEW

Compute n raised to the power x efficiently — the same problem as Pow(x,n) from the Recursion chapter (problem 127), revisited here to connect it explicitly to the bit-manipulation theme of this chapter.

Brute Force Approach NEW

Multiply n by itself x times in a loop — O(x) multiplications.

Why We Optimize NEW

Exponentiation by squaring, exactly as in problem 127: this time, it's worth explicitly connecting it to bit manipulation — the exponent x, viewed in binary, tells you exactly which powers of n (n^1, n^2, n^4, n^8, ...) need to be multiplied together to build n^x. Each bit of x that's set corresponds to including that particular squared-power of n in the final product. Seen this way, fast exponentiation is really 'read the bits of the exponent and multiply in the corresponding pre-squared powers' — a nice capstone connecting the divide-and-conquer idea from the Recursion chapter directly to the bit-level view this chapter has been building throughout, still achieving O(log x) instead of O(x).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate n^x efficiently using exponentiation by squaring.

Visual Diagram

```
Power: 2^10 = 1024

       Binary method:
       10 in binary = 1010

       2^10 = 2^(8+2) = 2^8 * 2^2 = 256 * 4 = 1024

       Iterative:
       result = 1, base = 2, exp = 10

       exp & 1 = 0 → exp=5, base=4
```

```
exp & 1 = 1 → result=4, exp=2, base=16
exp & 1 = 0 → exp=1, base=256
exp & 1 = 1 → result=1024, exp=0
Stop → result=1024

Power: 3^5 = 243
exp=5 (101)
exp & 1 = 1 → result=3, exp=2, base=9
exp & 1 = 0 → exp=1, base=81
exp & 1 = 1 → result=243, exp=0
Result=243
```

Solution Approach

Concept: Exponentiation by squaring.

Algorithm

```
Algorithm POWER(n, x):
           result = 1
           base = n
           exp = x

           While exp > 0:
               If exp & 1:
                   result *= base
               base *= base
               exp >>= 1

           Return result
```

Java Implementation

```
class Solution {
           // Iterative
           public long power(long n, int x) {
               long result = 1;
               long base = n;
               int exp = x;

                 while (exp > 0) {
                     if ((exp & 1) == 1) {
                         result *= base;
                     }

                   base *= base;
                   exp >>= 1;
               }
               return result;
           }
```

```
        // Recursive
        public long powerRecursive(long n, int x) {
            if (x == 0) return 1;
            if (x == 1) return n;

            long temp = powerRecursive(n, x / 2);
            if (x % 2 == 0) {
                return temp * temp;
            } else {
                return temp * temp * n;
            }
        }

        // Modular exponentiation
        public long modPower(long n, int x, int mod) {
            long result = 1;
            long base = n % mod;
            int exp = x;

            while (exp > 0) {
                if ((exp & 1) == 1) {
                    result = (result * base) % mod;
                }
                base = (base * base) % mod;
                exp >>= 1;
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log x) | Space Complexity: O(1) iterative, O(log x) recursive

Bit Manipulation Summary

Problem	Type	Time	Space	Key Technique
Bit Basics	Concepts	O(1)	O(1)	Bitwise operators
Check i-th Bit	Basic	O(1)	O(1)	n & (1<<i)
Check Odd	Basic	O(1)	O(1)	n & 1
Power of 2	Basic	O(1)	O(1)	n & (n-1)
Count Set Bits	Basic	O(k)	O(1)	Brian Kernighan
Set Rightmost Unset	Basic	O(1)	O(1)	n | (n+1)
Swap Numbers	Basic	O(1)	O(1)	XOR swap
Divide Integers	Basic	O(log n)	O(1)	Bit shifting
Count Flips	Interview	O(k)	O(1)	XOR + set bits
Odd Occurrence	Interview	O(n)	O(1)	XOR all
Power Set	Interview	$O(n*2^n)$	$O(2^n*n)$	Bitmask
XOR L to R	Interview	O(1)	O(1)	Pattern
Two Odd Occurring	Interview	O(n)	O(1)	XOR + partition
Prime Factors	Advanced	$O(\sqrt{n})$	O(k)	Division
All Divisors	Advanced	$O(\sqrt{n})$	O(k)	Check up to sqrt
Sieve	Advanced	O(n log log n)	O(n)	Marking multiples
Prime Factorization	Advanced	O(n log log n)	O(n)	SPF array
Power	Advanced	O(log n)	O(1)	Fast exponentiation

Stack & Queue, and Sliding Window — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

169. Implement Stack using Arrays

Understanding the Problem NEW

A Stack is a 'last in, first out' (LIFO) structure — think of a stack of plates: you can only add a plate to the top, and you can only remove the plate that's currently on top. Implement this using a plain array as the backing storage.

Brute Force Approach NEW

There isn't a 'brute' alternative here — this is about learning the building block itself. The natural question is just how to simulate LIFO behaviour using an array, which normally supports access to any index.

Why We Optimize NEW

You only need **one extra variable**: a `top` index tracking the current top of the stack. Push means placing the new value at `arr[top+1]` and incrementing top; pop means reading `arr[top]` and decrementing top. Both operations are O(1) — no shifting of other elements is needed, since you only ever touch the position right at the current top, unlike inserting/deleting in the middle of an array.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Stack data structure using array.

Visual Diagram

```
Stack: LIFO (Last In First Out)

       Operations:
       push(5): [5]
       push(3): [5,3]
       push(8): [5,3,8]
       pop(): [5,3] → returns 8
       peek(): returns 3
       isEmpty(): false

       Array Implementation:
       Index: 0 1 2 3 4
       Value: [5, 3, 8, _, _]
```

```
       Top = 2

       push(10):
       Index: 0 1 2 3 4
       Value: [5, 3, 8, 10, _]
       Top = 3
```

Solution Approach

Concept: Use array with top pointer.

Algorithm

```
Algorithm STACK_ARRAY:
           arr = new int[capacity]
           top = -1

           push(x):
               If top == capacity-1: Overflow
               arr[++top] = x

           pop():
               If top == -1: Underflow
               Return arr[top--]

           peek():
               If top == -1: Empty
               Return arr[top]
```

Java Implementation

```
class StackArray {
           private int[] arr;
           private int top;
           private int capacity;

           StackArray(int capacity) {
               this.capacity = capacity;
               arr = new int[capacity];
               top = -1;
           }

           public void push(int x) {
               if (top == capacity - 1) {
                   throw new RuntimeException("Stack Overflow");
               }
               arr[++top] = x;
           }

           public int pop() {
```

```
            if (top == -1) {
                throw new RuntimeException("Stack Underflow");
            }
            return arr[top--];
        }

        public int peek() {
            if (top == -1) {
                throw new RuntimeException("Stack is Empty");
            }
            return arr[top];
        }

        public boolean isEmpty() {
            return top == -1;
        }

        public int size() {
            return top + 1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(n)

170. Implement Queue using Arrays

Understanding the Problem NEW

A Queue is a 'first in, first out' (FIFO) structure — like a line at a checkout counter: whoever joined first gets served first. Implement this using an array.

Brute Force Approach NEW

A naive array-based queue might always insert at the end and remove from the front by shifting every remaining element left by one each time — O(n) per removal, since arrays aren't naturally suited to removing from the front cheaply.

Why We Optimize NEW

Instead of physically shifting elements after every removal, use a **circular array**: track a `front` and `rear` index that both wrap around back to the start of the array once they reach the end (using modulo arithmetic). This reuses array slots that have already been dequeued, without ever needing to shift remaining elements — both enqueue and dequeue become O(1), instead of O(n) for the shifting approach.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Queue data structure using array.

Visual Diagram

```
Queue: FIFO (First In First Out)

       Operations:
       enqueue(5): [5]
       enqueue(3): [5,3]
       enqueue(8): [5,3,8]
       dequeue(): [3,8] → returns 5
       front(): returns 3
       isEmpty(): false

       Circular Array Implementation:
       front = 0, rear = -1, size = 0
```

```
       enqueue: rear = (rear+1) % capacity
       dequeue: front = (front+1) % capacity
```

Solution Approach

Concept: Circular array to utilize space.

Algorithm

```
Algorithm QUEUE_ARRAY:
           arr = new int[capacity]
           front = 0, rear = -1, size = 0

            enqueue(x):
                If size == capacity: Overflow
                rear = (rear + 1) % capacity
                arr[rear] = x
                size++

            dequeue():
                If size == 0: Underflow
                val = arr[front]
                front = (front + 1) % capacity
                size--
                Return val
```

Java Implementation

```
class QueueArray {
           private int[] arr;
           private int front;
           private int rear;
           private int size;
           private int capacity;

            QueueArray(int capacity) {
                this.capacity = capacity;
                arr = new int[capacity];
                front = 0;
                rear = -1;
                size = 0;
            }
            public void enqueue(int x) {

              if (size == capacity) {
                  throw new RuntimeException("Queue Overflow");
              }
              rear = (rear + 1) % capacity;
              arr[rear] = x;
              size++;
```

```
        }

        public int dequeue() {
            if (size == 0) {
                throw new RuntimeException("Queue Underflow");
            }
            int val = arr[front];
            front = (front + 1) % capacity;
            size--;
            return val;
        }

        public int front() {
            if (size == 0) {
                throw new RuntimeException("Queue is Empty");
            }
            return arr[front];
        }

        public boolean isEmpty() {
            return size == 0;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(n)

171. Implement Stack using Queue

Understanding the Problem NEW

Implement a Stack (LIFO) using only Queues (FIFO) as your building blocks — an exercise in adapting one structure's behaviour using a fundamentally different one.

Brute Force Approach NEW

A naive approach might use just one queue and, on every push, dequeue and re-enqueue everything else after adding the new item, but doing this awkwardly on every single operation without a clear plan often leads to O(n) work per push *and* confusion about correctness.

Why We Optimize NEW

The clean approach: on every push, enqueue the new element into the queue, then **rotate the whole queue** by dequeuing and immediately re-enqueuing every element that came before it — this has the effect of moving the newest element to the very front of the queue. This makes push cost O(n) (since you rotate the whole queue) but pop trivially O(1) (just dequeue the front, which is always the most recently pushed element) — a deliberate trade-off, since the two structures have opposite natural strengths (queue front-access is cheap, but LIFO order requires 'faking' it via rotation).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Stack using two queues.

Visual Diagram

```
Stack using Queues (Push Efficient):
       Push: Add to queue2, then move all from queue1
       Pop: Remove from queue1

       Push 5:
       q1: [5], q2: []

       Push 3:
```

```
       q2: [3], move q1: [5] → q2: [3,5]
       Swap: q1: [3,5], q2: []

       Push 8:
       q2: [8], move q1: [3,5] → q2: [8,3,5]
       Swap: q1: [8,3,5], q2: []

       Pop(): 8 from q1
       q1: [3,5]

       Stack using Queues (Pop Efficient):
       Push: Add to q2, if q1 not empty move all
       Pop: Remove from q1
```

Solution Approach

Concept: Use two queues with swapping.

Algorithm (Push Efficient)

Algorithm STACK_USING_QUEUE:

q1, q2 = Queue()

push(x):

q2.enqueue(x)

While q1 not empty:

q2.enqueue(q1.dequeue())

Swap(q1, q2)

pop():

Return q1.dequeue()

peek():

Return q1.front()

Java Implementation

```
class StackUsingQueue {
           private Queue<Integer> q1;
           private Queue<Integer> q2;

           StackUsingQueue() {
               q1 = new LinkedList<>();
               q2 = new LinkedList<>();
```

```
        }

        // Push Efficient

        public void push(int x) {
            q2.add(x);
            while (!q1.isEmpty()) {
                q2.add(q1.remove());
            }
            Queue<Integer> temp = q1;
            q1 = q2;
            q2 = temp;
        }

        public int pop() {
            if (q1.isEmpty()) {
                throw new RuntimeException("Stack is Empty");
            }
            return q1.remove();
        }

        public int peek() {
            if (q1.isEmpty()) {
                throw new RuntimeException("Stack is Empty");
            }
            return q1.peek();
        }
    }
```

Time & Space Complexity

Time Complexity: Push O(n), Pop O(1) | Space Complexity: O(n)

172. Implement Queue using Stack

Understanding the Problem NEW

The reverse of the previous problem: implement a Queue (FIFO) using only Stacks (LIFO) as building blocks.

Brute Force Approach NEW

Similarly, you could reverse the entire first stack into a second stack on every single enqueue or dequeue operation to fix the ordering — correct but doing this on every operation is wasteful if operations are interleaved unpredictably.

Why We Optimize NEW

Use **two stacks** — an 'input' stack and an 'output' stack. Enqueue always just pushes onto the input stack (O(1)). Dequeue checks the output stack first: if it's empty, pour the *entire* input stack into it (which naturally reverses the order, turning LIFO into FIFO order), then pop from the output stack; if the output stack already has elements, just pop from it directly. The key insight is that each individual element only ever gets moved from input to output **once** over its lifetime, so even though a single dequeue call might occasionally cost O(n) (when the transfer happens), the **amortized** cost across many operations averages out to O(1) per operation — a valuable concept (amortized analysis) worth understanding here for the first time in this book.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Queue using two stacks.

Visual Diagram

```
Queue using Stacks (Enqueue Efficient):
       Enqueue: Push to stack1
       Dequeue: If stack2 empty, move all from stack1 to stack2

       Enqueue 5: s1: [5], s2: []
       Enqueue 3: s1: [5,3], s2: []
       Enqueue 8: s1: [5,3,8], s2: []
```

```
       Dequeue():
         Move s1 to s2: s1: [], s2: [8,3,5]
         Pop from s2: 5 → s2: [8,3]
         Return 5

       Queue using Stacks (Dequeue Efficient):
       Enqueue: Push to stack2, move to stack1 if needed
       Dequeue: Pop from stack1
```

Solution Approach

Concept: Two stacks with transfer on demand.

Algorithm (Enqueue Efficient)

Algorithm QUEUE_USING_STACK:

s1, s2 = Stack()

enqueue(x):

s1.push(x)

dequeue():

If s2.isEmpty():

While s1 not empty:

s2.push(s1.pop())

Return s2.pop()

Java Implementation

```
class QueueUsingStack {
           private Stack<Integer> s1;
           private Stack<Integer> s2;

           QueueUsingStack() {
               s1 = new Stack<>();
               s2 = new Stack<>();
           }

           // Enqueue Efficient
           public void enqueue(int x) {
               s1.push(x);
           }

           public int dequeue() {
               if (s2.isEmpty()) {
```

```
                while (!s1.isEmpty()) {
                    s2.push(s1.pop());

                }
            }
            if (s2.isEmpty()) {
                throw new RuntimeException("Queue is Empty");
            }
            return s2.pop();
        }

        public int front() {
            if (s2.isEmpty()) {
                while (!s1.isEmpty()) {
                    s2.push(s1.pop());
                }
            }
            if (s2.isEmpty()) {
                throw new RuntimeException("Queue is Empty");
            }
            return s2.peek();
        }
    }
```

Time & Space Complexity

Time Complexity: Enqueue O(1), Dequeue amortized O(1) | Space Complexity: O(n)

173. Implement Stack using LinkedList

Understanding the Problem NEW

Implement a Stack using a LinkedList instead of an array as the backing storage.

Brute Force Approach NEW

Using an array-backed stack (problem 169) works fine, but arrays have a fixed capacity that needs resizing (typically doubling and copying, an O(n) operation) once they fill up.

Why We Optimize NEW

A LinkedList naturally grows one node at a time with no pre-allocated capacity limit — pushing means adding a new node at the **head** (O(1), no shifting or resizing needed), and popping means removing the head node (O(1)). This avoids the occasional O(n) resize cost that array-backed implementations can incur, at the modest expense of a little extra memory per element for the 'next' pointer.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Stack using LinkedList.

Visual Diagram

```
LinkedList Stack:
       Head → [5] → [3] → [8] → null
               ↑
              Top

       push(10):
       Head → [10] → [5] → [3] → [8] → null

       pop():
       Head → [5] → [3] → [8] → null
       Returns 10

       Node Structure:
       class Node {
           int data;
           Node next;
```

```
            Node(int data) { this.data = data; }
        }
```

Solution Approach

Concept: Use LinkedList with head as top.

Algorithm

```
Algorithm STACK_LL:
            head = null

            push(x):
                newNode = Node(x)
                newNode.next = head
                head = newNode

            pop():
                If head == null: Underflow
                val = head.data
                head = head.next
                Return val
```

Java Implementation

```
class StackLinkedList {
            private static class Node {
                int data;
                Node next;
                Node(int data) {
                    this.data = data;
                }
            }

            private Node head;

            public void push(int x) {
                Node newNode = new Node(x);
                newNode.next = head;

               head = newNode;
            }

            public int pop() {
                if (head == null) {
                    throw new RuntimeException("Stack Underflow");
                }
                int val = head.data;
                head = head.next;
                return val;
```

```
        }

        public int peek() {
            if (head == null) {
                throw new RuntimeException("Stack is Empty");
            }
            return head.data;
        }

        public boolean isEmpty() {
            return head == null;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(n)

174. Implement Queue using LinkedList

Understanding the Problem NEW

Implement a Queue using a LinkedList instead of an array.

Brute Force Approach NEW

An array-backed queue (problem 170) using the circular-array trick works well, but still has a fixed maximum capacity that needs resizing occasionally.

Why We Optimize NEW

With a LinkedList, keep two references — `head` (front of queue, where you dequeue) and `tail` (rear, where you enqueue) — so both enqueue (add after tail) and dequeue (remove from head) are O(1), with no capacity limit and no need for the circular-index wraparound trick that an array implementation requires. The trade-off, as with problem 173, is a bit more memory overhead per element for the pointer fields.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Queue using LinkedList.

Visual Diagram

```
LinkedList Queue:
       Head → [5] → [3] → [8] → null ← Tail

       enqueue(10):
       Head → [5] → [3] → [8] → [10] → null ← Tail

       dequeue():
       Head → [3] → [8] → [10] → null ← Tail
       Returns 5

       Node Structure:
       class Node {
           int data;
           Node next;
```

```
            Node(int data) { this.data = data; }
        }
```

Solution Approach

Concept: Use head for front, tail for rear.

Algorithm

```
Algorithm QUEUE_LL:
            head = null, tail = null

             enqueue(x):
                 newNode = Node(x)
                 If tail == null:
                     head = tail = newNode
                 Else:
                     tail.next = newNode
                     tail = newNode

             dequeue():
                 If head == null: Underflow
                 val = head.data
                 head = head.next
                 If head == null: tail = null
                 Return val
```

Java Implementation

```
class QueueLinkedList {
            private static class Node {
                int data;
                Node next;
                Node(int data) {
                    this.data = data;
                }
            }

             private Node head;
             private Node tail;
             public void enqueue(int x) {

               Node newNode = new Node(x);
               if (tail == null) {
                   head = tail = newNode;
               } else {
                   tail.next = newNode;
                   tail = newNode;
               }
           }
```

```
        public int dequeue() {
            if (head == null) {
                throw new RuntimeException("Queue Underflow");
            }
            int val = head.data;
            head = head.next;
            if (head == null) {
                tail = null;
            }
            return val;
        }

        public int front() {
            if (head == null) {
                throw new RuntimeException("Queue is Empty");
            }
            return head.data;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(n)

175. Check for Balanced Parentheses

Understanding the Problem NEW

Given a string containing various types of brackets — `()`, `[]`, `{}` — check if every opening bracket has a matching, correctly-nested closing bracket. E.g. `"([{}])"` is balanced, but `"([)]"` is not (crossed, not nested).

Brute Force Approach NEW

Without a stack, you might try counting opening and closing brackets of each type separately and checking the counts match — but this misses ordering/nesting entirely (e.g. `"([)]"` has equal counts of each bracket type but is still invalid).

Why We Optimize NEW

A **stack** is exactly the right structure here because bracket matching is inherently LIFO: the *most recently opened* bracket must be the *next one closed*. Push every opening bracket onto the stack; on every closing bracket, check that it matches whatever is currently on top of the stack (popping it if so, or declaring the string unbalanced if not). If the stack is empty at the end, every bracket was properly matched. This correctly captures nesting order in a single O(n) pass, something a simple counting approach fundamentally cannot do.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if parentheses in string are balanced.

Visual Diagram

```
Input: "({[]})"
       Step 1: '(' push
       Step 2: '{' push
       Step 3: '[' push
       Step 4: ']' matches '[' → pop
       Step 5: '}' matches '{' → pop
       Step 6: ')' matches '(' → pop
       Result: Balanced ✓

       Input: "([)]"
```

```
Step 1: '(' push
Step 2: '[' push
Step 3: ')' doesn't match '[' → Unbalanced ✗

Input: "((()))"
Step 1: '(' push
Step 2: '(' push
Step 3: '(' push
Step 4: ')' matches '(' → pop
Step 5: ')' matches '(' → pop
End with '(' remaining → Unbalanced ✗
```

Solution Approach

Concept: Stack for opening brackets.

Algorithm

```
Algorithm IS_BALANCED(s):
           stack = Stack()
           For each char in s:
               If char in '({[':
                   stack.push(char)
               Else:
                   If stack.isEmpty(): Return false
                   top = stack.pop()
                   If not match(top, char): Return false
           Return stack.isEmpty()
```

Java Implementation

```
class Solution {
           public boolean isBalanced(String s) {
               Stack<Character> stack = new Stack<>();

                 for (char c : s.toCharArray()) {
                     if (c == '(' || c == '{' || c == '[') {
                         stack.push(c);
                     } else {
                         if (stack.isEmpty()) return false;
                         char top = stack.pop();
                         if (!isMatching(top, c)) return false;
                     }
                 }
                 return stack.isEmpty();

         }

         private boolean isMatching(char open, char close) {
             return (open == '(' && close == ')') ||
```

```
                    (open == '{' && close == '}') ||
                    (open == '[' && close == ']');
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

176. Implement Min Stack

Understanding the Problem NEW

Design a stack that supports the usual push/pop, plus a `getMin()` operation that returns the current minimum element in the stack — all in O(1) time.

Brute Force Approach NEW

You could scan the entire stack every time `getMin()` is called to find the current minimum — O(n) per call, which defeats the purpose if getMin() is called frequently.

Why We Optimize NEW

Maintain a **second stack** alongside the main one, which at every point tracks the minimum value *as of that point in the stack's history* — whenever you push a new value onto the main stack, you also push the smaller of (new value, current min) onto the min-stack; whenever you pop from the main stack, you also pop from the min-stack. This way, the top of the min-stack always instantly reflects the current minimum in O(1), because it's updated incrementally alongside every push/pop rather than recomputed from scratch on each query — trading a little extra memory (the second stack) for O(1) instead of O(n) queries.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement stack that supports getMin() in O(1).

Visual Diagram

```
Operations:
       push(5): stack=[5], minStack=[5] → min=5
       push(3): stack=[5,3], minStack=[5,3] → min=3
       push(8): stack=[5,3,8], minStack=[5,3] → min=3
       push(2): stack=[5,3,8,2], minStack=[5,3,2] → min=2
       pop(): stack=[5,3,8], minStack=[5,3] → min=3

       Alternative: Store only one stack with (value, min)
       Push: push (5,5), (3,3), (8,3), (2,2)
       Pop: pop (2,2) → min back to 3
```

Solution Approach

Concept: Maintain a separate min stack or store min with value.

Algorithm

```
Algorithm MIN_STACK:
           stack = Stack()
           minStack = Stack()

           push(x):
               stack.push(x)
               If minStack.isEmpty() or x <= minStack.peek():
                    minStack.push(x)

           pop():
               val = stack.pop()
               If val == minStack.peek():
                   minStack.pop()
               Return val

           getMin():
               Return minStack.peek()
```

Java Implementation

```
class MinStack {
           private Stack<Integer> stack;
           private Stack<Integer> minStack;

           MinStack() {
               stack = new Stack<>();
               minStack = new Stack<>();
           }

           public void push(int x) {
               stack.push(x);
               if (minStack.isEmpty() || x <= minStack.peek()) {
                   minStack.push(x);
               }
           }

           public void pop() {
               if (stack.isEmpty()) return;

              int val = stack.pop();
              if (val == minStack.peek()) {
                  minStack.pop();
              }
          }
```

```
        public int top() {
            return stack.peek();
        }

        public int getMin() {
            return minStack.peek();
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(n)

177. Infix to Postfix Conversion

Understanding the Problem NEW

Convert an infix expression (the normal way we write math, like `a+b*c`, where operators sit between operands) into postfix notation (like `abc*+`, where operators come after their operands — this is what makes evaluation unambiguous without needing parentheses or precedence rules at evaluation time).

Brute Force Approach NEW

Without a stack, you might try to track operator precedence and parenthesization with nested recursive parsing, similar to writing a small parser — doable, but considerably more complex than necessary for this specific conversion.

Why We Optimize NEW

A single stack of operators does this cleanly: scan left to right, sending operands directly to the output; when you see an operator, **pop and output any operators currently on the stack that have equal or higher precedence** (since they must be applied first), then push the new operator; parentheses are handled by pushing '(' and popping back to it when ')' is seen. This is a clean single-pass, O(n) algorithm that handles precedence and parentheses uniformly through the stack, rather than requiring a full recursive expression parser.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert infix expression to postfix notation.

Visual Diagram

```
Input: a + b * c
       Output: a b c * +

       Step-by-step:
       a: output "a"
       +: stack [+]
       b: output "a b"
       *: stack [+, *]
```

```
c: output "a b c"
End: pop all → "a b c * +"

Input: (a + b) * c
Output: a b + c *

Step-by-step:
(: stack [(]
a: output "a"
+: stack [(, +]
b: output "a b"
): pop until ( → output "a b +"
*: stack [*]
c: output "a b + c"
End: pop * → "a b + c *"
```

Solution Approach

Concept: Stack with operator precedence.

Algorithm

```
Algorithm INFIX_TO_POSTFIX(exp):
           stack = Stack()
           result = ""
           precedence = {'+':1, '-':1, '*':2, '/':2, '^':3}

            For each char in exp:
                If char is operand:
                    result += char
                Else If char == '(':
                    stack.push(char)
                Else If char == ')':
                    While stack.peek() != '(':
                        result += stack.pop()
                    stack.pop() // remove '('
                Else:
                    While !stack.isEmpty() and stack.peek() != '(' and
                          precedence[stack.peek()] >= precedence[char]:
                        result += stack.pop()
                    stack.push(char)

             While !stack.isEmpty():
                 result += stack.pop()

          Return result
```

Java Implementation

```
class Solution {
          public String infixToPostfix(String exp) {
              Map<Character, Integer> precedence = new HashMap<>();
              precedence.put('+', 1);
              precedence.put('-', 1);
              precedence.put('*', 2);
              precedence.put('/', 2);
              precedence.put('^', 3);

              Stack<Character> stack = new Stack<>();
              StringBuilder result = new StringBuilder();

              for (char c : exp.toCharArray()) {
                  if (Character.isLetterOrDigit(c)) {
                      result.append(c);
                  } else if (c == '(') {
                      stack.push(c);
                  } else if (c == ')') {
                      while (!stack.isEmpty() && stack.peek() != '(') {
                          result.append(stack.pop());
                      }
                      stack.pop(); // Remove '('
                  } else { // Operator
                      while (!stack.isEmpty() && stack.peek() != '(' &&
                             precedence.getOrDefault(stack.peek(), 0) >=
precedence.get(c)) {
                          result.append(stack.pop());
                      }
                      stack.push(c);
                  }
              }

              while (!stack.isEmpty()) {
                  result.append(stack.pop());
              }
              return result.toString();
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

178. Prefix to Infix Conversion

Understanding the Problem NEW

Convert a prefix expression (operator before its operands, like `+a*bc`) into infix notation (the normal `a+b*c` style, with parentheses added as needed to make precedence explicit).

Brute Force Approach NEW

You could try to build a full expression tree first and then read it back out in-order with parentheses — a valid approach, but more machinery (an explicit tree structure) than a simple stack-based method needs.

Why We Optimize NEW

Since prefix means 'operator first, then its two operands,' scanning the expression from **right to left** and using a stack works naturally: when you see an operand, push it; when you see an operator, pop the two most recent items (which are its two operands, since you're scanning backward), combine them into a single string wrapped in parentheses with the operator in the middle, and push that combined string back onto the stack. By the end, the stack holds exactly one item — the fully parenthesized infix expression — all without building an explicit tree data structure.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert prefix expression to infix notation.

Visual Diagram

```
Input: * + A B C
       Output: (A + B) * C

       Step-by-step (read right to left):
       C: push C
       B: push B
       A: push A
       +: pop A, B → (A + B) → push
       *: pop (A+B), C → ((A+B) * C) → push
```

```
Input: - / A B * C D
Output: (A / B) - (C * D)

Read right to left:
D: push D
C: push C
*: pop C, D → (C*D) → push
B: push B
A: push A
/: pop A, B → (A/B) → push
-: pop (A/B), (C*D) → (A/B)-(C*D)
```

Solution Approach

Concept: Stack with right-to-left traversal.

Algorithm

```
Algorithm PREFIX_TO_INFIX(exp):
           stack = Stack()
           For i = exp.length-1 down to 0:
               If exp[i] is operand:
                   stack.push(exp[i])
               Else:
                   op1 = stack.pop()
                   op2 = stack.pop()
                   stack.push("(" + op1 + exp[i] + op2 + ")")
           Return stack.pop()
```

Java Implementation

```
class Solution {
           public String prefixToInfix(String exp) {
               Stack<String> stack = new Stack<>();

                 for (int i = exp.length() - 1; i >= 0; i--) {
                     char c = exp.charAt(i);

                     if (Character.isLetterOrDigit(c)) {
                         stack.push(String.valueOf(c));
                     } else {
                         String op1 = stack.pop();
                         String op2 = stack.pop();
                         stack.push("(" + op1 + c + op2 + ")");
                     }

              }
              return stack.pop();
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

179. Prefix to Postfix Conversion

Understanding the Problem NEW

Convert a prefix expression directly into postfix notation, without going through infix as an intermediate step.

Brute Force Approach NEW

You could convert prefix → infix (problem 178) and then infix → postfix (problem 177) as two separate stages — technically correct, but doing two full conversions when a direct one exists is unnecessary extra work.

Why We Optimize NEW

Using the same right-to-left scanning idea as problem 178: when you see an operand, push it; when you see an operator, pop its two operands and instead of wrapping them with parentheses and the operator in the middle (infix style), simply **concatenate the two operands followed by the operator** (postfix style) and push that combined string back. This direct conversion skips the intermediate infix representation entirely, doing in one pass what the two-stage approach would do in two.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert prefix expression to postfix notation.

Visual Diagram

```
Input: * + A B C
       Output: A B + C *

       Step-by-step (read right to left):
       C: push C
       B: push B
       A: push A
       +: pop A, B → A B + → push
       *: pop AB+, C → AB+ C * → push

       Input: - / A B * C D
       Output: A B / C D * -
```

```
Read right to left:
D: push D
C: push C
*: pop C, D → C D * → push
B: push B
A: push A
/: pop A, B → A B / → push
-: pop AB/, CD* → AB/ CD* - → push
```

Solution Approach

Concept: Stack with right-to-left traversal.

Algorithm

```
Algorithm PREFIX_TO_POSTFIX(exp):
           stack = Stack()
           For i = exp.length-1 down to 0:
               If exp[i] is operand:
                   stack.push(exp[i])
               Else:
                   op1 = stack.pop()
                   op2 = stack.pop()
                   stack.push(op1 + op2 + exp[i])
           Return stack.pop()
```

Java Implementation

```
class Solution {
           public String prefixToPostfix(String exp) {
               Stack<String> stack = new Stack<>();

                 for (int i = exp.length() - 1; i >= 0; i--) {
                     char c = exp.charAt(i);

                     if (Character.isLetterOrDigit(c)) {
                         stack.push(String.valueOf(c));
                     } else {
                         String op1 = stack.pop();
                         String op2 = stack.pop();
                         stack.push(op1 + op2 + c);
                     }

              }
              return stack.pop();
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

180. Postfix to Prefix Conversion

Understanding the Problem NEW

Convert a postfix expression (operands then operator, like `abc*+`) into prefix notation (`+a*bc`).

Brute Force Approach NEW

Similarly, you could convert postfix → infix (problem 181) then infix → prefix (problem 182) — two conversions instead of one direct pass.

Why We Optimize NEW

Postfix means operands come *before* their operator, so this time scan **left to right**: push operands as you see them; when you hit an operator, pop the two most recent operands (in postfix, they appear before the operator) and combine them as `operator + operand1 + operand2` (prefix order), pushing that combined string back. This mirrors problem 179's approach but scanning in the opposite direction, since postfix and prefix put their operators on opposite sides relative to the operands.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert postfix expression to prefix notation.

Visual Diagram

```
Input: A B C * +
       Output: + A * B C

       Step-by-step:
       A: push A
       B: push B
       C: push C
       *: pop C, B → * B C → push
       +: pop *BC, A → + A * B C

       Input: A B / C D * -
       Output: - / A B * C D
```

```
A: push A
B: push B
/: pop B, A → / A B → push
C: push C
D: push D
*: pop D, C → * C D → push
-: pop *CD, /AB → - /AB *CD
```

Solution Approach

Concept: Stack with left-to-right traversal.

Algorithm

```
Algorithm POSTFIX_TO_PREFIX(exp):
           stack = Stack()
           For each char in exp:
               If char is operand:
                   stack.push(char)
               Else:
                   op2 = stack.pop()
                   op1 = stack.pop()
                   stack.push(char + op1 + op2)
           Return stack.pop()
```

Java Implementation

```
class Solution {
           public String postfixToPrefix(String exp) {
               Stack<String> stack = new Stack<>();

                 for (char c : exp.toCharArray()) {
                     if (Character.isLetterOrDigit(c)) {
                         stack.push(String.valueOf(c));
                     } else {
                         String op2 = stack.pop();
                         String op1 = stack.pop();
                         stack.push(c + op1 + op2);
                     }
                 }
                 return stack.pop();
            }

     }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

181. Postfix to Infix Conversion

Understanding the Problem NEW

Convert a postfix expression into infix notation (with appropriate parentheses).

Brute Force Approach NEW

You could build a full expression tree from the postfix expression and then read it back in-order — valid, but more structure than needed.

Why We Optimize NEW

Scan left to right with a stack: push operands; when you hit an operator, pop the two most recent operands and combine them as `(operand1 operator operand2)` — parenthesized infix style — pushing the combined string back. Same left-to-right scanning direction as problem 180 (since postfix always processes operators after seeing both their operands), just building a parenthesized infix string instead of a prefix one.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert postfix expression to infix notation.

Visual Diagram

```
Input: A B C * +
       Output: A + (B * C)

       Step-by-step:
       A: push A
       B: push B
       C: push C
       *: pop C, B → (B * C) → push
       +: pop (B*C), A → A + (B*C) → push

       Input: A B / C D * -
       Output: (A / B) - (C * D)

       A: push A
       B: push B
       /: pop B, A → (A/B) → push
```

```
        C: push C
        D: push D
        *: pop D, C → (C*D) → push
        -: pop (C*D), (A/B) → (A/B) - (C*D)
```

Solution Approach

Concept: Stack with left-to-right traversal.

Algorithm

```
Algorithm POSTFIX_TO_INFIX(exp):
            stack = Stack()
            For each char in exp:
                If char is operand:
                    stack.push(char)
                Else:
                    op2 = stack.pop()
                    op1 = stack.pop()
                    stack.push("(" + op1 + char + op2 + ")")
            Return stack.pop()
```

Java Implementation

```
class Solution {
            public String postfixToInfix(String exp) {
                Stack<String> stack = new Stack<>();

                  for (char c : exp.toCharArray()) {
                      if (Character.isLetterOrDigit(c)) {
                          stack.push(String.valueOf(c));
                      } else {
                          String op2 = stack.pop();
                          String op1 = stack.pop();
                          stack.push("(" + op1 + c + op2 + ")");
                      }
                  }
                  return stack.pop();
             }

      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

182. Convert Infix To Prefix Notation

Understanding the Problem NEW

Convert an infix expression directly into prefix notation.

Brute Force Approach NEW

You might try to directly adapt the infix-to-postfix algorithm (problem 177) with modified precedence rules scanned from a different direction, but getting the details right without a clear method is error-prone.

Why We Optimize NEW

There's an elegant reduction here: **reverse the infix expression**, swap every '(' with ')' and vice versa (since reversing an expression flips its parenthesis structure), run the standard infix-to-postfix algorithm (problem 177) on this modified string, and then **reverse the resulting postfix string** to get the prefix result. This clever transformation reuses the already-solved infix-to-postfix algorithm as a subroutine, rather than deriving a brand new algorithm from scratch for prefix conversion — a good example of solving a new problem by reducing it to one you've already solved.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert infix expression to prefix notation.

Visual Diagram

```
Input: A + B * C
       Output: + A * B C

       Step 1: Reverse the string: C * B + A
       Step 2: Convert to postfix (with swapped precedence)
       Step 3: Reverse the result

       Detailed:
       Reverse: C * B + A
       Postfix conversion:
       C: push C
       *: stack [*]
```

```
        B: push B
        +: pop *, push + → C B *
        A: push A
        Pop + → C B * A +
        Reverse: + A * B C

        Input: (A + B) * C
        Output: * + A B C

        Reverse: C * ) B + A (
        Postfix: C B A + *
        Reverse: * + A B C
```

Solution Approach

Concept: Reverse, swap parentheses, convert to postfix, reverse.

Algorithm

```
Algorithm INFIX_TO_PREFIX(exp):
            Reverse exp
            Swap '(' and ')' in reversed string
            Convert to Postfix (with swapped precedence)
            Reverse result
            Return result
```

Java Implementation

```
class Solution {
            public String infixToPrefix(String exp) {
                // Reverse the expression
                StringBuilder reversed = new StringBuilder(exp).reverse();

                  // Swap parentheses
                  for (int i = 0; i < reversed.length(); i++) {
                      if (reversed.charAt(i) == '(') {
                          reversed.setCharAt(i, ')');
                      } else if (reversed.charAt(i) == ')') {
                          reversed.setCharAt(i, '(');
                      }
                  }

                  // Convert to postfix

               String postfix = infixToPostfix(reversed.toString());

               // Reverse result
               return new StringBuilder(postfix).reverse().toString();
           }
```

```
          private String infixToPostfix(String exp) {
              Map<Character, Integer> precedence = new HashMap<>();
              precedence.put('+', 1);
              precedence.put('-', 1);
              precedence.put('*', 2);
              precedence.put('/', 2);
              precedence.put('^', 3);

              Stack<Character> stack = new Stack<>();
              StringBuilder result = new StringBuilder();

              for (char c : exp.toCharArray()) {
                  if (Character.isLetterOrDigit(c)) {
                      result.append(c);
                  } else if (c == '(') {
                      stack.push(c);
                  } else if (c == ')') {
                      while (!stack.isEmpty() && stack.peek() != '(') {
                           result.append(stack.pop());
                      }
                      stack.pop();
                  } else {
                      while (!stack.isEmpty() && stack.peek() != '(' &&
                              precedence.getOrDefault(stack.peek(), 0) >=
precedence.get(c)) {
                           result.append(stack.pop());
                      }
                      stack.push(c);
                  }
              }

              while (!stack.isEmpty()) {
                  result.append(stack.pop());
              }
              return result.toString();
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

183. Next Greater Element

Understanding the Problem NEW

For every element in an array, find the next element to its right that is strictly greater than it (or -1/None if no such element exists). E.g. in `[4,5,2,10]` : NGE of 4 is 5, NGE of 5 is 10, NGE of 2 is 10, NGE of 10 is none.

Brute Force Approach NEW

For each element, scan forward through every element to its right until you find one that's bigger — $O(n^2)$ in the worst case, since you might rescan large portions of the array repeatedly from different starting points.

Why We Optimize NEW

Process the array from **right to left**, maintaining a stack that always holds a **decreasing** sequence of 'candidate' values (potential next-greater-elements for whatever comes before them). For each new element, pop off anything on the stack that's smaller or equal to it (since those values can never be the answer for anything earlier — the current element is a better, closer candidate), then whatever remains on top of the stack (if anything) is the current element's actual next greater element; finally push the current element itself onto the stack. Because each element is pushed and popped from the stack at most once across the whole process, the total work is O(n) rather than $O(n^2)$ — this is the foundational 'monotonic stack' pattern that powers nearly every remaining problem through 195.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the next greater element for each array element.

Visual Diagram

```
Array: [4, 5, 2, 10, 8]
       Next Greater: [5, 10, 10, -1, -1]

       Step-by-step (traverse from right):
       i=4: stack=[], NGE[4]=-1, push 8
       i=3: stack=[8], 10>8 → NGE[3]=-1, push 10
```

```
        i=2: stack=[10], 2<10 → NGE[2]=10, push 2
        i=1: stack=[2,10], 5>2 → pop, 5<10 → NGE[1]=10, push 5
        i=0: stack=[5,10], 4<5 → NGE[0]=5, push 4

        Array: [1, 3, 2, 4]
        NGE: [3, 4, 4, -1]

        Traverse from right:
        i=3: [] → -1, push 4
        i=2: [4] → 4, push 2
        i=1: [2,4] → 4, push 3
        i=0: [3,4] → 3
```

Solution Approach

Concept: Stack maintains decreasing order.

Algorithm

```
Algorithm NEXT_GREATER(arr):
            stack = Stack()
            result = new int[n]

            For i = n-1 down to 0:
                While !stack.isEmpty() and stack.peek() <= arr[i]:
                    stack.pop()
                result[i] = stack.isEmpty() ? -1 : stack.peek()
                stack.push(arr[i])

            Return result
```

Java Implementation

```
class Solution {
            public int[] nextGreaterElement(int[] arr) {
                int n = arr.length;
                int[] result = new int[n];
                Stack<Integer> stack = new Stack<>();

                  for (int i = n - 1; i >= 0; i--) {
                      while (!stack.isEmpty() && stack.peek() <= arr[i]) {
                          stack.pop();
                      }
                      result[i] = stack.isEmpty() ? -1 : stack.peek();
                      stack.push(arr[i]);
                  }

               return result;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

184. Next Greater Element 2

Understanding the Problem NEW

Same as Next Greater Element, but the array is **circular** — meaning after the last element, you wrap back around to the beginning when searching for a next greater element.

Brute Force Approach NEW

You could physically duplicate the array (concatenate it with itself) and run ordinary Next Greater Element on the doubled array — this works but processes 2n elements when a smaller adjustment suffices.

Why We Optimize NEW

Rather than actually duplicating the array in memory, simulate a 'virtual' doubled array by iterating index `i` from `2n-1` i % n to access the actual array — the monotonic stack technique from problem 183 applies completely unchanged otherwise. This achieves the same effect as physically doubling the array, but with O(1) extra indexing arithmetic instead of O(n) extra memory for an actual duplicated array.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find next greater element in circular array.

Visual Diagram

```
Array: [2, 10, 12, 1, 11]
       NGE Circular: [10, 12, -1, 11, 12]

       Traverse array twice (2n):
       i=9 (4): [11] → -1, push 11
       i=8 (3): [11] → 11, push 1
       i=7 (2): [1,11] → 11? Wait, 12>1, push 12
       i=6 (1): [12] → 12, push 10
       i=5 (0): [10,12] → 10? Actually 10>2, push 10
       i=4 (4): [10] → 10? Actually 11>10, wait...

       Let's trace properly:
```

```
Array: [2, 10, 12, 1, 11]
i=9 (4): stack=[], NGE[4]=-1, push 11
i=8 (3): stack=[11], 1<11 → NGE[3]=11, push 1
i=7 (2): stack=[1,11], 12>1, pop; 12>11, pop; empty → NGE[2]=-1, push 12
i=6 (1): stack=[12], 10<12 → NGE[1]=12, push 10
i=5 (0): stack=[10,12], 2<10 → NGE[0]=10, push 2
i=4 (4): stack=[2,10,12], 11>2, pop; 11>10, pop; 11<12 → NGE[4]=12
i=3 (3): stack=[11,12], 1<11 → NGE[3]=11
```

Solution Approach

Concept: Traverse twice using modulo.

Algorithm

```
Algorithm NEXT_GREATER_CIRCULAR(arr):
           n = arr.length
           result = new int[n]
           stack = Stack()
           Arrays.fill(result, -1)

            For i = 0 to 2*n-1:
                index = i % n
                While !stack.isEmpty() && arr[stack.peek()] < arr[index]:
                    result[stack.pop()] = arr[index]
                If i < n:
                    stack.push(index)

            Return result
```

Java Implementation

```
class Solution {
           public int[] nextGreaterElements(int[] nums) {
               int n = nums.length;
               int[] result = new int[n];
               Arrays.fill(result, -1);
               Stack<Integer> stack = new Stack<>();

                 for (int i = 0; i < 2 * n; i++) {
                     int index = i % n;
                     while (!stack.isEmpty() && nums[stack.peek()] < nums[index]) {
                         result[stack.pop()] = nums[index];

                    }
                    if (i < n) {
                        stack.push(index);
                    }
              }
              return result;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

185. Next Smaller Element

Understanding the Problem NEW

The mirror image of Next Greater Element: for every element, find the next element to its right that is strictly smaller than it.

Brute Force Approach NEW

Same naive option as before: for each element, scan forward until you find a smaller one — O(n²) worst case.

Why We Optimize NEW

Exactly the same monotonic stack technique as problem 183, with the comparison direction flipped: maintain a stack of an **increasing** sequence of candidates (instead of decreasing), popping anything larger or equal since it can never be a valid 'next smaller' answer for what comes before it. Recognizing that 'next smaller' is just 'next greater' with the inequality flipped — reusing the exact same algorithm shape — is the actual lesson here, not a new technique.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the next smaller element for each array element.

Visual Diagram

```
Array: [4, 5, 2, 10, 8]
       Next Smaller: [2, 2, -1, 8, -1]

       Traverse from right:
       i=4: stack=[], NSE[4]=-1, push 8
       i=3: stack=[8], 10>8 → NSE[3]=8, push 10
       i=2: stack=[10,8], 2<10, pop; 2<8, pop; empty → NSE[2]=-1, push 2
       i=1: stack=[2], 5>2 → NSE[1]=2, push 5
       i=0: stack=[5,2], 4<5, pop; 4>2 → NSE[0]=2, push 4
```

Solution Approach

Concept: Similar to NGE but with smaller condition.

Algorithm

```
Algorithm NEXT_SMALLER(arr):
           stack = Stack()
           result = new int[n]

           For i = n-1 down to 0:
               While !stack.isEmpty() && stack.peek() >= arr[i]:
                   stack.pop()
               result[i] = stack.isEmpty() ? -1 : stack.peek()
               stack.push(arr[i])

           Return result
```

Java Implementation

```
class Solution {
           public int[] nextSmallerElement(int[] arr) {
               int n = arr.length;
               int[] result = new int[n];
               Stack<Integer> stack = new Stack<>();

                 for (int i = n - 1; i >= 0; i--) {
                     while (!stack.isEmpty() && stack.peek() >= arr[i]) {
                         stack.pop();
                     }
                     result[i] = stack.isEmpty() ? -1 : stack.peek();
                     stack.push(arr[i]);
                 }
                 return result;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

186. Number of NGEs to the Right

Understanding the Problem NEW

Instead of finding just the *first* next-greater-element, count *how many* elements to the right of each position are greater than it.

Brute Force Approach NEW

For each element, scan every element to its right and count how many are bigger — O(n²).

Why We Optimize NEW

This can also be solved with a monotonic stack, but tracking counts alongside values: rather than storing just candidate values on the stack, you also track how many elements have already been 'accounted for' relative to each candidate, incrementing counts as elements get popped in a way that reflects how many subsequent elements were bigger. This still processes each element a bounded number of times overall, keeping it at O(n) instead of O(n²), extending the same monotonic-stack contribution-counting idea used elsewhere in this chapter (like Sum of Subarray Minimums, problem 188).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count how many next greater elements exist to the right.

Visual Diagram

```
Array: [3, 4, 2, 7, 5, 8, 10, 6]
       For each element, count NGEs:

       i=0 (3): elements >3: 4,7,5,8,10,6 → 6
       i=1 (4): elements >4: 7,5,8,10,6 → 5
       i=2 (2): elements >2: 7,5,8,10,6 → 5
       i=3 (7): elements >7: 8,10 → 2
       i=4 (5): elements >5: 8,10,6 → 3
       i=5 (8): elements >8: 10 → 1
       i=6 (10): elements >10: none → 0
       i=7 (6): elements >6: none → 0
```

Solution Approach

Concept: Count greater elements to right.

Algorithm

```
Algorithm COUNT_NGE(arr):
           n = arr.length
           result = new int[n]

           For i = 0 to n-1:
               count = 0
               For j = i+1 to n-1:
                   If arr[j] > arr[i]: count++
               result[i] = count

           Return result
```

Java Implementation

```
class Solution {
           public int[] countNGE(int[] arr) {
               int n = arr.length;
               int[] result = new int[n];

                 for (int i = 0; i < n; i++) {
                     int count = 0;
                     for (int j = i + 1; j < n; j++) {
                         if (arr[j] > arr[i]) {
                             count++;
                         }
                     }
                     result[i] = count;
                 }
                 return result;
           }
      }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

187. Trapping Rainwater

Understanding the Problem NEW

Given a histogram of wall heights, calculate how much rainwater can be trapped between the walls after it rains. E.g. heights `[0,1,0,2,1,0,1,3,2,1,2,1]` traps 6 units of water.

Brute Force Approach NEW

For every position, scan left to find the tallest wall to its left and scan right to find the tallest wall to its right, then the water trapped at that position is `min(leftMax, rightMax) - height[i]` — correct, but this recomputes the left-max and right-max scans for every single position independently, giving O(n²).

Why We Optimize NEW

You don't need to rescan for every position — precompute the running maximum height **from the left** and **from the right** for every position in two single O(n) passes (storing them in arrays), then combine them in a third O(n) pass using the same formula. This turns O(n²) repeated scanning into three linear passes, O(n) total — reusing already-computed prefix/suffix maximums instead of recalculating them per position. (An even more memory-efficient two-pointer version exists that computes this with O(1) extra space instead of the two extra arrays, by cleverly advancing whichever side currently has the smaller max — worth knowing as a further refinement.)

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate how much rainwater can be trapped.

Visual Diagram

```
Array: [0,1,0,2,1,0,1,3,2,1,2,1]
       Water trapped = 6 units

       Visual:
                ■
            ■   ■ ■     ■
          ■   ■ ■ ■ ■ ■ ■
```

```
      ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■
      0 1 0 2 1 0 1 3 2 1 2 1

      Two-pointer approach:
      left=0, right=11, leftMax=0, rightMax=0, water=0

      height[0]=0 < height[11]=1:
        leftMax=0, water+=0, left++ → 1
      height[1]=1 < height[11]=1:
        leftMax=1, water+=0, left++ → 2
      height[2]=0 < height[11]=1:
        leftMax=1, water+=1-0=1, left++ → 3
      height[3]=2 > height[11]=1:
        rightMax=1, water+=0, right-- → 10
      ...
      Total water = 6
```

Solution Approach

Concept: Two pointers with max heights.

Algorithm

```
Algorithm TRAP_RAINWATER(height):
          left = 0, right = n-1
          leftMax = 0, rightMax = 0
          water = 0

          While left < right:
              If height[left] < height[right]:
                  If height[left] >= leftMax:
                      leftMax = height[left]
                  Else:
                      water += leftMax - height[left]
                  left++
              Else:
                  If height[right] >= rightMax:
                      rightMax = height[right]
                  Else:
                      water += rightMax - height[right]
                  right--

          Return water
```

Java Implementation

```
class Solution {
          public int trap(int[] height) {

             int left = 0, right = height.length - 1;
```

```
            int leftMax = 0, rightMax = 0;
            int water = 0;

            while (left < right) {
                if (height[left] < height[right]) {
                    if (height[left] >= leftMax) {
                        leftMax = height[left];
                    } else {
                        water += leftMax - height[left];
                    }
                    left++;
                } else {
                    if (height[right] >= rightMax) {
                        rightMax = height[right];
                    } else {
                        water += rightMax - height[right];
                    }
                    right--;
                }
            }
            return water;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

188. Sum of Subarray Minimums

Understanding the Problem NEW

For every possible subarray of the array, find its minimum element, and sum up all of these minimums across every subarray. Directly computing this for every subarray individually is expensive, since there are $O(n^2)$ subarrays.

Brute Force Approach NEW

Generate every subarray, find its minimum (an O(n) scan per subarray in the naive case), and sum them all — $O(n^3)$ total, or $O(n^2)$ if you at least track the running minimum incrementally as you extend each subarray rather than rescanning from scratch.

Why We Optimize NEW

Instead of thinking 'what is the minimum of each subarray,' flip the question around: **for each individual element, in how many subarrays is it the minimum?** An element's contribution to the total sum is (its value) × (the number of subarrays where it's the minimum). Using a monotonic stack, you can find, for each element, how far left and right it extends as 'the current minimum' (i.e. the distance to the nearest smaller element on each side) — multiplying those two distances tells you exactly how many subarrays have this element as their minimum. Summing (value × count) for every element gives the total in O(n), using the same 'contribution technique' that also solves Sum of Subarray Ranges (problem 190) — a much smarter reframing than directly computing every subarray's minimum.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find sum of minimums of all subarrays.

Visual Diagram

```
Array: [3, 1, 2, 4]

       All subarrays and minimums:
       [3] → 3
       [3,1] → 1
```

```
[3,1,2] → 1
[3,1,2,4] → 1
[1] → 1
[1,2] → 1
[1,2,4] → 1
[2] → 2
[2,4] → 2
[4] → 4

Sum = 3+1+1+1+1+1+1+2+2+4 = 17

Using Monotonic Stack:
For each element, find:
- Previous smaller (strict)
- Next smaller (non-strict)

[3, 1, 2, 4]
For 3: prev=-1, next=1 → (1-(-1))*(1-1)=2*0=0? Wrong

Actually: contribution = arr[i] * (i - prevSmaller) * (nextSmaller - i)
```

Solution Approach

Concept: Monotonic stack for contribution.

Algorithm

```
Algorithm SUM_SUBARRAY_MIN(arr):
           n = arr.length
           prevSmaller = new int[n]
           nextSmaller = new int[n]
           stack = Stack()

           // Previous smaller
           For i = 0 to n-1:
               While !stack.isEmpty() && arr[stack.peek()] > arr[i]:
                   stack.pop()
               prevSmaller[i] = stack.isEmpty() ? -1 : stack.peek()
               stack.push(i)

           stack.clear()
           // Next smaller
           For i = n-1 down to 0:
               While !stack.isEmpty() && arr[stack.peek()] >= arr[i]:
                   stack.pop()
               nextSmaller[i] = stack.isEmpty() ? n : stack.peek()
               stack.push(i)

           sum = 0
           For i = 0 to n-1:
```

```
            left = i - prevSmaller[i]
            right = nextSmaller[i] - i
            sum += arr[i] * left * right

        Return sum
```

Java Implementation

```
class Solution {
        public int sumSubarrayMins(int[] arr) {
            int n = arr.length;
            int[] prevSmaller = new int[n];
            int[] nextSmaller = new int[n];
            Stack<Integer> stack = new Stack<>();
            long sum = 0;
            int MOD = 1_000_000_007;

            // Previous smaller
            for (int i = 0; i < n; i++) {
                while (!stack.isEmpty() && arr[stack.peek()] > arr[i]) {
                    stack.pop();
                }
                prevSmaller[i] = stack.isEmpty() ? -1 : stack.peek();
                stack.push(i);
            }

            stack.clear();

            // Next smaller
            for (int i = n - 1; i >= 0; i--) {
                while (!stack.isEmpty() && arr[stack.peek()] >= arr[i]) {
                    stack.pop();
                }
                nextSmaller[i] = stack.isEmpty() ? n : stack.peek();
                stack.push(i);
            }

            for (int i = 0; i < n; i++) {
                long left = i - prevSmaller[i];
                long right = nextSmaller[i] - i;
                sum = (sum + arr[i] * left % MOD * right) % MOD;
            }
            return (int) sum;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

189. Asteroid Collision

Understanding the Problem NEW

Simulate a row of asteroids moving left or right (given as an array of signed integers, where sign indicates direction and magnitude indicates size); when two asteroids moving toward each other collide, the smaller one is destroyed (or both, if equal size), and larger destroys smaller. Determine the final state after all collisions resolve.

Brute Force Approach NEW

Simulate collisions with nested loops, re-scanning the array after each collision to check for new adjacent collisions that the resolution may have created — potentially $O(n^2)$ since each collision can trigger a re-scan.

Why We Optimize NEW

A **stack** naturally models this: process asteroids left to right, and whenever the current asteroid is moving left *and* the top of the stack is moving right (a genuine collision setup), resolve the collision by comparing sizes — possibly popping the stack repeatedly if the current asteroid keeps destroying smaller ones above it, or stopping early if it gets destroyed itself. Because each asteroid is pushed and popped from the stack at most once over the whole simulation, this achieves O(n) total, rather than repeatedly re-scanning the array after each individual collision.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Simulate asteroid collisions.

Visual Diagram

```
Input: [5, 10, -5]
       Output: [5, 10]

       Step 1: 5 → stack [5]
       Step 2: 10 → stack [5,10]
       Step 3: -5 → moving left, collides with 10
       10 > 5 → -5 destroyed, stack [5,10]
```

```
Input: [10, 2, -5]
Output: [10]

Step 1: 10 → [10]
Step 2: 2 → [10,2]
Step 3: -5 → collides with 2
2 < 5 → 2 destroyed, continue with 10
10 > 5 → -5 destroyed, stack [10]

Input: [-5, 10, -5]
Output: [-5, 10]

Step 1: -5 → [-5]
Step 2: 10 → [-5,10]
Step 3: -5 → collides with 10
10 > 5 → -5 destroyed, stack [-5,10]

Input: [8, -8]
Output: []
Step 1: 8 → [8]
Step 2: -8 → collides, equal → both destroyed, []
```

Solution Approach

Concept: Stack with collision handling.

Algorithm

```
Algorithm ASTEROID_COLLISION(asteroids):
           stack = Stack()

            For asteroid in asteroids:
                If asteroid > 0:
                    stack.push(asteroid)
                Else:
                    While !stack.isEmpty() && stack.peek() > 0 && stack.peek() < -
asteroid:
                        stack.pop()
                    If stack.isEmpty() || stack.peek() < 0:
                        stack.push(asteroid)
                    Else If stack.peek() == -asteroid:
                        stack.pop()

            Return stack to array
```

Java Implementation

```
class Solution {

          public int[] asteroidCollision(int[] asteroids) {
```

```
            Stack<Integer> stack = new Stack<>();

            for (int asteroid : asteroids) {
                if (asteroid > 0) {
                    stack.push(asteroid);
                } else {
                    while (!stack.isEmpty() && stack.peek() > 0 && stack.peek() <
-asteroid) {
                         stack.pop();
                    }
                    if (stack.isEmpty() || stack.peek() < 0) {
                         stack.push(asteroid);
                    } else if (stack.peek() == -asteroid) {
                         stack.pop();
                    }
                }
            }

            int[] result = new int[stack.size()];
            for (int i = stack.size() - 1; i >= 0; i--) {
                result[i] = stack.pop();
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

190. Sum of Subarray Ranges

Understanding the Problem NEW

For every subarray, compute its range (max minus min), and sum this value across all subarrays. This combines the ideas of Sum of Subarray Minimums (188) and its mirror image, Sum of Subarray Maximums.

Brute Force Approach NEW

Generate every subarray, compute its max and min directly (an O(n) scan each), sum the ranges — $O(n^3)$, or $O(n^2)$ with incremental tracking.

Why We Optimize NEW

Exactly the same contribution-based reframing as problem 188, applied twice: compute the total sum of subarray **minimums** using a monotonic stack (as in 188), separately compute the total sum of subarray **maximums** using the mirrored monotonic stack technique, and the answer is simply (sum of maximums) minus (sum of minimums) — since range = max - min for every subarray, and sums distribute over subtraction. This reuses the exact same O(n) contribution technique twice rather than inventing a new approach for 'range' specifically.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find sum of (max - min) for all subarrays.

Visual Diagram

```
Array: [1, 2, 3]
       Subarrays and ranges:
       [1] → 0
       [1,2] → 1
       [1,2,3] → 2
       [2] → 0
       [2,3] → 1
       [3] → 0
       Sum = 4

       Method: Sum(max) - Sum(min)
```

```
       Sum(max) for [1,2,3]:
       Using monotonic stack:
       Next greater: [1,2,3] → [1,-1,-1]
       Previous greater: [-1,-1,1]

       Sum of ranges = Sum(max contributions) - Sum(min contributions)
```

Solution Approach

Concept: Use contribution of each element.

Algorithm

```
Algorithm SUM_SUBARRAY_RANGE(arr):
           return sumMax(arr) - sumMin(arr)

       Algorithm sumMax(arr):
           prevGreater = findPrevGreater(arr)
           nextGreater = findNextGreater(arr)
           sum = 0
           For i = 0 to n-1:
               left = i - prevGreater[i]
               right = nextGreater[i] - i
               sum += arr[i] * left * right
           Return sum
```

Java Implementation

```
class Solution {
           public long subArrayRanges(int[] nums) {
               return sumMax(nums) - sumMin(nums);
           }

           private long sumMax(int[] nums) {
               int n = nums.length;
               long sum = 0;
               Stack<Integer> stack = new Stack<>();
               int[] prevGreater = new int[n];
               int[] nextGreater = new int[n];

                 for (int i = 0; i < n; i++) {
                     while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
                         stack.pop();
                     }

                  prevGreater[i] = stack.isEmpty() ? -1 : stack.peek();
                  stack.push(i);
              }

              stack.clear();
```

```
            for (int i = n - 1; i >= 0; i--) {
                while (!stack.isEmpty() && nums[stack.peek()] <= nums[i]) {
                    stack.pop();
                }
                nextGreater[i] = stack.isEmpty() ? n : stack.peek();
                stack.push(i);
            }

            for (int i = 0; i < n; i++) {
                sum += (long) nums[i] * (i - prevGreater[i]) * (nextGreater[i] -
i);
            }
            return sum;
        }

        private long sumMin(int[] nums) {
            int n = nums.length;
            long sum = 0;
            Stack<Integer> stack = new Stack<>();
            int[] prevSmaller = new int[n];
            int[] nextSmaller = new int[n];

            for (int i = 0; i < n; i++) {
                while (!stack.isEmpty() && nums[stack.peek()] > nums[i]) {
                    stack.pop();
                }
                prevSmaller[i] = stack.isEmpty() ? -1 : stack.peek();
                stack.push(i);
            }

            stack.clear();
            for (int i = n - 1; i >= 0; i--) {
                while (!stack.isEmpty() && nums[stack.peek()] >= nums[i]) {
                    stack.pop();
                }
                nextSmaller[i] = stack.isEmpty() ? n : stack.peek();
                stack.push(i);
            }

            for (int i = 0; i < n; i++) {
                sum += (long) nums[i] * (i - prevSmaller[i]) * (nextSmaller[i] -
i);
            }
            return sum;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

191. Remove K Digits

Understanding the Problem NEW

Given a number (as a string of digits) and a count K, remove exactly K digits so that the remaining digits (kept in their original order) form the smallest possible number.

Brute Force Approach NEW

Try every possible way of choosing which K digits to remove (there are C(n,K) ways) and check which resulting number is smallest — combinatorially expensive for anything but tiny inputs.

Why We Optimize NEW

A greedy, stack-based approach works: scan digits left to right, and whenever the current digit is **smaller** than the digit on top of the stack, pop the stack (removing that larger, earlier digit — using up one of your K removals) as long as you still have removals available, since removing a larger digit earlier always makes the resulting number smaller. This greedily builds an increasing sequence on the stack (removing 'bumps' that make the number larger than necessary), achieving the optimal result in a single O(n) pass instead of exploring every combination of removals.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Remove k digits to make smallest number.

Visual Diagram

```
Input: "1432219", k=3
       Output: "1219"

       Step-by-step:
       1: stack [1]
       4: stack [1,4]
       3: 3<4 → pop, stack [1], push 3 → [1,3]
       2: 2<3 → pop, stack [1], push 2 → [1,2]
       2: stack [1,2,2]
       1: 1<2 → pop, [1,2]; 1<2 → pop, [1]; push 1 → [1,1]
```

```
        9: stack [1,1,9]
        Remove last 1 → "1219"

        Input: "10200", k=1
        Output: "200"

        Step 1: 1 → [1]
        Step 2: 0<1 → pop, [0]
        Step 3: 2 → [0,2]
        Step 4: 0<2 → pop, [0,0]
        Step 5: 0 → [0,0,0]
        After removing k digits: "200" (remove leading zeros)
```

Solution Approach

Concept: Stack with monotonic increasing order.

Algorithm

```
Algorithm REMOVE_K_DIGITS(num, k):
           stack = Stack()

           For digit in num:
               While !stack.isEmpty() && k > 0 && stack.peek() > digit:
                   stack.pop()
                   k--
               stack.push(digit)

           While k > 0:
               stack.pop()
               k--

           // Remove leading zeros
           result = ""
           While !stack.isEmpty():
               result = stack.pop() + result

           // Remove leading zeros
           i = 0
           While i < result.length() && result.charAt(i) == '0':
               i++

           result = result.substring(i)
           Return result.isEmpty() ? "0" : result
```

Java Implementation

```
class Solution {
          public String removeKdigits(String num, int k) {
              Stack<Character> stack = new Stack<>();
```

```
            for (char digit : num.toCharArray()) {
                while (!stack.isEmpty() && k > 0 && stack.peek() > digit) {
                    stack.pop();
                    k--;
                }
                stack.push(digit);
            }

            while (k > 0 && !stack.isEmpty()) {
                stack.pop();
                k--;
            }

            StringBuilder result = new StringBuilder();
            while (!stack.isEmpty()) {
                result.append(stack.pop());
            }
            result.reverse();

            // Remove leading zeros
            int i = 0;
            while (i < result.length() && result.charAt(i) == '0') {
                i++;
            }
            result.delete(0, i);

            return result.length() == 0 ? "0" : result.toString();
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

192. Largest Rectangle in Histogram

Understanding the Problem NEW

Given a histogram (bar heights), find the area of the largest rectangle that can be formed using consecutive bars (the rectangle's height is limited by the shortest bar it spans).

Brute Force Approach NEW

For every pair of left and right boundaries, find the minimum height in that range and compute the area — O(n²) or O(n³) depending on how the minimum is found each time.

Why We Optimize NEW

For each bar, the largest rectangle *using that bar's height* extends as far left and right as possible while staying ≥ that bar's height — which is exactly 'how far until the nearest smaller bar on each side,' the same question a monotonic stack answers in O(n) (same core idea as problem 188's 'how far does this element extend as the minimum'). Compute the nearest-smaller-to-the-left and nearest-smaller-to-the-right for every bar using two monotonic stack passes, then the area for each bar is `height[i] × (right_boundary - left_boundary - 1)`, taking the maximum over all bars — O(n) total instead of O(n²).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the largest rectangle area in histogram.

Visual Diagram

```
Histogram: [2, 1, 5, 6, 2, 3]
       Largest rectangle = 10

       Visual:
            ■
          ■ ■
        ■ ■ ■
        ■ ■ ■   ■
      ■ ■ ■ ■ ■ ■
      2 1 5 6 2 3
```

```
Rectangle heights:
2: area = 2*1=2
1: area = 1*6=6
5: area = 5*2=10 (using height 5, width 2)
6: area = 6*1=6
2: area = 2*3=6
3: area = 3*1=3

Using Monotonic Stack:
For each bar, find previous smaller and next smaller
i=0 (2): prev=-1, next=1 → area=2*(1-(-1)-1)=2
i=1 (1): prev=-1, next=6 → area=1*6=6
i=2 (5): prev=1, next=3 → area=5*(3-1-1)=5
i=3 (6): prev=2, next=4 → area=6*(4-2-1)=6
i=4 (2): prev=1, next=6 → area=2*(6-1-1)=8
i=5 (3): prev=4, next=6 → area=3*(6-4-1)=3
Max = 10
```

Solution Approach

Concept: Monotonic stack for previous/next smaller.

Algorithm

```
Algorithm LARGEST_RECTANGLE(heights):
           stack = Stack()
           maxArea = 0
           n = heights.length

           For i = 0 to n:
               While !stack.isEmpty() && (i == n || heights[stack.peek()] >
heights[i]):
                   height = heights[stack.pop()]
                   width = stack.isEmpty() ? i : i - stack.peek() - 1
                   maxArea = Math.max(maxArea, height * width)
               stack.push(i)

           Return maxArea
```

Java Implementation

```
class Solution {
           public int largestRectangleArea(int[] heights) {
               Stack<Integer> stack = new Stack<>();
               int maxArea = 0;

              int n = heights.length;

              for (int i = 0; i <= n; i++) {
```

```
                int currentHeight = (i == n) ? 0 : heights[i];

                while (!stack.isEmpty() && heights[stack.peek()] > currentHeight)
{
                    int height = heights[stack.pop()];
                    int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                    maxArea = Math.max(maxArea, height * width);
                }
                stack.push(i);
            }
            return maxArea;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

193. Maximal Rectangles

Understanding the Problem NEW

Given a binary matrix (containing only 0s and 1s), find the area of the largest rectangle made entirely of 1s.

Brute Force Approach NEW

Check every possible rectangle (every pair of top-left and bottom-right corners) and verify all cells inside are 1 — extremely expensive, O((rows×cols)²) or worse.

Why We Optimize NEW

This reduces directly to the Largest Rectangle in Histogram problem (192), applied **row by row**: treat each row as the 'ground level' of a histogram, where the height of each column at that row is how many consecutive 1s are stacked directly above it (including the current row) — updating these running heights incrementally as you move down each row (extending the count if the cell is 1, resetting to 0 if it's 0). For each row's resulting histogram, run the O(cols) Largest Rectangle in Histogram algorithm, and track the maximum area seen across all rows — turning a 2D rectangle-search problem into rows × (an O(cols) 1D problem you've already solved), giving O(rows × cols) total.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find largest rectangle of 1s in binary matrix.

Visual Diagram

```
Matrix:
       1 0 1 0 0
       1 0 1 1 1
       1 1 1 1 1
       1 0 0 1 0

       Largest rectangle: area 6 (rows 1-2, cols 2-4)

       Step-by-step:
       Convert each row to histogram heights:
```

```
Row 0: [1,0,1,0,0] → histogram [1,0,1,0,0]
Row 1: [1,0,1,1,1] → [2,0,2,1,1]
Row 2: [1,1,1,1,1] → [3,1,3,2,2]
Row 3: [1,0,0,1,0] → [4,0,0,3,0]

For each row, find largest rectangle in histogram:
Row 0: max = 1
Row 1: max = 2 (height 2, width 1)
Row 2: max = 6 (height 2, width 3)
Row 3: max = 4
Result = 6
```

Solution Approach

Concept: Convert to histogram and use largest rectangle.

Algorithm

```
Algorithm MAXIMAL_RECTANGLE(matrix):
           If matrix empty: Return 0
           m = rows, n = cols
           heights = new int[n]
           maxArea = 0

            For i = 0 to m-1:
                For j = 0 to n-1:
                    If matrix[i][j] == '1':
                        heights[j]++
                    Else:
                        heights[j] = 0
                maxArea = Math.max(maxArea, largestRectangleArea(heights))

            Return maxArea
```

Java Implementation

```
class Solution {
           public int maximalRectangle(char[][] matrix) {
               if (matrix.length == 0) return 0;

                 int rows = matrix.length, cols = matrix[0].length;
                 int[] heights = new int[cols];
                 int maxArea = 0;
                 for (int i = 0; i < rows; i++) {

                  for (int j = 0; j < cols; j++) {
                      if (matrix[i][j] == '1') {
                          heights[j]++;
                      } else {
                          heights[j] = 0;
```

```
                    }
                }
                maxArea = Math.max(maxArea, largestRectangleArea(heights));
            }
            return maxArea;
        }

        private int largestRectangleArea(int[] heights) {
            Stack<Integer> stack = new Stack<>();
            int maxArea = 0;
            int n = heights.length;

            for (int i = 0; i <= n; i++) {
                int currentHeight = (i == n) ? 0 : heights[i];

                while (!stack.isEmpty() && heights[stack.peek()] > currentHeight)
{
                    int height = heights[stack.pop()];
                    int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                    maxArea = Math.max(maxArea, height * width);
                }
                stack.push(i);
            }
            return maxArea;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(n)

194. Sliding Window Maximum

Understanding the Problem NEW

Given an array and a window size K, find the maximum value within every sliding window of size K as it moves across the array from left to right.

Brute Force Approach NEW

For every window position, scan all K elements within it to find the max — O(n×K) total, recomputing the max from scratch for every single window even though consecutive windows overlap heavily.

Why We Optimize NEW

Use a **deque (double-ended queue)** that maintains a decreasing sequence of candidate indices: before adding a new index, remove from the back any indices whose values are smaller than the new one (since they can never be the max while the new, larger element is still in the window); also remove from the front any index that has fallen outside the current window's range. The front of the deque always holds the index of the current window's maximum. Because each index is added and removed from the deque at most once over the whole array, this achieves O(n) total, instead of O(n×K) — reusing the 'maintain a monotonic sequence, discard now-useless candidates' idea from the monotonic stack problems, but applied to a sliding window with a deque instead of a stack.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find maximum in each sliding window of size k.

Visual Diagram

```
Array: [1,3,-1,-3,5,3,6,7], k=3
       Output: [3,3,5,5,6,7]

       Step-by-step:
       Window 1: [1,3,-1] → max=3
       Window 2: [3,-1,-3] → max=3
       Window 3: [-1,-3,5] → max=5
       Window 4: [-3,5,3] → max=5
```

```
Window 5: [5,3,6] → max=6
Window 6: [3,6,7] → max=7

Using Deque:
i=0: deque=[0], arr[0]=1
i=1: deque=[1], arr[1]=3
i=2: deque=[1,2], arr[1]=3, arr[2]=-1 → max=3
i=3: remove 1, deque=[2,3], arr[3]=-3 → max=3
i=4: deque=[4], arr[4]=5 → max=5
```

Solution Approach

Concept: Deque maintains decreasing order.

Algorithm

```
Algorithm SLIDING_WINDOW_MAX(arr, k):
           deque = Deque()
           result = []

           For i = 0 to n-1:
               // Remove out of window
               While !deque.isEmpty() && deque.peekFirst() <= i-k:
                   deque.pollFirst()

                 // Remove smaller elements
                 While !deque.isEmpty() && arr[deque.peekLast()] <= arr[i]:
                     deque.pollLast()

                 deque.offerLast(i)

                 If i >= k-1:
                     result.add(arr[deque.peekFirst()])

           Return result
```

Java Implementation

```
class Solution {
           public int[] maxSlidingWindow(int[] nums, int k) {
               int n = nums.length;
               int[] result = new int[n - k + 1];
               Deque<Integer> deque = new ArrayDeque<>();
               for (int i = 0; i < n; i++) {

                  // Remove elements outside window
                  while (!deque.isEmpty() && deque.peekFirst() <= i - k) {
                      deque.pollFirst();
                  }
```

```
                // Remove smaller elements
                while (!deque.isEmpty() && nums[deque.peekLast()] <= nums[i]) {
                    deque.pollLast();
                }

                deque.offerLast(i);

                if (i >= k - 1) {
                    result[i - k + 1] = nums[deque.peekFirst()];
                }
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(k)

195. Stock Span Problem

Understanding the Problem NEW

Given a sequence of daily stock prices, compute the 'span' for each day — the number of consecutive days (including today) going backward where the price was less than or equal to today's price.

Brute Force Approach NEW

For each day, scan backward day by day until you find a higher price, counting how far back you went — O(n²) in the worst case (e.g. steadily increasing prices).

Why We Optimize NEW

This is Next Greater Element (problem 183) applied backward: maintain a monotonic stack of (price, span-so-far) pairs; for each new day, pop off any days from the stack whose price is ≤ today's price, **accumulating their spans** into today's span (since if a previous day's price was smaller, everything that day's span already covered is also covered by today going back further), then push today's price and computed span. Because each day is pushed and popped at most once, this achieves O(n) instead of O(n²) — a direct, if reframed, application of the monotonic stack idea used throughout this chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate stock span (consecutive days with price <= today).

Visual Diagram

```
Prices: [100, 80, 60, 70, 60, 75, 85]
       Span:   [1,   1, 1, 2, 1, 4, 6]

       Step-by-step:
       100 → span=1
       80 → span=1
       60 → span=1
       70 → 70>60, span=2
       60 → span=1
       75 → 75>60, >70, >80? No, span=4 (60,70,60)
       85 → 85>75, >80, >100? No, span=6
```

```
Using Stack:
i=0: stack=[0], span[0]=1
i=1: 80<100 → span=1, stack=[0,1]
i=2: 60<80 → span=1, stack=[0,1,2]
i=3: 70>60 → pop 2, stack=[0,1]; 70<80 → span=3-1=2
i=4: 60<70 → span=1
i=5: 75>60 → pop 4, >70 → pop 3, <80 → span=5-1=4
i=6: 85>75 → pop 5, >80 → pop 1, <100 → span=6-0=6
```

Solution Approach

Concept: Stack for previous greater element.

Algorithm

```
Algorithm STOCK_SPAN(prices):
           stack = Stack()
           span = new int[n]

           For i = 0 to n-1:
               While !stack.isEmpty() && prices[stack.peek()] <= prices[i]:
                   stack.pop()
               span[i] = stack.isEmpty() ? i+1 : i - stack.peek()
               stack.push(i)

           Return span
```

Java Implementation

```
class Solution {
           public int[] stockSpan(int[] prices) {
               int n = prices.length;
               int[] span = new int[n];
               Stack<Integer> stack = new Stack<>();

                 for (int i = 0; i < n; i++) {
                     while (!stack.isEmpty() && prices[stack.peek()] <= prices[i])
{
                         stack.pop();
                     }
                     span[i] = stack.isEmpty() ? i + 1 : i - stack.peek();
                     stack.push(i);
                 }
                 return span;

           }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

196. The Celebrity Problem

Understanding the Problem NEW

Among a group of N people, a 'celebrity' is someone who is known by everyone else but knows no one else themselves. Given a way to check 'does person A know person B' for any pair, find the celebrity (if one exists) using as few such checks as possible.

Brute Force Approach NEW

Check every pair of people to build a complete 'who knows who' picture, then look for the one person matching the celebrity definition — $O(n^2)$ checks, which is correct but doesn't exploit any structure in the celebrity property itself.

Why We Optimize NEW

The key insight: if person A knows person B, then A definitely *isn't* the celebrity (celebrities know no one) — so you can immediately discard A as a candidate. Using two pointers scanning through candidates and repeatedly eliminating whichever one 'knows' the other, you can narrow down to a **single remaining candidate** in O(n) checks (each check eliminates exactly one candidate), and then do one final O(n) verification pass to confirm that sole remaining candidate is actually known by everyone and knows no one. This 'elimination' strategy needs far fewer checks than examining every pair, dropping from $O(n^2)$ to O(n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the celebrity in a group of people.

Visual Diagram

```
Matrix where knows[a][b] = true if a knows b
       [0,0,1,0]
       [0,0,1,0]
       [0,0,0,0]
       [0,0,1,0]

       Celebrity is person 2:
       - Everyone knows person 2 (column 2 all true)
```

```
       - Person 2 knows no one (row 2 all false)

       Two-pointer approach:
       candidate = 0
       For i = 1 to n-1:
           If knows[candidate][i] == true:
               candidate = i

       Verify candidate:
       For i = 0 to n-1:
           If i != candidate:
               If knows[candidate][i] == true: return -1
               If knows[i][candidate] == false: return -1
       Return candidate
```

Solution Approach

Concept: Elimination method with verification.

Algorithm

```
Algorithm CELEBRITY(matrix):
           n = matrix.length
           candidate = 0

            For i = 1 to n-1:
                If matrix[candidate][i]:
                    candidate = i

            For i = 0 to n-1:
                If i != candidate:
                    If matrix[candidate][i]: return -1
                    If !matrix[i][candidate]: return -1

            Return candidate
```

Java Implementation

```
class Solution {
           public int findCelebrity(int[][] matrix) {
               int n = matrix.length;
               int candidate = 0;

                 // Find candidate
                 for (int i = 1; i < n; i++) {
                     if (matrix[candidate][i]) {
                         candidate = i;

                    }
               }
```

```
            // Verify candidate
            for (int i = 0; i < n; i++) {
                if (i == candidate) continue;

                // Celebrity knows no one
                if (matrix[candidate][i]) {
                    return -1;
                }

                // Everyone knows celebrity
                if (!matrix[i][candidate]) {
                    return -1;
                }
            }
            return candidate;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(1)

197. LRU Cache

Understanding the Problem NEW

Design a cache with a fixed capacity that evicts the **Least Recently Used** item when it's full and a new item needs to be inserted — and supports both get and put in O(1) time.

Brute Force Approach NEW

A simple array or list-based cache could work, but finding and updating 'how recently used' each item is, or finding the least-recently-used item to evict, would require scanning the whole cache — O(n) per operation, failing the O(1) requirement.

Why We Optimize NEW

Combine a **hashmap** (for O(1) lookup of any key's location) with a **doubly linked list** (which naturally supports O(1) removal and reinsertion at any position, given a direct node reference, as established back in the LinkedList chapter). Keep the DLL ordered by recency, with the most-recently-used item near one end; on every access, move that item to the recency end in O(1) using the DLL's O(1) removal/reinsertion; when eviction is needed, remove the item at the opposite (least-recently-used) end. This combination gives true O(1) for both get and put, directly reusing the DLL's O(1) arbitrary-position deletion capability that was highlighted as a key DLL advantage back in the LinkedList chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Least Recently Used cache.

Visual Diagram

```
Capacity = 3

      put(1,1): cache=[1]
      put(2,2): cache=[2,1]
      put(3,3): cache=[3,2,1]
      get(2): returns 2, cache=[2,3,1]
      put(4,4): cache full → remove 1, cache=[4,2,3]
```

```
get(1): returns -1
get(3): returns 3, cache=[3,4,2]

Data Structure:
HashMap for O(1) access
Doubly LinkedList for order
Head ←→ Most Recent ←→ Least Recent ←→ Tail
```

Solution Approach

Concept: HashMap + Doubly LinkedList.

Java Implementation

```
class LRUCache {
          private class Node {
              int key, value;
              Node prev, next;
              Node(int key, int value) {
                  this.key = key;
                  this.value = value;
              }
          }

          private int capacity;
          private Map<Integer, Node> map;
          private Node head, tail;

          public LRUCache(int capacity) {
              this.capacity = capacity;
              map = new HashMap<>();
              head = new Node(0, 0);
              tail = new Node(0, 0);
              head.next = tail;
              tail.prev = head;
          }

          public int get(int key) {
              if (!map.containsKey(key)) return -1;
              Node node = map.get(key);
              removeNode(node);
              addToFront(node);
              return node.value;
          }

          public void put(int key, int value) {
              if (map.containsKey(key)) {
                  Node node = map.get(key);

                   node.value = value;
                   removeNode(node);
```

```
                addToFront(node);
            } else {
                if (map.size() == capacity) {
                     Node lru = tail.prev;
                     removeNode(lru);
                     map.remove(lru.key);
                }
                Node newNode = new Node(key, value);
                map.put(key, newNode);
                addToFront(newNode);
            }
        }

        private void removeNode(Node node) {
            node.prev.next = node.next;
            node.next.prev = node.prev;
        }

        private void addToFront(Node node) {
            node.next = head.next;
            node.prev = head;
            head.next.prev = node;
            head.next = node;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(capacity)

198. LFU Cache

Understanding the Problem NEW

Similar to LRU Cache, but now eviction is based on **frequency of use** instead of recency — the Least Frequently Used item is evicted when the cache is full (with recency as a tiebreaker among equally-frequent items).

Brute Force Approach NEW

Track a frequency counter per key in a hashmap, and on eviction, scan all keys to find the minimum frequency (with some tiebreak logic) — O(n) per eviction, failing the O(1) goal.

Why We Optimize NEW

Maintain a hashmap from key to (value, frequency), *and* a second hashmap from frequency to a doubly linked list of all keys currently at that frequency (ordered by recency, so ties within the same frequency are broken LRU-style) — plus a running `minFrequency` counter. On every access, a key's frequency increases by 1, so it moves from its current frequency's DLL to the next frequency's DLL (each an O(1) DLL operation); eviction removes the least-recently-used key from the DLL at `minFrequency`. This layered structure — hashmap of DLLs, bucketed by frequency — achieves true O(1) for both get and put, extending LRU Cache's hashmap+DLL idea with an extra layer to also account for frequency.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Least Frequently Used cache.

Visual Diagram

```
Capacity = 3

      put(1,1): freq=1 cache=[1]
      put(2,2): freq=1 cache=[1,2]
      get(1): freq=2 cache=[1,2]
      put(3,3): freq=1 cache=[3,1,2]
      get(2): freq=2 cache=[2,3,1]
      put(4,4): cache full → remove 3 (freq=1, LRU), cache=[4,2,3,1]
```

```
Data Structures:
HashMap: key → Node
HashMap: frequency → DoublyLinkedList
minFreq: track minimum frequency
```

Solution Approach

Concept: HashMap + Frequency Maps.

Java Implementation

```
class LFUCache {
          private class Node {
              int key, value, freq;
              Node prev, next;
              Node(int key, int value) {
                  this.key = key;
                  this.value = value;
                  this.freq = 1;
              }
          }

          private class DoublyLinkedList {
              Node head, tail;
              DoublyLinkedList() {
                  head = new Node(0, 0);
                  tail = new Node(0, 0);
                  head.next = tail;
                  tail.prev = head;
              }

               void addToFront(Node node) {
                   node.next = head.next;
                   node.prev = head;
                   head.next.prev = node;
                   head.next = node;
               }

               void removeNode(Node node) {
                   node.prev.next = node.next;
                   node.next.prev = node.prev;
               }

               Node removeLast() {
                   if (head.next == tail) return null;
                   Node last = tail.prev;
                   removeNode(last);

                   return last;
               }
```

```
        boolean isEmpty() {
            return head.next == tail;
        }
    }

    private int capacity, minFreq;
    private Map<Integer, Node> keyMap;
    private Map<Integer, DoublyLinkedList> freqMap;

    public LFUCache(int capacity) {
        this.capacity = capacity;
        minFreq = 0;
        keyMap = new HashMap<>();
        freqMap = new HashMap<>();
    }

    public int get(int key) {
        if (!keyMap.containsKey(key)) return -1;
        Node node = keyMap.get(key);
        updateFreq(node);
        return node.value;
    }

    public void put(int key, int value) {
        if (capacity == 0) return;

        if (keyMap.containsKey(key)) {
            Node node = keyMap.get(key);
            node.value = value;
            updateFreq(node);
            return;
        }

        if (keyMap.size() == capacity) {
            DoublyLinkedList list = freqMap.get(minFreq);
            Node lru = list.removeLast();
            keyMap.remove(lru.key);
        }

        Node newNode = new Node(key, value);
        keyMap.put(key, newNode);
        minFreq = 1;
        freqMap.computeIfAbsent(1, k -> new DoublyLinkedList())
               .addToFront(newNode);
    }

    private void updateFreq(Node node) {
        int freq = node.freq;
        DoublyLinkedList list = freqMap.get(freq);
        list.removeNode(node);

        if (freq == minFreq && list.isEmpty()) {
            minFreq++;
```

```
            }

            node.freq++;
            freqMap.computeIfAbsent(node.freq, k -> new DoublyLinkedList())
                    .addToFront(node);
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(capacity)

Sliding Window & Two Pointer — Complete Guide

199. Longest Substring Without Repeating Characters

Understanding the Problem NEW

Given a string, find the length of the longest substring that doesn't contain any repeated characters. E.g. `"abcabcbb"` → the longest such substring is `"abc"`, length 3.

Brute Force Approach NEW

Check every possible substring and verify it has no repeated characters — $O(n^2)$ or $O(n^3)$ depending on how the repeat-check is done per substring.

Why We Optimize NEW

Use a **sliding window** with a hashmap tracking the last seen index of each character: expand the window's right edge one character at a time; if the new character has already been seen *within the current window*, jump the window's left edge forward to just past that character's last occurrence (rather than shrinking one step at a time), keeping the window always free of duplicates. Because both the left and right edges only ever move forward, never backward, the whole scan is O(n) — this exact sliding-window shape (expand right, jump/shrink left when a constraint is violated) is the template for essentially every remaining problem in this chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the longest substring without repeating characters.

Visual Diagram

```
Input: "abcabcbb"
       Output: 3 ("abc" or "bca" or "cab")

       Sliding Window:
       i=0, j=0: map={}, max=0
       a: map={a:1}, max=1, j=1
       b: map={a:1,b:1}, max=2, j=2
       c: map={a:1,b:1,c:1}, max=3, j=3
```

```
       a: map contains a → move i to index of a+1 (1)
          map={b:1,c:1,a:1}, max=3, j=4
       b: map contains b → move i to index of b+1 (2)
          map={c:1,a:1,b:1}, max=3, j=5
       c: map contains c → move i to index of c+1 (3)
          map={a:1,b:1,c:1}, max=3, j=6
       b: map contains b → move i to index of b+1 (4)
          max=3
       Result: 3

       Input: "bbbbb"
       Output: 1

       i=0, j=0: b → max=1
       j=1: b → i moves to 1
       j=2: b → i moves to 2
       ...
       max=1
```

Solution Approach

Concept: Sliding window with hashmap tracking last occurrence.

Algorithm

```
Algorithm LONGEST_UNIQUE_SUBSTRING(s):
           map = new HashMap<Character, Integer>()
           left = 0, maxLen = 0

           For right = 0 to n-1:
               If map contains s[right]:
                   left = max(left, map.get(s[right]) + 1)
               map.put(s[right], right)
               maxLen = max(maxLen, right - left + 1)

           Return maxLen
```

Java Implementation

```
class Solution {
           public int lengthOfLongestSubstring(String s) {
               Map<Character, Integer> map = new HashMap<>();
               int left = 0, maxLen = 0;

               for (int right = 0; right < s.length(); right++) {
                   char c = s.charAt(right);
                   if (map.containsKey(c)) {
                       left = Math.max(left, map.get(c) + 1);
                   }
                   map.put(c, right);
```

```
                maxLen = Math.max(maxLen, right - left + 1);
            }
            return maxLen;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(min(n, 26))

200. Max Consecutive Ones III

Understanding the Problem NEW

Given a binary array, find the longest subarray of all 1s achievable if you're allowed to flip at most K zeros to 1s.

Brute Force Approach NEW

Try every possible subarray, count its zeros, and check if that count is ≤ K — $O(n^2)$.

Why We Optimize NEW

Sliding window: expand the window's right edge, tracking a running count of zeros inside it; whenever that zero count exceeds K, shrink from the left until it's back within the allowed K — tracking the maximum window size seen throughout. Same 'expand right, shrink left on violation' template as problem 199, just with a numeric zero-count constraint.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find max consecutive ones with at most k zeros flipped.

Visual Diagram

```
Input: [1,1,1,0,0,0,1,1,1,1,0], k=2
       Output: 6

       Sliding Window:
       Window [1,1,1,0,0] → zeros=2, len=5
       Window [1,1,1,0,0,0] → zeros=3 > k → shrink
       Window [1,1,0,0,0] → zeros=3 > k → shrink
       Window [1,0,0,0] → zeros=3 > k → shrink
       Window [0,0,0] → zeros=3 > k → shrink
       Window [0,0,0,1,1,1,1] → zeros=3 > k → shrink
       Window [0,0,1,1,1,1,0] → zeros=3 > k → shrink
       Window [0,1,1,1,1,0] → zeros=2, len=6
       Result: 6

       Sliding Window:
       left=0, zeros=0, maxLen=0
```

```
right=0: 1 → zeros=0, len=1
right=1: 1 → zeros=0, len=2
right=2: 1 → zeros=0, len=3
right=3: 0 → zeros=1, len=4
right=4: 0 → zeros=2, len=5
right=5: 0 → zeros=3 → shrink
left=1: zeros=2 (remove first 1)
len=5
right=6: 1 → zeros=2, len=6
...
maxLen=6
```

Solution Approach

Concept: Sliding window tracking zeros count.

Algorithm

```
Algorithm MAX_CONSECUTIVE_ONES(nums, k):
           left = 0, zeros = 0, maxLen = 0

           For right = 0 to n-1:
               If nums[right] == 0:
                   zeros++

                 While zeros > k:
                     If nums[left] == 0:
                         zeros--
                     left++

                 maxLen = max(maxLen, right - left + 1)

           Return maxLen
```

Java Implementation

```
class Solution {
           public int longestOnes(int[] nums, int k) {
               int left = 0, zeros = 0, maxLen = 0;

              for (int right = 0; right < nums.length; right++) {
                  if (nums[right] == 0) {
                      zeros++;
                  }

                  while (zeros > k) {
                      if (nums[left] == 0) {
                          zeros--;
                      }
                      left++;
```

```
                }

                maxLen = Math.max(maxLen, right - left + 1);
            }
            return maxLen;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

201. Fruit Into Baskets

Understanding the Problem NEW

Given an array representing fruit types on trees in a row, find the longest subarray (contiguous run of trees) that contains at most 2 distinct fruit types — a real-world framing of 'longest subarray with at most K distinct elements' where K=2.

Brute Force Approach NEW

Check every subarray and count its distinct fruit types — $O(n^2)$ or $O(n^3)$.

Why We Optimize NEW

Sliding window with a hashmap tracking counts of each fruit type currently in the window: expand right, and whenever the window contains more than 2 distinct types, shrink from the left (decrementing/removing counts) until back down to 2 or fewer types — tracking the maximum window length. Exactly the same template again, this time with 'number of distinct keys in the frequency map' as the constraint being checked.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find longest subarray with at most 2 distinct elements.

Visual Diagram

```
Input: [1,2,1,2,3,2,3,1,1]
       Output: 4

       Sliding Window:
       Window [1,2,1,2] → 2 types, len=4
       When 3 appears: [1,2,1,2,3] → 3 types → shrink from left
       After shrink: [2,3,2,3,1,1] → Wait, let's trace properly

       Left=0, right=0: {1:1} → max=1
       right=1: {1:1,2:1} → max=2
       right=2: {1:2,2:1} → max=3
       right=3: {1:2,2:2} → max=4
       right=4: {1:2,2:2,3:1} → 3 types → shrink
```

```
          left=1: {1:1,2:2,3:1} → still 3
          left=2: {1:0,2:2,3:1} → 2 types, len=3
       right=5: {2:2,3:2} → len=4
       right=6: {2:2,3:2,1:1} → 3 types → shrink
          left=3: {2:1,3:2,1:1} → still 3
          left=4: {2:0,3:2,1:1} → 2 types, len=3
       right=7: {3:2,1:2} → len=4
       right=8: {3:2,1:3} → len=5? Wait, window [3,1,1] length 3
       Actually max=4

       Result: 4
```

Solution Approach

Concept: Sliding window with HashMap for distinct elements.

Algorithm

```
Algorithm FRUIT_INTO_BASKETS(fruits):
           left = 0, maxLen = 0
           map = new HashMap()

           For right = 0 to n-1:
               map.put(fruits[right], map.getOrDefault(fruits[right], 0) + 1)

                 While map.size() > 2:
                     map.put(fruits[left], map.get(fruits[left]) - 1)
                     If map.get(fruits[left]) == 0:
                         map.remove(fruits[left])
                     left++

                 maxLen = Math.max(maxLen, right - left + 1)

           Return maxLen
```

Java Implementation

```
class Solution {
           public int totalFruit(int[] fruits) {
               Map<Integer, Integer> map = new HashMap<>();
               int left = 0, maxLen = 0;
               for (int right = 0; right < fruits.length; right++) {

                  map.put(fruits[right], map.getOrDefault(fruits[right], 0) + 1);

                  while (map.size() > 2) {
                      map.put(fruits[left], map.get(fruits[left]) - 1);
                      if (map.get(fruits[left]) == 0) {
                          map.remove(fruits[left]);
                      }
```

```
                    left++;
                }

                maxLen = Math.max(maxLen, right - left + 1);
            }
            return maxLen;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

202. Longest Repeating Character Replacement

Understanding the Problem NEW

Given a string, find the length of the longest substring you can make entirely one repeated character, if you're allowed to change at most K characters.

Brute Force Approach NEW

Check every substring, count its most frequent character, and verify (length - max frequency) ≤ K — O(n²) or worse.

Why We Optimize NEW

Sliding window with a character frequency map: expand right, tracking the count of the *most frequent* character seen so far in the current window; the window is valid as long as `(window length - most frequent count) ≤ K` (meaning you'd need to change at most K characters to make the window uniform); if this is violated, shrink from the left. A subtle but important optimization: you don't need to recompute the true most-frequent-count exactly every time you shrink — you can allow it to be a slightly stale (but never overestimated) value and still get the correct final answer, since the window only grows when a strictly better max frequency is found. Same expand-right/shrink-left template, adapted with this frequency-based constraint.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Replace at most k characters to make longest repeating substring.

Visual Diagram

```
Input: "ABAB", k=2
       Output: 4 ("ABAB" → "AAAA")

       Window approach:
       right=0: 'A' → maxFreq=1, len=1, changes=0
       right=1: 'B' → maxFreq=1, len=2, changes=1
       right=2: 'A' → maxFreq=2, len=3, changes=1
       right=3: 'B' → maxFreq=2, len=4, changes=2
       maxLen=4
```

```
Input: "AABABBA", k=1
Output: 4

Window:
right=0: A → maxFreq=1, len=1
right=1: A → maxFreq=2, len=2
right=2: B → maxFreq=2, len=3, changes=1
right=3: A → maxFreq=3, len=4, changes=1
right=4: B → maxFreq=3, len=5, changes=2 > k → shrink
left=1: remove A → maxFreq still 3? No, recalc maxFreq
Actually maxFreq should be maintained differently

Better approach: maxFreq is max frequency in window
```

Solution Approach

Concept: Sliding window with character frequency.

Algorithm

```
Algorithm CHARACTER_REPLACEMENT(s, k):
           left = 0, maxFreq = 0, maxLen = 0
           freq = new int[26]

           For right = 0 to n-1:
               freq[s[right]-'A']++
               maxFreq = max(maxFreq, freq[s[right]-'A'])

                 // If window length - maxFreq > k, shrink
                 While (right - left + 1) - maxFreq > k:
                     freq[s[left]-'A']--
                     left++
                     // Recalculate maxFreq
                     maxFreq = getMaxFreq(freq)

                 maxLen = max(maxLen, right - left + 1)

           Return maxLen
```

Java Implementation

```
class Solution {
           public int characterReplacement(String s, int k) {
               int[] freq = new int[26];
               int left = 0, maxFreq = 0, maxLen = 0;

              for (int right = 0; right < s.length(); right++) {
                  freq[s.charAt(right) - 'A']++;
                  maxFreq = Math.max(maxFreq, freq[s.charAt(right) - 'A']);
```

```
                int windowLen = right - left + 1;
                if (windowLen - maxFreq > k) {
                    freq[s.charAt(left) - 'A']--;
                    left++;
                    // Recalculate maxFreq
                    maxFreq = getMaxFreq(freq);
                }

                maxLen = Math.max(maxLen, right - left + 1);
            }
            return maxLen;
        }

        private int getMaxFreq(int[] freq) {
            int max = 0;
            for (int f : freq) {
                max = Math.max(max, f);
            }
            return max;
        }
    }
```

Time & Space Complexity

Time Complexity: O(26*n) = O(n) | Space Complexity: O(1)

203. Binary Subarray with Sum

Understanding the Problem NEW

Given a binary array and a target sum, count how many subarrays sum to exactly that target.

Brute Force Approach NEW

Check every subarray's sum directly — O(n²).

Why We Optimize NEW

Since all elements are 0 or 1, you could use the prefix-sum-plus-hashmap technique from the Arrays chapter (problem 41) directly — but there's also a neat sliding-window alternative specific to non-negative integers: compute `atMost(target) - atMost(target-1)`, where `atMost(k)` counts subarrays with sum ≤ k using a simple sliding window (since sums only grow as the window expands, thanks to non-negative values). This 'exactly K = atMost(K) - atMost(K-1)' trick is a reusable pattern for turning an 'at most' sliding-window count into an 'exactly' count, and reappears in problems 204 and 208.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count subarrays with sum equal to target in binary array.

Visual Diagram

```
Input: [1,0,1,0,1], target=2
       Output: 4

       Subarrays with sum 2:
       [1,0,1] (0-2)
       [1,0,1,0] (0-3)
       [0,1,0,1] (1-4)
       [1,0,1] (2-4)
       Total: 4

       Two methods:
       1. Sliding Window (for non-negative)
       2. Prefix Sum with HashMap
```

```
Method 1 (Sliding Window):
For each left, find right for target and target-1
Count = atMost(target) - atMost(target-1)

Method 2 (Prefix Sum):
prefix = 0, map = {0:1}
For each num:
    prefix += num
    count += map.getOrDefault(prefix - target, 0)
    map.put(prefix, map.getOrDefault(prefix, 0) + 1)
```

Solution Approach

Concept: Prefix sum or sliding window.

Algorithm (Prefix Sum)

Algorithm BINARY_SUBARRAY_SUM(nums, target):

map = {0: 1}

prefix = 0, count = 0

For num in nums:

prefix += num

count += map.getOrDefault(prefix - target, 0)

map.put(prefix, map.getOrDefault(prefix, 0) + 1)

Return count

Java Implementation (Prefix Sum)

class Solution {

public int numSubarraysWithSum(int[] nums, int goal) {

Map<Integer, Integer> map = new HashMap<>();

map.put(0, 1);

int prefix = 0, count = 0;

for (int num : nums) {

prefix += num;

count += map.getOrDefault(prefix - goal, 0);

```
map.put(prefix, map.getOrDefault(prefix, 0) + 1);
}
return count;
}
}
```

Java Implementation (Sliding Window)

```
class Solution {
public int numSubarraysWithSum(int[] nums, int goal) {
return atMost(nums, goal) - atMost(nums, goal - 1);
}
private int atMost(int[] nums, int goal) {
if (goal < 0) return 0;
int left = 0, sum = 0, count = 0;
for (int right = 0; right < nums.length; right++) {
sum += nums[right];
while (sum > goal) {
sum -= nums[left];
left++;
}
count += right - left + 1;
}
return count;
}
}
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n) or O(1)

204. Count Number of Nice Subarrays

Understanding the Problem NEW

Count subarrays containing exactly K odd numbers.

Brute Force Approach NEW

Check every subarray and count its odd numbers directly — $O(n^2)$.

Why We Optimize NEW

Transform the problem: replace every even number with 0 and every odd number with 1, turning this into exactly the 'Binary Subarray with Sum' problem (203) with target K — and reuse the same `atMost(K) - atMost(K-1)` sliding-window trick. Recognizing that this 'different-looking' problem is actually a disguised version of one you already solved (by a simple even/odd → 0/1 relabeling) is the real insight here.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count subarrays with exactly k odd numbers.

Visual Diagram

```
Input: [1,1,2,1,1], k=3
       Output: 2

       Subarrays with exactly 3 odd numbers:
       [1,1,2,1] (0-3) → 3 odds
       [1,2,1,1] (1-4) → 3 odds
       Total: 2

       Method: Same as binary subarray with sum
       Treat odd as 1, even as 0
       Find subarrays with sum = k

       Using sliding window:
       count = atMost(k) - atMost(k-1)
```

Solution Approach

Concept: Convert to binary array.

Algorithm

```
Algorithm NICE_SUBARRAYS(nums, k):
           For i = 0 to n-1:
               nums[i] = nums[i] % 2

           Return atMost(nums, k) - atMost(nums, k-1)
```

Java Implementation

```
class Solution {
           public int numberOfSubarrays(int[] nums, int k) {
               for (int i = 0; i < nums.length; i++) {
                   nums[i] = nums[i] % 2;
               }
               return atMost(nums, k) - atMost(nums, k - 1);
           }

           private int atMost(int[] nums, int k) {
               if (k < 0) return 0;
               int left = 0, sum = 0, count = 0;

                 for (int right = 0; right < nums.length; right++) {
                     sum += nums[right];
                     while (sum > k) {
                         sum -= nums[left];
                         left++;
                     }
                     count += right - left + 1;
                 }
                 return count;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

205. Number of Substrings Containing All Three Characters

Understanding the Problem NEW

Given a string of only 'a', 'b', and 'c' characters, count the number of substrings that contain *at least one* of each of the three characters.

Brute Force Approach NEW

Check every substring and verify it contains all three characters — $O(n^2)$ or $O(n^3)$.

Why We Optimize NEW

Sliding window tracking the last-seen index of 'a', 'b', and 'c': for each right edge position, once the window contains all three characters, *every* substring starting anywhere from the current left edge up to the earliest of the three last-seen positions, and ending at the current right edge, must also contain all three — so you can add `(1 + min(last-seen positions) - left)` to the count in O(1) per right-edge step, rather than checking each qualifying substring individually. This is a slightly different flavor of sliding window — counting all valid substrings ending at each position in bulk, rather than tracking a single best window length — but built on the same expand-right-and-reason-about-what's-now-valid idea.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count substrings containing 'a', 'b', and 'c'.

Visual Diagram

```
Input: "abcabc"
        Output: 10

        Substrings containing a,b,c:
        All substrings with length >=3:
        "abc" (0-2)
        "abca" (0-3)
        "abcab" (0-4)
        "abcabc" (0-5)
        "bca" (1-3)
```

```
"bcab" (1-4)
"bcabc" (1-5)
"cab" (2-4)
"cabc" (2-5)
"abc" (3-5)
Total: 10

Sliding Window:
left=0, right=0: "a" → no
right=1: "ab" → no
right=2: "abc" → yes, count += n-right = 6-2=4
   left=1 → "bc" → no
right=3: "bca" → yes, count += 6-3=3
   left=2 → "ca" → no
right=4: "cab" → yes, count += 6-4=2
   left=3 → "ab" → no
right=5: "abc" → yes, count += 6-5=1
Total: 4+3+2+1=10
```

Solution Approach

Concept: Sliding window with frequency array.

Algorithm

```
Algorithm SUBSTRINGS_ALL_THREE(s):
            left = 0, count = 0
            freq = new int[3]

            For right = 0 to n-1:
                freq[s[right]-'a']++

                 While freq[0] > 0 && freq[1] > 0 && freq[2] > 0:
                     count += n - right
                     freq[s[left]-'a']--
                     left++

            Return count
```

Java Implementation

```
class Solution {
            public int numberOfSubstrings(String s) {
                int[] freq = new int[3];
                int left = 0, count = 0;

              int n = s.length();

              for (int right = 0; right < n; right++) {
                  freq[s.charAt(right) - 'a']++;
```

```
                while (freq[0] > 0 && freq[1] > 0 && freq[2] > 0) {
                    count += n - right;
                    freq[s.charAt(left) - 'a']--;
                    left++;
                }
            }
            return count;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

206. Maximum Points You Can Obtain from Cards

Understanding the Problem NEW

Given a row of cards each with a point value, and you must pick exactly K cards total, taking each pick from either the very front or the very back of the row — maximize total points.

Brute Force Approach NEW

Try every possible way of splitting K picks between 'taken from front' and 'taken from back' by simulating each split explicitly and summing — this is actually feasible in O(K) directly, but a truly naive recursive/exhaustive exploration of every pick order would be exponential.

Why We Optimize NEW

Reframe the problem: picking K cards from the two ends is equivalent to **leaving behind** a single contiguous subarray of length `n-K` from the middle (whatever you don't pick must be one unbroken chunk in the middle, since all picks come from the two ends). So the problem becomes: find the **minimum sum** contiguous subarray of length exactly `n-K`, and subtract it from the total sum of all cards — solvable with a simple fixed-size sliding window in O(n), turning an 'exploration of front/back split combinations' framing into a much simpler 'minimum fixed-window sum' framing.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum points by taking cards from either end.

Visual Diagram

```
Input: [1,2,3,4,5,6,1], k=3
       Output: 12

       Take from ends:
       Left: 1+2+3=6
       Right: 1+6+5=12
       Left: 1+6+1=8
       Right: 6+5+4=15? Wait k=3, can only take 3
```

```
Maximum: 12 (take 1,6,5 from right)

Sliding Window:
Take all k from left: [1,2,3] sum=6
Then slide:
Remove left, add right: [2,3,4? no, rightmost 1]
Actually: total = sum of all elements - sum of middle window of size n-k

n=7, k=3, window size = 4
Find minimum sum of 4 consecutive elements
Total sum = 22
Min window sum = 2+3+4+5=14
Max points = 22-14=8? Wait that's wrong

Actually:
k=3, can take from ends
Window of size n-k = 4 will be the remaining cards
We want min sum of 4 consecutive cards
Total sum = 22, min window sum = 2+3+4+5=14
Max points = 22-14=8

But expected is 12... Let's try:
Take all 3 from right: [1,6,5] = 12

The sliding window approach:
Take k from left: [1,2,3] = 6
Then slide window by removing from left and adding from right:
[1,2,3] → remove 1, add 1 → [2,3,1] = 6
[2,3,1] → remove 2, add 6 → [3,1,6] = 10
[3,1,6] → remove 3, add 5 → [1,6,5] = 12
Max = 12
```

Solution Approach

Concept: Sliding window on circular array.

Algorithm

```
Algorithm MAX_POINTS(cardPoints, k):
          n = cardPoints.length
          total = sum(cardPoints)
          windowSize = n - k
          minSum = sum of first windowSize elements
          currentSum = minSum

          For i = windowSize to n-1:

             currentSum = currentSum - cardPoints[i-windowSize] + cardPoints[i]
             minSum = min(minSum, currentSum)
```

```
          Return total - minSum
```

Java Implementation

```
class Solution {
          public int maxScore(int[] cardPoints, int k) {
              int n = cardPoints.length;
              int total = 0;
              for (int points : cardPoints) {
                  total += points;
              }

              int windowSize = n - k;
              int currentSum = 0;
              for (int i = 0; i < windowSize; i++) {
                  currentSum += cardPoints[i];
              }

              int minSum = currentSum;
              for (int i = windowSize; i < n; i++) {
                  currentSum = currentSum - cardPoints[i - windowSize] +
cardPoints[i];
                  minSum = Math.min(minSum, currentSum);
              }

              return total - minSum;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

207. Longest Substring with At Most K Distinct Characters

Understanding the Problem NEW

Find the longest substring containing at most K distinct characters — the general form of the Fruit Into Baskets problem (201), which was this same problem with K fixed at 2.

Brute Force Approach NEW

Check every substring and count its distinct characters — $O(n^2)$ or $O(n^3)$.

Why We Optimize NEW

Exactly the same sliding window with a frequency hashmap as problem 201, just with a general K instead of hardcoded 2: expand right, and if the number of distinct characters (keys in the frequency map) exceeds K, shrink from the left until it's back within K, tracking the maximum window length. This problem exists specifically to make explicit that Fruit Into Baskets was really 'K=2' of this more general template all along.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find longest substring with at most k distinct characters.

Visual Diagram

```
Input: "eceba", k=2
       Output: 3 ("ece")

       Sliding Window:
       left=0, right=0: "e" → {e:1}, len=1
       right=1: "ec" → {e:1,c:1}, len=2
       right=2: "ece" → {e:2,c:1}, len=3
       right=3: "eceb" → {e:2,c:1,b:1} → 3 distinct > k
          left=1: "ceb" → {e:1,c:1,b:1} → still 3
          left=2: "eb" → {e:1,b:1} → 2 distinct, len=2
       right=4: "eba" → {e:1,b:1,a:1} → 3 distinct
          left=3: "ba" → {b:1,a:1} → len=2
       maxLen=3
```

```
Input: "aa", k=1
Output: 2
```

Solution Approach

Concept: Sliding window with frequency map.

Algorithm

```
Algorithm LONGEST_K_DISTINCT(s, k):
           left = 0, maxLen = 0
           map = new HashMap()

           For right = 0 to n-1:
               map.put(s[right], map.getOrDefault(s[right], 0) + 1)

                 While map.size() > k:
                     map.put(s[left], map.get(s[left]) - 1)
                     If map.get(s[left]) == 0:
                         map.remove(s[left])
                     left++

                 maxLen = max(maxLen, right - left + 1)

           Return maxLen
```

Java Implementation

```
class Solution {
           public int lengthOfLongestSubstringKDistinct(String s, int k) {
               if (k == 0 || s.isEmpty()) return 0;

                 Map<Character, Integer> map = new HashMap<>();
                 int left = 0, maxLen = 0;

                 for (int right = 0; right < s.length(); right++) {
                     map.put(s.charAt(right), map.getOrDefault(s.charAt(right), 0)
+ 1);
                     while (map.size() > k) {

                      char leftChar = s.charAt(left);
                      map.put(leftChar, map.get(leftChar) - 1);
                      if (map.get(leftChar) == 0) {
                          map.remove(leftChar);
                      }
                      left++;
                  }

                  maxLen = Math.max(maxLen, right - left + 1);
```

```
            }
            return maxLen;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(k)

208. Subarray with K Different Integers

Understanding the Problem NEW

Count subarrays with **exactly** K distinct integers — the 'exactly' version building on the 'at most K distinct' sliding window from problem 207.

Brute Force Approach NEW

Check every subarray and count its distinct values — $O(n^2)$ or $O(n^3)$.

Why We Optimize NEW

Reuse the same `atMost(K) - atMost(K-1)` transformation from problems 203/204: build an `atMost(K distinct)` helper using the sliding window from problem 207 (which counts subarrays with at most K distinct values, by adding `(right - left + 1)` new valid subarrays at each right-edge step), then the answer is `atMost(K) - atMost(K-1)`. This is now the third appearance of the 'exactly = atMost(K) - atMost(K-1)' pattern in this chapter, worth committing to memory as a general technique whenever a sliding window naturally handles 'at most' constraints more easily than 'exactly' ones.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count subarrays with exactly k distinct integers.

Visual Diagram

```
Input: [1,2,1,2,3], k=2
       Output: 7

       Subarrays with exactly 2 distinct:
       [1,2] (0-1)
       [1,2,1] (0-2)
       [1,2,1,2] (0-3)
       [2,1] (1-2)
       [2,1,2] (1-3)
       [1,2] (2-3)
       [2,3] (3-4)
       Total: 7
```

```
       Method: atMost(k) - atMost(k-1)
```

Solution Approach

Concept: Sliding window with atMost function.

Algorithm

```
Algorithm SUBARRAYS_K_DISTINCT(nums, k):
           Return atMost(nums, k) - atMost(nums, k-1)

       Algorithm atMost(nums, k):
           left = 0, count = 0
           map = new HashMap()

           For right = 0 to n-1:
               map.put(nums[right], map.getOrDefault(nums[right], 0) + 1)

                 While map.size() > k:
                     map.put(nums[left], map.get(nums[left]) - 1)
                     If map.get(nums[left]) == 0:
                         map.remove(nums[left])
                     left++

                 count += right - left + 1

           Return count
```

Java Implementation

```
class Solution {
           public int subarraysWithKDistinct(int[] nums, int k) {
               return atMost(nums, k) - atMost(nums, k - 1);
           }

           private int atMost(int[] nums, int k) {
               if (k < 0) return 0;

                 Map<Integer, Integer> map = new HashMap<>();
                 int left = 0, count = 0;

                 for (int right = 0; right < nums.length; right++) {

                  map.put(nums[right], map.getOrDefault(nums[right], 0) + 1);

                  while (map.size() > k) {
                      map.put(nums[left], map.get(nums[left]) - 1);
                      if (map.get(nums[left]) == 0) {
                          map.remove(nums[left]);
```

```
                    }
                    left++;
                }

                count += right - left + 1;
            }
            return count;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(k)

209. Minimum Window Substring

Understanding the Problem NEW

Given a string S and a target string T, find the smallest substring (window) of S that contains every character of T (including duplicates — if T has two 'a's, the window needs at least two 'a's too).

Brute Force Approach NEW

Check every substring of S and verify it contains all required characters (with correct counts) — $O(n^2)$ substrings, each requiring an O(n) or O(alphabet) check, giving $O(n^2)$ to $O(n^3)$ total.

Why We Optimize NEW

Sliding window with a frequency map of T's required characters and a counter of 'how many required character-slots are currently satisfied': expand the right edge, and whenever the window fully satisfies every requirement, try **shrinking from the left** as much as possible while still satisfying every requirement, recording the smallest valid window found along the way. Because both edges only move forward overall (even though the left edge does its own shrink loop), the total work across the whole scan is O(n + m) — a more intricate version of the same expand-right/shrink-left template, adapted to track multiple required-character counts simultaneously instead of a single numeric constraint.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum window containing all characters of target.

Visual Diagram

```
Input: s = "ADOBECODEBANC", t = "ABC"
       Output: "BANC"

       Step-by-step:
       Window [A D O B E C] (0-5) → contains A,B,C, len=6
       Window [D O B E C O D E B A N C] → wait, let's trace

       left=0, right=0: "A" → missing B,C
```

```
right=1: "AD" → missing B,C
right=2: "ADO" → missing B,C
right=3: "ADOB" → missing C
right=4: "ADOBE" → missing C
right=5: "ADOBEC" → found! len=6, min="ADOBEC"
Shrink left:
left=1: "DOBEC" → missing A
left=2: "OBEC" → missing A
...
right=6: "DOBECOD" → missing A
right=7: "DOBECODE" → missing A
right=8: "DOBECODEB" → missing A
right=9: "DOBECODEBA" → found! len=10
Shrink left...
When right=12: "BANC" → found! len=4
Result: "BANC"
```

Solution Approach

Concept: Sliding window with character count.

Algorithm

```
Algorithm MIN_WINDOW(s, t):
           If s.length() < t.length(): Return ""

           map = new HashMap()
           For char in t: map.put(char, map.getOrDefault(char,0)+1)

           left = 0, minLen = INF, minStart = 0
           count = t.length() // characters to match

           For right = 0 to n-1:
               char c = s[right]
               If map.containsKey(c):
                   If map.get(c) > 0: count--
                   map.put(c, map.get(c)-1)

                 While count == 0:
                     If right - left + 1 < minLen:
                         minLen = right - left + 1
                         minStart = left

                    char leftChar = s[left]
                    If map.containsKey(leftChar):
                        map.put(leftChar, map.get(leftChar)+1)
                        If map.get(leftChar) > 0: count++

                  left++
```

```
        Return minLen == INF ? "" : s.substring(minStart, minStart+minLen)
```

Java Implementation

```
class Solution {
          public String minWindow(String s, String t) {
              if (s.length() < t.length()) return "";

              Map<Character, Integer> map = new HashMap<>();
              for (char c : t.toCharArray()) {
                  map.put(c, map.getOrDefault(c, 0) + 1);
              }

              int left = 0, minLen = Integer.MAX_VALUE, minStart = 0;
              int count = t.length();

              for (int right = 0; right < s.length(); right++) {
                  char c = s.charAt(right);
                  if (map.containsKey(c)) {
                      if (map.get(c) > 0) count--;
                      map.put(c, map.get(c) - 1);
                  }

                  while (count == 0) {
                      int windowLen = right - left + 1;
                      if (windowLen < minLen) {
                          minLen = windowLen;
                          minStart = left;
                      }

                      char leftChar = s.charAt(left);
                      if (map.containsKey(leftChar)) {
                          map.put(leftChar, map.get(leftChar) + 1);
                          if (map.get(leftChar) > 0) count++;
                      }
                      left++;
                  }
              }

              return minLen == Integer.MAX_VALUE ? "" : s.substring(minStart,
minStart + minLen);
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(m)

210. Minimum Window Subsequence

Understanding the Problem NEW

A variant of Minimum Window Substring: find the minimum window in S that contains T as a **subsequence** (characters of T must appear in the same relative order within the window, but don't need to be contiguous — unlike problem 209, where only the character counts mattered, not their order).

Brute Force Approach NEW

Check every substring of S and verify T is a valid subsequence of it (an O(n) check per substring using two pointers) — $O(n^2)$ to $O(n^3)$ total depending on substring generation cost.

Why We Optimize NEW

Because *order* matters here (unlike problem 209's frequency-only requirement), a frequency-map sliding window isn't sufficient — instead, use a **two-pointer subsequence-matching scan**: walk forward through S matching characters of T in order until a full match is found (marking a candidate window's end), then walk *backward* from that point re-matching T in reverse to find the tightest possible start of that window, then continue scanning forward in S from just after the previous match's start to look for the next candidate window. This differs from the standard sliding window template specifically because subsequence matching (not just character-count satisfaction) requires tracking positional order, not just counts — a good example of recognizing when the general sliding-window template needs a genuinely different mechanism rather than just a different constraint check.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum window in s containing t as subsequence.

Visual Diagram

```
Input: s = "abcdebdde", t = "bde"
      Output: "bcde"

      Step-by-step:
```

```
Find t as subsequence:
s[1]='b', s[3]='d', s[4]='e' → "bcde" len=4
s[1]='b', s[3]='d', s[7]='e' → "bcdde" len=5
s[1]='b', s[6]='d', s[7]='e' → "bcde" len=4
s[5]='b', s[6]='d', s[7]='e' → "bde" len=3
Minimum: "bcde" (4)

Two-pointer approach:
For each possible start:
  Find end where subsequence ends
  Then find start of subsequence
  Update minimum

Using DP for subsequence matching:
dp[i][j] = start position of subsequence
```

Solution Approach

Concept: Two-pointer subsequence matching.

Algorithm

```
Algorithm MIN_WINDOW_SUBSEQUENCE(s, t):
          n = s.length(), m = t.length()
          minLen = INF, minStart = -1

          For i = 0 to n-1:
              // Find end where subsequence matches
              j = 0
              While i < n && j < m:
                  If s[i] == t[j]: j++
                  i++

                 If j == m:
                     // Found subsequence ending at i-1
                     end = i-1
                     // Find start
                     j = m-1
                     start = end
                     While j >= 0:
                         If s[start] == t[j]: j--
                         start--
                     start++

                    If end - start + 1 < minLen:
                        minLen = end - start + 1
                        minStart = start

                    i = start + 1 // Reset for next search
```

```
        Return minLen == INF ? "" : s.substring(minStart, minStart+minLen)
```

Java Implementation

```
class Solution {
           public String minWindow(String s, String t) {
               int n = s.length(), m = t.length();
               int minLen = Integer.MAX_VALUE;
               int minStart = -1;

               for (int i = 0; i < n; i++) {
                   // Find end where t is subsequence
                   int j = 0;
                   int temp = i;
                   while (temp < n && j < m) {
                       if (s.charAt(temp) == t.charAt(j)) {
                           j++;
                       }
                       temp++;
                   }

                   if (j == m) {
                       // Found subsequence, now find the start
                       int end = temp - 1;
                       j = m - 1;
                       int start = end;
                       while (j >= 0) {
                           if (s.charAt(start) == t.charAt(j)) {
                                j--;
                           }
                           start--;
                       }
                       start++;

                       if (end - start + 1 < minLen) {
                           minLen = end - start + 1;
                           minStart = start;
                       }

                       // Reset to search for next window
                       i = start;
                   }
               }

               return minStart == -1 ? "" : s.substring(minStart, minStart +
minLen);
           }
       }
```

Time & Space Complexity

Time Complexity: O(n^2) worst case | Space Complexity: O(1)

Stack & Queue Summary

Problem	Type	Time	Space	Key Technique
Stack Array	Basic	O(1)	O(n)	Array with top
Queue Array	Basic	O(1)	O(n)	Circular array
Stack using Queue	Basic	Push O(n), Pop O(1)	O(n)	Two queues
Queue using Stack	Basic	Push O(1), Pop O(n)	O(n)	Two stacks
Stack LL	Basic	O(1)	O(n)	Linked list
Queue LL	Basic	O(1)	O(n)	Linked list
Balanced Parentheses	Basic	O(n)	O(n)	Stack matching
Min Stack	Basic	O(1)	O(n)	Min tracking
Infix-Postfix	Conversion	O(n)	O(n)	Precedence stack
Prefix-Infix	Conversion	O(n)	O(n)	Right-left traversal
Next Greater	Monotonic	O(n)	O(n)	Decreasing stack
Next Greater II	Monotonic	O(n)	O(n)	Circular array
Next Smaller	Monotonic	O(n)	O(n)	Stack with >=
Trapping Rainwater	Monotonic	O(n)	O(1)	Two pointers
Sum Subarray Min	Monotonic	O(n)	O(n)	Contribution
Asteroid Collision	Monotonic	O(n)	O(n)	Stack with collisions
Sum Subarray Ranges	Monotonic	O(n)	O(n)	Max - Min
Remove K Digits	Monotonic	O(n)	O(n)	Increasing stack
Largest Rectangle	Monotonic	O(n)	O(n)	Histogram
Maximal Rectangle	Monotonic	O(m*n)	O(n)	Histogram DP
Sliding Window Max	Implementation	O(n)	O(k)	Deque
Stock Span	Implementation	O(n)	O(n)	Previous greater
Celebrity Problem	Implementation	$O(n^2)$	O(1)	Elimination
LRU Cache	Implementation	O(1)	O(capacity)	HashMap + DLL
LFU Cache	Implementation	O(1)	O(capacity)	HashMap + Freq

Sliding Window & Two Pointer Summary

Problem	Difficulty	Time	Space	Key Technique
Longest Unique Substring	Medium	O(n)	O(1)	Map for last occurrence
Max Consecutive Ones III	Medium	O(n)	O(1)	Track zeros count
Fruit Into Baskets	Medium	O(n)	O(1)	Map for distinct count
Character Replacement	Medium	O(n)	O(1)	Frequency + maxFreq
Binary Subarray Sum	Medium	O(n)	O(n)	Prefix sum
Nice Subarrays	Medium	O(n)	O(1)	Convert to binary
All Three Characters	Medium	O(n)	O(1)	Frequency array
Max Points from Cards	Medium	O(n)	O(1)	Circular sliding window
Longest K Distinct	Hard	O(n)	O(k)	Map + shrink
K Different Integers	Hard	O(n)	O(k)	atMost(k) - atMost(k-1)
Minimum Window Substring	Hard	O(n)	O(m)	Map + count tracking
Minimum Window Subsequence	Hard	$O(n^2)$	O(1)	Subsequence matching

— End of enhanced Chapter 8: Stack & Queue / Sliding Window — remaining chapters continue in the full edition —

Heaps — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

211. Introduction to Priority Queues using Heaps

Understanding the Problem NEW

A **Heap** is a special tree-shaped structure that always keeps either the smallest (Min-Heap) or largest (Max-Heap) element instantly accessible at the top/root — and a **Priority Queue** is the abstract 'queue where the highest-priority item comes out first' concept that a heap is commonly used to implement efficiently.

Brute Force Approach NEW

Without a heap, you could keep an unsorted list and scan it every time you need the min/max — O(n) per query — or keep a fully sorted list, which gives O(1) access to the min/max but costs O(n) to insert a new element in the right sorted position.

Why We Optimize NEW

A heap strikes a middle ground: it doesn't maintain *full* sorted order (which is expensive to maintain on every insert), but it does maintain enough partial order (every parent node is smaller/larger than its children) to guarantee the min or max is always right at the root. This gives you O(log n) insert and O(log n) removal of the min/max, and O(1) to just *look at* the current min/max — a genuinely useful middle ground between 'no order' (fast insert, slow query) and 'full order' (slow insert, fast query), and it's exactly this trade-off that makes heaps the tool of choice for every 'kth largest/smallest,' 'merge k things,' and 'always process the highest priority item next' problem in this chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding Priority Queue and Heap data structures.

Visual Diagram

```
Min Heap (Priority Queue - ascending):
                    1
                  /    \
                3       5
               / \     /
             4     8 6
```

```
          Max Heap (Priority Queue - descending):
                    8
                  /    \
                 6       5
                / \     /
               4     3 1
          Heap Operations:

       - Insert: O(log n)
       - Extract Min/Max: O(log n)
       - Peek: O(1)
       - Heapify: O(n)

       Array Representation (Min Heap):
       Index: 0 1 2 3 4 5
       Value: 1 3 5 4 8 6
       Children: 2i+1, 2i+2
       Parent: (i-1)/2
```

Java Implementation

```
// Using Java's PriorityQueue (Min Heap by default)
       class PriorityQueueExample {
           public void demonstrate() {
               // Min Heap
               PriorityQueue<Integer> minHeap = new PriorityQueue<>();
               minHeap.add(5);
               minHeap.add(3);
               minHeap.add(8);
               minHeap.add(1);
               // Peek: 1 (smallest)

               // Max Heap
               PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Collections.reverseOrder());
               maxHeap.add(5);
               maxHeap.add(3);
               maxHeap.add(8);
               maxHeap.add(1);
               // Peek: 8 (largest)
           }
       }
```

Time & Space Complexity

Time Complexity: O(log n) for insert/delete | Space Complexity: O(n)

212. Min Heap and Max Heap Implementation

Understanding the Problem NEW

Implement a Min-Heap (smallest element always at the root) and Max-Heap (largest always at the root) from scratch, typically backed by a plain array using index arithmetic to represent the tree structure (for node at index i, children are at 2i+1 and 2i+2, parent at (i-1)/2).

Brute Force Approach NEW

You could use an actual tree structure with explicit node objects and pointers, but this adds pointer overhead and complexity compared to representing the tree implicitly with array indices.

Why We Optimize NEW

Because a heap is always a **complete** binary tree (filled left to right, level by level, with no gaps), its shape is entirely predictable — which means you can represent it as a plain array without storing any explicit pointers at all, just using simple index arithmetic to know where any node's parent or children live. Insertion adds the new element at the end of the array and 'bubbles it up' (sift-up) by repeatedly swapping with its parent while the heap property is violated; removal takes the root, replaces it with the last element, and 'sifts it down' by repeatedly swapping with the smaller/larger child. Both operations are O(log n), since the tree's height is always O(log n) thanks to being complete — the array representation trades away explicit pointers for compact, cache-friendly index arithmetic.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Min Heap and Max Heap from scratch.

Visual Diagram

```
Min Heap Implementation:
       size = 0
       capacity = 10
       heap = []
```

```
       insert(5): [5]
       insert(3): [3,5] (swap 5 and 3)
       insert(8): [3,5,8]
       insert(1): [1,3,8,5] (swap up)

       extractMin():
         1 → remove
         [3,5,8] (heapify down)

       Visual Heap:
                  1
                /   \
              3      8
            /
           5
```

Solution Approach

Concept: Array-based heap with sift up/down.

Algorithm

```
Algorithm MIN_HEAP:
           heap = new int[capacity]
           size = 0

            insert(key):
                If size == capacity: return
                heap[size] = key
                size++
                siftUp(size-1)

            siftUp(index):
                While index > 0:
                    parent = (index-1)/2
                    If heap[parent] <= heap[index]: break
                    swap(parent, index)
                    index = parent

            extractMin():
                If size == 0: return -1
                min = heap[0]
                heap[0] = heap[size-1]
                size--
                siftDown(0)
                Return min

            siftDown(index):
                While true:
                    left = 2*index+1, right = 2*index+2
```

```
                    smallest = index
                    If left < size and heap[left] < heap[smallest]: smallest = left
                    If right < size and heap[right] < heap[smallest]: smallest =
right
                    If smallest == index: break
                    swap(index, smallest)
                    index = smallest
```

Java Implementation

```
class MinHeap {
          private int[] heap;
          private int size;
          private int capacity;

          MinHeap(int capacity) {
              this.capacity = capacity;
              heap = new int[capacity];
              size = 0;
          }

          public void insert(int key) {
              if (size == capacity) {
                  throw new RuntimeException("Heap is full");
              }
              heap[size] = key;
              size++;
              siftUp(size - 1);
          }

          private void siftUp(int index) {
              while (index > 0) {
                  int parent = (index - 1) / 2;
                  if (heap[parent] <= heap[index]) break;
                  swap(parent, index);
                  index = parent;
              }
          }

          public int extractMin() {
              if (size == 0) {
                  throw new RuntimeException("Heap is empty");
              }
              int min = heap[0];
              heap[0] = heap[size - 1];
              size--;
              siftDown(0);
              return min;
          }

          private void siftDown(int index) {
              while (true) {
```

```
                int left = 2 * index + 1;
                int right = 2 * index + 2;
                int smallest = index;

                if (left < size && heap[left] < heap[smallest]) {
                    smallest = left;
                }
                if (right < size && heap[right] < heap[smallest]) {
                    smallest = right;
                }
                if (smallest == index) break;
                swap(index, smallest);
                index = smallest;
            }
        }

        public int peek() {

             if (size == 0) {
                 throw new RuntimeException("Heap is empty");
             }
             return heap[0];
         }

         private void swap(int i, int j) {
             int temp = heap[i];
             heap[i] = heap[j];
             heap[j] = temp;
         }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(n)

213. Check if an Array Represents a Min-Heap

Understanding the Problem NEW

Given an array, check if it represents a valid Min-Heap (using the standard array-index-to-tree mapping) — meaning every parent is ≤ both of its children.

Brute Force Approach NEW

There isn't really a slower alternative here — checking the heap property requires examining each parent-child relationship at least once regardless.

Why We Optimize NEW

You only need to check the **internal (non-leaf) nodes** — roughly the first half of the array — since leaf nodes have no children to violate the property against. For each internal node at index i, verify it's ≤ its children at 2i+1 and 2i+2 (when those indices exist). This is an O(n) single pass, checking exactly the relationships that matter rather than doing any unnecessary extra work.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if array represents a valid Min Heap.

Visual Diagram

```
Array: [1, 3, 5, 4, 8, 6]
       Check parent <= children:
       1 <= 3 ✓, 1 <= 5 ✓
       3 <= 4 ✓, 3 <= 8 ✓
       5 <= 6 ✓
       Result: True

       Array: [1, 3, 2, 4, 8, 6]
       Check: 1 <= 3 ✓, 1 <= 2 ✓
       3 <= 4 ✓, 3 <= 8 ✓
       2 <= 6 ✓
       Result: True

       Array: [1, 5, 3, 4, 8, 6]
       Check: 1 <= 5 ✓, 1 <= 3 ✓
```

```
        5 <= 4 ✗ → Not Min Heap
        Result: False
```

Solution Approach

Concept: Check heap property for all internal nodes.

Algorithm

```
Algorithm IS_MIN_HEAP(arr):
            n = arr.length
            For i = 0 to (n/2)-1:
                left = 2*i + 1
                right = 2*i + 2
                If left < n and arr[i] > arr[left]:
                    Return false
                If right < n and arr[i] > arr[right]:
                    Return false
            Return true
```

Java Implementation

```
class Solution {
            public boolean isMinHeap(int[] arr) {
                int n = arr.length;
                for (int i = 0; i < n / 2; i++) {
                    int left = 2 * i + 1;
                    int right = 2 * i + 2;

                      if (left < n && arr[i] > arr[left]) {
                          return false;
                      }
                      if (right < n && arr[i] > arr[right]) {
                          return false;
                      }
                  }
                  return true;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

214. Convert Min Heap to Max Heap

Understanding the Problem NEW

Given an array that's currently a valid Min-Heap, rearrange it in-place so it becomes a valid Max-Heap instead.

Brute Force Approach NEW

You could extract all elements one by one (removing the min each time, which is O(log n) per removal) and re-insert them into a fresh Max-Heap — O(n log n) total, throwing away and rebuilding the whole structure.

Why We Optimize NEW

Rather than rebuilding from scratch, you can convert in-place using the classic **'build-heap' technique**: starting from the last internal (non-leaf) node and working backward to the root, run 'sift-down' (using the Max-Heap comparison rule) on each node. Because you process nodes bottom-up, each node's subtree is already a valid max-heap by the time you sift it, so a single pass suffices — this well-known technique runs in O(n) total (not O(n log n)), because most nodes are near the bottom of the tree where sift-down does very little work, and only a few nodes near the root need the full O(log n) sift — a much cheaper way to establish heap order than n individual O(log n) insertions.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert a Min Heap array to Max Heap.

Visual Diagram

```
Min Heap: [1, 3, 5, 4, 8, 6]
       Max Heap: [8, 5, 6, 4, 1, 3]

       Step 1: Start from last internal node
       Index: 2 (value 5)
       Check children: left=5? Actually heap[2]=5, left=6? Let's recalc

       Array: [1, 3, 5, 4, 8, 6]
       Index: 0 1 2 3 4 5
```

```
Build Max Heap:
i=2: 5 > max(6) ? 5<6 → swap with 6 → [1,3,6,4,8,5]
i=1: 3 > max(4,8) ? 3<8 → swap with 8 → [1,8,6,4,3,5]
i=0: 1 > max(8,6) ? 1<8 → swap with 8 → [8,1,6,4,3,5]
     1 > max(4,3) ? 1<4 → swap with 4 → [8,4,6,1,3,5]
     1 > max(3,5) ? 1<5 → swap with 5 → [8,4,6,5,3,1]

Result: [8, 4, 6, 5, 3, 1]? Wait, let's recalc properly

Original: [1, 3, 5, 4, 8, 6]
          1
        /   \
      3       5
     / \     /
   4     8 6

Max Heap after heapify:
          8
        /   \
      6       5
     / \     /
   4     3 1

Array: [8, 6, 5, 4, 3, 1]
```

Solution Approach

Concept: Heapify all internal nodes.

Algorithm

```
Algorithm CONVERT_TO_MAX_HEAP(arr):
           n = arr.length
           For i = n/2 - 1 down to 0:
               maxHeapify(arr, n, i)
           Return arr

       Algorithm maxHeapify(arr, n, i):
           largest = i
           left = 2*i + 1
           right = 2*i + 2

           If left < n and arr[left] > arr[largest]:
               largest = left
           If right < n and arr[right] > arr[largest]:

               largest = right
           If largest != i:
               swap(arr[i], arr[largest])
               maxHeapify(arr, n, largest)
```

Java Implementation

```
class Solution {
           public void convertMinToMaxHeap(int[] arr) {
               int n = arr.length;
               for (int i = n / 2 - 1; i >= 0; i--) {
                   maxHeapify(arr, n, i);
               }
           }

           private void maxHeapify(int[] arr, int n, int i) {
               int largest = i;
               int left = 2 * i + 1;
               int right = 2 * i + 2;

               if (left < n && arr[left] > arr[largest]) {
                   largest = left;
               }
               if (right < n && arr[right] > arr[largest]) {
                   largest = right;
               }
               if (largest != i) {
                   swap(arr, i, largest);
                   maxHeapify(arr, n, largest);
               }
           }

           private void swap(int[] arr, int i, int j) {
               int temp = arr[i];
               arr[i] = arr[j];
               arr[j] = temp;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(log n)

215. Kth Largest Element in an Array

Understanding the Problem NEW

Find the K-th largest element in an unsorted array. E.g. in `[3,2,1,5,6,4]` with K=2, the answer is 5 (the 2nd largest value).

Brute Force Approach NEW

Sort the entire array and read off the element at the (n-K)-th position — O(n log n), which does more work than necessary since you don't actually need the whole array sorted, just to identify one specific rank.

Why We Optimize NEW

Maintain a **Min-Heap of size K**: process each element, and if the heap has fewer than K elements, add it; if it already has K elements and the new one is bigger than the heap's current minimum (the root), replace the root with the new element. After processing the whole array, the heap's root is exactly the K-th largest element — this works because the heap only ever needs to remember 'the K biggest values seen so far,' discarding anything that's too small to matter, achieving O(n log K) instead of O(n log n) — better when K is much smaller than n, since you're never paying for the cost of fully sorting elements you don't ultimately care about the relative order of.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the kth largest element in an array.

Visual Diagram

```
Array: [3, 2, 1, 5, 6, 4], k=2
       Output: 5

       Sorted: [6, 5, 4, 3, 2, 1]
       2nd largest = 5

       Using Min Heap of size k:
       i=0: 3 → [3]
       i=1: 2 → [2,3]
       i=2: 1 → [1,2,3]
```

```
        i=3: 5 → 5>1 → [2,3,5]
        i=4: 6 → 6>2 → [3,5,6]
        i=5: 4 → 4>3 → [4,5,6]
        Result: 4 (top of min heap)

        Wait, for kth largest with min heap size k:
        After processing all, heap contains k largest elements
        Top is kth largest → 4

        But expected is 5... Let's check:
        [3,2,1,5,6,4], k=2
        Heap of size 2:
        i=0: 3 → [3]
        i=1: 2 → [2,3]
        i=2: 1 → 1<2 → no change, [2,3]
        i=3: 5 → 5>2 → remove 2, add 5 → [3,5]
        i=4: 6 → 6>3 → remove 3, add 6 → [5,6]
        i=5: 4 → 4<5 → no change, [5,6]
        Result: 5 ✓
```

Solution Approach

Concept: Min heap of size k.

Algorithm

```
Algorithm KTH_LARGEST(arr, k):
           minHeap = PriorityQueue()
           For num in arr:
               minHeap.add(num)
               If minHeap.size() > k:
                   minHeap.poll()
           Return minHeap.peek()
```

Java Implementation

```
class Solution {
           public int findKthLargest(int[] nums, int k) {
               PriorityQueue<Integer> minHeap = new PriorityQueue<>();

                 for (int num : nums) {
                     minHeap.add(num);

                   if (minHeap.size() > k) {
                       minHeap.poll();
                   }
               }
               return minHeap.peek();
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n log k) | Space Complexity: O(k)

216. Kth Smallest Element in an Array

Understanding the Problem NEW

The mirror image of the previous problem: find the K-th smallest element in an unsorted array.

Brute Force Approach NEW

Sort the whole array and read off the K-th position — O(n log n).

Why We Optimize NEW

Same idea as problem 215, flipped: maintain a **Max-Heap of size K**, keeping the K smallest values seen so far and discarding anything bigger once the heap is full — the root ends up being the K-th smallest. Recognizing that 'Kth smallest' and 'Kth largest' are the same technique with the heap type flipped (Max-Heap vs Min-Heap) is the actual insight, not a new algorithm.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the kth smallest element in an array.

Visual Diagram

```
Array: [7, 10, 4, 3, 20, 15], k=3
       Output: 7

       Sorted: [3, 4, 7, 10, 15, 20]
       3rd smallest = 7

       Using Max Heap of size k:
       i=0: 7 → [7]
       i=1: 10 → [10,7]
       i=2: 4 → 4<10 → remove 10, add 4 → [7,4]
       i=3: 3 → 3<7 → remove 7, add 3 → [4,3]
       i=4: 20 → 20>4 → no change, [4,3]
       i=5: 15 → 15>4 → no change, [4,3]
       Result: 4

       Wait, expected is 7... Let's trace:
       Max Heap of size k contains k smallest elements
```

```
       Top is kth smallest

       i=0: 7 → [7]
       i=1: 10 → [10,7] (10 is top)
       i=2: 4 → 4<10 → pop 10, add 4 → [7,4]
       i=3: 3 → 3<7 → pop 7, add 3 → [4,3]
       i=4: 20 → 20>4 → no change
       i=5: 15 → 15>4 → no change
       Top = 4 → kth smallest = 4

       But sorted array: [3,4,7,10,15,20], 3rd smallest = 7

       Problem: For k=3, heap should contain [7,4,3] but max heap gives [4,3]?
Actually with size k, i
       Let's re-trace carefully:

       i=0: 7 → [7]
       i=1: 10 → [10,7]
       i=2: 4 → 4<10 → remove 10, add 4 → [7,4]
       i=3: 3 → 3<7 → remove 7, add 3 → [4,3] (size=2)
       i=4: 20 → 20>4 → no change
       i=5: 15 → 15>4 → no change
       Top = 4 (which is 2nd smallest, not 3rd)

       The issue is size tracking. Let's use approach: Add all, then remove n-k
times.

       Alternative: Use PriorityQueue with comparator and remove n-k largest.
```

Solution Approach

Concept: Max heap of size k.

Algorithm

```
Algorithm KTH_SMALLEST(arr, k):
           maxHeap = PriorityQueue(Collections.reverseOrder())
           For num in arr:
               maxHeap.add(num)
               If maxHeap.size() > k:

                   maxHeap.poll()
           Return maxHeap.peek()
```

Java Implementation

```
class Solution {
           public int findKthSmallest(int[] nums, int k) {
               PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Collections.reverseOrder());
```

```
            for (int num : nums) {
                maxHeap.add(num);
                if (maxHeap.size() > k) {
                    maxHeap.poll();
                }
            }
            return maxHeap.peek();
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log k) | Space Complexity: O(k)

217. Sort K-Sorted Array

Understanding the Problem NEW

Given an array where every element is at most K positions away from its correct sorted position (a 'K-sorted' or 'nearly sorted' array), sort it efficiently, exploiting this bound on displacement.

Brute Force Approach NEW

Ignore the K-sorted structure entirely and run a general-purpose sort like Quick Sort or Merge Sort — correct, O(n log n), but doesn't take advantage of the extra guarantee the problem gives you.

Why We Optimize NEW

Because any element can only be at most K positions from where it belongs, the true smallest remaining element is guaranteed to be found within the next K+1 elements at any point in the scan. Maintain a **Min-Heap of size K+1**: add the first K+1 elements, then repeatedly extract the minimum (guaranteed correct for the next output position) and add the next unseen element from the array, one at a time. This produces the fully sorted result in O(n log K) instead of O(n log n) — a real improvement when K is much smaller than n, directly exploiting the 'bounded displacement' guarantee that a general sort would ignore.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Sort an array where each element is at most k positions away.

Visual Diagram

```
Array: [6, 5, 3, 2, 8, 10, 9], k=3
       Output: [2, 3, 5, 6, 8, 9, 10]

       Using Min Heap:
       i=0 to k: heap=[6,5,3] → sorted: 3
       i=1 to k+1: heap=[6,5,2] → sorted: 2
       i=2 to k+2: heap=[6,5,8] → sorted: 5
       i=3 to k+3: heap=[6,8,10] → sorted: 6
       i=4 to k+4: heap=[8,10,9] → sorted: 8
       i=5: heap=[10,9] → sorted: 9
```

```
        i=6: heap=[10] → sorted: 10

        Result: [3, 2, 5, 6, 8, 9, 10]... Wait

        Let's re-trace:
        Add first k+1 elements: [6,5,3,2] → min=2
        Add 8: [6,5,3,8] → min=3
        Add 10: [6,5,8,10] → min=5
        Add 9: [6,8,10,9] → min=6
        Remaining: [8,9,10] → sort → [8,9,10]
        Result: [2,3,5,6,8,9,10]
```

Solution Approach

Concept: Min heap of size k+1.

Algorithm

```
Algorithm SORT_K_SORTED(arr, k):
           minHeap = PriorityQueue()
           result = []

           For i = 0 to k:
               minHeap.add(arr[i])

           For i = k+1 to n-1:
               result.add(minHeap.poll())
               minHeap.add(arr[i])

           While !minHeap.isEmpty():
               result.add(minHeap.poll())

           Return result
```

Java Implementation

```
class Solution {
           public int[] sortKSortedArray(int[] arr, int k) {
               PriorityQueue<Integer> minHeap = new PriorityQueue<>();
               int n = arr.length;
               int[] result = new int[n];

                 for (int i = 0; i <= Math.min(k, n - 1); i++) {
                     minHeap.add(arr[i]);
                 }

               int index = 0;
               for (int i = k + 1; i < n; i++) {
                   result[index++] = minHeap.poll();
                   minHeap.add(arr[i]);
```

```
            }

            while (!minHeap.isEmpty()) {
                result[index++] = minHeap.poll();
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log k) | Space Complexity: O(k)

218. Merge M Sorted Lists

Understanding the Problem NEW

Given M separate sorted linked lists, merge them all into one fully sorted linked list.

Brute Force Approach NEW

Merge the lists two at a time using the ordinary two-list merge from Merge Sort (Chapter 1) — merging list 1 with list 2, then that result with list 3, and so on. If each list has roughly n/M elements, this sequential merging costs O(n×M) in the worst case, since each of the M merge steps can touch up to all n elements accumulated so far.

Why We Optimize NEW

Instead of merging sequentially, use a **Min-Heap containing the current front node of every list**: repeatedly extract the overall minimum (across all M lists' current fronts), append it to the result, and push that extracted node's *next* node (from the same list) back into the heap. Since the heap only ever holds at most M elements at a time, each of the n total extractions costs O(log M), giving O(n log M) total — much better than O(n×M) when M is large, because you're comparing across all M lists' current candidates simultaneously via the heap, rather than fully re-merging accumulated results at every step.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Merge k sorted linked lists into one sorted list.

Visual Diagram

```
Lists:
       1 → 4 → 5
       1 → 3 → 4
       2 → 6

       Merged: 1 → 1 → 2 → 3 → 4 → 4 → 5 → 6

       Using Min Heap:
```

```
Step 1: Add first node of each list → [1,1,2]
Step 2: Poll 1 (from list1), add next 4
Step 3: Poll 1 (from list2), add next 3
Step 4: Poll 2, add next 6
Step 5: Poll 3, add next 4
Step 6: Poll 4, add next 5
Step 7: Poll 4, add next null
Step 8: Poll 5, no more
Step 9: Poll 6, no more
```

Solution Approach

Concept: Min heap with list nodes.

Algorithm

```
Algorithm MERGE_K_LISTS(lists):
           minHeap = PriorityQueue(comparator for node values)

            For each list in lists:
                If list is not null:
                    minHeap.add(list)

            dummy = Node(0)
            current = dummy

            While !minHeap.isEmpty():
                node = minHeap.poll()
                current.next = node
                current = current.next

                 If node.next != null:
                     minHeap.add(node.next)

            Return dummy.next
```

Java Implementation

```
class Solution {
           public ListNode mergeKLists(ListNode[] lists) {
               PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a, b) ->
a.val - b.val);

                 for (ListNode list : lists) {
                     if (list != null) {
                         minHeap.add(list);
                     }
                 }

               ListNode dummy = new ListNode(0);
```

```
            ListNode current = dummy;

            while (!minHeap.isEmpty()) {
                ListNode node = minHeap.poll();
                current.next = node;
                current = current.next;

                if (node.next != null) {
                    minHeap.add(node.next);
                }
            }
            return dummy.next;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log k) | Space Complexity: O(k)

219. Replace Each Array Element by Its Corresponding Rank

Understanding the Problem NEW

Replace every element in the array with its 'rank' — its 1-based position if the array were sorted (with equal elements getting the same rank). E.g. `[40,10,20,30]` → ranks `[4,1,2,3]`.

Brute Force Approach NEW

For each element, count how many elements in the array are smaller than or equal to it (a nested-loop comparison) — $O(n^2)$.

Why We Optimize NEW

Since a rank is fundamentally about relative sorted order, you can compute it by **sorting a copy of the array once** (or using a Min-Heap to extract elements in sorted order), assigning rank numbers as you go, and storing each value's rank in a hashmap — then look up each original element's rank from that map. This turns an $O(n^2)$ all-pairs comparison into O(n log n) for the sort/heap-based rank assignment plus O(n) for the final lookup pass — reusing the fact that ranks are just 'position in sorted order' rather than recomputing relative order pairwise for every element.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Replace each element with its rank (1-based smallest to largest).

Visual Diagram

```
Input: [10, 20, 15, 10, 30]
       Output: [1, 3, 2, 1, 4]

       Step 1: Sort unique values: [10, 15, 20, 30]
       Step 2: Map value → rank: 10→1, 15→2, 20→3, 30→4
       Step 3: Replace: [1, 3, 2, 1, 4]

       Using Min Heap:
       Create pairs (value, index)
```

```
Heap: (10,0), (10,3), (15,2), (20,1), (30,4)
Rank=1: pop (10,0) → result[0]=1
Rank=2: pop (10,3) → result[3]=1 (same rank for duplicates)
Rank=3: pop (15,2) → result[2]=2
Wait, for duplicates they should have same rank

Actually:
Rank=1: pop (10,0) → result[0]=1
Rank=1: pop (10,3) → result[3]=1
Rank=2: pop (15,2) → result[2]=2
Rank=3: pop (20,1) → result[1]=3
Rank=4: pop (30,4) → result[4]=4
Result: [1,3,2,1,4]
```

Solution Approach

Concept: Min heap or sorting with map.

Algorithm

```
Algorithm REPLACE_WITH_RANK(arr):
           pairs = new Pair[arr.length]
           For i = 0 to n-1:
               pairs[i] = (arr[i], i)
           Sort pairs by value

           result = new int[n]
           rank = 1

           For i = 0 to n-1:
               result[pairs[i].index] = rank
               If i+1 < n and pairs[i].value != pairs[i+1].value:
                   rank++

           Return result
```

Java Implementation

```
class Solution {
           public int[] replaceWithRank(int[] arr) {
               int n = arr.length;
               int[][] pairs = new int[n][2];

                 for (int i = 0; i < n; i++) {
                     pairs[i] = new int[]{arr[i], i};
                 }

               Arrays.sort(pairs, (a, b) -> a[0] - b[0]);

               int[] result = new int[n];
```

```
            int rank = 1;

            for (int i = 0; i < n; i++) {
                result[pairs[i][1]] = rank;
                if (i < n - 1 && pairs[i][0] != pairs[i + 1][0]) {
                    rank++;
                }
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

220. Task Scheduler

Understanding the Problem NEW

Given a list of tasks (each represented by a type) and a cooldown period N (the same task type can't run again until N other slots have passed, possibly filled with idle time), find the minimum total time needed to complete all tasks.

Brute Force Approach NEW

Try every possible ordering of tasks and idle slots, checking which satisfies the cooldown constraint with minimum total time — combinatorially expensive, since there are many possible orderings to consider.

Why We Optimize NEW

A greedy, frequency-based approach works well here: schedule the **most frequent remaining task type** first at each available slot (using a Max-Heap keyed by remaining count), since the most frequent task is the one most likely to run out of 'safe' cooldown slots if delayed — this greedy priority ensures no task type ever gets stuck waiting longer than necessary. Combined with tracking a cooldown queue of recently-used tasks, this achieves the optimal schedule in roughly O(total tasks × log(distinct task types)) instead of exploring every possible task ordering — a case where a heap-driven greedy choice ('always do the most urgent thing next') avoids a combinatorial search entirely.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Schedule tasks with cooldown between same tasks.

Visual Diagram

```
Tasks: ["A","A","A","B","B","B"], n=2
       Output: 8

       A B _ A B _ A B → 8 time units

       Frequency: A=3, B=3
       Most frequent = 3
       idleTime = (3-1) * 2 = 4
```

```
        Remaining tasks = 6 - 3 = 3
        idleTime -= 3 = 1
        Total = 3 + 6 = 9?

        Let's recalc:
        maxCount = 3, maxCountTasks = 2 (A and B both appear 3 times)
        partCount = maxCount - 1 = 2
        emptySlots = partCount * (n - (maxCountTasks - 1)) = 2 * (2 - 1) = 2
        availableTasks = 6 - maxCount * maxCountTasks = 6 - 6 = 0
        idle = max(0, emptySlots - availableTasks) = 2
        result = 6 + 2 = 8

        Alternative formula:
        result = max(tasks.length, (maxCount-1)*(n+1) + maxCountTasks)
        maxCount=3, maxCountTasks=2, n=2
        (3-1)*(2+1) + 2 = 2*3+2 = 8
        tasks.length=6
        result = max(6, 8) = 8
```

Solution Approach

Concept: Greedy with frequency counting.

Algorithm

```
Algorithm TASK_SCHEDULER(tasks, n):
           freq = new int[26]
           For task in tasks: freq[task-'A']++

           maxCount = 0, maxCountTasks = 0
           For count in freq:
               If count > maxCount:
                   maxCount = count
                   maxCountTasks = 1
               Else If count == maxCount:
                   maxCountTasks++

           result = Math.max(tasks.length, (maxCount-1)*(n+1) + maxCountTasks)
           Return result
```

Java Implementation

```
class Solution {
           public int leastInterval(char[] tasks, int n) {
               int[] freq = new int[26];
               for (char task : tasks) {
                   freq[task - 'A']++;

              }
```

```
            int maxCount = 0;
            int maxCountTasks = 0;

            for (int count : freq) {
                if (count > maxCount) {
                    maxCount = count;
                    maxCountTasks = 1;
                } else if (count == maxCount) {
                    maxCountTasks++;
                }
            }

            int result = Math.max(tasks.length, (maxCount - 1) * (n + 1) +
maxCountTasks);
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

221. Hands of Straights

Understanding the Problem NEW

Given a hand of cards (numbers), determine if they can be grouped into groups of consecutive-value straights of a given size W (e.g. cards `[1,2,3,6,2,3,4,7,8]`, W=3 → groups `[1,2,3]`, `[2,3,4]`, `[6,7,8]`).

Brute Force Approach NEW

Try every possible way to partition the cards into groups and check if each is a valid consecutive run — combinatorially expensive without a clear strategy.

Why We Optimize NEW

A greedy approach using a frequency map and a **Min-Heap of distinct card values**: always start a new group from the **smallest remaining** card value (since if the smallest card can't start a valid consecutive run, no arrangement can ever place it anywhere else, as there's nothing smaller left to pair it with), then greedily consume that value and the next W-1 consecutive values to form one group, decrementing their counts. Repeating this — always resolving the smallest remaining value first — greedily builds valid groups without ever needing to explore alternative groupings, since the smallest remaining card's fate is always forced.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if cards can be grouped into consecutive sequences.

Visual Diagram

```
Hand: [1,2,3,6,2,3,4,7,8], groupSize=3
       Output: true

       Groups: [1,2,3], [2,3,4], [6,7,8]

       Using Min Heap:
       Map frequencies: 1:1,2:2,3:2,4:1,6:1,7:1,8:1
       Start with 1: group [1,2,3]
         freq[1]=0, freq[2]=1, freq[3]=1
       Start with 2: group [2,3,4]
```

```
  freq[2]=0, freq[3]=0, freq[4]=0
Start with 6: group [6,7,8]
  freq[6]=0, freq[7]=0, freq[8]=0
Result: true

Hand: [1,2,3,4,5], groupSize=4
Output: false
Start with 1: [1,2,3,4] → 5 remaining
Can't form another group → false
```

Solution Approach

Concept: Min heap + frequency map.

Algorithm

```
Algorithm HANDS_OF_STRAIGHTS(hand, groupSize):
           If hand.length % groupSize != 0: return false

           freq = new HashMap()
           For card in hand: freq[card]++

           minHeap = new PriorityQueue(freq.keySet())

           While !minHeap.isEmpty():
               first = minHeap.peek()
               For i = 0 to groupSize-1:
                   current = first + i
                   If !freq.containsKey(current) or freq.get(current) == 0:
                       return false
                   freq.put(current, freq.get(current)-1)
                   If freq.get(current) == 0:
                       If minHeap.peek() != current: return false
                       minHeap.poll()

           Return true
```

Java Implementation

```
class Solution {
           public boolean isNStraightHand(int[] hand, int groupSize) {
               if (hand.length % groupSize != 0) return false;

                 Map<Integer, Integer> freq = new HashMap<>();
                 for (int card : hand) {

                   freq.put(card, freq.getOrDefault(card, 0) + 1);
               }

               PriorityQueue<Integer> minHeap = new PriorityQueue<>(freq.keySet());
```

```
            while (!minHeap.isEmpty()) {
                int first = minHeap.peek();

                for (int i = 0; i < groupSize; i++) {
                    int current = first + i;
                    if (!freq.containsKey(current) || freq.get(current) == 0) {
                        return false;
                    }
                    freq.put(current, freq.get(current) - 1);
                    if (freq.get(current) == 0) {
                        if (minHeap.peek() != current) {
                            return false;
                        }
                        minHeap.poll();
                    }
                }
            }
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

222. Design Twitter

Understanding the Problem NEW

Design a simplified Twitter: users can post tweets, follow/unfollow other users, and retrieve their 10 most recent tweets from themselves and everyone they follow, merged in reverse-chronological order.

Brute Force Approach NEW

To get the feed, you could gather every tweet from the user and everyone they follow into one list, then sort by timestamp and take the top 10 — correct, but sorting potentially thousands of tweets just to get the top 10 wastes most of that sorting effort.

Why We Optimize NEW

This is essentially the 'Merge M Sorted Lists' problem (218) in disguise: each followed user's tweet history is already naturally in reverse-chronological (sorted) order, so use a **Max-Heap** containing the most recent tweet from the user and each followed user, repeatedly extracting the overall most recent tweet and pushing that same user's next-most-recent tweet back into the heap, stopping after 10 extractions. This reuses the exact same 'heap over multiple sorted sources' technique as merging sorted lists, achieving O(10 × log(number of followed users)) for feed retrieval instead of sorting every tweet from every followed user.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Design Twitter with post, get feed, follow, unfollow.

Visual Diagram

```
Users: 1, 2, 3
       Tweets: user1: [tweet1, tweet4], user2: [tweet2], user3: [tweet3]

       getNewsFeed(1) → [tweet4, tweet3, tweet2, tweet1] (latest 10)

       Data Structures:
       Map<Integer, User> users
```

```
        User: {
            tweets: LinkedList<Tweet>
            following: Set<Integer>
        }
        Tweet: { id, timestamp }

        postTweet(userId, tweetId):
            user = users.get(userId)
            user.tweets.addFirst(tweetId)

        getNewsFeed(userId):
            tweets = []
            For each followee in user.following:
                Add all tweets from followee
            Sort by timestamp desc
            Return top 10
```

Solution Approach

Concept: HashMap + Heap for feed.

Java Implementation

```
class Twitter {
            private class Tweet {
                int tweetId;
                int timestamp;
                Tweet next;
                Tweet(int tweetId, int timestamp) {
                    this.tweetId = tweetId;
                    this.timestamp = timestamp;
                }
            }

             private class User {
                 int userId;
                 Set<Integer> following;
                 Tweet tweets;

                 User(int userId) {
                     this.userId = userId;
                     following = new HashSet<>();
                     following.add(userId);
                 }
             }

            private Map<Integer, User> users;
            private int timestamp;

            public Twitter() {
                users = new HashMap<>();
```

```
                timestamp = 0;
            }

            public void postTweet(int userId, int tweetId) {
                users.computeIfAbsent(userId, k -> new User(userId));
                Tweet tweet = new Tweet(tweetId, timestamp++);
                tweet.next = users.get(userId).tweets;
                users.get(userId).tweets = tweet;
            }

            public List<Integer> getNewsFeed(int userId) {
                users.computeIfAbsent(userId, k -> new User(userId));

                PriorityQueue<Tweet> heap = new PriorityQueue<>((a, b) ->
a.timestamp - b.timestamp);

                for (int followeeId : users.get(userId).following) {
                    if (!users.containsKey(followeeId)) continue;
                    Tweet tweet = users.get(followeeId).tweets;
                    while (tweet != null && heap.size() < 10) {
                        heap.add(tweet);
                        tweet = tweet.next;
                    }
                    if (heap.size() == 10 && tweet != null) {
                        while (tweet != null) {
                            if (tweet.timestamp > heap.peek().timestamp) {
                                heap.poll();
                                heap.add(tweet);
                            }
                            tweet = tweet.next;
                        }
                    }
                }

                List<Integer> result = new ArrayList<>();
                while (!heap.isEmpty()) {
                    result.add(heap.poll().tweetId);
                }
                Collections.reverse(result);
                return result;
            }

            public void follow(int followerId, int followeeId) {
                users.computeIfAbsent(followerId, k -> new User(followerId));
                users.computeIfAbsent(followeeId, k -> new User(followeeId));
                users.get(followerId).following.add(followeeId);
            }

            public void unfollow(int followerId, int followeeId) {
                if (followerId != followeeId) {
                    users.get(followerId).following.remove(followeeId);
                }
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n log k) | Space Complexity: O(n)

223. Connect n Ropes with Minimal Cost

Understanding the Problem NEW

Given ropes of various lengths, connect them all into one rope, where connecting two ropes costs (sum of their lengths), and you want to minimize the *total* cost across all connections.

Brute Force Approach NEW

Try every possible order of connecting ropes and compute the total cost for each — combinatorially expensive (the number of possible connection orders grows very quickly).

Why We Optimize NEW

The greedy insight: always connect the **two currently-smallest** ropes at each step. This minimizes total cost because each rope's length gets 'counted' once for every connection it participates in downstream — so keeping small ropes small for as long as possible (by combining them early, before they get folded into larger and larger combined ropes) minimizes how many times their length contributes to the total. A **Min-Heap** makes 'always grab the two smallest' an O(log n) operation per step, giving O(n log n) total — turning a combinatorially expensive ordering search into a straightforward greedy-with-heap process.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Connect ropes with minimal cost (sum of lengths).

Visual Diagram

```
Ropes: [4, 3, 2, 6]
       Minimal cost: 29

       Step 1: Connect 2+3=5, cost=5, ropes=[4,5,6]
       Step 2: Connect 4+5=9, cost=5+9=14, ropes=[6,9]
       Step 3: Connect 6+9=15, cost=14+15=29

       Using Min Heap:
       [2,3,4,6]
       Step 1: 2+3=5, cost=5, add 5 → [4,5,6]
```

```
       Step 2: 4+5=9, cost=14, add 9 → [6,9]
       Step 3: 6+9=15, cost=29
```

Solution Approach

Concept: Min heap always connect two smallest.

Algorithm

```
Algorithm MIN_COST_CONNECT(ropes):
           minHeap = PriorityQueue(ropes)
           cost = 0

           While minHeap.size() > 1:
               first = minHeap.poll()
               second = minHeap.poll()
               sum = first + second
               cost += sum
               minHeap.add(sum)

           Return cost
```

Java Implementation

```
class Solution {
           public int minCostConnectRopes(int[] ropes) {
               PriorityQueue<Integer> minHeap = new PriorityQueue<>();
               for (int rope : ropes) {
                   minHeap.add(rope);
               }

                 int cost = 0;
                 while (minHeap.size() > 1) {
                     int first = minHeap.poll();
                     int second = minHeap.poll();
                     int sum = first + second;
                     cost += sum;
                     minHeap.add(sum);
                 }
                 return cost;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

224. Kth Largest Element in a Stream of Data

Understanding the Problem NEW

Design a structure that, given a continuous stream of incoming numbers (arriving one at a time, not known in advance), can always report the current K-th largest number seen so far at any point.

Brute Force Approach NEW

Store every number seen so far in a list, and whenever asked for the K-th largest, sort the entire list and read off the answer — O(n log n) per query, terrible if queries happen frequently as the stream grows.

Why We Optimize NEW

Maintain a running **Min-Heap of size K**, exactly like problem 215, but now incrementally: every time a new number arrives, add it to the heap (evicting the current minimum if the heap exceeds size K) — the root is always the current K-th largest, retrievable in O(1) at any moment, with each new number processed in O(log K). This is the 'streaming' version of problem 215's static-array technique — the heap's small, bounded size (K, not n) is precisely what makes it practical to maintain incrementally across a continuously growing stream rather than re-sorting everything on every query.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find kth largest in a stream of numbers.

Visual Diagram

```
Stream: [4, 5, 8, 2, 3, 5, 10, 9, 4], k=3

       After each:
       4 → [4] → kth largest = -1 (not enough elements)
       5 → [4,5] → -1
       8 → [4,5,8] → 4
       2 → [4,5,8,2] → min=2? Wait, heap should keep size k

       Min Heap of size k:
       4: [4]
```

```
5: [4,5]
8: [4,5,8]
2: 2<4 → no change, [4,5,8] → kth=4
3: 3<4 → no change, [4,5,8] → kth=4
5: 5>4 → remove 4, add 5 → [5,5,8] → kth=5
10: 10>5 → remove 5, add 10 → [5,8,10] → kth=5
9: 9>5 → remove 5, add 9 → [8,9,10] → kth=8
4: 4<8 → no change, [8,9,10] → kth=8
```

Solution Approach

Concept: Min heap of size k.

Algorithm

```
Algorithm KTH_LARGEST_STREAM(stream, k):
           minHeap = PriorityQueue()
           For num in stream:
               minHeap.add(num)
               If minHeap.size() > k:
                   minHeap.poll()
               If minHeap.size() == k:
                   print minHeap.peek()
               Else:
                   print -1
```

Java Implementation

```
class KthLargest {
           private PriorityQueue<Integer> minHeap;
           private int k;

           public KthLargest(int k, int[] nums) {
               this.k = k;
               minHeap = new PriorityQueue<>();
               for (int num : nums) {
                   add(num);
               }
           }

           public int add(int val) {
               minHeap.add(val);
               if (minHeap.size() > k) {
                   minHeap.poll();
               }

               return minHeap.peek();
           }
       }
```

Time & Space Complexity

Time Complexity: O(log k) per add | Space Complexity: O(k)

225. Maximum Sum Combination

Understanding the Problem NEW

Given two arrays, find the K pairs (one element from each array) with the largest sums, without generating and sorting every possible pair (which would be a times b total pairs).

Brute Force Approach NEW

Generate every possible pair (one from each array), compute their sums, sort all pairs by sum, and take the top K — O(n×m log(n×m)), generating far more pairs than needed if K is small.

Why We Optimize NEW

Sort both arrays first (so the largest possible sums are 'near the top' of each), then use a **Max-Heap** seeded with a few promising initial candidate pairs (e.g. pairing the largest elements from each array), extracting the current best sum and pushing its 'neighbouring' unexplored pairs (like decrementing one index) back into the heap, using a visited-set to avoid processing the same pair twice — repeating this K times. This explores only the pairs that could plausibly be in the top K, guided by the heap always expanding from the currently-best-known candidate, achieving roughly O(K log K) instead of generating and sorting all n×m pairs upfront.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find k maximum sum combinations from two arrays.

Visual Diagram

```
A = [1, 4, 2], B = [3, 2, 5], k = 3
       All combinations:
       1+5=6, 1+3=4, 1+2=3
       4+5=9, 4+3=7, 4+2=6
       2+5=7, 2+3=5, 2+2=4
       Top 3: [9,7,7]

       Using Max Heap:
```

```
Sort: A=[4,2,1], B=[5,3,2]
Start: (4+5, 0, 0) → 9
Add: (4+3,0,1), (2+5,1,0)
Pop: 7 (4+3) → add (4+2,0,2)
Pop: 7 (2+5) → add (2+3,1,1)
Result: [9,7,7]
```

Solution Approach

Concept: Max heap with visited set.

Algorithm

```
Algorithm MAX_SUM_COMBINATION(A, B, k):
           Sort A descending, B descending
           maxHeap = PriorityQueue(max)
           visited = HashSet()

           maxHeap.add((A[0]+B[0], 0, 0))
           visited.add((0,0))
           result = []

           While k-- > 0:
               (sum, i, j) = maxHeap.poll()
               result.add(sum)

                 If i+1 < n and (i+1,j) not visited:
                     maxHeap.add((A[i+1]+B[j], i+1, j))
                     visited.add((i+1,j))
                 If j+1 < n and (i,j+1) not visited:
                     maxHeap.add((A[i]+B[j+1], i, j+1))
                     visited.add((i,j+1))

           Return result
```

Java Implementation

```
class Solution {
           public List<Integer> maxSumCombinations(int[] A, int[] B, int k) {
               Arrays.sort(A);
               Arrays.sort(B);
               int n = A.length;
               List<Integer> result = new ArrayList<>();

                 PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> b[0]
- a[0]);
                 Set<String> visited = new HashSet<>();
                 maxHeap.add(new int[]{A[n-1] + B[n-1], n-1, n-1});

               visited.add((n-1) + "," + (n-1));
```

```
            while (k-- > 0 && !maxHeap.isEmpty()) {
                int[] top = maxHeap.poll();
                int sum = top[0], i = top[1], j = top[2];
                result.add(sum);

                String key1 = (i-1) + "," + j;
                if (i > 0 && !visited.contains(key1)) {
                    maxHeap.add(new int[]{A[i-1] + B[j], i-1, j});
                    visited.add(key1);
                }

                String key2 = i + "," + (j-1);
                if (j > 0 && !visited.contains(key2)) {
                    maxHeap.add(new int[]{A[i] + B[j-1], i, j-1});
                    visited.add(key2);
                }
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(k log k) | Space Complexity: O(k)

226. Find Median from Data Stream

Understanding the Problem NEW

Given a continuous stream of numbers, maintain the ability to report the current median (the middle value, or average of the two middle values if the count is even) at any point as new numbers arrive.

Brute Force Approach NEW

Keep all numbers seen so far in a list, and whenever asked for the median, sort the whole list and read off the middle — O(n log n) per query, which is far too slow if the median is queried frequently as the stream grows.

Why We Optimize NEW

Maintain **two heaps simultaneously**: a Max-Heap holding the smaller half of the numbers seen so far, and a Min-Heap holding the larger half — kept balanced in size (differing by at most 1). The median is then always either the root of whichever heap has more elements, or the average of both roots if they're equal in size. Every new number is inserted into the appropriate heap (based on comparison with the current roots) and the two heaps are rebalanced if needed (moving the root from one to the other) — all in O(log n) per insertion, with the median retrievable in O(1) at any time. This 'two balanced heaps, split at the median' structure is the classic solution for maintaining a running median efficiently in a streaming context.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find median of a stream of numbers.

Visual Diagram

```
Stream: [1, 2, 3, 4, 5]

       After 1: [1] → median=1
       After 2: [1,2] → median=(1+2)/2=1.5
       After 3: [1,2,3] → median=2
       After 4: [1,2,3,4] → median=(2+3)/2=2.5
       After 5: [1,2,3,4,5] → median=3
```

```
Data Structure:
Max Heap (lower half): [1,2]
Min Heap (upper half): [3,4,5]

Add 6:
  maxHeap: [1,2], minHeap: [3,4,5,6]
  Balance: maxHeap: [1,2,3], minHeap: [4,5,6]
  Median: (3+4)/2=3.5
```

Solution Approach

Concept: Two heaps (max for lower, min for upper).

Algorithm

```
Algorithm MEDIAN_STREAM:
           maxHeap = PriorityQueue(max) // lower half
           minHeap = PriorityQueue(min) // upper half

           addNum(num):
               If maxHeap.isEmpty() or num <= maxHeap.peek():
                   maxHeap.add(num)
               Else:
                   minHeap.add(num)

                 // Balance
                 If maxHeap.size() > minHeap.size() + 1:
                     minHeap.add(maxHeap.poll())
                 Else If minHeap.size() > maxHeap.size():
                     maxHeap.add(minHeap.poll())

           findMedian():
               If maxHeap.size() > minHeap.size():
                   Return maxHeap.peek()
               Else:
                   Return (maxHeap.peek() + minHeap.peek()) / 2.0
```

Java Implementation

```
class MedianFinder {
           private PriorityQueue<Integer> maxHeap;
           private PriorityQueue<Integer> minHeap;

           public MedianFinder() {
               maxHeap = new PriorityQueue<>(Collections.reverseOrder());
               minHeap = new PriorityQueue<>();
           }

           public void addNum(int num) {
```

```
            if (maxHeap.isEmpty() || num <= maxHeap.peek()) {
                maxHeap.add(num);
            } else {
                minHeap.add(num);
            }

            if (maxHeap.size() > minHeap.size() + 1) {
                minHeap.add(maxHeap.poll());
            } else if (minHeap.size() > maxHeap.size()) {
                maxHeap.add(minHeap.poll());
            }
        }

        public double findMedian() {
            if (maxHeap.size() > minHeap.size()) {
                return maxHeap.peek();
            } else {
                return (maxHeap.peek() + minHeap.peek()) / 2.0;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(n)

227. K Most Frequent Elements

Understanding the Problem NEW

Find the K most frequently occurring elements in an array. E.g. `[1,1,1,2,2,3]`, K=2 → `[1,2]` (1 appears 3 times, 2 appears twice).

Brute Force Approach NEW

Count frequencies with a hashmap, then sort all distinct elements by frequency and take the top K — O(n log n) for the sort, doing more ordering work than necessary if you only need the top K, not a full ranking of everyone.

Why We Optimize NEW

Count frequencies with a hashmap (O(n)), then maintain a **Min-Heap of size K** keyed by frequency (exactly the same 'keep only the K best, discard the rest' pattern as problem 215's Kth Largest Element) — achieving O(n log K) instead of O(n log n) for the ranking step. (An alternative **bucket sort** approach achieves O(n) instead: since frequency can range from 1 to n, create n buckets indexed by frequency, place each distinct element into its frequency's bucket, then read off the top K by scanning buckets from highest frequency down — avoiding comparison-based sorting/heap operations entirely by exploiting the bounded range of possible frequency values, similar in spirit to how the Sieve of Eratosthenes avoids per-number primality checks by using array-indexed buckets.)

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find k most frequent elements in an array.

Visual Diagram

```
Input: [1,1,1,2,2,3], k=2
       Output: [1,2]

       Frequency: 1:3, 2:2, 3:1

       Method 1: Min Heap
       Build frequency map
```

```
Add entries to min heap of size k
Result: [1,2]

Method 2: Bucket Sort
Frequency: [3,2,1]
Buckets:
0: []
1: [3]
2: [2]
3: [1]
Collect k elements: [1,2]
```

Solution Approach

Concept: Min heap or bucket sort.

Algorithm (Min Heap)

Algorithm K_FREQUENT(arr, k):

freq = new HashMap()

For num in arr: freq[num]++

minHeap = PriorityQueue(comparator by frequency)

For entry in freq:

minHeap.add(entry)

If minHeap.size() > k:

minHeap.poll()

Return minHeap entries

Java Implementation (Min Heap)

class Solution {

public int[] topKFrequent(int[] nums, int k) {

Map<Integer, Integer> freq = new HashMap<>();

for (int num : nums) {

freq.put(num, freq.getOrDefault(num, 0) + 1);

}

PriorityQueue<Map.Entry<Integer, Integer>> minHeap =

```
new PriorityQueue<>((a, b) -> a.getValue() - b.getValue());
for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
minHeap.add(entry);
if (minHeap.size() > k) {
minHeap.poll();
}
}
int[] result = new int[k];
for (int i = 0; i < k; i++) {
result[i] = minHeap.poll().getKey();
}
return result;
}
}
```

Java Implementation (Bucket Sort) - O(n)

```
class Solution {
public int[] topKFrequent(int[] nums, int k) {
Map<Integer, Integer> freq = new HashMap<>();
for (int num : nums) {
freq.put(num, freq.getOrDefault(num, 0) + 1);
}
List<Integer>[] buckets = new List[nums.length + 1];
for (int key : freq.keySet()) {
int count = freq.get(key);
if (buckets[count] == null) {
buckets[count] = new ArrayList<>();
```

```
}
buckets[count].add(key);
}
int[] result = new int[k];
int index = 0;
for (int i = buckets.length - 1; i >= 0 && index < k; i--) {
if (buckets[i] != null) {
for (int num : buckets[i]) {
result[index++] = num;
if (index == k) break;
}
}
}
return result;
}
}
```

Time & Space Complexity

Time Complexity: O(n log k) or O(n) | Space Complexity: O(n)

Heap Summary

Problem	Difficulty	Time	Space	Key Technique
Heap Basics	Learning	O(log n)	O(n)	Array-based heap
Min Heap Check	Learning	O(n)	O(1)	Property validation
Min to Max Heap	Learning	O(n)	O(log n)	Heapify all
Kth Largest	Medium	O(n log k)	O(k)	Min heap size k
Kth Smallest	Medium	O(n log k)	O(k)	Max heap size k
Sort K-Sorted	Medium	O(n log k)	O(k)	Min heap sliding
Merge K Lists	Medium	O(n log k)	O(k)	Heap with nodes
Replace with Rank	Medium	O(n log n)	O(n)	Sorting
Task Scheduler	Medium	O(n)	O(1)	Greedy + math
Hands of Straights	Medium	O(n log n)	O(n)	Heap + map
Design Twitter	Hard	O(n log k)	O(n)	Heap + maps
Connect Ropes	Hard	O(n log n)	O(n)	Min heap
Kth Largest Stream	Hard	O(log k)	O(k)	Min heap
Max Sum Combination	Hard	O(k log k)	O(k)	Heap + visited
Median Stream	Hard	O(log n)	O(n)	Two heaps
K Frequent Elements	Hard	O(n log k)	O(n)	Min heap or bucket

Greedy Algorithms — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

228. Assign Cookies

Understanding the Problem NEW

You have children each with a 'greed factor' (the minimum cookie size that would satisfy them) and cookies of various sizes. Assign cookies to children to maximize how many children end up satisfied (each cookie can go to at most one child).

Brute Force Approach NEW

Try every possible way of assigning cookies to children and count satisfied children for each assignment — combinatorially expensive, exploring assignments that are obviously wasteful (like giving a huge cookie to an easily-satisfied child, wasting its size on someone who didn't need it).

Why We Optimize NEW

Sort both the children's greed factors and the cookie sizes. Then greedily walk through both sorted lists together: give the smallest available cookie that's big enough to satisfy the current least-greedy child — if it's big enough, both move forward; if not, only the cookie pointer moves forward (that cookie is too small for anyone remaining, or rather, that child needs a bigger cookie found later). Matching the smallest sufficient cookie to the least-greedy unsatisfied child (rather than 'wasting' a big cookie on someone who needed little) leaves the largest cookies available for the more demanding children — this greedy 'smallest fit' choice, after sorting, achieves the optimal number of satisfied children in O(n log n) instead of exploring every assignment combination.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Assign cookies to children to maximize satisfied children.

Visual Diagram

```
Children Greed: [1, 2, 3]
          Cookies Size: [1, 1]
          Output: 1 (only child with greed 1 gets cookie)

          Sort both:
          Children: [1, 2, 3]
```

```
Cookies: [1, 1]

i=0 (greed=1), j=0 (cookie=1): 1>=1 → satisfied, i=1, j=1
i=1 (greed=2), j=1 (cookie=1): 2>1 → no, j=2
i=1, j=2: end
Result: 1

Children Greed: [1, 2], Cookies: [1, 2, 3]
Sorted: [1, 2], [1, 2, 3]
i=0: 1>=1 → satisfied, i=1, j=1
i=1: 2>=2 → satisfied, i=2, j=2
Result: 2
```

Solution Approach

Concept: Sort both, assign smallest cookie that satisfies greed.

Algorithm

```
Algorithm ASSIGN_COOKIES(greed, cookies):
          Sort(greed), Sort(cookies)
          i = 0, j = 0

          While i < greed.length and j < cookies.length:
              If cookies[j] >= greed[i]:
                  i++
              j++

          Return i
```

Java Implementation

```
class Solution {
          public int findContentChildren(int[] g, int[] s) {
              Arrays.sort(g);
              Arrays.sort(s);

                int i = 0, j = 0;
                while (i < g.length && j < s.length) {
                    if (s[j] >= g[i]) {
                         i++;
                    }
                    j++;
                }
                return i;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n log n + m log m) | Space Complexity: O(1)

229. Fractional Knapsack Problem

Understanding the Problem NEW

You have a knapsack with a weight capacity, and a set of items each with a weight and value — unlike the classic 0/1 Knapsack, here you're allowed to take a *fraction* of an item (like pouring out part of a sack of grain). Maximize the total value you can carry.

Brute Force Approach NEW

Try every possible combination of fractions for each item and compute total value — since fractions are continuous, there are infinitely many combinations to consider without a clear strategy, making truly brute enumeration impossible; even a coarse discretized brute-force would be extremely expensive.

Why We Optimize NEW

Because you can take *any fraction* of an item, the right measure of an item's 'efficiency' is its **value-per-unit-weight ratio** (value/weight). Sort items by this ratio in descending order, then greedily take as much as possible of the highest-ratio item first, then the next, and so on, taking only a fraction of the last item that fits when the capacity runs out. Because fractional taking is allowed, there's no risk of 'almost fitting but not quite' the way there is in 0/1 Knapsack — greedily maximizing value-density at every step is always optimal here, achieving O(n log n) for the sort, whereas the 0/1 version (no fractions allowed) genuinely requires the more expensive Dynamic Programming approach covered in the next chapter, since greedy can fail there.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximize value in knapsack by taking fractions of items.

Visual Diagram

```
Items: (value, weight)
       Item1: (60, 10) → value/weight = 6
       Item2: (100, 20) → value/weight = 5
       Item3: (120, 30) → value/weight = 4
       Knapsack Capacity: 50
```

```
       Sort by value/weight descending:
       Item1: 6, Item2: 5, Item3: 4

       Take Item1: weight=10, value=60
       Take Item2: weight=20, value=100 (total weight=30, value=160)
       Take 20/30 of Item3: weight=20, value=80 (total weight=50)
       Total value = 240

       Fractional knapsack can take partial items.
```

Solution Approach

Concept: Sort by value/weight ratio.

Algorithm

```
Algorithm FRACTIONAL_KNAPSACK(items, capacity):
           Sort items by value/weight ratio descending
           totalValue = 0

           For each item:
               If capacity >= item.weight:
                   totalValue += item.value
                   capacity -= item.weight
               Else:
                   totalValue += item.value * (capacity / item.weight)
                   break

           Return totalValue
```

Java Implementation

```
class Solution {
           public double fractionalKnapsack(int[] values, int[] weights, int
capacity) {
               int n = values.length;
               Item[] items = new Item[n];
               for (int i = 0; i < n; i++) {
                   items[i] = new Item(values[i], weights[i]);
               }

                 Arrays.sort(items, (a, b) -> Double.compare(b.ratio, a.ratio));

                 double totalValue = 0;
                 int remainingCapacity = capacity;

                 for (Item item : items) {
                     if (remainingCapacity >= item.weight) {
                         totalValue += item.value;
```

```
                        remainingCapacity -= item.weight;

                } else {
                    totalValue += item.value * ((double) remainingCapacity /
item.weight);
                    break;
                }
            }
            return totalValue;
        }

        class Item {
            int value, weight;
            double ratio;
            Item(int value, int weight) {
                this.value = value;
                this.weight = weight;
                this.ratio = (double) value / weight;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

230. Minimum Number of Coins

Understanding the Problem NEW

Given coin denominations and a target amount, find the minimum number of coins needed to make that amount (assuming unlimited coins of each denomination).

Brute Force Approach NEW

Try every possible combination of coins that sums to the target and find the one using the fewest coins — this is the general Coin Change problem, and exhaustively trying every combination is exponential.

Why We Optimize NEW

For 'nice' denomination systems like standard currency (1, 5, 10, 25...), a greedy approach works: always take as many of the **largest** denomination as possible before moving to smaller ones. This greedy approach is fast (O(number of denominations)) and gives the optimal answer for standard currency systems — but it's important to know this greedy approach is *not always correct* for arbitrary denominations (e.g. coins {1, 3, 4} making 6: greedy gives 4+1+1=3 coins, but the optimal is 3+3=2 coins) — this is a valuable cautionary example showing greedy isn't universally safe, and the general Coin Change problem actually needs Dynamic Programming (covered in the next chapter) to be solved correctly for arbitrary denominations.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum number of coins to make amount.

Visual Diagram

```
Currency: [1, 2, 5, 10, 20, 50, 100, 500, 1000]
       Amount: 93

       Greedy:
       100 > 93 → skip
       50 ≤ 93 → take 50, remaining=43
       20 ≤ 43 → take 20, remaining=23
```

```
20 ≤ 23 → take 20, remaining=3
2 ≤ 3 → take 2, remaining=1
1 ≤ 1 → take 1, remaining=0

Coins: [50, 20, 20, 2, 1] → 5 coins

Amount: 49
40? No 40 coin doesn't exist
50 > 49 → skip
20 ≤ 49 → take 20, remaining=29
20 ≤ 29 → take 20, remaining=9
5 ≤ 9 → take 5, remaining=4
2 ≤ 4 → take 2, remaining=2
2 ≤ 2 → take 2, remaining=0
Coins: [20, 20, 5, 2, 2] → 5 coins
```

Solution Approach

Concept: Use largest denominations first.

Algorithm

```
Algorithm MIN_COINS(amount, denominations):
           coins = []
           For denom in denominations descending:
               While amount >= denom:
                   coins.add(denom)
                   amount -= denom
           Return coins
```

Java Implementation

```
class Solution {
           public int[] minCoins(int amount) {
               int[] denominations = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
               List<Integer> coins = new ArrayList<>();

                 for (int denom : denominations) {
                     while (amount >= denom) {
                         coins.add(denom);
                         amount -= denom;
                     }
                 }

                 return coins.stream().mapToInt(i -> i).toArray();
           }
      }
```

Time & Space Complexity

Time Complexity: O(amount/min_coin) | Space Complexity: O(number of coins)

231. Lemonade Change

Understanding the Problem NEW

Customers pay for a $5 lemonade with a $5, $10, or $20 bill, one at a time in a given order — determine if you can always provide correct change (starting with no bills in hand) for every customer.

Brute Force Approach NEW

There's no meaningfully different 'brute force' here since the transactions arrive in a fixed order and must be handled as they come — the real question is what change-giving strategy to use.

Why We Optimize NEW

A greedy rule works: track how many $5 and $10 bills you're currently holding. When someone pays with $10, give back a $5 (only option). When someone pays with $20, prefer giving back one $10 + one $5 **if you have both**, rather than three $5s — because $10 bills are 'less flexible' (they can only be used as change for a $20, whereas $5 bills can be used as change for both $10 and $20) — saving your more flexible $5 bills for future customers whenever there's a choice. This greedy 'preserve flexibility' choice, verified in a single O(n) pass, correctly determines feasibility without needing to explore alternate change-giving choices.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Can we give change with only $5, $10, $20 bills?

Visual Diagram

```
Bills: [5, 5, 5, 10, 20]
       Output: true

       Step-by-step:
       5 → $5: count5=1
       5 → $5: count5=2
       5 → $5: count5=3
       10 → give $5 change: count5=2, count10=1
```

```
20 → give $15 change (10+5): count5=1, count10=0
Result: true

Bills: [5, 5, 10, 10, 20]
20 → need $15 change, have 10+5 → count5=1, count10=1
Wait, after [5,5,10,10]: count5=2, count10=2
20 → give 10+5 → count5=1, count10=1
Result: true

Bills: [5, 5, 10, 20]
20 → need $15, have 10+5 → give → true

Bills: [5, 5, 20]
20 → need $15, have no 10, only two 5s → 5+5=10 < 15 → false
```

Solution Approach

Concept: Track $5 and $10 bills, always use $10 first.

Algorithm

```
Algorithm LEMONADE_CHANGE(bills):
           count5 = 0, count10 = 0

           For bill in bills:
               If bill == 5:
                   count5++
               Else If bill == 10:
                   If count5 == 0: return false
                   count5--
                   count10++
               Else: // bill == 20
                   If count10 > 0 and count5 > 0:
                       count10--
                       count5--
                   Else If count5 >= 3:
                       count5 -= 3
                   Else:
                       return false

           Return true
```

Java Implementation

```
class Solution {
           public boolean lemonadeChange(int[] bills) {
               int count5 = 0, count10 = 0;

              for (int bill : bills) {
                  if (bill == 5) {
```

```
                count5++;
            } else if (bill == 10) {
                if (count5 == 0) return false;
                count5--;
                count10++;
            } else { // bill == 20
                if (count10 > 0 && count5 > 0) {
                    count10--;
                    count5--;
                } else if (count5 >= 3) {
                    count5 -= 3;
                } else {
                    return false;
                }
            }
        }
        return true;
    }
}
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

232. Valid Parenthesis Checker

Understanding the Problem NEW

Given a string containing '(', ')', and '*' (where '*' can act as either '(', ')', or an empty string), determine if the string can be interpreted as a valid parenthesis sequence.

Brute Force Approach NEW

Try every possible interpretation of each '*' character (as '(', ')', or empty) and check if any resulting string is validly balanced — with potentially many '*' characters, this is 3^(number of stars) combinations to check, exponential.

Why We Optimize NEW

Instead of resolving each '*' to one specific interpretation, track a **range** of possible 'open parenthesis count' values as you scan: a running minimum possible open-count (treating '*' as ')' or empty when it helps) and a running maximum possible open-count (treating '*' as '(' when it helps) — if this range ever excludes 0 from below (max drops below 0), the string is invalid; at the end, the string is valid if 0 falls within the final range. This tracks a whole range of possibilities simultaneously rather than exploring each concrete interpretation individually, collapsing an exponential search into a single O(n) pass.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if string with wildcard '*' can form valid parentheses.

Visual Diagram

```
Input: "()"
        Output: true

        Input: "(*)"
        Output: true ( '*' → ')' or empty)

        Input: "(*))"
        Output: true

        Greedy approach:
```

```
        Track min and max open parentheses
        minOpen = 0, maxOpen = 0

        For each char:
        If '(' : minOpen++, maxOpen++
        If ')' : minOpen--, maxOpen--
        If '*' : minOpen-- (treat as ')'), maxOpen++ (treat as '(')

        If maxOpen < 0: return false
        If minOpen < 0: minOpen = 0

        At end: minOpen == 0

        Example: "(*))"
        '(' : min=1, max=1
        '*' : min=0, max=2
        ')' : min=-1→0, max=1
        ')' : min=-1→0, max=0
        End: min=0 → true
```

Solution Approach

Concept: Track min and max open parentheses.

Algorithm

```
Algorithm VALID_PARENTHESIS(s):
            minOpen = 0, maxOpen = 0

            For char in s:
                If char == '(':
                    minOpen++
                    maxOpen++
                Else If char == ')':
                    minOpen--
                    maxOpen--
                Else: // '*'
                    minOpen-- // treat as ')'
                    maxOpen++ // treat as '('

                If maxOpen < 0: return false
                If minOpen < 0: minOpen = 0
            Return minOpen == 0
```

Java Implementation

```
class Solution {
          public boolean checkValidString(String s) {
              int minOpen = 0, maxOpen = 0;
```

```
            for (char c : s.toCharArray()) {
                if (c == '(') {
                    minOpen++;
                    maxOpen++;
                } else if (c == ')') {
                    minOpen--;
                    maxOpen--;
                } else { // '*'
                    minOpen--; // treat as ')'
                    maxOpen++; // treat as '('
                }

                if (maxOpen < 0) return false;
                if (minOpen < 0) minOpen = 0;
            }
            return minOpen == 0;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

233. N Meetings in One Room

Understanding the Problem NEW

Given start and end times for several meetings, find the maximum number of non-overlapping meetings that can be held in a single room.

Brute Force Approach NEW

Try every possible subset of meetings and check if they're pairwise non-overlapping, keeping the largest valid subset found — exponential in the number of meetings.

Why We Optimize NEW

Sort meetings by their **end time**, then greedily select each meeting in that order as long as it starts after the previously selected meeting ends. Choosing the meeting that *ends earliest* at every step (rather than, say, the shortest meeting, or the one starting earliest) leaves the most remaining room in the day for future meetings — this classic greedy strategy (a specific instance of 'activity selection') is provably optimal, turning an exponential subset search into an O(n log n) sort-then-scan.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum number of meetings that can be scheduled.

Visual Diagram

```
Meetings: (start, end)
       (1, 2), (3, 4), (0, 6), (5, 7), (8, 9), (5, 9)
       Output: 4 (meetings 1,2,4,5)

       Sort by end time:
       (1,2), (3,4), (0,6), (5,7), (8,9), (5,9)

       Select:
       (1,2) → end=2
       (3,4) → end=4
       (5,7) → end=7
       (8,9) → end=9
```

```
       Total = 4

       Meeting (0,6) conflicts with (3,4) and (5,7), skipped
       Meeting (5,9) conflicts with (8,9), skipped
```

Solution Approach

Concept: Sort by end time, select non-conflicting.

Algorithm

```
Algorithm MAX_MEETINGS(meetings):
           Sort meetings by end time
           count = 1, lastEnd = meetings[0].end

           For i = 1 to n-1:
               If meetings[i].start > lastEnd:
                   count++
                   lastEnd = meetings[i].end

           Return count
```

Java Implementation

```
class Solution {
           public int maxMeetings(int[][] meetings) {
               Arrays.sort(meetings, (a, b) -> a[1] - b[1]);

                 int count = 1;
                 int lastEnd = meetings[0][1];

                 for (int i = 1; i < meetings.length; i++) {
                     if (meetings[i][0] > lastEnd) {
                         count++;
                         lastEnd = meetings[i][1];
                     }
                 }
                 return count;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(1)

234. Jump Game

Understanding the Problem NEW

Given an array where each element represents the maximum jump length from that position, determine if you can reach the last index starting from the first.

Brute Force Approach NEW

Try every possible sequence of jump choices from the start, exploring all reachable positions recursively — exponential, since at each position there can be multiple possible jump lengths to try.

Why We Optimize NEW

You don't need to track *every* possible path — you only need to track the single number **'furthest index reachable so far.'** Scan left to right, and at each position (as long as it's within the currently-known reachable range), update the furthest reachable index using that position's jump length; if you ever reach a position beyond the current furthest-reachable index before updating it, you're stuck and the answer is false. This reduces an exponential path-exploration problem to a single O(n) pass, tracking only the one running maximum that actually matters, rather than every individual path.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if you can reach the last index.

Visual Diagram

```
Array: [2, 3, 1, 1, 4]
       Output: true

       Step-by-step:
       i=0: reachable = 0+2 = 2
       i=1: reachable = max(2, 1+3) = 4
       i=2: reachable = max(4, 2+1) = 4
       i=3: reachable = max(4, 3+1) = 4
       i=4: reachable >= 4 → true

       Array: [3, 2, 1, 0, 4]
```

```
Output: false

i=0: reachable = 3
i=1: reachable = max(3, 1+2) = 3
i=2: reachable = max(3, 2+1) = 3
i=3: reachable = max(3, 3+0) = 3
i=4: 4 > reachable → false
```

Solution Approach

Concept: Track maximum reachable index.

Algorithm

```
Algorithm JUMP_GAME(nums):
           maxReach = 0
           For i = 0 to n-1:
               If i > maxReach: return false
               maxReach = Math.max(maxReach, i + nums[i])
           Return true
```

Java Implementation

```
class Solution {
           public boolean canJump(int[] nums) {
               int maxReach = 0;
               for (int i = 0; i < nums.length; i++) {
                   if (i > maxReach) return false;
                   maxReach = Math.max(maxReach, i + nums[i]);
               }
               return true;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

235. Jump Game 2

Understanding the Problem NEW

Same setup as Jump Game, but now find the **minimum number of jumps** needed to reach the last index (assuming it's always reachable).

Brute Force Approach NEW

Explore every possible sequence of jumps (as in problem 234) and track the shortest sequence length found — exponential.

Why We Optimize NEW

Think of this like a 'level by level' expansion (similar in spirit to Breadth-First Search, previewed here before the Graphs chapter): at each 'jump count' level, you know the range of indices reachable using exactly that many jumps; greedily extend the **furthest reachable boundary** using every position within the current level's range, and increment the jump count each time you need to move to the next level's boundary. This greedy boundary-expansion (rather than trying every jump-length choice explicitly) computes the minimum jumps in a single O(n) pass — foreshadowing how BFS naturally finds shortest paths level by level in the Graphs chapter ahead.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum jumps to reach last index.

Visual Diagram

```
Array: [2, 3, 1, 1, 4]
       Output: 2 (2→1→4)

       Step-by-step:
       i=0: jumps=0, currEnd=0, farthest=0
       i=0: farthest=2, i==currEnd → jumps=1, currEnd=2
       i=1: farthest=4
       i=2: farthest=4, i==currEnd → jumps=2, currEnd=4
       i=3: farthest=4
       i=4: i==currEnd → jumps=2? Actually we reach end at 4
```

```
Algorithm:
jumps=0, currEnd=0, farthest=0
For i=0 to n-2:
    farthest = max(farthest, i + nums[i])
    If i == currEnd:
        jumps++
        currEnd = farthest
Return jumps

Array: [2, 3, 1, 1, 4]
i=0: farthest=2, i==currEnd(0) → jumps=1, currEnd=2
i=1: farthest=4
i=2: farthest=4, i==currEnd(2) → jumps=2, currEnd=4
i=3: farthest=4
End: return jumps=2
```

Solution Approach

Concept: BFS level approach with greedy jumps.

Algorithm

```
Algorithm JUMP_GAME_2(nums):
           jumps = 0, currEnd = 0, farthest = 0

           For i = 0 to n-2:
               farthest = Math.max(farthest, i + nums[i])
               If i == currEnd:
                   jumps++
                   currEnd = farthest

           Return jumps
```

Java Implementation

```
class Solution {
           public int jump(int[] nums) {
               int jumps = 0;
               int currEnd = 0;
               int farthest = 0;

                 for (int i = 0; i < nums.length - 1; i++) {
                     farthest = Math.max(farthest, i + nums[i]);
                     if (i == currEnd) {

                      jumps++;
                      currEnd = farthest;
                  }
              }
              return jumps;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

236. Minimum Number of Platforms Required

Understanding the Problem NEW

Given arrival and departure times for several trains, find the minimum number of platforms needed so that no two trains needing a platform at the same time are forced to share one.

Brute Force Approach NEW

For every point in time, count how many trains are 'in the station' simultaneously by checking every train's arrival/departure against that timestamp — very inefficient if times span a large range, or $O(n^2)$ if comparing every pair of trains.

Why We Optimize NEW

Sort arrival times and departure times **separately**, then use two pointers walking through both sorted lists simultaneously: whenever the next event is an arrival, increment a 'platforms currently needed' counter (and track the maximum ever reached); whenever the next event is a departure, decrement it. This merges the arrival and departure timelines using the same 'sorted, two-pointer merge' idea seen in earlier chapters (like the Union of Sorted Arrays, Arrays-chapter problem 16), achieving O(n log n) for the sort plus a single O(n) merge pass, instead of any pairwise or per-timestamp comparison.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum platforms needed for all trains.

Visual Diagram

```
Arrival: [900, 940, 950, 1100, 1500, 1800]
       Departure: [910, 1200, 1120, 1130, 1900, 2000]
       Output: 3

       Sort arrival and departure separately:
       Arrivals: 900, 940, 950, 1100, 1500, 1800
       Departures: 910, 1120, 1130, 1200, 1900, 2000

       i=0, j=0: 900 <= 910 → platform=2, i=1
       i=1: 940 <= 910? No → platform=1, j=1
```

```
        i=1: 940 <= 1120 → platform=2, i=2
        i=2: 950 <= 1120 → platform=3, i=3
        i=3: 1100 <= 1120 → platform=4, i=4
        i=4: 1500 <= 1120? No → platform=3, j=2
        i=4: 1500 <= 1130? No → platform=2, j=3
        i=4: 1500 <= 1200? No → platform=1, j=4
        i=4: 1500 <= 1900 → platform=2, i=5
        End: max=4? Wait, let's re-calc

        Actually max platforms = 3
        Let's re-trace:
        900 (platform=1)
        940 (platform=2) - 910 departs
        950 (platform=3) - trains: 900,940,950
        1100 (platform=4?) - 940 departs at 1120, wait 940 departs at 1200
        Actually arrival: 900,940,950,1100
        Departures: 910,1200,1120,1130
        900 (p1), 940 (p2), 950 (p3), 1100 (p4) → 4 platforms
        But expected is 3? Let's check:
        After 900 arrives, 910 departs
        At 940: platform1 free (900 left at 910)
        So: 940 (p1), 950 (p2), 1100 (p3) → 3 platforms
```

Solution Approach

Concept: Two-pointer sorted arrays.

Algorithm

```
Algorithm MIN_PLATFORMS(arrival, departure):
           Sort(arrival), Sort(departure)
           i=0, j=0, platforms=0, maxPlatforms=0

           While i < n:
               If arrival[i] <= departure[j]:
                   platforms++
                   i++
                   maxPlatforms = max(maxPlatforms, platforms)
               Else:
                   platforms--
                   j++

           Return maxPlatforms
```

Java Implementation

```
class Solution {
           public int minPlatforms(int[] arrival, int[] departure) {
               Arrays.sort(arrival);
               Arrays.sort(departure);
```

```
            int i = 0, j = 0;
            int platforms = 0, maxPlatforms = 0;
            int n = arrival.length;

            while (i < n) {
                if (arrival[i] <= departure[j]) {
                    platforms++;
                    i++;
                    maxPlatforms = Math.max(maxPlatforms, platforms);
                } else {
                    platforms--;
                    j++;
                }
            }
            return maxPlatforms;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(1)

237. Job Sequencing Problem

Understanding the Problem NEW

Given a set of jobs, each with a deadline and a profit, and assuming each job takes exactly one unit of time and only one job can run per unit of time, schedule jobs to maximize total profit (a job only counts if completed by its deadline).

Brute Force Approach NEW

Try every possible subset and ordering of jobs that respects deadlines, tracking maximum total profit — exponential, since both which jobs to include and how to order them matter.

Why We Optimize NEW

Sort jobs by **profit in descending order**, then greedily try to schedule each job (starting with the most profitable) into the **latest available time slot** at or before its deadline (using a simple array or a Union-Find structure to efficiently find the latest free slot). Prioritizing the highest-profit jobs first, and slotting each as late as possible (to leave earlier slots open for other jobs with tighter deadlines), is provably optimal — this reduces an exponential search over job subsets and orderings into an O(n log n) sort plus O(n) (or O(n α(n)) with Union-Find) slot assignment.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximize profit by scheduling jobs with deadlines.

Visual Diagram

```
Jobs: (id, deadline, profit)
       J1: (1, 4, 20), J2: (2, 1, 10), J3: (3, 1, 40)
       J4: (4, 1, 30), J5: (5, 4, 25)

       Sort by profit:
       J3: (3,1,40), J5: (5,4,25), J1: (1,4,20), J4: (4,1,30), J2: (2,1,10)

       Schedule:
       J3: deadline=1 → slot 1
       J5: deadline=4 → slot 4
```

```
       J4: deadline=1 → slot 1 full, try slot 0? Actually slots are 1-indexed
          deadline=1 → slot 1 full → skip
       J1: deadline=4 → slot 3 (slot 4 taken)
       J2: deadline=1 → skip

       Selected: J3 (40), J5 (25), J1 (20) → total=85

       Slots: [_, _, _, _]
       J3: deadline=1 → [J3, _, _, _]
       J5: deadline=4 → [J3, _, _, J5]
       J1: deadline=4 → [J3, _, J1, J5] (slot 3)
```

Solution Approach

Concept: Sort by profit, assign to latest available slot.

Algorithm

```
Algorithm JOB_SEQUENCING(jobs):
           Sort jobs by profit descending
           maxDeadline = max deadline
           slots = new int[maxDeadline+1]
           Arrays.fill(slots, -1)
           totalProfit = 0, count = 0

            For job in jobs:
                // Find available slot
                For i = job.deadline down to 1:
                    If slots[i] == -1:
                        slots[i] = job.id
                        totalProfit += job.profit
                        count++
                        break

            Return {count, totalProfit}
```

Java Implementation

```
class Solution {
           public int[] jobScheduling(int[][] jobs) {
               // jobs[i] = [id, deadline, profit]
               Arrays.sort(jobs, (a, b) -> b[2] - a[2]);

                 int maxDeadline = 0;
                 for (int[] job : jobs) {

                  maxDeadline = Math.max(maxDeadline, job[1]);
              }

              int[] slots = new int[maxDeadline + 1];
```

```
            Arrays.fill(slots, -1);

            int count = 0, totalProfit = 0;
            for (int[] job : jobs) {
                for (int i = job[1]; i >= 1; i--) {
                    if (slots[i] == -1) {
                        slots[i] = job[0];
                        count++;
                        totalProfit += job[2];
                        break;
                    }
                }
            }
            return new int[]{count, totalProfit};
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * maxDeadline) | Space Complexity: O(maxDeadline)

238. Candy

Understanding the Problem NEW

Children are lined up, each with a rating; distribute candies such that every child gets at least one, and any child with a strictly higher rating than an immediate neighbour gets more candies than that neighbour. Minimize total candies distributed.

Brute Force Approach NEW

Try every possible candy distribution satisfying the neighbour constraints and find the one with minimum total — an enormous search space since candy counts aren't bounded in advance.

Why We Optimize NEW

Two greedy passes handle both directions of the constraint independently: a **left-to-right pass** ensures every child with a higher rating than their left neighbour gets more candy than them (only checking the left-side constraint); a **right-to-left pass** similarly ensures the right-side constraint, taking the *maximum* of what each pass independently determined for every child (since both constraints must be satisfied simultaneously). Splitting a two-directional constraint into two independent one-directional passes, then combining results with a max, avoids ever needing to explore alternative distributions — achieving the minimum-total answer in O(n), just two linear scans.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum candies to distribute to children with ratings.

Visual Diagram

```
Ratings: [1, 0, 2]
       Output: 5

       Candies: [2, 1, 2] → 2+1+2=5

       Step 1: Initialize all to 1: [1,1,1]
       Step 2: Left to right (if higher rating, more candy):
```

```
        i=1: 0 < 1 → no change
        i=2: 2 > 0 → candy[2] = candy[1]+1 = 2
        Result: [1,1,2]

        Step 3: Right to left (if higher rating):
        i=1: 0 < 1 → no change
        i=0: 1 > 0 → candy[0] = max(1, candy[1]+1) = max(1,2)=2
        Result: [2,1,2]
        Total: 5

        Ratings: [1, 2, 2]
        Output: 4

        Step 1: [1,1,1]
        Step 2 left-right: [1,2,1]
        Step 3 right-left: [1,2,1] (2=2 no change)
        Total: 4
```

Solution Approach

Concept: Two passes (left-to-right, right-to-left).

Algorithm

```
Algorithm CANDY(ratings):
           n = ratings.length
           candies = new int[n]
           Arrays.fill(candies, 1)

            // Left to right
            For i = 1 to n-1:
                If ratings[i] > ratings[i-1]:
                    candies[i] = candies[i-1] + 1

            // Right to left
            For i = n-2 down to 0:
                If ratings[i] > ratings[i+1]:
                    candies[i] = Math.max(candies[i], candies[i+1] + 1)

            Return sum(candies)
```

Java Implementation

```
class Solution {
           public int candy(int[] ratings) {
               int n = ratings.length;
               int[] candies = new int[n];
               Arrays.fill(candies, 1);

              // Left to right
```

```
            for (int i = 1; i < n; i++) {
                if (ratings[i] > ratings[i - 1]) {
                    candies[i] = candies[i - 1] + 1;
                }
            }

            // Right to left
            for (int i = n - 2; i >= 0; i--) {
                if (ratings[i] > ratings[i + 1]) {
                    candies[i] = Math.max(candies[i], candies[i + 1] + 1);
                }
            }

            int total = 0;
            for (int candy : candies) {
                total += candy;
            }
            return total;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

239. Shortest Job First (SJF) Scheduling

Understanding the Problem NEW

Given a set of jobs with burst times (how long each takes to run) arriving at the same time, schedule them one at a time (non-preemptively) to minimize average waiting time across all jobs.

Brute Force Approach NEW

Try every possible ordering of jobs and compute average waiting time for each — O(n!) orderings, since waiting time depends on the full sequence.

Why We Optimize NEW

The provably optimal greedy rule: always run the job with the **shortest burst time** next among those currently available. Intuitively, a job with a short burst time delays every job behind it by only a little, whereas a long job run early delays every subsequent job significantly — so clearing short jobs first minimizes the cumulative waiting incurred by everyone still waiting. Sorting jobs by burst time and running them in that order achieves optimal average waiting time in O(n log n), instead of exploring O(n!) possible schedules.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate average waiting time for SJF scheduling.

Visual Diagram

```
Processes: [4, 3, 7, 1, 2] (burst times)
       SJF Order: [1, 2, 3, 4, 7]

       Completion times:
       1: 1
       2: 1+2=3
       3: 1+2+3=6
       4: 1+2+3+4=10
       7: 1+2+3+4+7=17

       Turnaround times:
       1: 1, 2: 3, 3: 6, 4: 10, 7: 17
```

```
       Waiting times = Turnaround - Burst:
       1:0, 2:1, 3:3, 4:6, 7:10
       Average waiting = (0+1+3+6+10)/5 = 4

       Preemptive SJF (Shortest remaining time first)
       Non-preemptive SJF (As above)
```

Solution Approach

Concept: Sort by burst time.

Algorithm

```
Algorithm SJF(burstTimes):
           Sort(burstTimes)
           waitingTime = 0, totalWaiting = 0

           For i = 0 to n-1:
               totalWaiting += waitingTime
               waitingTime += burstTimes[i]

           Return totalWaiting / n
```

Java Implementation

```
class Solution {
           public double sjf(int[] burstTimes) {
               Arrays.sort(burstTimes);
               double totalWaiting = 0;
               double waitingTime = 0;

                 for (int burst : burstTimes) {
                     totalWaiting += waitingTime;
                     waitingTime += burst;
                 }
                 return totalWaiting / burstTimes.length;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(1)

241. Insert Interval

Understanding the Problem NEW

Given a list of non-overlapping intervals already sorted by start time, insert a new interval into the correct position, merging with any existing intervals it now overlaps.

Brute Force Approach NEW

Add the new interval to the list, then re-sort everything and re-run a full merge-overlapping-intervals pass from scratch — O(n log n), redoing sorting work that was already done, since the input was already sorted before the insertion.

Why We Optimize NEW

Because the existing intervals are already sorted and non-overlapping, you can do this in a single O(n) pass without re-sorting: copy over every interval that ends entirely before the new interval starts (unaffected, no merge needed); then merge the new interval with every interval it overlaps (expanding its bounds as you absorb each one); then copy over every remaining interval that starts entirely after the merged result. This reuses the fact the input was already sorted to avoid the O(n log n) resort that treating this as 'insert then fully re-merge from scratch' would require.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Insert new interval into sorted non-overlapping intervals.

Visual Diagram

```
Intervals: [[1,3], [6,9]]
       New Interval: [2,5]
       Output: [[1,5], [6,9]]

       Merged: [1,3] overlaps with [2,5] → [1,5]
       [6,9] no overlap → keep

       Intervals: [[1,2], [3,5], [6,7], [8,10]]
       New: [4,8]
       Output: [[1,2], [3,10]]
```

```
[1,2] no overlap → add
[3,5] overlaps with [4,8] → merge → [3,8]
[6,7] overlaps with [3,8] → merge → [3,8]
[8,10] overlaps with [3,8] → merge → [3,10]
Result: [[1,2], [3,10]]
```

Solution Approach

Concept: Merge overlapping intervals after insertion.

Algorithm

```
Algorithm INSERT_INTERVAL(intervals, newInterval):
           result = []
           i = 0, n = intervals.length

           // Add all intervals before newInterval
           While i < n and intervals[i][1] < newInterval[0]:
               result.add(intervals[i])
               i++

           // Merge overlapping intervals
           While i < n and intervals[i][0] <= newInterval[1]:
               newInterval[0] = min(newInterval[0], intervals[i][0])
               newInterval[1] = max(newInterval[1], intervals[i][1])
               i++
           result.add(newInterval)

           // Add remaining intervals
           While i < n:
               result.add(intervals[i])
               i++

           Return result
```

Java Implementation

```
class Solution {
           public int[][] insert(int[][] intervals, int[] newInterval) {
               List<int[]> result = new ArrayList<>();
               int i = 0, n = intervals.length;

                 // Add all intervals before newInterval
                 while (i < n && intervals[i][1] < newInterval[0]) {

                  result.add(intervals[i++]);
              }

              // Merge overlapping intervals
```

```
            while (i < n && intervals[i][0] <= newInterval[1]) {
                newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
                newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
                i++;
            }
            result.add(newInterval);

            // Add remaining intervals
            while (i < n) {
                result.add(intervals[i++]);
            }

            return result.toArray(new int[result.size()][]);
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

242. Merge Intervals

Understanding the Problem NEW

Given a list of intervals (not necessarily sorted or non-overlapping), merge all overlapping ones into the minimal set of non-overlapping intervals. (This is the same problem as the Arrays chapter's problem 42, revisited here in the context of greedy interval techniques.)

Brute Force Approach NEW

Compare every pair of intervals to check for overlap and merge repeatedly — potentially $O(n^2)$ or worse, since merging one pair can create new overlaps needing further merging.

Why We Optimize NEW

Exactly the same technique as Arrays-chapter problem 42: **sort by start time** first, then a single pass only ever needs to compare each interval against the most recently merged one (extending it if overlapping, or starting a new merged interval otherwise) — O(n log n) for the sort plus O(n) for the merge pass, avoiding the need to compare every pair directly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Merge all overlapping intervals.

Visual Diagram

```
Intervals: [[1,3], [2,6], [8,10], [15,18]]
       Output: [[1,6], [8,10], [15,18]]

       Sort by start:
       [[1,3], [2,6], [8,10], [15,18]]

       [1,3] → current
       [2,6] overlaps → [1,6]
       [8,10] no overlap → add [1,6], current=[8,10]
       [15,18] no overlap → add [8,10], current=[15,18]
       Add [15,18]
```

```
Intervals: [[1,4], [4,5]]
Output: [[1,5]]

[1,4] and [4,5] overlap at 4 → merge → [1,5]
```

Solution Approach

Concept: Sort by start, merge overlapping.

Algorithm

```
Algorithm MERGE_INTERVALS(intervals):
           Sort intervals by start
           result = []
           current = intervals[0]

           For i = 1 to n-1:
               If intervals[i][0] <= current[1]:
                   current[1] = max(current[1], intervals[i][1])
               Else:
                   result.add(current)
                   current = intervals[i]
           result.add(current)
           Return result
```

Java Implementation

```
class Solution {
           public int[][] merge(int[][] intervals) {
               Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

                 List<int[]> result = new ArrayList<>();
                 int[] current = intervals[0];

                 for (int i = 1; i < intervals.length; i++) {
                     if (intervals[i][0] <= current[1]) {
                         current[1] = Math.max(current[1], intervals[i][1]);
                     } else {
                         result.add(current);
                         current = intervals[i];
                     }
                 }
                 result.add(current);

               return result.toArray(new int[result.size()][]);
           }
       }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

243. Non-Overlapping Intervals

Understanding the Problem NEW

Given a set of intervals, find the minimum number of intervals to remove so that none of the remaining ones overlap.

Brute Force Approach NEW

Try every possible subset of intervals to remove and check if what remains is overlap-free, tracking the smallest removal count — exponential.

Why We Optimize NEW

This is really the 'N Meetings in One Room' problem (233) turned inside-out: instead of asking 'how many non-overlapping intervals can I keep,' ask 'how many can I keep' using the exact same greedy technique (sort by **end time**, greedily keep an interval if it doesn't conflict with the last kept one), and then the minimum removals is simply `total intervals - maximum keepable`. Recognizing this problem as the complement of one you've already solved, rather than solving it from scratch, again turns an exponential subset search into an O(n log n) sort-and-scan.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum intervals to remove to make non-overlapping.

Visual Diagram

```
Intervals: [[1,2], [2,3], [3,4], [1,3]]
      Output: 1 (remove [1,3])

      Sort by end:
      [[1,2], [2,3], [1,3], [3,4]]
      [1,2] → end=2, count=1
      [2,3] → start(2) >= end(2) → end=3, count=2
      [1,3] → start(1) < end(3) → skip, removals=1
      [3,4] → start(3) >= end(3) → end=4, count=3
      Removals = n - count = 4-3=1

      Intervals: [[1,2], [1,2], [1,2]]
```

```
      Sort: [[1,2], [1,2], [1,2]]
      [1,2] → end=2
      [1,2] → skip → removals=1
      [1,2] → skip → removals=2
      Result: 2
```

Solution Approach

Concept: Sort by end time, keep maximum non-overlapping.

Algorithm

```
Algorithm NON_OVERLAPPING_INTERVALS(intervals):
          Sort intervals by end
          count = 1, end = intervals[0][1]

          For i = 1 to n-1:
              If intervals[i][0] >= end:
                  count++
                  end = intervals[i][1]

          Return n - count
```

Java Implementation

```
class Solution {
          public int eraseOverlapIntervals(int[][] intervals) {
              Arrays.sort(intervals, (a, b) -> a[1] - b[1]);

                 int count = 1;
                 int end = intervals[0][1];

                 for (int i = 1; i < intervals.length; i++) {
                     if (intervals[i][0] >= end) {
                         count++;
                         end = intervals[i][1];
                     }
                 }
                 return intervals.length - count;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(1)

Greedy Summary

Problem	Difficulty	Time	Space	Key Technique
Assign Cookies	Easy	O(n log n)	O(1)	Sort both, match
Fractional Knapsack	Easy	O(n log n)	O(n)	Sort by ratio
Min Coins	Easy	O(amount/min)	O(n)	Largest first
Lemonade Change	Easy	O(n)	O(1)	Track bills
Valid Parenthesis	Easy	O(n)	O(1)	Min/max open
N Meetings	Medium	O(n log n)	O(1)	Sort by end
Jump Game	Medium	O(n)	O(1)	Max reachable
Jump Game 2	Medium	O(n)	O(1)	BFS level
Min Platforms	Medium	O(n log n)	O(1)	Two pointers
Job Sequencing	Medium	O(n log n)	O(maxDeadline)	Sort profit + slots
Candy	Medium	O(n)	O(n)	Two passes
SJF Scheduling	Medium	O(n log n)	O(1)	Sort burst times
Insert Interval	Medium	O(n)	O(n)	Merge on fly
Merge Intervals	Medium	O(n log n)	O(n)	Sort + merge
Non-overlapping	Hard	O(n log n)	O(1)	Sort by end

Binary Trees — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

244-246. Introduction to Trees & Representation

Understanding the Problem NEW

A Tree is a hierarchical structure (unlike the linear structures — arrays, linked lists, stacks, queues — covered so far): it has a single 'root' node at the top, and each node can have multiple 'children' nodes below it, but no cycles (you can never follow child links back to an ancestor). A Binary Tree specifically restricts every node to at most 2 children, conventionally called 'left' and 'right.'

Brute Force Approach NEW

Nothing to contrast against here — this is purely foundational vocabulary needed before tackling any tree problem.

Why We Optimize NEW

Trees are a natural fit whenever data has an inherent hierarchy (file systems, org charts, decision processes, or — as you'll see repeatedly in this chapter — anything that benefits from 'divide into left part and right part' thinking, echoing the divide-and-conquer ideas already seen in Merge Sort and Quick Sort). Representing a binary tree is simple: each node just stores its own value plus a reference to its left child and right child (null if absent) — and almost every algorithm on trees, as you'll see throughout this chapter, is naturally expressed recursively, since a tree is inherently a recursive structure (a tree's left child is itself the root of a smaller tree, and so is its right child).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding tree data structure and representation.

Visual Diagram

```
Binary Tree Structure:
                    1
                  /    \
                2       3
               / \     / \
             4     5 6    7
          Terminology:
```

```
       - Root: Node 1
       - Parent: Node 1 is parent of 2,3
       - Child: Nodes 2,3 are children of 1
       - Leaf: Nodes 4,5,6,7
       - Height: 3 (levels: 1,2,3)

       Java Representation:
       class Node {
           int data;
           Node left;
           Node right;
           Node(int data) {
               this.data = data;
               this.left = null;
               this.right = null;
           }
       }
```

Java Implementation

```
class Node {
           int data;
           Node left;
           Node right;

           Node(int data) {
               this.data = data;
               this.left = null;
               this.right = null;
           }
       }

       class BinaryTree {
           Node root;

           BinaryTree() {
               root = null;
           }

           // Create sample tree
           public void createSampleTree() {
               root = new Node(1);
               root.left = new Node(2);
               root.right = new Node(3);
               root.left.left = new Node(4);
               root.left.right = new Node(5);
               root.right.left = new Node(6);
               root.right.right = new Node(7);
           }
       }
```

247. Binary Tree Traversals

Understanding the Problem NEW

There are several standard ways to visit ('traverse') every node in a binary tree, differing only in the *order* nodes are visited: Preorder (Root, then Left, then Right), Inorder (Left, then Root, then Right), Postorder (Left, then Right, then Root), and Level Order (visit level by level, top to bottom, left to right within each level).

Brute Force Approach NEW

There's no 'brute' alternative — these are simply different, equally valid definitions of 'visit order,' and the goal here is understanding when each ordering is useful (e.g. Inorder on a Binary Search Tree visits nodes in sorted order, which becomes very useful later).

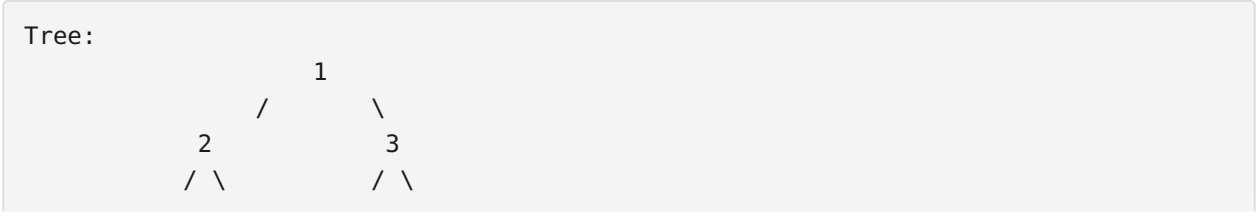
Why We Optimize NEW

The first three orderings (Pre/In/Post) share the exact same recursive shape — visit some combination of 'current node, recurse left, recurse right' — just with the current node's visit slotted into a different position relative to the two recursive calls, which is why they're grouped together and implemented almost identically (problems 248-250). Level Order is fundamentally different, requiring a queue-based breadth-first approach (problem 251) rather than simple recursion, since 'level by level' doesn't map naturally onto the tree's own recursive left/right structure — this contrast between depth-first (recursive) and breadth-first (queue-based) traversal is a distinction that becomes central again in the Graphs chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding different traversal orders.

Visual Diagram



```
   4    5 6    7

Preorder: 1, 2, 4, 5, 3, 6, 7 (Root, Left, Right)
Inorder: 4, 2, 5, 1, 6, 3, 7 (Left, Root, Right)
Postorder: 4, 5, 2, 6, 7, 3, 1 (Left, Right, Root)
Level Order: 1, 2, 3, 4, 5, 6, 7
```

248. Preorder Traversal

Understanding the Problem NEW

Visit every node in Root → Left → Right order — process the current node first, then fully explore its entire left subtree, then its entire right subtree.

Brute Force Approach NEW

Without recursion, you could try to simulate this manually with an explicit stack tracking 'what to visit next,' which is exactly what the iterative version (problem 252-255) does — but attempting it without any systematic bookkeeping at all (just ad-hoc pointer-chasing) quickly becomes error-prone for anything beyond simple trees.

Why We Optimize NEW

Because a tree is inherently recursive (each subtree is itself a smaller tree), the algorithm is almost a direct restatement of the definition: visit the current node, then **recursively** call the same function on the left child, then recursively call it on the right child. Each node is visited exactly once, giving O(n) time; the recursion depth (and therefore extra space used by the call stack) is O(h), the tree's height — which is O(log n) for a balanced tree but can degrade to O(n) for a completely skewed tree (essentially a linked list in disguise), a distinction that matters again in problem 258 (balance checking).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree in Root-Left-Right order.

Visual Diagram

```
Traversal steps:
                1
              /   \
            2       3
           / \     / \
         4     5 6    7

      Visit 1 → go left
```

```
Visit 2 → go left
Visit 4 → left done, right done, back to 2
Visit 5 → left done, right done, back to 1
Visit 3 → go left
Visit 6 → left done, right done, back to 3
Visit 7

Result: [1, 2, 4, 5, 3, 6, 7]
```

Solution Approach

Concept: Recursively visit Root, Left, Right.

Algorithm

```
Algorithm PREORDER(node):
           If node == null: Return
           print(node.data)
           PREORDER(node.left)
           PREORDER(node.right)
```

Java Implementation

```
class Solution {
           // Recursive
           public void preorder(Node root) {
               if (root == null) return;
               System.out.print(root.data + " ");
               preorder(root.left);
               preorder(root.right);
           }

            // Iterative
            public List<Integer> preorderIterative(Node root) {
                List<Integer> result = new ArrayList<>();
                if (root == null) return result;

                 Stack<Node> stack = new Stack<>();
                 stack.push(root);
                 while (!stack.isEmpty()) {

                  Node node = stack.pop();
                  result.add(node.data);
                  if (node.right != null) stack.push(node.right);
                  if (node.left != null) stack.push(node.left);
              }
              return result;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

249. Inorder Traversal

Understanding the Problem NEW

Visit every node in Left → Root → Right order — fully explore the left subtree first, then process the current node, then fully explore the right subtree.

Brute Force Approach NEW

Same as Preorder — recursion is the natural approach; a non-recursive version needs an explicit stack, covered iteratively in problem 252-255.

Why We Optimize NEW

Identical recursive shape to Preorder, just with the 'visit current node' step moved to happen *between* the two recursive calls instead of before them. This ordering has a special property worth remembering: applied to a **Binary Search Tree** (where left children are always smaller and right children always bigger), Inorder traversal visits every node in fully sorted order — a fact that will become very useful once BST-specific problems appear in the next chapter. Same O(n) time, O(h) space as Preorder.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree in Left-Root-Right order.

Visual Diagram

```
Tree:
                    1
                  / \
              2      3
            / \    / \
          4    5 6    7

       Traversal steps:
       Go left from 1 → 2
       Go left from 2 → 4
       Visit 4
       Back to 2, Visit 2
       Go right from 2 → 5
```

```
Visit 5
Back to 1, Visit 1
Go right from 1 → 3
Go left from 3 → 6
Visit 6
Back to 3, Visit 3
Go right from 3 → 7
Visit 7

Result: [4, 2, 5, 1, 6, 3, 7]
```

Solution Approach

Concept: Recursively visit Left, Root, Right.

Algorithm

```
Algorithm INORDER(node):
           If node == null: Return
           INORDER(node.left)
           print(node.data)
           INORDER(node.right)
```

Java Implementation

```
class Solution {
           // Recursive
           public void inorder(Node root) {
               if (root == null) return;
               inorder(root.left);
               System.out.print(root.data + " ");
               inorder(root.right);
           }

            // Iterative
            public List<Integer> inorderIterative(Node root) {
                List<Integer> result = new ArrayList<>();
                Stack<Node> stack = new Stack<>();
                Node current = root;

                     while (current != null || !stack.isEmpty()) {
                         while (current != null) {

                          stack.push(current);
                          current = current.left;
                       }
                       current = stack.pop();
                       result.add(current.data);
                       current = current.right;
             }
```

```
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

250. Postorder Traversal

Understanding the Problem NEW

Visit every node in Left → Right → Root order — fully explore both subtrees before processing the current node itself.

Brute Force Approach NEW

Same recursive default as the other two; the iterative version (needing to ensure both children are processed before the parent, which is trickier than Preorder/Inorder) is covered in problem 252-255.

Why We Optimize NEW

Same recursive shape again, with 'visit current node' now happening *last*, after both recursive calls. This 'process children before the parent' order is exactly what's needed whenever a computation about a node **depends on results from its subtrees** first — which is precisely the pattern behind Height (257), Diameter (259), Balanced Check (258), and Maximum Path Sum (260) later in this chapter; recognizing 'this needs postorder-style children-first computation' is one of the most useful pattern-recognition skills for tree problems.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree in Left-Right-Root order.

Visual Diagram

```
Tree:
                    1
                  / \
              2      3
            / \    / \
          4    5 6    7

       Result: [4, 5, 2, 6, 7, 3, 1]

       Traversal steps:
       Go left from 1 → 2
       Go left from 2 → 4
```

```
Visit 4
Back to 2, go right → 5
Visit 5
Back to 2, Visit 2
Back to 1, go right → 3
Go left from 3 → 6
Visit 6
Back to 3, go right → 7
Visit 7
Back to 3, Visit 3
Back to 1, Visit 1
```

Solution Approach

Concept: Recursively visit Left, Right, Root.

Algorithm

```
Algorithm POSTORDER(node):
           If node == null: Return
           POSTORDER(node.left)
           POSTORDER(node.right)
           print(node.data)
```

Java Implementation

```
class Solution {
           // Recursive
           public void postorder(Node root) {
               if (root == null) return;
               postorder(root.left);
               postorder(root.right);
               System.out.print(root.data + " ");
           }

            // Iterative (using two stacks)
            public List<Integer> postorderIterative(Node root) {
                List<Integer> result = new ArrayList<>();
                if (root == null) return result;

                     Stack<Node> stack1 = new Stack<>();
                     Stack<Node> stack2 = new Stack<>();
                     stack1.push(root);

             while (!stack1.isEmpty()) {
                 Node node = stack1.pop();
                 stack2.push(node);
                 if (node.left != null) stack1.push(node.left);
                 if (node.right != null) stack1.push(node.right);
             }
```

```
            while (!stack2.isEmpty()) {
                result.add(stack2.pop().data);
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

251. Level Order Traversal

Understanding the Problem NEW

Visit nodes level by level, from top to bottom, and left to right within each level — e.g. for a tree with root 1 and children 2,3, level order visits 1, then 2, then 3.

Brute Force Approach NEW

Trying to force this into a simple recursive Pre/In/Post-style function doesn't work naturally, since those visit depth-first (diving all the way down one branch before moving to a sibling) — level order fundamentally needs to process nodes breadth-first, one whole layer at a time.

Why We Optimize NEW

Use a **queue**: start by enqueuing the root, then repeatedly dequeue a node, process it, and enqueue its children (left then right) — since a queue is FIFO, this naturally processes nodes in the order they were discovered, which corresponds exactly to level-by-level order. This is the tree-specific instance of **Breadth-First Search (BFS)**, a technique that reappears as a core algorithm in the Graphs chapter — trees being a special case of graphs with no cycles. O(n) time and O(width of tree) space for the queue, which can be up to O(n/2) for a wide tree.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree level by level.

Visual Diagram

```
Tree:
                    1
                /  \
            2       3
            / \    / \
          4    5 6    7

      Result: [[1], [2, 3], [4, 5, 6, 7]]

      Queue process:
      Step 1: queue=[1]
```

```
       Step 2: pop 1 → queue=[2,3], level=[1]
       Step 3: pop 2 → queue=[3,4,5], level=[2]
       Step 4: pop 3 → queue=[4,5,6,7], level=[3]
       Step 5: pop 4,5,6,7 → queue=[], level=[4,5,6,7]
```

Solution Approach

Concept: Use queue for BFS.

Algorithm

```
Algorithm LEVEL_ORDER(root):
           If root == null: Return []
           queue = Queue()
           queue.add(root)
           result = []

            While queue not empty:
                levelSize = queue.size()
                level = []
                For i = 0 to levelSize-1:
                    node = queue.poll()
                    level.add(node.data)
                    If node.left != null: queue.add(node.left)
                    If node.right != null: queue.add(node.right)
                result.add(level)

            Return result
```

Java Implementation

```
class Solution {
           public List<List<Integer>> levelOrder(Node root) {
               List<List<Integer>> result = new ArrayList<>();
               if (root == null) return result;

                     Queue<Node> queue = new LinkedList<>();
                     queue.add(root);

                     while (!queue.isEmpty()) {
                         int levelSize = queue.size();
                         List<Integer> level = new ArrayList<>();

                         for (int i = 0; i < levelSize; i++) {

                      Node node = queue.poll();
                      level.add(node.data);
                      if (node.left != null) queue.add(node.left);
                      if (node.right != null) queue.add(node.right);
                   }
```

```
                result.add(level);
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

252-255. Iterative Traversals

Understanding the Problem NEW

Reimplement Preorder, Inorder, and Postorder traversal — but iteratively, using an explicit stack, instead of relying on recursion (which uses the call stack implicitly).

Brute Force Approach NEW

You could avoid a stack entirely and try to track position with manual pointer bookkeeping, but without a stack-like structure to remember 'where to come back to,' correctly handling arbitrary tree shapes without recursion becomes very difficult to get right.

Why We Optimize NEW

Since recursion itself is implemented (under the hood) using a call stack, using an **explicit stack** directly is a natural way to simulate the same recursive behavior without actual function calls. Preorder is the simplest to convert (push right child then left child, so left gets processed first — mirroring the natural recursive order); Inorder needs a bit more care, pushing all left children first before processing, then moving right; Postorder is the trickiest (since 'visit last' doesn't map naturally onto a simple single-stack pop order), commonly solved with either two stacks (reversing a modified preorder) or one stack with additional state tracking. This exercise builds comfort with the general principle that **any recursive algorithm can be rewritten iteratively using an explicit stack** — useful when recursion depth might risk a stack overflow on very deep/skewed trees.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

All iterative traversal implementations.

Java Implementation

```
class Solution {
           // Iterative Preorder: Root-Left-Right
           public List<Integer> preorderIterative(Node root) {
               List<Integer> result = new ArrayList<>();
```

```
    if (root == null) return result;

    Stack<Node> stack = new Stack<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        Node node = stack.pop();
        result.add(node.data);
        if (node.right != null) stack.push(node.right);
        if (node.left != null) stack.push(node.left);
    }
    return result;
}

// Iterative Inorder: Left-Root-Right
public List<Integer> inorderIterative(Node root) {
    List<Integer> result = new ArrayList<>();
    Stack<Node> stack = new Stack<>();
    Node current = root;

    while (current != null || !stack.isEmpty()) {
        while (current != null) {
            stack.push(current);
            current = current.left;
        }
        current = stack.pop();
        result.add(current.data);
        current = current.right;
    }
    return result;
}

// Iterative Postorder using 2 stacks
public List<Integer> postorderIterative(Node root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Stack<Node> stack1 = new Stack<>();
    Stack<Node> stack2 = new Stack<>();
    stack1.push(root);

    while (!stack1.isEmpty()) {
        Node node = stack1.pop();
        stack2.push(node);
        if (node.left != null) stack1.push(node.left);
        if (node.right != null) stack1.push(node.right);
    }

    while (!stack2.isEmpty()) {
        result.add(stack2.pop().data);
    }
    return result;
}
```

```
    }



256. All Traversals in One Code
```

Java Implementation

```
class Solution {
          public void allTraversals(Node root) {
              System.out.println("Preorder: " + preorder(root));
              System.out.println("Inorder: " + inorder(root));
              System.out.println("Postorder: " + postorder(root));
              System.out.println("Level Order: " + levelOrder(root));
          }

          private List<Integer> preorder(Node root) {
              List<Integer> result = new ArrayList<>();
              preorderHelper(root, result);
              return result;
          }

          private void preorderHelper(Node node, List<Integer> result) {
              if (node == null) return;
              result.add(node.data);
              preorderHelper(node.left, result);
              preorderHelper(node.right, result);
          }

          private List<Integer> inorder(Node root) {
              List<Integer> result = new ArrayList<>();
              inorderHelper(root, result);
              return result;
          }

          private void inorderHelper(Node node, List<Integer> result) {
              if (node == null) return;
              inorderHelper(node.left, result);
              result.add(node.data);
              inorderHelper(node.right, result);
          }

          private List<Integer> postorder(Node root) {
              List<Integer> result = new ArrayList<>();
              postorderHelper(root, result);
              return result;
          }

          private void postorderHelper(Node node, List<Integer> result) {
              if (node == null) return;
              postorderHelper(node.left, result);
```

```
            postorderHelper(node.right, result);
            result.add(node.data);
        }

        private List<List<Integer>> levelOrder(Node root) {
            List<List<Integer>> result = new ArrayList<>();
            if (root == null) return result;

            Queue<Node> queue = new LinkedList<>();
            queue.add(root);

            while (!queue.isEmpty()) {
                int size = queue.size();
                List<Integer> level = new ArrayList<>();
                for (int i = 0; i < size; i++) {
                    Node node = queue.poll();
                    level.add(node.data);

                if (node.left != null) queue.add(node.left);
                if (node.right != null) queue.add(node.right);
            }
            result.add(level);
        }
        return result;
    }
}
```

13.2 Medium Problems

256. All Traversals in One Code

Understanding the Problem NEW

This isn't a new algorithm — it's simply all four traversal methods (Preorder, Inorder, Postorder, Level Order) collected together in one place for easy side-by-side reference and comparison.

Brute Force Approach NEW

N/A — this is a reference/consolidation exercise rather than a new problem to solve.

Why We Optimize NEW

Seeing all four traversals together highlights how similar Pre/In/Post really are (same recursive shape, different visit-order placement) while Level Order stands clearly apart as the one requiring a fundamentally different (queue-based, breadth-first) approach — reinforcing the pattern-recognition distinction established in problem 247.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

All four traversal methods (preorder, inorder, postorder, level order) implemented together in one class for easy reference and comparison.

Java Implementation

```
class Solution {
          public void allTraversals(Node root) {
              System.out.println("Preorder: " + preorder(root));
              System.out.println("Inorder: " + inorder(root));
              System.out.println("Postorder: " + postorder(root));
              System.out.println("Level Order: " + levelOrder(root));
          }

          private List<Integer> preorder(Node root) {
              List<Integer> result = new ArrayList<>();
              preorderHelper(root, result);
              return result;
          }
```

```
    private void preorderHelper(Node node, List<Integer> result) {
        if (node == null) return;
        result.add(node.data);
        preorderHelper(node.left, result);
        preorderHelper(node.right, result);
    }

    private List<Integer> inorder(Node root) {
        List<Integer> result = new ArrayList<>();
        inorderHelper(root, result);
        return result;
    }

    private void inorderHelper(Node node, List<Integer> result) {
        if (node == null) return;
        inorderHelper(node.left, result);
        result.add(node.data);
        inorderHelper(node.right, result);
    }

    private List<Integer> postorder(Node root) {
        List<Integer> result = new ArrayList<>();
        postorderHelper(root, result);
        return result;
    }

    private void postorderHelper(Node node, List<Integer> result) {
        if (node == null) return;
        postorderHelper(node.left, result);
        postorderHelper(node.right, result);
        result.add(node.data);
    }

    private List<List<Integer>> levelOrder(Node root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;

        Queue<Node> queue = new LinkedList<>();
        queue.add(root);

        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                Node node = queue.poll();
                level.add(node.data);
          if (node.left != null) queue.add(node.left);
          if (node.right != null) queue.add(node.right);
      }
      result.add(level);
    }
    return result;
```

```
    }
}
```

257. Height of a Binary Tree

Understanding the Problem NEW

Find the height of a binary tree — the number of edges (or nodes, depending on convention) on the longest path from the root down to any leaf.

Brute Force Approach NEW

You could try to enumerate every root-to-leaf path explicitly and take the longest one's length — technically correct, but generating every full path when you only need the length is more work than necessary.

Why We Optimize NEW

A node's height only depends on the heights of its *children* — specifically, `height(node) = 1 + max(height(left), height(right))`, with a height of 0 for an empty/null node. This is a natural **postorder-style** recursion (children's results are needed before the current node's result can be computed, echoing the pattern flagged in problem 250): recursively compute both subtrees' heights first, then combine them for the current node. This computes the whole tree's height in a single O(n) pass, visiting each node exactly once, rather than explicitly enumerating and comparing every root-to-leaf path.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find maximum depth (height) of binary tree.

Visual Diagram

```
Tree:
                  1        height = 3
               /     \
             2         3
           / \
         4    5

      Height = max(2, 1) + 1 = 3

      Recursive:
      height(1) = 1 + max(height(2), height(3))
```

```
       height(2) = 1 + max(height(4), height(5))
       height(4) = 1 + max(0, 0) = 1
       height(5) = 1
       height(2) = 1 + max(1, 1) = 2
       height(3) = 1
       height(1) = 1 + max(2, 1) = 3
```

Solution Approach

Concept: Recursively find max depth.

Algorithm

```
Algorithm HEIGHT(node):
           If node == null: Return 0
           leftHeight = HEIGHT(node.left)
           rightHeight = HEIGHT(node.right)
           Return 1 + max(leftHeight, rightHeight)
```

Java Implementation

```
class Solution {
           public int height(Node root) {
               if (root == null) return 0;
               return 1 + Math.max(height(root.left), height(root.right));
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

258. Check if Binary Tree is Height-Balanced

Understanding the Problem NEW

Check whether a binary tree is 'height-balanced' — meaning for *every* node in the tree, the height difference between its left and right subtrees is at most 1.

Brute Force Approach NEW

A naive approach computes the height of every node's subtrees independently by calling the Height function (problem 257) separately at each node — but since Height itself is O(n), and you'd call it at every one of the n nodes, this gives O(n²) total, redundantly recomputing heights of the same lower subtrees over and over from different starting points.

Why We Optimize NEW

Instead of computing height and checking balance as two separate passes, do them **together in a single postorder traversal**: at each node, first recursively get the left and right subtree's height *and* whether they're balanced; if either subtree is already unbalanced, immediately propagate that failure upward (no need to keep computing anything further, using a sentinel value like -1 to signal 'already found unbalanced'); otherwise, check the current node's own height difference and return the combined height. This computes both height and balance-status simultaneously in one O(n) pass, instead of the O(n²) that comes from recomputing subtree heights redundantly for every node.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if tree is height-balanced (diff ≤ 1 at every node).

Visual Diagram

```
Balanced:
                  1
               /    \
             2       3
            / \
          4     5
      Height diff = |2-1| = 1 ✓
```

```
      Unbalanced:
                1
              /
            2
          /
        3
      Height diff = |3-0| = 3 ✗

      Approach: Check balance at each node
      For node 1: leftHeight=3, rightHeight=0 → diff=3 → false
```

Solution Approach

Concept: Recursively check balance and height.

Algorithm

```
Algorithm IS_BALANCED(node):
          If node == null: Return true
          left = height(node.left)
          right = height(node.right)
          If abs(left-right) > 1: Return false
          Return IS_BALANCED(node.left) && IS_BALANCED(node.right)
```

Java Implementation

```
class Solution {
          public boolean isBalanced(Node root) {
              return checkBalance(root) != -1;
          }

           private int checkBalance(Node node) {
               if (node == null) return 0;

                int left = checkBalance(node.left);
                if (left == -1) return -1;
                int right = checkBalance(node.right);
                if (right == -1) return -1;

                if (Math.abs(left - right) > 1) return -1;
                return 1 + Math.max(left, right);
           }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

259. Diameter of Binary Tree

Understanding the Problem NEW

Find the diameter of a binary tree — the length of the longest path between *any* two nodes in the tree (this path may or may not pass through the root).

Brute Force Approach NEW

For every possible pair of nodes, compute the path length between them and track the maximum — extremely expensive ($O(n^2)$ pairs, each requiring path-length computation), and doesn't exploit any tree structure.

Why We Optimize NEW

The key insight: the longest path passing *through* any specific node is exactly `height(left subtree) + height(right subtree)` (the path dips down the left side, up through the node, down the right side) — so the overall diameter is simply the **maximum of this value across every node in the tree**. Compute this during the same postorder height-computation traversal as problem 257: at each node, after getting both subtrees' heights, check if `leftHeight + rightHeight` beats the best diameter seen so far (tracked in a shared variable), then return the height as usual for the parent's use. This piggybacks the diameter calculation onto the height calculation you're already doing, achieving $O(n)$ total instead of $O(n^2)$ — the same 'compute once, reuse for every node's local check' idea as problem 258.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find longest path between any two nodes.

Visual Diagram

```
Tree:
                    1
                  /   \
               2        3
             / \
           4   5
```

```
       Diameter = 4 (path: 4→2→1→3 or 5→2→1→3)

       For each node:
       Diameter = max(leftDiameter, rightDiameter, leftHeight + rightHeight)
```

Solution Approach

Concept: Diameter = max path through node or in subtrees.

Algorithm

```
Algorithm DIAMETER(node):
           If node == null: Return {height:0, diameter:0}
           left = DIAMETER(node.left)
           right = DIAMETER(node.right)

            height = 1 + max(left.height, right.height)
            diameter = max(left.diameter, right.diameter, left.height +
right.height)
            Return {height, diameter}
```

Java Implementation

```
class Solution {
           private int diameter = 0;

            public int diameterOfBinaryTree(Node root) {
                height(root);
                return diameter;
            }

            private int height(Node node) {
                if (node == null) return 0;
                int left = height(node.left);
                int right = height(node.right);
                diameter = Math.max(diameter, left + right);
                return 1 + Math.max(left, right);
            }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

260. Maximum Path Sum

Understanding the Problem NEW

Find the maximum path sum in a binary tree — the largest possible sum along any path between two nodes (again, not necessarily through the root, and nodes can have negative values).

Brute Force Approach NEW

Check every possible pair of nodes and compute the path sum between them — $O(n^2)$ or worse, and doesn't exploit tree structure.

Why We Optimize NEW

Similar to Diameter (259), but now summing values instead of counting edges, with an added twist for negative values: at each node, recursively get the **maximum single-direction path sum contribution** from the left and right subtrees (clamped to 0 if negative, since including a negative-sum branch would only hurt — you'd rather not extend the path into it at all), then the best path *through* this node is `node.value + max(0, leftContribution) + max(0, rightContribution)`, checked against a running global maximum; but critically, the value *returned* to the parent (for use in path sums going further up the tree) can only include **one** side (left or right, not both, since a path can't branch in two directions once it leaves a node upward) — this distinction between 'best path through this node' (both sides allowed) and 'best contribution passed up' (only one side allowed) is the key subtlety, computed together in one O(n) postorder pass.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find maximum path sum between any two nodes.

Visual Diagram

```
Tree:
              -10
              / \
           9          20
                 /        \
```

```
                15         7

      Maximum path sum = 42 (15→20→7)

      For each node:
      maxSum = max(maxSum, leftMax + rightMax + node.val)
      return max(node.val + max(leftMax, rightMax), node.val)
```

Solution Approach

Concept: Track max path sum through each node.

Algorithm

```
Algorithm MAX_PATH_SUM(node):
          maxSum = -INF
          maxGain(node):
              If node == null: Return 0
              left = max(0, maxGain(node.left))
              right = max(0, maxGain(node.right))
              maxSum = max(maxSum, left + right + node.val)
              Return node.val + max(left, right)
```

Java Implementation

```
class Solution {
          private int maxSum = Integer.MIN_VALUE;

          public int maxPathSum(Node root) {
              maxGain(root);
              return maxSum;
          }

          private int maxGain(Node node) {
              if (node == null) return 0;
              int left = Math.max(0, maxGain(node.left));
              int right = Math.max(0, maxGain(node.right));
              maxSum = Math.max(maxSum, left + right + node.data);
              return node.data + Math.max(left, right);
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

261. Check if Two Trees are Identical

Understanding the Problem NEW

Check whether two binary trees are structurally identical — same shape, and the same value at every corresponding position.

Brute Force Approach NEW

You could flatten both trees into some linear representation (like a traversal sequence) and compare those — but a naive traversal-based comparison alone isn't fully reliable, since different tree shapes can sometimes produce the same traversal sequence (e.g. a single traversal order doesn't always uniquely determine a tree's shape), so this needs care to be fully correct.

Why We Optimize NEW

Direct recursive comparison is both simpler and unambiguous: two trees are identical if their root values match *and* their left subtrees are recursively identical *and* their right subtrees are recursively identical (with two null nodes trivially being 'identical,' and a null compared against a non-null being an immediate mismatch). This compares both trees simultaneously in a single O(min(n,m)) pass (stopping early the moment any mismatch is found), directly checking structure and values together rather than relying on a potentially ambiguous flattened representation.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if two binary trees are identical.

Visual Diagram

```
Tree 1:        Tree 2:
          1             1
         / \           / \
        2   3         2   3

       Identical: Both have same structure and values

       Tree 1:      Tree 2:
```

```
          1           1
         / \         / \
        2     3     2    4
       Not identical (values differ at right child)
```

Solution Approach

Concept: Recursively compare both trees.

Algorithm

```
Algorithm IS_IDENTICAL(t1, t2):
           If t1 == null and t2 == null: Return true
           If t1 == null or t2 == null: Return false
           If t1.data != t2.data: Return false
           Return IS_IDENTICAL(t1.left, t2.left) &&
                  IS_IDENTICAL(t1.right, t2.right)
```

Java Implementation

```
class Solution {
           public boolean isIdentical(Node p, Node q) {
               if (p == null && q == null) return true;
               if (p == null || q == null) return false;
               if (p.data != q.data) return false;
               return isIdentical(p.left, q.left) && isIdentical(p.right, q.right);
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

262. Zig Zag Traversal

Understanding the Problem NEW

Traverse the tree level by level (like Level Order, problem 251), but alternate the direction of each level — left-to-right, then right-to-left, then left-to-right again, and so on.

Brute Force Approach NEW

You could run ordinary level order traversal and then reverse every other level's results afterward — this works but does the reversal as a separate post-processing pass rather than integrating it into the traversal itself.

Why We Optimize NEW

Use the same BFS/queue approach as Level Order, but track a simple **direction flag** that toggles after every level; when building the result list for a level, either append values normally or insert them at the front (or just reverse that level's collected list before adding it to the final result), based on the current direction. This integrates the zigzag behavior directly into the same O(n) BFS traversal, rather than requiring a separate reversal pass — a small but direct extension of the Level Order technique from problem 251.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree in zigzag level order.

Visual Diagram

```
Tree:
                    1
                  / \
              2      3
            / \    / \
          4    5 6    7

       Zigzag: [[1], [3,2], [4,5,6,7]]

       Queue process with direction flag:
       Level 0: left-to-right → [1]
```

```
       Level 1: right-to-left → [3,2]
       Level 2: left-to-right → [4,5,6,7]
```

Solution Approach

Concept: Level order with direction toggle.

Algorithm

```
Algorithm ZIGZAG(root):
           If root == null: Return []
           queue = Queue()
           queue.add(root)
           result = []
           leftToRight = true

           While queue not empty:
               levelSize = queue.size()
               level = new LinkedList()

                     For i = 0 to levelSize-1:
                         node = queue.poll()
                         If leftToRight:
                             level.addLast(node.data)
                         Else:
                             level.addFirst(node.data)
                         If node.left: queue.add(node.left)
                         If node.right: queue.add(node.right)

                     result.add(level)
                     leftToRight = !leftToRight

           Return result
```

Java Implementation

```
class Solution {
           public List<List<Integer>> zigzagLevelOrder(Node root) {
               List<List<Integer>> result = new ArrayList<>();
               if (root == null) return result;

                     Queue<Node> queue = new LinkedList<>();
                     queue.add(root);
                     boolean leftToRight = true;

              while (!queue.isEmpty()) {
                  int levelSize = queue.size();
                  List<Integer> level = new ArrayList<>();

                  for (int i = 0; i < levelSize; i++) {
```

```
                    Node node = queue.poll();
                    if (leftToRight) {
                        level.add(node.data);
                    } else {
                        level.add(0, node.data);
                    }
                    if (node.left != null) queue.add(node.left);
                    if (node.right != null) queue.add(node.right);
                }
                result.add(level);
                leftToRight = !leftToRight;
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

263. Boundary Traversal

Understanding the Problem NEW

Print the 'boundary' of the tree — the left edge (top to bottom), the leaf nodes (left to right), and the right edge (bottom to top), each visited without duplication where they overlap (e.g. the root and leaf boundary nodes shouldn't be printed twice).

Brute Force Approach NEW

Trying to handle this with one single traversal and complex conditional logic to detect 'is this a boundary node' on the fly is possible but genuinely awkward to get right cleanly.

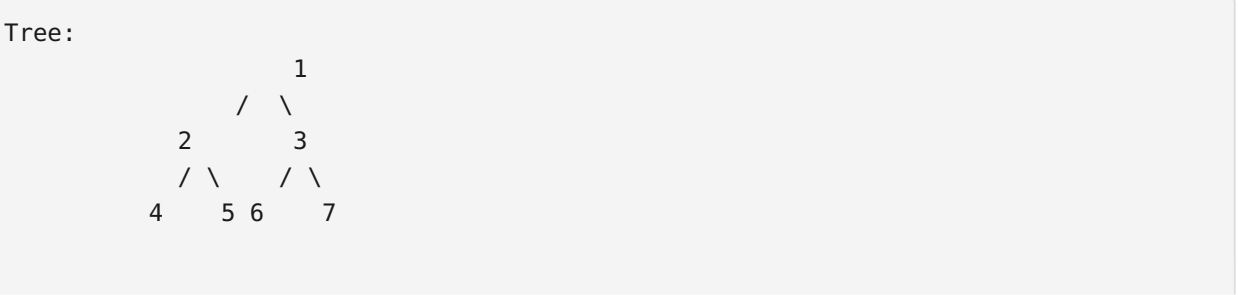
Why We Optimize NEW

Break the problem into **three clearly separate, simpler sub-traversals** and concatenate their results: (1) the left boundary, found by repeatedly going left (or right, if left is absent) from the root, excluding leaves; (2) all leaf nodes, found via any standard traversal (e.g. inorder) filtering for nodes with no children; (3) the right boundary, found similarly to the left boundary but mirrored, and printed in reverse (bottom to top) order. Decomposing a seemingly complex, unified traversal into three independent, individually simple traversals — each easy to verify correct on its own — is the real technique here, rather than trying to handle every case within one intricate traversal function.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse boundary of binary tree.

Visual Diagram



```
       Boundary: [1, 2, 4, 5, 6, 7, 3]
       (Left boundary + Leaves + Right boundary reverse)

       Components:
       1. Left boundary (excluding leaves): [1, 2]
       2. Leaves: [4, 5, 6, 7]
       3. Right boundary (excluding leaves, reverse): [3]
```

Solution Approach

Concept: Three parts: left boundary, leaves, right boundary.

Algorithm

```
Algorithm BOUNDARY_TRAVERSAL(root):
           result = []
           If root == null: Return result

           If !isLeaf(root): result.add(root.data)
           addLeftBoundary(root.left, result)
           addLeaves(root, result)
           addRightBoundary(root.right, result)
           Return result
```

Java Implementation

```
class Solution {
           public List<Integer> boundaryTraversal(Node root) {
               List<Integer> result = new ArrayList<>();
               if (root == null) return result;

                    if (!isLeaf(root)) result.add(root.data);
                    addLeftBoundary(root.left, result);
                    addLeaves(root, result);
                    addRightBoundary(root.right, result);
                    return result;
           }

           private void addLeftBoundary(Node node, List<Integer> result) {
               while (node != null) {
                   if (!isLeaf(node)) result.add(node.data);
                   node = (node.left != null) ? node.left : node.right;
               }
           }

           private void addLeaves(Node node, List<Integer> result) {
               if (node == null) return;
               if (isLeaf(node)) {

                  result.add(node.data);
```

```
                return;
            }
            addLeaves(node.left, result);
            addLeaves(node.right, result);
        }

        private void addRightBoundary(Node node, List<Integer> result) {
            Stack<Integer> stack = new Stack<>();
            while (node != null) {
                if (!isLeaf(node)) stack.push(node.data);
                node = (node.right != null) ? node.right : node.left;
            }
            while (!stack.isEmpty()) {
                result.add(stack.pop());
            }
        }

        private boolean isLeaf(Node node) {
            return node.left == null && node.right == null;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

264. Vertical Order Traversal

Understanding the Problem NEW

Group nodes by their 'vertical column' — imagine dropping a vertical line through the root; nodes directly below-left get negative column numbers, below-right get positive ones, and the same column number groups nodes into the same vertical line, printed top to bottom (with a tiebreak rule for nodes at the same row and column).

Brute Force Approach NEW

You could try to compute columns without any systematic approach, but without tracking each node's column and row together, correctly grouping and ordering nodes (especially handling ties) is easy to get wrong.

Why We Optimize NEW

Do a **BFS (level order) traversal, but track each node's horizontal distance (column) and row (depth)** alongside it, storing them in a map keyed by column, with values being lists of (row, node-value) — using BFS specifically (rather than a depth-first traversal) ensures that when multiple nodes land in the same column and row, they're naturally encountered in left-to-right order, simplifying the tie-breaking rule. After the traversal, sort the map's keys (columns) and, within each column, sort by row (and value, for ties at the same row) to produce the final grouped output. This turns an initially confusing 2D-positioning problem into 'BFS while tracking extra coordinates,' directly reusing the Level Order traversal technique with additional bookkeeping.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree by vertical columns.

Visual Diagram

```
Tree:
                  1
              /   \
           2       3
           / \     / \
```

```
          4    5 6    7

       Vertical order:
       Column -2: [4]
       Column -1: [2]
       Column 0: [1,5,6]
       Column 1: [3]
       Column 2: [7]

       HD (Horizontal Distance):
       Root: 0
       Left child: -1
       Right child: +1
```

Solution Approach

Concept: BFS with horizontal distances.

Algorithm

```
Algorithm VERTICAL_TRAVERSAL(root):
           map = new TreeMap<Integer, List<Integer>>()
           queue = Queue of (node, hd)
           queue.add((root, 0))

           While queue not empty:
               (node, hd) = queue.poll()
               map.get(hd).add(node.data)
               If node.left: queue.add((node.left, hd-1))
               If node.right: queue.add((node.right, hd+1))

           Return map values
```

Java Implementation

```
class Solution {
           public List<List<Integer>> verticalTraversal(Node root) {
               TreeMap<Integer, List<Integer>> map = new TreeMap<>();
               Queue<Pair<Node, Integer>> queue = new LinkedList<>();
               queue.add(new Pair<>(root, 0));

                    while (!queue.isEmpty()) {
                        Pair<Node, Integer> pair = queue.poll();
                        Node node = pair.getKey();
                        int hd = pair.getValue();

                        map.computeIfAbsent(hd, k -> new
ArrayList<>()).add(node.data);

                        if (node.left != null) queue.add(new Pair<>(node.left, hd -
```

```
1));
                        if (node.right != null) queue.add(new Pair<>(node.right, hd
+ 1));

                }

                return new ArrayList<>(map.values());
            }
        }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

265. Top View of Binary Tree

Understanding the Problem NEW

Print the nodes visible when looking at the tree from directly above — for each vertical column (as defined in Vertical Order Traversal, 264), only the **topmost** node in that column is visible.

Brute Force Approach NEW

You could compute the full vertical order traversal (264) and then, for each column, sort by row and take the first node — correct, but requires the full grouping-and-sorting machinery of vertical order when you only need the very first node per column.

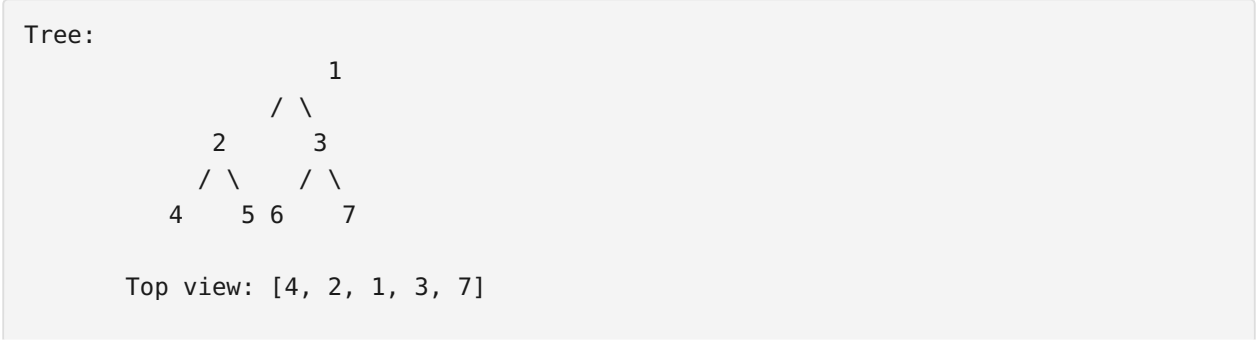
Why We Optimize NEW

Since you only need the *first* node encountered per column, and BFS naturally processes nodes in top-to-bottom order, you can simplify: do a BFS traversal tracking horizontal distance, and for each column, record a value in a map **only if that column hasn't been seen yet** (ignoring any subsequent, lower node in the same column). This achieves the same 'top view' result with a lighter-weight single pass and a simpler map (no need to store every node's row, and no post-traversal sorting by row needed), rather than fully solving the more general vertical-order problem and filtering afterward.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print top view nodes visible from top.

Visual Diagram



```
For each HD, first node encountered in level order
HD -2: 4
HD -1: 2
HD 0: 1
HD 1: 3
HD 2: 7
```

Solution Approach

Concept: BFS with first occurrence per HD.

Algorithm

```
Algorithm TOP_VIEW(root):
           map = new TreeMap<Integer, Integer>()
           queue = Queue of (node, hd)
           queue.add((root, 0))

           While queue not empty:
               (node, hd) = queue.poll()
               If hd not in map:
                   map.put(hd, node.data)
               If node.left: queue.add((node.left, hd-1))
               If node.right: queue.add((node.right, hd+1))

           Return map values
```

Java Implementation

```
class Solution {
           public List<Integer> topView(Node root) {
               List<Integer> result = new ArrayList<>();
               if (root == null) return result;

                     Map<Integer, Integer> map = new TreeMap<>();
                     Queue<Pair<Node, Integer>> queue = new LinkedList<>();
                     queue.add(new Pair<>(root, 0));

                     while (!queue.isEmpty()) {
                         Pair<Node, Integer> pair = queue.poll();
                         Node node = pair.getKey();
                         int hd = pair.getValue();

                         if (!map.containsKey(hd)) {
                             map.put(hd, node.data);
                         }

                   if (node.left != null) queue.add(new Pair<>(node.left, hd - 1));
                   if (node.right != null) queue.add(new Pair<>(node.right, hd +
```

```
1));
            }

            return new ArrayList<>(map.values());
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

266. Bottom View of Binary Tree

Understanding the Problem NEW

The mirror image of Top View: print the nodes visible from directly below — for each vertical column, only the **bottommost** node is visible.

Brute Force Approach NEW

Same idea as Top View — you could compute the full vertical order and take the last node in each column's sorted-by-row list.

Why We Optimize NEW

Same simplified BFS approach as Top View (265), but with one key difference: instead of only recording a column's value on *first* occurrence, **always overwrite** the map's entry for that column with the current node — since BFS processes top-to-bottom, the last write for any given column will naturally end up being the bottommost node. A one-line change (always overwrite vs. only-write-if-absent) is the entire difference between Top View and Bottom View, again reinforcing that grasping the underlying BFS-with-column-tracking technique lets you solve several visually distinct-sounding problems with minimal changes.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print bottom view nodes visible from bottom.

Visual Diagram

```
Tree:
                    1
                /   \
            2       3
            / \     / \
          4    5 6    7

       Bottom view: [4, 5, 6, 7]

       For each HD, last node encountered in level order
       HD -2: 4
       HD -1: 2 (but 5 is lower) → 5
```

```
       HD 0: 1 (but 5,6 are lower) → 5? Wait, both 5 and 6 at HD 0
       Actually bottom view: [4, 5, 6, 7]
```

Solution Approach

Concept: BFS with last occurrence per HD.

Algorithm

```
Algorithm BOTTOM_VIEW(root):
           map = new TreeMap<Integer, Integer>()
           queue = Queue of (node, hd)
           queue.add((root, 0))

           While queue not empty:
               (node, hd) = queue.poll()
               map.put(hd, node.data) // overwrite for bottom
               If node.left: queue.add((node.left, hd-1))
               If node.right: queue.add((node.right, hd+1))

           Return map values
```

Java Implementation

```
class Solution {
           public List<Integer> bottomView(Node root) {
               List<Integer> result = new ArrayList<>();
               if (root == null) return result;

                    Map<Integer, Integer> map = new TreeMap<>();
                    Queue<Pair<Node, Integer>> queue = new LinkedList<>();
                    queue.add(new Pair<>(root, 0));

                    while (!queue.isEmpty()) {
                        Pair<Node, Integer> pair = queue.poll();
                        Node node = pair.getKey();
                        int hd = pair.getValue();

                        map.put(hd, node.data); // Overwrite for bottom view

                        if (node.left != null) queue.add(new Pair<>(node.left, hd -
1));
                        if (node.right != null) queue.add(new Pair<>(node.right, hd
+ 1));
                    }

               return new ArrayList<>(map.values());
           }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

267. Right/Left View of Binary Tree

Understanding the Problem NEW

Print the nodes visible when looking at the tree from the right side (or left side) — for each *level*, only the rightmost (or leftmost) node is visible.

Brute Force Approach NEW

You could compute the full level order traversal (251) and then, for each level's list, take the last (or first) element — correct, but builds and stores every node in every level when only one per level is actually needed for the final answer.

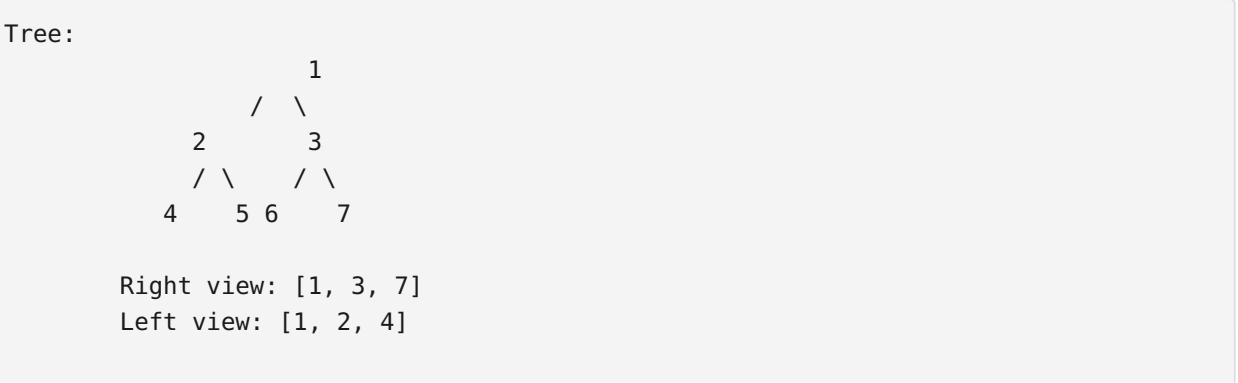
Why We Optimize NEW

Do a standard BFS, but for each level, only keep track of the **last node processed in that level** for Right View (or the *first* node for Left View) — you don't need to store the whole level's contents, just note which node was last (or first) as you dequeue through that level's nodes, adding only that one to the result. This is a lighter-weight variant of Level Order that only retains one value per level instead of the full level's list, directly building on the same BFS traversal used throughout this 'views of a tree' family of problems (Top, Bottom, Left/Right View all share the same BFS backbone, varying only in which node per grouping gets kept).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print nodes visible from right or left side.

Visual Diagram



```
        Right view - last node at each level
        Level 0: 1
        Level 1: 3
        Level 2: 7
```

Solution Approach

Concept: Level order with first/last per level.

Algorithm

```
Algorithm RIGHT_VIEW(root):
            queue = Queue()
            queue.add(root)
            result = []

            While queue not empty:
                size = queue.size()
                For i = 0 to size-1:
                    node = queue.poll()
                    If i == size-1: result.add(node.data)
                    If node.left: queue.add(node.left)
                    If node.right: queue.add(node.right)

            Return result
```

Java Implementation

```
class Solution {
            // Right View
            public List<Integer> rightView(Node root) {
                List<Integer> result = new ArrayList<>();
                if (root == null) return result;

                     Queue<Node> queue = new LinkedList<>();
                     queue.add(root);

                     while (!queue.isEmpty()) {
                         int size = queue.size();
                         for (int i = 0; i < size; i++) {
                             Node node = queue.poll();
                             if (i == size - 1) result.add(node.data);
                             if (node.left != null) queue.add(node.left);
                             if (node.right != null) queue.add(node.right);
                         }

              }
              return result;
          }
```

```
        // Left View
        public List<Integer> leftView(Node root) {
            List<Integer> result = new ArrayList<>();
            if (root == null) return result;

            Queue<Node> queue = new LinkedList<>();
            queue.add(root);

            while (!queue.isEmpty()) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    Node node = queue.poll();
                    if (i == 0) result.add(node.data);
                    if (node.left != null) queue.add(node.left);
                    if (node.right != null) queue.add(node.right);
                }
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

268. Symmetric Binary Tree

Understanding the Problem NEW

Check whether a binary tree is symmetric — meaning it looks the same as its own mirror image around the vertical center line through the root.

Brute Force Approach NEW

You could try to build an explicit mirrored copy of the tree and then check if it's identical to the original (reusing the 'check identical trees' logic from problem 261) — this works, but constructing a whole extra mirrored tree just to compare it is unnecessary overhead.

Why We Optimize NEW

Rather than building a physical mirror, do a recursive comparison directly: check that the tree's **left subtree is a mirror of the right subtree** — where 'is a mirror of' means each node's value matches, its left child mirrors the other tree's right child, and its right child mirrors the other tree's left child (crossed comparisons, not same-side ones, which is the key difference from problem 261's identical-tree check). This achieves the symmetry check in a single O(n) recursive pass with no extra tree construction, by directly encoding the 'mirror' relationship into the comparison itself rather than materializing an actual mirrored copy first.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if tree is symmetric around its center.

Visual Diagram

```
Symmetric:
                1
              /   \
             2     2
            / \   / \
           3   4 4   3

      Not symmetric:
                1
              /   \
```

```
          2       2
         / \     / \
        3     4 5    3

      Mirror check:
      isMirror(left, right):
          If both null: true
          If one null: false
          If values differ: false
          Return isMirror(left.left, right.right) &&
                 isMirror(left.right, right.left)
```

Solution Approach

Concept: Mirror comparison recursively.

Algorithm

```
Algorithm IS_SYMMETRIC(root):
          If root == null: Return true
          Return isMirror(root.left, root.right)

      Algorithm isMirror(left, right):
          If left == null && right == null: Return true
          If left == null || right == null: Return false
          Return left.data == right.data &&
                 isMirror(left.left, right.right) &&
                 isMirror(left.right, right.left)
```

Java Implementation

```
class Solution {
          public boolean isSymmetric(Node root) {
              if (root == null) return true;
              return isMirror(root.left, root.right);
          }

          private boolean isMirror(Node left, Node right) {
              if (left == null && right == null) return true;
              if (left == null || right == null) return false;
              return left.data == right.data &&
                     isMirror(left.left, right.right) &&
                     isMirror(left.right, right.left);
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

269. Root to Node Path in Binary Tree

Understanding the Problem NEW

Find the path of nodes from the root down to a specific target node/value.

Brute Force Approach NEW

You could search the tree with any traversal, but without a clear way to 'remember the path so far,' correctly reconstructing the exact path taken (rather than just confirming the node exists) requires deliberate bookkeeping.

Why We Optimize NEW

Use **backtracking** during a DFS traversal: maintain a running list representing 'the path so far,' adding the current node before recursing into children; if the current node is the target, you're done (the current path list is the answer); otherwise, try recursing left, then right — if a recursive call into a child doesn't find the target, **remove that child from the path** (backtrack) before trying the other side, since that branch wasn't the correct one. This is the same 'try, recurse, undo if unsuccessful' backtracking discipline seen throughout the Recursion chapter (Word Search, Rat in a Maze, N-Queens), applied here to tree path-finding instead of grid/board exploration.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find path from root to a given node.

Visual Diagram

```
Tree:
                    1
                /  \
            2       3
            / \    / \
          4    5 6    7

       Path to 5: [1, 2, 5]

       Recursive search:
       findPath(1, 5):
```

```
         check left: findPath(2, 5)
           check left: findPath(4, 5) → false
           check right: findPath(5, 5) → add 5, return true
           add 2, return true
         add 1, return true
```

Solution Approach

Concept: Recursively search and backtrack.

Algorithm

```
Algorithm ROOT_TO_NODE(root, target):
           path = []
           If getPath(root, target, path):
               Return path
           Return []

       Algorithm getPath(node, target, path):
           If node == null: Return false
           path.add(node.data)
           If node.data == target: Return true
           If getPath(node.left, target, path) or
              getPath(node.right, target, path):
               Return true
           path.remove(path.size()-1)
           Return false
```

Java Implementation

```
class Solution {
           public List<Integer> rootToNodePath(Node root, int target) {
               List<Integer> path = new ArrayList<>();
               getPath(root, target, path);
               return path;
           }

           private boolean getPath(Node node, int target, List<Integer> path) {
               if (node == null) return false;

                    path.add(node.data);

              if (node.data == target) return true;

              if (getPath(node.left, target, path) || getPath(node.right, target,
path)) {
                   return true;
              }

              path.remove(path.size() - 1);
```

```
            return false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

270. LCA in Binary Tree

Understanding the Problem NEW

Find the Lowest Common Ancestor (LCA) of two given nodes — the deepest node in the tree that has both target nodes as descendants (a node is considered its own descendant for this purpose).

Brute Force Approach NEW

You could find the root-to-node path for both targets separately (reusing problem 269's technique) and then compare the two paths to find where they last agree — correct, and actually a perfectly reasonable O(n) approach, but it requires storing and comparing two full paths explicitly.

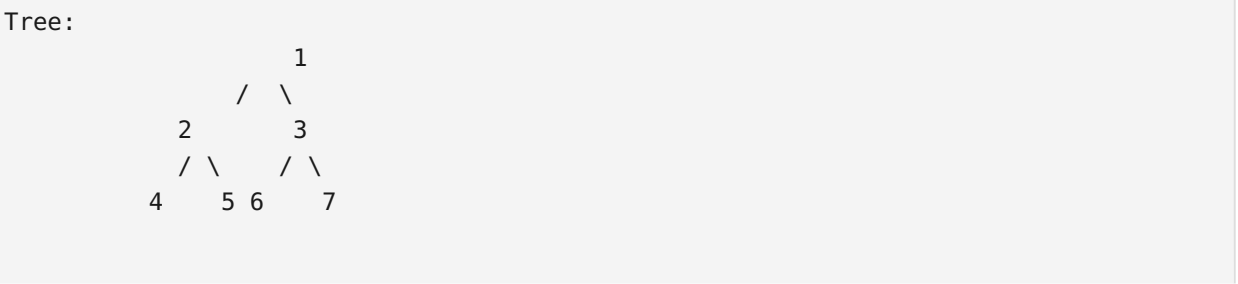
Why We Optimize NEW

A more elegant single-pass approach: recursively search both left and right subtrees for either target; if a subtree call returns a non-null result from *both* sides (meaning one target was found on the left and the other on the right), the current node *is* the LCA; if only one side returns non-null, propagate that result upward (the LCA must be further up, or that returned node itself might be the LCA if it happens to be an ancestor of the other target too). This finds the LCA in a single O(n) traversal without explicitly constructing and comparing two separate root-to-node paths, by letting the recursion itself discover where the two search paths 'merge.'

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find Lowest Common Ancestor of two nodes.

Visual Diagram



```
LCA(4, 5) = 2
LCA(4, 7) = 1
LCA(2, 5) = 2

Recursive:
findLCA(1, 4, 7):
  left = findLCA(2, 4, 7)
    left = findLCA(4, 4, 7) → returns 4
    right = findLCA(5, 4, 7) → returns null
    returns 4
  right = findLCA(3, 4, 7) → returns null
  returns 4? Wait, expected 1

Better approach:
If node is null: return null
If node is p or q: return node
left = findLCA(node.left, p, q)
right = findLCA(node.right, p, q)
If left != null && right != null: return node
Return left != null ? left : right
```

Solution Approach

Concept: Recursively find both nodes.

Algorithm

```
Algorithm LCA(node, p, q):
           If node == null: Return null
           If node == p || node == q: Return node

           left = LCA(node.left, p, q)
           right = LCA(node.right, p, q)

           If left != null && right != null: Return node
           Return left != null ? left : right
```

Java Implementation

```
class Solution {
           public Node lowestCommonAncestor(Node root, Node p, Node q) {
               if (root == null) return null;
               if (root == p || root == q) return root;

                    Node left = lowestCommonAncestor(root.left, p, q);
                    Node right = lowestCommonAncestor(root.right, p, q);

                    if (left != null && right != null) return root;

              return left != null ? left : right;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

271. Maximum Width of Binary Tree

Understanding the Problem NEW

Find the maximum width of a binary tree — the largest number of 'slots' between the leftmost and rightmost node at any level, *including* the slots for missing (null) nodes in between (not just counting actual nodes present).

Brute Force Approach NEW

You could do ordinary level order traversal and just count how many actual nodes are present per level — but this misses the requirement to count the full span including gaps from missing nodes, e.g. a level with nodes only at the far-left and far-right position (with nothing in between) should count the full span, not just 2.

Why We Optimize NEW

During BFS, assign each node an **index** following the standard 'complete binary tree' numbering (root=0, left child = 2×parent+1, right child = 2×parent+2) — then a level's width is simply `(last index at that level) - (first index at that level) + 1`, which correctly accounts for gaps since the indices reflect true positions, not just how many nodes actually exist. A practical detail: these indices can grow very large for deep, sparse trees, so re-basing each level's indices relative to that level's first index (subtracting it off) before computing child indices prevents integer overflow — this reuses the same BFS backbone as the 'views' problems, with index-tracking added as the specific extra bookkeeping this problem needs.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find maximum width (including null nodes).

Visual Diagram

```
Tree:
                    1
                /       \
            2             3
          / \                 \
        4   5                  8
```

```
Max width = 4 (level 2: [4,5,null,8])

Using indexing:
Root: index = 1
Left: 2*index, Right: 2*index+1
Level 0: [1] → width=1
Level 1: [2,3] → width=2
Level 2: [4,5,null,8] → width=4
```

Solution Approach

Concept: BFS with indexing.

Algorithm

```
Algorithm MAX_WIDTH(root):
           queue = Queue of (node, index)
           queue.add((root, 1))
           maxWidth = 0

           While queue not empty:
               size = queue.size()
               first = queue.peek().index
               last = first

                      For i = 0 to size-1:
                          (node, index) = queue.poll()
                          last = index
                          If node.left: queue.add((node.left, 2*index))
                          If node.right: queue.add((node.right, 2*index+1))

                      maxWidth = max(maxWidth, last - first + 1)

           Return maxWidth
```

Java Implementation

```
class Solution {
           public int maxWidth(Node root) {
               if (root == null) return 0;

                      Queue<Pair<Node, Integer>> queue = new LinkedList<>();
                      queue.add(new Pair<>(root, 1));
                      int maxWidth = 0;

                      while (!queue.isEmpty()) {
                          int size = queue.size();
                          int first = queue.peek().getValue();
```

```
                int last = first;

                for (int i = 0; i < size; i++) {
                    Pair<Node, Integer> pair = queue.poll();
                    Node node = pair.getKey();
                    int index = pair.getValue();
                    last = index;

                    if (node.left != null) {
                        queue.add(new Pair<>(node.left, 2L * index));
                    }
                    if (node.right != null) {
                        queue.add(new Pair<>(node.right, 2L * index + 1));
                    }
                }
                maxWidth = Math.max(maxWidth, last - first + 1);
            }
            return maxWidth;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

272. Check for Children Sum Property

Understanding the Problem NEW

Check whether every node in the tree satisfies the 'children sum property' — meaning each non-leaf node's value equals the sum of its immediate children's values (leaf nodes are trivially fine, having no children to sum).

Brute Force Approach NEW

Check each node against its immediate children individually with a simple traversal — this is actually already efficient and there isn't a slower 'brute' alternative to meaningfully contrast against; the interesting part of this problem is usually the related task of *fixing* a tree to satisfy this property, which requires more careful top-down adjustment.

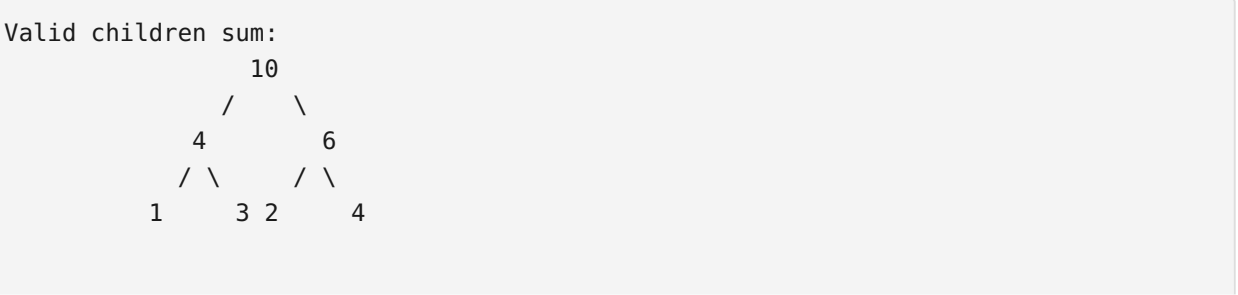
Why We Optimize NEW

A straightforward single traversal suffices to *check* the property: for each node, verify `node.value == (left child's value, or 0 if absent) + (right child's value, or 0 if absent)`, recursing into children regardless, achieving O(n) time with no special technique beyond a direct property check per node — worth noting mainly because a very similar-looking problem ('make the tree satisfy this property by adjusting values') requires a more careful top-down pass (propagating value changes downward from parent to children) rather than the simple bottom-up check shown here.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if every node equals sum of children.

Visual Diagram



```
       10 = 4+6 ✓
       4 = 1+3 ✓
       6 = 2+4 ✓

       Invalid:
                     10
                /         \
             5              4
             / \            / \
            1   3          2   4

       10 ≠ 5+4 ✗
```

Solution Approach

Concept: Recursively check children sum.

Algorithm

```
Algorithm CHILDREN_SUM(node):
           If node == null: Return true
           If node.left == null && node.right == null: Return true

            sum = (node.left != null ? node.left.data : 0) +
                  (node.right != null ? node.right.data : 0)

            Return node.data == sum &&
                   CHILDREN_SUM(node.left) &&
                   CHILDREN_SUM(node.right)
```

Java Implementation

```
class Solution {
           public boolean isChildrenSumProperty(Node root) {
               if (root == null) return true;
               if (root.left == null && root.right == null) return true;

                     int sum = 0;
                     if (root.left != null) sum += root.left.data;
                     if (root.right != null) sum += root.right.data;

                     return root.data == sum &&
                            isChildrenSumProperty(root.left) &&
                            isChildrenSumProperty(root.right);
            }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

273. Nodes at Distance K in Binary Tree

Understanding the Problem NEW

Given a target node, find every node in the tree that is exactly K edges away from it — which could be nodes below the target (descendants) or nodes reached by going up through ancestors and then back down other branches.

Brute Force Approach NEW

Compute the distance from the target to every other node individually (e.g. via a fresh search from the target for each candidate node) — O(n) per node, $O(n^2)$ overall, since 'distance in a tree' isn't naturally something a simple top-down recursion computes directly for arbitrary node pairs.

Why We Optimize NEW

The trick: a binary tree's parent-child links only go one direction (down), but distance to nodes that require going *up* through ancestors needs a way to traverse upward too — so first do a traversal building a **parent-pointer map** (recording each node's parent, since the tree itself doesn't store this), then treat the tree as an undirected graph (where you can move to a parent, left child, or right child, all as equally valid 'neighbours') and run a **BFS starting from the target node**, stopping exactly K levels out. This converts a tree-distance problem into a graph BFS problem by adding the missing 'upward' links via the parent map — the same core BFS technique from Level Order, just applied to a graph-like view of the tree rather than the tree's native root-to-leaf direction, and directly foreshadowing BFS's central role in the upcoming Graphs chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all nodes at distance K from a given node.

Visual Diagram

```
Tree:
                  1
              /   \
           2       3
           / \     / \
```

```
          4    5 6    7

       Target: 2, K = 1
       Nodes at distance 1: [1, 4, 5]

       Target: 2, K = 2
       Nodes: [3, 4, 5, 6, 7? Wait, 6 and 7 are distance 3 from 2]

       Distance 2 from 2:
       - Through parent: 2→1→3 → 3
       - Through left child: 2→4 (distance 1) → no child
       - Through right child: 2→5 (distance 1) → no child
       Actually 4 and 5 are distance 1, not 2

       Distance 2 from 2: [3, 6, 7]
```

Solution Approach

Concept: Find target, then search in subtrees.

Algorithm

```
Algorithm NODES_AT_DISTANCE_K(root, target, k):
           map = new HashMap<Node, Node>() // parent map
           DFS to fill parent map

           queue = Queue()
           queue.add(target)
           visited = HashSet()
           visited.add(target)
           distance = 0
           result = []

           While queue not empty and distance <= k:
               size = queue.size()
               For i = 0 to size-1:
                   node = queue.poll()
                   If distance == k: result.add(node.data)
                   // Add parent, left, right if not visited
                   If node.parent not visited: add to queue
                   If node.left not visited: add to queue
                   If node.right not visited: add to queue
               distance++

           Return result
```

Java Implementation

```
class Solution {
           public List<Integer> distanceK(Node root, Node target, int k) {
```

```
            Map<Node, Node> parent = new HashMap<>();
            getParentMap(root, null, parent);

            Queue<Node> queue = new LinkedList<>();
            Set<Node> visited = new HashSet<>();
            queue.add(target);
            visited.add(target);
            List<Integer> result = new ArrayList<>();
            int distance = 0;

            while (!queue.isEmpty() && distance <= k) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    Node node = queue.poll();
                    if (distance == k) {
                        result.add(node.data);
                    }
                    if (node.left != null && !visited.contains(node.left)) {
                        queue.add(node.left);
                        visited.add(node.left);
                    }
                    if (node.right != null && !visited.contains(node.right)) {
                        queue.add(node.right);
                        visited.add(node.right);
                    }
                    if (parent.get(node) != null && !
visited.contains(parent.get(node))) {
                        queue.add(parent.get(node));
                        visited.add(parent.get(node));
                    }
                }
                distance++;
            }
            return result;
        }

        private void getParentMap(Node node, Node parent, Map<Node, Node> map) {
            if (node == null) return;
            map.put(node, parent);
            getParentMap(node.left, node, map);
            getParentMap(node.right, node, map);
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

274. Minimum Time to Burn Binary Tree

Understanding the Problem NEW

Given a target node that starts on fire, and fire spreads to adjacent nodes (parent, left child, right child) one time-unit per step, find the minimum time needed for the fire to reach every node in the tree.

Brute Force Approach NEW

Simulate the fire spreading minute by minute across the whole tree structure directly using the tree's native (parent-to-child only) links — but since fire can also spread *upward* to a parent, the tree's one-directional child links alone aren't sufficient to simulate this correctly without extra bookkeeping.

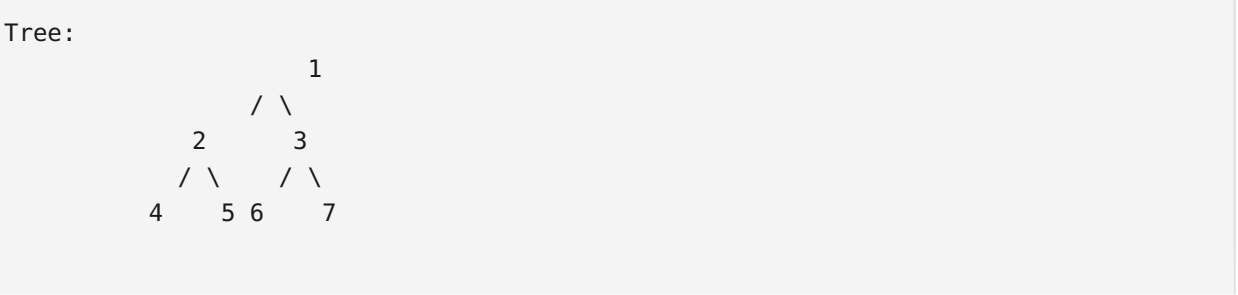
Why We Optimize NEW

Exactly the same fix as Nodes at Distance K (273): build a parent-pointer map first, then run a **BFS from the target node** treating parent/left/right as equally valid neighbours (since fire doesn't care about the 'down-only' direction trees normally impose) — the total time to burn the whole tree is simply the number of BFS levels needed to reach every node (the eccentricity of the target node in this now-undirected view of the tree). This is essentially the same technique as problem 273, extended to find the maximum distance to *any* node rather than just nodes at exactly distance K — another direct reuse of 'add parent pointers, then BFS' to handle upward movement that plain trees don't natively support.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find time to burn entire tree from a target node.

Visual Diagram



```
       Burn from 2:
       Time 0: 2
       Time 1: 1, 4, 5
       Time 2: 3, 6, 7
       Total time = 2

       Same as finding max distance from target
```

Solution Approach

Concept: BFS from target with parent map.

Algorithm

```
Algorithm BURN_TIME(root, target):
           parent = getParentMap(root)
           queue = Queue()
           queue.add(target)
           visited = HashSet()
           visited.add(target)
           time = 0

           While queue not empty:
               size = queue.size()
               burned = false
               For i = 0 to size-1:
                   node = queue.poll()
                   If node.left not visited:
                       burned = true
                       queue.add(node.left)
                       visited.add(node.left)
                   If node.right not visited:
                       burned = true
                       queue.add(node.right)
                       visited.add(node.right)
                   If parent.get(node) not visited:
                       burned = true
                       queue.add(parent.get(node))
                       visited.add(parent.get(node))
               If burned: time++

           Return time
```

Java Implementation

```
class Solution {
           public int burnTree(Node root, Node target) {
               Map<Node, Node> parent = new HashMap<>();

              getParentMap(root, null, parent);
```

```
            Queue<Node> queue = new LinkedList<>();
            Set<Node> visited = new HashSet<>();
            queue.add(target);
            visited.add(target);
            int time = 0;

            while (!queue.isEmpty()) {
                int size = queue.size();
                boolean burned = false;

                for (int i = 0; i < size; i++) {
                    Node node = queue.poll();

                    if (node.left != null && !visited.contains(node.left)) {
                        burned = true;
                        queue.add(node.left);
                        visited.add(node.left);
                    }
                    if (node.right != null && !visited.contains(node.right)) {
                        burned = true;
                        queue.add(node.right);
                        visited.add(node.right);
                    }
                    if (parent.get(node) != null && !
visited.contains(parent.get(node))) {
                        burned = true;
                        queue.add(parent.get(node));
                        visited.add(parent.get(node));
                    }
                }
                if (burned) time++;
            }
            return time;
        }

        private void getParentMap(Node node, Node parent, Map<Node, Node> map) {
            if (node == null) return;
            map.put(node, parent);
            getParentMap(node.left, node, map);
            getParentMap(node.right, node, map);
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

275. Count Total Nodes in Complete Binary Tree

Understanding the Problem NEW

Given a **Complete** Binary Tree specifically (every level full except possibly the last, which fills left to right with no gaps), count the total number of nodes faster than a plain O(n) traversal would.

Brute Force Approach NEW

Visit every single node with a standard traversal and count them — correct, O(n), but ignores the extra structural guarantee ('complete tree') the problem specifically provides.

Why We Optimize NEW

Exploit the completeness guarantee: for any node, compute the height by always going **left** only, and separately the height by always going **right** only — if these two heights are equal, the subtree rooted here is a *perfect*2^height - 1, with no need to visit every individual node; if the heights differ, recurse into both children separately and combine their counts (only one side, at most, will actually be 'incomplete' and require deeper recursion, thanks to the completeness property). This achieves O(log²n) time — recursing O(log n) levels deep, each doing O(log n) work to compute left/right height — dramatically better than the O(n) a general (non-complete) tree would require, purely by exploiting the specific structural guarantee this problem provides that general trees don't have.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count nodes in complete binary tree faster than O(n).

Visual Diagram

```
Complete Binary Tree:
                1
              /    \
            2       3
          / \     / \
        4     5 6    7
```

```
       Count = 7

       Complete binary tree property:
       If leftHeight == rightHeight: nodes = 2^height - 1
       Else: 1 + count(left) + count(right)
```

Solution Approach

Concept: Use complete tree property.

Algorithm

```
Algorithm COUNT_NODES(root):
           If root == null: Return 0
           leftHeight = getLeftHeight(root)
           rightHeight = getRightHeight(root)

           If leftHeight == rightHeight:
               Return (1 << leftHeight) - 1
           Else:
               Return 1 + COUNT_NODES(root.left) + COUNT_NODES(root.right)
```

Java Implementation

```
class Solution {
           public int countNodes(Node root) {
               if (root == null) return 0;
               int leftHeight = getLeftHeight(root);
               int rightHeight = getRightHeight(root);

                 if (leftHeight == rightHeight) {
                     return (1 << leftHeight) - 1;
                 }
                 return 1 + countNodes(root.left) + countNodes(root.right);
           }

           private int getLeftHeight(Node node) {
               int height = 0;
               while (node != null) {
                   height++;
                   node = node.left;
               }
               return height;
           }

           private int getRightHeight(Node node) {
               int height = 0;
               while (node != null) {

                   height++;
```

```
                node = node.right;
            }
            return height;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log² n) | Space Complexity: O(log n)

276-278. Construct Binary Trees from Traversals

Understanding the Problem NEW

Given two of a tree's traversal sequences (like Preorder + Inorder, or Postorder + Inorder), reconstruct the original tree structure that would produce exactly those sequences.

Brute Force Approach NEW

Without a clear plan, you might try many possible tree shapes and check if replaying their traversals matches the given sequences — extremely wasteful, since there's a direct constructive method available instead of a guess-and-check search.

Why We Optimize NEW

The key insight: in **Preorder**, the very first element is always the root; in **Inorder**, everything to the *left* of that same root's position belongs to the left subtree, and everything to the *right* belongs to the right subtree. So you can recursively: take the current preorder's first element as the root, find that value's position in the current inorder slice to split it into left/right portions, and recurse to build the left and right subtrees from the corresponding slices of both sequences. Using a hashmap to look up each value's position in the inorder sequence (instead of scanning for it each time) brings this down to O(n) total, rather than an $O(n^2)$ approach that re-scans the inorder sequence for every node, or a combinatorial guess-and-check search — this exact 'first element = root, split by its position in the other sequence' technique works symmetrically for Postorder + Inorder too, just using postorder's *last* element as the root instead of preorder's first.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Construct trees from given traversals.

Visual Diagram

```
Inorder: [4, 2, 5, 1, 6, 3, 7]
       Preorder: [1, 2, 4, 5, 3, 6, 7]
```

```
      Tree:
                  1
               /  \
            2       3
            / \    / \
          4    5 6    7

      Build process:
      Preorder[0] = 1 → root
      Find 1 in inorder → index 3
      Left inorder: [4,2,5], Right inorder: [6,3,7]
      Left preorder: [2,4,5], Right preorder: [3,6,7]
      Recursively build
```

Java Implementation

```
class Solution {
           // Build from inorder and preorder
           public Node buildTreeInPre(int[] inorder, int[] preorder) {
               Map<Integer, Integer> map = new HashMap<>();
               for (int i = 0; i < inorder.length; i++) {
                   map.put(inorder[i], i);
               }
               return buildInPre(preorder, 0, preorder.length - 1,
                                 inorder, 0, inorder.length - 1, map);
           }

           private Node buildInPre(int[] preorder, int preStart, int preEnd,
                                   int[] inorder, int inStart, int inEnd,
                                   Map<Integer, Integer> map) {
               if (preStart > preEnd || inStart > inEnd) return null;

                    int rootVal = preorder[preStart];
                    Node root = new Node(rootVal);
                    int rootIndex = map.get(rootVal);
                    int leftSize = rootIndex - inStart;

                    root.left = buildInPre(preorder, preStart + 1, preStart +
leftSize,
                                           inorder, inStart, rootIndex - 1, map);
                    root.right = buildInPre(preorder, preStart + leftSize + 1,
preEnd,
                                            inorder, rootIndex + 1, inEnd, map);
                    return root;
           }

           // Build from inorder and postorder
           public Node buildTreeInPost(int[] inorder, int[] postorder) {
               Map<Integer, Integer> map = new HashMap<>();
               for (int i = 0; i < inorder.length; i++) {
                   map.put(inorder[i], i);
```

```
            }
            return buildInPost(postorder, 0, postorder.length - 1,
                               inorder, 0, inorder.length - 1, map);
        }

        private Node buildInPost(int[] postorder, int postStart, int postEnd,
                                 int[] inorder, int inStart, int inEnd,
                                 Map<Integer, Integer> map) {
            if (postStart > postEnd || inStart > inEnd) return null;

            int rootVal = postorder[postEnd];
            Node root = new Node(rootVal);
            int rootIndex = map.get(rootVal);
            int leftSize = rootIndex - inStart;

            root.left = buildInPost(postorder, postStart, postStart + leftSize -
1,
                                    inorder, inStart, rootIndex - 1, map);
            root.right = buildInPost(postorder, postStart + leftSize, postEnd -
1,
                                     inorder, rootIndex + 1, inEnd, map);
            return root;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

279. Serialize and Deserialize Binary Tree

Understanding the Problem NEW

Design a way to convert a binary tree into a single string (serialize), and then reconstruct the exact original tree from that string (deserialize) — useful for saving a tree to disk or sending it over a network.

Brute Force Approach NEW

You might try to serialize using just one traversal's sequence (e.g. only preorder values), but a single traversal sequence alone is generally *not enough* to uniquely reconstruct a tree's shape (multiple different trees can share the same preorder sequence, for instance), so this naive approach can fail to reconstruct correctly.

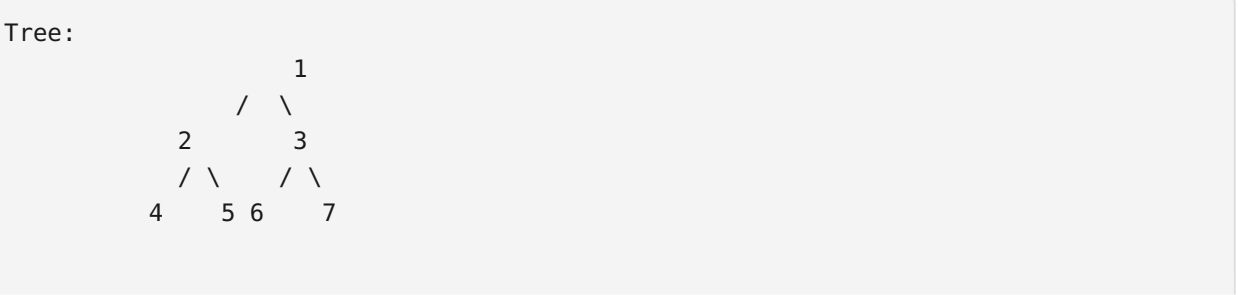
Why We Optimize NEW

The fix: serialize using preorder traversal, but explicitly include **markers for null/missing children** (e.g. writing 'N' whenever a child is absent, not just skipping it silently) — this extra information about *where* the gaps are is exactly what disambiguates the tree's shape, making a single traversal sufficient after all. Deserializing then just replays the same sequence with a pointer/index, recursively rebuilding left and right children (creating a null node whenever the 'N' marker is encountered), mirroring exactly how the tree was written out. Both serialize and deserialize run in O(n) time — the key lesson being that a traversal alone is ambiguous, but a traversal *plus explicit null markers* is not.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert tree to string and back.

Visual Diagram



```
       Serialized: [1,2,3,4,5,6,7] (level order)

       Using preorder with null markers:
       Serialized: [1,2,4,null,null,5,null,null,3,6,null,null,7,null,null]

       Deserialize:
       Build from preorder sequence
```

Solution Approach

Concept: Preorder traversal with null markers.

Algorithm

```
Algorithm SERIALIZE(root):
           result = []
           serializeHelper(root, result)
           Return String.join(",", result)

       Algorithm serializeHelper(node, result):
           If node == null:
               result.add("null")
               Return
           result.add(String.valueOf(node.data))
           serializeHelper(node.left, result)
           serializeHelper(node.right, result)

       Algorithm DESERIALIZE(data):
           queue = Queue of data split by comma
           Return deserializeHelper(queue)

       Algorithm deserializeHelper(queue):
           val = queue.poll()
           If val == "null": Return null
           node = new Node(Integer(val))
           node.left = deserializeHelper(queue)
           node.right = deserializeHelper(queue)
           Return node
```

Java Implementation

```
class Solution {
           // Serialize
           public String serialize(Node root) {
               StringBuilder sb = new StringBuilder();
               serializeHelper(root, sb);
               return sb.toString();
           }

          private void serializeHelper(Node node, StringBuilder sb) {
```

```
            if (node == null) {
                sb.append("null,");
                return;
            }
            sb.append(node.data).append(",");
            serializeHelper(node.left, sb);
            serializeHelper(node.right, sb);
        }

        // Deserialize
        public Node deserialize(String data) {
            Queue<String> queue = new
LinkedList<>(Arrays.asList(data.split(",")));
            return deserializeHelper(queue);
        }

        private Node deserializeHelper(Queue<String> queue) {
            String val = queue.poll();
            if (val.equals("null")) return null;
            Node node = new Node(Integer.parseInt(val));
            node.left = deserializeHelper(queue);
            node.right = deserializeHelper(queue);
            return node;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

280-281. Morris Traversals

Understanding the Problem NEW

Perform tree traversals (Preorder, Inorder) using O(1) extra space — no recursion (which uses O(h) call-stack space) and no explicit stack (which uses O(h) space too).

Brute Force Approach NEW

Ordinary recursive or stack-based traversals (as in problems 248-249, 252-255) both use O(h) extra space for tracking 'where to return to' — perfectly fine for most trees, but this problem specifically challenges you to eliminate that space entirely.

Why We Optimize NEW

Morris Traversal achieves this with a clever trick: instead of using a stack to remember 'where to come back to' after finishing a left subtree, temporarily create a **threaded link** — find the current node's in-order predecessor (the rightmost node in its left subtree) and set *that* node's normally-null right pointer to point back to the current node. This lets you navigate back up the tree later without any stack, since the tree's own (temporarily borrowed) pointers now encode the return path. Once you've used a threaded link to return to a node, you detect and remove it (restoring the tree to its original structure) before moving on. This achieves true O(1) extra space (beyond the traversal output itself) at the cost of temporarily modifying and then carefully restoring the tree's pointers — a genuinely different category of technique from anything else in this chapter, trading a bit of pointer-manipulation complexity for a real space-complexity improvement from O(h) to O(1).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree without recursion or stack (O(1) space).

Visual Diagram

```
Morris Inorder:
                1
```

```
               /    \
             2       3
            / \     / \
          4     5 6    7

       Algorithm: Create threaded links to predecessor
       1. If left child exists, find predecessor (rightmost in left subtree)
       2. Set predecessor.right = current (create thread)
       3. Move left
       4. If no left child, visit node, move right
```

Java Implementation

```
class Solution {
           // Morris Inorder Traversal
           public List<Integer> morrisInorder(Node root) {
               List<Integer> result = new ArrayList<>();
               Node current = root;

                while (current != null) {
                    if (current.left == null) {
                        result.add(current.data);
                        current = current.right;
                    } else {
                        // Find predecessor
                        Node predecessor = current.left;
                        while (predecessor.right != null && predecessor.right !=
current) {
                             predecessor = predecessor.right;
                        }

                           if (predecessor.right == null) {
                               // Create thread
                               predecessor.right = current;
                               current = current.left;
                           } else {
                               // Remove thread and visit
                               predecessor.right = null;
                               result.add(current.data);
                               current = current.right;
                           }
                    }
                }
                return result;
           }

           // Morris Preorder Traversal
           public List<Integer> morrisPreorder(Node root) {
               List<Integer> result = new ArrayList<>();
               Node current = root;

                while (current != null) {
```

```
                    if (current.left == null) {
                        result.add(current.data);
                        current = current.right;

                } else {
                    Node predecessor = current.left;
                    while (predecessor.right != null && predecessor.right !=
current) {
                         predecessor = predecessor.right;
                    }

                    if (predecessor.right == null) {
                        result.add(current.data);
                        predecessor.right = current;
                        current = current.left;
                    } else {
                        predecessor.right = null;
                        current = current.right;
                    }
                }
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

282. Flatten Binary Tree to LinkedList

Understanding the Problem NEW

Rearrange a binary tree in-place into a 'linked list' shape — every node's left child becomes null, and its right child becomes the next node in preorder traversal order (so following right pointers alone reconstructs the exact preorder sequence).

Brute Force Approach NEW

You could first compute the full preorder traversal into a separate list (O(n) extra space), then rebuild the tree's pointers to match that list's order — correct, but uses O(n) extra space for the intermediate list when the rearrangement could be done without it.

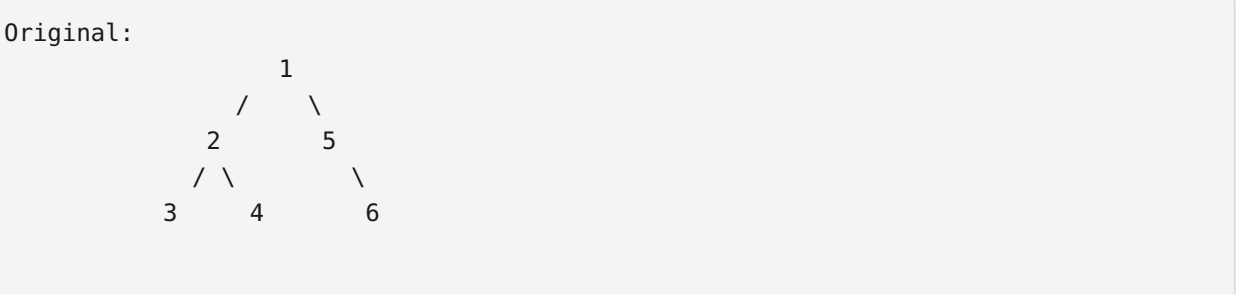
Why We Optimize NEW

A clean in-place approach reuses the exact same trick as Morris Traversal (280-281): for each node, if it has a left child, find the **rightmost node in that left subtree** (the predecessor) and attach the current node's original right subtree there — then move the entire left subtree over to become the new right subtree, and clear the left pointer. Repeating this at every node (moving to the new right child each time) flattens the tree in a single O(n) pass using O(1) extra space, by directly rewiring existing pointers in a Morris-traversal-like manner rather than building an intermediate list and reconstructing from it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Flatten tree to linked list in preorder.

Visual Diagram



```
        Flattened (right child as next):
        1 → 2 → 3 → 4 → 5 → 6

        Morris-like traversal:
        For each node:
          If left exists:
            Find rightmost of left subtree
            Connect rightmost.right = node.right
            node.right = node.left
            node.left = null
          Move to right
```

Solution Approach

Concept: In-place Morris-like flattening.

Algorithm

```
Algorithm FLATTEN(root):
            current = root

             While current != null:
                 If current.left != null:
                     // Find rightmost of left subtree
                     rightmost = current.left
                     While rightmost.right != null:
                         rightmost = rightmost.right

                       // Connect rightmost to current.right
                       rightmost.right = current.right
                       current.right = current.left
                       current.left = null

                  current = current.right
```

Java Implementation

```
class Solution {
            public void flatten(Node root) {
                Node current = root;

                  while (current != null) {
                      if (current.left != null) {
                          // Find rightmost of left subtree
                          Node rightmost = current.left;
                          while (rightmost.right != null) {
                              rightmost = rightmost.right;
                          }

                      // Connect rightmost to current.right
```

```
                    rightmost.right = current.right;
                    current.right = current.left;
                    current.left = null;
                }
                current = current.right;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

Binary Tree Summary

Problem	Difficulty	Time	Space	Key Technique
Traversals	Basic	O(n)	O(h)	DFS/BFS
Height	Medium	O(n)	O(h)	Recursion
Balanced	Medium	O(n)	O(h)	Recursive check
Diameter	Medium	O(n)	O(h)	Height + path
Max Path Sum	Medium	O(n)	O(h)	Return max gain
Identical	Medium	O(n)	O(h)	Recursive comparison
Zigzag	Medium	O(n)	O(n)	Queue + toggle
Boundary	Medium	O(n)	O(h)	Three parts
Vertical	Medium	O(n log n)	O(n)	HD map
Top View	Medium	O(n)	O(n)	BFS + first
Bottom View	Medium	O(n)	O(n)	BFS + last
Left/Right View	Medium	O(n)	O(n)	Level order
Symmetric	Medium	O(n)	O(h)	Mirror check
Root to Path	Hard	O(n)	O(h)	Backtracking
LCA	Hard	O(n)	O(h)	Recursive search
Max Width	Hard	O(n)	O(n)	Indexing
Children Sum	Hard	O(n)	O(h)	Recursive check
Distance K	Hard	O(n)	O(n)	Parent map + BFS
Burn Time	Hard	O(n)	O(n)	Parent map + BFS
Count Complete	Hard	$O(\log^2 n)$	O(log n)	Height property
Build Tree	Hard	O(n)	O(n)	Mapping indexes
Serialize	Hard	O(n)	O(n)	Preorder with null
Morris Traversal	Hard	O(n)	O(1)	Threaded links
Flatten	Hard	O(n)	O(1)	Morris-like

Graphs — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

299. Graph and Types

Understanding the Problem NEW

A Graph is the most general connection-based structure covered so far — unlike a Tree (which has one root and no cycles), a graph is simply a set of 'nodes' (vertices) connected by 'edges,' with no restrictions: any node can connect to any other node, cycles are allowed, and edges can be one-directional (Directed Graph) or two-directional (Undirected Graph), and may carry weights (costs) or not.

Brute Force Approach NEW

Nothing to contrast against here — this is foundational vocabulary. It's worth explicitly noting that a Tree is actually just a special, restricted kind of graph (connected, no cycles, exactly one path between any two nodes) — everything learned about trees generalizes, with extra care needed once cycles become possible.

Why We Optimize NEW

Graphs model an enormous range of real-world problems — social networks, maps/road systems, dependency chains, computer networks — which is why this is one of the largest chapters in the book. The biggest conceptual jump from trees: since graphs can have cycles, naive recursive traversal (which worked fine on trees) can now loop forever unless you explicitly track **visited nodes** — a single new piece of bookkeeping that becomes essential in almost every algorithm in this chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding graph data structure and its types.

Visual Diagram

```
Graph Types:
       1. Undirected Graph:
          A --- B
          |     |
          C --- D
```

```
2. Directed Graph (Digraph):
   A → B
   ↓   ↓
   C → D

3. Weighted Graph:
   A --5-- B
   |       |
   2       3
   |       |
   C --4-- D

4. Cyclic vs Acyclic:
   Cyclic: A → B → C → A
   Acyclic: A → B → C

Graph Terminology:
- Vertices/Nodes: A, B, C, D
- Edges: Connections between vertices
- Degree: Number of edges incident to vertex
- Path: Sequence of vertices
- Cycle: Path that starts and ends at same vertex
```

300-301. Graph Representation

Understanding the Problem NEW

Before writing any graph algorithm, you need a way to actually store the graph in memory. The two standard representations are an **Adjacency Matrix** (an n×n grid where cell [i][j] indicates whether an edge exists between i and j) and an **Adjacency List** (for each node, a list of its directly connected neighbours).

Brute Force Approach NEW

An adjacency matrix is simple to reason about and offers O(1) 'is there an edge between i and j?' lookups, but it always uses O(n²) space regardless of how many edges actually exist — wasteful for 'sparse' graphs (relatively few edges compared to the number of possible pairs).

Why We Optimize NEW

An **Adjacency List** uses O(n + edges) space instead — only storing the connections that actually exist — which is dramatically better for sparse graphs (most real-world graphs are sparse: a social network doesn't have every person connected to every other person). The trade-off is that checking 'is there a specific edge between i and j' becomes O(degree of i) instead of O(1) — but since most graph algorithms need to 'visit all neighbours of a node' far more often than 'check one specific pair,' the adjacency list's efficient neighbour iteration usually wins out, which is why it's the default representation used throughout the rest of this chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Different ways to represent graphs.

Visual Diagram

```
Graph:
           0 --- 1
           |     |
           3 --- 2

       Adjacency Matrix (4x4):
```

```
    0 1 2 3
0: 0 1 0 1
1: 1 0 1 0
2: 0 1 0 1
3: 1 0 1 0

Adjacency List:
0 → [1, 3]
1 → [0, 2]
2 → [1, 3]

3 → [0, 2]

Edge List:
[(0,1), (0,3), (1,2), (2,3)]
```

Java Implementation

```
// Adjacency List
class Graph {
    private List<List<Integer>> adjList;
    private int vertices;

    Graph(int vertices) {
        this.vertices = vertices;
        adjList = new ArrayList<>();
        for (int i = 0; i < vertices; i++) {
            adjList.add(new ArrayList<>());
        }
    }

    void addEdge(int u, int v) {
        adjList.get(u).add(v);
        adjList.get(v).add(u); // For undirected
    }

    List<Integer> getNeighbors(int v) {
        return adjList.get(v);
    }
}

// Adjacency Matrix
class GraphMatrix {
    private int[][] matrix;
    private int vertices;

    GraphMatrix(int vertices) {
        this.vertices = vertices;
        matrix = new int[vertices][vertices];
    }

    void addEdge(int u, int v) {
```

```
            matrix[u][v] = 1;
            matrix[v][u] = 1; // For undirected
        }
    }
```

Time & Space Complexity

Time Complexity:

302. Connected Components

Understanding the Problem NEW

Find all the 'connected components' of a graph — separate clusters of nodes where every node in a cluster can reach every other node in the same cluster, but nodes in different clusters can't reach each other at all.

Brute Force Approach NEW

You could check every pair of nodes for reachability individually — $O(n^2)$ pairs, each requiring a traversal to check connectivity, giving a very expensive $O(n^3)$ or worse overall.

Why We Optimize NEW

Instead of checking pairs, do a single traversal (BFS or DFS) starting from any unvisited node — this traversal will naturally reach *every* node in that node's entire connected component, all in one go. Mark every visited node, then repeat by starting a new traversal from the next unvisited node (a new component), continuing until every node has been visited. This finds every connected component in a total of $O(V + E)$ — each node and edge is only ever processed once across all the traversals combined, rather than repeatedly testing pairwise reachability.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find connected components in undirected graph.

Visual Diagram

```
Graph:
      Component 1: 0-1-2
      Component 2: 3-4
      Component 3: 5

      Visited array:
      0: visited
      1: visited
      2: visited
      3: visited
```

```
      4: visited
      5: visited

      Components count = 3
```

Solution Approach

Concept: DFS/BFS on unvisited nodes.

Algorithm

```
Algorithm COUNT_COMPONENTS(graph):
          visited = new boolean[V]
          count = 0

          For i = 0 to V-1:
              If !visited[i]:
                  DFS(i, visited)
                  count++
          Return count
```

Java Implementation

```
class Solution {
          public int countComponents(int n, int[][] edges) {
              List<List<Integer>> graph = new ArrayList<>();
              for (int i = 0; i < n; i++) {
                  graph.add(new ArrayList<>());
              }
              for (int[] edge : edges) {
                  graph.get(edge[0]).add(edge[1]);
                  graph.get(edge[1]).add(edge[0]);
              }

                 boolean[] visited = new boolean[n];
                 int count = 0;

                 for (int i = 0; i < n; i++) {
                     if (!visited[i]) {
                         dfs(i, graph, visited);
                         count++;
                     }
                 }
                 return count;
          }

          private void dfs(int node, List<List<Integer>> graph, boolean[] visited)
{
              visited[node] = true;
              for (int neighbor : graph.get(node)) {
```

```
                if (!visited[neighbor]) {
                    dfs(neighbor, graph, visited);
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

303. BFS (Breadth First Search)

Understanding the Problem NEW

Traverse a graph breadth-first — visiting the starting node, then all of its direct neighbours, then all of *their* unvisited neighbours, and so on, level by level, exactly like Level Order traversal on a tree (problem 251), but now with a graph's added possibility of cycles.

Brute Force Approach NEW

Without tracking visited nodes, a naive traversal could revisit the same nodes repeatedly through a cycle, looping forever — the entire challenge here (unlike the tree version) is correctly handling cycles.

Why We Optimize NEW

Use the same **queue-based approach** as tree Level Order, but add a **visited set**: mark a node visited the moment it's *enqueued* (not when dequeued) to prevent the same node from being added to the queue multiple times through different paths. This guarantees each node and edge is processed exactly once, achieving O(V + E) time — the visited-set is the one new ingredient graphs require that trees didn't need, since a tree's lack of cycles made accidental revisiting structurally impossible.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse graph level by level.

Visual Diagram

```
Graph:
           0 --- 1
           |     |
           3 --- 2

       BFS from 0:
       Queue: [0]
       Visit 0, add neighbors: [1,3]
       Visit 1, add neighbors: [2]
       Visit 3
```

```
       Visit 2

       Order: 0, 1, 3, 2

       BFS Tree:
            0
           / \
         1     3
         |
         2
```

Solution Approach

Concept: Queue for level order traversal.

Algorithm

```
Algorithm BFS(start):
           queue = Queue()
           visited = boolean[V]
           queue.add(start)
           visited[start] = true

           While queue not empty:
               node = queue.poll()
               print(node)
               For neighbor in graph[node]:
                   If !visited[neighbor]:
                       visited[neighbor] = true
                       queue.add(neighbor)
```

Java Implementation

```
class Solution {
           public void bfs(int start, List<List<Integer>> graph) {
               boolean[] visited = new boolean[graph.size()];
               Queue<Integer> queue = new LinkedList<>();
               queue.add(start);
               visited[start] = true;

                 while (!queue.isEmpty()) {
                     int node = queue.poll();
                     System.out.print(node + " ");

                     for (int neighbor : graph.get(node)) {

                      if (!visited[neighbor]) {
                          visited[neighbor] = true;
                          queue.add(neighbor);
                      }
```

```
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

304. DFS (Depth First Search)

Understanding the Problem NEW

Traverse a graph depth-first — go as deep as possible down one path before backtracking, similar to the recursive Preorder traversal idea from trees (problem 248), but again needing to handle cycles.

Brute Force Approach NEW

Same risk as BFS without care: naive recursion could loop forever through a cycle without a way to detect 'I've already been here.'

Why We Optimize NEW

Use the same **recursive (or explicit-stack) approach** as tree preorder, adding a **visited set** exactly like BFS does: mark each node visited as soon as you arrive, and only recurse into neighbours that haven't been visited yet. This achieves O(V + E) time just like BFS — the key structural choice between BFS and DFS is queue vs. stack/recursion, but the 'add a visited set to handle cycles' fix is identical for both, and this same DFS-with-visited-tracking template becomes the backbone for a large fraction of the remaining problems in this chapter (cycle detection, topological sort, bridges, articulation points).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse graph using recursion/stack.

Visual Diagram

```
Graph:
           0 --- 1
           |     |
           3 --- 2

       DFS from 0:
       Visit 0
       Visit neighbor 1
       Visit neighbor 2
       Visit neighbor 3
       Backtrack
```

```
       Order: 0, 1, 2, 3

       DFS Tree:
           0
           |
           1
           |
           2
           |
           3
```

Solution Approach

Concept: Stack/Recursion for depth traversal.

Algorithm

```
Algorithm DFS(node, visited):
           visited[node] = true
           print(node)
           For neighbor in graph[node]:
               If !visited[neighbor]:
                   DFS(neighbor, visited)
```

Java Implementation

```
class Solution {
           // Recursive DFS
           public void dfs(int node, List<List<Integer>> graph, boolean[] visited)
{
               visited[node] = true;
               System.out.print(node + " ");

                 for (int neighbor : graph.get(node)) {
                     if (!visited[neighbor]) {
                         dfs(neighbor, graph, visited);
                     }
                 }
           }

           // Iterative DFS
           public void dfsIterative(int start, List<List<Integer>> graph) {
               boolean[] visited = new boolean[graph.size()];
               Stack<Integer> stack = new Stack<>();

              stack.push(start);

              while (!stack.isEmpty()) {
                  int node = stack.pop();
```

```
                if (!visited[node]) {
                    visited[node] = true;
                    System.out.print(node + " ");

                    for (int neighbor : graph.get(node)) {
                        if (!visited[neighbor]) {
                            stack.push(neighbor);
                        }
                    }
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

305. Number of Provinces

Understanding the Problem NEW

Given an adjacency matrix representing connections between 'provinces' (cities directly connected if 1, else 0), count how many separate provinces (connected groups) exist — a direct real-world framing of Connected Components (302).

Brute Force Approach NEW

Check every pair of cities for a connecting path individually — $O(n^2)$ to $O(n^3)$.

Why We Optimize NEW

Exactly the same technique as problem 302: run DFS or BFS from each unvisited city, marking every city reachable from it as part of the same province, and count how many separate traversals were needed. Recognizing that 'count provinces' is literally 'count connected components' with different words is the main insight — no new algorithm required.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count connected components in graph (provinces).

Visual Diagram

```
Matrix:
       [1,1,0]
       [1,1,0]
       [0,0,1]

       Province 1: [0,1]
       Province 2: [2]
       Count = 2

       DFS from 0: visits 0,1
       DFS from 2: visits 2
```

Solution Approach

Concept: DFS on adjacency matrix.

Algorithm

```
Algorithm FIND_PROVINCES(isConnected):
           n = isConnected.length
           visited = new boolean[n]
           count = 0

           For i = 0 to n-1:
               If !visited[i]:
                   dfs(i, isConnected, visited)
                   count++
           Return count
```

Java Implementation

```
class Solution {
           public int findCircleNum(int[][] isConnected) {
               int n = isConnected.length;
               boolean[] visited = new boolean[n];
               int count = 0;

                 for (int i = 0; i < n; i++) {
                     if (!visited[i]) {
                         dfs(i, isConnected, visited);
                         count++;
                     }
                 }
                 return count;
           }

           private void dfs(int node, int[][] isConnected, boolean[] visited) {
               visited[node] = true;
               for (int i = 0; i < isConnected.length; i++) {
                   if (isConnected[node][i] == 1 && !visited[i]) {
                       dfs(i, isConnected, visited);
                   }

               }
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

306. Connected Components in Matrix

Understanding the Problem NEW

Given a 2D binary grid (representing land 'connected component regions'), find the connected components — but now 'neighbours' means adjacent grid cells (up/down/left/right), not an explicit edge list.

Brute Force Approach NEW

You could try to convert the grid into an explicit graph (with nodes for each cell and edges to adjacent cells) and then run standard connected-components logic — this works but adds unnecessary conversion overhead.

Why We Optimize NEW

Treat the grid **directly** as an implicit graph: 'neighbours' of a cell are simply its up/down/left/right adjacent cells (computed via simple index arithmetic, no explicit edge list needed), and run DFS/BFS exactly as in problem 302, marking visited cells directly in the grid (or a separate visited grid). This is the first of several grid-based graph problems in this chapter — recognizing that a grid *is* a graph in disguise (with implicit adjacency via neighbouring indices) avoids the overhead of explicitly building a graph structure for something whose connectivity is already implied by position.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find connected components in a binary matrix.

Visual Diagram

```
Matrix:
       1 1 0
       0 1 0
       0 0 1

       Components: 2
       Component 1: (0,0), (0,1), (1,1)
       Component 2: (2,2)
```

```
       DFS: Visit all connected 1s
```

Solution Approach

Concept: DFS on grid with 4 directions.

Algorithm

```
Algorithm COUNT_COMPONENTS_MATRIX(grid):
           m = grid.length, n = grid[0].length
           visited = boolean[m][n]
           count = 0

           For i = 0 to m-1:
               For j = 0 to n-1:
                   If grid[i][j] == 1 and !visited[i][j]:
                        dfs(i, j, grid, visited)
                        count++
           Return count
```

Java Implementation

```
class Solution {
           public int countComponents(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               boolean[][] visited = new boolean[m][n];
               int count = 0;

                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         if (grid[i][j] == 1 && !visited[i][j]) {
                             dfs(i, j, grid, visited);
                             count++;
                         }
                     }
                 }
                 return count;
           }

           private void dfs(int i, int j, int[][] grid, boolean[][] visited) {
               int m = grid.length, n = grid[0].length;
               if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] == 0 ||
visited[i][j]) {
                   return;
               }

                 visited[i][j] = true;

              int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
```

```
            for (int[] dir : dirs) {
                dfs(i + dir[0], j + dir[1], grid, visited);
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

307. Rotten Oranges

Understanding the Problem NEW

Given a grid where cells are empty, contain a fresh orange, or a rotten orange, and rot spreads to adjacent fresh oranges each minute, find the minimum time for all oranges to rot (or -1 if impossible).

Brute Force Approach NEW

Simulate minute by minute, scanning the *entire* grid on every single minute to find which fresh oranges are now adjacent to a rotten one — O(minutes × grid size), redundantly rescanning the whole grid repeatedly even though most cells aren't relevant on most minutes.

Why We Optimize NEW

This calls for **Multi-Source BFS**: instead of starting BFS from just one node, start it simultaneously from *every* already-rotten orange at once (enqueue them all before beginning), then proceed with standard BFS — since BFS naturally processes level by level, and each level here corresponds exactly to one minute passing, this correctly simulates simultaneous spreading from multiple sources without redundant rescanning. This achieves O(rows × cols) total, visiting each cell once, instead of repeatedly rescanning the whole grid every single minute — 'multi-source BFS' (seed the queue with several starting points before the traversal begins) is a small but powerful variation on ordinary BFS that reappears in problem 311.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Time to rot all oranges in a grid.

Visual Diagram

```
Grid:
       [2,1,1]
       [1,1,0]
       [0,1,1]

       Time 0: (0,0) is rotten
```

```
       Time 1: (0,1), (1,0) rot
       Time 2: (0,2), (1,1), (2,1) rot
       Time 3: (2,2) rot
       Output: 3

       BFS with multiple sources:
       Queue all rotten oranges
       Level by level traversal
```

Solution Approach

Concept: Multi-source BFS.

Algorithm

```
Algorithm ROTTEN_ORANGES(grid):
           queue = Queue()
           fresh = 0
           time = 0

            For each cell:
                If grid[i][j] == 2: queue.add((i,j))
                If grid[i][j] == 1: fresh++

            While queue not empty and fresh > 0:
                size = queue.size()
                For each in queue:
                    Try all 4 directions
                    If neighbor is fresh:
                        grid[neighbor] = 2
                        queue.add(neighbor)
                        fresh--
                time++

            Return fresh == 0 ? time : -1
```

Java Implementation

```
class Solution {
           public int orangesRotting(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               Queue<int[]> queue = new LinkedList<>();
               int fresh = 0;

                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         if (grid[i][j] == 2) {
                             queue.add(new int[]{i, j});
                         } else if (grid[i][j] == 1) {
```

```
                        fresh++;
                    }
                }
            }

            if (fresh == 0) return 0;
            int time = 0;
            int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

            while (!queue.isEmpty() && fresh > 0) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    int[] cell = queue.poll();
                    for (int[] dir : dirs) {
                        int x = cell[0] + dir[0];
                        int y = cell[1] + dir[1];
                        if (x >= 0 && x < m && y >= 0 && y < n && grid[x][y] ==
1) {
                            grid[x][y] = 2;
                            queue.add(new int[]{x, y});
                            fresh--;
                        }
                    }
                }
                time++;
            }
            return fresh == 0 ? time : -1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

308. Flood Fill

Understanding the Problem NEW

Given a grid (like an image) and a starting pixel, change the color of that pixel and every pixel connected to it (via matching color, adjacent cells) to a new color — the 'paint bucket' tool in image editors.

Brute Force Approach NEW

There's no slower alternative worth contrasting — this is essentially Connected Components (302/306) applied directly, where the 'connection' is 'adjacent cell with the same original color.'

Why We Optimize NEW

Run DFS (or BFS) from the starting pixel, only recursing into neighbouring cells that share the *original* color (checked before it gets overwritten, to avoid immediately re-matching the new color and causing infinite recursion if old and new colors happen to be checked incorrectly), changing each visited cell's color as you go. This is a direct, minimal-twist application of grid-based connected-component traversal (306) — the 'new insight' here is really just recognizing that a common image-editing tool is exactly a graph traversal problem in disguise.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Fill connected region with new color.

Visual Diagram

```
Image:
       [1,1,1]
       [1,1,0]
       [1,0,1]
       Start: (1,1), NewColor: 2

       Result:
       [2,2,2]
       [2,2,0]
       [2,0,1]
```

```
       DFS: Replace all connected 1s with 2
```

Solution Approach

Concept: DFS with color replacement.

Algorithm

```
Algorithm FLOOD_FILL(image, sr, sc, newColor):
           if image[sr][sc] == newColor: return image
           oldColor = image[sr][sc]
           dfs(image, sr, sc, oldColor, newColor)
           return image

       Algorithm dfs(image, i, j, oldColor, newColor):
           if out of bounds or image[i][j] != oldColor: return
           image[i][j] = newColor
           dfs all 4 directions
```

Java Implementation

```
class Solution {
           public int[][] floodFill(int[][] image, int sr, int sc, int newColor) {
               int oldColor = image[sr][sc];
               if (oldColor == newColor) return image;
               dfs(image, sr, sc, oldColor, newColor);
               return image;
           }

           private void dfs(int[][] image, int i, int j, int oldColor, int
newColor) {
               int m = image.length, n = image[0].length;
               if (i < 0 || i >= m || j < 0 || j >= n || image[i][j] != oldColor) {
                   return;
               }

                 image[i][j] = newColor;
                 int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
                 for (int[] dir : dirs) {
                     dfs(image, i + dir[0], j + dir[1], oldColor, newColor);
                 }
           }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

309-310. Cycle Detection in Undirected Graph

Understanding the Problem NEW

Detect whether an undirected graph contains a cycle (a path that starts and ends at the same node without reusing any edge).

Brute Force Approach NEW

You could try to enumerate every possible path and check if any revisits a node — exponential, since the number of possible paths in a graph can be very large.

Why We Optimize NEW

During a DFS traversal, if you ever encounter an **already-visited node that isn't the node you just came from** (i.e., not simply your immediate parent in the traversal), you've found a cycle — an edge going back to some earlier-visited node other than your direct predecessor. This requires **tracking the parent** of each DFS call explicitly (since in an undirected graph, the edge back to your immediate parent is expected and not a cycle — you arrived via that edge, so seeing it again isn't a 'true' new cycle). This detects a cycle in a single O(V+E) DFS traversal, instead of exponentially enumerating paths — this 'track parent, flag a cycle if you see a visited non-parent node' technique is specific to *undirected* graphs; directed graphs need a different approach (three-state coloring, problem 318), since a directed edge's 'reverse' direction isn't automatically expected the way it is here.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Detect cycles in undirected graph.

Visual Diagram

```
Graph with cycle:
       0 --- 1
       |     |
       3 --- 2

       DFS from 0:
       Visit 0
```

```
Visit 1
Visit 2
Visit 3 → neighbor 0 already visited and not parent → cycle

Graph without cycle:
0 --- 1
|     |
3     2

DFS from 0:
Visit 0
Visit 1
Visit 2
Backtrack
Visit 3 → no cycle
```

Solution Approach

Concept: DFS with parent tracking.

Algorithm (DFS)

Algorithm HAS_CYCLE_DFS(graph):

visited = boolean[V]

For i = 0 to V-1:

If !visited[i]:

If dfs(i, -1, visited): Return true

Return false

Algorithm dfs(node, parent, visited):

visited[node] = true

For neighbor in graph[node]:

If !visited[neighbor]:

If dfs(neighbor, node, visited): Return true

Else If neighbor != parent:

Return true

Return false

Java Implementation

```
class Solution {
           public boolean hasCycle(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(edge[1]);

                  graph.get(edge[1]).add(edge[0]);
              }

              boolean[] visited = new boolean[n];
              for (int i = 0; i < n; i++) {
                  if (!visited[i]) {
                      if (dfs(i, -1, graph, visited)) {
                          return true;
                      }
                  }
              }
              return false;
          }

          private boolean dfs(int node, int parent, List<List<Integer>> graph,
boolean[] visited) {
              visited[node] = true;
              for (int neighbor : graph.get(node)) {
                  if (!visited[neighbor]) {
                      if (dfs(neighbor, node, graph, visited)) {
                          return true;
                      }
                  } else if (neighbor != parent) {
                      return true;
                  }
              }
              return false;
          }
      }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

311. 0/1 Matrix

Understanding the Problem NEW

Given a binary matrix, for every cell find the distance to the nearest 0 (using standard grid adjacency, one step per adjacent cell).

Brute Force Approach NEW

For each cell containing a 1, run a fresh BFS (or scan outward) to find the nearest 0 — O(cells) BFS calls, each potentially O(cells), giving O(cells²) overall — heavily redundant since many searches will re-explore overlapping territory.

Why We Optimize NEW

Exactly the **Multi-Source BFS** technique from Rotten Oranges (307): seed the BFS queue with *every* cell that's already a 0 (distance 0) simultaneously, then expand outward level by level — each cell's distance is naturally determined by which BFS 'wave' first reaches it. This computes every cell's distance in a single O(rows × cols) pass, instead of O(cells²) from re-running BFS individually from every 1-cell — reinforcing multi-source BFS as a reusable technique whenever you need 'distance to nearest occurrence of X' across an entire grid at once, rather than per-cell searches.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find distance of each cell to nearest 0.

Visual Diagram

```
Matrix:
       [0,0,0]
       [0,1,0]
       [1,1,1]

       Result:
       [0,0,0]
       [0,1,0]
       [1,2,1]

       BFS from all zeros:
```

```
        Add all 0s to queue
        BFS level by level
        Distance = level
```

Solution Approach

Concept: Multi-source BFS.

Algorithm

```
Algorithm UPDATE_MATRIX(matrix):
            m = matrix.length, n = matrix[0].length
            dist = new int[m][n]
            queue = Queue()

            For each cell:
                If matrix[i][j] == 0:
                    dist[i][j] = 0
                    queue.add((i,j))
                Else:
                    dist[i][j] = INF

            BFS and update distances
            Return dist
```

Java Implementation

```
class Solution {
            public int[][] updateMatrix(int[][] mat) {
                int m = mat.length, n = mat[0].length;
                int[][] dist = new int[m][n];
                Queue<int[]> queue = new LinkedList<>();

                  for (int i = 0; i < m; i++) {
                      for (int j = 0; j < n; j++) {
                          if (mat[i][j] == 0) {
                              dist[i][j] = 0;
                              queue.add(new int[]{i, j});
                          } else {
                              dist[i][j] = Integer.MAX_VALUE;
                          }
                      }
                  }
                  int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

               while (!queue.isEmpty()) {
                   int[] cell = queue.poll();
                   for (int[] dir : dirs) {
                       int x = cell[0] + dir[0];
                       int y = cell[1] + dir[1];
```

```
                    if (x >= 0 && x < m && y >= 0 && y < n) {
                        if (dist[x][y] > dist[cell[0]][cell[1]] + 1) {
                            dist[x][y] = dist[cell[0]][cell[1]] + 1;
                            queue.add(new int[]{x, y});
                        }
                    }
                }
            }
            return dist;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

312. Surrounded Regions

Understanding the Problem NEW

Given a grid of 'X' and 'O' characters, capture (flip to 'X') every region of 'O's that is fully surrounded by 'X's — but regions of 'O' touching the grid's border should NOT be captured, since they're not actually 'surrounded.'

Brute Force Approach NEW

You might try to check, for each 'O' region, whether it's surrounded by scanning its entire boundary and checking every adjacent cell — awkward to get right directly, especially distinguishing 'touches the border' from 'surrounded' cleanly.

Why We Optimize NEW

Flip the perspective: instead of trying to detect 'surrounded' regions directly, **first mark every 'O' region that touches the border as safe** (run DFS/BFS starting from every border cell that's an 'O', marking everything reachable from there) — since anything connected to the border obviously can't be fully surrounded. Afterward, in a single pass, flip every remaining *unmarked* 'O' to 'X' (these are exactly the truly surrounded ones) and restore the marked (border-connected) ones back to 'O'. This 'identify the safe/exempt cells first via traversal from the boundary, then handle everything else' strategy avoids ambiguous case-by-case surrounded-region detection, and reappears almost identically in problem 313.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Capture surrounded regions (flip 'O' to 'X').

Visual Diagram

```
Board:
       X X X X
       X 0 0 X
       X X 0 X
       X 0 X X

       Result:
```

```
X X X X
X X X X
X X X X
X O X X

Approach:
Mark boundary 'O' and connected 'O' as safe
Flip all other 'O' to 'X'
```

Solution Approach

Concept: DFS from boundaries.

Algorithm

```
Algorithm SURROUNDED_REGIONS(board):
           m = board.length, n = board[0].length

            // Mark boundary 'O's
            For each boundary cell:
                If board[i][j] == 'O':
                    dfs(i, j, board)

            // Flip remaining 'O' to 'X'
            // Flip marked to 'O'
```

Java Implementation

```
class Solution {
           public void solve(char[][] board) {
               int m = board.length, n = board[0].length;

                 // Mark boundary O's and connected ones
                 for (int i = 0; i < m; i++) {
                     if (board[i][0] == 'O') dfs(i, 0, board);
                     if (board[i][n-1] == 'O') dfs(i, n-1, board);
                 }
                 for (int j = 0; j < n; j++) {
                     if (board[0][j] == 'O') dfs(0, j, board);
                     if (board[m-1][j] == 'O') dfs(m-1, j, board);
                 }

                 // Flip remaining
                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         if (board[i][j] == 'O') board[i][j] = 'X';
                         if (board[i][j] == '#') board[i][j] = 'O';
                     }

             }
```

```
        }

        private void dfs(int i, int j, char[][] board) {
            int m = board.length, n = board[0].length;
            if (i < 0 || i >= m || j < 0 || j >= n || board[i][j] != 'O') return;

            board[i][j] = '#';
            int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
            for (int[] dir : dirs) {
                dfs(i + dir[0], j + dir[1], board);
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

313. Number of Enclaves

Understanding the Problem NEW

Count the number of land cells in a grid that *cannot* reach the grid's boundary by moving through adjacent land cells — the complement of 'land connected to the border.'

Brute Force Approach NEW

For every land cell, check if there's a path to the boundary — O(cells) searches, each potentially O(cells), redundant and expensive.

Why We Optimize NEW

Exactly the same 'mark the boundary-connected cells first, then count what's left' strategy as Surrounded Regions (312): run DFS/BFS from every boundary land cell, marking everything reachable; then simply count land cells that were *never*

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count land cells that cannot reach boundary.

Visual Diagram

```
Grid:
       0 0 0 0
       1 0 1 0
       0 1 1 0
       0 0 0 0

       Enclaves: 3 (cells at (1,2), (2,1), (2,2))

       Approach:
       Mark all land connected to boundary
       Count remaining land cells
```

Solution Approach

Concept: DFS from boundary.

Algorithm

```
Algorithm ENCLAVES(grid):
           m = grid.length, n = grid[0].length
           visited = boolean[m][n]

           // DFS from boundary land
           For each boundary:
               If grid[i][j] == 1:
                   dfs(i, j, grid, visited)

           // Count unvisited land
           count = 0
           For each cell:
               If grid[i][j] == 1 and !visited[i][j]:
                   count++
           Return count
```

Java Implementation

```
class Solution {
           public int numEnclaves(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               boolean[][] visited = new boolean[m][n];

                 // Mark boundary connected land
                 for (int i = 0; i < m; i++) {
                     if (grid[i][0] == 1) dfs(i, 0, grid, visited);
                     if (grid[i][n-1] == 1) dfs(i, n-1, grid, visited);
                 }
                 for (int j = 0; j < n; j++) {
                     if (grid[0][j] == 1) dfs(0, j, grid, visited);
                     if (grid[m-1][j] == 1) dfs(m-1, j, grid, visited);
                 }

                 // Count enclaves
                 int count = 0;
                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {

                      if (grid[i][j] == 1 && !visited[i][j]) {
                          count++;
                      }
                    }
               }
               return count;
           }

           private void dfs(int i, int j, int[][] grid, boolean[][] visited) {
               int m = grid.length, n = grid[0].length;
               if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] == 0 ||
```

```
visited[i][j]) {
                return;
            }
            visited[i][j] = true;
            int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
            for (int[] dir : dirs) {
                dfs(i + dir[0], j + dir[1], grid, visited);
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

314. Word Ladder - 1

Understanding the Problem NEW

Given a start word and end word, and a dictionary of valid words, find the length of the shortest sequence of single-letter-changed words (each step must be a valid dictionary word) connecting start to end.

Brute Force Approach NEW

Try every possible sequence of word transformations via exhaustive search (trying every position/letter change combination at each step) — exponential, since the number of possible transformation paths grows very quickly.

Why We Optimize NEW

Treat this as a **graph where each word is a node**, and an edge exists between two words if they differ by exactly one letter — then the shortest transformation sequence is exactly the **shortest path** in this implicit graph, found via ordinary BFS from the start word (since BFS naturally finds shortest paths in unweighted graphs, as established in problem 303/326). Rather than building the whole word-graph explicitly upfront (which could be expensive), you generate a word's 'neighbours' on the fly by trying every possible single-letter substitution at every position and checking if the result is in the dictionary — achieving roughly O(word length × alphabet size × number of words) instead of an exponential path search, by recognizing 'shortest transformation sequence' as literally a shortest-path-in-an-unweighted-graph problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Shortest transformation sequence from beginWord to endWord.

Visual Diagram

```
beginWord = "hit", endWord = "cog"
       wordList = ["hot","dot","dog","lot","log","cog"]

       hit → hot → dot → dog → cog
       Length = 5
```

```
       BFS:
       Level 0: hit
       Level 1: hot
       Level 2: dot, lot
       Level 3: dog, log
       Level 4: cog
```

Solution Approach

Concept: BFS with word transformations.

Algorithm

```
Algorithm LADDER_LENGTH(beginWord, endWord, wordList):
           wordSet = Set(wordList)
           queue = Queue()
           queue.add(beginWord)
           level = 1

           While queue not empty:
               size = queue.size()
               For i = 0 to size-1:
                   word = queue.poll()
                   If word == endWord: Return level
                   For each char in word:
                       For c in 'a' to 'z':
                           next = word with char replaced
                           If wordSet.contains(next):
                               wordSet.remove(next)
                               queue.add(next)
               level++

           Return 0
```

Java Implementation

```
class Solution {
           public int ladderLength(String beginWord, String endWord, List<String>
wordList) {
               Set<String> wordSet = new HashSet<>(wordList);
               if (!wordSet.contains(endWord)) return 0;

                 Queue<String> queue = new LinkedList<>();
                 queue.add(beginWord);
                 int level = 1;

                 while (!queue.isEmpty()) {
                     int size = queue.size();
                     for (int i = 0; i < size; i++) {
                         String word = queue.poll();
```

```
                    if (word.equals(endWord)) return level;

                    char[] chars = word.toCharArray();
                    for (int j = 0; j < chars.length; j++) {
                        char original = chars[j];
                        for (char c = 'a'; c <= 'z'; c++) {
                            chars[j] = c;
                            String next = new String(chars);
                            if (wordSet.contains(next)) {
                                wordSet.remove(next);
                                queue.add(next);
                            }
                        }
                        chars[j] = original;
                    }
                }
                level++;
            }
            return 0;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * L * 26) | Space Complexity: O(n)

315. Word Ladder - 2

Understanding the Problem NEW

Same as Word Ladder-1, but now find *every* shortest transformation sequence (all paths of the minimum length), not just the length of one.

Brute Force Approach NEW

Explore every possible transformation path exhaustively and filter for the ones matching the minimum length found — extremely expensive without first knowing what that minimum length even is.

Why We Optimize NEW

Do this in **two phases**: first, run BFS exactly as in Word Ladder-1 to establish each word's shortest distance from the start (not stopping at just one path, but recording every word's level); then, do a **second pass (DFS or backward reconstruction)** that only follows edges from a word at distance `d` to a word at distance `d+1` — reconstructing every path that strictly increases in BFS-level at each step. This 'BFS to establish distances, then DFS restricted to distance-respecting edges to enumerate all paths' two-phase approach avoids exploring paths that couldn't possibly be shortest, which a blind full-path exploration would waste enormous time on.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all shortest transformation sequences.

Visual Diagram

```
beginWord = "hit", endWord = "cog"
       wordList = ["hot","dot","dog","lot","log","cog"]

       Paths:
       hit → hot → dot → dog → cog
       hit → hot → lot → log → cog

       BFS + DFS:
```

```
        Build parent map during BFS
        DFS to construct paths
```

Solution Approach

Concept: BFS for distances, DFS for paths.

Algorithm

```
Algorithm LADDER_PATHS(beginWord, endWord, wordList):
            Build adjacency list
            BFS to find distances
            DFS from endWord to beginWord
            Return all paths
```

Java Implementation

```
class Solution {
            public List<List<String>> findLadders(String beginWord, String endWord,
List<String> wordLi
                Set<String> wordSet = new HashSet<>(wordList);
                List<List<String>> result = new ArrayList<>();
                if (!wordSet.contains(endWord)) return result;

                  Map<String, List<String>> adj = new HashMap<>();
                  Map<String, Integer> dist = new HashMap<>();

                  // BFS
                  Queue<String> queue = new LinkedList<>();
                  queue.add(beginWord);
                  dist.put(beginWord, 0);

                  while (!queue.isEmpty()) {
                      String word = queue.poll();
                      char[] chars = word.toCharArray();
                      for (int i = 0; i < chars.length; i++) {
                          char original = chars[i];
                          for (char c = 'a'; c <= 'z'; c++) {
                              chars[i] = c;
                              String next = new String(chars);
                              if (wordSet.contains(next) && !next.equals(word)) {
                                  adj.computeIfAbsent(word, k -> new
ArrayList<>()).add(next);
                                  if (!dist.containsKey(next)) {
                                      dist.put(next, dist.get(word) + 1);
                                      queue.add(next);
                                  }
                              }
                          }
```

```
                    chars[i] = original;
                }
            }

            // DFS to build paths
            List<String> path = new ArrayList<>();
            path.add(beginWord);
            dfs(beginWord, endWord, dist, adj, path, result);
            return result;
        }

        private void dfs(String word, String endWord, Map<String, Integer> dist,
                         Map<String, List<String>> adj, List<String> path,
                         List<List<String>> result) {
            if (word.equals(endWord)) {
                result.add(new ArrayList<>(path));
                return;
            }

            for (String next : adj.getOrDefault(word, new ArrayList<>())) {
                if (dist.containsKey(next) && dist.get(next) == dist.get(word) +
1) {
                    path.add(next);
                    dfs(next, endWord, dist, adj, path, result);
                    path.remove(path.size() - 1);
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * L * 26) | Space Complexity: O(n * paths)

316. Number of Distinct Islands

Understanding the Problem NEW

Given a grid of land ('1') and water ('0'), count how many *distinct shapes* of islands exist — two islands with the exact same shape (even if in different grid positions) count as the same shape, only counted once.

Brute Force Approach NEW

You could compare every pair of islands' full cell coordinate sets to check if they're the same shape (accounting for translation) — O(islands²) comparisons, each potentially expensive.

Why We Optimize NEW

During DFS/BFS traversal of each island, record the **relative path/shape** taken (e.g. the sequence of directions moved from the island's starting cell, or each cell's position relative to the island's top-left corner) rather than absolute coordinates — this creates a canonical 'shape signature' that's identical for two islands with the same shape regardless of where they sit in the grid. Store these signatures in a **hash set** and count distinct entries. This turns an O(islands²) pairwise shape-comparison problem into an O(cells) traversal-with-signature-recording problem, since a hash set naturally deduplicates matching signatures in O(1) average per insertion.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count distinct island shapes.

Visual Diagram

```
Grid:
       1 1 0 1 1
       1 0 0 1 1
       0 0 0 0 0

       Island 1: (0,0),(0,1),(1,0) → shape: 1,1,1
       Island 2: (0,3),(0,4),(1,3),(1,4) → shape: 1,1,1,1
```

```
       Distinct islands: 2

       Normalize shape by subtracting base coordinates
```

Solution Approach

Concept: DFS with shape normalization.

Algorithm

```
Algorithm COUNT_DISTINCT_ISLANDS(grid):
           visited = boolean[m][n]
           islands = HashSet()

           For each cell:
               If grid[i][j] == 1 and !visited[i][j]:
                   shape = []
                   dfs(i, j, i, j, shape, visited, grid)
                   islands.add(shape)

           Return islands.size()
```

Java Implementation

```
class Solution {
           public int countDistinctIslands(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               boolean[][] visited = new boolean[m][n];
               Set<List<String>> islands = new HashSet<>();

                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         if (grid[i][j] == 1 && !visited[i][j]) {
                             List<String> shape = new ArrayList<>();
                             dfs(i, j, i, j, grid, visited, shape);
                             islands.add(shape);
                         }
                     }
                 }
                 return islands.size();
           }

           private void dfs(int i, int j, int baseI, int baseJ, int[][] grid,
                            boolean[][] visited, List<String> shape) {
               int m = grid.length, n = grid[0].length;
               if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] == 0 ||
visited[i][j]) {
                   return;

              }
```

```
            visited[i][j] = true;
            shape.add((i - baseI) + "," + (j - baseJ));
            int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
            for (int[] dir : dirs) {
                dfs(i + dir[0], j + dir[1], baseI, baseJ, grid, visited, shape);
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

317. Bipartite Graph (DFS)

Understanding the Problem NEW

Check whether a graph is 'bipartite' — meaning its nodes can be split into two groups such that every edge connects a node in one group to a node in the other (no edge connects two nodes within the same group). Equivalent to asking: can the graph be properly colored with just 2 colors?

Brute Force Approach NEW

Try every possible way of splitting nodes into two groups and check if all edges cross between groups — 2^n possible splits, exponential.

Why We Optimize NEW

Use DFS (or BFS), **assigning alternating colors** as you traverse: color the starting node, then every neighbour must get the *opposite* color; if you ever find a neighbour that's already colored and it **matches** the current node's color (rather than being the opposite), the graph isn't bipartite — an edge would then connect two same-colored nodes, violating the definition. This checks bipartiteness in a single O(V+E) traversal, instead of an exponential search over every possible 2-way split, by building the coloring incrementally and detecting a contradiction the moment one arises.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if graph is bipartite (2-colorable).

Visual Diagram

```
Bipartite:
       0 --- 1
       |     |
       3 --- 2

       Color assignment:
       0: red, 1: blue, 2: red, 3: blue

       Non-bipartite (odd cycle):
       0 --- 1
```

```
|     |
3 --- 2
Cannot color with 2 colors
```

Solution Approach

Concept: DFS with alternating colors.

Algorithm

```
Algorithm IS_BIPARTITE(graph):
           color = new int[V] // -1: uncolored, 0: red, 1: blue

            For i = 0 to V-1:
                If color[i] == -1:
                    If !dfs(i, 0, color, graph):
                        Return false
            Return true

       Algorithm dfs(node, c, color, graph):
           color[node] = c
           For neighbor in graph[node]:
               If color[neighbor] == -1:
                   If !dfs(neighbor, 1-c, color, graph):
                       Return false
               Else If color[neighbor] == c:
                   Return false
           Return true
```

Java Implementation

```
class Solution {
           public boolean isBipartite(int[][] graph) {
               int n = graph.length;
               int[] color = new int[n];
               Arrays.fill(color, -1);

                 for (int i = 0; i < n; i++) {
                     if (color[i] == -1) {
                         if (!dfs(i, 0, color, graph)) {
                              return false;
                         }
                     }
                 }
                 return true;

         }

          private boolean dfs(int node, int c, int[] color, int[][] graph) {
              color[node] = c;
```

```
            for (int neighbor : graph[node]) {
                if (color[neighbor] == -1) {
                    if (!dfs(neighbor, 1 - c, color, graph)) {
                         return false;
                    }
                } else if (color[neighbor] == c) {
                    return false;
                }
            }
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

318. Cycle Detection in Directed Graph (DFS)

Understanding the Problem NEW

Detect a cycle in a *directed* graph — trickier than the undirected version (309-310), since in a directed graph, encountering an already-visited node doesn't automatically mean a cycle (it might just be a different, valid path converging on a node you've already finished exploring elsewhere).

Brute Force Approach NEW

Reusing the undirected 'track only the immediate parent' technique directly would give wrong answers here, since directed graphs can have multiple valid paths converging on the same node without that being a cycle — so a naive reuse of 309-310's technique doesn't work.

Why We Optimize NEW

Use **three-state coloring** during DFS instead of a simple visited/unvisited flag: 'white' (not yet visited), 'gray' (currently being explored, i.e. on the current DFS path / call stack), and 'black' (fully finished exploring). A cycle exists specifically if you ever encounter a **'gray' node** — meaning you've looped back to something still actively on your current path (a true cycle), as opposed to a 'black' node (already fully explored via some other path, which is fine and expected in directed graphs with converging paths). This distinguishes genuine back-edges (cycles) from harmless cross-edges/forward-edges that are completely normal in directed graphs, something the simpler undirected-graph technique can't tell apart.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Detect cycles in directed graph.

Visual Diagram

```
Graph with cycle:
       0 → 1 → 2 → 0

       DFS with 3 states:
       0: unvisited
```

```
       1: visiting (in current recursion stack)
       2: visited (processed)

       0: visiting
       1: visiting
       2: visiting
       0: visiting → cycle found

       Graph without cycle:
       0 → 1 → 2
       All nodes processed → no cycle
```

Solution Approach

Concept: DFS with 3-state coloring.

Algorithm

```
Algorithm HAS_CYCLE_DIRECTED(graph):
           state = new int[V] // 0: unvisited, 1: visiting, 2: visited

           For i = 0 to V-1:
               If state[i] == 0:
                   If dfs(i, state, graph):
                        Return true
           Return false

       Algorithm dfs(node, state, graph):
           state[node] = 1 // visiting
           For neighbor in graph[node]:
               If state[neighbor] == 0:
                   If dfs(neighbor, state, graph): Return true
               Else If state[neighbor] == 1:
                   Return true // back edge
           state[node] = 2 // processed
           Return false
```

Java Implementation

```
class Solution {
           public boolean hasCycle(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(edge[1]);
               }

                 int[] state = new int[n]; // 0: unvisited, 1: visiting, 2: visited
```

```
            for (int i = 0; i < n; i++) {
                if (state[i] == 0) {
                    if (dfs(i, state, graph)) {
                        return true;
                    }
                }
            }
            return false;
        }

        private boolean dfs(int node, int[] state, List<List<Integer>> graph) {
            state[node] = 1; // visiting

            for (int neighbor : graph.get(node)) {
                if (state[neighbor] == 0) {
                    if (dfs(neighbor, state, graph)) {
                        return true;
                    }
                } else if (state[neighbor] == 1) {
                    return true; // back edge
                }
            }
            state[node] = 2; // processed
            return false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

319. Topological Sort

Understanding the Problem NEW

Given a Directed Acyclic Graph (DAG — a directed graph with no cycles), find a 'topological order' — an ordering of all nodes such that every edge points from an earlier node to a later one in the ordering (like a valid sequence of course prerequisites).

Brute Force Approach NEW

Try every possible permutation of nodes and check if it satisfies every edge's ordering constraint — n! permutations, completely infeasible.

Why We Optimize NEW

Run DFS from every unvisited node, and the moment a node **finishes** being fully explored (all its descendants have been visited), push it onto a stack; once all nodes have been processed this way, popping the entire stack gives a valid topological order. This works because a node only gets pushed after everything reachable from it has already been pushed (and will therefore come out of the stack pop *after* — meaning positioned earlier in the resulting order — everything it points to), guaranteeing the ordering constraint for every edge. This achieves the topological order in a single O(V+E) DFS pass (using the 'black/finished' three-state idea from problem 318 to know when a node is truly done), instead of the factorial brute-force permutation check.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Order vertices so all edges go forward.

Visual Diagram

```
Graph:
       0 → 1 → 3
       ↓   ↓
       2   4

       Topological Order: [0, 1, 2, 3, 4] or [0, 2, 1, 3, 4]
```

```
       DFS Algorithm:
       Visit 0 → visit 1 → visit 3 → visit 4 → visit 2
       Add to stack: 4, 3, 2, 1, 0
       Reverse: 0, 1, 2, 3, 4
```

Solution Approach

Concept: DFS with stack.

Algorithm

```
Algorithm TOPO_SORT_DFS(graph):
           visited = boolean[V]
           stack = Stack()

           For i = 0 to V-1:
               If !visited[i]:
                   dfs(i, visited, stack, graph)

           Return stack (reversed)

       Algorithm dfs(node, visited, stack, graph):
           visited[node] = true
           For neighbor in graph[node]:
               If !visited[neighbor]:
                   dfs(neighbor, visited, stack, graph)
           stack.push(node)
```

Java Implementation

```
class Solution {
           public List<Integer> topologicalSort(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(edge[1]);
               }

                 boolean[] visited = new boolean[n];
                 Stack<Integer> stack = new Stack<>();

                 for (int i = 0; i < n; i++) {
                     if (!visited[i]) {

                      dfs(i, graph, visited, stack);
                  }
              }
```

```
            List<Integer> result = new ArrayList<>();
            while (!stack.isEmpty()) {
                result.add(stack.pop());
            }
            return result;
        }

        private void dfs(int node, List<List<Integer>> graph, boolean[] visited,
Stack<Integer> sta
            visited[node] = true;
            for (int neighbor : graph.get(node)) {
                if (!visited[neighbor]) {
                    dfs(neighbor, graph, visited, stack);
                }
            }
            stack.push(node);
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

320. Kahn's Algorithm

Understanding the Problem NEW

An alternative way to compute topological order (same goal as problem 319), this time using BFS instead of DFS.

Brute Force Approach NEW

Same factorial permutation-checking alternative as 319 would apply here too if attempted naively.

Why We Optimize NEW

Kahn's Algorithm: first compute the **in-degree** (number of incoming edges) for every node; any node with in-degree 0 has no unmet prerequisites, so it can safely go first — enqueue all such nodes initially. Then repeatedly dequeue a node, add it to the topological order, and 'remove' its outgoing edges by decrementing the in-degree of each neighbour; any neighbour whose in-degree drops to 0 as a result gets enqueued next. This BFS-based approach achieves the same O(V+E) result as DFS-based topological sort (319), but has the added benefit of naturally detecting cycles too (problem 321) — if not all nodes get processed by the end, some nodes' in-degrees never dropped to 0, meaning a cycle prevented them from ever becoming 'ready.'

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Topological sort using indegree.

Visual Diagram

```
Graph:
       0 → 1 → 3
       ↓   ↓
       2   4

       Indegrees:
       0:0, 1:1, 2:1, 3:1, 4:1

       Queue: [0]
       Pop 0, add neighbors: indegree[1]=0, indegree[2]=0
```

```
       Queue: [1,2]
       Pop 1, add neighbors: indegree[3]=0, indegree[4]=0
       Queue: [2,3,4]
       Pop 2,3,4

       Order: [0, 1, 2, 3, 4]
```

Solution Approach

Concept: Queue with indegree 0 nodes.

Algorithm

```
Algorithm KAHN(graph):
           indegree = new int[V]
           For each edge u→v:
               indegree[v]++

           queue = Queue()
           For i = 0 to V-1:
               If indegree[i] == 0:
                   queue.add(i)

           result = []
           While queue not empty:
               node = queue.poll()
               result.add(node)
               For neighbor in graph[node]:
                   indegree[neighbor]--
                   If indegree[neighbor] == 0:
                       queue.add(neighbor)

           Return result
```

Java Implementation

```
class Solution {
           public List<Integer> topologicalSortKahn(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               int[] indegree = new int[n];
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(edge[1]);

                  indegree[edge[1]]++;
              }

              Queue<Integer> queue = new LinkedList<>();
```

```
            for (int i = 0; i < n; i++) {
                if (indegree[i] == 0) {
                    queue.add(i);
                }
            }

            List<Integer> result = new ArrayList<>();
            while (!queue.isEmpty()) {
                int node = queue.poll();
                result.add(node);
                for (int neighbor : graph.get(node)) {
                    indegree[neighbor]--;
                    if (indegree[neighbor] == 0) {
                        queue.add(neighbor);
                    }
                }
            }
            return result.size() == n ? result : new ArrayList<>();
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

321. Cycle Detection in Directed Graph (Kahn's)

Understanding the Problem NEW

Detect whether a directed graph has a cycle, using Kahn's Algorithm's BFS-based approach rather than the DFS three-color approach from problem 318.

Brute Force Approach NEW

Same exponential-path-enumeration alternative as any cycle-detection problem.

Why We Optimize NEW

Run Kahn's Algorithm (320) exactly as normal, but simply **count how many nodes get successfully processed** (added to the order). If that count equals the total number of nodes, the graph is acyclic (every node eventually had its dependencies satisfied); if it's *less* than the total, some nodes never had their in-degree reach 0 — meaning they're stuck in a cycle where each depends on another that never gets 'freed.' This gives an alternative cycle-detection method to problem 318's DFS coloring, useful particularly because it naturally falls out of running Kahn's Algorithm anyway if you needed the topological order too.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Detect cycle using Kahn's algorithm.

Visual Diagram

```
Graph with cycle:
       0 → 1 → 2 → 0

       Indegrees:
       0:1, 1:1, 2:1
       No node with indegree 0 → cycle exists

       Graph without cycle:
       0 → 1 → 2

       Indegrees:
       0:0, 1:1, 2:1
       Queue: [0]
```

```
       Pop 0 → indegree[1]=0 → queue: [1]
       Pop 1 → indegree[2]=0 → queue: [2]
       Pop 2 → all processed → no cycle
```

Solution Approach

Concept: Kahn's algorithm with count.

Algorithm

```
Algorithm HAS_CYCLE_KAHN(graph):
           indegree = new int[V]
           For each edge u→v: indegree[v]++

           queue = Queue with indegree 0 nodes
           count = 0

           While queue not empty:
               node = queue.poll()
               count++
               For neighbor in graph[node]:
                   indegree[neighbor]--
                   If indegree[neighbor] == 0:
                       queue.add(neighbor)

           Return count != V // cycle exists
```

Java Implementation

```
class Solution {
           public boolean hasCycleKahn(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               int[] indegree = new int[n];
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(edge[1]);
                   indegree[edge[1]]++;
               }

                 Queue<Integer> queue = new LinkedList<>();

              for (int i = 0; i < n; i++) {
                  if (indegree[i] == 0) {
                      queue.add(i);
                  }
              }

              int count = 0;
```

```
            while (!queue.isEmpty()) {
                int node = queue.poll();
                count++;
                for (int neighbor : graph.get(node)) {
                    indegree[neighbor]--;
                    if (indegree[neighbor] == 0) {
                         queue.add(neighbor);
                    }
                }
            }
            return count != n;
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

322. Course Schedule - I

Understanding the Problem NEW

Given a number of courses and a list of prerequisite pairs, determine if it's possible to finish all courses (i.e., is there a valid completion order, or do the prerequisites form an impossible cycle?).

Brute Force Approach NEW

Try every possible course order and check if all prerequisites are satisfied — n! orderings, infeasible.

Why We Optimize NEW

This is directly Cycle Detection in a Directed Graph (318 or 321) in disguise: model courses as nodes and prerequisites as directed edges, then the courses can all be finished **if and only if the graph has no cycle** (a cycle would mean course A needs B, which needs C, which circles back to needing A — an impossible deadlock). Reusing either the DFS three-color method or Kahn's Algorithm answers this in O(V+E), rather than exploring every possible course ordering — recognizing the real-world framing as a direct instance of an already-solved graph problem is the entire insight here.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if all courses can be completed.

Visual Diagram

```
numCourses = 2, prerequisites = [[1,0]]
       Graph: 0 → 1
       Can complete: Yes

       numCourses = 2, prerequisites = [[1,0],[0,1]]
       Graph: 0 → 1 → 0 (cycle)
       Can complete: No

       Check for cycle in directed graph
```

Solution Approach

Concept: Cycle detection in directed graph.

Algorithm

```
Algorithm CAN_FINISH(numCourses, prerequisites):
           Build graph
           Return !hasCycle_Kahn(graph)
```

Java Implementation

```
class Solution {
           public boolean canFinish(int numCourses, int[][] prerequisites) {
               List<List<Integer>> graph = new ArrayList<>();
               int[] indegree = new int[numCourses];
               for (int i = 0; i < numCourses; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] pre : prerequisites) {
                   graph.get(pre[1]).add(pre[0]);
                   indegree[pre[0]]++;
               }

                 Queue<Integer> queue = new LinkedList<>();
                 for (int i = 0; i < numCourses; i++) {
                     if (indegree[i] == 0) {
                         queue.add(i);
                     }
                 }

                 int count = 0;
                 while (!queue.isEmpty()) {
                     int course = queue.poll();
                     count++;
                     for (int next : graph.get(course)) {
                         indegree[next]--;
                         if (indegree[next] == 0) {
                              queue.add(next);
                         }
                     }
                 }
                 return count == numCourses;
           }
       }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V+E)

323. Course Schedule - II

Understanding the Problem NEW

Same setup as Course Schedule-I, but now return an actual **valid order** to complete all courses (or an empty result if impossible).

Brute Force Approach NEW

Same exponential permutation search as a naive approach to 322.

Why We Optimize NEW

This is directly Topological Sort (319 or 320) in disguise — the valid course order *is* a topological order of the prerequisite graph, and if Kahn's Algorithm (320) doesn't manage to process every node, that's simultaneously your signal that it's impossible (same cycle-detection fact from problem 321). One algorithm run answers both 'is it possible' and 'what's a valid order,' achieved in O(V+E), reusing topological sort rather than solving 'finishability' and 'find an order' as two separate problems.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Return order of courses to complete all.

Visual Diagram

```
numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
       Graph: 0 → 1 → 3
              0 → 2 → 3

       Topological Order: [0, 1, 2, 3] or [0, 2, 1, 3]

       Kahn's Algorithm:
       Indegrees: 0:0, 1:1, 2:1, 3:2
       Queue: [0]
       Pop 0 → indegree[1]=0, indegree[2]=0
       Queue: [1,2]
       Pop 1 → indegree[3]=1
       Pop 2 → indegree[3]=0
       Queue: [3]
```

```
       Pop 3

       Order: [0, 1, 2, 3]
```

Solution Approach

Concept: Kahn's algorithm for topological order.

Algorithm

```
Algorithm FIND_ORDER(numCourses, prerequisites):
           Build graph and indegree
           Kahn's algorithm
           Return order if count == numCourses else []
```

Java Implementation

```
class Solution {
           public int[] findOrder(int numCourses, int[][] prerequisites) {
               List<List<Integer>> graph = new ArrayList<>();
               int[] indegree = new int[numCourses];
               for (int i = 0; i < numCourses; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] pre : prerequisites) {
                   graph.get(pre[1]).add(pre[0]);
                   indegree[pre[0]]++;
               }

                 Queue<Integer> queue = new LinkedList<>();
                 for (int i = 0; i < numCourses; i++) {
                     if (indegree[i] == 0) {
                         queue.add(i);
                     }
                 }

                 int[] order = new int[numCourses];
                 int index = 0;
                 while (!queue.isEmpty()) {
                     int course = queue.poll();
                     order[index++] = course;

                  for (int next : graph.get(course)) {
                      indegree[next]--;
                      if (indegree[next] == 0) {
                          queue.add(next);
                      }
                  }
              }
              return index == numCourses ? order : new int[0];
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V+E)

324. Find Eventual Safe States

Understanding the Problem NEW

Given a directed graph, find all 'safe' nodes — nodes from which every possible path eventually leads to a terminal node (a node with no outgoing edges), meaning you can never get stuck in an infinite cycle no matter which path you take from a safe node.

Brute Force Approach NEW

For every node, explore every possible path starting from it and check if all of them terminate — exponential, since the number of paths from a node can grow very large.

Why We Optimize NEW

A node is unsafe **if and only if it's part of a cycle, or can reach a cycle** — so this is a cycle-detection problem in disguise, solvable with the same **three-state DFS coloring** from problem 318: a node is 'safe' if DFS from it never encounters a 'gray' (currently-on-path) node — meaning no cycle is reachable from it. Reusing the three-color technique (with memoization of each node's safe/unsafe status to avoid recomputing for shared sub-paths) determines every node's safety in a single O(V+E) pass, instead of exponentially exploring every path from every node.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find nodes that eventually reach terminal nodes.

Visual Diagram

```
Graph:
       0 → 1 → 2
       ↓   ↓
       3   4

       Terminal nodes: 3, 4
       Safe nodes: 0,1,2,3,4
```

```
       DFS with 3 states:
       0: unvisited
       1: visiting (potentially unsafe)
       2: processed (safe)
```

Solution Approach

Concept: DFS with 3-state coloring.

Algorithm

```
Algorithm SAFE_STATES(graph):
           state = new int[V] // 0: unvisited, 1: visiting, 2: safe
           result = []

           For i = 0 to V-1:
               If dfs(i, state, graph):
                   result.add(i)
           Return result

       Algorithm dfs(node, state, graph):
           If state[node] == 1: Return false
           If state[node] == 2: Return true

           state[node] = 1 // visiting
           For neighbor in graph[node]:
               If !dfs(neighbor, state, graph):
                   Return false
           state[node] = 2 // safe
           Return true
```

Java Implementation

```
class Solution {
           public List<Integer> eventualSafeNodes(int[][] graph) {
               int n = graph.length;
               int[] state = new int[n]; // 0: unvisited, 1: visiting, 2: safe
               List<Integer> result = new ArrayList<>();

                 for (int i = 0; i < n; i++) {
                     if (dfs(i, state, graph)) {
                         result.add(i);
                     }
                 }
                 return result;
           }
           private boolean dfs(int node, int[] state, int[][] graph) {

              if (state[node] == 1) return false;
              if (state[node] == 2) return true;
```

```
            state[node] = 1; // visiting
            for (int neighbor : graph[node]) {
                if (!dfs(neighbor, state, graph)) {
                    return false;
                }
            }
            state[node] = 2; // safe
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

325. Alien Dictionary

Understanding the Problem NEW

Given a list of words from an alien language sorted according to that language's (unknown) alphabetical order, determine what that order actually is.

Brute Force Approach NEW

Try every possible ordering of the alphabet's letters and check which one is consistent with the given sorted word list — with up to 26 letters, that's 26! possible orderings, astronomically infeasible.

Why We Optimize NEW

Compare each pair of **adjacent words** in the list: the first position where they differ tells you that the earlier word's letter must come **before** the later word's letter in the alien alphabet — build a directed graph of these letter-precedence constraints from every adjacent word pair, then the actual alien alphabet order is simply a **topological sort** (319/320) of this constraint graph. This turns an exponential guess-the-full-ordering problem into 'extract pairwise ordering constraints, then topologically sort them' — directly reusing the topological sort technique, with an added subtlety: if a later word is a *strict prefix* of an earlier word (e.g. 'abc' appearing after 'ab'), that's an immediate contradiction (a prefix must always come first), which needs an explicit special check since the letter-by-letter comparison wouldn't otherwise catch it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find order of letters in alien language.

Visual Diagram

```
Words: ["wrt", "wrf", "er", "ett", "rftt"]
       Output: "wertf"

       Compare adjacent words:
       wrt vs wrf → t < f
       wrf vs er → w < e
       er vs ett → r < t
```

```
       ett vs rftt → e < r

       Graph: t→f, w→e, r→t, e→r
       Topological: w→e→r→t→f = "wertf"
```

Solution Approach

Concept: Build graph and topological sort.

Algorithm

```
Algorithm ALIEN_DICTIONARY(words):
           graph = new HashMap<Character, Set<Character>>()
           indegree = new HashMap<Character, Integer>()

            // Initialize all characters
            For each word, add all chars to graph

            // Compare adjacent words
            For i = 0 to words.length-2:
                word1 = words[i], word2 = words[i+1]
                For j = 0 to min(word1.length, word2.length):
                    If word1[j] != word2[j]:
                        If word2[j] not in graph[word1[j]]:
                            graph[word1[j]].add(word2[j])
                            indegree[word2[j]]++
                        Break

            // Topological sort
            Queue = all chars with indegree 0
            Build result
            Return result if all chars included else ""
```

Java Implementation

```
class Solution {
           public String alienOrder(String[] words) {
               Map<Character, Set<Character>> graph = new HashMap<>();
               Map<Character, Integer> indegree = new HashMap<>();

                 // Initialize
                 for (String word : words) {
                     for (char c : word.toCharArray()) {
                         graph.putIfAbsent(c, new HashSet<>());
                         indegree.putIfAbsent(c, 0);
                     }
                 }
                 // Build graph

             for (int i = 0; i < words.length - 1; i++) {
```

```
                String word1 = words[i], word2 = words[i+1];
                int minLen = Math.min(word1.length(), word2.length());
                boolean found = false;

                for (int j = 0; j < minLen; j++) {
                    char c1 = word1.charAt(j), c2 = word2.charAt(j);
                    if (c1 != c2) {
                        if (!graph.get(c1).contains(c2)) {
                            graph.get(c1).add(c2);
                            indegree.put(c2, indegree.get(c2) + 1);
                        }
                        found = true;
                        break;
                    }
                }
                if (!found && word1.length() > word2.length()) {
                    return "";
                }
            }

            // Topological sort
            Queue<Character> queue = new LinkedList<>();
            for (char c : indegree.keySet()) {
                if (indegree.get(c) == 0) {
                    queue.add(c);
                }
            }

            StringBuilder result = new StringBuilder();
            while (!queue.isEmpty()) {
                char c = queue.poll();
                result.append(c);
                for (char next : graph.get(c)) {
                    indegree.put(next, indegree.get(next) - 1);
                    if (indegree.get(next) == 0) {
                        queue.add(next);
                    }
                }
            }
            return result.length() == indegree.size() ? result.toString() : "";
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * L) | Space Complexity: O(V+E)

326. Shortest Path in Undirected Graph with Unit Weight

Understanding the Problem NEW

Find the shortest distance from a source node to every other node, in an undirected graph where every edge has the same weight (unit weight, i.e. an unweighted graph).

Brute Force Approach NEW

You could run a separate search from the source to each individual target node — O(V) separate traversals, each O(V+E), giving O(V×(V+E)) overall, hugely redundant since a single traversal can actually discover distances to everyone at once.

Why We Optimize NEW

Plain **BFS from the source** already solves this optimally in one pass: since BFS explores level by level, and every edge costs the same 'one step,' the level at which BFS reaches a node *is* exactly that node's shortest distance from the source. A single O(V+E) BFS traversal gives every node's shortest distance simultaneously — the 'unit weight' condition is precisely what makes plain BFS sufficient here; the moment edges have different weights, this stops working and you need Dijkstra's Algorithm (328) instead, which is exactly the next problem's motivation.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find shortest distance from source to all nodes.

Visual Diagram

```
Graph:
       0 --- 1
       |     |
       3 --- 2

       BFS from 0:
       dist[0] = 0
```

```
       dist[1] = 1, dist[3] = 1
       dist[2] = 2

       BFS queue:
       [0] → pop 0, add 1,3
       [1,3] → pop 1, add 2
       [3,2] → pop 3
       [2] → pop 2
```

Solution Approach

Concept: BFS for unweighted graph.

Algorithm

```
Algorithm SHORTEST_PATH_UNWEIGHTED(graph, source):
           dist = new int[V]
           Arrays.fill(dist, -1)
           queue = Queue()
           dist[source] = 0
           queue.add(source)

           While queue not empty:
               node = queue.poll()
               For neighbor in graph[node]:
                   If dist[neighbor] == -1:
                       dist[neighbor] = dist[node] + 1
                       queue.add(neighbor)
           Return dist
```

Java Implementation

```
class Solution {
           public int[] shortestPath(int n, int[][] edges, int source) {
               List<List<Integer>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(edge[1]);
                   graph.get(edge[1]).add(edge[0]);
               }

                  int[] dist = new int[n];
                  Arrays.fill(dist, -1);

              Queue<Integer> queue = new LinkedList<>();
              dist[source] = 0;
              queue.add(source);
```

```
            while (!queue.isEmpty()) {
                int node = queue.poll();
                for (int neighbor : graph.get(node)) {
                    if (dist[neighbor] == -1) {
                         dist[neighbor] = dist[node] + 1;
                         queue.add(neighbor);
                    }
                }
            }
            return dist;
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

327. Shortest Path in DAG

Understanding the Problem NEW

Find the shortest path from a source to every other node, in a **Directed Acyclic Graph (DAG)** where edges can have different (possibly non-unit) weights.

Brute Force Approach NEW

You could try Dijkstra's Algorithm (328) directly, which would work — but it does more work than necessary here, since a DAG's acyclic structure allows for something even simpler and faster.

Why We Optimize NEW

Because the graph is acyclic, you can first compute a **topological order** (319) of the nodes, then process nodes *in that order*, 'relaxing' each outgoing edge (i.e., updating a neighbour's best-known distance if going through the current node gives a shorter path) — since topological order guarantees you always finish computing a node's final shortest distance before you ever need to use it to update something later in the order, one single pass through the topologically sorted nodes suffices. This achieves O(V+E) — genuinely faster than Dijkstra's O(E log V) — by exploiting the DAG's acyclic property (no need for the priority-queue-based 'always process the currently-closest unprocessed node next' strategy that general graphs with cycles require).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find shortest path in Directed Acyclic Graph.

Visual Diagram

```
DAG:
       0 → 1 (weight 5)
       0 → 2 (weight 3)
       1 → 3 (weight 6)
       2 → 3 (weight 2)
       2 → 4 (weight 4)
       3 → 4 (weight 1)
```

```
Topological order: [0, 2, 1, 3, 4]
Distances from 0:
dist[0] = 0
dist[1] = 5
dist[2] = 3
dist[3] = min(5+6=11, 3+2=5) = 5
dist[4] = min(3+4=7, 5+1=6) = 6
```

Solution Approach

Concept: Topological order + relaxation.

Algorithm

```
Algorithm SHORTEST_PATH_DAG(graph, source):
           topo = topologicalSort(graph)
           dist = new int[V]
           Arrays.fill(dist, INF)
           dist[source] = 0

           For node in topo:
               If dist[node] != INF:
                   For neighbor, weight in graph[node]:
                       dist[neighbor] = min(dist[neighbor], dist[node] + weight)

           Return dist
```

Java Implementation

```
class Solution {
           public int[] shortestPathDAG(int n, int[][] edges, int source) {
               List<List<int[]>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(new int[]{edge[1], edge[2]});
               }

                 // Topological sort
                 boolean[] visited = new boolean[n];
                 Stack<Integer> stack = new Stack<>();
                 for (int i = 0; i < n; i++) {
                     if (!visited[i]) {
                         dfs(i, graph, visited, stack);
                     }
                 }

              // Shortest path
              int[] dist = new int[n];
```

```
            Arrays.fill(dist, Integer.MAX_VALUE);
            dist[source] = 0;

            while (!stack.isEmpty()) {
                int node = stack.pop();
                if (dist[node] != Integer.MAX_VALUE) {
                    for (int[] neighbor : graph.get(node)) {
                        int next = neighbor[0], weight = neighbor[1];
                        dist[next] = Math.min(dist[next], dist[node] + weight);
                    }
                }
            }
            return dist;
        }

        private void dfs(int node, List<List<int[]>> graph, boolean[] visited,
Stack<Integer> stack
            visited[node] = true;
            for (int[] neighbor : graph.get(node)) {
                if (!visited[neighbor[0]]) {
                    dfs(neighbor[0], graph, visited, stack);
                }
            }
            stack.push(node);
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

328. Dijkstra's Algorithm

Understanding the Problem NEW

Find the shortest path from a source to every other node in a graph with **non-negative** edge weights (edges can now have different costs, and the graph can have cycles, unlike problem 327's DAG restriction).

Brute Force Approach NEW

Try every possible path from source to each target and compare total weights — exponential, since the number of distinct paths in a graph with cycles can be unbounded (well, bounded but combinatorially huge) unless a cycle is avoided during enumeration, which adds further complexity to a naive approach.

Why We Optimize NEW

Dijkstra's Algorithm: repeatedly pick the **currently closest unprocessed node** (using a Min-Heap/Priority Queue to always retrieve it in O(log V)), and 'relax' all of its outgoing edges (update neighbours' distances if a shorter path is found through this node) — since you always process the globally-closest remaining node next, once a node is processed, its shortest distance is guaranteed final (any other path to it would have to go through an already-processed, farther node, which can't yield something shorter given non-negative weights). This greedy 'always finalize the closest remaining node next' strategy, powered by a heap, achieves O((V+E) log V) instead of exponential path enumeration — but critically depends on **non-negative weights** for its greedy correctness guarantee, which is exactly why negative edges need Bellman-Ford (336) instead.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Shortest path in weighted graph (non-negative edges).

Visual Diagram

```
Graph:
       0 → 1 (4)
       0 → 2 (1)
       1 → 3 (1)
```

```
       2 → 1 (2)
       2 → 3 (5)
       3 → 4 (3)

       From 0:
       dist[0] = 0
       dist[2] = 1
       dist[1] = min(4, 1+2=3) = 3
       dist[3] = min(3+1=4, 1+5=6) = 4
       dist[4] = 7

       Priority Queue:
       (0,0) → pop 0
       (1,2) → pop 2
       (3,1) → pop 1
       (4,3) → pop 3
       (7,4) → pop 4
```

Solution Approach

Concept: Priority Queue with distance relaxation.

Algorithm

```
Algorithm DIJKSTRA(graph, source):
           dist = new int[V]
           Arrays.fill(dist, INF)
           dist[source] = 0
           pq = PriorityQueue((d, node) -> d)
           pq.add((0, source))

           While pq not empty:
               (d, node) = pq.poll()
               If d > dist[node]: continue
               For neighbor, weight in graph[node]:
                   If dist[node] + weight < dist[neighbor]:
                       dist[neighbor] = dist[node] + weight
                       pq.add((dist[neighbor], neighbor))

           Return dist
```

Java Implementation

```
class Solution {
           public int[] dijkstra(int n, int[][] edges, int source) {
               List<List<int[]>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
```

```
                graph.get(edge[0]).add(new int[]{edge[1], edge[2]});

                graph.get(edge[1]).add(new int[]{edge[0], edge[2]});
            }

            int[] dist = new int[n];
            Arrays.fill(dist, Integer.MAX_VALUE);
            dist[source] = 0;

            PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
            pq.add(new int[]{0, source});

            while (!pq.isEmpty()) {
                int[] curr = pq.poll();
                int d = curr[0], node = curr[1];
                if (d > dist[node]) continue;

                for (int[] neighbor : graph.get(node)) {
                    int next = neighbor[0], weight = neighbor[1];
                    if (dist[node] + weight < dist[next]) {
                        dist[next] = dist[node] + weight;
                        pq.add(new int[]{dist[next], next});
                    }
                }
            }
            return dist;
        }
    }
```

Time & Space Complexity

Time Complexity: O((V+E) log V) | Space Complexity: O(V)

329. Why Priority Queue is Used in Dijkstra

Understanding the Problem NEW

This isn't a new problem — it's a dedicated explanation of *why* Dijkstra's Algorithm specifically needs a Priority Queue (Min-Heap) rather than, say, a plain queue or a simple array scan.

Brute Force Approach NEW

You could scan the entire list of unprocessed nodes each time to find the current minimum-distance one — O(V) per selection, O(V²) total across all selections, which is actually a valid (if slower) variant of Dijkstra's Algorithm sometimes used for very dense graphs.

Why We Optimize NEW

A Priority Queue retrieves the current minimum in O(log V) instead of O(V), which matters because Dijkstra performs this 'find the closest unprocessed node' operation up to V times — using a heap brings the total down to O((V+E) log V) instead of O(V²), a meaningful improvement for sparse graphs (where E is much smaller than V²). This connects directly back to the Heaps chapter's core lesson (problem 211): a heap is exactly the right structure whenever you repeatedly need 'the current best/smallest candidate' from a changing set, which is precisely Dijkstra's repeated need throughout its execution.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understand the role of priority queue in Dijkstra.

Visual Diagram

```
Without Priority Queue (using simple queue):
       0 → 1 (10)
       0 → 2 (5)
       2 → 1 (2)

       Queue approach:
       Process 0 → dist[1]=10, dist[2]=5
       Process 2 → dist[1]=7 (update)
```

```
Process 1 → no updates
Works but process order matters

With Priority Queue:
Always process smallest distance first
Guarantees optimal distance when popped
No need to process nodes multiple times

Example:
pq: (0,0)
Pop 0 → add (5,2), (10,1)
Pop 2 → add (7,1)
Pop 1 → final
```

Java Implementation

```
class Solution {
           public int[] dijkstraWithPQ(int n, int[][] edges, int source) {
               // Same as Dijkstra implementation above
               // PQ ensures O((V+E)logV) complexity
               // Without PQ, it would be O(V²)

                // PQ is used because:
                // 1. Always extracts minimum distance
                // 2. Each node processed once
                // 3. Efficient O(log V) extraction
                // 4. Handles dynamic updates
                // 5. Guarantees correctness
                return new int[0];
            }
       }
```

Time & Space Complexity

Time Complexity: O((V+E) log V) | Space Complexity: O(V)

330. Shortest Path in a Binary Maze

Understanding the Problem NEW

Given a binary grid (0s are open cells, 1s are blocked), find the shortest path from top-left to bottom-right, moving through adjacent open cells (including diagonals, in some variants).

Brute Force Approach NEW

Try every possible path via exhaustive DFS exploration, tracking the shortest one found — exponential, since the number of paths through a grid can grow very large.

Why We Optimize NEW

Since every move costs the same '1 step' (unweighted, like problem 326), plain **BFS from the start cell** finds the shortest path directly — the level at which BFS reaches the destination cell is exactly the shortest path length. This is Shortest Path with Unit Weight (326), applied to an implicit grid-graph (306's 'grid is a graph in disguise' idea) instead of an explicit adjacency list — no new algorithm needed, just recognizing the grid as an unweighted graph where BFS is the right tool.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find shortest path in a binary maze.

Visual Diagram

```
Grid:
       0 0 0
       1 1 0
       0 0 0
       Start: (0,0), End: (2,2)

       Path: (0,0)→(0,1)→(0,2)→(1,2)→(2,2) = 5 steps

       BFS with 4 directions:
       Queue: [(0,0)]
       Level 1: (0,1)
```

```
       Level 2: (0,2)
       Level 3: (1,2)
       Level 4: (2,2)
```

Solution Approach

Concept: BFS on grid.

Algorithm

```
Algorithm SHORTEST_PATH_MAZE(grid, start, end):
           m = grid.length, n = grid[0].length
           visited = boolean[m][n]
           queue = Queue()
           queue.add(start)
           visited[start[0]][start[1]] = true
           dist = 0

           While queue not empty:
               size = queue.size()
               For i = 0 to size-1:
                   cell = queue.poll()
                   If cell == end: Return dist
                   For each direction:
                       If valid and grid[x][y] == 0 and !visited:
                           queue.add((x,y))
                           visited[x][y] = true
               dist++
           Return -1
```

Java Implementation

```
class Solution {
           public int shortestPath(int[][] grid, int[] start, int[] end) {
               int m = grid.length, n = grid[0].length;
               boolean[][] visited = new boolean[m][n];
               Queue<int[]> queue = new LinkedList<>();
               queue.add(start);
               visited[start[0]][start[1]] = true;
               int dist = 0;
               int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

                 while (!queue.isEmpty()) {
                     int size = queue.size();

                  for (int i = 0; i < size; i++) {
                      int[] cell = queue.poll();
                      if (cell[0] == end[0] && cell[1] == end[1]) {
                          return dist;
                      }
```

```
                    for (int[] dir : dirs) {
                        int x = cell[0] + dir[0];
                        int y = cell[1] + dir[1];
                        if (x >= 0 && x < m && y >= 0 && y < n &&
                            grid[x][y] == 0 && !visited[x][y]) {
                            queue.add(new int[]{x, y});
                            visited[x][y] = true;
                        }
                    }
                }
                dist++;
            }
            return -1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

331. Path with Minimum Effort

Understanding the Problem NEW

Given a grid where moving between adjacent cells costs the absolute difference in their values ('effort'), find a path from top-left to bottom-right that minimizes the **maximum single-step effort** along the path (not the total effort — just the worst single step).

Brute Force Approach NEW

Try every possible path and compute each one's maximum step-effort, tracking the overall minimum — exponential path exploration.

Why We Optimize NEW

This needs a **modified Dijkstra's Algorithm**: instead of tracking and minimizing the *sum* of edge weights along a path (as ordinary Dijkstra does), track and minimize the **maximum** edge weight seen so far along a path — when 'relaxing' an edge, a path is considered better if it reduces the *worst step so far*, not the total. The same priority-queue-driven 'always expand the currently best path next' structure from Dijkstra applies unchanged, just with the comparison/update rule changed from 'sum' to 'max' — a good example of how Dijkstra's greedy priority-queue framework generalizes to different path-cost definitions, not just simple additive weights.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find path with minimum maximum effort.

Visual Diagram

```
Grid:
       [1,2,2]
       [3,8,2]
       [5,3,5]
       Effort = max absolute difference

       Path: (0,0)→(0,1)→(0,2)→(1,2)→(2,2)
       Efforts: |2-1|=1, |2-2|=0, |8-2|=6, |5-8|=3
```

```
       Max effort = 6

       Dijkstra variation:
       dist[i][j] = min max effort to reach (i,j)
       Priority queue by effort
```

Solution Approach

Concept: Modified Dijkstra.

Algorithm

```
Algorithm MINIMUM_EFFORT(heights):
           m = heights.length, n = heights[0].length
           dist = new int[m][n]
           Arrays.fill(dist, INF)
           dist[0][0] = 0
           pq = PriorityQueue((effort, x, y))
           pq.add((0, 0, 0))

           While pq not empty:
               (effort, x, y) = pq.poll()
               If effort > dist[x][y]: continue
               If x == m-1 && y == n-1: return effort
               For each direction:
                   newEffort = max(effort, abs(heights[x][y] - heights[nx][ny]))
                   If newEffort < dist[nx][ny]:
                       dist[nx][ny] = newEffort
                       pq.add((newEffort, nx, ny))
           Return -1
```

Java Implementation

```
class Solution {
           public int minimumEffortPath(int[][] heights) {
               int m = heights.length, n = heights[0].length;
               int[][] dist = new int[m][n];
               for (int[] row : dist) {
                   Arrays.fill(row, Integer.MAX_VALUE);
               }
               dist[0][0] = 0;

                 PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] -
b[0]);
                 pq.add(new int[]{0, 0, 0});
                 int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

                 while (!pq.isEmpty()) {

                  int[] curr = pq.poll();
```

```
                int effort = curr[0], x = curr[1], y = curr[2];
                if (effort > dist[x][y]) continue;
                if (x == m-1 && y == n-1) return effort;

                for (int[] dir : dirs) {
                    int nx = x + dir[0];
                    int ny = y + dir[1];
                    if (nx >= 0 && nx < m && ny >= 0 && ny < n) {
                        int newEffort = Math.max(effort,
                            Math.abs(heights[x][y] - heights[nx][ny]));
                        if (newEffort < dist[nx][ny]) {
                            dist[nx][ny] = newEffort;
                            pq.add(new int[]{newEffort, nx, ny});
                        }
                    }
                }
            }
            return -1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n log(m*n)) | Space Complexity: O(m*n)

332. Cheapest Flights Within K Stops

Understanding the Problem NEW

Find the cheapest flight route from a source city to a destination, allowed at most K stopover cities along the way (a constraint that ordinary shortest-path algorithms don't account for).

Brute Force Approach NEW

Try every possible route with up to K+1 flights and compare total costs — the number of possible routes can grow exponentially with the number of cities and allowed stops.

Why We Optimize NEW

Dijkstra's Algorithm doesn't directly work here because its greedy 'always finalize the closest node' guarantee assumes you don't care *how many edges* a path used — but here, a route's validity depends on stop count, not just cost. Instead, use a **Bellman-Ford-style approach**: relax all edges, but only allow up to **K+1 rounds** of relaxation (each round representing one additional flight taken), tracking the best cost reachable within that many flights specifically. This bounded-rounds relaxation (rather than Dijkstra's unbounded greedy processing) is exactly what correctly respects the 'at most K stops' constraint — a case where the general shortest-path problem needs a different algorithm once an extra constraint (hop count) is added on top of just minimizing total cost.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find cheapest flight with at most k stops.

Visual Diagram

```
n=3, flights=[[0,1,100],[1,2,100],[0,2,500]], k=1
       Path: 0→1→2 cost=200

       BFS with DP:
       dp[node][stops] = min cost
       Bellman-Ford variant
```

```
       dp[0][0] = 0
       For stops = 1 to k+1:
           For each flight:
               dp[flight.to][stops] = min(dp[flight.to][stops],
                                          dp[flight.from][stops-1] + cost)
```

Solution Approach

Concept: Bellman-Ford with stop constraint.

Algorithm

```
Algorithm CHEAPEST_FLIGHT(n, flights, src, dst, k):
           dist = new int[n]
           Arrays.fill(dist, INF)
           dist[src] = 0

           For i = 1 to k+1:
               temp = dist.clone()
               For each flight (from, to, price):
                   If dist[from] != INF:
                       temp[to] = min(temp[to], dist[from] + price)
               dist = temp
               If dist[dst] updated: track

           Return dist[dst] == INF ? -1 : dist[dst]
```

Java Implementation

```
class Solution {
           public int findCheapestPrice(int n, int[][] flights, int src, int dst,
int k) {
               int[] dist = new int[n];
               Arrays.fill(dist, Integer.MAX_VALUE);
               dist[src] = 0;

                 for (int i = 0; i <= k; i++) {
                     int[] temp = Arrays.copyOf(dist, n);
                     for (int[] flight : flights) {
                         int from = flight[0], to = flight[1], price = flight[2];
                         if (dist[from] != Integer.MAX_VALUE) {
                             temp[to] = Math.min(temp[to], dist[from] + price);
                         }
                     }
                     dist = temp;
                 }
                 return dist[dst] == Integer.MAX_VALUE ? -1 : dist[dst];
           }
       }
```

Time & Space Complexity

Time Complexity: O(k * E) | Space Complexity: O(V)

333. Network Delay Time

Understanding the Problem NEW

Given a network of nodes and directed weighted edges representing signal transmission times, find the time for a signal sent from a source node to reach *every* node in the network (or determine it's impossible).

Brute Force Approach NEW

Try every possible path from the source to every target node and take the minimum for each — exponential path enumeration.

Why We Optimize NEW

This is directly Dijkstra's Algorithm (328) applied as-is: run Dijkstra from the source node to get the shortest distance to every other node, and the answer is simply the **maximum** of all these shortest distances (since the whole network 'finishes receiving the signal' only once the slowest-to-reach node gets it) — or -1/ impossible if any node's distance remains unreachable. No new technique needed here — just directly applying Dijkstra and taking the max of the resulting distance array.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Time for signal to reach all nodes.

Visual Diagram

```
Graph:
       1 → 2 (1)
       2 → 3 (1)
       1 → 3 (2)

       From 1:
       dist[1]=0, dist[2]=1, dist[3]=2
       Max distance = 2

       Dijkstra from source
       Return max distance
```

Solution Approach

Concept: Dijkstra's algorithm.

Algorithm

```
Algorithm NETWORK_DELAY(times, n, k):
           graph = adjacency list
           dist = Dijkstra(k)
           maxTime = max(dist[1..n])
           Return maxTime == INF ? -1 : maxTime
```

Java Implementation

```
class Solution {
           public int networkDelayTime(int[][] times, int n, int k) {
               List<List<int[]>> graph = new ArrayList<>();
               for (int i = 0; i <= n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] time : times) {
                   graph.get(time[0]).add(new int[]{time[1], time[2]});
               }

                 int[] dist = new int[n + 1];
                 Arrays.fill(dist, Integer.MAX_VALUE);
                 dist[k] = 0;

                 PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] -
b[1]);
                 pq.add(new int[]{k, 0});

                 while (!pq.isEmpty()) {
                     int[] curr = pq.poll();
                     int node = curr[0], d = curr[1];
                     if (d > dist[node]) continue;

                    for (int[] neighbor : graph.get(node)) {
                        int next = neighbor[0], weight = neighbor[1];
                        if (dist[node] + weight < dist[next]) {
                            dist[next] = dist[node] + weight;
                            pq.add(new int[]{next, dist[next]});
                        }
                    }

             }

             int maxTime = 0;
             for (int i = 1; i <= n; i++) {
                 if (dist[i] == Integer.MAX_VALUE) return -1;
                 maxTime = Math.max(maxTime, dist[i]);
```

```
            }
            return maxTime;
        }
    }
```

Time & Space Complexity

Time Complexity: O((V+E) log V) | Space Complexity: O(V+E)

334. Number of Ways to Arrive at Destination

Understanding the Problem NEW

Find not just the shortest path length from source to destination, but **how many distinct shortest paths** exist (paths that all tie for the minimum possible distance).

Brute Force Approach NEW

Enumerate every possible path, group them by total distance, and count how many share the minimum — exponential.

Why We Optimize NEW

Extend Dijkstra's Algorithm with an extra **'ways' counter** per node: whenever you relax an edge and find a *strictly shorter* path to a neighbour, reset that neighbour's ways-count to match the current node's ways-count (a completely new best path type); but if you find a path that *ties* the neighbour's current best distance exactly, **add** the current node's ways-count to the neighbour's existing ways-count (an additional equally-good path has been found). This piggybacks path-counting onto the same Dijkstra traversal, achieving the same O((V+E) log V) complexity while tracking one extra piece of bookkeeping per node, rather than needing a separate exponential enumeration of all paths.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count number of shortest paths to destination.

Visual Diagram

```
Graph:
       0 → 1 (1)
       0 → 2 (2)
       1 → 2 (1)
       1 → 3 (3)
       2 → 3 (1)

       Number of shortest paths from 0 to 3:
       0→1→2→3 (1+1+1=3)
       0→2→3 (2+1=3)
```

```
       0→1→3 (1+3=4)
       0→1→2→3 (1+1+1=3)
       Total: 2 paths

       Modified Dijkstra:
       dist[v] = shortest distance
       ways[v] = number of ways
       If dist[node] + weight < dist[next]:
           dist[next] = dist[node] + weight
           ways[next] = ways[node]
       If dist[node] + weight == dist[next]:
           ways[next] += ways[node]
```

Solution Approach

Concept: Dijkstra with ways tracking.

Algorithm

```
Algorithm COUNT_WAYS(n, roads):
           Build graph
           dist = new long[n], ways = new long[n]
           dist[source] = 0, ways[source] = 1
           pq = PriorityQueue()

           While pq not empty:
               (d, node) = pq.poll()
               If d > dist[node]: continue
               For (neighbor, weight) in graph[node]:
                   If dist[node] + weight < dist[neighbor]:
                       dist[neighbor] = dist[node] + weight
                       ways[neighbor] = ways[node]
                       pq.add((dist[neighbor], neighbor))
                   Else If dist[node] + weight == dist[neighbor]:
                       ways[neighbor] = (ways[neighbor] + ways[node]) % MOD
           Return ways[dst]
```

Java Implementation

```
class Solution {
           public int countPaths(int n, int[][] roads) {
               final int MOD = 1_000_000_007;
               List<List<int[]>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());

              }
              for (int[] road : roads) {
                  graph.get(road[0]).add(new int[]{road[1], road[2]});
                  graph.get(road[1]).add(new int[]{road[0], road[2]});
```

```
            }

            long[] dist = new long[n];
            long[] ways = new long[n];
            Arrays.fill(dist, Long.MAX_VALUE);
            dist[0] = 0;
            ways[0] = 1;

            PriorityQueue<long[]> pq = new PriorityQueue<>((a, b) ->
                Long.compare(a[1], b[1]));
            pq.add(new long[]{0, 0});

            while (!pq.isEmpty()) {
                long[] curr = pq.poll();
                int node = (int) curr[0];
                long d = curr[1];
                if (d > dist[node]) continue;

                for (int[] neighbor : graph.get(node)) {
                    int next = neighbor[0], weight = neighbor[1];
                    if (dist[node] + weight < dist[next]) {
                        dist[next] = dist[node] + weight;
                        ways[next] = ways[node];
                        pq.add(new long[]{next, dist[next]});
                    } else if (dist[node] + weight == dist[next]) {
                        ways[next] = (ways[next] + ways[node]) % MOD;
                    }
                }
            }
            return (int) ways[n-1];
        }
    }
```

Time & Space Complexity

Time Complexity: O((V+E) log V) | Space Complexity: O(V+E)

335. Minimum Steps to Reach End from Start

Understanding the Problem NEW

Given a starting value and target value, and a set of allowed transformation operations, find the minimum number of operations to reach the target from the start — a generalization of Word Ladder (314) to arbitrary 'states' and 'operations' rather than specifically words and letter changes.

Brute Force Approach NEW

Try every possible sequence of operations exhaustively — exponential, since the number of possible operation sequences grows very quickly.

Why We Optimize NEW

Exactly the same insight as Word Ladder: model each reachable 'state' as a graph node, with an edge to every state reachable via one allowed operation, and the minimum number of operations is exactly the **shortest path** length from the start state to the target state in this implicit graph — found via plain BFS (since every operation, like every word-transformation step, costs one unit), generating each state's neighbours on the fly via the allowed operations rather than building the whole state graph upfront. This is the general template Word Ladder was a specific instance of: 'minimum operations to transform X into Y' problems are almost always BFS-on-an-implicit-state-graph problems in disguise.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum steps to reach target using operations.

Visual Diagram

```
Start: 0, Target: 10
       Operations: 2x, x+1, x-1

       BFS:
       Level 0: [0]
       Level 1: [1, -1, 0] → [1, -1]
       Level 2: [2, 0, 2, -2] → [2, -2]
       Level 3: [4, 1, 4, -4] → [4, -4]
```

```
Level 4: [8, 3, 8, -8] → [8, -8]
Level 5: [16, 7, 16, -16] → [16,7,16,-16]
Continue until 10 found
```

Solution Approach

Concept: BFS on state space.

Algorithm

```
Algorithm MIN_STEPS(start, target, operations):
           visited = Set()
           queue = Queue()
           queue.add(start)
           visited.add(start)
           steps = 0

           While queue not empty:
               size = queue.size()
               For i = 0 to size-1:
                   node = queue.poll()
                   If node == target: Return steps
                   For each operation:
                       next = operation(node)
                       If !visited.contains(next):
                           visited.add(next)
                           queue.add(next)
               steps++
           Return -1
```

Java Implementation

```
class Solution {
           public int minSteps(int start, int target, int[] ops) {
               Set<Integer> visited = new HashSet<>();
               Queue<Integer> queue = new LinkedList<>();
               queue.add(start);
               visited.add(start);
               int steps = 0;

                 while (!queue.isEmpty()) {
                     int size = queue.size();
                     for (int i = 0; i < size; i++) {
                         int node = queue.poll();
                         if (node == target) return steps;

                         for (int op : ops) {

                           int next = applyOp(node, op);
                           if (next >= 0 && next <= 1000 && !
```

```
visited.contains(next)) {
                                visited.add(next);
                                queue.add(next);
                            }
                        }
                    }
                    steps++;
                }
                return -1;
            }

            private int applyOp(int node, int op) {
                // Example operations: 0: +1, 1: -1, 2: *2
                if (op == 0) return node + 1;
                if (op == 1) return node - 1;
                if (op == 2) return node * 2;
                return node;
            }
        }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

336. Bellman Ford Algorithm

Understanding the Problem NEW

Find shortest paths from a source node in a graph that may contain **negative** edge weights (something Dijkstra's Algorithm cannot correctly handle, since its greedy 'finalize the closest node' guarantee breaks down when a later negative edge could still improve an already-finalized node's distance).

Brute Force Approach NEW

You might try running Dijkstra anyway — but it can produce incorrect results with negative edges, since its core greedy assumption (processing nodes in increasing distance order guarantees correctness) is specifically violated by negative weights.

Why We Optimize NEW

Bellman-Ford takes a different approach: relax *every* edge in the graph, repeated for **V-1 rounds** (where V is the number of nodes) — this many rounds is guaranteed sufficient because the longest possible shortest path (without cycles) can have at most V-1 edges, so after V-1 full relaxation rounds, every shortest distance is guaranteed to have stabilized, even accounting for negative edges. As a valuable bonus, running one *additional* (V-th) relaxation round and checking if **any** distance still improves reveals whether the graph contains a **negative cycle** (a cycle whose total weight is negative, which would make 'shortest path' undefined, since you could loop around it forever to keep decreasing the total). This costs O(V×E) — slower than Dijkstra's O((V+E) log V) — but correctly handles negative weights, which Dijkstra's greedy priority-queue approach fundamentally cannot.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Shortest path with negative edges (no negative cycle).

Visual Diagram

```
Graph:
       0 → 1 (4)
```

```
0 → 2 (1)
2 → 1 (2)
1 → 3 (1)
2 → 3 (5)

From 0:
dist[0] = 0
Iteration 1: dist[2]=1, dist[1]=4
Iteration 2: dist[1]=3, dist[3]=4
Iteration 3: dist[3]=4 (no change)

Negative cycle detection:
If any relaxation in V-th iteration → negative cycle
```

Solution Approach

Concept: V-1 relaxations.

Algorithm

```
Algorithm BELLMAN_FORD(graph, source):
           dist = new int[V]
           Arrays.fill(dist, INF)
           dist[source] = 0

           For i = 1 to V-1:
               For each edge (u, v, weight):
                   If dist[u] != INF and dist[u] + weight < dist[v]:
                       dist[v] = dist[u] + weight

           // Check negative cycle
           For each edge (u, v, weight):
               If dist[u] != INF and dist[u] + weight < dist[v]:
                   Return false // Negative cycle exists

           Return true
```

Java Implementation

```
class Solution {
           public int[] bellmanFord(int n, int[][] edges, int source) {
               int[] dist = new int[n];
               Arrays.fill(dist, Integer.MAX_VALUE);
               dist[source] = 0;

                 // V-1 iterations
                 for (int i = 1; i < n; i++) {
                     for (int[] edge : edges) {
                         int u = edge[0], v = edge[1], w = edge[2];
                         if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v])
```

```
{
                            dist[v] = dist[u] + w;
                        }
                    }

            }

            // Check negative cycle
            for (int[] edge : edges) {
                int u = edge[0], v = edge[1], w = edge[2];
                if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                    return null; // Negative cycle exists
                }
            }
            return dist;
        }
    }
```

Time & Space Complexity

Time Complexity: O(V*E) | Space Complexity: O(V)

337. Floyd Warshall Algorithm

Understanding the Problem NEW

Find the shortest path between *every* pair of nodes in a graph (all-pairs shortest paths), rather than just from a single source.

Brute Force Approach NEW

Run Dijkstra's Algorithm (or Bellman-Ford, if negative edges are present) separately from every single node as a source — O(V) separate shortest-path computations, each O((V+E) log V) or O(V×E), giving O(V² log V) or O(V²×E) overall.

Why We Optimize NEW

Floyd-Warshall takes a fundamentally different, dynamic-programming-based approach: for every pair (i, j), check whether routing *through* an intermediate node k gives a shorter path than the current best known — trying every possible intermediate node k, one at a time, updating the entire distance matrix after considering each k. This achieves all-pairs shortest paths in O(V³) with a wonderfully simple triple-nested loop, and — unlike running Dijkstra V separate times — it naturally handles negative edge weights too (though still not negative cycles), making it a genuinely different algorithmic strategy (incremental 'allow one more intermediate node at a time' DP) rather than just 'repeat single-source Dijkstra V times.'

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

All-pairs shortest path.

Visual Diagram

```
Graph:
       0 → 1 (4)
       0 → 2 (1)
       2 → 1 (2)
       1 → 3 (1)
       2 → 3 (5)
```

```
       Matrix:
       dist[0][1] = min(4, 1+2=3) = 3
       dist[0][3] = min(5, 1+2+1=4) = 4
       dist[2][3] = min(5, 2+1=3) = 3

       For k from 0 to V-1:
           For i from 0 to V-1:
               For j from 0 to V-1:
                   dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

Solution Approach

Concept: Dynamic programming with intermediate nodes.

Algorithm

```
Algorithm FLOYD_WARSHALL(graph):
           dist = graph matrix
           For k = 0 to V-1:
               For i = 0 to V-1:
                   For j = 0 to V-1:
                       If dist[i][k] + dist[k][j] < dist[i][j]:
                           dist[i][j] = dist[i][k] + dist[k][j]
           Return dist
```

Java Implementation

```
class Solution {
           public int[][] floydWarshall(int[][] graph) {
               int n = graph.length;
               int[][] dist = new int[n][n];
               for (int i = 0; i < n; i++) {
                   for (int j = 0; j < n; j++) {
                        dist[i][j] = graph[i][j];
                        if (i != j && dist[i][j] == 0) {
                            dist[i][j] = Integer.MAX_VALUE / 2;
                        }
                   }
               }

                 for (int k = 0; k < n; k++) {
                     for (int i = 0; i < n; i++) {
                         for (int j = 0; j < n; j++) {
                             if (dist[i][k] + dist[k][j] < dist[i][j]) {
                                 dist[i][j] = dist[i][k] + dist[k][j];
                             }
                         }
                     }

              }
```

```
            return dist;
        }
    }
```

Time & Space Complexity

Time Complexity: $O(V^3)$ | Space Complexity: $O(V^2)$

338. Find City with Smallest Number of Neighbors

Understanding the Problem NEW

Given a graph of cities with weighted edges and a distance threshold, find the city that has the **fewest** other cities reachable within that threshold distance (with ties broken by choosing the city with the largest index).

Brute Force Approach NEW

Run a separate shortest-path computation (Dijkstra) from every city and count reachable cities within the threshold for each — O(V) Dijkstra runs, each O((V+E) log V).

Why We Optimize NEW

Since you need shortest distances between *every* pair of cities anyway (to count each city's reachable neighbours), this is a natural fit for **Floyd-Warshall** (337) computed once for the whole graph — after that single $O(V^3)$ computation, counting each city's reachable-within-threshold neighbours is just a simple O(V) scan per city ($O(V^2)$ total) over the completed distance matrix. This reuses Floyd-Warshall's all-pairs result directly, rather than running V separate single-source searches, since the problem's need for 'every city's reachability count' aligns exactly with what an all-pairs shortest-path computation already provides.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find city with minimum reachable cities within threshold.

Visual Diagram

```
n=4, edges=[[0,1,3],[1,2,1],[1,3,4],[2,3,1]], threshold=4
       Floyd Warshall distances:
       From 0: [0,3,4,5] → 2 cities (1,2)
       From 1: [3,0,1,2] → 3 cities (0,2,3)
       From 2: [4,1,0,1] → 3 cities (0,1,3)
       From 3: [5,2,1,0] → 2 cities (1,2)
       Result: 0 (tie break by larger index? Actually smaller index with min count)
```

Solution Approach

Concept: Floyd Warshall + counting.

Algorithm

```
Algorithm FIND_CITY(n, edges, threshold):
           dist = FloydWarshall(edges)
           minCount = INF, city = -1

           For i = 0 to n-1:
               count = 0
               For j = 0 to n-1:
                   If i != j and dist[i][j] <= threshold:
                       count++
               If count <= minCount:
                   minCount = count
                   city = i
           Return city
```

Java Implementation

```
class Solution {
           public int findTheCity(int n, int[][] edges, int threshold) {
               int[][] dist = new int[n][n];
               for (int i = 0; i < n; i++) {
                   Arrays.fill(dist[i], Integer.MAX_VALUE / 2);
                   dist[i][i] = 0;
               }
               for (int[] edge : edges) {
                   dist[edge[0]][edge[1]] = edge[2];
                   dist[edge[1]][edge[0]] = edge[2];
               }

                 // Floyd Warshall
                 for (int k = 0; k < n; k++) {
                     for (int i = 0; i < n; i++) {
                         for (int j = 0; j < n; j++) {
                             dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k]
[j]);
                         }
                     }
                 }

                 int minCount = n;
                 int result = -1;
                 for (int i = 0; i < n; i++) {
                     int count = 0;

                   for (int j = 0; j < n; j++) {
                       if (i != j && dist[i][j] <= threshold) {
```

```
                        count++;
                    }
                }
                if (count <= minCount) {
                    minCount = count;
                    result = i;
                }
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: $O(n^3)$ | Space Complexity: $O(n^2)$

339-340. Minimum Spanning Tree & Prim's Algorithm

Understanding the Problem NEW

Given a connected, undirected, weighted graph, find a **Minimum Spanning Tree (MST)** — a subset of edges that connects all nodes together (forming a tree, no cycles) with the minimum possible total edge weight.

Brute Force Approach NEW

Try every possible subset of edges that forms a spanning tree and compare total weights — the number of possible spanning trees can be astronomically large (exponential) for even moderately sized graphs.

Why We Optimize NEW

Prim's Algorithm builds the MST greedily, growing a single connected tree one edge at a time: starting from any node, repeatedly add the **cheapest edge** that connects the current tree to a new, not-yet-included node (using a Min-Heap/ Priority Queue — the same heap-driven 'always grab the current best candidate' pattern from Dijkstra — to efficiently find this cheapest connecting edge at each step). Because adding the cheapest available connecting edge at every step never creates a worse result than any alternative choice (a classic greedy-exchange argument), this builds a guaranteed-optimal MST in O(E log V), instead of exploring the exponentially many possible spanning trees directly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find MST with minimum total edge weight.

Visual Diagram

```
Graph:
       0 --4-- 1
       |       |
       3       2
       2 --3-- 3 --1-- 4

       Prim's Algorithm from 0:
```

```
       MST: 0--3 (2), 3--4 (1), 4--2 (3), 2--1 (2)
       Total: 2+1+3+2 = 8
```

Solution Approach

Concept: Priority queue with visited nodes.

Algorithm

```
Algorithm PRIM(graph):
           visited = boolean[V]
           pq = PriorityQueue((weight, node))
           pq.add((0, 0))
           totalWeight = 0

           While pq not empty:
               (w, node) = pq.poll()
               If visited[node]: continue
               visited[node] = true
               totalWeight += w
               For neighbor, weight in graph[node]:
                   If !visited[neighbor]:
                       pq.add((weight, neighbor))
           Return totalWeight
```

Java Implementation

```
class Solution {
           public int primMST(int n, int[][] edges) {
               List<List<int[]>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(new int[]{edge[1], edge[2]});
                   graph.get(edge[1]).add(new int[]{edge[0], edge[2]});
               }

                 boolean[] visited = new boolean[n];
                 PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] -
b[1]);
                 pq.add(new int[]{0, 0});
                 int totalWeight = 0;

                 while (!pq.isEmpty()) {
                     int[] curr = pq.poll();

                  int node = curr[0], weight = curr[1];
                  if (visited[node]) continue;
```

```
                visited[node] = true;
                totalWeight += weight;

                for (int[] neighbor : graph.get(node)) {
                    if (!visited[neighbor[0]]) {
                        pq.add(new int[]{neighbor[0], neighbor[1]});
                    }
                }
            }
            return totalWeight;
        }
    }
```

Time & Space Complexity

Time Complexity: O(E log V) | Space Complexity: O(V+E)

341-342. Disjoint Set (Union Find)

Understanding the Problem NEW

A **Disjoint Set (Union-Find)** is a specialized data structure for efficiently answering 'are these two elements in the same group?' and 'merge these two groups together' — critical for the next several MST and connectivity-focused problems.

Brute Force Approach NEW

Without a specialized structure, you could represent groups as explicit lists and merge them by concatenating lists — checking group membership might be O(n) per query (scanning a list), and merging two groups could cost O(size of the smaller list) each time, adding up to potentially $O(n^2)$ across many merge operations.

Why We Optimize NEW

Union-Find represents each group as a **tree** where every node points to a 'parent,' with the group's identity determined by its tree's root — **'find'** follows parent pointers up to the root (answering 'which group am I in'), and **'union'** merges two groups by attaching one tree's root under the other's. Two crucial optimizations make this extremely fast: **path compression** (during 'find,' make every visited node point directly to the root, flattening future lookups) and **union by rank/size** (always attach the smaller/shallower tree under the larger/deeper one's root, keeping trees flat). Together, these bring both operations down to nearly O(1) amortized (technically $O(\alpha(n))$, the inverse Ackermann function — effectively constant for any realistic input size) — a dramatic improvement over the naive list-based approach's O(n) per operation, and this structure becomes essential for Kruskal's Algorithm (343) and many later connectivity problems in this chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Union-Find data structure for connectivity.

Visual Diagram

```
Elements: 0,1,2,3,4
       Initial: Each is own parent

       Union(0,1): parent[0]=1
       Union(2,3): parent[2]=3
       Union(1,3): parent[1]=3
       Find(0): 0→1→3 → returns 3
       Find(2): 2→3 → returns 3
       Connected: 0 and 2 are connected

       Union by Rank:
       - Smaller tree attached to larger
       - O(α(n)) amortized
```

Solution Approach

Concept: Parent array with path compression.

Algorithm

```
Algorithm DSU:
           parent = new int[n]
           rank = new int[n]

           find(x):
               If parent[x] != x:
                    parent[x] = find(parent[x])
               Return parent[x]

           union(x, y):
               px = find(x), py = find(y)
               If px == py: return
               If rank[px] < rank[py]: parent[px] = py
               Else If rank[px] > rank[py]: parent[py] = px
               Else: parent[py] = px, rank[px]++
```

Java Implementation

```
class DSU {
           private int[] parent;
           private int[] rank;

           DSU(int n) {
               parent = new int[n];
               rank = new int[n];
               for (int i = 0; i < n; i++) {
                   parent[i] = i;
               }
```

```
        }

        public int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]); // Path compression
            }
            return parent[x];

        }

        public void union(int x, int y) {
            int px = find(x);
            int py = find(y);
            if (px == py) return;

            if (rank[px] < rank[py]) {
                parent[px] = py;
            } else if (rank[px] > rank[py]) {
                parent[py] = px;
            } else {
                parent[py] = px;
                rank[px]++;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(α(n)) amortized | Space Complexity: O(n)

343. Kruskal's Algorithm

Understanding the Problem NEW

An alternative way to find a Minimum Spanning Tree (same goal as Prim's Algorithm, 339-340), this time by considering edges globally sorted by weight rather than growing one connected tree.

Brute Force Approach NEW

Same exponential spanning-tree enumeration as a naive approach to Prim's problem.

Why We Optimize NEW

Kruskal's Algorithm: sort *all* edges in the graph by weight, ascending, then process them one at a time in that order, adding an edge to the MST only if its two endpoints are **currently in different components** (checked instantly using Union-Find's 'find' operation from problems 341-342) — skipping any edge that would connect two nodes already in the same component (which would create a cycle, not allowed in a tree). This greedy 'take the globally cheapest edge that doesn't create a cycle' strategy, powered by Union-Find for near-instant cycle checking, achieves O(E log E) (dominated by the sort), and is often preferred over Prim's when the edge list is more naturally available than an adjacency structure, or when the graph is sparse.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

MST using Kruskal's algorithm.

Visual Diagram

```
Graph:
       0 --4-- 1
       |       |
       3       2
       2 --3-- 3 --1-- 4

       Sort edges by weight:
       (3,4) = 1
```

```
       (0,3) = 2
       (2,3) = 3
       (2,4) = 3
       (0,1) = 4
       (1,2) = 2

       Union:
       (3,4) → union 3,4
       (0,3) → union 0,3
       (2,3) → union 2,3 (connects 2 to 3)
       (2,4) → already connected
       (0,1) → union 0,1 (connects 1 to 0)
       Total: 1+2+3+2 = 8
```

Solution Approach

Concept: Sort edges, union if different components.

Algorithm

```
Algorithm KRUSKAL(edges, n):
           Sort edges by weight
           dsu = DSU(n)
           totalWeight = 0
           count = 0

           For each edge (u, v, w):
               If dsu.find(u) != dsu.find(v):
                   dsu.union(u, v)
                   totalWeight += w
                   count++
                   If count == n-1: break
           Return totalWeight
```

Java Implementation

```
class Solution {
           public int kruskalMST(int n, int[][] edges) {
               Arrays.sort(edges, (a, b) -> a[2] - b[2]);
               DSU dsu = new DSU(n);
               int totalWeight = 0;
               int count = 0;

                 for (int[] edge : edges) {
                     int u = edge[0], v = edge[1], w = edge[2];
                     if (dsu.find(u) != dsu.find(v)) {
                         dsu.union(u, v);

                      totalWeight += w;
                      count++;
```

```
                    if (count == n - 1) break;
                }
            }
            return totalWeight;
        }
    }
```

Time & Space Complexity

Time Complexity: O(E log E) | Space Complexity: O(V)

344. Number of Operations to Make Network Connected

Understanding the Problem NEW

Given a network of computers with some direct connections, and the ability to move an existing cable to connect two currently-disconnected computers, find the minimum number of cable moves needed to connect the entire network (or -1 if there aren't enough cables).

Brute Force Approach NEW

Try every possible way of removing and re-adding cables and check which sequence connects everything with the fewest moves — combinatorially expensive without structure.

Why We Optimize NEW

Use **Union-Find**: process every existing connection, unioning the two computers it connects — after processing all connections, count how many **separate components** remain (using Union-Find's structure to identify this) and how many 'extra' cables exist beyond what's needed to connect within each component (a component with k computers only needs k-1 cables to stay connected; any additional cables within it are 'spare' and can be relocated). If there are at least (number of components - 1) spare cables available, the answer is simply (number of components - 1) — since connecting C separate components together always takes exactly C-1 new connecting cables, mirroring how a spanning tree connects components with the minimum number of edges. This reduces a seemingly complex cable-rearrangement problem to counting components via Union-Find, an O(E) process.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum connections to connect all computers.

Visual Diagram

```
n=4, connections=[[0,1],[0,2],[1,2]]
       Output: 1 (connect 3 to any)
```

```
Count components:
Component 1: [0,1,2]
Component 2: [3]
Need: components-1 = 1 extra connection

If extra cables < components-1: impossible
Extra cables = edges - (n - components)
```

Solution Approach

Concept: DSU to count components.

Algorithm

```
Algorithm MAKE_CONNECTED(n, connections):
           If connections.length < n-1: Return -1
           dsu = DSU(n)
           For each connection: dsu.union(u, v)

           components = 0
           For i = 0 to n-1:
               If dsu.find(i) == i: components++

           Return components - 1
```

Java Implementation

```
class Solution {
           public int makeConnected(int n, int[][] connections) {
               if (connections.length < n - 1) return -1;

                 DSU dsu = new DSU(n);
                 for (int[] conn : connections) {
                     dsu.union(conn[0], conn[1]);
                 }

                 int components = 0;
                 for (int i = 0; i < n; i++) {
                     if (dsu.find(i) == i) {
                         components++;
                     }
                 }
                 return components - 1;
           }
      }
```

Time & Space Complexity

Time Complexity: O(E α(n)) | Space Complexity: O(n)

345. Most Stones Removed with Same Row/Column

Understanding the Problem NEW

Given stones on a 2D grid, you can remove a stone if it shares a row or column with another remaining stone; find the maximum number of stones that can be removed.

Brute Force Approach NEW

Try every possible order and choice of stone removals and track the maximum removable — combinatorially expensive, since removal order and choices interact in complex ways.

Why We Optimize NEW

Reframe using **Union-Find**, but cleverly: instead of treating stones as the elements to union, union **rows and columns** together whenever a stone connects them (using a shared numbering scheme, e.g. row indices and negated/offset column indices to keep them distinct in the same Union-Find structure) — any group of stones connected (directly or transitively) via shared rows/columns forms one Union-Find component, and within any single component, you can always remove all but one stone (by removing them in the right order, always leaving one 'anchor' stone until the very end). So the answer is simply `(total stones) - (number of distinct components)`, computed via one pass unioning rows/columns per stone — turning a seemingly complex order-dependent removal puzzle into a straightforward component-counting problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum stones that can be removed.

Visual Diagram

```
Stones: [[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]
       Groups (connected by row/column):
       Group 1: [0,0], [0,1], [1,0], [2,0]? etc.

       DSU on rows and columns:
       For each stone (i,j):
```

```
            union(i, j+n) // row and column nodes

        Components = count of unique parents
        Removed = stones.length - components
```

Solution Approach

Concept: DSU with row and column nodes.

Algorithm

```
Algorithm REMOVE_STONES(stones):
            n = stones.length
            dsu = DSU(2 * maxCoord)

            For each stone (x, y):
                dsu.union(x, y + maxCoord)

            components = 0
            For each stone (x, y):
                If dsu.find(x) == x: components++
            Return n - components
```

Java Implementation

```
class Solution {
            public int removeStones(int[][] stones) {
                int maxCoord = 10000;
                DSU dsu = new DSU(2 * maxCoord + 1);

                  for (int[] stone : stones) {
                      dsu.union(stone[0], stone[1] + maxCoord);
                  }

                  Set<Integer> components = new HashSet<>();
                  for (int[] stone : stones) {
                      components.add(dsu.find(stone[0]));
                  }
                  return stones.length - components.size();
            }
        }
```

Time & Space Complexity

Time Complexity: O(n α(n)) | Space Complexity: O(n)

346. Accounts Merge

Understanding the Problem NEW

Given a list of accounts, each with a name and a list of associated emails, merge accounts that share at least one email (since they must belong to the same actual person), combining all their emails into one sorted list per merged account.

Brute Force Approach NEW

Compare every pair of accounts to check for a shared email, merging matches, and repeat until no more merges are possible — O(accounts²) comparisons, potentially requiring multiple passes since merging two accounts could newly connect them to a third.

Why We Optimize NEW

Use **Union-Find over emails**: treat every distinct email as an element, and for each account, union all of its emails together (since they all belong to the same person by virtue of appearing in the same account) — after processing every account this way, any two emails ending up in the same Union-Find component belong to the same actual person, regardless of how many accounts originally separated them. Finally, group emails by their root/component and attach the appropriate name to each group. This turns a potentially multi-pass, pairwise-comparison merging problem into a single pass of Union-Find operations (near-O(1) each), followed by one grouping pass — directly reusing Union-Find's efficient 'these things are connected, merge them' capability for a very different-looking real-world problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Merge accounts with common emails.

Visual Diagram

```
Accounts:
      ["John", "johnsmith@mail.com", "john_newyork@mail.com"]
      ["John", "johnsmith@mail.com", "john00@mail.com"]
      ["Mary", "mary@mail.com"]
```

```
      ["John", "johnnybravo@mail.com"]

      Output:
      ["John", "john00@mail.com", "john_newyork@mail.com", "johnsmith@mail.com"]
      ["John", "johnnybravo@mail.com"]
      ["Mary", "mary@mail.com"]

      DSU on emails
      Map email → name
```

Solution Approach

Concept: DSU on emails.

Algorithm

```
Algorithm ACCOUNTS_MERGE(accounts):
          parent = HashMap<String, String>
          emailToName = HashMap<String, String>

          For each account:
              For each email in account[1:]:
                  emailToName.put(email, account[0])
                  parent.put(email, email)
                  If it's not first email:
                      union(firstEmail, email)

          groups = HashMap<String, List<String>>
          For each email in parent:
              root = find(email)
              groups.putIfAbsent(root, new ArrayList<>())
              groups.get(root).add(email)

          result = []
          For each group:
              sorted emails, add name
          Return result
```

Java Implementation

```
class Solution {
          public List<List<String>> accountsMerge(List<List<String>> accounts) {
              Map<String, String> parent = new HashMap<>();
              Map<String, String> emailToName = new HashMap<>();

                 // Initialize DSU
                 for (List<String> account : accounts) {
                     String name = account.get(0);
                     for (int i = 1; i < account.size(); i++) {
                         String email = account.get(i);
```

```
                    parent.put(email, email);

                emailToName.put(email, name);
            }
        }

        // Union emails in same account
        for (List<String> account : accounts) {
            String firstEmail = account.get(1);
            for (int i = 2; i < account.size(); i++) {
                union(parent, firstEmail, account.get(i));
            }
        }

        // Group emails
        Map<String, List<String>> groups = new HashMap<>();
        for (String email : parent.keySet()) {
            String root = find(parent, email);
            groups.computeIfAbsent(root, k -> new ArrayList<>()).add(email);
        }

        // Build result
        List<List<String>> result = new ArrayList<>();
        for (List<String> emails : groups.values()) {
            Collections.sort(emails);
            List<String> account = new ArrayList<>();
            account.add(emailToName.get(emails.get(0)));
            account.addAll(emails);
            result.add(account);
        }
        return result;
    }

    private String find(Map<String, String> parent, String x) {
        if (!parent.get(x).equals(x)) {
            parent.put(x, find(parent, parent.get(x)));
        }
        return parent.get(x);
    }

    private void union(Map<String, String> parent, String x, String y) {
        String px = find(parent, x);
        String py = find(parent, y);
        if (!px.equals(py)) {
            parent.put(px, py);
        }
    }
}
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

347. Number of Islands II

Understanding the Problem NEW

Given a grid that starts entirely water, and a sequence of 'add land' operations (one cell at a time), report the number of distinct islands **after each individual operation** — requiring an efficient way to track connectivity as land is added incrementally, rather than recomputing from scratch each time.

Brute Force Approach NEW

After each single land addition, re-run a full connected-components scan (like problem 306) over the entire grid — O(rows×cols) per addition, giving O(operations × rows × cols) overall, extremely wasteful since only one cell changes between consecutive queries.

Why We Optimize NEW

Use **Union-Find**, incrementally: when a new land cell is added, initially count it as a brand new island (increment a running island count), then check its up/down/left/right neighbours — for every neighbour that's *already land* and in a *different* Union-Find component, union them together and **decrement** the running island count by 1 (since two separate islands just merged into one). This maintains the running island count incrementally, giving each query an O(α(n)) (near-constant) answer instead of a full O(rows×cols) rescan — exploiting the fact that only local changes near the newly-added cell can possibly affect connectivity, rather than needing to reconsider the whole grid.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Number of islands after each add operation.

Visual Diagram

```
n=3, m=3, positions = [[0,0],[0,1],[1,2],[2,1]]
       After [0,0]: 1 island
       After [0,1]: 1 island (connected)
       After [1,2]: 2 islands
       After [2,1]: 2 islands? Actually [2,1] connects to [1,2]
```

Solution Approach

Concept: DSU with dynamic additions.

Algorithm

```
Algorithm NUM_ISLANDS_II(n, m, positions):
           grid = new int[n][m]
           dsu = DSU(n*m)
           directions = [[1,0],[-1,0],[0,1],[0,-1]]
           result = []
           count = 0

           For each position (x, y):
               If grid[x][y] == 1:
                   result.add(count)
                   continue
               grid[x][y] = 1
               count++
               index = x*m + y
               For each direction:
                   nx, ny = x+dx, y+dy
                   If valid and grid[nx][ny] == 1:
                       neighbor = nx*m + ny
                       If dsu.find(index) != dsu.find(neighbor):
                            dsu.union(index, neighbor)
                            count--
               result.add(count)
           Return result
```

Java Implementation

```
class Solution {
           public List<Integer> numIslands2(int n, int m, int[][] positions) {
               int[][] grid = new int[n][m];
               DSU dsu = new DSU(n * m);
               List<Integer> result = new ArrayList<>();
               int count = 0;
               int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

                 for (int[] pos : positions) {
                     int x = pos[0], y = pos[1];
                     if (grid[x][y] == 1) {
                         result.add(count);
                         continue;
                     }
                     grid[x][y] = 1;
                     count++;
                     int index = x * m + y;

                   for (int[] dir : dirs) {
```

```
                        int nx = x + dir[0];
                        int ny = y + dir[1];
                        if (nx >= 0 && nx < n && ny >= 0 && ny < m && grid[nx][ny]
== 1) {
                            int neighbor = nx * m + ny;
                            if (dsu.find(index) != dsu.find(neighbor)) {
                                dsu.union(index, neighbor);
                                count--;
                            }
                        }
                    }
                    result.add(count);
                }
                return result;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n * α(n)) | Space Complexity: O(n)

348. Making a Large Island

Understanding the Problem NEW

Given a binary grid, you're allowed to change *at most one* 0 to a 1; find the size of the largest possible island achievable by choosing the best possible cell to flip.

Brute Force Approach NEW

Try flipping every single 0 cell one at a time, and for each, recompute the resulting largest island size from scratch via a full grid traversal — O(cells) flips considered, each requiring O(cells) to re-evaluate, giving O(cells²) overall.

Why We Optimize NEW

First, run one pass identifying and **labeling every existing island** with a unique ID, also recording each island's size (using DFS/BFS, or equivalently Union-Find grouping connected land cells). Then, for every 0 cell, look at its up/down/left/right neighbours' island IDs — a flip at that cell would merge together *all distinct* islands touching it (plus the newly flipped cell itself), so the resulting size is `1 + sum of sizes of each distinct neighbouring island` (being careful not to double-count the same island if it touches the 0 cell from multiple sides). Precomputing island sizes once (O(cells)) and then evaluating every 0 cell's potential in O(1) (just checking up to 4 neighbours) brings the total down to O(cells), instead of O(cells²) from re-scanning the whole grid for every candidate flip.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Make largest island by flipping one 0 to 1.

Visual Diagram

```
Grid:
       1 0 1
       0 1 0
       1 0 1

       Max island after flipping one 0 = 5
```

```
       DSU on all 1s, then check each 0
```

Solution Approach

Concept: DSU on land cells.

Algorithm

```
Algorithm LARGEST_ISLAND(grid):
           n = grid.length
           dsu = DSU(n*n)

           // Union all adjacent 1s
           For each cell with grid[i][j] == 1:
               For each neighbor with 1:
                   union(index, neighbor)

           // For each 0, check unique adjacent islands
           max = max component size
           For each cell with grid[i][j] == 0:
               Set roots of adjacent 1s
               size = 1 + sum of component sizes
               max = max(max, size)
           Return max
```

Java Implementation

```
class Solution {
           public int largestIsland(int[][] grid) {
               int n = grid.length;
               DSU dsu = new DSU(n * n);
               int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

                 // Union all adjacent 1s
                 for (int i = 0; i < n; i++) {
                     for (int j = 0; j < n; j++) {
                         if (grid[i][j] == 1) {
                             int index = i * n + j;
                             for (int[] dir : dirs) {
                                 int ni = i + dir[0];
                                 int nj = j + dir[1];
                                 if (ni >= 0 && ni < n && nj >= 0 && nj < n &&
grid[ni][nj] == 1) {
                                     dsu.union(index, ni * n + nj);
                                 }
                             }
                         }
                     }
                 }
```

```
                // Get component sizes
                Map<Integer, Integer> sizeMap = new HashMap<>();
                for (int i = 0; i < n; i++) {
                    for (int j = 0; j < n; j++) {
                        if (grid[i][j] == 1) {
                            int root = dsu.find(i * n + j);
                            sizeMap.put(root, sizeMap.getOrDefault(root, 0) + 1);
                        }
                    }
                }

                int max = 0;
                for (int size : sizeMap.values()) {
                    max = Math.max(max, size);
                }

                // Try flipping each 0
                for (int i = 0; i < n; i++) {
                    for (int j = 0; j < n; j++) {
                        if (grid[i][j] == 0) {
                            Set<Integer> roots = new HashSet<>();
                            for (int[] dir : dirs) {
                                int ni = i + dir[0];
                                int nj = j + dir[1];
                                if (ni >= 0 && ni < n && nj >= 0 && nj < n &&
grid[ni][nj] == 1) {
                                    roots.add(dsu.find(ni * n + nj));
                                }
                            }
                            int size = 1;
                            for (int root : roots) {
                                size += sizeMap.get(root);
                            }
                            max = Math.max(max, size);
                        }
                    }
                }
                return max;
            }
        }
```

Time & Space Complexity

Time Complexity: $O(n^2 \alpha(n))$ | Space Complexity: $O(n^2)$

349. Swim in Rising Water

Understanding the Problem NEW

Given a grid representing elevations, and rising water starting at time 0, find the minimum time at which you can travel from the top-left to the bottom-right, where you can only move to a cell once the current water level (time) is ≥ that cell's elevation.

Brute Force Approach NEW

Try every possible path and, for each, compute the minimum time needed (the maximum elevation along that path) — exponential path exploration.

Why We Optimize NEW

This is exactly Path with Minimum Effort (331) in a new costume: you need the path that minimizes the **maximum elevation encountered** along the way (not the sum of elevations) — solved with the same **modified Dijkstra's Algorithm** that tracks 'minimize the max value seen so far' instead of 'minimize the total sum,' using a priority queue to always expand the currently-best (lowest max-elevation-so-far) path first. Recognizing this as the same underlying 'minimize the bottleneck value along a path' problem as 331, just with a different real-world story (water rising vs. hiking effort), avoids needing to design a new algorithm from scratch.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum time to reach bottom-right.

Visual Diagram

```
Grid:
       [0,2]
       [1,3]
       Time = 3

       Dijkstra with threshold:
       Start with time = grid[0][0]
```

```
       PQ by time
       Visit neighbors if time >= grid[neighbor]
```

Solution Approach

Concept: Dijkstra with elevation condition.

Algorithm

```
Algorithm SWIM_IN_WATER(grid):
           n = grid.length
           visited = boolean[n][n]
           pq = PriorityQueue((time, x, y))
           pq.add((grid[0][0], 0, 0))

            While pq not empty:
                (time, x, y) = pq.poll()
                If x == n-1 && y == n-1: return time
                For each neighbor:
                    newTime = Math.max(time, grid[nx][ny])
                    If !visited[nx][ny]:
                        visited[nx][ny] = true
                        pq.add((newTime, nx, ny))
            Return -1
```

Java Implementation

```
class Solution {
           public int swimInWater(int[][] grid) {
               int n = grid.length;
               boolean[][] visited = new boolean[n][n];
               PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] -
b[0]);
               pq.add(new int[]{grid[0][0], 0, 0});
               visited[0][0] = true;
               int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

                  while (!pq.isEmpty()) {
                      int[] curr = pq.poll();
                      int time = curr[0], x = curr[1], y = curr[2];
                      if (x == n-1 && y == n-1) return time;

                     for (int[] dir : dirs) {
                         int nx = x + dir[0];
                         int ny = y + dir[1];
                         if (nx >= 0 && nx < n && ny >= 0 && ny < n && !visited[nx]
[ny]) {
                             visited[nx][ny] = true;
                             pq.add(new int[]{Math.max(time, grid[nx][ny]), nx,
ny});
```

```
                        }

                    }
                }
                return -1;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n² log n) | Space Complexity: O(n²)

350. Bridges in Graph

Understanding the Problem NEW

Find all 'bridges' in an undirected graph — edges whose removal would disconnect the graph (split it into more components than it currently has).

Brute Force Approach NEW

For every single edge, temporarily remove it and run a full connected-components check to see if the graph became more disconnected — O(E) edges to check, each requiring an O(V+E) traversal, giving O(E×(V+E)) overall.

Why We Optimize NEW

Use a single DFS traversal tracking two values per node: its **discovery time** (when it was first visited) and its **low-link value** (the earliest discovery time reachable from it, including through back-edges to ancestors) — an edge (u, v) is a bridge specifically if `low[v] > discovery[u]`, meaning v (and everything below it in the DFS tree) has *no other way* back to u or anything above u except through this exact edge. This computes discovery and low-link values for every node in a single O(V+E) DFS pass, identifying all bridges at once, instead of the O(E×(V+E)) cost of testing every edge's removal individually — this discovery-time/low-link technique is the foundation for both this problem and Articulation Points (351).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all bridges in an undirected graph.

Visual Diagram

```
Graph:
       0 --- 1
       |     |
       3 --- 2

       Bridge: 1-2? Wait, 1-2 is not a bridge
       Bridges: 0-3? Actually all edges are bridges? Let's check
       0-1: Removing disconnects 0 from 1,2,3 → bridge
```

```
1-2: Removing disconnects 2 from 0,1,3 → bridge
2-3: Removing disconnects 3 from 0,1,2 → bridge
3-0: Removing disconnects 0 from 1,2,3 → bridge
All edges are bridges (tree)

With cycle:
0 --- 1
|     |
3 --- 2
No bridges (cycle)
```

Solution Approach

Concept: DFS with discovery and low times.

Algorithm

```
Algorithm BRIDGES(graph):
           disc = new int[V], low = new int[V]
           visited = boolean[V]
           time = 0
           bridges = []

           DFS(node, parent):
               visited[node] = true
               disc[node] = low[node] = ++time

                  For neighbor in graph[node]:
                      If !visited[neighbor]:
                          dfs(neighbor, node)
                          low[node] = min(low[node], low[neighbor])
                          If low[neighbor] > disc[node]:
                              bridges.add((node, neighbor))
                      Else If neighbor != parent:
                          low[node] = min(low[node], disc[neighbor])
```

Java Implementation

```
class Solution {
           private int time = 0;

           public List<int[]> findBridges(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {

                  graph.add(new ArrayList<>());
              }
              for (int[] edge : edges) {
                  graph.get(edge[0]).add(edge[1]);
                  graph.get(edge[1]).add(edge[0]);
```

```
            }

            int[] disc = new int[n];
            int[] low = new int[n];
            boolean[] visited = new boolean[n];
            List<int[]> bridges = new ArrayList<>();

            for (int i = 0; i < n; i++) {
                if (!visited[i]) {
                    dfs(i, -1, graph, disc, low, visited, bridges);
                }
            }
            return bridges;
        }

        private void dfs(int node, int parent, List<List<Integer>> graph,
                         int[] disc, int[] low, boolean[] visited, List<int[]>
bridges) {
            visited[node] = true;
            disc[node] = low[node] = ++time;

            for (int neighbor : graph.get(node)) {
                if (!visited[neighbor]) {
                    dfs(neighbor, node, graph, disc, low, visited, bridges);
                    low[node] = Math.min(low[node], low[neighbor]);
                    if (low[neighbor] > disc[node]) {
                        bridges.add(new int[]{node, neighbor});
                    }
                } else if (neighbor != parent) {
                    low[node] = Math.min(low[node], disc[neighbor]);
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

351. Articulation Point

Understanding the Problem NEW

Find all 'articulation points' (cut vertices) in an undirected graph — nodes whose removal would increase the number of connected components (splitting the graph apart).

Brute Force Approach NEW

For every single node, temporarily remove it (and its edges) and check if the graph becomes more disconnected — O(V) nodes to check, each requiring an O(V+E) traversal.

Why We Optimize NEW

Extend the exact same discovery-time/low-link DFS technique from Bridges (350): a non-root node u is an articulation point if it has a child v in the DFS tree where `low[v] ≥ discovery[u]` (meaning v's subtree has no back-edge escaping past u, so removing u would strand it) — with a special-case rule for the DFS root (which is an articulation point specifically if it has more than one child in the DFS tree, since those children's subtrees can only be connected to each other through the root). This single O(V+E) DFS pass, essentially the same low-link machinery as Bridges but with a slightly different comparison condition (≥ instead of >, plus the root special case), finds every articulation point at once.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find articulation points (cut vertices).

Visual Diagram

```
Graph:
        0 --- 1
        |     |
        3 --- 2

        Articulation points: 1? Actually removing 1 disconnects 2 from 0,3
        Articulation points: 0,1,2,3 (tree)
```

```
        With cycle:
        0 --- 1
        |     |
        3 --- 2
        No articulation points
```

Solution Approach

Concept: DFS with discovery and low times.

Algorithm

```
Algorithm ARTICULATION_POINTS(graph):
            disc = new int[V], low = new int[V]
            visited = boolean[V]
            isArticulation = boolean[V]
            time = 0
            childCount = 0

            DFS(node, parent):
                visited[node] = true
                disc[node] = low[node] = ++time
                childCount = 0

                   For neighbor in graph[node]:
                       If !visited[neighbor]:
                           childCount++
                           dfs(neighbor, node)
                           low[node] = min(low[node], low[neighbor])
                           If parent == -1 && childCount > 1:
                               isArticulation[node] = true
                           If parent != -1 && low[neighbor] >= disc[node]:
                               isArticulation[node] = true
                       Else If neighbor != parent:
                           low[node] = min(low[node], disc[neighbor])
```

Java Implementation

```
class Solution {
            private int time = 0;

            public List<Integer> articulationPoints(int n, int[][] edges) {
                List<List<Integer>> graph = new ArrayList<>();
                for (int i = 0; i < n; i++) {
                    graph.add(new ArrayList<>());
                }
                for (int[] edge : edges) {

                  graph.get(edge[0]).add(edge[1]);
                  graph.get(edge[1]).add(edge[0]);
```

```
            }

            int[] disc = new int[n];
            int[] low = new int[n];
            boolean[] visited = new boolean[n];
            boolean[] isArticulation = new boolean[n];

            for (int i = 0; i < n; i++) {
                if (!visited[i]) {
                    dfs(i, -1, graph, disc, low, visited, isArticulation);
                }
            }

            List<Integer> result = new ArrayList<>();
            for (int i = 0; i < n; i++) {
                if (isArticulation[i]) result.add(i);
            }
            return result;
        }

        private void dfs(int node, int parent, List<List<Integer>> graph,
                         int[] disc, int[] low, boolean[] visited, boolean[]
isArticulation) {
            visited[node] = true;
            disc[node] = low[node] = ++time;
            int children = 0;

            for (int neighbor : graph.get(node)) {
                if (!visited[neighbor]) {
                    children++;
                    dfs(neighbor, node, graph, disc, low, visited,
isArticulation);
                    low[node] = Math.min(low[node], low[neighbor]);

                    if (parent == -1 && children > 1) {
                        isArticulation[node] = true;
                    }
                    if (parent != -1 && low[neighbor] >= disc[node]) {
                        isArticulation[node] = true;
                    }
                } else if (neighbor != parent) {
                    low[node] = Math.min(low[node], disc[neighbor]);
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

352. Kosaraju's Algorithm

Understanding the Problem NEW

Find all 'Strongly Connected Components' (SCCs) in a *directed* graph — maximal groups of nodes where every node in the group can reach every other node in the same group via directed paths (a much stricter notion of 'connected' than the undirected version, since direction matters).

Brute Force Approach NEW

For every pair of nodes, check if each can reach the other (two separate reachability checks per pair) — $O(V^2)$ pairs, each requiring an $O(V+E)$ traversal, giving $O(V^2 \times (V+E))$ overall.

Why We Optimize NEW

Kosaraju's Algorithm uses an elegant two-pass DFS strategy: first, do a DFS on the original graph, pushing each node onto a stack as it *finishes* (exactly like Topological Sort, problem 319); then, **reverse every edge** in the graph (transpose it); finally, pop nodes off the stack one at a time, and for each not-yet-visited node, run DFS on the *reversed* graph — each such DFS call discovers exactly one full SCC. This works because reversing the graph and processing in the finish-time-descending order (from the first pass) guarantees that a DFS in the second pass can't 'escape' into a different SCC without having already been captured within its own. Two $O(V+E)$ DFS passes total (plus the $O(V+E)$ edge-reversal step) gives $O(V+E)$ overall — dramatically better than the $O(V^2 \times (V+E))$ pairwise reachability check, by cleverly using the finishing-time ordering to guarantee each second-pass DFS stays contained within exactly one SCC.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find strongly connected components in directed graph.

Visual Diagram

```
Graph:
       0 → 1 → 2
       ■ ↓
       3 ← 4
```

```
Step 1: DFS order
DFS from 0: 0→1→2→4→3
Order: [0,1,2,4,3]

Step 2: Reverse graph
0 ← 1 ← 2
■ ↑
3 → 4

Step 3: DFS on reversed graph in reverse order
From 0: 0→3→4→1→2
SCC: [0,1,2,3,4]
```

Solution Approach

Concept: Two DFS passes.

Algorithm

```
Algorithm KOSARAJU(graph):
           // Pass 1: Ordering
           visited = boolean[V]
           stack = Stack()
           For i = 0 to V-1:
               If !visited[i]:
                   dfs1(i, visited, stack, graph)

           // Pass 2: Reverse graph
           revGraph = reverse(graph)
           visited = new boolean[V]
           sccs = []

           While stack not empty:
               node = stack.pop()
               If !visited[node]:
                   scc = []
                   dfs2(node, visited, scc, revGraph)
                   sccs.add(scc)
           Return sccs
```

Java Implementation

```
class Solution {
           public List<List<Integer>> kosaraju(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               List<List<Integer>> reverseGraph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
                   reverseGraph.add(new ArrayList<>());
```

```
            }

            for (int[] edge : edges) {
                graph.get(edge[0]).add(edge[1]);
                reverseGraph.get(edge[1]).add(edge[0]);
            }

            // Pass 1: Ordering
            boolean[] visited = new boolean[n];
            Stack<Integer> stack = new Stack<>();
            for (int i = 0; i < n; i++) {
                if (!visited[i]) {
                    dfs1(i, graph, visited, stack);
                }
            }

            // Pass 2: SCC
            Arrays.fill(visited, false);
            List<List<Integer>> sccs = new ArrayList<>();
            while (!stack.isEmpty()) {
                int node = stack.pop();
                if (!visited[node]) {
                    List<Integer> scc = new ArrayList<>();
                    dfs2(node, reverseGraph, visited, scc);
                    sccs.add(scc);
                }
            }
            return sccs;
        }

        private void dfs1(int node, List<List<Integer>> graph, boolean[] visited,
Stack<Integer> st
            visited[node] = true;
            for (int neighbor : graph.get(node)) {
                if (!visited[neighbor]) {
                    dfs1(neighbor, graph, visited, stack);
                }
            }
            stack.push(node);
        }

        private void dfs2(int node, List<List<Integer>> graph, boolean[] visited,
List<Integer> scc
            visited[node] = true;
            scc.add(node);
            for (int neighbor : graph.get(node)) {
                if (!visited[neighbor]) {
                    dfs2(neighbor, graph, visited, scc);
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V+E)

Graph Summary

Problem	Type	Time	Space	Key Technique
BFS/DFS	Learning	O(V+E)	O(V)	Queue/Stack
Provinces	BFS/DFS	$O(n^2)$	O(n)	DFS on matrix
Rotten Oranges	BFS/DFS	O(m*n)	O(m*n)	Multi-source BFS
Flood Fill	BFS/DFS	O(m*n)	O(m*n)	DFS with color
Cycle Undirected	BFS/DFS	O(V+E)	O(V)	Parent DFS
0/1 Matrix	BFS/DFS	O(m*n)	O(m*n)	Multi-source BFS
Surrounded Regions	BFS/DFS	O(m*n)	O(m*n)	Boundary DFS
Number of Enclaves	BFS/DFS	O(m*n)	O(m*n)	Boundary DFS
Word Ladder	BFS/DFS	O(n*L*26)	O(n)	BFS transformation
Distinct Islands	BFS/DFS	O(m*n)	O(m*n)	Shape normalization
Bipartite	BFS/DFS	O(V+E)	O(V)	2-coloring
Cycle Directed	BFS/DFS	O(V+E)	O(V)	3-state DFS
Topological Sort	Topo	O(V+E)	O(V)	DFS/Kahn
Kahn's Algorithm	Topo	O(V+E)	O(V)	Indegree queue
Course Schedule	Topo	O(V+E)	O(V)	Cycle detection
Alien Dictionary	Topo	O(n*L)	O(V+E)	Graph + Topo
Shortest Path UG	SP	O(V+E)	O(V)	BFS
Shortest Path DAG	SP	O(V+E)	O(V)	Topo + relaxation
Dijkstra	SP	O((V+E)logV)	O(V)	Priority queue
Binary Maze	SP	O(m*n)	O(m*n)	BFS on grid
Minimum Effort	SP	O(m*n log(mn))	O(m*n)	Modified Dijkstra
Cheapest Flights	SP	O(k*E)	O(V)	Bellman-Ford
Network Delay	SP	O((V+E)logV)	O(V)	Dijkstra
Count Ways	SP	O((V+E)logV)	O(V)	Dijkstra + DP
Bellman-Ford	SP	O(V*E)	O(V)	Relaxation
Floyd Warshall	SP	$O(V^3)$	$O(V^2)$	All-pairs DP
Prim's	MST	O(E log V)	O(V+E)	Priority queue
DSU	MST	$O(\alpha(n))$	O(n)	Union-Find

Kruskal	MST	O(E log E)	O(V)	DSU on sorted edges
Make Connected	DSU	O(E α(n))	O(n)	DSU counting
Accounts Merge	DSU	O(n log n)	O(n)	DSU on emails
Islands II	DSU	O(n α(n))	O(n)	Dynamic DSU
Largest Island	DSU	O(n² α(n))	O(n²)	DSU + flipping
Swim in Water	DSU/Dijkstra	O(n² log n)	O(n²)	Dijkstra with elevation
Bridges	Advanced	O(V+E)	O(V)	DFS low/disc
Articulation Point	Advanced	O(V+E)	O(V)	DFS low/disc
Kosaraju	Advanced	O(V+E)	O(V+E)	Two-pass DFS

— End of enhanced Chapter 12: Graphs — remaining chapters continue in the full edition —

Dynamic Programming — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

353. Dynamic Programming Introduction

Understanding the Problem NEW

Dynamic Programming (DP) is a technique for speeding up recursive problems that recompute the **same subproblem** many times — instead of solving it fresh every time it's encountered, you solve it once and **remember (cache)** the answer for reuse. This chapter's whole theme is spotting 'overlapping subproblems' hiding inside a recursive solution and eliminating that redundant recomputation.

Brute Force Approach NEW

A plain recursive solution to a DP problem (like naive Fibonacci, computing fib(n) = fib(n-1) + fib(n-2)) recomputes the same smaller subproblems exponentially many times — e.g. fib(5) calls fib(3) twice, fib(2) three times, and so on, giving $O(2^n)$ time for something that has only n distinct subproblems.

Why We Optimize NEW

Two equivalent ways to eliminate the redundant recomputation: **Memoization** (top-down) keeps the natural recursive structure, but stores each subproblem's answer in an array/hashmap the first time it's computed, and simply returns the cached value on any later request instead of recomputing; **Tabulation** (bottom-up) instead builds up the answer iteratively from the smallest subproblems to the largest, filling a table in order, with no recursion at all. Both bring a problem like naive Fibonacci down from $O(2^n)$ to O(n) — this 'recognize repeated subproblems, cache instead of recompute' idea is the single unifying theme behind every problem in this chapter, however different they look on the surface.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding DP - optimization over recursion with overlapping subproblems.

Visual Diagram

```
Fibonacci: f(n) = f(n-1) + f(n-2)

       Recursion Tree (f(5)):
                              f(5)
```

```
                          /       \
                    f(4)            f(3)
                  /      \        /      \
              f(3)      f(2)    f(2)    f(1)
            /    \     /    \ /    \
          f(2) f(1) f(1) f(0) f(1) f(0)
        /    \
       f(1) f(0)

       DP Approaches:
       1. Memoization (Top-Down)
       2. Tabulation (Bottom-Up)
       3. Space Optimization

       DP Steps:
       1. Identify DP state
       2. Write recurrence
       3. Base cases
       4. Fill DP table / Memoize
```

Java Implementation

```
class Solution {

           // 1. Memoization (Top-Down)
           public int fibMemo(int n) {
               int[] dp = new int[n + 1];
               Arrays.fill(dp, -1);
               return fibHelper(n, dp);
           }

           private int fibHelper(int n, int[] dp) {
               if (n <= 1) return n;
               if (dp[n] != -1) return dp[n];
               dp[n] = fibHelper(n - 1, dp) + fibHelper(n - 2, dp);
               return dp[n];
           }

           // 2. Tabulation (Bottom-Up)
           public int fibTab(int n) {
               if (n <= 1) return n;
               int[] dp = new int[n + 1];
               dp[0] = 0;
               dp[1] = 1;
               for (int i = 2; i <= n; i++) {
                   dp[i] = dp[i - 1] + dp[i - 2];
               }
               return dp[n];
           }

           // 3. Space Optimized
           public int fibSpace(int n) {
```

```
            if (n <= 1) return n;
            int prev2 = 0, prev1 = 1;
            for (int i = 2; i <= n; i++) {
                int curr = prev1 + prev2;
                prev2 = prev1;
                prev1 = curr;
            }
            return prev1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n) or O(1)

354. Climbing Stairs

Understanding the Problem NEW

Count the number of distinct ways to climb n stairs, if you can take either 1 or 2 steps at a time.

Brute Force Approach NEW

A direct recursive solution — `ways(n) = ways(n-1) + ways(n-2)` (since your last step was either a 1-step from stair n-1, or a 2-step from stair n-2) — looks correct but recomputes the same smaller values of `ways(k)` exponentially many times, exactly like naive Fibonacci, giving $O(2^n)$.

Why We Optimize NEW

This is literally the Fibonacci recurrence in disguise. Apply **memoization** (cache each `ways(k)` the first time it's computed) or **tabulation** (build up `ways(1), ways(2), ..., ways(n)` iteratively in an array) to eliminate the repeated recomputation, bringing it down to O(n) time (and this can be further reduced to O(1) space, since each answer only ever needs the previous two values, not the whole table) — the very first and simplest illustration of the memoization/tabulation idea introduced in problem 353.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Ways to climb n stairs taking 1 or 2 steps.

Visual Diagram

```
n = 4:
       Ways:
       1+1+1+1
       1+1+2
       1+2+1
       2+1+1
       2+2
       Total: 5

       DP: dp[i] = dp[i-1] + dp[i-2]
       dp[0] = 1
```

```
        dp[1] = 1
        dp[2] = 2
        dp[3] = 3
        dp[4] = 5
```

Solution Approach

Concept: Fibonacci pattern.

Algorithm

```
Algorithm CLIMB_STAIRS(n):
            If n <= 1: Return 1
            dp[0] = 1, dp[1] = 1
            For i = 2 to n:
                dp[i] = dp[i-1] + dp[i-2]
            Return dp[n]
```

Java Implementation

```
class Solution {
            public int climbStairs(int n) {
                if (n <= 1) return 1;
                int prev2 = 1, prev1 = 1;
                for (int i = 2; i <= n; i++) {
                    int curr = prev1 + prev2;
                    prev2 = prev1;
                    prev1 = curr;
                }
                return prev1;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

355. Frog Jump (DP-3)

Understanding the Problem NEW

A frog needs to jump across stones (positions 0 to n-1), able to jump either 1 or 2 stones ahead each time, paying a cost equal to the height difference between the stones it jumps between; find the minimum total cost to reach the last stone.

Brute Force Approach NEW

A direct recursion — `minCost(i) = min(minCost(i-1) + cost(i-1,i), minCost(i-2) + cost(i-2,i))` — again recomputes the same `minCost(k)` for smaller k exponentially many times if implemented naively without caching.

Why We Optimize NEW

Same fix as Climbing Stairs: memoize (or tabulate) `minCost(i)` for each stone, since each value only needs to be computed once and reused whenever a later stone's calculation needs it. This is essentially 'Climbing Stairs, but minimizing a cost instead of counting ways' — recognizing the shared recursive shape (each state depends only on the previous 1-2 states) is what tells you the same memoization/tabulation fix applies, turning $O(2^n)$ into O(n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum cost to reach last stone with jumps of 1 or 2.

Visual Diagram

```
Heights: [10, 30, 40, 20]
       Costs:
       dp[0] = 0
       dp[1] = |30-10| = 20
       dp[2] = min(|40-30| + dp[1], |40-10| + dp[0]) = min(10+20, 30+0) = 30
       dp[3] = min(|20-40| + dp[2], |20-30| + dp[1]) = min(20+30, 10+20) = 30
       Result: 30
```

Solution Approach

Concept: DP with two-step transitions.

Algorithm

```
Algorithm FROG_JUMP(heights):
           n = heights.length
           dp = new int[n]
           dp[0] = 0
           dp[1] = abs(heights[1] - heights[0])

           For i = 2 to n-1:
               one = dp[i-1] + abs(heights[i] - heights[i-1])
               two = dp[i-2] + abs(heights[i] - heights[i-2])
               dp[i] = min(one, two)

           Return dp[n-1]
```

Java Implementation

```
class Solution {
           public int frogJump(int[] heights) {
               int n = heights.length;
               int[] dp = new int[n];
               dp[0] = 0;
               dp[1] = Math.abs(heights[1] - heights[0]);

                 for (int i = 2; i < n; i++) {
                     int one = dp[i-1] + Math.abs(heights[i] - heights[i-1]);
                     int two = dp[i-2] + Math.abs(heights[i] - heights[i-2]);
                     dp[i] = Math.min(one, two);
                 }
                 return dp[n-1];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

356. Frog Jump with K Distances (DP-4)

Understanding the Problem NEW

Same as Frog Jump, but now the frog can jump up to K stones ahead (not just 1 or 2), still minimizing total jump cost.

Brute Force Approach NEW

Direct recursion — `minCost(i) = min over j=1..K of (minCost(i-j) + cost(i-j, i))` — without caching, this recomputes shared subproblems exponentially, and even with caching, naively trying all K options at every position costs O(n×K) if not careful.

Why We Optimize NEW

Same memoization/tabulation fix as problem 355, generalized: cache `minCost(i)` for each position, computed by trying all K possible jump-back distances — achieving O(n×K) total (n positions, each considering up to K options), a direct generalization of the 2-choice version to a K-choice version, with the same underlying 'cache each position's answer once' principle.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Frog can jump up to k steps.

Visual Diagram

```
Heights: [10, 30, 40, 50, 20], k=3
       dp[i] = min over j=i-k..i-1 of dp[j] + abs(h[i] - h[j])
```

Solution Approach

Concept: DP with k-step transitions.

Algorithm

```
Algorithm FROG_JUMP_K(heights, k):
           n = heights.length
           dp = new int[n]
```

```
            dp[0] = 0

            For i = 1 to n-1:
                minCost = INF
                For j = max(0, i-k) to i-1:
                    minCost = min(minCost, dp[j] + abs(heights[i] - heights[j]))
                dp[i] = minCost

            Return dp[n-1]
```

Java Implementation

```
class Solution {
            public int frogJumpK(int[] heights, int k) {
                int n = heights.length;
                int[] dp = new int[n];
                dp[0] = 0;

                  for (int i = 1; i < n; i++) {
                      int minCost = Integer.MAX_VALUE;
                      for (int j = Math.max(0, i - k); j < i; j++) {
                          minCost = Math.min(minCost,
                              dp[j] + Math.abs(heights[i] - heights[j]));
                      }
                      dp[i] = minCost;
                  }
                  return dp[n-1];
            }
       }
```

Time & Space Complexity

Time Complexity: O(n*k) | Space Complexity: O(n)

357. Maximum Sum of Non-Adjacent Elements

Understanding the Problem NEW

Given an array, find the maximum sum achievable by selecting elements such that no two selected elements are adjacent to each other.

Brute Force Approach NEW

Try every possible subset that avoids adjacent pairs and compute sums — exponential (2^n possible 'valid or invalid' subsets to consider without a clear plan).

Why We Optimize NEW

At each position i, you face a simple binary choice: **either include** element i (in which case you must skip i-1, adding `arr[i] + bestSum(i-2)`) **or exclude** it (keeping `bestSum(i-1)` unchanged) — take whichever is larger. This recursive choice, memoized (or tabulated) per position, computes the answer in O(n) instead of exponential subset enumeration — and since each position only ever needs the previous two computed values, this can be further optimized to O(1) space, just like Climbing Stairs, by keeping two running variables instead of a full array.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum sum without taking adjacent elements.

Visual Diagram

```
Array: [2, 1, 4, 9]
       dp[0] = 2
       dp[1] = max(2, 1) = 2
       dp[2] = max(dp[1], dp[0] + 4) = max(2, 2+4=6) = 6
       dp[3] = max(dp[2], dp[1] + 9) = max(6, 2+9=11) = 11
       Result: 11 (2+9)

       Recurrence: dp[i] = max(dp[i-1], dp[i-2] + arr[i])
```

Solution Approach

Concept: Include/exclude current element.

Algorithm

```
Algorithm MAX_NON_ADJACENT(arr):
           n = arr.length
           if n == 1: return arr[0]
           prev2 = arr[0]
           prev1 = max(arr[0], arr[1])

           For i = 2 to n-1:
               curr = max(prev1, prev2 + arr[i])
               prev2 = prev1
               prev1 = curr

           Return prev1
```

Java Implementation

```
class Solution {
           public int maxNonAdjacentSum(int[] nums) {
               int n = nums.length;
               if (n == 0) return 0;
               if (n == 1) return nums[0];

                 int prev2 = nums[0];
                 int prev1 = Math.max(nums[0], nums[1]);

                 for (int i = 2; i < n; i++) {
                     int curr = Math.max(prev1, prev2 + nums[i]);
                     prev2 = prev1;
                     prev1 = curr;
                 }
                 return prev1;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

358. House Robber (DP-6)

Understanding the Problem NEW

The exact same problem as Maximum Sum of Non-Adjacent Elements (357), just given a real-world story: a robber can't rob two adjacent houses, maximize total loot.

Brute Force Approach NEW

Same exponential subset enumeration as 357 without a clear DP plan.

Why We Optimize NEW

Identical technique to problem 357 — recognizing this is the same underlying DP recurrence wearing a different name is the entire point of including both versions back to back, reinforcing that many 'differently-named' problems are really the same recurrence relation.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum money without robbing adjacent houses.

Visual Diagram

```
Houses: [2, 7, 9, 3, 1]
      dp[0] = 2
      dp[1] = max(2,7) = 7
      dp[2] = max(7, 2+9=11) = 11
      dp[3] = max(11, 7+3=10) = 11
      dp[4] = max(11, 11+1=12) = 12
      Result: 12 (2+9+1)

      Same as max non-adjacent sum.
```

Solution Approach

Concept: Same as max non-adjacent sum.

Algorithm

```
Algorithm HOUSE_ROBBER(nums):
          return MAX_NON_ADJACENT(nums)
```

Java Implementation

```
class Solution {
          public int rob(int[] nums) {
              if (nums.length == 0) return 0;
              if (nums.length == 1) return nums[0];

                 int prev2 = nums[0];
                 int prev1 = Math.max(nums[0], nums[1]);

                 for (int i = 2; i < nums.length; i++) {
                     int curr = Math.max(prev1, prev2 + nums[i]);
                     prev2 = prev1;
                     prev1 = curr;
                 }
                 return prev1;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

359. Ninja's Training (DP-7)

Understanding the Problem NEW

A ninja trains over n days, choosing one of 3 possible activities each day (with associated points), but can't repeat the *same* activity on two consecutive days; maximize total points across all days.

Brute Force Approach NEW

Try every possible sequence of daily activity choices respecting the no-repeat-adjacent-day constraint — 3^n possible sequences, exponential.

Why We Optimize NEW

Track, for each day, the best achievable total points *given which activity was chosen the previous day* — a slightly richer DP state than the simple 'best sum so far' of House Robber, since the decision now depends on more than just 'did I pick the previous position or not.' At each day, for each of the 3 possible 'yesterday's activity' states, compute the best total by choosing any activity *except* yesterday's, using the previous day's already-computed best values for each of those combinations. This memoized/tabulated approach computes the answer in $O(n \times 3 \times 3)$ (a small constant multiplier per day) instead of the exponential 3^n sequence enumeration, by tracking a slightly larger 'state' (day, previous activity) rather than just 'day.'

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Max points with no same activity on consecutive days.

Visual Diagram

```
Points: [[10,40,70], [20,50,80], [30,60,90]]
       dp[day][activity] = max points

       Day 0: [10,40,70]
       Day 1:
         activity 0: 20 + max(40,70) = 20+70=90
         activity 1: 50 + max(10,70) = 50+70=120
         activity 2: 80 + max(10,40) = 80+40=120
```

```
       Day 2:
         activity 0: 30 + max(120,120) = 150
         activity 1: 60 + max(90,120) = 180
         activity 2: 90 + max(90,120) = 210
       Result: 210
```

Solution Approach

Concept: 2D DP with activity constraint.

Algorithm

```
Algorithm NINJA_TRAINING(points):
           n = points.length
           dp = new int[n][3]
           dp[0] = points[0]

           For i = 1 to n-1:
               For j = 0 to 2:
                   For k = 0 to 2:
                       If k != j:
                           dp[i][j] = max(dp[i][j], dp[i-1][k] + points[i][j])

           Return max(dp[n-1])
```

Java Implementation

```
class Solution {
           public int ninjaTraining(int[][] points) {
               int n = points.length;
               int[][] dp = new int[n][3];
               dp[0] = points[0];

                 for (int i = 1; i < n; i++) {
                     for (int j = 0; j < 3; j++) {
                         for (int k = 0; k < 3; k++) {
                             if (k != j) {
                                 dp[i][j] = Math.max(dp[i][j], dp[i-1][k] +
points[i][j]);
                             }
                         }
                     }
                 }
                 return Math.max(dp[n-1][0], Math.max(dp[n-1][1], dp[n-1][2]));
           }

     }
```

Time & Space Complexity

Time Complexity: O(n*3*3) | Space Complexity: O(n*3)

360. Grid Unique Paths (DP-8)

Understanding the Problem NEW

Count the number of unique paths from the top-left to bottom-right of an m×n grid, moving only right or down at each step.

Brute Force Approach NEW

Try every possible sequence of right/down moves recursively and count how many reach the destination — exponential, since the recursive branching (right or down at every cell) explores many overlapping paths through shared intermediate cells repeatedly.

Why We Optimize NEW

The number of paths to reach any cell (r, c) is simply `paths(r-1, c) + paths(r, c-1)` (you must have arrived from either directly above or directly to the left) — memoizing (or tabulating, filling the grid row by row) this per-cell value means each cell's path-count is computed exactly once and reused by its neighbours below/right, achieving O(m×n) instead of the exponential path-by-path recursive enumeration. (There's also a purely combinatorial closed-form using binomial coefficients — choosing which of the (m+n-2) total moves are 'down' — but the DP approach generalizes more easily to variants like the next problem, which adds obstacles.)

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Number of ways to reach bottom-right from top-left.

Visual Diagram

```
Grid: 3x3
       Ways:
       [1,1,1]
       [1,2,3]
       [1,3,6]
       dp[i][j] = dp[i-1][j] + dp[i][j-1]
       Result: 6
```

```
       Combinatorics: C(m+n-2, m-1)
```

Solution Approach

Concept: DP with left and top cells.

Algorithm

```
Algorithm UNIQUE_PATHS(m, n):
           dp = new int[m][n]
           For i = 0 to m-1: dp[i][0] = 1
           For j = 0 to n-1: dp[0][j] = 1

            For i = 1 to m-1:
                For j = 1 to n-1:
                    dp[i][j] = dp[i-1][j] + dp[i][j-1]

            Return dp[m-1][n-1]
```

Java Implementation

```
class Solution {
           public int uniquePaths(int m, int n) {
               int[][] dp = new int[m][n];
               for (int i = 0; i < m; i++) dp[i][0] = 1;
               for (int j = 0; j < n; j++) dp[0][j] = 1;

                 for (int i = 1; i < m; i++) {
                     for (int j = 1; j < n; j++) {
                         dp[i][j] = dp[i-1][j] + dp[i][j-1];
                     }
                 }
                 return dp[m-1][n-1];
            }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

361. Grid Unique Paths 2 (DP-9)

Understanding the Problem NEW

Same as Unique Paths, but now some grid cells are obstacles that can't be passed through — count unique paths avoiding all obstacles.

Brute Force Approach NEW

Same exponential path enumeration as problem 360, now also needing to check at every step whether the path has hit an obstacle.

Why We Optimize NEW

Identical DP recurrence to problem 360, with one adjustment: any cell that's an obstacle simply has 0 paths through it (instead of summing from its neighbours, its value is forced to 0, and it doesn't contribute paths to cells after it). This shows how a DP recurrence can be trivially extended to handle a new constraint (obstacles) by adding one conditional check, without needing a fundamentally different algorithm — the underlying 'sum of ways from above and from the left' structure is unchanged.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Unique paths with obstacles.

Visual Diagram

```
Grid:
       [0,0,0]
       [0,1,0]
       [0,0,0]
       dp:
       [1,1,1]
       [1,0,1]
       [1,1,2]
       Result: 2

       If obstacle: dp[i][j] = 0
       Else: dp[i][j] = dp[i-1][j] + dp[i][j-1]
```

Solution Approach

Concept: DP with obstacle handling.

Algorithm

```
Algorithm UNIQUE_PATHS_OBSTACLE(grid):
           m = grid.length, n = grid[0].length
           if grid[0][0] == 1: return 0
           dp = new int[m][n]
           dp[0][0] = 1

           For i = 0 to m-1:
               For j = 0 to n-1:
                   If grid[i][j] == 1: dp[i][j] = 0
                   Else:
                       If i > 0: dp[i][j] += dp[i-1][j]
                       If j > 0: dp[i][j] += dp[i][j-1]

           Return dp[m-1][n-1]
```

Java Implementation

```
class Solution {
           public int uniquePathsWithObstacles(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               if (grid[0][0] == 1) return 0;

                 int[][] dp = new int[m][n];
                 dp[0][0] = 1;

                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         if (grid[i][j] == 1) {
                             dp[i][j] = 0;
                         } else {
                             if (i > 0) dp[i][j] += dp[i-1][j];
                             if (j > 0) dp[i][j] += dp[i][j-1];
                         }
                     }
                 }
                 return dp[m-1][n-1];

          }
      }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

362. Minimum Path Sum in Grid (DP-10)

Understanding the Problem NEW

Given a grid of costs, find the minimum sum path from top-left to bottom-right, moving only right or down.

Brute Force Approach NEW

Try every possible right/down path and sum costs, tracking the minimum — same exponential path enumeration as Unique Paths.

Why We Optimize NEW

Same recurrence shape as Unique Paths, with 'sum the ways' replaced by 'take the minimum': `minCost(r,c) = grid[r][c] + min(minCost(r-1,c), minCost(r,c-1))`, memoized/tabulated per cell. This directly reuses the grid-DP recurrence pattern from problems 360-361, just swapping the combination operation from addition (counting paths) to minimum (minimizing cost) — a recurring theme throughout this chapter: the same recursive 'shape' often applies to counting, summing, and optimizing versions of what's structurally the same problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum path sum from top-left to bottom-right.

Visual Diagram

```
Grid:
       [1,3,1]
       [1,5,1]
       [4,2,1]
       dp:
       [1,4,5]
       [2,7,6]
       [6,8,7]
       Result: 7 (1→3→1→1→1)

       dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])
```

Solution Approach

Concept: DP with min of left and top.

Algorithm

```
Algorithm MIN_PATH_SUM(grid):
           m = grid.length, n = grid[0].length
           dp = new int[m][n]
           dp[0][0] = grid[0][0]

           For i = 1 to m-1: dp[i][0] = dp[i-1][0] + grid[i][0]
           For j = 1 to n-1: dp[0][j] = dp[0][j-1] + grid[0][j]

           For i = 1 to m-1:
               For j = 1 to n-1:
                   dp[i][j] = grid[i][j] + Math.min(dp[i-1][j], dp[i][j-1])

           Return dp[m-1][n-1]
```

Java Implementation

```
class Solution {
           public int minPathSum(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               int[][] dp = new int[m][n];
               dp[0][0] = grid[0][0];

                 for (int i = 1; i < m; i++) dp[i][0] = dp[i-1][0] + grid[i][0];
                 for (int j = 1; j < n; j++) dp[0][j] = dp[0][j-1] + grid[0][j];

                 for (int i = 1; i < m; i++) {
                     for (int j = 1; j < n; j++) {
                         dp[i][j] = grid[i][j] + Math.min(dp[i-1][j], dp[i][j-1]);
                     }
                 }
                 return dp[m-1][n-1];
           }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

363. Minimum Path Sum in Triangular Grid (DP-11)

Understanding the Problem NEW

Given a triangular arrangement of numbers, find the minimum sum path from the top to the bottom row, where each step can move to one of the two numbers directly below the current position.

Brute Force Approach NEW

Try every possible path down the triangle and sum values — exponential (2 choices at each of n rows).

Why We Optimize NEW

Exactly the same 'minimum path sum' DP shape as problem 362, just on a triangular grid instead of a rectangular one: `minCost(r,c) = triangle[r][c] + min(minCost(r+1,c), minCost(r+1,c+1))` (computed bottom-up, since a triangle's natural 'end' is the single top cell, making bottom-up tabulation especially clean here), memoized/tabulated per position — same O(rows×cols) improvement over exponential path enumeration.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum path sum in triangular grid.

Visual Diagram

```
Triangle:
       [2]
       [3,4]
       [6,5,7]
       [4,1,8,3]

       dp:
       [2]
       [5,6]
       [11,10,13]
       [15,11,18,16]
       Result: 11
```

```
Bottom-up: dp[i][j] = triangle[i][j] + min(dp[i+1][j], dp[i+1][j+1])
```

Solution Approach

Concept: DP from bottom to top.

Algorithm

```
Algorithm MIN_PATH_TRIANGLE(triangle):
           n = triangle.size()
           dp = new int[n]
           dp = triangle[n-1]

           For i = n-2 down to 0:
               For j = 0 to i:
                   dp[j] = triangle[i][j] + Math.min(dp[j], dp[j+1])

           Return dp[0]
```

Java Implementation

```
class Solution {
           public int minimumTotal(List<List<Integer>> triangle) {
               int n = triangle.size();
               int[] dp = new int[n];
               for (int i = 0; i < n; i++) {
                   dp[i] = triangle.get(n-1).get(i);
               }

                 for (int i = n-2; i >= 0; i--) {
                     for (int j = 0; j <= i; j++) {
                         dp[j] = triangle.get(i).get(j) + Math.min(dp[j], dp[j+1]);
                     }
                 }
                 return dp[0];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

364. Minimum/Maximum Falling Path Sum (DP-12)

Understanding the Problem NEW

Given a grid, find the minimum (or maximum) sum path that starts anywhere in the top row and ends anywhere in the bottom row, moving down to any of the 3 cells directly below (down-left, straight down, down-right) at each step.

Brute Force Approach NEW

Try every possible path from any top-row starting cell through the grid — exponential, now with 3 choices per step instead of 2.

Why We Optimize NEW

Same grid-DP recurrence pattern as Minimum Path Sum (362), just with 3 possible 'sources' per cell instead of 2: `minCost(r,c) = grid[r][c] + min(minCost(r-1,c-1), minCost(r-1,c), minCost(r-1,c+1))`, and since the path can *start* anywhere in the top row, the final answer is the minimum value across the *entire* bottom row after filling the table — same O(rows×cols) improvement, extended to a slightly wider set of allowed moves per step.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum sum path falling from top to bottom.

Visual Diagram

```
Matrix:
       [2,1,3]
       [6,5,4]
       [7,8,9]

       dp[0] = [2,1,3]
       dp[1][0] = 6 + min(2,1) = 7
       dp[1][1] = 5 + min(2,1,3) = 6
       dp[1][2] = 4 + min(1,3) = 5
       dp[2][0] = 7 + min(7,6) = 13
       dp[2][1] = 8 + min(7,6,5) = 13
       dp[2][2] = 9 + min(6,5) = 14
       Result: 13
```

```
dp[i][j] = matrix[i][j] + min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])
```

Solution Approach

Concept: DP with three directions.

Algorithm

```
Algorithm MIN_FALLING_PATH(matrix):
           m = matrix.length, n = matrix[0].length
           dp = new int[m][n]
           dp[0] = matrix[0]

           For i = 1 to m-1:
               For j = 0 to n-1:
                   minVal = dp[i-1][j]
                   If j > 0: minVal = min(minVal, dp[i-1][j-1])
                   If j < n-1: minVal = min(minVal, dp[i-1][j+1])
                   dp[i][j] = matrix[i][j] + minVal

           Return min(dp[m-1])
```

Java Implementation

```
class Solution {
           public int minFallingPathSum(int[][] matrix) {
               int m = matrix.length, n = matrix[0].length;
               int[][] dp = new int[m][n];
               dp[0] = matrix[0];

                 for (int i = 1; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         int minVal = dp[i-1][j];
                         if (j > 0) minVal = Math.min(minVal, dp[i-1][j-1]);
                         if (j < n-1) minVal = Math.min(minVal, dp[i-1][j+1]);
                         dp[i][j] = matrix[i][j] + minVal;
                     }
                 }

                 int result = Integer.MAX_VALUE;
                 for (int val : dp[m-1]) {

                  result = Math.min(result, val);
              }
              return result;
          }
      }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

365. 3D DP: Ninja and His Friends (DP-13)

Understanding the Problem NEW

Two 'ninjas' start at the top corners of a grid and both move downward (each choosing among 3 directions per step, like problem 364) simultaneously, collecting values from cells they visit (a cell visited by both counts only once); maximize the total collected.

Brute Force Approach NEW

Try every possible combination of both ninjas' paths simultaneously — exponential in the combined branching of both ninjas' choices.

Why We Optimize NEW

Extend the DP **state** to track *both* ninjas' positions at once: since both move down one row per step together, the DP state is `(row, ninja1_column, ninja2_column)`, and at each state, try all 3×3=9 combinations of both ninjas' possible moves, taking the best. This is the chapter's first genuinely **3D DP** — the key generalization is recognizing that when a problem involves multiple 'agents' moving through the same structure, the DP state needs to track *all* of their positions together (not each independently), since their choices interact (the shared-cell rule). Still solvable in polynomial time — O(rows × cols² × 9) — despite the seemingly higher-dimensional complexity, by memoizing/tabulating over the combined state space rather than exploring the exponential combination of independent path choices.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Two ninjas collect maximum cherries.

Visual Diagram

```
Grid 3x3:
       [1,2,3]
       [4,5,6]
       [7,8,9]

       Two ninjas start from (0,0) and (0,1)
```

```
       Move together to bottom

       dp[step][i1][i2] = max cherries
       Same cell counted once
```

Solution Approach

Concept: 3D DP with same row movement.

Algorithm

```
Algorithm MAX_CHERRIES(grid):
           m = grid.length, n = grid[0].length
           dp = new int[m][n][n]

           For i = 0 to m-1:
               For j1 = 0 to n-1:
                   For j2 = 0 to n-1:
                       If j1 == j2:
                           dp[i][j1][j2] = grid[i][j1]
                       Else:
                           dp[i][j1][j2] = grid[i][j1] + grid[i][j2]

           For i = m-2 down to 0:
               For j1 = 0 to n-1:
                   For j2 = 0 to n-1:
                       maxVal = 0
                       For d1 = -1 to 1:
                           For d2 = -1 to 1:
                                If j1+d1 >=0 && j1+d1<n && j2+d2>=0 && j2+d2<n:
                                    maxVal = max(maxVal, dp[i+1][j1+d1][j2+d2])
                       dp[i][j1][j2] += maxVal
           Return dp[0][0][n-1]
```

Java Implementation

```
class Solution {
           public int maxCherries(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               int[][][] dp = new int[m][n][n];

                 for (int j1 = 0; j1 < n; j1++) {
                     for (int j2 = 0; j2 < n; j2++) {
                         dp[m-1][j1][j2] = grid[m-1][j1] + (j1 == j2 ? 0 :
grid[m-1][j2]);
                     }
                 }

                 for (int i = m-2; i >= 0; i--) {
                     for (int j1 = 0; j1 < n; j1++) {
```

```
                        for (int j2 = 0; j2 < n; j2++) {
                            int maxVal = 0;
                            for (int d1 = -1; d1 <= 1; d1++) {
                                for (int d2 = -1; d2 <= 1; d2++) {
                                    int nj1 = j1 + d1;
                                    int nj2 = j2 + d2;
                                    if (nj1 >= 0 && nj1 < n && nj2 >= 0 && nj2 < n)
{
                                        maxVal = Math.max(maxVal, dp[i+1][nj1]
[nj2]);
                                    }
                                }
                            }
                            dp[i][j1][j2] = grid[i][j1] + (j1 == j2 ? 0 : grid[i]
[j2]) + maxVal;
                        }
                    }
                }
                return dp[0][0][n-1];
            }
        }
```

Time & Space Complexity

Time Complexity: O(m * n² * 9) | Space Complexity: O(m * n²)

366. Subset Sum Equal to Target (DP-14)

Understanding the Problem NEW

Given an array and a target sum, determine if *any* subset of the array sums to exactly that target.

Brute Force Approach NEW

Try every one of the 2^n possible subsets and check their sums — exponential, matching the include/exclude recursion tree from the Recursion chapter (problem 133), but now this same tree has massively overlapping subproblems worth caching.

Why We Optimize NEW

Define the DP state as `canMake(index, remainingTarget)` — can the elements from `index` onward form exactly `remainingTarget`? — with the same include/exclude recursive choice as the Recursion chapter's subset problems, but now **memoized** on the (index, remainingTarget) pair, since many different subset choices lead to the exact same (index, remainingTarget) state and would otherwise be recomputed repeatedly. This converts the exponential $O(2^n)$ recursion into O(n×target) — the first, and clearest, example in this chapter of a problem you already know how to solve recursively (from Chapter 6) becoming dramatically faster purely by adding memoization on top of the exact same recursive structure.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if any subset sums to target.

Visual Diagram

```
Array: [2, 3, 7, 8, 10], Target: 11

       dp[i][j] = possible with first i elements to get sum j
       dp:
             0 1 2 3 4 5 6 7 8 9 10 11
       0:   [T,F,F,F,F,F,F,F,F,F,F,F]
       1(2): [T,F,T,F,F,F,F,F,F,F,F,F]
```

```
       2(3): [T,F,T,T,F,T,F,F,F,F,F,F]
       ...
       Result: True

       Recurrence: dp[i][j] = dp[i-1][j] || dp[i-1][j-arr[i]]
```

Solution Approach

Concept: Include/exclude element.

Algorithm

```
Algorithm SUBSET_SUM(arr, target):
           n = arr.length
           dp = boolean[n+1][target+1]
           dp[0][0] = true

           For i = 1 to n:
               For j = 0 to target:
                   dp[i][j] = dp[i-1][j]
                   If j >= arr[i-1]:
                       dp[i][j] = dp[i][j] || dp[i-1][j-arr[i-1]]

           Return dp[n][target]
```

Java Implementation

```
class Solution {
           public boolean subsetSum(int[] arr, int target) {
               int n = arr.length;
               boolean[][] dp = new boolean[n+1][target+1];
               dp[0][0] = true;

                 for (int i = 1; i <= n; i++) {
                     for (int j = 0; j <= target; j++) {
                         dp[i][j] = dp[i-1][j];
                         if (j >= arr[i-1]) {
                             dp[i][j] = dp[i][j] || dp[i-1][j-arr[i-1]];
                         }
                     }
                 }
                 return dp[n][target];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n*target) | Space Complexity: O(n*target)

367. Partition Equal Subset Sum (DP-15)

Understanding the Problem NEW

Given an array, determine if it can be partitioned into two subsets with *equal* sum.

Brute Force Approach NEW

Try every possible way to split the array into two groups and check if their sums match — exponential (2^n splits).

Why We Optimize NEW

This reduces directly to Subset Sum (366): the array can be split into two equal-sum halves if and only if **some subset sums to exactly half the total array sum** (if the total sum is odd, it's immediately impossible, since two equal integer halves couldn't sum to an odd total). Reusing the exact Subset Sum DP with target = totalSum/2 answers this directly, achieving O(n×totalSum) — another case of recognizing a new-sounding problem as a direct instance of one already solved.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Partition array into two subsets with equal sum.

Visual Diagram

```
Array: [1, 5, 11, 5]
       Sum = 22, target = 11
       Subset sum 11 exists: [1,5,5] = 11
       Result: True

       Array: [1, 2, 3, 5]
       Sum = 11, target = 5.5 (not integer)
       Result: False

       If total sum odd: return false
       Check subset sum = total/2
```

Solution Approach

Concept: Check subset sum = total/2.

Algorithm

```
Algorithm EQUAL_SUBSET(nums):
           total = sum(nums)
           If total % 2 != 0: Return false
           target = total / 2
           Return SUBSET_SUM(nums, target)
```

Java Implementation

```
class Solution {
           public boolean canPartition(int[] nums) {
               int total = 0;
               for (int num : nums) total += num;
               if (total % 2 != 0) return false;

                 int target = total / 2;
                 int n = nums.length;
                 boolean[] dp = new boolean[target + 1];
                 dp[0] = true;

                 for (int num : nums) {
                     for (int j = target; j >= num; j--) {
                         dp[j] = dp[j] || dp[j - num];
                     }
                 }
                 return dp[target];
           }
      }
```

Time & Space Complexity

Time Complexity: O(n*target) | Space Complexity: O(target)

368. Partition Set Into 2 Subsets With Minimum Difference (DP-16)

Understanding the Problem NEW

Given an array, partition it into two subsets such that the *difference* between their sums is as small as possible (not necessarily zero, unlike problem 367).

Brute Force Approach NEW

Try every possible split and compute the resulting difference, tracking the minimum — exponential.

Why We Optimize NEW

Extend Subset Sum (366) further: compute *which totals are achievable* by any subset (not just check one specific target, but find **every** achievable subset-sum up to the total array sum, using the same DP table but reading off every reachable value instead of just one). For every achievable subset-sum `s`, the resulting difference is `|totalSum - 2s|` (one subset sums to s, the other to totalSum-s); take the minimum of this over every achievable s. This reuses the exact same Subset Sum DP table, just extracting more information from it (every reachable sum, not just a yes/no for one target) — the same O(n×totalSum) table serves multiple related questions.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Partition array to minimize difference.

Visual Diagram

```
Array: [1, 2, 3, 4]
       Possible partitions:
       [1,4] and [2,3] → diff=0
       Result: 0

       Find sum closest to total/2.
```

Solution Approach

Concept: Subset sum closest to total/2.

Algorithm

```
Algorithm MIN_PARTITION_DIFF(arr):
           total = sum(arr)
           target = total / 2
           Find subset sum ≤ target closest to target
           diff = total - 2 * subsetSum
           Return diff
```

Java Implementation

```
class Solution {
           public int minPartitionDiff(int[] nums) {
               int total = 0;
               for (int num : nums) total += num;
               int target = total / 2;

                 boolean[] dp = new boolean[target + 1];
                 dp[0] = true;

                 for (int num : nums) {
                     for (int j = target; j >= num; j--) {
                         dp[j] = dp[j] || dp[j - num];
                     }
                 }

                 int closest = 0;
                 for (int i = target; i >= 0; i--) {
                     if (dp[i]) {
                         closest = i;
                         break;
                     }
                 }
                 return total - 2 * closest;
            }
      }
```

Time & Space Complexity

Time Complexity: O(n*target) | Space Complexity: O(target)

369. Count Subsets with Sum K (DP-17)

Understanding the Problem NEW

Given an array and a target, count *how many* subsets sum to exactly that target (not just whether one exists, unlike problem 366).

Brute Force Approach NEW

Enumerate every subset, check its sum, count matches — exponential.

Why We Optimize NEW

Same include/exclude DP state as Subset Sum (366), but instead of storing a true/false 'can this be achieved,' store a **count** — the number of ways to reach `remainingTarget` from `index` onward is the sum of the ways achieved by including vs. excluding the current element. Memoized/tabulated exactly like 366, just changing the combination step from OR (either way works) to addition (count both ways) — a small but important shift from 'existence' DP to 'counting' DP that reappears throughout this chapter (much like the Recursion chapter's print/count/check variations on the same tree, problem 134).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count subsets with sum equal to target.

Visual Diagram

```
Array: [2, 3, 5, 6], Target: 8
       Subsets: [2,6], [3,5]
       Count: 2

       dp[i][j] = dp[i-1][j] + dp[i-1][j-arr[i]]
```

Solution Approach

Concept: Count ways with include/exclude.

Algorithm

```
Algorithm COUNT_SUBSETS(arr, target):
           n = arr.length
           dp = new int[n+1][target+1]
           dp[0][0] = 1

           For i = 1 to n:
               For j = 0 to target:
                   dp[i][j] = dp[i-1][j]
                   If j >= arr[i-1]:
                       dp[i][j] += dp[i-1][j-arr[i-1]]

           Return dp[n][target]
```

Java Implementation

```
class Solution {
           public int countSubsets(int[] nums, int target) {
               int n = nums.length;
               int[] dp = new int[target + 1];
               dp[0] = 1;

                 for (int num : nums) {
                     for (int j = target; j >= num; j--) {
                         dp[j] += dp[j - num];
                     }
                 }
                 return dp[target];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n*target) | Space Complexity: O(target)

370. Count Partitions with Given Difference (DP-18)

Understanding the Problem NEW

Count the number of ways to partition the array into two subsets whose sums differ by exactly a given value.

Brute Force Approach NEW

Enumerate every split and check the resulting difference — exponential.

Why We Optimize NEW

Same algebraic reduction as problem 368, combined with the counting version from 369: if the two subsets differ by `diff` and sum to `totalSum` together, one subset's sum must be exactly `(totalSum + diff) / 2` (solving the two simultaneous equations) — so the answer is simply **Count Subsets with Sum K** (369) applied with this specific target. Another instance of reducing a new-sounding problem to an already-solved one via a bit of algebra, rather than designing a new DP from scratch.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count partitions with difference = diff.

Visual Diagram

```
Array: [1, 1, 2, 3], diff=1
       Partitions with diff=1:
       {1,3} and {1,2} → |4-3|=1
       {1,1,2} and {3} → |4-3|=1
       {1,1,3} and {2} → |5-2|=3
       Count: 2

       Formula: s1 - s2 = diff, s1 + s2 = total
       s1 = (total + diff) / 2
       Count subsets with sum = s1
```

Solution Approach

Concept: Reduce to subset sum count.

Algorithm

```
Algorithm COUNT_PARTITIONS(arr, diff):
           total = sum(arr)
           If (total + diff) % 2 != 0: Return 0
           target = (total + diff) / 2
           Return COUNT_SUBSETS(arr, target)
```

Java Implementation

```
class Solution {
           public int countPartitions(int[] nums, int diff) {
               int total = 0;
               for (int num : nums) total += num;

                 if ((total + diff) % 2 != 0) return 0;
                 int target = (total + diff) / 2;

                 int[] dp = new int[target + 1];
                 dp[0] = 1;

                 for (int num : nums) {
                     for (int j = target; j >= num; j--) {
                         dp[j] += dp[j - num];
                     }
                 }
                 return dp[target];
            }
       }
```

Time & Space Complexity

Time Complexity: O(n*target) | Space Complexity: O(target)

371. 0/1 Knapsack (DP-19)

Understanding the Problem NEW

The classic 0/1 Knapsack: given items with weights and values, and a knapsack with a maximum weight capacity, maximize total value, where **each item can be used at most once** (unlike Fractional Knapsack from the Greedy chapter, no partial items are allowed here, which is exactly why greedy doesn't work and DP is needed).

Brute Force Approach NEW

Try every one of the 2^n possible subsets of items and check which fit within the weight capacity while maximizing value — exponential, mirroring the same include/exclude recursion tree as Combination Sum (Recursion chapter, problem 137), but without the 'unlimited reuse' twist.

Why We Optimize NEW

DP state `bestValue(index, remainingCapacity)` : at each item, either **exclude** it (keep `bestValue(index+1, remainingCapacity)`) or, if it fits, **include** it (`item.value + bestValue(index+1, remainingCapacity - item.weight)`) — take the better option, memoized/tabulated over the (index, remainingCapacity) state space. This is the specific reason 0/1 Knapsack needs DP rather than the greedy 'sort by value/weight ratio' approach that worked for the Fractional version (Greedy chapter, problem 229) — since items can't be split, the greedy choice can be provably wrong here, and only exploring (and caching) the include/exclude decision space correctly finds the optimum, in O(n×capacity) instead of O(2^n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Max value with weight capacity.

Visual Diagram

```
Items: weight=[1,3,4,5], value=[1,4,5,7], W=7
       dp[W] = max value
       dp[7] = max(dp[6] + 1, dp[4] + 4, dp[3] + 5, dp[2] + 7)
```

```
       Result: 11 (items 3+4: weight=4+3? Actually 5+4=9 >7)
       Correct: items 2+3: weight=3+4=7, value=4+5=9
```

Solution Approach

Concept: DP on capacity.

Algorithm

```
Algorithm KNAPSACK(weights, values, W):
           n = weights.length
           dp = new int[W+1]

           For i = 0 to n-1:
               For j = W down to weights[i]:
                   dp[j] = max(dp[j], dp[j-weights[i]] + values[i])

           Return dp[W]
```

Java Implementation

```
class Solution {
           public int knapsack(int[] weights, int[] values, int W) {
               int n = weights.length;
               int[] dp = new int[W + 1];

                 for (int i = 0; i < n; i++) {
                     for (int j = W; j >= weights[i]; j--) {
                         dp[j] = Math.max(dp[j], dp[j - weights[i]] + values[i]);
                     }
                 }
                 return dp[W];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n*W) | Space Complexity: O(W)

372. Minimum Coins (DP-20)

Understanding the Problem NEW

Given coin denominations (unlimited supply of each) and a target amount, find the minimum number of coins needed to make that amount exactly — the same real-world problem as the Greedy chapter's Minimum Number of Coins (230), but now solved with DP so it's **correct for any denomination system**, not just 'well-behaved' ones.

Brute Force Approach NEW

Try every possible combination of coin counts summing to the target — exponential without structure, or the greedy approach from 230, which (as flagged there) can give a wrong answer for arbitrary denominations.

Why We Optimize NEW

DP state `minCoins(remainingAmount)`: try using each available denomination once, recursively solving for `remainingAmount - coin`, taking the best (fewest coins) option across all denominations tried — memoized/tabulated per remaining amount. Unlike the greedy approach, this DP is **always correct** regardless of the denomination system, since it genuinely explores every relevant combination (via caching, not blind greed) rather than assuming 'always take the largest coin that fits' is safe — directly resolving the caveat flagged back in the Greedy chapter's coin problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum coins to make amount.

Visual Diagram

```
Coins: [1,2,5], Amount: 11
       Min coins: 3 (5+5+1)

       dp[amount] = min coins
       dp[0] = 0
       dp[1] = 1
       dp[2] = 1
```

```
dp[3] = min(dp[2]+1, dp[1]+1) = 2
dp[4] = min(dp[2]+1, dp[1]+1) = 2
dp[5] = 1
dp[6] = min(dp[5]+1, dp[4]+1) = 2
dp[7] = min(dp[5]+1, dp[2]+1) = 2
dp[8] = min(dp[5]+1, dp[3]+1) = 3
dp[9] = min(dp[5]+1, dp[4]+1) = 3? Actually 5+2+2 = 3
dp[10] = 2
dp[11] = min(dp[10]+1, dp[9]+1, dp[6]+1) = 3
Result: 3
```

Solution Approach

Concept: DP with coin choices.

Algorithm

```
Algorithm MIN_COINS(coins, amount):
           dp = new int[amount+1]
           Arrays.fill(dp, INF)
           dp[0] = 0

           For i = 1 to amount:
               For coin in coins:
                   If coin <= i:
                       dp[i] = min(dp[i], dp[i-coin] + 1)

           Return dp[amount] == INF ? -1 : dp[amount]
```

Java Implementation

```
class Solution {
           public int minCoins(int[] coins, int amount) {
               int[] dp = new int[amount + 1];
               Arrays.fill(dp, Integer.MAX_VALUE);
               dp[0] = 0;

                 for (int i = 1; i <= amount; i++) {
                     for (int coin : coins) {
                         if (coin <= i && dp[i - coin] != Integer.MAX_VALUE) {
                             dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                         }
                     }
                 }
                 return dp[amount] == Integer.MAX_VALUE ? -1 : dp[amount];
           }
      }
```

Time & Space Complexity

Time Complexity: O(amount * coins) | Space Complexity: O(amount)

373. Target Sum (DP-21)

Understanding the Problem NEW

Given an array of numbers, assign either a '+' or '-' sign to each one so that the resulting sum equals a given target; count how many such sign-assignments achieve the target.

Brute Force Approach NEW

Try every one of the 2^n possible +/- sign combinations and check which sum to the target — exponential.

Why We Optimize NEW

A clever algebraic reduction turns this into Subset Sum-style counting: split the numbers into a 'positive group' P and 'negative group' N — then `sum(P) - sum(N) = target` and `sum(P) + sum(N) = totalSum`, which together mean `sum(P) = (totalSum + target) / 2`. So the answer is exactly **Count Subsets with Sum K** (369), with K set to this computed value — yet another instance of algebraically transforming a new-looking problem into the same Subset-Sum-counting DP already built, rather than inventing a new algorithm.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Assign +/- to get target sum.

Visual Diagram

```
Array: [1,1,1,1,1], Target: 3
       Ways: 5

       Formula: s1 - s2 = target, s1 + s2 = total
       s1 = (total + target) / 2
       Count subsets with sum = s1

       total = 5, target = 3
       s1 = (5+3)/2 = 4
       Count subsets sum 4 = 5
```

Solution Approach

Concept: Reduce to subset sum count.

Algorithm

```
Algorithm TARGET_SUM(nums, target):
           total = sum(nums)
           If (total + target) % 2 != 0: Return 0
           target = (total + target) / 2
           Return COUNT_SUBSETS(nums, target)
```

Java Implementation

```
class Solution {
           public int findTargetSumWays(int[] nums, int target) {
               int total = 0;
               for (int num : nums) total += num;

                 if ((total + target) % 2 != 0 || Math.abs(target) > total) return
0;
                 int sum = (total + target) / 2;

                 int[] dp = new int[sum + 1];
                 dp[0] = 1;

                 for (int num : nums) {
                     for (int j = sum; j >= num; j--) {
                         dp[j] += dp[j - num];
                     }
                 }
                 return dp[sum];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n*sum) | Space Complexity: O(sum)

374. Coin Change 2 (DP-22)

Understanding the Problem NEW

Given coin denominations (unlimited supply) and a target amount, count the number of *distinct combinations* of coins that sum to the target (order doesn't matter — using one 5 then one 10 is the same combination as one 10 then one 5).

Brute Force Approach NEW

Try every possible combination of coin counts — exponential without structure, and naively trying every *ordering* of coin choices would overcount combinations that use the same coins in a different order.

Why We Optimize NEW

DP state `ways(coinIndex, remainingAmount)`: to avoid counting the same combination multiple times in different orders, process coin denominations **one at a time, in a fixed order**, deciding at each denomination 'how many of this coin to use' before moving to the next denomination — rather than trying combinations in every possible order. This 'process denominations in a fixed sequence, using each 0 or more times before moving on' structure is what correctly counts combinations instead of ordered sequences, achieving O(denominations × amount) via memoization/tabulation over the (coinIndex, remainingAmount) state.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Number of ways to make amount with coins (unlimited).

Visual Diagram

```
Coins: [1,2,5], Amount: 5
       Ways: 4
       5
       2+2+1
       2+1+1+1
       1+1+1+1+1
```

```
        dp[amount] = ways
        dp[0] = 1
        For coin in coins:
            For j = coin to amount:
                dp[j] += dp[j-coin]
```

Solution Approach

Concept: Unbounded knapsack count.

Algorithm

```
Algorithm COIN_CHANGE_2(coins, amount):
            dp = new int[amount+1]
            dp[0] = 1

            For coin in coins:
                For j = coin to amount:
                    dp[j] += dp[j-coin]

            Return dp[amount]
```

Java Implementation

```
class Solution {
            public int change(int amount, int[] coins) {
                int[] dp = new int[amount + 1];
                dp[0] = 1;

                  for (int coin : coins) {
                      for (int j = coin; j <= amount; j++) {
                          dp[j] += dp[j - coin];
                      }
                  }
                  return dp[amount];
            }
        }
```

Time & Space Complexity

Time Complexity: O(coins*amount) | Space Complexity: O(amount)

375. Unbounded Knapsack (DP-23)

Understanding the Problem NEW

Unbounded Knapsack: same as 0/1 Knapsack (371), but each item can now be used **unlimited** times instead of just once.

Brute Force Approach NEW

Same exponential exploration as 0/1 Knapsack without caching, now with an unbounded number of possible item-repetition combinations.

Why We Optimize NEW

Nearly identical DP recurrence to 0/1 Knapsack, with one crucial change: when you 'include' an item, you recurse back to the **same item index** (since it can be reused) rather than advancing to the next item — mirroring the exact same 'stay at the same candidate when reusing is allowed' pattern from Combination Sum in the Recursion chapter (problem 137). This single change to the recursive transition (stay vs. advance) is the entire difference between the bounded (0/1) and unbounded knapsack variants, both solved with the same O(n×capacity) memoized/tabulated DP shape.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Max value with unlimited item usage.

Visual Diagram

```
Items: weight=[1,2,3], value=[10,15,40], W=6
       Max value: 80 (take 40 twice? weight 3*2=6, value 40*2=80)

       dp[W] = max value
       For each weight w:
           For j = w to W:
               dp[j] = max(dp[j], dp[j-w] + value)
```

Solution Approach

Concept: Unbounded knapsack.

Algorithm

```
Algorithm UNBOUNDED_KNAPSACK(weights, values, W):
           n = weights.length
           dp = new int[W+1]

           For i = 0 to n-1:
               For j = weights[i] to W:
                   dp[j] = max(dp[j], dp[j-weights[i]] + values[i])

           Return dp[W]
```

Java Implementation

```
class Solution {
           public int unboundedKnapsack(int[] weights, int[] values, int W) {
               int n = weights.length;
               int[] dp = new int[W + 1];

                 for (int i = 0; i < n; i++) {
                     for (int j = weights[i]; j <= W; j++) {
                         dp[j] = Math.max(dp[j], dp[j - weights[i]] + values[i]);
                     }
                 }
                 return dp[W];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n*W) | Space Complexity: O(W)

376. Rod Cutting Problem (DP-24)

Understanding the Problem NEW

Given a rod of length n and prices for each possible cut length, find the maximum total value obtainable by cutting the rod into pieces (each piece's value determined by its length) and selling them.

Brute Force Approach NEW

Try every possible way to partition the rod's length into pieces — exponential, since the number of ways to partition an integer grows very quickly.

Why We Optimize NEW

This is Unbounded Knapsack (375) in disguise: each possible cut-length is like an 'item' with a weight (the length) and a value (its price), and you're trying to fill a knapsack of capacity n using unlimited repetitions of any length — directly reusing the same unbounded-knapsack DP recurrence and complexity, O(n×n) here since capacity and number of 'items' (possible lengths) are both n. Recognizing that 'cut a rod for max value' and 'fill a knapsack unlimited times' are structurally the same problem is the entire insight, rather than deriving a new DP from scratch.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Max value by cutting rod.

Visual Diagram

```
Length: 1,2,3,4
       Price: [2,5,7,8]
       Max value for length 4: 10 (2+2+2? Actually 4)
       Options:
       1+1+1+1 = 8
       2+2 = 10
       3+1 = 9
       4 = 8
       Result: 10
```

```
        dp[n] = max(dp[n], price[i] + dp[n-i-1])
```

Solution Approach

Concept: Unbounded knapsack with lengths.

Algorithm

```
Algorithm ROD_CUTTING(price, n):
            dp = new int[n+1]
            dp[0] = 0

            For i = 1 to n:
                For j = 1 to i:
                    dp[i] = max(dp[i], price[j-1] + dp[i-j])

            Return dp[n]
```

Java Implementation

```
class Solution {
            public int rodCutting(int[] price, int n) {
                int[] dp = new int[n + 1];
                dp[0] = 0;

                  for (int i = 1; i <= n; i++) {
                      for (int j = 1; j <= i; j++) {
                          dp[i] = Math.max(dp[i], price[j-1] + dp[i-j]);
                      }
                  }
                  return dp[n];
            }
        }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

377. Longest Common Subsequence (DP-25)

Understanding the Problem NEW

Find the length of the Longest Common Subsequence (LCS) between two strings — the longest sequence of characters that appears in both strings in the same relative order (not necessarily contiguous). E.g. LCS of 'abcde' and 'ace' is 'ace,' length 3.

Brute Force Approach NEW

Try every one of the 2^n subsequences of the first string and check if each is also a subsequence of the second — exponential, and extremely wasteful given the massive overlap between different subsequence attempts.

Why We Optimize NEW

DP state `lcs(i, j)` — the LCS length considering only the first i characters of string 1 and first j characters of string 2: if the characters at positions i and j match, `lcs(i,j) = 1 + lcs(i-1,j-1)` (extend the match); otherwise, `lcs(i,j) = max(lcs(i-1,j), lcs(i,j-1))` (try dropping a character from either string). This **2D DP over two string indices** — a new kind of DP state shape compared to the single-index problems earlier in this chapter — is memoized/tabulated to give O(n×m) instead of exponential subsequence enumeration, and this exact 2D-index DP shape becomes the backbone for nearly every remaining string-comparison problem in this chapter (LCS variants, Edit Distance, Wildcard Matching).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Length of longest common subsequence.

Visual Diagram

```
s1 = "abcde", s2 = "ace"
       LCS = "ace" length=3

       dp[i][j] = LCS length for first i of s1, first j of s2
       If s1[i-1] == s2[j-1]: dp[i][j] = 1 + dp[i-1][j-1]
       Else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

```
        "" a c e
    ""  0 0 0 0
    a   0 1 1 1
    b   0 1 1 1
    c   0 1 2 2
    d   0 1 2 2
    e   0 1 2 3
```

Solution Approach

Concept: DP with matching and skipping.

Algorithm

```
Algorithm LCS(s1, s2):
          m = s1.length, n = s2.length
          dp = int[m+1][n+1]

           For i = 1 to m:
               For j = 1 to n:
                   If s1[i-1] == s2[j-1]:
                       dp[i][j] = 1 + dp[i-1][j-1]
                   Else:
                       dp[i][j] = max(dp[i-1][j], dp[i][j-1])

           Return dp[m][n]
```

Java Implementation

```
class Solution {
          public int lcs(String s1, String s2) {
              int m = s1.length(), n = s2.length();
              int[][] dp = new int[m+1][n+1];

                for (int i = 1; i <= m; i++) {
                    for (int j = 1; j <= n; j++) {
                        if (s1.charAt(i-1) == s2.charAt(j-1)) {
                            dp[i][j] = 1 + dp[i-1][j-1];
                        } else {
                            dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                        }
                    }
                }
                return dp[m][n];
           }

     }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

378. Print Longest Common Subsequence (DP-26)

Understanding the Problem NEW

Same as LCS, but now print the actual longest common subsequence string, not just its length.

Brute Force Approach NEW

You could try every subsequence and check for the longest match directly — exponential, same as plain LCS without a plan for reconstruction.

Why We Optimize NEW

First compute the full LCS DP table as in problem 377, then **walk backward through the table** from `lcs(n,m)`: whenever the characters at the current positions match, that character is part of the LCS (add it to the front of the result and move diagonally back); otherwise, move in whichever direction (i-1 or j-1) the table shows the value came from. This backward-reconstruction-from-a-filled-DP-table technique — compute the table forward, then trace an optimal path backward through it to recover the actual solution, not just its value — is a general technique that applies to many DP problems where you need the actual answer, not just its length/value/count.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print the LCS string.

Visual Diagram

```
s1 = "abcde", s2 = "ace"
       LCS = "ace"

       Backtrack:
       dp[5][3] = 3 → s1[4]=e, s2[2]=e match → include 'e'
       dp[4][2] = 2 → s1[3]=d, s2[1]=c no match → go left (dp[4][1]=2)
       dp[4][1] = 2 → s1[3]=d, s2[0]=a no match → go left (dp[3][1]=1)
       dp[3][1] = 1 → s1[2]=c, s2[0]=a no match → go up (dp[2][1]=1)
       dp[2][1] = 1 → s1[1]=b, s2[0]=a no match → go up (dp[1][1]=1)
```

```
       dp[1][1] = 1 → s1[0]=a, s2[0]=a match → include 'a'
       Result: "ace"
```

Solution Approach

Concept: Backtrack from DP table.

Algorithm

```
Algorithm PRINT_LCS(s1, s2, dp):
           i = m, j = n
           lcs = []

            While i > 0 && j > 0:
                If s1[i-1] == s2[j-1]:
                    lcs.add(s1[i-1])
                    i--; j--
                Else If dp[i-1][j] > dp[i][j-1]:
                    i--
                Else:
                    j--

            Return reverse(lcs)
```

Java Implementation

```
class Solution {
           public String printLCS(String s1, String s2) {
               int m = s1.length(), n = s2.length();
               int[][] dp = new int[m+1][n+1];

                 // Fill DP table
                 for (int i = 1; i <= m; i++) {
                     for (int j = 1; j <= n; j++) {
                         if (s1.charAt(i-1) == s2.charAt(j-1)) {
                             dp[i][j] = 1 + dp[i-1][j-1];
                         } else {
                             dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                         }
                     }
                 }

                 // Backtrack
                 StringBuilder result = new StringBuilder();
                 int i = m, j = n;
                 while (i > 0 && j > 0) {

                  if (s1.charAt(i-1) == s2.charAt(j-1)) {
                      result.append(s1.charAt(i-1));
                      i--; j--;
```

```
                } else if (dp[i-1][j] > dp[i][j-1]) {
                    i--;
                } else {
                    j--;
                }
            }
            return result.reverse().toString();
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

379. Longest Common Substring (DP-27)

Understanding the Problem NEW

Find the length of the Longest Common **Substring** (not subsequence) — a contiguous run of characters common to both strings, unlike LCS which allows gaps.

Brute Force Approach NEW

Check every possible substring of string 1 against every possible substring of string 2 for a match — O(n²×m²) or worse.

Why We Optimize NEW

Nearly the same 2D DP table shape as LCS (377), but with one crucial change: since a substring must be *contiguous*, a mismatch can't just 'skip' a character the way LCS does — instead, `dp(i,j) = 1 + dp(i-1,j-1)` on a match, but `dp(i,j) = 0` entirely on a mismatch (breaking the current run rather than trying to continue it via skips), and the final answer is the **maximum value found anywhere in the table** (not just the bottom-right corner, since the longest common substring could end at any position). This small but important change — reset to 0 on mismatch instead of taking a max of skip-options — is what converts the LCS recurrence into the Longest Common Substring recurrence.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Length of longest common substring.

Visual Diagram

```
s1 = "abcd", s2 = "bcde"
       LCS substring = "bcd" length=3

       If match: dp[i][j] = 1 + dp[i-1][j-1]
       Else: dp[i][j] = 0
       Track max

            "" b c d e
       ""   0 0 0 0 0
       a    0 0 0 0 0
```

```
       b    0 1 0 0 0
       c    0 0 2 0 0
       d    0 0 0 3 0
```

Solution Approach

Concept: DP with match only.

Algorithm

```
Algorithm LCS_SUBSTRING(s1, s2):
           m = s1.length, n = s2.length
           dp = int[m+1][n+1]
           maxLen = 0

            For i = 1 to m:
                For j = 1 to n:
                    If s1[i-1] == s2[j-1]:
                        dp[i][j] = 1 + dp[i-1][j-1]
                        maxLen = max(maxLen, dp[i][j])

            Return maxLen
```

Java Implementation

```
class Solution {
           public int longestCommonSubstring(String s1, String s2) {
               int m = s1.length(), n = s2.length();
               int[][] dp = new int[m+1][n+1];
               int maxLen = 0;

                 for (int i = 1; i <= m; i++) {
                     for (int j = 1; j <= n; j++) {
                         if (s1.charAt(i-1) == s2.charAt(j-1)) {
                             dp[i][j] = 1 + dp[i-1][j-1];
                             maxLen = Math.max(maxLen, dp[i][j]);
                         }
                     }
                 }
                 return maxLen;
            }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

380. Longest Palindromic Subsequence (DP-28)

Understanding the Problem NEW

Find the length of the Longest Palindromic Subsequence within a single string — the longest subsequence that reads the same forwards and backwards.

Brute Force Approach NEW

Try every one of the 2^n subsequences and check which are palindromes, tracking the longest — exponential.

Why We Optimize NEW

A clever reduction: the Longest Palindromic Subsequence of a string is exactly the **Longest Common Subsequence (377) between the string and its own reverse** — since any subsequence common to both the string and its reverse must, by construction, read the same both ways. This directly reuses the LCS DP with no modification needed beyond feeding it the string and its reverse as the two inputs — another strong example of reducing a new problem to one already solved, this time via a clever observation rather than a direct structural similarity.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Length of longest palindromic subsequence.

Visual Diagram

```
s = "bbbab"
       LPS = "bbbb" length=4

       dp[i][j] = LPS length for s[i..j]
       If i == j: dp[i][j] = 1
       If s[i] == s[j]: dp[i][j] = 2 + dp[i+1][j-1]
       Else: dp[i][j] = max(dp[i+1][j], dp[i][j-1])

       For "bbbab":
       dp[0][4] = 4 (bbbb)
```

Solution Approach

Concept: DP on intervals.

Algorithm

```
Algorithm LPS(s):
           n = s.length()
           dp = int[n][n]

           For i = 0 to n-1: dp[i][i] = 1

           For len = 2 to n:
               For i = 0 to n-len:
                   j = i + len - 1
                   If s[i] == s[j]:
                       dp[i][j] = 2 + (len == 2 ? 0 : dp[i+1][j-1])
                   Else:
                       dp[i][j] = max(dp[i+1][j], dp[i][j-1])

           Return dp[0][n-1]
```

Java Implementation

```
class Solution {
           public int longestPalindromeSubseq(String s) {
               int n = s.length();
               int[][] dp = new int[n][n];

                 for (int i = 0; i < n; i++) dp[i][i] = 1;

                 for (int len = 2; len <= n; len++) {
                     for (int i = 0; i <= n - len; i++) {
                         int j = i + len - 1;
                         if (s.charAt(i) == s.charAt(j)) {
                             dp[i][j] = 2 + (len == 2 ? 0 : dp[i+1][j-1]);
                         } else {
                             dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
                         }
                     }
                 }
                 return dp[0][n-1];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n²)

381. Minimum Insertions to Make Palindrome (DP-29)

Understanding the Problem NEW

Find the minimum number of character insertions needed to turn a given string into a palindrome.

Brute Force Approach NEW

Try every possible way of inserting characters and check which produces a palindrome with the fewest insertions — exponential without structure.

Why We Optimize NEW

Another elegant reduction: the minimum insertions needed equals `string length - Longest Palindromic Subsequence length` (380) — because the LPS represents the 'already palindromic core' you can keep as-is, and every character *not* part of that core needs a matching insertion elsewhere to become symmetric. Directly reusing the LPS DP (which itself reuses LCS) to answer this, rather than solving insertion-counting from scratch, is the whole insight — reinforcing how a handful of core DP recurrences (LCS being the most reused one) power a surprising number of seemingly distinct string problems.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum insertions to make string palindrome.

Visual Diagram

```
s = "mbadm"
       LPS = "madam" (or "mbdm" no, "mam" length 3)
       Minimum insertions = n - LPS = 5 - 3 = 2

       "mbadm" → "madam" insert 'a' at pos 1? "mbda"...
       Actually: "mbadm" → "mabdamb"? Too complex.

       Formula: minInsertions = n - LPS(s)
```

Solution Approach

Concept: LPS length and total length.

Algorithm

```
Algorithm MIN_INSERTIONS(s):
           n = s.length()
           lps = LPS(s)
           Return n - lps
```

Java Implementation

```
class Solution {
           public int minInsertions(String s) {
               int n = s.length();
               int lps = longestPalindromeSubseq(s);
               return n - lps;
           }

           private int longestPalindromeSubseq(String s) {
               int n = s.length();
               int[][] dp = new int[n][n];

                 for (int i = 0; i < n; i++) dp[i][i] = 1;

                 for (int len = 2; len <= n; len++) {
                     for (int i = 0; i <= n - len; i++) {
                         int j = i + len - 1;
                         if (s.charAt(i) == s.charAt(j)) {
                             dp[i][j] = 2 + (len == 2 ? 0 : dp[i+1][j-1]);
                         } else {
                             dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
                         }
                     }
                 }
                 return dp[0][n-1];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n²)

382. Minimum Insertions/Deletions to Convert (DP-30)

Understanding the Problem NEW

Find the minimum number of insertions and deletions needed to convert one string into another.

Brute Force Approach NEW

Try every possible sequence of insert/delete operations — exponential without structure.

Why We Optimize NEW

Reduces directly to LCS (377): whatever characters are part of the LCS between the two strings can stay untouched; every other character in string 1 must be **deleted** (`len(s1) - LCS` deletions), and every other character in string 2 must be **inserted** (`len(s2) - LCS` insertions). Computing LCS once and doing simple arithmetic on the lengths answers this directly — yet another problem that reduces cleanly to the same LCS computation, rather than needing a new DP recurrence.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum operations to convert s1 to s2.

Visual Diagram

```
s1 = "heap", s2 = "pea"
       Delete 'h','p' and insert 'p'? Actually:
       "heap" → "pea"
       Delete 'h', delete 'p', insert 'p'?
       Minimum: 3

       Formula:
       LCS = "ea" length=2
       Deletions = m - LCS = 4-2=2
       Insertions = n - LCS = 3-2=1
       Total = 3
```

Solution Approach

Concept: LCS approach.

Algorithm

```
Algorithm MIN_OPERATIONS(s1, s2):
           lcs = LCS(s1, s2)
           deletions = s1.length - lcs
           insertions = s2.length - lcs
           Return deletions + insertions
```

Java Implementation

```
class Solution {
           public int minOperations(String s1, String s2) {
               int m = s1.length(), n = s2.length();
               int lcs = longestCommonSubsequence(s1, s2);
               return (m - lcs) + (n - lcs);
           }

           private int longestCommonSubsequence(String s1, String s2) {
               int m = s1.length(), n = s2.length();
               int[][] dp = new int[m+1][n+1];

                 for (int i = 1; i <= m; i++) {
                     for (int j = 1; j <= n; j++) {
                         if (s1.charAt(i-1) == s2.charAt(j-1)) {
                             dp[i][j] = 1 + dp[i-1][j-1];
                         } else {
                             dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                         }
                     }
                 }
                 return dp[m][n];
           }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

383. Shortest Common Supersequence (DP-31)

Understanding the Problem NEW

Find the Shortest Common Supersequence (SCS) — the shortest string that contains both given strings as subsequences.

Brute Force Approach NEW

Try building candidate supersequences directly and check if both strings are subsequences of each — without structure, this is a huge, poorly-defined search space.

Why We Optimize NEW

The SCS is built by **merging the two strings along their LCS (377)**: characters that are part of the LCS only need to appear once in the result, while characters *not* part of the LCS from either string must all appear (since they need to be 'passed through' individually). Concretely, reconstruct using the same backward-walk technique as Print LCS (378), but this time, when characters don't match, include *both* mismatched characters in the result (rather than skipping) before continuing the backward walk. This is a direct extension of the LCS-reconstruction technique from problem 378, adapted to build a merged supersequence instead of just extracting the common part.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find shortest string containing both as subsequences.

Visual Diagram

```
s1 = "abac", s2 = "cab"
       SCS = "cabac" or "caba"?
       LCS = "ab" length=2
       SCS length = m + n - LCS = 4+3-2=5

       Build SCS:
       1. Take LCS "ab"
       2. Add remaining chars in order
       Result: "cabac" (c + ab + ac)
```

Solution Approach

Concept: LCS based construction.

Algorithm

```
Algorithm SHORTEST_COMMON_SUPERSEQUENCE(s1, s2):
           lcs = LCS(s1, s2)
           Build SCS by merging s1 and s2 with LCS
```

Java Implementation

```
class Solution {
           public String shortestCommonSupersequence(String s1, String s2) {
               int m = s1.length(), n = s2.length();
               int[][] dp = new int[m+1][n+1];

                 // Fill LCS DP
                 for (int i = 1; i <= m; i++) {
                     for (int j = 1; j <= n; j++) {
                         if (s1.charAt(i-1) == s2.charAt(j-1)) {
                             dp[i][j] = 1 + dp[i-1][j-1];
                         } else {
                             dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                         }
                     }
                 }

                 // Build SCS
                 StringBuilder result = new StringBuilder();
                 int i = m, j = n;
                 while (i > 0 && j > 0) {
                     if (s1.charAt(i-1) == s2.charAt(j-1)) {
                         result.append(s1.charAt(i-1));
                         i--; j--;
                     } else if (dp[i-1][j] > dp[i][j-1]) {
                         result.append(s1.charAt(i-1));
                         i--;
                     } else {
                         result.append(s2.charAt(j-1));
                         j--;
                     }
                 }

                   while (i > 0) result.append(s1.charAt(--i));

              while (j > 0) result.append(s2.charAt(--j));

              return result.reverse().toString();
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

384. Distinct Subsequences (DP-32)

Understanding the Problem NEW

Count the number of distinct ways string T appears as a subsequence within string S. E.g. S='rabbbit', T='rabbit' → 3 ways (choosing different 'b's).

Brute Force Approach NEW

Try every one of the 2^|S| subsequences of S and check how many equal T — exponential.

Why We Optimize NEW

DP state `ways(i, j)` — number of ways to form the first j characters of T using the first i characters of S: if `S[i] == T[j]`, you have **two choices** — use this S character to match T (`ways(i-1, j-1)`) or skip it and still try to match using earlier S characters (`ways(i-1, j)`) — sum both; if they don't match, you must skip the S character (`ways(i-1, j)` only). This 2D DP (same index-pair shape as LCS, but counting instead of taking max/length) achieves O(n×m) instead of exponential subsequence enumeration, another instance of the 'counting variant' of a familiar 2D string-DP recurrence.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count distinct subsequences of s matching t.

Visual Diagram

```
s = "rabbbit", t = "rabbit"
       Distinct subsequences: 3

       dp[i][j] = ways using first i of s to form first j of t
       If match: dp[i][j] = dp[i-1][j] + dp[i-1][j-1]
       Else: dp[i][j] = dp[i-1][j]

           "" r a b b i t
       "" 1 0 0 0 0 0 0
       r  1 1 0 0 0 0 0
       a  1 1 1 0 0 0 0
       b  1 1 1 1 0 0 0
```

```
       b   1 1 1 2 0 0 0
       i   1 1 1 2 2 0 0
       t   1 1 1 2 2 2 0?
       Result: 3
```

Solution Approach

Concept: DP with match inclusion.

Algorithm

```
Algorithm DISTINCT_SUBSEQUENCES(s, t):
           m = s.length(), n = t.length()
           dp = int[m+1][n+1]
           dp[i][0] = 1

           For i = 1 to m:
               For j = 1 to n:
                   dp[i][j] = dp[i-1][j]
                   If s[i-1] == t[j-1]:
                       dp[i][j] += dp[i-1][j-1]

           Return dp[m][n]
```

Java Implementation

```
class Solution {
           public int numDistinct(String s, String t) {
               int m = s.length(), n = t.length();
               int[] dp = new int[n + 1];
               dp[0] = 1;

                 for (int i = 1; i <= m; i++) {
                     for (int j = n; j >= 1; j--) {
                         if (s.charAt(i-1) == t.charAt(j-1)) {
                             dp[j] += dp[j-1];
                         }
                     }
                 }
                 return dp[n];
           }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(n)

385. Edit Distance (DP-33)

Understanding the Problem NEW

Find the minimum number of operations (insert, delete, or replace a character) needed to convert one string into another — a generalization of problem 382 that also allows character replacement, not just insert/delete.

Brute Force Approach NEW

Try every possible sequence of edit operations — exponential without structure.

Why We Optimize NEW

DP state `editDist(i, j)`: if the current characters match, no operation is needed here (`editDist(i-1, j-1)`); if they don't match, try all three operations and take the cheapest — **insert** (`1 + editDist(i, j-1)`), **delete** (`1 + editDist(i-1, j)`), or **replace** (`1 + editDist(i-1, j-1)`) — memoized/tabulated over the same 2D (i,j) index-pair shape used throughout this string-DP family. This is the most general string-transformation DP in the chapter (subsuming problem 382's insert/delete-only version as a special case where replacement is simply never the cheapest option), achieving O(n×m) instead of exponential operation-sequence exploration.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum operations to convert s1 to s2.

Visual Diagram

```
s1 = "horse", s2 = "ros"
       Operations: 3
       "horse" → "rorse" (replace h→r)
       "rorse" → "rose" (delete r)
       "rose" → "ros" (delete e)

       dp[i][j] = min operations
       If match: dp[i][j] = dp[i-1][j-1]
       Else: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
```

Solution Approach

Concept: DP with delete/insert/replace.

Algorithm

```
Algorithm EDIT_DISTANCE(s1, s2):
           m = s1.length, n = s2.length
           dp = int[m+1][n+1]

           For i = 0 to m: dp[i][0] = i
           For j = 0 to n: dp[0][j] = j

           For i = 1 to m:
               For j = 1 to n:
                   If s1[i-1] == s2[j-1]:
                       dp[i][j] = dp[i-1][j-1]
                   Else:
                       dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])

           Return dp[m][n]
```

Java Implementation

```
class Solution {
           public int minDistance(String s1, String s2) {
               int m = s1.length(), n = s2.length();
               int[][] dp = new int[m+1][n+1];

                 for (int i = 0; i <= m; i++) dp[i][0] = i;
                 for (int j = 0; j <= n; j++) dp[0][j] = j;

                 for (int i = 1; i <= m; i++) {
                     for (int j = 1; j <= n; j++) {
                         if (s1.charAt(i-1) == s2.charAt(j-1)) {
                             dp[i][j] = dp[i-1][j-1];
                         } else {
                             dp[i][j] = 1 + Math.min(dp[i-1][j],
                                           Math.min(dp[i][j-1], dp[i-1][j-1]));
                         }
                     }
                 }
                 return dp[m][n];
           }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

386. Wildcard Matching (DP-34)

Understanding the Problem NEW

Determine if a pattern (containing literal characters, '?' meaning any single character, and '*' meaning any sequence of characters, including empty) matches a given string entirely.

Brute Force Approach NEW

Try every possible way '*' characters could expand and check if the resulting expanded pattern matches the string exactly — exponential, since each '*' could expand to many different lengths.

Why We Optimize NEW

Same 2D DP index-pair shape as Edit Distance/LCS, but with match-checking logic tailored to wildcards: `match(i,j)` is true if the current pattern character is '?' or matches the string character exactly (then check `match(i-1,j-1)`); if it's '*', it can either match zero characters (check `match(i,j-1)`, moving past the '*' in the pattern) or match one more string character while staying on the same '*' (check `match(i-1,j)`) — take true if *either* option works. This reuses the same 2D-index DP structure as the rest of the string family, with the recurrence's match-logic adapted specifically to handle the two wildcard types, rather than exhaustively trying every possible '*' expansion length.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Wildcard matching with '*' and '?'.

Visual Diagram

```
s = "adceb", p = "*a*b"
       Result: true

       dp[i][j] = pattern matches s[0..i-1]
       If p[j-1] == '?': dp[i][j] = dp[i-1][j-1]
       If p[j-1] == '*': dp[i][j] = dp[i][j-1] || dp[i-1][j]
       If chars match: dp[i][j] = dp[i-1][j-1]
```

Solution Approach

Concept: DP with wildcard handling.

Algorithm

```
Algorithm WILDCARD_MATCH(s, p):
           m = s.length, n = p.length
           dp = boolean[m+1][n+1]
           dp[0][0] = true

            For j = 1 to n:
                If p[j-1] == '*': dp[0][j] = dp[0][j-1]

            For i = 1 to m:
                For j = 1 to n:
                    If p[j-1] == '?':
                        dp[i][j] = dp[i-1][j-1]
                    Else If p[j-1] == '*':
                        dp[i][j] = dp[i][j-1] || dp[i-1][j]
                    Else:
                        dp[i][j] = s[i-1] == p[j-1] && dp[i-1][j-1]

            Return dp[m][n]
```

Java Implementation

```
class Solution {
           public boolean isMatch(String s, String p) {
               int m = s.length(), n = p.length();
               boolean[][] dp = new boolean[m+1][n+1];
               dp[0][0] = true;

                 for (int j = 1; j <= n; j++) {
                     if (p.charAt(j-1) == '*') {
                         dp[0][j] = dp[0][j-1];
                     }
                 }

                 for (int i = 1; i <= m; i++) {
                     for (int j = 1; j <= n; j++) {
                         char sc = s.charAt(i-1);
                         char pc = p.charAt(j-1);

                         if (pc == '?') {
                             dp[i][j] = dp[i-1][j-1];
                         } else if (pc == '*') {

                          dp[i][j] = dp[i][j-1] || dp[i-1][j];
                      } else {
                          dp[i][j] = sc == pc && dp[i-1][j-1];
```

```
                    }
                }
            }
            return dp[m][n];
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

387. Best Time to Buy and Sell Stock I (DP-35)

Understanding the Problem NEW

Given daily stock prices, find the maximum profit from **one** buy and one sell transaction — the exact same problem as the Arrays chapter's Stock Buy and Sell (problem 27), revisited here to connect it to the DP family of stock problems that follow.

Brute Force Approach NEW

Check every pair of buy-day/sell-day combinations — $O(n^2)$, same as the naive approach flagged back in the Arrays chapter.

Why We Optimize NEW

As established back in problem 27: track a running minimum price seen so far, and at each day compute `currentPrice - minSoFar`, keeping the best — O(n) time, O(1) space, no DP table actually needed for this simplest single-transaction version. It's included here specifically as the 'base case' anchor for the more complex multi-transaction stock variants (388-392) that follow, each adding one more layer of complexity on top of this same core idea.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum profit with one transaction.

Visual Diagram

```
Prices: [7,1,5,3,6,4]
       Max profit: 5 (buy at 1, sell at 6)

       Track min price and max profit.
```

Solution Approach

Concept: Single pass tracking min.

Algorithm

```
Algorithm MAX_PROFIT(prices):
           minPrice = INF
           maxProfit = 0

           For price in prices:
               minPrice = min(minPrice, price)
               maxProfit = max(maxProfit, price - minPrice)

           Return maxProfit
```

Java Implementation

```
class Solution {
           public int maxProfit(int[] prices) {
               int minPrice = Integer.MAX_VALUE;
               int maxProfit = 0;

                 for (int price : prices) {
                     minPrice = Math.min(minPrice, price);
                     maxProfit = Math.max(maxProfit, price - minPrice);
                 }
                 return maxProfit;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

388. Buy and Sell Stock II (DP-36)

Understanding the Problem NEW

Same stock setup, but now **unlimited transactions** are allowed (buy and sell multiple times, though you must sell before buying again).

Brute Force Approach NEW

Try every possible combination of buy/sell day pairs — exponential without structure.

Why We Optimize NEW

With unlimited transactions and no cooldown/fee, the optimal strategy is actually simple and greedy: capture **every single upward price movement** (buy the day before any price increase, sell the day of the increase) — summing `max(0, price[i] - price[i-1])do genuinely need DP because their added restrictions (transaction limits, cooldown, fees) break this simple greedy strategy.`

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum profit with unlimited transactions.

Visual Diagram

```
Prices: [7,1,5,3,6,4]
       Profit: 7 (buy 1 sell 5 = 4, buy 3 sell 6 = 3)
       Total: 7

       Sum all positive differences:
       (5-1)+(6-3) = 4+3=7
```

Solution Approach

Concept: Sum all positive differences.

Algorithm

```
Algorithm MAX_PROFIT_II(prices):
           profit = 0
           For i = 1 to n-1:
               If prices[i] > prices[i-1]:
                   profit += prices[i] - prices[i-1]
           Return profit
```

Java Implementation

```
class Solution {
           public int maxProfit(int[] prices) {
               int profit = 0;
               for (int i = 1; i < prices.length; i++) {
                   if (prices[i] > prices[i-1]) {
                        profit += prices[i] - prices[i-1];
                   }
               }
               return profit;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

389. Buy and Sell Stocks III (DP-37)

Understanding the Problem NEW

Same stock setup, but now limited to **at most 2** transactions total.

Brute Force Approach NEW

Try every possible pair of non-overlapping buy-sell windows and compute their combined profit — $O(n^4)$ or worse if done naively by checking all combinations of two transaction windows.

Why We Optimize NEW

Now the transaction limit genuinely requires DP: track state `(day, transactionNumber, holding-or-not)` — for each day, and each of the first/second transaction 'slots,' track the best profit achievable whether you currently hold a stock or not, updating each day based on the previous day's values for buying, selling, or doing nothing. This is where the stock problems stop being solvable by simple greedy summation (as in the unlimited-transactions version, 388) and genuinely need a DP table tracking the extra 'which transaction number, holding or not' state — $O(n)$ time with a small constant state-space multiplier.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum profit with at most 2 transactions.

Visual Diagram

```
Prices: [3,3,5,0,0,3,1,4]
       Max profit: 6 (buy 3 sell 5 = 2, buy 0 sell 4 = 4)

       DP with 2 transactions.
```

Solution Approach

Concept: DP with transaction count.

Algorithm

```
Algorithm MAX_PROFIT_III(prices):
           n = prices.length
           profit = new int[3][n]

           For t = 1 to 2:
               maxDiff = -prices[0]
               For i = 1 to n-1:
                   profit[t][i] = Math.max(profit[t][i-1], prices[i] + maxDiff)
                   maxDiff = Math.max(maxDiff, profit[t-1][i-1] - prices[i])

           Return profit[2][n-1]
```

Java Implementation

```
class Solution {
           public int maxProfit(int[] prices) {
               int n = prices.length;
               if (n < 2) return 0;

                 int[][] dp = new int[3][n];

                 for (int t = 1; t <= 2; t++) {
                     int maxDiff = -prices[0];
```

```
                    for (int i = 1; i < n; i++) {
                        dp[t][i] = Math.max(dp[t][i-1], prices[i] + maxDiff);
                        maxDiff = Math.max(maxDiff, dp[t-1][i-1] - prices[i]);
                    }
                }
                return dp[2][n-1];
            }
        }
```

Time & Space Complexity

Time Complexity: O(2*n) | Space Complexity: O(3*n)

390. Buy and Sell Stock IV (DP-38)

Understanding the Problem NEW

Generalizes problem 389 to **at most K** transactions (K given, not fixed at 2).

Brute Force Approach NEW

Same exponential/combinatorial transaction-window exploration as 389, worse for larger K.

Why We Optimize NEW

Direct generalization of 389's DP state: `(day, transactionNumber from 1 to K, holding-or-not)`, with the exact same buy/sell/hold transition logic per day, just extended to K transaction slots instead of a hardcoded 2. Achieves O(n×K) — the same DP shape as 389, parameterized to any K rather than a fixed constant.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum profit with at most k transactions.

Visual Diagram

```
Prices: [3,2,6,5,0,3], k=2
       Max profit: 7 (buy 2 sell 6 = 4, buy 0 sell 3 = 3)

       Same as III but with k.
```

Solution Approach

Concept: DP with k transactions.

Algorithm

```
Algorithm MAX_PROFIT_IV(k, prices):
           n = prices.length
           If k >= n/2: return maxProfit_II(prices)

           dp = new int[k+1][n]
           For t = 1 to k:
               maxDiff = -prices[0]
               For i = 1 to n-1:
                   dp[t][i] = Math.max(dp[t][i-1], prices[i] + maxDiff)
                   maxDiff = Math.max(maxDiff, dp[t-1][i-1] - prices[i])

           Return dp[k][n-1]
```

Java Implementation

```
class Solution {
           public int maxProfit(int k, int[] prices) {
               int n = prices.length;
               if (n < 2) return 0;
               if (k >= n/2) return maxProfitUnlimited(prices);

                 int[][] dp = new int[k+1][n];
                 for (int t = 1; t <= k; t++) {
                     int maxDiff = -prices[0];
                     for (int i = 1; i < n; i++) {
                         dp[t][i] = Math.max(dp[t][i-1], prices[i] + maxDiff);
                         maxDiff = Math.max(maxDiff, dp[t-1][i-1] - prices[i]);
                     }
```

```
                }
                return dp[k][n-1];
            }

            private int maxProfitUnlimited(int[] prices) {
                int profit = 0;
                for (int i = 1; i < prices.length; i++) {
                    if (prices[i] > prices[i-1]) profit += prices[i] - prices[i-1];
                }
                return profit;
            }
        }
```

Time & Space Complexity

Time Complexity: O(k*n) | Space Complexity: O(k*n)

391. Buy and Sell Stocks With Cooldown (DP-39)

Understanding the Problem NEW

Unlimited transactions again, but now with a **cooldown**: after selling, you must wait one day before buying again.

Brute Force Approach NEW

Same exponential exploration without structure as any unconstrained-transaction-sequence problem.

Why We Optimize NEW

Track DP states for each day representing 'holding a stock,' 'not holding, and not in cooldown,' and 'not holding, just sold (in cooldown)' — the cooldown constraint means the 'can buy today' transition must specifically come from the 'not holding, cooldown already over' state, not directly from 'just sold.' This adds one extra state category compared to the simpler unlimited-transactions case, directly modeling the cooldown rule as an explicit state rather than trying to bolt it onto the greedy strategy from problem 388, which can't accommodate this extra day-of-restriction rule.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum profit with cooldown after selling.

Visual Diagram

```
Prices: [1,2,3,0,2]
       Max profit: 3 (buy 1 sell 3 = 2, cooldown, buy 0 sell 2 = 2)
       Total: 3? Actually buy 1 sell 2 = 1, buy 0 sell 2 = 2 total=3

       States:
       hold[i] = max profit holding stock
       sold[i] = max profit sold stock
       rest[i] = max profit in cooldown
```

Solution Approach

Concept: State machine DP.

Algorithm

```
Algorithm MAX_PROFIT_COOLDOWN(prices):
           hold = -prices[0], sold = 0, rest = 0

           For i = 1 to n-1:
               newHold = max(hold, rest - prices[i])
               newSold = hold + prices[i]
               newRest = max(rest, sold)
               hold, sold, rest = newHold, newSold, newRest

           Return max(sold, rest)
```

Java Implementation

```
class Solution {
           public int maxProfit(int[] prices) {
               int n = prices.length;
               if (n < 2) return 0;

                 int hold = -prices[0];
```

```
                int sold = 0;
                int rest = 0;

                for (int i = 1; i < n; i++) {
                    int newHold = Math.max(hold, rest - prices[i]);
                    int newSold = hold + prices[i];
                    int newRest = Math.max(rest, sold);
                    hold = newHold;
                    sold = newSold;
                    rest = newRest;
                }
                return Math.max(sold, rest);
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

392. Buy and Sell Stocks With Transaction Fee (DP-40)

Understanding the Problem NEW

Unlimited transactions, but now every *sale* incurs a fixed transaction fee, which must be subtracted from that transaction's profit.

Brute Force Approach NEW

Same exponential exploration without structure.

Why We Optimize NEW

Very similar DP state to the basic unlimited-transactions case (388), tracking 'holding' vs 'not holding' per day, but the profit calculation upon selling now subtracts the fee: `sell profit = price[i] - buyPrice - fee`. This small tweak to the transition (subtract the fee at the point of selling) is enough to correctly handle the fee constraint — worth noting this specific variant sits between the fully-greedy 388 (no state needed at all) and the more state-heavy cooldown version (391), since a flat fee doesn't require an extra 'cooldown' state category, just an adjusted profit calculation.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum profit with transaction fee.

Visual Diagram

```
Prices: [1,3,2,8,4,9], fee=2
       Max profit: 8 (buy 1 sell 8 = 7-2=5, buy 4 sell 9 = 5-2=3)
       Total: 8

       States:
       cash = max profit without stock
       hold = max profit with stock
```

Solution Approach

Concept: State machine DP with fee.

Algorithm

```
Algorithm MAX_PROFIT_FEE(prices, fee):
           cash = 0, hold = -prices[0]

           For i = 1 to n-1:
               cash = Math.max(cash, hold + prices[i] - fee)
               hold = Math.max(hold, cash - prices[i])

           Return cash
```

Java Implementation

```
class Solution {
           public int maxProfit(int[] prices, int fee) {
               int cash = 0;
               int hold = -prices[0];

                 for (int i = 1; i < prices.length; i++) {
                     cash = Math.max(cash, hold + prices[i] - fee);
```

```
                    hold = Math.max(hold, cash - prices[i]);
                }
                return cash;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

393. Longest Increasing Subsequence (DP-41)

Understanding the Problem NEW

Find the length of the Longest Increasing Subsequence (LIS) in an array — the longest subsequence (not necessarily contiguous) where every element is strictly greater than the previous one.

Brute Force Approach NEW

Try every one of the 2^n subsequences and check which are increasing, tracking the longest — exponential.

Why We Optimize NEW

DP state `lis(i)` — the length of the longest increasing subsequence *ending exactly at index i*: `lis(i) = 1 + max(lis(j))` for every `j < i` where `arr[j] < arr[i]` (or just 1, if no such j exists) — memoized/tabulated per index, giving $O(n^2)$ (checking all previous j for each i). This can be further optimized to O(n log n) using a clever trick: maintain a separate array tracking 'the smallest possible tail value for an increasing subsequence of each length,' and use binary search (from Chapter 3!) to find where each new element fits in — a nice full-circle callback to binary search being useful even in a DP-heavy chapter, whenever a monotonic structure can be exploited.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Length of longest increasing subsequence.

Visual Diagram

```
Array: [10,9,2,5,3,7,101,18]
       LIS = 4 ([2,3,7,101] or [2,5,7,101])

       dp[i] = 1 + max(dp[j]) for j<i and nums[j]<nums[i]
       dp: [1,1,1,2,2,3,4,4]
       Result: 4

       Binary Search Method:
       tails = [10]
       9 → [9]
       2 → [2]
       5 → [2,5]
       3 → [2,3]
       7 → [2,3,7]
       101 → [2,3,7,101]
       18 → [2,3,7,18]
       Length = 4
```

Solution Approach

Concept: DP or binary search.

Algorithm (DP)

Algorithm LIS(nums):

n = nums.length

dp = new int[n]

```
Arrays.fill(dp, 1)

maxLen = 1

For i = 1 to n-1:

For j = 0 to i-1:

If nums[j] < nums[i]:

dp[i] = max(dp[i], dp[j] + 1)

maxLen = max(maxLen, dp[i])

Return maxLen
```

Java Implementation (DP)

```
class Solution {

public int lengthOfLIS(int[] nums) {

int n = nums.length;

int[] dp = new int[n];
```

```
Arrays.fill(dp, 1);

int maxLen = 1;

for (int i = 1; i < n; i++) {

for (int j = 0; j < i; j++) {

if (nums[j] < nums[i]) {

dp[i] = Math.max(dp[i], dp[j] + 1);

}

}

maxLen = Math.max(maxLen, dp[i]);

}

return maxLen;

}

}
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

394. Printing Longest Increasing Subsequence (DP-42)

Understanding the Problem NEW

Same as LIS, but print the actual subsequence, not just its length.

Brute Force Approach NEW

Same exponential subsequence enumeration as plain LIS without a reconstruction plan.

Why We Optimize NEW

While computing the LIS DP table (393), additionally track, for each index, **which previous index** it extended from (whichever j gave the best `lis(j)` value used to compute `lis(i)`) — then after finding the index with the overall maximum LIS length, walk backward through these 'came from' pointers to reconstruct the actual subsequence. Same backward-reconstruction-from-a-filled-table technique as Print LCS (378), applied here to LIS's DP table instead.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print the LIS.

Visual Diagram

```
Array: [10,9,2,5,3,7,101,18]
       LIS: [2,3,7,101]

       Backtrack from dp array.
```

Solution Approach

Concept: DP with parent tracking.

Algorithm

```
Algorithm PRINT_LIS(nums):
           n = nums.length
           dp = new int[n], parent = new int[n]
           Arrays.fill(dp, 1)
           Arrays.fill(parent, -1)
           maxIndex = 0

            For i = 1 to n-1:
                For j = 0 to i-1:
                    If nums[j] < nums[i] && dp[j]+1 > dp[i]:
                        dp[i] = dp[j]+1
                        parent[i] = j
                If dp[i] > dp[maxIndex]: maxIndex = i

            // Backtrack
            lis = []
            i = maxIndex
            While i != -1:
                lis.add(nums[i])
                i = parent[i]
            Return reverse(lis)
```

Java Implementation

```
class Solution {
           public List<Integer> printLIS(int[] nums) {
               int n = nums.length;
               int[] dp = new int[n];
               int[] parent = new int[n];
               Arrays.fill(dp, 1);
               Arrays.fill(parent, -1);
               int maxIndex = 0;

                 for (int i = 1; i < n; i++) {
                     for (int j = 0; j < i; j++) {
                         if (nums[j] < nums[i] && dp[j] + 1 > dp[i]) {
                             dp[i] = dp[j] + 1;
                             parent[i] = j;
                         }
                     }
                     if (dp[i] > dp[maxIndex]) maxIndex = i;
                 }

                 List<Integer> lis = new ArrayList<>();

               int i = maxIndex;
               while (i != -1) {
                   lis.add(nums[i]);
                   i = parent[i];
               }
               Collections.reverse(lis);
               return lis;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

Problem

LIS in O(n log n).

Visual Diagram

```
Array: [10,9,2,5,3,7,101,18]
       tails = []
       10 → [10]
       9 → [9]
       2 → [2]
       5 → [2,5]
       3 → [2,3]
       7 → [2,3,7]
       101 → [2,3,7,101]
       18 → [2,3,7,18]
       Length = 4
```

Solution Approach

Concept: Binary search on tails.

Algorithm

```
Algorithm LIS_BINARY_SEARCH(nums):
           tails = []

            For num in nums:
                pos = lowerBound(tails, num)
                If pos == tails.length:
                    tails.add(num)
                Else:
                    tails[pos] = num

            Return tails.length
```

Java Implementation

```
class Solution {
           public int lengthOfLIS(int[] nums) {
               List<Integer> tails = new ArrayList<>();

                 for (int num : nums) {
                     int pos = Collections.binarySearch(tails, num);
                     if (pos < 0) pos = -(pos + 1);
```

```
                    if (pos == tails.size()) {
                        tails.add(num);
                    } else {
                        tails.set(pos, num);
                    }
                }
                return tails.size();
            }
        }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

396. Largest Divisible Subset (DP-44)

Understanding the Problem NEW

Find the largest subset of an array where every pair of elements satisfies the divisibility property (each element divides, or is divided by, every other element when the subset is sorted).

Brute Force Approach NEW

Try every one of the 2^n subsets and check the pairwise divisibility condition for each — exponential.

Why We Optimize NEW

After **sorting** the array, this becomes structurally identical to LIS (393): a valid divisible subset, once sorted, is exactly a subsequence where each next element is *divisible by* the previous one (analogous to LIS's 'greater than' condition) — reusing the exact same `dp(i) = 1 + max(dp(j))` recurrence, just replacing the 'greater than' comparison with 'divisible by.' Sorting first is what makes this reduction valid, echoing a similar 'sort first, then apply a familiar recurrence' pattern seen in several Greedy-chapter problems.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Largest subset where every pair is divisible.

Visual Diagram

```
Array: [1,2,3,4,8]
       LDS: [1,2,4,8] (or [1,3])

       Sort array.
       dp[i] = largest subset ending at i
       If nums[i] % nums[j] == 0: dp[i] = max(dp[i], dp[j]+1)
```

Solution Approach

Concept: DP on sorted array.

Algorithm

```
Algorithm LARGEST_DIVISIBLE_SUBSET(nums):
           Sort(nums)
           n = nums.length
           dp = new int[n]
           parent = new int[n]
           Arrays.fill(dp, 1)
           Arrays.fill(parent, -1)
           maxIndex = 0

           For i = 1 to n-1:
               For j = 0 to i-1:
                   If nums[i] % nums[j] == 0 && dp[j]+1 > dp[i]:
                       dp[i] = dp[j]+1
                       parent[i] = j
               If dp[i] > dp[maxIndex]: maxIndex = i

           // Backtrack
           subset = []
           i = maxIndex
           While i != -1:
               subset.add(nums[i])
               i = parent[i]
           Return reverse(subset)
```

Java Implementation

```
class Solution {
           public List<Integer> largestDivisibleSubset(int[] nums) {
               Arrays.sort(nums);
               int n = nums.length;
               int[] dp = new int[n];
               int[] parent = new int[n];
               Arrays.fill(dp, 1);
               Arrays.fill(parent, -1);
               int maxIndex = 0;

                 for (int i = 1; i < n; i++) {
                     for (int j = 0; j < i; j++) {
                         if (nums[i] % nums[j] == 0 && dp[j] + 1 > dp[i]) {
                             dp[i] = dp[j] + 1;
                             parent[i] = j;
                         }

                    }
                    if (dp[i] > dp[maxIndex]) maxIndex = i;
               }

               List<Integer> result = new ArrayList<>();
               int i = maxIndex;
               while (i != -1) {
                   result.add(nums[i]);
                   i = parent[i];
               }
               Collections.reverse(result);
               return result;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

397. Longest String Chain (DP-45)

Understanding the Problem NEW

Given a list of words, find the longest chain where each word in the chain is formed by adding exactly one character to the previous word (in any position), and each intermediate word must also exist in the given list.

Brute Force Approach NEW

Try every possible ordering/subset of words and check the chain-forming condition — exponential.

Why We Optimize NEW

After **sorting words by length**, this becomes an LIS-shaped problem again: `dp(word) = 1 + dp(predecessor)` where 'predecessor' is any word in the list that's exactly one character shorter and forms the current word when one character is added — checked by trying every possible single-character removal from the current word and looking it up (e.g. via a hashmap of word → best chain length). Same 'sort first, then apply an LIS-shaped recurrence over a customized 'compatibility' condition instead of simple numeric comparison' pattern as problem 396, just with a string-based 'predecessor' check instead of divisibility.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Longest chain where each string differs by one char.

Visual Diagram

```
Words: ["a","b","ba","bca","bda","bdca"]
       Longest chain: ["a","ba","bda","bdca"] length=4

       Sort by length
       dp[i] = 1 + max(dp[j]) where s[j] is predecessor
```

Solution Approach

Concept: DP on sorted strings.

Algorithm

```
Algorithm LONGEST_STRING_CHAIN(words):
           Sort by length
           dp = new int[n]
           Arrays.fill(dp, 1)
           maxLen = 1

           For i = 1 to n-1:
               For j = 0 to i-1:
                   If isPredecessor(words[j], words[i]):
                       dp[i] = max(dp[i], dp[j]+1)
               maxLen = max(maxLen, dp[i])

           Return maxLen

       Algorithm isPredecessor(s1, s2):
           If s2.length != s1.length+1: Return false
           i = j = 0
           diff = 0
           While i < s1.length && j < s2.length:
               If s1[i] != s2[j]:
                    diff++
                    j++
```

```
                Else:
                     i++; j++
           Return diff <= 1
```

Java Implementation

```
class Solution {
           public int longestStrChain(String[] words) {
               Arrays.sort(words, (a, b) -> a.length() - b.length());
               int n = words.length;
               int[] dp = new int[n];
               Arrays.fill(dp, 1);
               int maxLen = 1;

                 for (int i = 1; i < n; i++) {
                     for (int j = 0; j < i; j++) {
                         if (isPredecessor(words[j], words[i])) {
                             dp[i] = Math.max(dp[i], dp[j] + 1);
                         }
                     }
                     maxLen = Math.max(maxLen, dp[i]);

               }
               return maxLen;
           }

           private boolean isPredecessor(String s1, String s2) {
               if (s2.length() != s1.length() + 1) return false;
               int i = 0, j = 0;
               while (i < s1.length() && j < s2.length()) {
                   if (s1.charAt(i) == s2.charAt(j)) {
                       i++;
                       j++;
                   } else {
                       j++;
                   }
               }
               return i == s1.length();
           }
       }
```

Time & Space Complexity

Time Complexity: O(n² * L) | Space Complexity: O(n)

398. Longest Bitonic Subsequence (DP-46)

Understanding the Problem NEW

Find the longest 'bitonic' subsequence — one that strictly increases and then strictly decreases (like a mountain shape), possibly with only an increasing or only a decreasing part.

Brute Force Approach NEW

Try every subsequence and check the increase-then-decrease shape — exponential.

Why We Optimize NEW

Compute two LIS-style DP tables: one for the **longest increasing subsequence ending at each index** (exactly problem 393's DP, left to right), and one for the **longest decreasing subsequence starting at each index** (the same DP, but scanning right to left, or equivalently LIS on the reversed array) — then for each index, the best bitonic sequence 'peaking' there is `(increasing length ending here) + (decreasing length starting here) - 1` (subtracting 1 to avoid double-counting the peak element itself), and the answer is the maximum over all indices. This combines two separately-computed LIS tables — computed left-to-right and right-to-left — directly reusing the LIS recurrence twice rather than designing a new algorithm for the mountain shape.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Longest increasing then decreasing subsequence.

Visual Diagram

```
Array: [1, 11, 2, 10, 4, 5, 2, 1]
       LBS: [1, 2, 10, 4, 2, 1] length=6

       Compute:
       lis[i] = LIS ending at i
       lds[i] = LDS starting at i
       bitonic[i] = lis[i] + lds[i] - 1
```

Solution Approach

Concept: LIS + LDS.

Algorithm

```
Algorithm BITONIC_SUBSEQUENCE(nums):
           n = nums.length
           lis = LIS_ending_at_each(nums)
           lds = LDS_starting_at_each(nums)

           maxLen = 0
           For i = 0 to n-1:
               maxLen = max(maxLen, lis[i] + lds[i] - 1)

           Return maxLen
```

Java Implementation

```
class Solution {
           public int longestBitonicSubsequence(int[] nums) {
               int n = nums.length;
```

```
                int[] lis = new int[n];
                int[] lds = new int[n];
                Arrays.fill(lis, 1);
                Arrays.fill(lds, 1);

                  // LIS ending at i
                  for (int i = 1; i < n; i++) {
                      for (int j = 0; j < i; j++) {
                          if (nums[j] < nums[i]) {
                              lis[i] = Math.max(lis[i], lis[j] + 1);
                          }
                      }
                  }

                  // LDS starting at i
                  for (int i = n-2; i >= 0; i--) {
                      for (int j = n-1; j > i; j--) {
                          if (nums[j] < nums[i]) {
                              lds[i] = Math.max(lds[i], lds[j] + 1);
                          }
                      }
                  }

                  int maxLen = 0;
                  for (int i = 0; i < n; i++) {

                    maxLen = Math.max(maxLen, lis[i] + lds[i] - 1);
                }
                return maxLen;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

399. Number of Longest Increasing Subsequences (DP-47)

Understanding the Problem NEW

Count how many distinct Longest Increasing Subsequences exist (not just find the length of one, but count all subsequences that achieve that maximum length).

Brute Force Approach NEW

Enumerate every subsequence, filter for increasing ones matching the maximum length, and count — exponential.

Why We Optimize NEW

Extend the LIS DP (393) with a parallel **count array**: alongside `lis(i)` (the length), track `count(i)` (how many distinct subsequences of that maximum length end at index i) — when extending from a previous index j, if `lis(j)+1` creates a *new best* length at i, reset `count(i)` to `count(j)`; if it *ties* the current best length at i, **add** `count(j)` to `count(i)` (multiple ways to reach the same best length). Sum `count(i)` over every index that achieves the overall maximum LIS length. This is the same 'add a parallel counting array alongside the main DP table' technique seen in graph shortest-path counting (Number of Ways to Arrive at Destination, problem 334) — applied here to LIS instead of shortest paths.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count of LIS.

Visual Diagram

```
Array: [1,3,5,4,7]
       LIS length = 3, count = 2 ([1,3,5,7] and [1,3,4,7])

       dp[i] = length of LIS ending at i
       count[i] = number of LIS ending at i
```

Solution Approach

Concept: DP with length and count.

Algorithm

```
Algorithm COUNT_LIS(nums):
           n = nums.length
           dp = new int[n]
           count = new int[n]
           Arrays.fill(dp, 1)
           Arrays.fill(count, 1)
           maxLen = 1

           For i = 1 to n-1:
               For j = 0 to i-1:
                   If nums[j] < nums[i]:
                       If dp[j] + 1 > dp[i]:
                           dp[i] = dp[j] + 1
                           count[i] = count[j]
                       Else If dp[j] + 1 == dp[i]:
                           count[i] += count[j]
               maxLen = max(maxLen, dp[i])

           total = 0
           For i = 0 to n-1:
               If dp[i] == maxLen:
```

```
                total += count[i]
        Return total
```

Java Implementation

```
class Solution {
        public int findNumberOfLIS(int[] nums) {
            int n = nums.length;
            int[] dp = new int[n];
            int[] count = new int[n];
            Arrays.fill(dp, 1);
            Arrays.fill(count, 1);
            int maxLen = 1;

              for (int i = 1; i < n; i++) {
                  for (int j = 0; j < i; j++) {
                      if (nums[j] < nums[i]) {
                          if (dp[j] + 1 > dp[i]) {
                              dp[i] = dp[j] + 1;
                              count[i] = count[j];
                          } else if (dp[j] + 1 == dp[i]) {
                              count[i] += count[j];

                           }
                       }
                  }
                  maxLen = Math.max(maxLen, dp[i]);
              }

              int total = 0;
              for (int i = 0; i < n; i++) {
                  if (dp[i] == maxLen) {
                      total += count[i];
                  }
              }
              return total;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

400. Matrix Chain Multiplication (DP-48)

Understanding the Problem NEW

Given a chain of matrices to multiply together, find the minimum number of scalar multiplications needed, by choosing the optimal order/parenthesization to multiply them in (matrix multiplication is associative, so different groupings give the same result but very different costs).

Brute Force Approach NEW

Try every possible way to parenthesize the matrix chain — this follows the Catalan number sequence, growing exponentially with the number of matrices.

Why We Optimize NEW

DP state `minCost(i, j)` — the minimum cost to multiply matrices i through j: try every possible **split point k** between i and j, computing `minCost(i,k) + minCost(k+1,j) + (cost of multiplying the two resulting combined matrices)`, taking the minimum over every possible k. This is a genuinely new DP **shape** for this chapter — an 'interval DP,' where the state is a *range* (i,j) rather than a single index or index-pair-between-two-sequences — memoized/tabulated over all $O(n^2)$ possible (i,j) ranges, each trying $O(n)$ split points, giving $O(n^3)$ instead of the exponential Catalan-number-scaling brute force. This interval-DP shape (state = a range, transition = try every split point) becomes the template for several remaining 'hard'

> problems in this chapter (Cutting a Stick, Burst Balloons, Palindrome Partitioning-II).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum scalar multiplications to multiply matrices.

Visual Diagram

```
Dimensions: [10, 30, 5, 60]
       Matrices: A1(10x30), A2(30x5), A3(5x60)
       Min: 4500
       Order: (A1*A2)*A3 = 10*30*5 + 10*5*60 = 1500+3000=4500

       dp[i][j] = min cost to multiply matrices i to j
       dp[i][j] = min(dp[i][k] + dp[k+1][j] + dims[i]*dims[k+1]*dims[j+1])
```

Solution Approach

Concept: Partition DP.

Algorithm

```
Algorithm MCM(dims):
           n = dims.length - 1
           dp = int[n][n]

           For len = 2 to n:
               For i = 0 to n-len:
                   j = i + len - 1
                   dp[i][j] = INF
                   For k = i to j-1:
                       cost = dp[i][k] + dp[k+1][j] + dims[i]*dims[k+1]*dims[j+1]
                       dp[i][j] = min(dp[i][j], cost)
```

```
            Return dp[0][n-1]
```

Java Implementation

```
class Solution {
            public int matrixChainOrder(int[] dims) {
                int n = dims.length - 1;
                int[][] dp = new int[n][n];

                  for (int len = 2; len <= n; len++) {
                      for (int i = 0; i <= n - len; i++) {
                          int j = i + len - 1;
                          dp[i][j] = Integer.MAX_VALUE;
                          for (int k = i; k < j; k++) {
                              int cost = dp[i][k] + dp[k+1][j] + dims[i] * dims[k+1]
* dims[j+1];
                              dp[i][j] = Math.min(dp[i][j], cost);
                          }
                      }
                  }
                  return dp[0][n-1];
            }
       }
```

Time & Space Complexity

Time Complexity: O(n³) | Space Complexity: O(n²)

401. Matrix Chain Multiplication Bottom-Up (DP-49)

Understanding the Problem NEW

The exact same Matrix Chain Multiplication problem (400), solved bottom-up (tabulation) instead of top-down (memoization).

Brute Force Approach NEW

Same exponential Catalan-scaling brute force as 400 without any DP.

Why We Optimize NEW

Fill the `minCost(i,j)` table by **increasing interval length** — first all length-1 ranges (trivially cost 0, a single matrix), then length-2, then length-3, and so on, since computing any `minCost(i,j)` requires smaller sub-ranges to already be filled in. This bottom-up ordering (process by increasing range size, rather than recursing top-down) is a general pattern for tabulating *any* interval DP, worth remembering for problems 402-406 which share this same interval-DP shape.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

MCM bottom-up approach.

Visual Diagram

```
Same as above with iterative DP.
```

Java Implementation

```
class Solution {
           public int matrixChainOrder(int[] dims) {
               int n = dims.length;
               int[][] dp = new int[n][n];

               for (int len = 2; len < n; len++) {
                   for (int i = 1; i < n - len + 1; i++) {
                       int j = i + len - 1;
                       dp[i][j] = Integer.MAX_VALUE;
                       for (int k = i; k < j; k++) {
                           int cost = dp[i][k] + dp[k+1][j] + dims[i-1] * dims[k] *
dims[j];
                           dp[i][j] = Math.min(dp[i][j], cost);
                       }
                   }
               }
               return dp[1][n-1];
           }
       }
```

Time & Space Complexity

Time Complexity: $O(n^3)$ | Space Complexity: $O(n^2)$

402. Minimum Cost to Cut the Stick (DP-50)

Understanding the Problem NEW

Given a stick of a certain length and a set of positions where cuts must be made, find the minimum total cost to make all the cuts — where each individual cut costs the length of the piece being cut at that moment (so cutting order matters a lot).

Brute Force Approach NEW

Try every possible order of making the cuts and compute the total cost for each — n! orderings, factorial and infeasible for more than a few cuts.

Why We Optimize NEW

Same interval-DP shape as Matrix Chain Multiplication: `minCost(left, right)` — the minimum cost to make all required cuts strictly between positions left and right — tries every possible cut position `c` within that range as the *first* cut made in that sub-range, paying `(right - left)` for that cut (the current piece's length) plus the cost of recursively handling the two resulting sub-pieces. This directly reuses the 'range as state, try every split point' interval-DP template from Matrix Chain Multiplication, achieving $O(n^3)$ instead of the factorial brute-force ordering search.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum cost to cut a stick at given positions.

Visual Diagram

```
n=7, cuts=[1,3,4,5]
       Min cost = 16

       Add 0 and n to cuts, sort.
       dp[i][j] = min cost to cut between cuts[i] and cuts[j]
       dp[i][j] = min(dp[i][k] + dp[k][j] + (cuts[j]-cuts[i]))
```

Solution Approach

Concept: Partition DP.

Algorithm

```
Algorithm MIN_COST_CUT(n, cuts):
           cuts = [0] + sort(cuts) + [n]
           m = cuts.length
           dp = int[m][m]

           For len = 2 to m-1:
               For i = 0 to m-len-1:
                   j = i + len
                   dp[i][j] = INF
                   For k = i+1 to j-1:
                       dp[i][j] = min(dp[i][j], dp[i][k] + dp[k][j] + cuts[j] -
cuts[i])

           Return dp[0][m-1]
```

Java Implementation

```
class Solution {
           public int minCost(int n, int[] cuts) {
               int m = cuts.length + 2;
               int[] newCuts = new int[m];
               newCuts[0] = 0;
               newCuts[m-1] = n;
               System.arraycopy(cuts, 0, newCuts, 1, cuts.length);
               Arrays.sort(newCuts);

                 int[][] dp = new int[m][m];
                 for (int len = 2; len < m; len++) {
                     for (int i = 0; i < m - len; i++) {
                         int j = i + len;
                         dp[i][j] = Integer.MAX_VALUE;
                         for (int k = i + 1; k < j; k++) {
                             dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k][j] +
newCuts[j] - newCuts[i]
                         }
                     }
                 }
                 return dp[0][m-1];
           }
       }
```

Time & Space Complexity

Time Complexity: O(m³) | Space Complexity: O(m²)

403. Burst Balloons (DP-51)

Understanding the Problem NEW

Given balloons each with a value, bursting a balloon earns coins equal to the product of its value and its two current neighbours' values (after bursting, the balloon's former neighbours become adjacent to each other); find the maximum total coins from bursting all balloons in some order.

Brute Force Approach NEW

Try every possible order of bursting all balloons — n! orderings, factorial and infeasible.

Why We Optimize NEW

The trick that makes this an interval DP: instead of thinking 'which balloon do I burst first,' think '**which balloon do I burst LAST** within a given range' — because the last balloon burst in a range has, by definition, the two range-boundary balloons as its final neighbours (since everything else in the range is already gone). DP state `maxCoins(left, right)` tries every possible balloon `k` as the 'last burst' within that range, computing `maxCoins(left,k) + maxCoins(k,right) + value[left]×value[k]×value[right]`. This 'think about what happens last, not first' reframing (rather than the more intuitive 'first choice' framing that doesn't decompose cleanly) is the key insight, still fitting the same interval-DP template as Matrix Chain Multiplication and Cutting a Stick, achieving $O(n^3)$ instead of factorial ordering exploration.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Max coins by bursting balloons.

Visual Diagram

```
Balloons: [3,1,5,8]
       Max coins: 167
       Order: 1, 5, 3, 8

       Add 1 at both ends: [1,3,1,5,8,1]
       dp[i][j] = max coins from bursting i+1..j-1
       dp[i][j] = max(dp[i][k] + dp[k][j] + nums[i]*nums[k]*nums[j])
```

Solution Approach

Concept: Partition DP.

Algorithm

```
Algorithm MAX_COINS(nums):
           n = nums.length
           newNums = [1] + nums + [1]
           dp = int[n+2][n+2]

           For len = 1 to n:
               For i = 1 to n-len+1:
                   j = i + len - 1
                   For k = i to j:
                       dp[i][j] = max(dp[i][j], dp[i][k-1] + dp[k+1][j] +
newNums[i-1]*newNums[k]*newN

           Return dp[1][n]
```

Java Implementation

```
class Solution {
           public int maxCoins(int[] nums) {
               int n = nums.length;
               int[] newNums = new int[n + 2];
               newNums[0] = 1;
               newNums[n+1] = 1;
               System.arraycopy(nums, 0, newNums, 1, n);

                 int[][] dp = new int[n+2][n+2];
                 for (int len = 1; len <= n; len++) {
                     for (int i = 1; i <= n - len + 1; i++) {
                         int j = i + len - 1;
                         for (int k = i; k <= j; k++) {
                             dp[i][j] = Math.max(dp[i][j],
                                  dp[i][k-1] + dp[k+1][j] + newNums[i-1] *
newNums[k] * newNums[j+1]);
                         }
                     }
                 }
                 return dp[1][n];
           }
       }
```

Time & Space Complexity

Time Complexity: $O(n^3)$ | Space Complexity: $O(n^2)$

404. Evaluate Boolean Expression to True (DP-52)

Understanding the Problem NEW

Given a boolean expression with operands (True/False) and operators (AND, OR, XOR), count the number of ways to parenthesize the expression such that it evaluates to True.

Brute Force Approach NEW

Try every possible parenthesization — following the same Catalan-number exponential growth as Matrix Chain Multiplication's naive approach.

Why We Optimize NEW

Interval DP again, but tracking **two** counts per range instead of one: `waysTrue(left,right)` and `waysFalse(left,right)` — for every split point (every operator position) within the range, compute how many ways the left and right sub-expressions can each evaluate to True or False, then combine them according to the specific operator's truth table (e.g. for OR, the only way to get False is both sides False; every other combination gives True) to determine how many combined ways yield True vs False overall. This extends the interval-DP template with a 'track two parallel counts instead of one value' twist — similar in spirit to how LIS-counting (399) added a parallel count array alongside its main DP value.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Ways to evaluate expression to True.

Visual Diagram

```
s = "T|F", result=true
       Ways: 1

       dp[i][j][0/1] = ways to get False/True
       If operator == '|':
           True = leftTrue*rightTrue + leftTrue*rightFalse + leftFalse*rightTrue
           False = leftFalse*rightFalse
```

Solution Approach

Concept: Partition DP.

Algorithm

```
Algorithm BOOLEAN_EVALUATION(s, result):
           n = s.length
           dp = new int[n][n][2]

           For i = 0 to n-1 step 2:
               dp[i][i][1] = s[i] == 'T' ? 1 : 0
               dp[i][i][0] = s[i] == 'F' ? 1 : 0

           For len = 3 to n step 2:
               For i = 0 to n-len step 2:
                   j = i + len - 1
                   For k = i+1 to j-1 step 2:
                       For op in [T,F]:
                           // Count ways for each operator
           Return dp[0][n-1][1]
```

Java Implementation

```
class Solution {
           public int countWays(String s) {
               int n = s.length();
               int[][][] dp = new int[n][n][2];

                 for (int i = 0; i < n; i += 2) {
                     dp[i][i][0] = s.charAt(i) == 'F' ? 1 : 0;
                     dp[i][i][1] = s.charAt(i) == 'T' ? 1 : 0;
                 }

                 for (int len = 3; len <= n; len += 2) {
                     for (int i = 0; i <= n - len; i += 2) {
                         int j = i + len - 1;
                         for (int k = i + 1; k < j; k += 2) {
                             char op = s.charAt(k);
                             int lt = dp[i][k-1][1], lf = dp[i][k-1][0];
                             int rt = dp[k+1][j][1], rf = dp[k+1][j][0];

                             if (op == '|') {
                                 dp[i][j][1] += lt*rt + lt*rf + lf*rt;
                                 dp[i][j][0] += lf*rf;
                             } else if (op == '&') {
                                 dp[i][j][1] += lt*rt;

                               dp[i][j][0] += lf*rf + lt*rf + lf*rt;
                           } else { // '^'
                               dp[i][j][1] += lt*rf + lf*rt;
                               dp[i][j][0] += lt*rt + lf*rf;
                           }
                       }
                   }
               }
               return dp[0][n-1][1];
           }
       }
```

Time & Space Complexity

Time Complexity: $O(n^3)$ | Space Complexity: $O(n^2)$

405. Palindrome Partitioning - II (DP-53)

Understanding the Problem NEW

Find the minimum number of cuts needed to partition a string into pieces that are all palindromes.

Brute Force Approach NEW

Try every possible way to place cuts and check if every resulting piece is a palindrome, tracking the minimum cuts — exponential (similar to the Recursion chapter's Palindrome Partitioning problem, 143, but this version only needs the minimum cut *count*, not every possible partition).

Why We Optimize NEW

DP state `minCuts(i)` — the minimum cuts needed for the first i characters: for every possible last piece (from some start position j to i), if that substring is a palindrome (precomputed once via a separate O(n²) palindrome-checking table, so each check is O(1) rather than re-verified from scratch), then `minCuts(i) = min(minCuts(j-1) + 1)` over all valid j. This connects back to the Recursion chapter's backtracking approach (143, which explored every valid partition) by instead just tracking the minimum cut count via DP — a good example of a problem sitting on the boundary between 'backtracking generates every valid answer' (Recursion chapter) and 'DP finds just the optimal count' (this chapter), achieving O(n²) instead of the recursion chapter's more exploratory approach.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum cuts for palindrome partitions.

Visual Diagram

```
s = "aab"
       Min cuts: 1 ("aa","b")

       dp[i] = min cuts for s[0..i]
       If palindrome: dp[i] = 0
       Else: dp[i] = min(dp[j] + 1) for j<i and s[j+1..i] palindrome
```

Solution Approach

Concept: DP with palindrome check.

Algorithm

```
Algorithm MIN_CUTS(s):
           n = s.length()
           isPal = boolean[n][n]
           dp = new int[n]

           For len = 1 to n:
               For i = 0 to n-len:
                   j = i + len - 1
                   If s[i] == s[j] && (len <= 2 || isPal[i+1][j-1]):
                       isPal[i][j] = true

           For i = 0 to n-1:
               If isPal[0][i]:
                   dp[i] = 0
               Else:
                   dp[i] = INF
                   For j = 0 to i-1:
                       If isPal[j+1][i]:
```

```
                            dp[i] = min(dp[i], dp[j] + 1)

            Return dp[n-1]
```

Java Implementation

```
class Solution {
            public int minCut(String s) {
                int n = s.length();
                boolean[][] isPal = new boolean[n][n];

                  for (int len = 1; len <= n; len++) {
                      for (int i = 0; i <= n - len; i++) {
                          int j = i + len - 1;
                          if (s.charAt(i) == s.charAt(j) && (len <= 2 || isPal[i+1]
[j-1])) {
                              isPal[i][j] = true;
                          }
                      }
                  }

                  int[] dp = new int[n];
                  for (int i = 0; i < n; i++) {
                      if (isPal[0][i]) {
                          dp[i] = 0;

                    } else {
                        dp[i] = Integer.MAX_VALUE;
                        for (int j = 0; j < i; j++) {
                             if (isPal[j+1][i]) {
                                 dp[i] = Math.min(dp[i], dp[j] + 1);
                             }
                        }
                    }
                }
                return dp[n-1];
            }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n²)

406. Partition Array for Maximum Sum (DP-54)

Understanding the Problem NEW

Partition an array into contiguous subarrays of length at most K, where each subarray's elements all get replaced by that subarray's maximum value; maximize the total sum after this transformation.

Brute Force Approach NEW

Try every possible way to partition the array into valid-length groups — exponential without structure, since there are many ways to place partition boundaries.

Why We Optimize NEW

DP state `maxSum(i)` — the best achievable sum for the first i elements: try every possible **last group length** from 1 to K ending at position i, computing `maxSum(i) = max over length L of (maxSum(i-L) + L × max(last L elements))`. This is a 1D DP (unlike the interval DPs of 400-404) but still requires trying multiple 'how far back does the last group extend' options at each position — similar in flavor to the K-distance Frog Jump (356), just with a more involved per-position computation (finding the max of the last L elements) rather than a simple cost lookup.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Partition array to maximize sum of max * length.

Visual Diagram

```
Array: [1,15,7,9,2,5,10], k=3
       Max sum: 84

       dp[i] = max sum for first i elements
       dp[i] = max(dp[i-k] + max * k) for k=1..min(k,i)
```

Solution Approach

Concept: DP with sliding window.

Algorithm

```
Algorithm PARTITION_MAX_SUM(arr, k):
           n = arr.length
           dp = new int[n+1]

           For i = 1 to n:
               maxVal = 0
               For j = i-1 down to max(0, i-k):
                   maxVal = max(maxVal, arr[j])
                   dp[i] = max(dp[i], dp[j] + maxVal * (i-j))

           Return dp[n]
```

Java Implementation

```
class Solution {
           public int maxSumAfterPartitioning(int[] arr, int k) {
               int n = arr.length;
               int[] dp = new int[n+1];

                 for (int i = 1; i <= n; i++) {
```

```
                    int maxVal = 0;
                    for (int j = i-1; j >= Math.max(0, i-k); j--) {
                        maxVal = Math.max(maxVal, arr[j]);
                        dp[i] = Math.max(dp[i], dp[j] + maxVal * (i-j));
                    }
                }
                return dp[n];
            }
        }
```

Time & Space Complexity

Time Complexity: O(n*k) | Space Complexity: O(n)

407. Maximum Rectangle Area with all 1's (DP-55)

Understanding the Problem NEW

Given a binary matrix, find the area of the largest rectangle containing only 1s — the same problem as the Stack/Queue chapter's Maximal Rectangle (problem 193), revisited here to connect it to this chapter's DP techniques.

Brute Force Approach NEW

Check every possible rectangle directly — extremely expensive, as flagged back in problem 193.

Why We Optimize NEW

As established in problem 193: treat each row as the 'ground level' of a histogram (building a running 'height' for each column based on consecutive 1s above), and apply the Largest Rectangle in Histogram technique (a monotonic-stack technique from the Stack/Queue chapter, problem 192) to each row's histogram — O(rows × cols) total. This problem is placed here specifically to highlight that not every 'hard' problem in the DP chapter needs a DP solution — sometimes recognizing a connection to a technique from an entirely different chapter (monotonic stacks) is the real insight, and forcing a DP-only lens onto every problem in a DP chapter would be a mistake.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Largest rectangle of 1s in binary matrix.

Visual Diagram

```
Matrix:
       1 0 1 0 0
       1 0 1 1 1
       1 1 1 1 1
       1 0 0 1 0

       Max area: 6 (rows 1-2, cols 2-4)

       Convert to histogram heights:
       Row0: [1,0,1,0,0]
       Row1: [2,0,2,1,1]
       Row2: [3,1,3,2,2]
       Row3: [4,0,0,3,0]
       Max rectangle in histogram = 6
```

Solution Approach

Concept: Histogram DP.

Algorithm

```
Algorithm MAXIMAL_RECTANGLE(matrix):
           m = matrix.length, n = matrix[0].length
           heights = new int[n]
           maxArea = 0

            For i = 0 to m-1:
                For j = 0 to n-1:
                    If matrix[i][j] == '1':
                        heights[j]++
                    Else:
                        heights[j] = 0
                maxArea = max(maxArea, largestRectangleArea(heights))
```

```
            Return maxArea
```

Java Implementation

```
class Solution {
           public int maximalRectangle(char[][] matrix) {
               if (matrix.length == 0) return 0;
               int m = matrix.length, n = matrix[0].length;
               int[] heights = new int[n];
               int maxArea = 0;

                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         heights[j] = matrix[i][j] == '1' ? heights[j] + 1 : 0;
                     }
                     maxArea = Math.max(maxArea, largestRectangleArea(heights));
                 }
                 return maxArea;

          }

          private int largestRectangleArea(int[] heights) {
              Stack<Integer> stack = new Stack<>();
              int maxArea = 0;
              int n = heights.length;

              for (int i = 0; i <= n; i++) {
                  int h = (i == n) ? 0 : heights[i];
                  while (!stack.isEmpty() && heights[stack.peek()] > h) {
                      int height = heights[stack.pop()];
                      int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                      maxArea = Math.max(maxArea, height * width);
                  }
                  stack.push(i);
              }
              return maxArea;
          }
      }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(n)

408. Count Square Submatrices with All Ones (DP-56)

Understanding the Problem NEW

Given a binary matrix, count the total number of square submatrices consisting entirely of 1s (of any size, from 1×1 up to the largest possible).

Brute Force Approach NEW

Check every possible square (every top-left corner and every possible size) directly — O((rows×cols)²) or worse.

Why We Optimize NEW

DP state `largestSquareEndingAt(r,c)` — the size of the largest all-1s square with its bottom-right corner at (r,c): if the current cell is a 1, `dp(r,c) = 1 + min(dp(r-1,c), dp(r,c-1), dp(r-1,c-1))` (limited by the smallest of the three neighboring squares, since all three must support at least that size for a bigger square to form) — and critically, **the value at each cell directly tells you how many valid squares have their bottom-right corner there** (a cell with dp value 3 means there are 3 valid squares — sizes 1, 2, and 3 — ending there), so summing `dp(r,c)` over the entire grid gives the total count directly. This closing problem elegantly reuses the classic 'largest square ending here' DP recurrence, with the added observation that the DP *value itself* (not just its maximum) directly answers a counting question — a fitting note to end the chapter on, given how often this book has shown the same DP table serving

> multiple related questions (length, count, or reconstruction) with only a small twist in how the final answer is read off.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count all square submatrices of 1s.

Visual Diagram

```
Matrix:
       [1,1,1]
       [1,1,1]
       [1,1,1]

       Squares:
       1x1: 9
       2x2: 4
       3x3: 1
       Total: 14

       dp[i][j] = size of largest square ending at (i,j)
       If matrix[i][j] == 1:
           dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
       Else: dp[i][j] = 0
```

Solution Approach

Concept: DP with min of neighbors.

Algorithm

```
Algorithm COUNT_SQUARES(matrix):
           m = matrix.length, n = matrix[0].length
           dp = int[m][n]
           total = 0
```

```
            For i = 0 to m-1:
                For j = 0 to n-1:
                    If matrix[i][j] == 1:
                        If i == 0 || j == 0:
                            dp[i][j] = 1
                        Else:
                            dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
                        total += dp[i][j]

            Return total
```

Java Implementation

```
class Solution {
            public int countSquares(int[][] matrix) {
                int m = matrix.length, n = matrix[0].length;
                int[][] dp = new int[m][n];
                int total = 0;

                  for (int i = 0; i < m; i++) {
                      for (int j = 0; j < n; j++) {
                          if (matrix[i][j] == 1) {
                              if (i == 0 || j == 0) {
                                  dp[i][j] = 1;
                              } else {
                                  dp[i][j] = 1 + Math.min(dp[i-1][j],
                                              Math.min(dp[i][j-1], dp[i-1][j-1]));
                              }

                               total += dp[i][j];
                           }
                       }
                   }
                   return total;
               }
          }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

Dynamic Programming Summary

Problem	Category	Time	Space	Key Technique
Fibonacci/Intro	1D DP	O(n)	O(1)	DP basics
Climbing Stairs	1D DP	O(n)	O(1)	Fibonacci pattern
Frog Jump	1D DP	O(n)	O(n)	Two-step transition
Frog Jump K	1D DP	O(n*k)	O(n)	K-step transition
Max Non-Adjacent	1D DP	O(n)	O(1)	Include/Exclude
House Robber	1D DP	O(n)	O(1)	Include/Exclude
Ninja Training	2D DP	O(n)	O(n*3)	Activity constraint
Unique Paths	Grid	O(m*n)	O(m*n)	Left + Top
Min Path Sum	Grid	O(m*n)	O(m*n)	Min of neighbors
Falling Path	Grid	O(m*n)	O(m*n)	Three directions
Subset Sum	Subsequence	O(n*target)	O(target)	Include/Exclude
Equal Subset	Subsequence	O(n*target)	O(target)	Target sum
Knapsack	Subsequence	O(n*W)	O(W)	Weight capacity
Min Coins	Subsequence	O(amount*coins)	O(amount)	Unbounded
Coin Change 2	Subsequence	O(coins*amount)	O(amount)	Unbounded count
LCS	Strings	O(m*n)	O(m*n)	Match/Skip
Edit Distance	Strings	O(m*n)	O(m*n)	Delete/Insert/ Replace
Wildcard Matching	Strings	O(m*n)	O(m*n)	Pattern matching
Stock I	Stocks	O(n)	O(1)	Min price
Stock II	Stocks	O(n)	O(1)	Positive differences
Stock IV	Stocks	O(k*n)	O(k*n)	DP with transactions
LIS	LIS	$O(n^2)$	O(n)	DP on indices
LIS Binary	LIS	O(n log n)	O(n)	Binary search
String Chain	LIS	$O(n^2*L)$	O(n)	Predecessor check
MCM	Partition	$O(n^3)$	$O(n^2)$	Interval DP

Burst Balloons	Partition	$O(n^3)$	$O(n^2)$	Interval DP
Max Rectangle	Squares	O(m*n)	O(n)	Histogram DP
Count Squares	Squares	O(m*n)	O(m*n)	Min of neighbors

— End of enhanced Chapter 13: Dynamic Programming —

Tries & Strings — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

409. Implement TRIE | INSERT | SEARCH | STARTSWITH

Understanding the Problem NEW

A Trie (pronounced 'try,' from 'retrieval') is a tree specifically designed for storing strings, where each node represents one character, and following a path from the root spells out a prefix — words that share a common prefix share the same path down the tree. Implement `insert` (add a word), `search` (check if an exact word exists), and `startsWith` (check if any word has this prefix).

Brute Force Approach NEW

Without a Trie, you could store all words in a hashset — `search` would be O(word length) (fine), but `startsWith` would require scanning *every* stored word to check if any of them start with the given prefix — O(number of words × word length), since a plain hashset has no concept of 'shared prefixes.'

Why We Optimize NEW

A Trie's whole design directly targets this weakness: since words sharing a prefix share the same initial path in the tree, checking `startsWith` just means **walking down the tree following the prefix's characters** — if you can walk the whole prefix without hitting a missing child, the prefix exists, in O(prefix length) regardless of how many words are stored. `insert` and `search` are similarly O(word length) — walking down, creating nodes as needed (insert) or checking they all exist and the final node is marked 'end of word' (search). This is the entire reason Tries exist: they turn prefix-related queries from 'expensive because you must consider every stored word' into 'cheap because shared prefixes are pre-organized by the tree's structure itself.'

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Trie (Prefix Tree) with insert, search, and startsWith operations.

Visual Diagram

```
Trie Structure:
                    root
```

```
                /   |  \
              a     b   c
            /       |     \
          p         o      a
        /           |        \
      p             b        t
    [apple] [bob]          [cat]

    Insert "apple":
    root → a → p → p → l → e
    Mark end at 'e'

    Search "app":
    root → a → p → p → not end → false

    StartsWith "app":
    root → a → p → p → exists → true
```

Solution Approach

Concept: Each node has children array and end marker.

Algorithm

```
Algorithm TRIE:
           class Node:
               children = Node[26]
               isEnd = false

           insert(word):
               node = root
               For char in word:
                   index = char - 'a'
                   If node.children[index] == null:
                       node.children[index] = new Node()
                   node = node.children[index]
               node.isEnd = true

           search(word):
               node = root
               For char in word:
                   index = char - 'a'
                   If node.children[index] == null:
                       Return false

                  node = node.children[index]
              Return node.isEnd

          startsWith(prefix):
              node = root
              For char in prefix:
```

```
                index = char - 'a'
                If node.children[index] == null:
                    Return false
                node = node.children[index]
            Return true
```

Java Implementation

```
class Trie {
          private class Node {
              Node[] children;
              boolean isEnd;

              Node() {
                  children = new Node[26];
                  isEnd = false;
              }
          }

          private Node root;

          public Trie() {
              root = new Node();
          }

          public void insert(String word) {
              Node node = root;
              for (char c : word.toCharArray()) {
                  int index = c - 'a';
                  if (node.children[index] == null) {
                      node.children[index] = new Node();
                  }
                  node = node.children[index];
              }
              node.isEnd = true;
          }

          public boolean search(String word) {
              Node node = root;
              for (char c : word.toCharArray()) {
                  int index = c - 'a';
                  if (node.children[index] == null) {
                      return false;
                  }
                  node = node.children[index];
              }
              return node.isEnd;
          }

          public boolean startsWith(String prefix) {
              Node node = root;
              for (char c : prefix.toCharArray()) {
```

```
                int index = c - 'a';
                if (node.children[index] == null) {
                    return false;
                }
                node = node.children[index];
            }
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(L) | Space Complexity: O(26 * nodes)

410. Implement Trie - 2 (Prefix Tree)

Understanding the Problem NEW

Extend the basic Trie to also support counting: how many words currently stored have a given prefix, and how many equal a given word exactly (useful for a Trie that supports duplicates and deletion).

Brute Force Approach NEW

Without extra bookkeeping in the Trie nodes, you'd have to fall back to scanning every stored word to count prefix or exact-word matches — the same O(words × length) weakness a plain Trie was built to avoid.

Why We Optimize NEW

Store two small counters at **every node**: `countPrefix` (how many words pass through this node) and `countEndsHere` (how many words end exactly at this node) — both updated incrementally during `insert` (incrementing along the path) and `delete` (decrementing along the path, removing nodes whose counts drop to zero). This keeps prefix-count and word-count queries at O(word/prefix length), same as basic search, by maintaining running counts as extra state on nodes you're already visiting during insert/search anyway — no need to ever scan the full word list.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Extended Trie with count of words and prefixes.

Visual Diagram

```
Insert "apple":
       root → a → p → p → l → e
       Counts: each node prefix count++, end at 'e'

       Insert "app":
       root → a → p → p
       Counts: prefix count++, end at 'p'
```

```
countWordsEqualTo("app"): 1
countWordsStartingWith("app"): 2
```

Solution Approach

Concept: Maintain prefix count and end count.

Algorithm

```
Algorithm TRIE_2:
           class Node:
               children = Node[26]
               prefixCount = 0
               endCount = 0

           insert(word):
               node = root
               For char in word:
                   index = char - 'a'
                   If node.children[index] == null:
                       node.children[index] = new Node()
                   node = node.children[index]
                   node.prefixCount++
               node.endCount++

           countWordsEqualTo(word):
               node = root
               For char in word:
                   index = char - 'a'
                   If node.children[index] == null:
                       Return 0
                   node = node.children[index]
               Return node.endCount

           countWordsStartingWith(prefix):
               node = root
               For char in prefix:
                   index = char - 'a'
                   If node.children[index] == null:
                       Return 0
                   node = node.children[index]
               Return node.prefixCount

           erase(word):
               // Decrement counts along path
```

Java Implementation

```
class Trie2 {
         private class Node {
```

```
        Node[] children;
        int prefixCount;
        int endCount;

        Node() {
            children = new Node[26];
            prefixCount = 0;
            endCount = 0;
        }
    }

    private Node root;

    public Trie2() {
        root = new Node();
    }

    public void insert(String word) {
        Node node = root;
        for (char c : word.toCharArray()) {
            int index = c - 'a';
            if (node.children[index] == null) {
                node.children[index] = new Node();
            }
            node = node.children[index];
            node.prefixCount++;
        }
        node.endCount++;
    }

    public int countWordsEqualTo(String word) {
        Node node = root;
        for (char c : word.toCharArray()) {
            int index = c - 'a';
            if (node.children[index] == null) {
                return 0;
            }
            node = node.children[index];
        }
        return node.endCount;
    }

    public int countWordsStartingWith(String prefix) {
        Node node = root;
        for (char c : prefix.toCharArray()) {
            int index = c - 'a';
            if (node.children[index] == null) {
                return 0;
            }
            node = node.children[index];
        }
        return node.prefixCount;
    }
```

```
        public void erase(String word) {
            if (countWordsEqualTo(word) == 0) return;
            eraseHelper(root, word, 0);
        }

        private boolean eraseHelper(Node node, String word, int index) {
            if (index == word.length()) {
                node.endCount--;
                return true;
            }

            int idx = word.charAt(index) - 'a';
            if (eraseHelper(node.children[idx], word, index + 1)) {
                node.children[idx].prefixCount--;
                if (node.children[idx].prefixCount == 0) {
                    node.children[idx] = null;
                }
                return true;
            }
            return false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(L) | Space Complexity: O(26 * nodes)

411. Longest String with All Prefixes

Understanding the Problem NEW

Given a list of words, find the longest word such that every one of its prefixes (of every length) is also present somewhere in the list.

Brute Force Approach NEW

For every word, check whether every one of its prefixes exists in the word list — using a hashset, each prefix check is O(1) average, but checking all prefixes of all words gives O(total characters × average word length) if not organized well, and doesn't easily let you build up longer valid words incrementally.

Why We Optimize NEW

Insert every word into a **Trie**, marking end-of-word nodes; then do a DFS/BFS traversal of the Trie, but only continuing into a child if that child is itself marked as an end-of-word (ensuring the prefix built so far is a genuine complete word from the list, not just a partial path) — track the longest such path found. This directly exploits the Trie's prefix-sharing structure: 'is every prefix also a complete word' becomes 'does every step down this path pass through an end-of-word marker,' checkable during a single traversal rather than independently verifying every prefix of every word against a separate hashset.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find longest string where all prefixes exist.

Visual Diagram

```
Words: ["a", "banana", "app", "appl", "ap", "apply", "apple"]
       Longest: "apple" (all prefixes: a, ap, app, appl, apple exist)

       Using Trie:
       Insert all words, mark end
       DFS from root to find longest word
```

Solution Approach

Concept: Trie with DFS traversal.

Algorithm

```
Algorithm LONGEST_WORD(words):
           Insert all words into Trie
           DFS from root:
               If node.isEnd:
                   For each child:
                       DFS(child)
                   Update max length
           Return longest word
```

Java Implementation

```
class Solution {
           private String longest = "";

            public String longestWord(String[] words) {
                Trie trie = new Trie();
                for (String word : words) {
                    trie.insert(word);
                }
                dfs(trie.root, new StringBuilder());
                return longest;
            }

            private void dfs(Trie.Node node, StringBuilder sb) {
                for (int i = 0; i < 26; i++) {
                    Trie.Node child = node.children[i];
                    if (child != null && child.isEnd) {
                        sb.append((char)('a' + i));
                        if (sb.length() > longest.length()) {
                            longest = sb.toString();
                        }
                        dfs(child, sb);
                        sb.deleteCharAt(sb.length() - 1);
                    }
                }
            }
      }
```

Time & Space Complexity

Time Complexity: O(26 * L) | Space Complexity: O(26 * nodes)

412. Number of Distinct Substrings in a String

Understanding the Problem NEW

Count the number of distinct (non-repeating) substrings of a given string.

Brute Force Approach NEW

Generate every possible substring ($O(n^2)$ of them) and use a hashset to deduplicate — $O(n^2)$ substrings, each potentially $O(n)$ to hash/compare, giving $O(n^3)$ in the worst case.

Why We Optimize NEW

Insert every suffix of the string into a **Trie** character by character — since every substring is a prefix of some suffix, this Trie ends up containing exactly one path for every distinct substring. Counting distinct substrings is then simply **counting the number of nodes created** in the Trie (each new node represents one newly-discovered distinct substring that didn't exist as a prefix before) — achieved in $O(n^2)$ total (n suffixes, each up to length n), better than the $O(n^3)$ hashset approach, by letting the Trie's structure itself naturally deduplicate shared prefixes as they're inserted, rather than explicitly generating and hashing every substring separately.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count distinct substrings using Trie.

Visual Diagram

```
s = "ababa"
       Distinct substrings: 9
       "a", "b", "ab", "ba", "aba", "bab", "abab", "baba", "ababa"

       Insert all suffixes:
       "ababa" → a,b,a,b,a
       "baba" → b,a,b,a
       "aba" → a,b,a
       "ba" → b,a
       "a" → a
```

```
       Count distinct nodes in Trie = 9
```

Solution Approach

Concept: Insert suffixes into Trie.

Algorithm

```
Algorithm DISTINCT_SUBSTRINGS(s):
           trie = new Trie()
           count = 0

            For i = 0 to n-1:
                node = root
                For j = i to n-1:
                    index = s[j] - 'a'
                    If node.children[index] == null:
                        node.children[index] = new Node()
                        count++
                    node = node.children[index]

            Return count
```

Java Implementation

```
class Solution {
           public int countDistinctSubstrings(String s) {
               Node root = new Node();
               int count = 0;
               int n = s.length();

                 for (int i = 0; i < n; i++) {
                     Node node = root;
                     for (int j = i; j < n; j++) {
                         int idx = s.charAt(j) - 'a';
                         if (node.children[idx] == null) {
                             node.children[idx] = new Node();
                             count++;
                         }
                         node = node.children[idx];
                     }
                 }
                 return count + 1; // +1 for empty substring
            }

           private class Node {
               Node[] children = new Node[26];
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n²)

413. Bit PreRequisites for TRIE Problems

Understanding the Problem NEW

Before tackling XOR-based Trie problems, this section reviews the bit-manipulation prerequisites: representing numbers in fixed-width binary and understanding how a Trie can be built over the *bits* of numbers (rather than characters of a string) to answer XOR-maximization queries efficiently.

Brute Force Approach NEW

Nothing to contrast — this is preparatory material connecting back to the Bit Manipulation chapter's XOR properties (problems 151-168), needed before the following problems make sense.

Why We Optimize NEW

The key idea being set up here: just as a string-based Trie organizes words by shared character-prefixes, a **bit-Trie** organizes numbers by shared binary-prefixes (processing each number's bits from most-significant to least-significant) — and because XOR-ing two numbers gives the largest result when their bits differ as much as possible, a bit-Trie lets you greedily search, at each bit position, for 'the opposite bit if it's available' — turning an $O(n^2)$ all-pairs XOR check into something much faster, which is exactly what problems 414-415 build on.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding bit manipulation for Trie.

Visual Diagram

```
Binary numbers: [3, 10, 5, 25, 2, 8]
       3 = 00011
       10 = 01010
       5 = 00101
       25 = 11001
       2 = 00010
       8 = 01000

       Bit Trie:
                     root
```

```
                  /      \
                 0        1
                / \      / \
               0   1 1      0
               ...
```

Solution Approach

Concept: Build Trie with bits 0 and 1.

Java Implementation

```
class BitTrie {
           private class Node {
               Node[] children = new Node[2];
           }

            private Node root;

            BitTrie() {
                root = new Node();
            }

            void insert(int num) {
                Node node = root;
                for (int i = 31; i >= 0; i--) {
                    int bit = (num >> i) & 1;
                    if (node.children[bit] == null) {
                        node.children[bit] = new Node();
                    }
                    node = node.children[bit];
                }
            }

            int maxXor(int num) {
                Node node = root;
                int maxXor = 0;
                for (int i = 31; i >= 0; i--) {
                    int bit = (num >> i) & 1;
                    int opposite = 1 - bit;
                    if (node.children[opposite] != null) {
                        maxXor |= (1 << i);
                        node = node.children[opposite];
                    } else {
                        node = node.children[bit];

            }
        }
        return maxXor;
```

```
    }
}
```

414. Maximum XOR of Two Numbers in an Array

Understanding the Problem NEW

Given an array of numbers, find the maximum XOR value achievable between any two of them.

Brute Force Approach NEW

Check every pair of numbers and compute their XOR directly — O(n²) pairs.

Why We Optimize NEW

Insert every number into a **bit-Trie** (each number's binary representation, most-significant bit first, forming one root-to-leaf path). Then, for each number, walk the Trie trying to **greedily choose the opposite bit** at every position (since XOR-ing differing bits gives 1s, maximizing the result) — if the opposite bit's branch exists in the Trie, take it (this contributes a 1 to the result at this position); otherwise, you're forced to take the same-bit branch (contributing a 0). Doing this for every number, each in O(bit-width) time using the pre-built Trie, achieves O(n × bit-width) total instead of O(n²) — directly reusing the greedy 'always prefer the branch that helps most' idea, applied to bits instead of characters.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find maximum XOR of any two numbers.

Visual Diagram

```
Array: [3, 10, 5, 25, 2, 8]
       Max XOR: 28 (25 ^ 2 = 11001 ^ 00010 = 11011 = 27)
       Actually max is 28 (25 ^ 5 = 28)

       Insert all numbers into bit Trie
       For each num, find max XOR with existing nums
```

Solution Approach

Concept: Bit Trie with XOR query.

Algorithm

```
Algorithm MAX_XOR(nums):
           root = new BitTrie()
           maxXor = 0

           For num in nums:
               root.insert(num)
               maxXor = max(maxXor, root.maxXor(num))

           Return maxXor
```

Java Implementation

```
class Solution {
           public int findMaximumXOR(int[] nums) {
               BitTrie trie = new BitTrie();
               int maxXor = 0;

                 for (int num : nums) {
                     trie.insert(num);
                     maxXor = Math.max(maxXor, trie.maxXor(num));
                 }
                 return maxXor;
           }

           private class BitTrie {
               private class Node {
                   Node[] children = new Node[2];
               }

                 private Node root;

                 BitTrie() {
                     root = new Node();
                 }

                 void insert(int num) {
                     Node node = root;
                     for (int i = 31; i >= 0; i--) {
                         int bit = (num >> i) & 1;
                         if (node.children[bit] == null) {
                             node.children[bit] = new Node();
                         }

                           node = node.children[bit];
                       }
               }

               int maxXor(int num) {
                   Node node = root;
```

```
                int maxXor = 0;
                for (int i = 31; i >= 0; i--) {
                    int bit = (num >> i) & 1;
                    int opposite = 1 - bit;
                    if (node.children[opposite] != null) {
                        maxXor |= (1 << i);
                        node = node.children[opposite];
                    } else {
                        node = node.children[bit];
                    }
                }
                return maxXor;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * 32) | Space Complexity: O(n * 32)

415. Maximum XOR With an Element From Array

Understanding the Problem NEW

A variant of Maximum XOR: given several queries, each asking for the maximum XOR achievable between a given value x and any array element that is ≤ some limit m, answer all queries efficiently.

Brute Force Approach NEW

For each query, filter the array down to elements ≤ m, then check every remaining pair with x for XOR — O(queries × n) in the worst case.

Why We Optimize NEW

Sort the array, and **sort the queries by their limit m**; then process both in increasing order together, inserting array elements into the bit-Trie (414) only once they're ≤ the current query's limit (since queries are processed in increasing-m order, elements only ever need to be added, never removed) — then answer each query using the same greedy bit-Trie walk as problem 414, but only among elements already inserted (i.e., only those ≤ m). This 'sort queries and process incrementally, only ever adding to the Trie' technique avoids rebuilding or re-filtering the Trie per query, achieving roughly O((n + queries) log n) total instead of O(queries × n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

For each query (xi, mi), find max xi XOR y where y ≤ mi.

Visual Diagram

```
nums = [0,1,2,3,4]
       queries = [[3,1],[1,3],[5,6]]

       Query [3,1]: nums ≤ 1: [0,1], max xor = 3^0=3
       Query [1,3]: nums ≤ 3: [0,1,2,3], max xor = 1^2=3
       Query [5,6]: nums ≤ 6: all, max xor = 5^2=7
```

Solution Approach

Concept: Sort queries and nums, offline processing.

Algorithm

```
Algorithm MAX_XOR_QUERIES(nums, queries):
           Sort nums
           Sort queries by mi
           Trie trie = new Trie()
           result = new int[queries.length]
           index = 0

           For query in queries:
               While index < nums.length && nums[index] <= mi:
                   trie.insert(nums[index])
                   index++
               If trie.isEmpty(): result[i] = -1
               Else: result[i] = trie.maxXor(xi)

           Return result
```

Java Implementation

```
class Solution {
           public int[] maximizeXor(int[] nums, int[][] queries) {
               Arrays.sort(nums);
               int n = queries.length;
               int[][] q = new int[n][3];
               for (int i = 0; i < n; i++) {
                   q[i][0] = queries[i][0];
                   q[i][1] = queries[i][1];
                   q[i][2] = i;
               }
               Arrays.sort(q, (a, b) -> a[1] - b[1]);

                 BitTrie trie = new BitTrie();
                 int[] result = new int[n];
                 int index = 0;

                 for (int[] query : q) {
                     int xi = query[0], mi = query[1], qi = query[2];
                     while (index < nums.length && nums[index] <= mi) {
                         trie.insert(nums[index]);
                         index++;
                     }
                     result[qi] = index == 0 ? -1 : trie.maxXor(xi);
                 }

               return result;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O((n+m) * 32) | Space Complexity: O(n * 32)

416. Minimum Number of Bracket Reversals

Understanding the Problem NEW

Given a string of brackets that might not be balanced (but has an even length), find the minimum number of bracket reversals (flipping '(' to ')' or vice versa) needed to make it valid.

Brute Force Approach NEW

Try every possible combination of reversals and check validity — exponential without structure.

Why We Optimize NEW

After removing every already-matched pair (using a stack, similar to the Balanced Parentheses check from the Stack/Queue chapter, problem 175), what remains is some number of unmatched `)` characters followed by unmatched `(` characters (a specific, simple leftover pattern once matched pairs are stripped away) — counting the leftover unmatched `)`s as `m` and unmatched `(`s as `n`, the answer is `ceil(m/2) + ceil(n/2)` (each mismatched pair of the *same* type needs exactly one reversal to fix, and any single leftover odd one needs two reversals since it must flip type entirely). This reduces the reversal-counting problem to a straightforward stack-based reduction plus simple counting, reusing the same stack-matching idea from the Stack/Queue chapter rather than exploring reversal combinations directly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum reversals to make valid parentheses.

Visual Diagram

```
s = "}{{}}{{{"
      Invalid, min reversals = 2

      Count unmatched:
      Step 1: remove valid pairs
```

```
      remaining = "}{{"
      Step 2: count = 2, result = ceil(count/2)
```

Solution Approach

Concept: Count unmatched brackets.

Algorithm

```
Algorithm MIN_REVERSALS(s):
          If n % 2 != 0: Return -1
          left = 0, right = 0

          For char in s:
              If char == '{':
                  left++
              Else:
                  If left == 0:
                      right++
                  Else:
                      left--

          Return (left + 1) / 2 + (right + 1) / 2
```

Java Implementation

```
class Solution {
          public int minReversals(String s) {
              int n = s.length();
              if (n % 2 != 0) return -1;

                 int open = 0, close = 0;
                 for (char c : s.toCharArray()) {
                     if (c == '{') {
                         open++;
                     } else {
                         if (open == 0) {
                              close++;
                         } else {
                              open--;
                         }
                     }
                 }
                 return (open + 1) / 2 + (close + 1) / 2;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

417. Count and Say

Understanding the Problem NEW

Generate the n-th term of the 'count-and-say' sequence, where each term describes the previous term by counting consecutive runs of the same digit (e.g. '1' → '11' [one 1] → '21' [two 1s] → '1211' [one 2, one 1]).

Brute Force Approach NEW

There's no faster alternative — you genuinely need to build each term from the previous one in sequence, since each term is defined purely in terms of describing its predecessor.

Why We Optimize NEW

Simply simulate the definition directly: for each term, scan through the previous term's digits, counting consecutive runs, and build the next term as (count, digit) pairs concatenated together — O(length of term) per step, and since each term's length can roughly grow with each step, the overall cost through n terms depends on how quickly the sequence grows, but there's no cleverer shortcut than direct simulation here, since the sequence has no simpler closed-form pattern to exploit.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Generate nth count-and-say sequence.

Visual Diagram

```
1: "1"
       2: "11" (one 1)
       3: "21" (two 1s)
       4: "1211" (one 2, one 1)
       5: "111221" (one 1, one 2, two 1s)

       Build next from current:
       Current: "1211"
       Count runs:
```

```
       1→1, 2→1, 11→2
       Next: "111221"
```

Solution Approach

Concept: Run-length encoding.

Algorithm

```
Algorithm COUNT_AND_SAY(n):
           result = "1"
           For i = 2 to n:
               next = ""
               count = 1
               For j = 1 to result.length():
                   If j == result.length() || result[j] != result[j-1]:
                        next += count + result[j-1]
                        count = 1
                   Else:
                        count++
               result = next
           Return result
```

Java Implementation

```
class Solution {
           public String countAndSay(int n) {
               String result = "1";
               for (int i = 2; i <= n; i++) {
                   StringBuilder next = new StringBuilder();
                   int count = 1;
                   for (int j = 1; j < result.length(); j++) {
                        if (result.charAt(j) != result.charAt(j-1)) {
                            next.append(count).append(result.charAt(j-1));
                            count = 1;
                        } else {
                            count++;
                        }
                   }
                   next.append(count).append(result.charAt(result.length()-1));
                   result = next.toString();
               }
               return result;
           }
       }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(2^n)

418-419. Rabin Karp Algorithm

Understanding the Problem NEW

Search for a pattern within a text string — the classic pattern-matching problem, here solved with the **Rabin-Karp** algorithm, which uses hashing to speed up comparisons.

Brute Force Approach NEW

The naive approach checks the pattern against every possible starting position in the text, comparing character by character at each position — O(n×m) in the worst case (n = text length, m = pattern length), since a near-match can require almost a full pattern-length comparison before failing.

Why We Optimize NEW

Compute a **rolling hash** of the pattern and of every same-length window of the text — critically, the hash of the *next* window can be computed from the *current* window's hash in O(1) (by removing the outgoing character's contribution and adding the incoming character's, using modular arithmetic), rather than rehashing the whole window from scratch. Only when a window's hash matches the pattern's hash do you do a full character-by-character verification (to rule out hash collisions) — in practice, this achieves close to O(n+m) average time, since most non-matching windows are ruled out by a quick O(1) hash comparison instead of a full O(m) character comparison, though a poor hash function (with many collisions) can degrade this back toward O(n×m) in the worst case.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Pattern matching using rolling hash.

Visual Diagram

```
Text: "ababcab", Pattern: "abc"
       Hash("abc") = 123
       Rolling hash:
       "aba" → remove 'a', add 'c' → "abc"
       When hash matches, verify actual string
```

Solution Approach

Concept: Rolling hash with modulo.

Algorithm

```
Algorithm RABIN_KARP(text, pattern):
           n = text.length, m = pattern.length
           if m > n: return []
           d = 256, q = 101 (prime)

           // Compute hash
           p = 0, t = 0, h = d^(m-1) % q
           For i = 0 to m-1:
               p = (p*d + pattern[i]) % q
               t = (t*d + text[i]) % q

           For i = 0 to n-m:
               If p == t:
                   If text.substring(i, i+m) == pattern:
                       return i
               If i < n-m:
                   t = (d*(t - text[i]*h) + text[i+m]) % q
                   If t < 0: t += q
           Return -1
```

Java Implementation

```
class Solution {
           public int rabinKarp(String text, String pattern) {
               int n = text.length(), m = pattern.length();
               if (m > n) return -1;

                 int d = 256, q = 101;
                 int h = 1;
                 for (int i = 0; i < m-1; i++) {
                     h = (h * d) % q;
                 }

                 int p = 0, t = 0;
                 for (int i = 0; i < m; i++) {
                     p = (p * d + pattern.charAt(i)) % q;
                     t = (t * d + text.charAt(i)) % q;
                 }

                 for (int i = 0; i <= n - m; i++) {
                     if (p == t) {
                         if (text.substring(i, i+m).equals(pattern)) {
                             return i;

                      }
```

```
                }
                if (i < n - m) {
                    t = (d * (t - text.charAt(i) * h) + text.charAt(i+m)) % q;
                    if (t < 0) t += q;
                }
            }
            return -1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n+m) average | Space Complexity: O(1)

420. Z-Function

Understanding the Problem NEW

Compute the Z-array for a string — where `Z[i]` is the length of the longest substring starting at position i that matches a prefix of the string itself. This array is a powerful building block for various pattern-matching tasks.

Brute Force Approach NEW

Compute each `Z[i]` independently by directly comparing characters starting at position i against the string's prefix, character by character — O(n) per position, O(n²) overall, since many of these comparisons redundantly re-examine characters already compared for a previous position.

Why We Optimize NEW

Maintain a **'Z-box'** — the rightmost-reaching interval [L, R] found so far where a prefix-match is known to extend — and when computing a new position, if it falls within an existing Z-box, you can initialize its comparison using already-known information from an earlier corresponding position (since the Z-box guarantees that segment already matches the prefix), only extending the comparison further if needed, rather than starting completely from scratch. This 'reuse previously discovered matching information via the current Z-box' technique computes the entire Z-array in O(n) total, instead of the O(n²) that comes from treating every position as an independent, from-scratch comparison.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Compute Z-array for pattern matching.

Visual Diagram

```
s = "aabcaabx"
       Z-array:
       [0,1,0,0,3,1,0,0]

       Z[i] = longest prefix of s starting at i
       At i=4: "aabx" matches prefix "aab" → 3
```

Solution Approach

Concept: Z-algorithm with [L,R] window.

Algorithm

```
Algorithm Z_FUNCTION(s):
           n = s.length()
           z = new int[n]
           l = 0, r = 0

           For i = 1 to n-1:
               If i <= r:
                   z[i] = min(r - i + 1, z[i - l])
               While i + z[i] < n && s[z[i]] == s[i + z[i]]:
                   z[i]++
               If i + z[i] - 1 > r:
                   l = i
                   r = i + z[i] - 1

           Return z
```

Java Implementation

```
class Solution {
           public int[] zFunction(String s) {
               int n = s.length();
               int[] z = new int[n];
               int l = 0, r = 0;

                 for (int i = 1; i < n; i++) {
                     if (i <= r) {
                         z[i] = Math.min(r - i + 1, z[i - l]);
                     }
                     while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) {
                         z[i]++;
                     }
                     if (i + z[i] - 1 > r) {
                         l = i;
                         r = i + z[i] - 1;
                     }
                 }
                 return z;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

421. KMP Algorithm / LPS (pi) Array

Understanding the Problem NEW

The Knuth-Morris-Pratt (KMP) algorithm for pattern matching, built around computing the LPS (Longest Proper Prefix that is also a Suffix) array for the pattern, used to search a pattern within a text efficiently.

Brute Force Approach NEW

The naive approach (same as Rabin-Karp's baseline) checks the pattern against every text position independently, and on a mismatch, restarts the comparison from the very next text position and the very start of the pattern — O(n×m) worst case, since it discards all the partial-match information gained before the mismatch.

Why We Optimize NEW

Precompute the pattern's **LPS array** (for each prefix of the pattern, the length of its longest proper prefix that's also a suffix of that same prefix) — this captures exactly how much of a partial match can be 'reused' when a mismatch occurs, since the pattern itself is compared against itself to find repeated internal structure. During the actual text search, on a mismatch, instead of restarting the pattern from position 0, you jump the pattern pointer back to `LPS[previous position]` — skipping past comparisons that the LPS array guarantees would fail anyway, since that structure was already confirmed not to match. This achieves O(n+m) total (O(m) to build the LPS array, O(n) for the search), a genuine improvement over the O(n×m) naive approach, by never re-examining text characters that have already been compared once — a different, generally faster-worst-case alternative to Rabin-Karp's hash-based approach.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Pattern matching using KMP.

Visual Diagram

```
Pattern: "ababc"
       LPS: [0,0,1,2,0]
```

```
       LPS[i] = longest proper prefix which is also suffix
       "abab" → prefix "ab", suffix "ab" → 2

       KMP search:
       Text: "abababc"
       Pattern: "ababc"
       Match found at index 2
```

Solution Approach

Concept: LPS array for failure function.

Algorithm

```
Algorithm KMP(text, pattern):
           lps = computeLPS(pattern)
           i = 0, j = 0

            While i < text.length:
                If text[i] == pattern[j]:
                    i++; j++
                    If j == pattern.length:
                        return i - j
                Else If j > 0:
                    j = lps[j-1]
                Else:
                    i++
            Return -1
```

Java Implementation

```
class Solution {
           public int kmpSearch(String text, String pattern) {
               int[] lps = computeLPS(pattern);
               int i = 0, j = 0;

                 while (i < text.length()) {
                     if (text.charAt(i) == pattern.charAt(j)) {
                         i++;
                         j++;
                         if (j == pattern.length()) {
                              return i - j;
                         }
                     } else if (j > 0) {
                         j = lps[j-1];
                     } else {
                         i++;
                     }
                  }
```

```
                return -1;
            }
            private int[] computeLPS(String pattern) {

              int[] lps = new int[pattern.length()];
              int len = 0;
              int i = 1;

              while (i < pattern.length()) {
                  if (pattern.charAt(i) == pattern.charAt(len)) {
                      len++;
                      lps[i] = len;
                      i++;
                  } else if (len > 0) {
                      len = lps[len - 1];
                  } else {
                      lps[i] = 0;
                      i++;
                  }
              }
              return lps;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n+m) | Space Complexity: O(m)

422. Shortest Palindrome

Understanding the Problem NEW

Given a string, find the shortest palindrome you can create by adding characters only to the *front* of the string.

Brute Force Approach NEW

Try every possible prefix to add and check if the resulting string is a palindrome — O(n²) checks (n possible insertion points, each requiring an O(n) palindrome check), or worse if not done carefully.

Why We Optimize NEW

The key question is: what's the **longest palindromic prefix** of the string (starting from index 0)? — everything *after* that palindromic prefix needs to have its reverse added to the front. To find this efficiently, construct a new string as `string + separator + reverse(string)` and compute its **KMP LPS array** (421) — the LPS value at the very last position of this combined string tells you exactly how long of a prefix of the original string is also a palindrome (since the LPS captures the longest overlap between the string and its own reverse, arranged this way). This reuses the KMP LPS computation (O(n)) to answer a seemingly different palindrome question, rather than checking every possible prefix insertion directly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Add characters to front to make palindrome.

Visual Diagram

```
s = "aacecaaa"
       Shortest palindrome: "aaacecaaa"

       Find longest palindrome prefix:
       "aacecaa" is palindrome? "aacecaa" reversed "aacecaa" yes
       Add remaining "aa" to front? Actually "aa"+"aacecaaa" = "aaaacecaaa"

       Using KMP:
```

```
       s + "#" + reverse(s)
       Find LPS of combined string
```

Solution Approach

Concept: KMP LPS on combined string.

Algorithm

```
Algorithm SHORTEST_PALINDROME(s):
           rev = reverse(s)
           combined = s + "#" + rev
           lps = computeLPS(combined)
           toAdd = rev.substring(0, s.length() - lps[lps.length-1])
           Return toAdd + s
```

Java Implementation

```
class Solution {
           public String shortestPalindrome(String s) {
               String rev = new StringBuilder(s).reverse().toString();
               String combined = s + "#" + rev;
               int[] lps = computeLPS(combined);
               int len = lps[lps.length - 1];
               String toAdd = rev.substring(0, s.length() - len);
               return toAdd + s;
           }

           private int[] computeLPS(String s) {
               int[] lps = new int[s.length()];
               int len = 0, i = 1;
               while (i < s.length()) {
                   if (s.charAt(i) == s.charAt(len)) {
                       len++;
                       lps[i] = len;
                       i++;
                   } else if (len > 0) {
                       len = lps[len - 1];
                   } else {
                       i++;
                   }
               }
               return lps;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

423. Longest Happy Prefix

Understanding the Problem NEW

Find the length of the longest 'happy prefix' of a string — a substring that is both a proper prefix (not the whole string) and a proper suffix of the same string.

Brute Force Approach NEW

Check every possible prefix length and see if the same-length suffix matches it — $O(n^2)$ in the worst case (n possible lengths, each requiring an O(n) comparison).

Why We Optimize NEW

This is *exactly* what the KMP **LPS array** (421) computes at its very last position — the LPS array's definition ('longest proper prefix that is also a suffix,' computed for every prefix of the string) directly gives the answer for the whole string at its final index, with no modification needed. This problem exists specifically to make explicit that the LPS array — usually thought of as just an internal building block for KMP's pattern search — directly answers this seemingly separate-sounding question on its own, in the O(n) time it already takes to build the array.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Longest prefix which is also suffix.

Visual Diagram

```
s = "ababab"
       Longest happy prefix: "abab"
       LPS array: [0,0,1,2,3,4]
       Result: s.substring(0, 4) = "abab"

       s = "level"
       LPS: [0,0,0,0,1]
       Result: "l"
```

Solution Approach

Concept: KMP LPS array.

Algorithm

```
Algorithm LONGEST_HAPPY_PREFIX(s):
           lps = computeLPS(s)
           len = lps[lps.length - 1]
           Return len == 0 ? "" : s.substring(0, len)
```

Java Implementation

```
class Solution {
           public String longestHappyPrefix(String s) {
               int[] lps = computeLPS(s);
               int len = lps[lps.length - 1];
               return len == 0 ? "" : s.substring(0, len);
           }

            private int[] computeLPS(String s) {
                int[] lps = new int[s.length()];
                int len = 0, i = 1;
                while (i < s.length()) {
                    if (s.charAt(i) == s.charAt(len)) {
                        len++;
                        lps[i] = len;
                        i++;
                    } else if (len > 0) {
                        len = lps[len - 1];
                    } else {
                        i++;
                    }
                }
                return lps;
            }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

424. Count Palindromic Subsequences in Given String

Understanding the Problem NEW

Count the number of palindromic subsequences within a string (not necessarily distinct — every valid selection counts, even if it produces the same characters as another selection).

Brute Force Approach NEW

Try every one of the 2^n subsequences and check which are palindromes — exponential.

Why We Optimize NEW

This connects back to the Dynamic Programming chapter's interval-DP family: define `countPalindromes(left, right)` using inclusion-exclusion — count the palindromic subsequences within `[left+1, right]` plus those within `[left, right-1]`, but subtract the double-counted ones in `[left+1, right-1]` to avoid overcounting, and if the characters at `left` and `right` match, add back in the subsequences formed by wrapping that matching pair around every subsequence (including the empty one) within `[left+1, right-1]`. This is the same 'range as DP state' interval-DP shape from the DP chapter (Matrix Chain Multiplication, Burst Balloons), applied here to a counting question about palindromic subsequences, achieving $O(n^2)$ instead of the exponential 2^n subsequence enumeration — a fitting final problem for the book, since it ties this last chapter's string-matching machinery (Tries, KMP, Z-function) back to the interval-DP technique from the chapter just before it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count palindromic subsequences in string.

Visual Diagram

```
s = "bccb"
       Palindromic subsequences: 7
       "b", "c", "b", "c", "bb", "cc", "bccb"
```

```
dp[i][j] = count of palindromic subsequences in s[i..j]
If s[i] == s[j]:
    dp[i][j] = dp[i+1][j] + dp[i][j-1] + 1
Else:
    dp[i][j] = dp[i+1][j] + dp[i][j-1] - dp[i+1][j-1]
```

Solution Approach

Concept: DP with inclusion-exclusion.

Algorithm

```
Algorithm COUNT_PAL_SUBSEQ(s):
           n = s.length()
           dp = int[n][n]

           For i = 0 to n-1: dp[i][i] = 1

           For len = 2 to n:
               For i = 0 to n-len:
                   j = i + len - 1
                   If s[i] == s[j]:
                       dp[i][j] = dp[i+1][j] + dp[i][j-1] + 1
                   Else:
                       dp[i][j] = dp[i+1][j] + dp[i][j-1] - dp[i+1][j-1]

           Return dp[0][n-1]
```

Java Implementation

```
class Solution {
           public int countPalindromicSubsequences(String s) {
               int n = s.length();
               int[][] dp = new int[n][n];

                 for (int i = 0; i < n; i++) {
                     dp[i][i] = 1;
                 }

                 for (int len = 2; len <= n; len++) {
                     for (int i = 0; i <= n - len; i++) {
                         int j = i + len - 1;
                         if (s.charAt(i) == s.charAt(j)) {
                             dp[i][j] = dp[i+1][j] + dp[i][j-1] + 1;
                         } else {
                             dp[i][j] = dp[i+1][j] + dp[i][j-1] - dp[i+1][j-1];
                         }
                     }
                 }
```

```
                return dp[0][n-1];
            }

        }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n²)

Tries & Strings Summary

Problem	Category	Time	Space	Key Technique
Basic Trie	Trie	O(L)	O(26*nodes)	Prefix Tree
Trie 2	Trie	O(L)	O(26*nodes)	Counts tracking
Longest Word	Trie	O(26*L)	O(26*nodes)	DFS traversal
Distinct Substrings	Trie	$O(n^2)$	$O(n^2)$	Suffix insertion
Max XOR	Bit Trie	O(n*32)	O(n*32)	Bit Trie
Max XOR Queries	Bit Trie	O((n+m)*32)	O(n*32)	Offline processing
Min Reversals	Strings	O(n)	O(1)	Unmatched count
Count and Say	Strings	O(2^n)	O(2^n)	Run-length encoding
Rabin Karp	Strings	O(n+m)	O(1)	Rolling hash
Z-Function	Strings	O(n)	O(n)	Z-array
KMP	Strings	O(n+m)	O(m)	LPS array
Shortest Palindrome	Strings	O(n)	O(n)	KMP + reverse
Happy Prefix	Strings	O(n)	O(n)	LPS array
Count Pal Subseq	Strings	$O(n^2)$	$O(n^2)$	Interval DP

— End of enhanced Chapter 14: Tries & Strings — End of Enhanced Beginner Edition —